



ORACLE
NetSuite

SuiteScript 2.x



2025.2

January 7, 2026



Copyright © 2005, 2025, Oracle and/or its affiliates.

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software, software documentation, data (as defined in the Federal Acquisition Regulation), or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software," "commercial computer software documentation," or "limited rights data" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed, or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle®, Java, and MySQL are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

If this document is in public or private pre-General Availability status:

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

If this document is in private pre-General Availability status:

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, timing, and pricing of any features or functionality described in this document may change and remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Documentation Accessibility

For information about Oracle's commitment to accessibility, visit the Oracle Accessibility Program website at <https://www.oracle.com/corporate/accessibility>.

Access to Oracle Support

Oracle customers that have purchased support have access to electronic support through My Oracle Support. For information, visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=info> or visit <http://www.oracle.com/pls/topic/lookup?ctx=acc&id=trs> if you are hearing impaired.

Sample Code

Oracle may provide sample code in SuiteAnswers, the Help Center, User Guides, or elsewhere through help links. All such sample code is provided "as is" and "as available", for use only with an authorized NetSuite Service account, and is made available as a SuiteCloud Technology subject to the SuiteCloud Terms of Service at www.netsuite.com/tos.

Oracle may modify or remove sample code at any time without notice.

No Excessive Use of the Service

As the Service is a multi-tenant service offering on shared databases, Customer may not use the Service in excess of limits or thresholds that Oracle considers commercially reasonable for the Service. If Oracle reasonably concludes that a Customer's use is excessive and/or will cause immediate or ongoing performance issues for one or more of Oracle's other customers, Oracle may slow down or throttle Customer's excess use until such time that Customer's use stays within reasonable limits. If Customer's particular usage pattern requires a higher limit or threshold, then the Customer should procure a subscription to the Service that accommodates a higher limit and/or threshold that more effectively aligns with the Customer's actual usage pattern.

Beta Features

This software and related documentation are provided under a license agreement containing restrictions on use and disclosure and are protected by intellectual property laws. Except as expressly permitted in your license agreement or allowed by law, you may not use, copy, reproduce, translate, broadcast, modify, license, transmit, distribute, exhibit, perform, publish, or display any part, in any form, or by any means. Reverse engineering, disassembly, or decompilation of this software, unless required by law for interoperability, is prohibited.

The information contained herein is subject to change without notice and is not warranted to be error-free. If you find any errors, please report them to us in writing.

If this is software or related documentation that is delivered to the U.S. Government or anyone licensing it on behalf of the U.S. Government, then the following notice is applicable:

U.S. GOVERNMENT END USERS: Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs) and Oracle computer documentation or other Oracle data delivered to or accessed by U.S. Government end users are "commercial computer software" or "commercial computer software documentation" pursuant to the applicable Federal Acquisition Regulation and agency-specific supplemental regulations. As such, the use, reproduction, duplication, release, display, disclosure, modification, preparation of derivative works, and/or adaptation of i) Oracle programs (including any operating system, integrated software, any programs embedded, installed or activated on delivered hardware, and modifications of such programs), ii) Oracle computer documentation and/or iii) other Oracle data, is subject to the rights and limitations specified in the license contained in the applicable contract. The terms governing the U.S. Government's use of Oracle cloud services are defined by the applicable contract for such services. No other rights are granted to the U.S. Government.

This software or hardware is developed for general use in a variety of information management applications. It is not developed or intended for use in any inherently dangerous applications, including applications that may create a risk of personal injury. If you use this software or hardware in dangerous applications, then you shall be responsible to take all appropriate fail-safe, backup, redundancy, and other measures to ensure its safe use. Oracle Corporation and its affiliates disclaim any liability for any damages caused by use of this software or hardware in dangerous applications.

Oracle and Java are registered trademarks of Oracle and/or its affiliates. Other names may be trademarks of their respective owners.

Intel and Intel Inside are trademarks or registered trademarks of Intel Corporation. All SPARC trademarks are used under license and are trademarks or registered trademarks of SPARC International, Inc. AMD, Epyc, and the AMD logo are trademarks or registered trademarks of Advanced Micro Devices. UNIX is a registered trademark of The Open Group.

This software or hardware and documentation may provide access to or information about content, products, and services from third parties. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to third-party content, products, and services unless otherwise set forth in an applicable agreement between you and Oracle. Oracle Corporation and its affiliates will not be responsible for any loss, costs, or damages incurred due to your access to or use of third-party content, products, or services, except as set forth in an applicable agreement between you and Oracle.

This documentation is in pre-General Availability status and is intended for demonstration and preliminary use only. It may not be specific to the hardware on which you are using the software. Oracle Corporation and its affiliates are not responsible for and expressly disclaim all warranties of any kind with respect to this documentation and will not be responsible for any loss, costs, or damages incurred due to the use of this documentation.

The information contained in this document is for informational sharing purposes only and should be considered in your capacity as a customer advisory board member or pursuant to your pre-General Availability trial agreement only. It is not a commitment to deliver any material, code, or functionality, and should not be relied upon in making purchasing decisions. The development, release, and timing of any features or functionality described in this document remains at the sole discretion of Oracle.

This document in any form, software or printed matter, contains proprietary information that is the exclusive property of Oracle. Your access to and use of this confidential material is subject to the terms and conditions of your Oracle Master Agreement, Oracle License and Services Agreement, Oracle PartnerNetwork Agreement, Oracle distribution agreement, or other license agreement which has been executed by you and Oracle and with which you agree to comply. This document and information contained herein may not be disclosed, copied, reproduced, or distributed to anyone outside Oracle without prior written consent of Oracle. This document is not part of your license agreement nor can it be incorporated into any contractual agreement with Oracle or its subsidiaries or affiliates.

Table of Contents

SuiteScript 2.x API Introduction	1
SuiteScript 2.x Hello World	1
Key SuiteScript 2.x Development Concepts	1
Enable the Client SuiteScript Feature	3
Create the Client SuiteScript Script File	4
Upload the Script File to NetSuite	8
Create a Script Record and Script Deployment Record	8
Test the Hello World Script	9
Next Steps in Learning SuiteScript 2.x	10
SuiteScript 2.x Script Basics	10
SuiteScript 2.x Anatomy of a Script	12
SuiteScript 2.x Script Creation Process	14
SuiteScript 2.x Advantages	15
SuiteScript 2.x Terminology	18
SuiteScript 2.x Developer Resources	20
SuiteScript Reserved Words	21
SuiteScript Versioning Guidelines	21
SuiteScript 2.1	24
Executing Scripts Using SuiteScript 2.1	24
Executing a Single Script Using SuiteScript 2.1	25
Enabling SuiteScript 2.1 at the Account Level	25
SuiteScript 2.1 Language Examples	28
Differences Between SuiteScript 2.0 and SuiteScript 2.1	32
SuiteScript 2.x AI APIs	40
SuiteScript 2.x Generative AI APIs	41
Using Your Own OCI Configuration for SuiteScript Generative AI APIs	43
SuiteScript 2.x Analytic APIs	45
Workbook API	46
Workbook API Concepts	47
Tutorial: Creating a Dataset Using the Workbook API	61
Tutorial: Creating a Workbook Using the Workbook API	72
Workbook API Limitations	90
SuiteScript 2.x Script Types	91
SuiteScript 2.x Bundle Installation Script Type	95
SuiteScript 2.x Bundle Installation Script Reference	96
SuiteScript 2.x Bundle Installation Script Entry Points and API	99
SuiteScript 2.x Client Script Type	102
SuiteScript 2.x Client Script Reference	106
SuiteScript 2.x Client Script Entry Points and API	110
SuiteScript 2.1 Custom Tool Script Type	119
SuiteScript 2.1 Custom Tool Script Type Reference	120
SuiteScript 2.1 Custom Tool Script Type Code Samples	123
SuiteScript 2.x Map/Reduce Script Type	128
SuiteScript 2.x Map/Reduce Script Reference	129
SuiteScript 2.x Map/Reduce Script Entry Points and API	161
SuiteScript 2.x Map/Reduce Script Code Samples	199
SuiteScript 2.x Mass Update Script Type	205
SuiteScript 2.x Mass Update Script Entry Points and API	207
SuiteScript 2.x Portlet Script Type	208
SuiteScript 2.x Portlet Script Reference	209
SuiteScript 2.x Portlet Script Entry Points and API	214
SuiteScript 2.x RESTlet Script Type	224
SuiteScript 2.x RESTlet Reference	225

SuiteScript 2.x RESTlet Script and Request Examples	232
SuiteScript 2.x RESTlet Script Entry Points and API	239
SuiteScript 2.x Scheduled Script Type	241
SuiteScript 2.x Scheduled Script Reference	244
SuiteScript 2.x Scheduled Script Entry Points and API	255
SuiteScript 2.x SDF Installation Script Type	257
SuiteScript 2.x SDF Installation Script Entry Points and API	258
SuiteScript 2.x Suitelet Script Type	259
SuiteScript 2.x Suitelet Script Reference	260
SuiteScript 2.x Suitelet Script Entry Points and API	269
SuiteScript 2.x Suitelet Script Type Code Samples	270
SuiteScript 2.x User Event Script Type	303
SuiteScript 2.x User Event Script Reference	305
SuiteScript 2.x User Event Script Entry Points and API	315
SuiteScript 2.x Workflow Action Script Type	320
Creating and Using Workflow Action Scripts	321
SuiteScript 2.x Workflow Action Script Entry Points and API	323
SuiteScript 2.x Record Actions and Macros	324
Overview of Record Action and Macro APIs	324
Supported Record Actions	325
Supported Record Macros	326
SuiteScript 2.x JSDoc Validation	328
Controlling Access to Scripts and Custom Modules	330
SuiteScript 2.x Entry Point Script Creation and Deployment	333
Record-Level and Form-Level Script Deployments	333
SuiteScript 2.x Entry Point Script Validation	334
Entry Point Script Validation Guidelines	334
Entry Point Script Validation Examples	337
Entry Point Script Validation Error Reference	339
SuiteScript 2.x Record-Level Script Deployments	344
Script Record Creation	344
Script Deployment	349
Viewing System Notes	355
SuiteScript 2.x Form-Level Script Deployments	356
Attaching a Client Script to a Form	357
Configuring a Custom Action	358
SuiteScript 2.x Custom Modules	359
SuiteScript 2.x Anatomy of a Custom Module Script	360
SuiteScript 2.x Custom Module Tutorial	362
Module Dependency Paths	370
Naming a Custom Module	372
Custom Module Examples	373
Troubleshooting Errors	377
Custom Modules Frequently Asked Questions	377
SuiteScript 2.x Scripting Records and Subrecords	379
SuiteScript 2.x Scripting Records	379
SuiteScript 2.x Standard and Dynamic Modes	379
SuiteScript 2.x Record Modules	380
SuiteScript 2.x Scripting Subrecords	381
About Subrecords	381
Subrecord Scripting in SuiteScript 2.x Compared With 1.0	394
Scripting Subrecords that Occur on Sublist Lines	395
Scripting Subrecords that Occur in Body Fields	414
SuiteScript 2.x Custom Pages	431
SuiteScript 2.x Custom Forms	431

Sample Custom Form Script	437
SuiteScript 2.x Custom List Pages	450
Sample Custom List Page Script	451
SuiteScript 2.x Working with UI Objects	454
Custom UI Development	455
UI Component Overview	455
SuiteScript 2.x UI Modules	457
Using HTML	457
Creating Custom Assistants	461
Transitioning from SuiteScript 1.0 to SuiteScript 2.x	466
Overview of the Differences Between SuiteScript 1.0 and SuiteScript 2.x	467
Differences Between SuiteScript 1.0 and SuiteScript 2.x Script Types	470
Differences in Similar SuiteScript 1.0 and SuiteScript 2.x Capabilities	474
SuiteScript 1.0 APIs Not Directly Mapped to a SuiteScript 2.x Module	475
Converting a SuiteScript 1.0 Script to a SuiteScript 2.x Script	477
Sample SuiteScript 1.0 to SuiteScript 2.0 and SuiteScript 2.1 Conversions	478
Confirm bundle installation, enable features, create account	479
Set default fields values on a record	480
Disable fields on a record	481
Display user profile information	482
Update fields on sales order and estimate records	483
Build a simple portlet that posts data to a servlet	483
Using RESTlet get, post, put, and delete entry points and methods	485
Fulfill and bill sales orders each day	486
Create a simple form	487
Implement end of month sales order promotions	489
Set the sales rep on the record in a workflow	490
Create an item receipt from a purchase order	491
Create and run a joined search	492
Create CSV imports	493
Using SuiteScript 2.x Scripts with SuiteScript 1.0 Scripts	494
nlapiRequestRestlet(scriptId, deploymentId, urlParams, body, headers, httpMethod)	494

SuiteScript 2.x API Introduction

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following help topics show how to write scripts using the SuiteScript 2.x API:

- [SuiteScript 2.x Hello World](#) – To help you understand how SuiteScript 2.x works, this topic walks you through how to set up a basic customization.
- [SuiteScript 2.x Script Basics](#) – Some components are common to all SuiteScript 2.x scripts. This topic covers a few of them.
- [SuiteScript 2.x Anatomy of a Script](#) – All SuiteScript 2.x entry point scripts follow the same basic structure. This topic explains that structure.
- [SuiteScript 2.x Script Creation Process](#) – You can create a SuiteScript 2.x script by following a simple process flow. This topic explains the steps.
- [SuiteScript 2.x Advantages](#) – SuiteScript 2.x is a complete redesign of the SuiteScript model from 1.0. This topic covers some of the advantages of SuiteScript 2.x over 1.0.
- [SuiteScript 2.x Terminology](#) – Some terms are defined differently in SuiteScript 2.0 and 2.1. This topic lists and explains these terms.
- [SuiteScript 2.x Developer Resources](#) – There are several resources to help you use SuiteScript 2.x. This topic lists both internal and external resources, plus help topics for developing SuiteScripts.
- [SuiteScript Reserved Words](#) – SuiteScript has some reserved words you can't use as variable or function names. This topic explains what they are.
- [SuiteScript Versioning Guidelines](#) – This topic explains the SuiteScript versioning system.

SuiteScript 2.x Hello World

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

To help you understand how SuiteScript 2.x works, this tutorial walks you through how to set up a basic customization. After you finish the steps in this tutorial, you'll see a "Hello, World!" message whenever you load a NetSuite task record.

See the following sections:

- [Key SuiteScript 2.x Development Concepts](#)
- [Enable the Client SuiteScript Feature](#)
- [Create the Client SuiteScript Script File](#)
- [Upload the Script File to NetSuite](#)
- [Create a Script Record and Script Deployment Record](#)
- [Test the Hello World Script](#)
- [Next Steps in Learning SuiteScript 2.x](#)

Key SuiteScript 2.x Development Concepts

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Before you get started, it might help to review a few key concepts and terms that are central to SuiteScript 2.x development. Some of these terms appear in the steps later in this tutorial. You can read about these concepts now or skip ahead to [Enable the Client SuiteScript Feature](#).

SuiteScript Versions

The sample script here uses SuiteScript 2.0. There's also a newer version, SuiteScript 2.1, which supports new language features from ES2019. You can write your scripts in either SuiteScript 2.0 or 2.1.

- For help writing scripts in SuiteScript 2.x, see [SuiteScript 2.x Entry Point Script Creation and Deployment](#).
- For more info about SuiteScript versions and 2.1, see [SuiteScript Versioning Guidelines](#) and [SuiteScript 2.1](#).

SuiteScript Reserved Words

Like most programming languages, SuiteScript has a list of reserved words you can't use as variable or function names in your scripts. For example, `var` is used to declare a variable, and `try` marks the start of a try-catch block. For more information, see [SuiteScript Reserved Words](#).

SuiteScript 2.x Script Types and Entry Points

Before you start writing your script, you need to pick which predefined **script type** to use. Each script type is made for certain situations and specific triggering events. The following are some of the available [SuiteScript 2.x Script Types](#):

- The [SuiteScript 2.x Client Script Type](#) is for scripts that should run in the browser.
- The [SuiteScript 2.x Scheduled Script Type](#) is for server scripts that should run at a specific time or on a recurring schedule.
- The [SuiteScript 2.x RESTlet Script Type](#) is for server scripts that should run when called over HTTP by an application external to NetSuite.

Each script type has one or more **entry points** unique to that type. An entry point is where the system hands control of NetSuite to your script. When you add an entry point to your script, you're telling the system to do something when that entry point is triggered. This runs a function in your script, called an **entry point function**.

For many script types, entry points are like events, or the different things that can happen, to trigger your script. For example, client script entry points are events that can happen in a browser session, like [fieldChanged\(scriptContext\)](#) when a field value changes, or [pageInit\(scriptContext\)](#) when a page loads. In contrast, the scheduled script type has only one entry point, called [execute](#). This is when a schedule runs the script or a user runs it manually.

In the example in this tutorial, you want a dialog alert to pop up when someone loads the NetSuite task record page in a browser. So, this example uses the client script type and the `pageInit` entry point.

SuiteScript 2.x Modules

SuiteScript 2.x uses [Modular Architecture](#). All its APIs are organized into standard modules, and each module's name shows what it does. For example, the [N/record Module](#) lets you work with NetSuite records, while [N/https Module](#) lets you make HTTPS requests to external web services.

Most modules need to be loaded in your script before you can use their APIs. Loading a JavaScript module is kind of like importing a library in Java—it gives you access to code that's defined somewhere

else. In an entry point script, you load modules through the [define Object](#). You list the modules you want to load as arguments.

Some APIs are globally available, so you can use them without loading their modules first. You'll find a list of these globally available APIs in [SuiteScript 2.x Global Objects](#).

The example in this tutorial uses both: some globally available APIs, and a method that only works after you load the right module.

Entry Point Scripts Versus Custom Module Scripts

The example script includes everything in one file. But sometimes you'll want scripts that use logic from other files. In SuiteScript 2.x, these extra files are called **custom module scripts**.

The main script file—the one that sets the script type, entry point, and entry point function—is called an **entry point script**. Entry point scripts have their own formatting rules, which are different from custom module scripts. The next steps in this tutorial point out some of these requirements.

This topic doesn't cover custom module scripts. For more on those, see [SuiteScript 2.x Custom Modules](#).

Enable the Client SuiteScript Feature

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Before you can do the rest of the steps in this tutorial, you'll need to make sure the Client SuiteScript feature is enabled in your NetSuite account.

If you're not sure whether the feature's enabled, check by going to Customization > Scripting > Scripts. If you can follow that path, the feature is enabled. If you don't see the option, it might be because you don't have permission or the menu path was customized. If you're still not sure, ask your account administrator.

An account administrator or anyone with the Enable Features permission can turn on the feature. To enable it, follow these steps.

To enable the Client SuiteScript feature:

1. Go to Setup > Company > Enable Features.
2. On the **SuiteCloud** subtab, check the **Client SuiteScript** box.
3. If the box is already checked, skip to [Create the Client SuiteScript Script File](#).

SuiteScript

☒ **Client SuiteScript**
Use industry-standard JavaScript to do advanced client-side customization of your forms.

☐ **Server SuiteScript**
Use industry-standard JavaScript to do advanced server-side customization of your business processes.

☐ **SuiteScript Server Pages**
Create interactive Web applications with customizable SuiteScript to deliver a rich user experience.

You'll see a window with the terms of service.

4. If you agree to the terms, scroll to the bottom and click **I Agree**.
5. You don't need Server SuiteScript for these steps, but if you plan to do more SuiteScript development later, you may want to check the **Server SuiteScript** box. If you check the box, you'll see another window with the terms of service. Click **I Agree**.
6. Click **Save**.

Create the Client SuiteScript Script File

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

First, you need to create a script file called `helloWorld.js`. Choose one of the following options:

- Create your own `helloWorld.js` script. See [Create the Client SuiteScript Script File Step by Step](#).
- Copy the full script. See [Copy the Full Script](#).

All entry point scripts need to include the required components described in [SuiteScript 2.x Anatomy of a Script](#).

Create the Client SuiteScript Script File Step by Step

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

To create the script file:

1. Open a new file in your favorite text editor.
2. Add two JSDoc tags:
 - `@NApiVersion` – This is the version of SuiteScript you're using.
 - `@NScriptType` – This is the script type you're using.

After each JSDoc tag, add the right value, like you see in the following snippet.

```
1 /**
2  *@NApiVersion 2.0
3  *@NScriptType ClientScript
4  */
5
6 ...
```

Every SuiteScript 2.x entry point script needs to have these tags, and you can only use one value for each. For more details and a list of valid values, check out [SuiteScript 2.x JSDoc Tags](#).

3. Add the [define Object](#). Every entry point script needs to use it.

Use `['N/ui/dialog']` as the define function's first argument for the define function. This argument is an array of string values for the modules you want to load.

```
1 ...
2 define(['N/ui/dialog'],
3
4     // In Step 4, you will add additional code here.
5
```

```
6 | });
```

The `N/ui/dialog` Module has methods for showing different types of dialogs. You load it so your script can use one of those methods.

4. Declare a callback function as the second argument for the `define` function, and give it one argument called **dialog**.

```
1 | ...
2 |     function(dialog) {
3 |
4 |         // In Step 5, you will add additional code here.
5 |
6 |     }
7 | ...
```

If you're not familiar with callback functions, know that this function will hold all the other logic for your script. Also, the number of arguments in your callback function has to match the number of modules you load with the `define` function. Each argument is an object that lets you access its module, so it's a good idea to name each argument after its module.

In this example, the `define` function loads one module, so the callback function only needs one argument.

5. Inside the callback function, declare a function named **helloWorld**.

```
1 | ...
2 |     function helloWorld() {
3 |
4 |         // In steps 6-10, you will add additional code here.
5 |
6 |     }
7 | ...
```

Later, you'll set the `helloWorld` function as the script's entry point function. Every entry point script needs to have one.

A function only counts as an entry point function if it's linked to an entry point. You'll make that link in Step 11.

6. Inside the entry point function, create an object called **options**.

Many SuiteScript 2.x methods need or can take a plain JavaScript object as their argument. The method you use to create the "Hello, World!" dialog is one of them. This method takes an object with two parameters: `title` and `message`.

```
1 | ...
2 |     var options = {
3 |         title: 'Hello!',
4 |         message: 'Hello, World!'
5 |     };
6 | ...
```

7. Add a **try/catch statement**. You don't have to, but it helps your script handle errors gracefully. If an error happens and your try/catch handles it, your script—and any others on the page—can keep running. Try/catch can also help you figure out why something went wrong. Keep in mind, try/catch is a JavaScript feature, not something specific to SuiteScript 2.x.

A basic try/catch has two parts: the `try` block holds the code you want to run, and the `catch` block runs if there's a JavaScript error in the `try` block.

Add the try/catch statement after the object you created in Step 6.

```
1 | ...
2 |     try {
3 |
```

```

4      // In steps 8 and 9, you will add additional code here.
5
6      } catch (e) {
7
8      // In Step 10, you will add additional code here.
9
10     }
11     ...

```

8. For the first action in the try block, call the [N/ui/dialog Module](#) `alert()` method. This method pops up a dialog with a title, a message, and an OK button.

You call this method using the **dialog** object you set up in Step 4, along with the method name **alert**. To set the dialog's title and message, pass in the **options** object.

```

1      ...
2      dialog.alert(options);
3      ...

```

For more details about this method, check out the [dialog.alert\(options\)](#) reference page. The page title matches the code you've added to your script, but keep in mind that reference pages for standard module methods might not always match your naming conventions. For example, if you named your callback argument message, you'd call the alert method with `message.alert(options)`.

Also, in the documentation, each method's argument is usually called `options`, but in your script, you can name the object whatever you want. You can even create the object right when you call the method.

9. Add some logic to create a log entry when the dialog shows up successfully. You can do this with the globally available [log.debug\(options\)](#) method.

The `debug()` method is called on the `log` object. Unlike the `dialog` object, which you had to load, the `log` object is always available in every script. That's why you don't give it a name—you can always write `log.debug` to use this method.

This method takes an object with two properties: a title and a detailed message. This time, create the object right in the method call. That's different from what you did in Step 4, where you made the object first and then passed it by name to `alert()`.

```

1      ...
2      log.debug ({
3          title: 'Success',
4          details: 'Alert displayed successfully'
5      });
6      ...

```

The log entry shows up in the UI when someone triggers the dialog alert. You'll find out how to view the log in [Test the Hello World Script](#).

10. In the catch block, add some logic to create a log entry if there's an error. You can use the globally available [log.error\(options\)](#) method for this. The `log.error()` method works a lot like `log.debug()`, but the log entry is marked as an error instead of a debug message.

```

1      ...
2      log.error ({
3          title: e.name,
4          details: e.message
5      });
6      ...

```

11. Right after the entry point function, add a return statement. In every SuiteScript 2.x entry point script, your return statement needs at least one line with two parts:

- An entry point, which is [pageInit\(scriptContext\)](#) here.
- A function, which is `helloWorld` here.

```

1 | ...
2 |     return {
3 |         pageInit: helloWorld
4 |     };
5 | ...

```

Because of this reference, `helloWorld` counts as an entry point function.

Even though this script uses only one entry point, your return statement can include more if you want. You can use multiple entry points if they all match the script type set by the `@NScriptType` tag at the top of your file. For example, besides `pageInit(scriptContext)`, the client script type also has `saveRecord(scriptContext)`. So if you want the script to do one thing when the page loads and something else when the user clicks Save, you can use both entry points. For an example of a script with multiple entry points, check [SuiteScript Client Script Sample](#).

12. Save the file and name it `helloWorld.js`.

Copy the Full Script

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

This section shows the full sample script. If you haven't already created the script file by following the steps in [Create the Client SuiteScript Script File Step by Step](#), copy and paste the following code into a text file. Save the file and name it `helloWorld.js`.

```

1 | /**
2 |  * @NApiVersion 2.0
3 |  * @NScriptType ClientScript
4 |  */
5 |
6 |
7 | define(['N/ui/dialog'],
8 |
9 |     function(dialog) {
10 |
11 |         function helloWorld() {
12 |
13 |             var options = {
14 |                 title: 'Hello!',
15 |                 message: 'Hello, World!'
16 |             };
17 |
18 |             try {
19 |
20 |                 dialog.alert(options);
21 |
22 |                 log.debug ({
23 |                     title: 'Success',
24 |                     details: 'Alert displayed successfully'
25 |                 });
26 |
27 |             } catch (e) {
28 |
29 |                 log.error ({
30 |                     title: e.name,
31 |                     details: e.message
32 |                 });
33 |             }
34 |         }
35 |
36 |         return {
37 |             pageInit: helloWorld
38 |         };

```

```
39 | });
```

 **Note:** If the script sample is split across multiple pages, make sure you copy all of the code.

Upload the Script File to NetSuite

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

After you've created your entry point script file, upload it to your NetSuite File Cabinet.

To upload the script file:

1. In the NetSuite UI, go to Documents > File > SuiteScripts.
2. In the left pane, select the SuiteScripts folder and click **Add File**.
3. Follow the prompts to find the helloWorld.js file on your computer and upload it.

Keep in mind that even after you upload the file, you can edit it right in the File Cabinet if you need to. For details, see the help topic [Editing Files in the File Cabinet](#).

Create a Script Record and Script Deployment Record

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

In general, before an entry point script can run in your account, you'll need to create a script record for the script file. You'll also need to create a script deployment record.

The script deployment record holds part of the logic for when the script runs. Some of that logic is in the script itself, based on the entry point you use. For example, if you use the [pageInit\(scriptContext\)](#) entry point, you're telling the system to run the script when a page loads. But you also need to say which specific page should trigger the script. In other words, you have to tell the system which record type this script should run on, and you do that with the script deployment record.

You can create these records programmatically, or you can set them up in the UI, as shown in the following steps.

To create the script record and script deployment record:

1. Go to Customization > Scripting > Scripts > New.
2. In the **Script File** dropdown list, select helloWorld.js.

If you haven't uploaded the file yet (see Step 3), you can do it right from this page. Point to the right of the dropdown list to see a plus icon. Click the plus icon to open a window where you can upload your file.

3. After you selected the file, click **Create Script Record**.

The system will show a new script record, with helloWorld.js listed on the **Scripts** subtab.

4. Fill out the required fields like this:

- In the **Name** field, enter **Hello World Client Script**.

- In the **ID** field, enter **_cs_helloworld**.
 - 5. Click the **Deployments** subtab to create the deployment record.
 - 6. Add a line to the sublist like this:
 - Set the **Applies to** dropdown to **Task**. If you want the dialog to show up for a different record type, select that from the list instead.
 - In the **ID** field, enter **_cs_helloworld**.
- Leave the other fields as they are. The **Status** field is set to **Testing**, which means the script won't deploy for other users. If you want to make this customization available to everyone later, edit the deployment and set the status to **Released**.
7. Click **Save**.

The system creates the script and script deployment records.

Test the Hello World Script

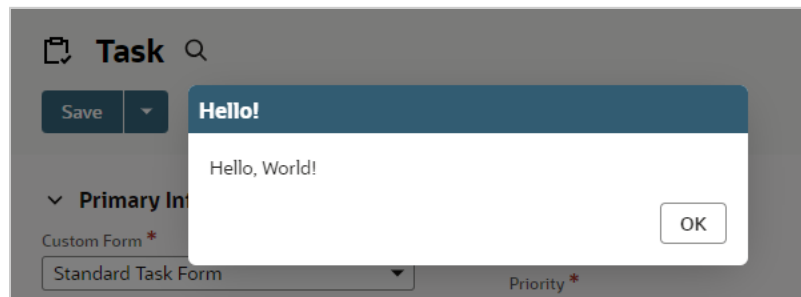
Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Now that the script is deployed, you should make sure it runs as expected.

To test the script:

1. Check that the dialog alert pops up when it should:
 1. Open a task record by going to Activities > Scheduling > Tasks > New.

If the script is working properly, the system will show a dialog alert.



2. Confirm and close the dialog by clicking **OK**.
2. Check that the expected log entry has been saved to the script deployment record:
 1. Go to Customization > Scripting > Script Deployments.
 2. Find your deployment and click **View**.

Script Deployments						Scripts
Filters						
☐ Show Undeployed						Total: 1
Internal ID	Edit View	ID	Script	Record Type	Last Modified	
4021	Edit View	customdeploy_cs_helloworld	Hello World	Task	21.5.2021 3:59 pm	

3. Click the **Execution Log** subtab.
- The subtab should show an entry like the following.

Audience • Scripts • Context Filtering • Execution Log • System Notes								
View			Type					
Default Script Notes View			- All -					
Customize View			Remove all			Refresh		
#	View	Type	Title	Date	Time	User	Details	Remove
1	View	Debug	Success	4.10.2024	11:41 am	Smith, John	Alert displayed successfully	Remove

You'll also see the same entry on the Execution Log of the script record.

If there was an error, the error log entry you created would show up instead of the debug entry.

Next Steps in Learning SuiteScript 2.x

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Now that you've deployed your first script, you might want to check out other topics in the [SuiteScript 2.x API Introduction](#). Or, if you want to experiment with other script samples, try these:

- Another commonly used script type is the user event type. This one's for server scripts that run when users do certain things with records. To try a simple user event script, see [SuiteScript 2.x User Event Script Tutorial](#).
- For a slightly more complex user event script, check out [SuiteScript 2.x User Event Script Sample](#).
- For an example of a custom module script and a user event script that calls a custom module function, see [SuiteScript 2.x Custom Module Tutorial](#).

SuiteScript 2.x Script Basics

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Some components are common to all SuiteScript 2.x scripts. This topic covers a few of them and includes the following sections:

- [Modular Architecture](#)
- [Objects as Arguments](#)

Note: If you haven't already, review [SuiteScript 2.x Hello World](#), and pay special attention to the [Key SuiteScript 2.x Development Concepts](#) section.

Modular Architecture

SuiteScript 2.x is modular, meaning all its APIs are organized into standard modules. When your script loads a standard module, it can use any of that module's APIs.

Every SuiteScript 2.x entry point and custom module script needs to be set up to create a module. This doesn't apply to scripts meant only for on-demand debugging in the NetSuite Debugger or a browser console. For example, an entry point script has to create a module that the NetSuite application can use.

In the same way, a custom module script also needs to create a module that other scripts can load and use. In both cases, the module should be set up to return an object, which is usually a function.

You create a module using the globally available [define Object](#). You can use the define function to create and load modules in your script (see [SuiteScript 2.x Hello World](#)).

The define function can take one to three arguments, but most examples in this help center use two:

- The first argument is a list of dependencies, like standard [SuiteScript 2.x Modules](#). These modules load when the script runs.
- The second argument is a callback function. This function needs a return statement that identifies at least one standard or custom entry point. The entry point should be linked to an object that's returned when it's called—usually, that object is a function.

If you want to learn more about SuiteScript 2.x's modular architecture, check out the [Asynchronous Module Definition \(AMD\) Specification](#), which SuiteScript 2.x uses. But you don't need to know AMD in depth to use SuiteScript 2.x.

 **Note:** Some samples in this help center use the [require Function](#) instead of the define function. Samples that use require are meant for on-demand debugging, either in the NetSuite Debugger or a browser console. If you want to use a script for anything other than on-demand debugging, it needs to use the [define Object](#).

Objects as Arguments

In SuiteScript 2.x, the arguments you pass to methods are usually objects. For two examples of why this matters, check out the following sections:

- [Using Objects When Calling Standard Methods](#)
- [Context Objects for Standard and Custom Entry Points](#)

Using Objects When Calling Standard Methods

Many SuiteScript 2.x methods need you to provide one or more pieces of information, all wrapped up in a single object. This object has one or more properties, which can be required or optional. You need to set a value for each required property, as well as any optional ones you use in your script.

When you set a value on a record, you use the [Record.setValue\(options\)](#) method. With this method, you need to provide two things: the ID of the field and the value you want to set. To do that, you create an object with a **fieldId** property and a **value** property, then pass that object to the method. For example:

```
1 var myObject = {
2   fieldId: 'Hello!',
3   value: 'Hello, World!'
4 };
5
6 myRecord.setValue(myObject);
```

Another way to pass an object is to define it right inside the method call. For example:

```
1 myRecord.setValue({
2   fieldId: 'Hello!',
3   value: 'Hello, World!'
4 });
```

In the documentation for standard SuiteScript 2.x methods, each method's argument is usually called **options**. But in your script, you can name the object whatever you want. For an example, see [SuiteScript 2.x User Event Script Tutorial](#).

Note: In SuiteScript 2.x, you can't add inline comments inside an object. You might not see a specific error, but your script won't work right if you include comments inside an object.

Context Objects for Standard and Custom Entry Points

Every SuiteScript 2.x script—whether it's an entry point script or a custom module script—needs to use an entry point. Each entry point should match up with an object you define in the script, which is usually a function. When it is, it's called an entry point function.

Most of the time, an entry point function gets access to an object provided by the system. This object lets your script access data and take actions based on the context where the script is running, which is why it's often called a context object.

Depending on what your script needs to do, this object can be a key part of your script. You can name it whatever you like, but most examples here call it **context**.

Note: For more about entry points, see the [Key SuiteScript 2.x Development Concepts](#) section of [SuiteScript 2.x Hello World](#).

Context Objects in Entry Point Scripts

Every standard script type has entry points specific to that type. Most of these entry points can access a context object that gives you data or methods. The properties of this object depend on the entry point you're using. For details about the context object for each entry point, check the documentation for that entry point.

For an example of how context objects work in a user event script, see [SuiteScript 2.x User Event Script Tutorial](#).

Context Objects in Custom Module Scripts

Similar to entry point scripts, a custom module script's entry point function can also get a context object. But in this case, the object isn't provided by NetSuite—it comes from the SuiteScript that calls the module.

If you want a custom entry point function to get an object, the calling SuiteScript needs to be set up for that. The calling script has to create the object, set any needed properties, and pass it to the custom module script, which can then use it.

For an example, see [SuiteScript 2.x Custom Module Tutorial](#).

Note: To see how these components show up in scripts, check out [SuiteScript 2.x Anatomy of a Script](#) and [SuiteScript 2.x Anatomy of a Custom Module Script](#).

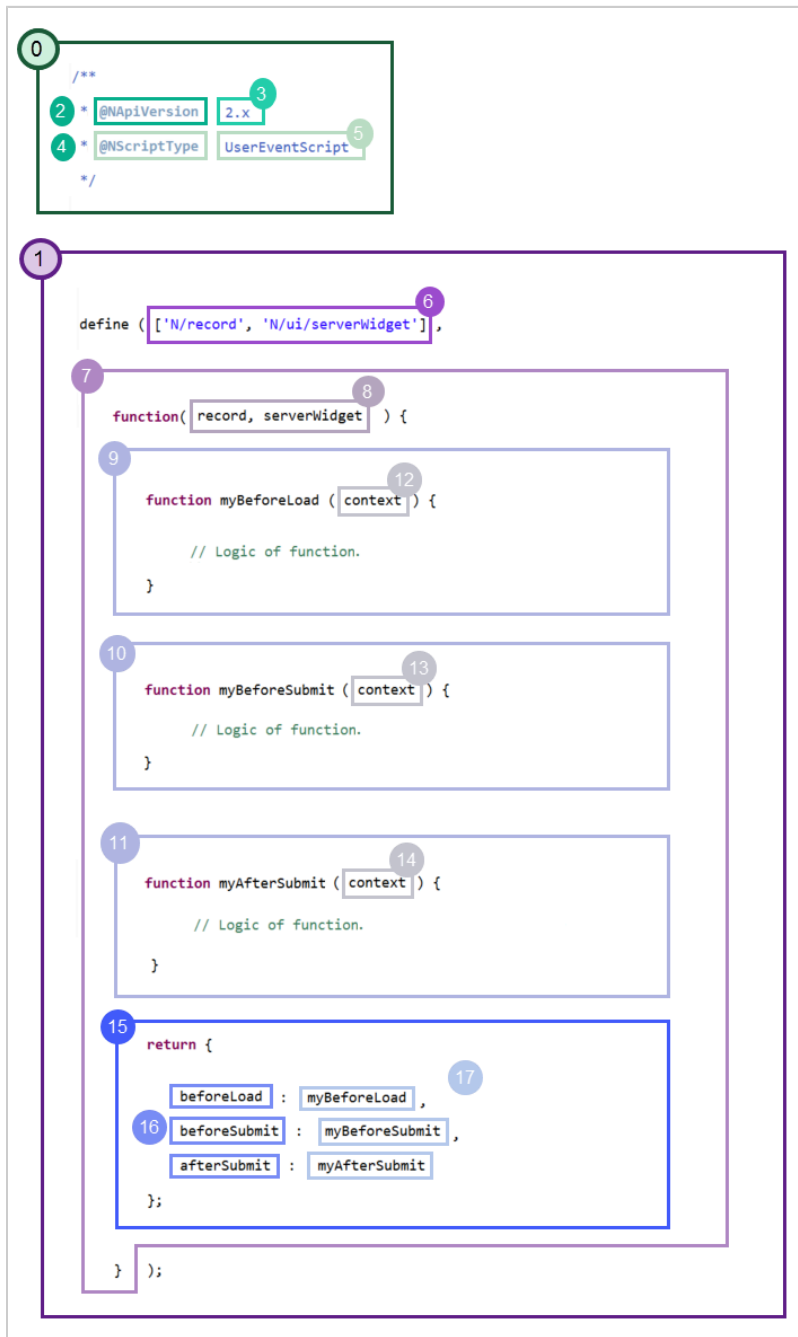
SuiteScript 2.x Anatomy of a Script

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

All SuiteScript 2.x entry point scripts follow the same basic structure, as shown in the diagram below. Check out the table after the diagram for an explanation of each part. For a walkthrough of the script

creation steps, see [SuiteScript 2.x Script Creation Process](#), and for step-by-step instructions to create a sample script, see [SuiteScript 2.x Hello World](#).

Note: If you need help with any of the terms in this topic, [SuiteScript 2.x Hello World](#).



General Area	Callout	Description
0 — JSDoc tags	2 and 3	The <code>@NApiVersion</code> tag is required in an entry point script, and its value can be 2.0, 2.x, or 2.X.
	4 and 5	The <code>@NScriptType</code> tag is also required. The value isn't case sensitive, but using Pascal case, like in this example, makes it easier to read.

General Area	Callout	Description
1 — define statement	6	The first argument for the define function is a list of dependencies, or modules the script loads. In this script, it uses the to work with records and the to work with forms.
	7	The second argument for the define function is a callback function.
	8	The callback function's arguments represent the modules you listed as dependencies. The first argument is for the N/record Module , and the second is for N/ui/serverWidget Module . The order of these matches the order in the define function's dependency list. (Callout 6) You can use these objects anywhere in the callback function to access the module APIs. You can name them whatever you like, but it's best to use names similar to the module names.
	9, 10, and 11	These are the entry point functions. To use a function, you need to name it in the return statement next to an entry point, like in Callout 17.
	12, 13, and 14	This is the context object that's provided to each entry point function. What's included in this object depends on the entry point you're using. For more details, see Context Objects for Standard and Custom Entry Points .
	15	The callback function's return statement.
	16	The entry points used. The script needs to use at least one entry point, and any entry points you use must match the script type set by the @NScriptType tag (see Callouts 4 and 5).
	17	References to the script's entry point functions. For each entry point, your script needs to point to an entry point function you've defined elsewhere in the script.

SuiteScript 2.x Script Creation Process

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Here's the basic process flow for creating a SuiteScript 2.x script. For a more detailed explanation with a sample script, check out [SuiteScript 2.x Hello World](#). If you want to see a diagram of a valid script's structure, see [SuiteScript 2.x Anatomy of a Script](#).

Stage	Description	Additional Information
1	Use the define() function to load SuiteScript 2.0 modules in your entry point script. This is the script you attach to the script record.	Modular Architecture SuiteScript 2.x Global Objects
2	Add the required JSDoc tags to your entry point script.	SuiteScript 2.x JSDoc Validation
3	Add at least one entry point function to your script. An entry point function is a named function that runs when an entry point is triggered.  Important: Your entry point script can only use one script type. For example, it can't return both a beforeLoad entry point and an onRequest entry point.	SuiteScript 2.x Script Types
4	Organize your supporting code into custom modules (instead of SuiteScript 1.0 libraries). Create these modules with the define() function, then load them in your entry point script.	SuiteScript 2.x Global Objects

Stage	Description	Additional Information
5	Upload and deploy your script to NetSuite.	SuiteScript 2.x Entry Point Script Creation and Deployment

SuiteScript 2.x Advantages

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

SuiteScript 2.x is a complete redesign of the SuiteScript model from 1.0. This topic covers some of the advantages SuiteScript 2.x has over 1.0.

i Note: Use SuiteScript 2.x for any new scripts you create, and think about converting your SuiteScript 1.0 scripts to SuiteScript 2.0 or 2.1. SuiteScript 1.0 isn't being updated anymore, and there are no new features or enhancements for it. While SuiteScript 1.0 scripts are still supported, you should use SuiteScript 2.0 or 2.1 for any new or majorly updated scripts to take advantage of new features, APIs, and functionality improvements.

- [Complexity Management and Intuitive Code Organization](#)
- [Automatic Dependency Management](#)
- [Modern Programming Syntax and Behavior](#)
- [Functionality Enhancements](#)

Complexity Management and Intuitive Code Organization

SuiteScript 2.x is built around modularity. Modern SuiteApps need complex scripts with lots of code and files, and modularity helps manage that complexity. It also adds encapsulation, makes code organization more intuitive, and prevents global variable or method naming conflicts.

SuiteScript 2.x comes with a full set of new APIs, all organized into modules based on what they do. For example, you use the N/file module to work with files in NetSuite. Your script only loads the modules it needs. In SuiteScript 1.0, all APIs are in one big global namespace, so every script uses the whole namespace no matter which APIs it needs.

SuiteScript 2.x also lets you create your own custom modules. You can use these to organize your code (instead of SuiteScript 1.0 libraries). You can even add custom modules to SuiteApps and share them with third parties.

For additional information, see the following help topics:

- [SuiteScript 2.x Hello World](#)
- [SuiteScript 2.x Script Basics](#)
- [SuiteScript 2.x Modules](#)
- [SuiteScript 2.x Entry Point Script Creation and Deployment](#)
- [SuiteScript 2.x Custom Modules](#)

Note: SuiteScript 2.x uses the Asynchronous Module Definition (AMD) specification to define and load JavaScript modules and their dependencies. For more about AMD, see <http://requirejs.org/docs/whyamd.html> and <https://github.com/amdjs/amdjs-api/blob/master/AMD.md>.

Automatic Dependency Management

SuiteScript 2.x gives you built-in dependency management. You define which modules need to load before your script runs, and the module loader takes care of loading those modules and all their dependencies automatically. This means you can focus on your logic instead of worrying about dependencies and load order.

For additional information, see the following help topics:

- [SuiteScript 2.x Hello World](#)
- [SuiteScript 2.x Script Basics](#)
- [SuiteScript 2.x Entry Point Script Creation and Deployment](#)
- [SuiteScript 2.x Custom Modules](#)

Modern Programming Syntax and Behavior

SuiteScript 2.x is designed to be similar to JavaScript, so it's easier for JavaScript developers to pick up. The syntax is straightforward and the behavior is consistent. Some of the enhancements include:

- **Third party JavaScript API support:** SuiteScript 2.x supports all standard JavaScript. The SuiteScript 2.x APIs let you access NetSuite features programmatically. For generic logic, you can use custom modules to load your favorite third-party JavaScript APIs.
- **SuiteScript 1.0 nlapi/nlobj prefix retirement:** SuiteScript 2.x is designed to look and work like modern JavaScript, so its methods and objects don't use the nlapi or nlobj prefixes anymore. This change fits with the modular structure of SuiteScript 2.x—methods and objects are now inside modules, while SuiteScript 1.0 kept everything in the nlapi and nlobj namespaces.
- **Usage of properties and enumerations:** SuiteScript 2.x uses properties and enumerations. Most getter and setter methods from SuiteScript 1.0 are now properties, and enumerations are used for common constants like standard record types.

Note: JavaScript doesn't have a built-in enumeration type. In SuiteScript 2.x, an "enumeration" (or enum) is a plain JavaScript object with a flat, map-like structure, where each key points to a read-only string value.

- **Updated sublist and column indexing:** Most programming languages start indexing at 0, but SuiteScript 1.0 started sublist and column indexing at 1. SuiteScript 2.x now starts indexing at 0 to match modern JavaScript.
- There's also a new version, SuiteScript 2.1, which is the latest minor version. It adds support for more ECMAScript features and syntax. For more info, see [SuiteScript 2.1](#).

For additional information, see the following help topics:

- [SuiteScript 2.x Hello World](#)
- [SuiteScript 2.x Entry Point Script Creation and Deployment](#)
- [SuiteScript 2.x Custom Modules](#)

Functionality Enhancements

The following enhancements are exclusive to SuiteScript 2.x:

- [Map/Reduce Script Type](#)
- [Asynchronous Processing \(Promises\)](#)
- [SFTP File Transfer API](#)
- [Cache API](#)
- [Search Pagination API](#)
- [Flat File Streaming API](#)
- [Expanded Support for HTTP Content Type Headers](#)
- [New Encryption/Encoding Functionality](#)

Map/Reduce Script Type

SuiteScript 2.x introduces a new server script type based on the map/reduce model. Map/reduce scripts give you a structured way to process lots of records on the server. If you're a SuiteCloud Plus user, you can even process records in parallel across multiple processors, and you get to choose how many processors to use right from the script deployment record.

For additional information, see [SuiteScript 2.x Map/Reduce Script Type](#).

Asynchronous Processing (Promises)

Promises are JavaScript objects that represent the result of an asynchronous process. Once you create a promise, it acts as a placeholder for whether an operation will succeed or fail. While a promise is waiting in the background, the rest of your script can keep running.

In SuiteScript 2.x, all client scripts and some server scripts support promises. With promises, you can write asynchronous code that's both intuitive and efficient. SuiteScript 2.x includes promise APIs for certain modules, and you can also create custom promises in any client script.

For additional information see the help topic [Promise Object](#).

SFTP File Transfer API

SuiteScript 2.x supports SFTP (Secure File Transfer Protocol), so you can securely transfer files between NetSuite and external FTP servers. SFTP is packaged with SSH and works like FTP, but with a secure connection. To authorize the server, you'll need a password GUID (Globally Unique Identifier) and either of the following:

- DSA (Digital Signature Algorithm)
- ECDSA (Elliptical Curve Digital Signature Algorithm)
- RSA (cryptosystem) host key

For additional information, see the help topic [N/sftp Module](#).

Cache API

The SuiteScript 2.x Cache API lets you load data into a cache and share it with one or more scripts. This feature helps cut down the time it takes to retrieve data.

For additional information, see the help topic [N/cache Module](#).

Search Pagination API

The SuiteScript 2.x Search Pagination API lets you page through search results. This makes scripts run faster and gives you an easy way to work with large sets of search data.

For additional information, see the help topic [N/search Module](#).

Flat File Streaming API

With SuiteScript 1.0, you can't easily access files larger than 10MB. You'd have to split big files into smaller ones to read, write, or append their contents in memory.

The SuiteScript 2.x Flat File Streaming API lets you efficiently process and stream large CSV and plain text files. You can load and edit each line in memory, then put the lines back together. The 10MB limit now only applies to individual lines, not the whole file.

For additional information, see the help topic [N/file Module](#).

Expanded Support for HTTP Content Type Headers

SuiteScript 2.x supports all HTTP content types, for both client requests and server response headers.

For additional information, see the help topics [N/http Module](#) and [N/https Module](#).

New Encryption/Encoding Functionality

SuiteScript 2.x adds better encryption, decryption, and hashing with the [N/crypto Module](#), plus extra encoding features in the [N/encode Module](#).

SuiteScript 2.x Terminology

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: These definitions reflect the usage of the terms as they apply to SuiteScript 2.0 and SuiteScript 2.1. Some terms may have alternate meanings outside of the SuiteScript context.

Term	Definition
Class	A class is a programming template that allows you to create complex objects; classes encapsulate data with code to work on that data. This term applies only to SuiteScript 2.1.
Constants	A constant reference to a value. This term applies only to SuiteScript 2.1.
Custom Module	A user-defined module that serves as a JavaScript library or supporting logic. This module is separate from your entry point script. Your entry point script loads this module as a dependency. For more information about custom modules, see SuiteScript 2.x Custom Modules .
Deferred Dynamic Mode	See the definition for Standard Mode.
Destructuring	A JavaScript expression that allows us to extract data from arrays, objects, and maps and set them into new, distinct variables. This applies only to SuiteScript 2.1.
Dynamic Mode	See the definition for Standard Mode.

Term	Definition
Entry Point	An entry point represents the juncture at which the system grants control of the NetSuite application to the script. Each script type includes one or more entry points that are exclusive to that type. When that entry point is invoked, the system knows to execute its corresponding entry point function. For more information about entry points, see SuiteScript 2.x Entry Point Script Creation and Deployment, SuiteScript 2.x Hello World, and SuiteScript 2.x Script Types.
Entry Point Function	A function that executes when an entry point is invoked. For more information about entry point functions, see SuiteScript 2.x Entry Point Script Creation and Deployment and SuiteScript 2.x Hello World.
Entry Point Script	An entry point script is the primary script that you attach to the script record. This script identifies the script type, entry points, and entry point functions. Each entry point script must include at least one entry point and entry point function. For more information about entry point scripts, see SuiteScript 2.x Entry Point Script Creation and Deployment and SuiteScript 2.x Hello World.
Enum	JavaScript does not include an enumeration type. The SuiteScript 2.x documentation utilizes the term enumeration (or enum) to describe a plain JavaScript object with a flat, map-like structure. Within this object, each key points to a read-only string value.
Governance	Governance ensures that an individual script does not consume an unreasonable amount of resources. The SuiteScript governance model is based on usage units. Certain API calls cost a specific number of usage units. Each script type also has a maximum number of usage units that it can expend per execution. If the number of allowable usage units is exceeded, the script is terminated and an error is thrown. For more information about governance, see the help topic SuiteScript Governance and Limits.
Script	A script refers to an aggregate of ■ An entry point script file ■ All custom modules used by the entry point script ■ The script record associated with the entry point script For more information about the components that make up a script, see SuiteScript 2.x Entry Point Script Validation, SuiteScript 2.x Custom Modules, Script Record Creation, and SuiteScript 2.x Hello World.
Script Deployment	A script deployment determines some of the associated script's runtime behavior. The settings you can define on a script deployment record include: ■ The record types the script executes against ■ When the script is executed ■ The audience and role restrictions ■ The script log levels The settings found on the deployment record vary based on script type. For more information about script deployments, see SuiteScript 2.x Entry Point Script Creation and Deployment and SuiteScript 2.x Hello World.
Script Type	SuiteScript 2.x supports several script types. Each script type is designed for specific types, situations, and triggering events. For more information about script types, see SuiteScript 2.x Script Types.
Standard Mode	There are two modes you can operate in when you work with a record in SuiteScript:

Term	Definition
	- In standard mode, the record's body fields and sublist line items aren't sourced, calculated, or validated until the record is saved. - In dynamic mode, the record's body fields and sublist line items are sourced, calculated, and validated in real-time. A record in dynamic mode emulates the behavior of a record in the UI.  Note: Standard mode is also referred to as deferred dynamic mode. For more information about standard and dynamic modes, see SuiteScript 2.x Standard and Dynamic Modes and record.Record.
Usage Units	See the definition for Governance.

SuiteScript 2.x Developer Resources

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

SuiteScript 2.x Help

See the following help sections for information about developing with SuiteScript 2.x:

- [SuiteScript 1.0 to SuiteScript 2.x API Map](#)
- [SuiteScript 2.x Script Types](#)
- [SuiteScript 2.x Global Objects](#)
- [SuiteScript 2.x Modules](#)
- [SuiteScript 2.x JSDoc Validation](#)
- [SuiteScript 2.x Entry Point Script Creation and Deployment](#)
- [SuiteScript 2.x Custom Modules](#)
- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript 2.x Custom Pages](#)

SuiteScript 2.x Internal Resources

The following internal resources are available in addition to the SuiteScript 2.x help.

Resource	Description
NetSuite User Group	Official forum for the NetSuite community. Use the search field to find the SuiteScript 2.x board.
SuiteScript 2.0: Extend NetSuite with JavaScript	Oracle Training Course
SuiteScript 2.0 for Experienced Developers	Oracle Training Course
 SuiteScript Help Overview video	Video from the Oracle Video Hub

SuiteScript 2.x External Resources

The following external resources are available in addition to the SuiteScript 2.x help.

Resource	Description
AMD Specification	SuiteScript 2.x implements its modular architecture with the Asynchronous Module Definition (AMD) specification. AMD is used to define and load JavaScript modules and their dependencies.
RequireJS	RequireJS also implements the AMD specification.
https://www.promisejs.org/	Tutorials on JavaScript promises
Eloquent JavaScript	Free online JavaScript book
You Don't Know JS	Free online JavaScript book series
Object-oriented JavaScript for Beginners	MDN article on object oriented JavaScript
StackOverflow – SuiteScript 2.0	Online community for developers

SuiteScript Reserved Words

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

In the ECMAScript specification, reserved words are identifiers that have special meaning. For example, the `var` reserved word indicates a variable declaration, and the `try` reserved word indicates the start of a try-catch block. You can't use reserved words as variable names, labels, or function names in languages based on the ECMAScript specification, such as JavaScript. For more information about the ECMAScript specification, see [JavaScript language resources](#).

SuiteScript is based on the ECMAScript specification, and you can't use ECMAScript reserved words as variable names or function names in your SuiteScript scripts. This restriction applies to both SuiteScript 2.0 and SuiteScript 2.1. If you use a reserved word as a variable name or function name, your script may produce a syntax error when it runs.

To view the list of reserved words, visit the following link:

[JavaScript Reserved Words](#)

You shouldn't use any of the reserved words that are included in ECMAScript 2015. For best compatibility, you should also avoid using any reserved words that may be supported in future ECMAScript editions.

i Note: SuiteScript 2.x provides global objects including the [log Object](#) and the [util Object](#). If you're using the words 'log' or 'util' as variable names in your SuiteScript 1.0 scripts and you have both SuiteScript 1.0 and SuiteScript 2.x scripts running on the same record, you'll receive an error. Both the 'log' and 'util' words are reserved in SuiteScript 2.x. You'll need to update your SuiteScript 1.0 scripts that use 'log' or 'util' as variables names.

SuiteScript Versioning Guidelines

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

See the following sections for information about SuiteScript versioning:

- [SuiteScript 2.1](#)
- [SuiteScript Versioning](#)
- [Version Cohabitation Rules](#)



Important: If you're using SuiteScript 1.0 for your scripts, consider converting these scripts to SuiteScript 2.0 or 2.1. Use SuiteScript 2.0 and 2.1 to take advantage of new features, APIs, and functionality enhancements. For more information, see [SuiteScript 2.x Advantages](#) and [SuiteScript 2.1](#).

SuiteScript 2.1

A new version of SuiteScript is now available. This new version, SuiteScript 2.1, includes the existing features and functionality that SuiteScript 2.0 offers, and it also supports new language capabilities that were introduced in the ECMAScript 2019 (ES2019) edition of the ECMAScript specification. SuiteScript 2.1 supports all server script types, such as user event scripts, scheduled scripts, and Suitelets, and it is backward compatible with SuiteScript 2.0.

This latest version of SuiteScript is separate from previous SuiteScript versions (1.0 and 2.0). You can create and run SuiteScript 2.1 scripts alongside SuiteScript 1.0 and 2.0 scripts in your account.

To learn more about SuiteScript 2.1, see [SuiteScript 2.1](#).

SuiteScript Versioning

SuiteScript 2.0 and all future releases of SuiteScript will maintain the following versioning system.

Version Type	Numbering Pattern	Description
Major	Version 2.0, 3.0, 4.0	Major versions of SuiteScript include significant functionality changes and improvements. Major versions aren't backward compatible with previous versions.
Minor	Version 2.0, 2.1, 2.2	Minor versions of SuiteScript include enhancements to existing features. Minor versions are backward compatible with all versions released since the last major version. For example, SuiteScript 2.1 is backward compatible with SuiteScript 2.0, but it is not backward compatible with SuiteScript 1.0.
Patch	Does not apply	Patch versions of SuiteScript are included with regular NetSuite bug fix releases. Patch versions are backward compatible with all versions released since the last major version.

You can also specify a SuiteScript version of 2.x in your scripts. The 2.x value usually represents the latest version of SuiteScript that is generally available and does not represent any versions that are released as beta features. However, this doesn't apply to SuiteScript 2.1. In this release, the 2.x value indicates that a script uses SuiteScript 2.0, not SuiteScript 2.1. You can still use SuiteScript 2.1 and all of its features in your server scripts, but your 2.x scripts won't automatically run as SuiteScript 2.1 scripts until a future release. For more information about specifying a version for your script, see [SuiteScript 2.x JSDoc Validation](#) and [SuiteScript Versions](#).

You can use an account-level preference, Execute SuiteScript 2.x Server Scripts As, to specify that the 2.x value should represent SuiteScript 2.1 instead of SuiteScript 2.0. Setting this preference to 2.1 applies to

all scripts in your account that use the 2.x value. For more information about this preference and how to use it, see [Enabling SuiteScript 2.1 at the Account Level](#).

Version Cohabitation Rules

You can't use APIs from different major versions of SuiteScript in one script. For example, you cannot use SuiteScript 1.0 APIs and SuiteScript 2.0 APIs in the same script. However, you can have multiple scripts in your account that use different SuiteScript versions. These scripts can be deployed in the same account, in the same SuiteApp, and on the same record.

You can use `nlapiRequestRestlet(scriptId, deploymentId, urlParams, body, headers, httpMethod)` in SuiteScript 1.0 scripts to call RESTlets written in SuiteScript 2.x code. For more information, see [Using SuiteScript 2.x Scripts with SuiteScript 1.0 Scripts](#).

You can't use SuiteScript 2.1 custom modules with SuiteScript 2.0 entry point scripts. For more information about custom modules, see [SuiteScript 2.x Custom Modules](#).

The Map/Reduce script type is a new functionality introduced with SuiteScript 2.0. You cannot use `nlapiScheduleScript(scriptId, deployId, params)` to schedule a Map/Reduce script. See [SuiteScript 2.x Map/Reduce Script Type](#) for more information.

SuiteScript 2.1

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

SuiteScript 2.1 is the latest version of SuiteScript and can be used for server and client scripts. For server scripts, the Graal runtime engine is used. This engine supports ECMAScript 2023 which lets you use new language capabilities and functions in your scripts and may also improve script performance. For client scripts, you can include functions and features supported by the ECMAScript version your browser uses.

SuiteScript 2.1 supports several new ECMAScript features you can use in your scripts. For example, the spread operator lets you expand an iterable inside another object or array. You can use the spread operator to combine values from two objects into a new one. For more information about other powerful new language features supported in SuiteScript 2.1, see [SuiteScript 2.1 Language Examples](#).

SuiteScript 2.0 uses a different runtime engine and is based on a less sophisticated version of ECMAScript (ES5.1). For more information about ECMAScript, see [JavaScript language resources](#).

A few notes:

- The functionality provided by the SuiteScript API is the same for SuiteScript 2.0 and SuiteScript 2.1, with a few exceptions, such as the N/lm and N/pgp modules. For more information about this API, see the help topic [SuiteScript 2.x API Reference](#).
- You can debug SuiteScript 2.1 scripts using your browser's debugger. For more information about debugging SuiteScript 2.1 scripts, see the help topic [Debugging SuiteScript 2.1 Scripts](#).
- Some SuiteScript 2.1 features are not fully supported in client scripts, such as for subrecords. For example, you may get an error if your client script is deployed for All Records, since that includes subrecords. In this case, if you need to deploy to All Records, update your 2.1 client script to use ES5.1 features instead of newer ECMAScript ones. ECMAScript features.
- SuiteScript 2.1 client scripts are not currently supported in the Scriptable Cart. For more information about the Scriptable Cart, see the help topics [Scriptable Cart](#) and [SuiteScript for Scriptable Cart](#).
- The SuiteTax Engine does not support SuiteScript 2.1. To use the full functionality of the SuiteTax Engine, you must use SuiteScript 2.0. For more information about the SuiteTax Engine, see the help topic [SuiteTax Engine](#).

For more information about how to execute your script as a SuiteScript 2.1 script, see [Executing Scripts Using SuiteScript 2.1](#).

Executing Scripts Using SuiteScript 2.1

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The Execute SuiteScript 2.x Server Scripts As account-level preference lets you specify which version of SuiteScript (SuiteScript 2.0 or SuiteScript 2.1) is used when 2.x is specified in the @NApiVersion JSDoc tag in the script. With this preference, you can execute all of your SuiteScript 2.x scripts as SuiteScript 2.1 scripts without having to make changes to any script. For more information about this preference, see [Enabling SuiteScript 2.1 at the Account Level](#).

Currently, by default, a value of 2.x for the @NApiVersion JSDoc tag will resolve to 2.0 when the script is uploaded and executed. This means that all scripts that include @NApiVersion 2.x are executed as SuiteScript 2.0 scripts. In a future release, scripts with 2.x will automatically run as SuiteScript 2.1 scripts by default. When this happens, account administrators will receive notifications that 2.x annotated scripts will begin executing as SuiteScript 2.1 scripts. Notifications will be sent to account administrators if there is at least one script in their account that is annotated with the @NApiVersion JSDoc tag value of 2.x.

There are two ways you can execute your scripts using SuiteScript 2.1:

- **Execute a single script using SuiteScript 2.1** — You can use the @NApiVersion JSDoc tag in your script file to indicate explicitly that the script should execute as a SuiteScript 2.1 script. For more information, see [Executing a Single Script Using SuiteScript 2.1](#).
- **Execute all SuiteScript 2.x scripts using SuiteScript 2.1** — You can use the Execute SuiteScript 2.x Server Scripts As account-level preference to specify that all scripts that specify 2.x for the @NApiVersion JSDoc tag value should execute as SuiteScript 2.1 scripts. For more information, see [Enabling SuiteScript 2.1 at the Account Level](#).

For more information about the @NApiVersion JSDoc tag, see [SuiteScript 2.x JSDoc Validation](#).

Executing a Single Script Using SuiteScript 2.1

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

To execute a single script as a SuiteScript 2.1 script, you must annotate the script to explicitly use SuiteScript 2.1. This lets you execute a single script as a SuiteScript 2.1 script without affecting any other scripts in your account. This approach is useful if you want to use SuiteScript 2.1 features in only a few of your scripts. It is also useful if you aren't ready to execute all SuiteScript 2.x annotated scripts in your account as SuiteScript 2.1 scripts.

To execute a single script using SuiteScript 2.1:

1. In your script file, include the following JSDoc comment block at the beginning of the file:

```
1 | /**
2 |  * @NApiVersion 2.1
3 |  */
```

2. Add other JSDoc tags (such as @NScriptType) as appropriate. For more information, see [SuiteScript 2.x JSDoc Validation](#).
3. Upload and deploy your script. Your script will be validated as a SuiteScript 2.1 script. For more information, see [SuiteScript 2.x Entry Point Script Creation and Deployment](#).

Enabling SuiteScript 2.1 at the Account Level

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

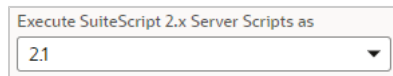
When you use the @NApiVersion JSDoc tag in a script file, you can specify a value of 2.x. This value usually represents the latest version of SuiteScript that is generally available. However, in this release, this value represents SuiteScript 2.0 by default. You can use the Execute SuiteScript 2.x Server Scripts As account-level preference to specify that the 2.x value should represent SuiteScript 2.1 instead of SuiteScript 2.0. Setting this preference to 2.1 applies to all scripts in your account that use the 2.x value. Scripts that use the 2.0 or 2.1 value for the @NApiVersion JSDoc tag are not affected. In other words, this preference doesn't affect scripts that specify 2.0 or 2.1 directly in the @NApiVersion tag.

You can use the Execute SuiteScript 2.x Server Scripts As preference to test how your SuiteScript 2.x scripts will execute using SuiteScript 2.1. This is helpful if you want to test all of your 2.x scripts as SuiteScript 2.1 scripts without having to specifically annotate each script with the @NApiVersion JSDoc tag value of 2.1. Using this preference, you don't need to change any script files to switch the SuiteScript version they execute as.

 [Account-Level Preference for SuiteScript 2.x Scripts](#)

To enable SuiteScript 2.1 at the account level:

1. Go to Setup > Company > Preferences > General Preferences.
2. In the **Execute SuiteScript 2.x Server Scripts As** dropdown list, select **2.1**.



The image shows a dropdown menu with the text 'Execute SuiteScript 2.x Server Scripts as' above it. The dropdown is open, showing '2.1' as the selected option.

3. Click **Save**.

When you use this account-level preference, you should be understand how validation of the script occurs. If you have a SuiteScript annotated as @NApiVersion 2.x and you set the Execute SuiteScript 2.x Server Scripts As preference to 2.1, syntax of the script is validated as if the script was a SuiteScript 2.0 script. If you add syntax that is supported only in SuiteScript 2.1, you receive a syntax error when you upload the script file.

This approach is designed to ensure that SuiteScript 2.x scripts in your account continue to work even if the Execute SuiteScript 2.x Server Scripts As preference is changed from 2.1 back to 2.0. To test new SuiteScript 2.1 features and syntax, you should update the @NApiVersion JSDoc tag in your script files from 2.x to 2.1 explicitly. You can also upload new scripts as SuiteScript 2.1 scripts.

The following table summarizes when errors can occur with SuiteScript 2.1 scripts:

Scenario	Outcome
1. Set the Execute SuiteScript 2.x Server Scripts As preference to 2.1. 2. Open the script record of an executing script that uses an @NApiVersion value of 2.x. 3. Edit the script and add code that is supported only in SuiteScript 2.1 (such as using the Spread operator). 4. Save the updated script.	A syntax error occurs when you save the script. You can't save a script where you have added SuiteScript 2.1 syntax even if the Execute SuiteScript 2.x Server Scripts As preference is set to 2.1. To resolve this issue, change the @NApiVersion value to 2.1 to explicitly execute the script as a SuiteScript 2.1 script.
1. Upload a script that includes an @NApiVersion value of 2.x and also includes SuiteScript 2.1 syntax (such as using the Spread operator). 2. Change the value of the Execute SuiteScript 2.1 Server Scripts As preference to 2.0.	The script no longer works. When the account-level preference is set to SuiteScript 2.0, all 2.x scripts execute as SuiteScript 2.0 scripts. SuiteScript 2.1 syntax is not valid in a SuiteScript 2.0 script. To resolve this issue, do one of the following: ■ Change the @NApiVersion value to 2.1 to explicitly execute the script as a SuiteScript 2.1 script. ■ Remove the SuiteScript 2.1 syntax from the script.

The following table compares how you can use 2.0 or 2.1 syntax, set the @NApiVersion JSDoc tag, and choose the Execute SuiteScript 2.x Server Scripts As preference.

Syntax	@NApi Version	Execute SuiteScript 2.x Server Scripts As preference	Upload the script file	Create a script record	Deploy the script	Execute the script
2.0	2.0	2.0	Success	Success	Success. Script deployment is not affected by SuiteScript syntax, @NApiVersion	Success
		2.1	Success	Success		Success
	2.1	2.0	Success	Success		Success

Syntax	@NApi Version	Execute SuiteScript 2.x Server Scripts As preference	Upload the script file	Create a script record	Deploy the script	Execute the script
2.1	2.x	2.1	Success	Success	value or preference setting.	Success
		2.0	Success	Success		Success
		2.1	Success	Success		Success
	2.0	2.0	Syntax error	The script file can't be uploaded, therefore, a script record can't be created.		The script file can't be uploaded, therefore, it can't be executed.
		2.1	Syntax error	The script file can't be uploaded, therefore, a script record can't be created.		The script file can't be uploaded, therefore, it can't be executed.
		2.1	Success	Success		Unsuccessful — type of error will vary
	2.x	2.0	Syntax error	The script file can't be uploaded, therefore, a script record can't be created.		The script file can't be uploaded, therefore, it can't be executed.
		2.1	Success	Success		Success
		2.1	Syntax error	The script file can't be uploaded, therefore, a script record can't be created.		The script file can't be uploaded, therefore, it can't be executed.

You can check which version of SuiteScript is used to execute your script by looking at the **Execute As Version** field on the script record. For more information, see [Version Information on the Script Record](#).

Version Information on the Script Record

Using the SuiteScript 2.1 value for the Execute SuiteScript 2.x Server Scripts As preference overrides the script version specified using the @NApiVersion JSDoc tag within the script. Because of this, it can be useful to verify the version of SuiteScript used to execute the script. The script record includes two fields that provide information about SuiteScript version:

- **API Version** — This field indicates the version of SuiteScript that is specified in the script file. It always shows a specific version of SuiteScript. For example, if you specify a version of 2.x in the script file, the

value of this field is set to 2.0 (the value that the 2.x value maps to in this release by default). For more information about this field, see [Script Record Creation](#).

API Version
2.1

- **Execute As Version** — This field indicates the version of SuiteScript that is being used to execute the script. This field is only shown on the script record if the Execute SuiteScript 2.x Server Scripts As preference is set to 2.1, and is the only indication that the script is executed as a SuiteScript 2.1 script based on the Execute SuiteScript 2.x Server Scripts As preference. The original SuiteScript version (as indicated in the API Version field) is displayed when the script is listed in NetSuite (for example, in script lists, script deployment lists, scripted records lists, and so on).

Execute as version
2.1

SuiteScript 2.1 Language Examples

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

SuiteScript 2.1 runs on the Graal runtime engine, so it supports the ECMAScript features that Graal does (ES2019). For example, SuiteScript 2.1 includes support for constants, block-scoped variables and functions, customizable iterator behavior, typed arrays, and more. You can take advantage of these features when you write SuiteScript 2.1 scripts. Also consider converting your existing SuiteScript 2.0 scripts to SuiteScript 2.1, since some of these new features may help you improve performance or refactor your code.

The following sections include examples of using ECMAScript features supported in SuiteScript 2.1:

- [Spread Operator](#)
- [Classes](#)
- [Destructuring](#)
- [Rest Operator](#)
- [Asynchronous Server-Side Promises](#)
- [Additional ECMAScript Features](#)

For more information about new language features, see [JavaScript language resources](#).

Spread Operator

You can use the spread operator to combine values from two objects into a new one:

```

1  const createDynamicRecord = (options) => {
2      const defaultOptions = {
3          isDynamic: true
4      };
5
6      // Spread values from two Objects into a new Object, and create a new record
7      return record.create({
8          ...defaultOptions,
9          ...options
10     });
11 }
12
13 const cashSale = createDynamicRecord({

```

```

14     type: record.Type.CASH_SALE,
15     defaultValues: {
16         'subsidiary' : '1'
17     }
18 });

```

In this example, you create an arrow function called `createDynamicRecord()` that uses default parameter values (`defaultOptions`) that apply to all records you create with the function. When you call this function, you can add more parameter values, and those get merged with the default parameter values when `record.create(options)` is called.

Classes



Note: If your script contains SuiteScript 2.1 syntax that includes classes and will be included in a bundle, the `@NModuleScope` JSDoc tag must be set to `SameAccount` or `TargetAccount`.

You can create classes to represent common structures:

```

1  /**
2   * @NApiVersion 2.1
3   */
4  define(['N/currency', 'N/query'],
5  (currency, query) => {
6      // Define a class called Transaction
7      class Transaction {
8          constructor(amount, currency, date, id) {
9              this.amount = amount;
10             this.currency = currency;
11             this.date = date;
12             this.id = id;
13         }
14
15         // Create a helper function to check if the transaction exists
16         exists() {
17             return this.id && query.create({
18                 type: query.Type.TRANSACTION,
19                 condition: query.createCondition({
20                     fieldId: 'id',
21                     operator: query.Operator.IS,
22                     values: this.id
23                 })
24             }).run().results.length > 0;
25         }
26
27         // Create a helper function to convert transaction amount to a desired currency
28         toCurrency(targetCurrency) {
29             return this.amount * currency.exchangeRate({
30                 date: this.date,
31                 source: this.currency,
32                 target: targetCurrency
33             });
34         }
35     }
36
37     // Define Sale and Purchase classes that inherits helper methods from the Transaction class
38     class Sale extends Transaction {
39         constructor(amount, currency, date, customer, id) {
40             super(amount, currency, date, id)
41             this.customer = customer;
42         }
43     }
44
45     class Purchase extends Transaction {
46         constructor(amount, currency, date, vendor, id) {
47             super(amount, currency, date, id)
48             this.vendor = vendor;

```

```

49     }
50   }
51
52   // Use case: A user attempts to create a Sale or Purchase object from a search or query
53   // result set. The user can use the createSale() and createPurchase() functions
54   // to create the desired objects.
55   return {
56     createSale(amount, currency, date, customer, id) {
57       return new Sale(amount, currency, date, customer, id);
58     },
59     createPurchase(amount, currency, date, vendor, id) {
60       return new Purchase(amount, currency, date, vendor, id);
61     }
62   }
63 }
64 );

```

Destructuring

You can use destructuring to bind variables to different properties of an Object:

```

1  const name = 'data.csv';
2  const isOnline = true;
3  const fileType = file.Type.CSV;
4  const contents = "Alpha,Bravo,Charlie,Delta";
5
6  // Create a file using destructured properties
7  const dataFile = file.create({name, fileType, isOnline, contents});

```

Rest Operator

You can use destructured assignment to obtain a single property from an Object, and you can use the rest operator to create a new Object with the remaining properties:

```

1  const {fileType, ...fileProps} = file.load({
2    id: 1234
3  });
4  log.debug("fileType", fileType);
5  log.debug("isOnline?", fileProps.isOnline);

```

Asynchronous Server-Side Promises

SuiteScript 2.1 fully supports non-blocking asynchronous server-side promises expressed using the `async`, `await`, and `promise` keywords for a subset of modules: `N/http`, `N/https`, `N/query`, `N/search`, and `N/transaction`. See the help topic [Promise Object](#) for a list of promise methods supported in server scripts.

When using server-side promises in SuiteScript, you should consider the following:

- This capability is not for bulk processing use cases where an out-of-band solution, such as a work queue, may suffice.
- This capability is mainly for in-process and distinct operations, such as business functions on a list of transactions when ordering does not matter.

The following code sample shows the use of the `async` and `await` keywords:

```

1  async function() {
2    let myQuery = query.load ({
3      id: myQueryId

```

```

4   });
5   let myResult = await myQuery.run().then (function(result) {
6       // do stuff
7   });
8 }

```

For more information about JavaScript promises, see https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise.

For more information about asynchronous JavaScript programming, see <https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Asynchronous>.

For more information about the Promise object in SuiteScript, see the help topic [Promise Object](#).

Additional ECMAScript Features

You can also use the following ECMAScript features when writing SuiteScript 2.1 code:

- `Array.prototype.flat([depth=1])`
 - Recursively flattens nested arrays into a single array, up to a given depth
 - Can be used to filter out empty arrays
- `Array.prototype.flat([depth]) ...`
Flattens nested arrays returned from sub-tasks
- `Array.prototype.flatMap(callbackFn[, thisArg])`
 - Equivalent to first calling `map(callbackFn)` on an array then flattening (one level deep)
 - Superior performance
- `Object.fromEntries(iterable)`
 - Constructs an object from an iterable of key-value pairs
 - A reverse operation to `Object.entries(obj)`
 - While `Object.entries` does not emit key-value pairs for Symbol keys, `Object.fromEntries` fully supports Symbols
- `Promise.any(promises)` - fulfilled
 - Takes an iterable of Promises
 - Returns a Promise that is fulfilled as soon as the first promise in the iterable is fulfilled, with the value of that first fulfilled promise
- `Promise.any(promises)` and `AggregateError` - rejected
 - When all Promises in the iterable reject, the returned Promise is also rejected
 - `AggregateError` is a new Error sub-type; it has a property called `errors` that holds an array of inner errors
 - The inner errors may or may not be Error instances
- `Promise.allSettled(promises)`
 - Returns a Promise that resolves when all input promises are settled
 - Resolved value is an array of result objects corresponding to the input promises
 - By comparison, `Promise.all` rejects based on the first observed rejection among the input promises
- Nullish coalescing operator
 - Returns a right-hand side operand if the left-hand side operand is null or undefined (nullish); otherwise returns left-hand side

- Short-circuits
 - Makes your code shorter and easier to read
 - Proper replacement for the bug-prone use of `||` (logical OR operator)
- Optional chaining
 - Similar to `?.` operator except that it short-circuits to undefined if the left-hand side is nullish, protecting you from null reference errors
 - Makes your code shorter and easier to read
- Logical assignment operators - `&&=`, `||=`, `??=`
 - Combine condition and assignment into a single operation
 - Short-circuits
 - `a ( a = b )` becomes `a = b`
- Miscellaneous
 - `String.prototype.trimStart`
 - `String.prototype.trimEnd`
 - `BigInt`
 - `String.prototype.matchAll`
 - `String.prototype.replaceAll`
 - `WeakRef`
 - Underscore separators in numerical literals

Differences Between SuiteScript 2.0 and SuiteScript 2.1

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

SuiteScript 2.1 uses a different runtime engine than SuiteScript 2.0 and supports ECMAScript language features that are not supported in SuiteScript 2.0. Some capabilities in SuiteScript 2.0 scripts will function differently when the script is executed using SuiteScript 2.1.

Here are some key differences between SuiteScript 2.0 and 2.1:

- SuiteScript 2.1 scripts are debugged using the browser debugger instead of the SuiteScript debugger. For more information about debugging SuiteScript 2.1 scripts, see the help topic [Debugging SuiteScript 2.1 Scripts](#).
- The [N/llm Module](#) and the [N/pgp Module](#) are only supported in SuiteScript 2.1. The [N/crypto/random Module](#), when used in server scripts, is also only supported in SuiteScript 2.1.
- [Reserved Words as Identifiers](#)
- [Error Object Properties](#)
- [Invalid JSON Parsing](#)
- [Strict Mode](#)
- [Reassignment of const Variables](#)
- [Behavior of for...each...in Statement](#)
- [Formats for Converting Dates to Local Date Strings](#)

- [Conditional Catch Blocks](#)
- [The toSource Method](#)
- [Set Decimal Number with Trailing Zeros](#)
- [String Differences for RESTlet Post Method](#)
- [RESTlet Return Type Difference](#)
- [parseInt Difference](#)
- [Promises in SuiteScript 2.1 Server Scripts](#)

Reserved Words as Identifiers

In ECMAScript, reserved words are identifiers that have special meaning. For example, the reserved word `var` indicates a variable declaration. You can't use reserved words as variable names, labels, or function names in JavaScript or ECMAScript. Since SuiteScript 2.1 supports a newer ECMAScript version (ECMAScript 2023) than SuiteScript 2.0 (ECMAScript 5.1), the list of reserved words is different. SuiteScript 2.1 is stricter about not letting you use reserved words in scripts.

Scenario	SuiteScript 2.0 Behavior	SuiteScript 2.1 Behavior
You use the <code>extends</code> identifier in one of the following ways in a script: <pre> 1 myObj({extends: 'test'}); 2 function myFunction(extends) {} 3 function extends() {} 4 var extends = 1; </pre>	The script executes and does not generate an error. The <code>extends</code> identifier is not a reserved word in ECMAScript 5.1.	The script generates a syntax error. The <code>extends</code> identifier is a reserved word in ECMAScript 2023.

To avoid this issue, don't use any reserved words from any edition of ECMAScript (including those planned for future editions using ES.Next). For more information, see [SuiteScript Reserved Words](#).

Error Object Properties

SuiteScript supports the ECMAScript [Error](#) object to represent errors that can occur in scripts. You can call [JSON.stringify\(obj\)](#) on an Error object to obtain a string representation of the error. If you pass an argument to the Error object constructor, the string representation in SuiteScript 2.0 includes the argument as the value of the message property. The string representation in SuiteScript 2.1 does not include the message property.

This difference is because of how [JSON.stringify\(obj\)](#) handles enumerable and non-enumerable properties. In SuiteScript 2.0, the output of [JSON.stringify\(obj\)](#) includes the content of both enumerable and non-enumerable properties. In SuiteScript 2.1, the output of [JSON.stringify\(obj\)](#) includes only the content of enumerable properties and does not include the content of non-enumerable properties, as per the ECMAScript specification.

When you pass an argument to the Error object constructor, a non-enumerable message property is defined. If you change the value of this property after the Error object is created, the property remains non-enumerable. So, when an Error object is created with an argument, the message property is not included in the [JSON.stringify\(obj\)](#) output in SuiteScript 2.1 scripts.

By contrast, when you do not pass an argument to the Error object constructor, the message property is not defined. If you try to set the value of this property after the Error object is created, a new message property is defined. Properties that are defined in this way are always enumerable. So, when an Error

object is created without arguments, the message property is included in the `JSON.stringify(obj)` output in SuiteScript 2.1 scripts.

Scenario	SuiteScript 2.0 Behavior	SuiteScript 2.1 Behavior
Create an Error object without an argument in a script: <pre>1 var myFirstError = new Error(); 2 myFirstError.message = 'A message'; 3 4 var firstErrorJson = JSON.stringify(myFirstError);</pre>	The firstErrorJson variable contains the following: <pre>1 { 2 'message' : 'A message' 3 }</pre>	The firstErrorJson variable contains the following: <pre>1 { 2 'message' : 'A message' 3 }</pre>
Create an Error object with an argument in a script: <pre>1 var mySecondError = new Error(''); 2 mySecondError.message = 'A message'; 3 4 var secondErrorJson = JSON.stringify(mySecondError);</pre>	The secondErrorJson variable contains the following: <pre>1 { 2 'message' : 'A message' 3 }</pre>	The secondErrorJson variable contains the following: <pre>1 {}</pre>

To avoid this issue, use only enumerable properties for objects. For error messages, create Error objects with no arguments. For more information, see the help topics [JSON.stringify\(obj\)](#) and [Enumerability and ownership of properties](#).

Invalid JSON Parsing

SuiteScript can parse JSON strings that you use in your scripts. When invalid JSON is encountered, a syntax error is generated. The type and format of the generated error message is different in SuiteScript 2.1 than in SuiteScript 2.0. Also, including trailing commas in JSON strings is accepted in SuiteScript 2.0 but generates an error in SuiteScript 2.1.

Scenario	SuiteScript 2.0 Behavior	SuiteScript 2.1 Behavior
Parse the following JSON string in a script: <pre>1 var obj = 'b'; 2 JSON.parse(obj);</pre>	The following error is generated: <i>org.mozilla.javascript.EcmaError: SyntaxError: Unexpected token: b</i>	The following (or similar) error is generated: <i>SyntaxError: Invalid JSON: <json>:1:0 Expected json literal but found b</i>
Parse the following JSON string in a script: <pre>1 JSON.parse('{ "mykey" : 1, }');</pre>	The script executes successfully.	The following (or similar) error is generated: <i>SyntaxError: Invalid JSON: <json>:1:0 Expected json literal but found ,</i>

To avoid this issue, make sure you use the JSON format that is compatible with the ECMAScript specification. For more information, see the help topic [JSON object](#).

Strict Mode

ECMAScript 5 introduced strict mode. If in a SuiteScript 2.0 script you assign a value to a variable without first declaring the variable using the `var` or `const` keyword, the script executes without error. If in a SuiteScript 2.1 script you assign a value to a variable without using the `var`, `let`, or `const` keyword, an error is thrown. For more information about strict mode, see [Strict Mode](#).

Scenario	SuiteScript 2.0 Behavior	SuiteScript 2.1 Behavior
Place a portion of your script (or your entire script) into strict mode and assign a value to an undeclared variable: <pre> 1 'use strict'; 2 x = 122; 3 log.debug('x = ', x); </pre>	Script execution completes without throwing an error.	The following error is generated: <i>ReferenceError: x is not defined</i>

Reassignment of const Variables

JavaScript `const` variables create a read-only reference to a value. When a `const` variable is assigned a value, the variable identifier cannot be used again (reassigned). In SuiteScript 2.0, no error is thrown when you reassign a `const` variable. In SuiteScript 2.1, a `TypeError` is thrown.

Scenario	SuiteScript 2.0 Behavior	SuiteScript 2.1 Behavior
Set a value for a <code>const</code> variable and then try to assign it a new value: <pre> 1 const recordId = 1; 2 recordId = 2; 3 log.debug("recordId", recordId); </pre>	The script executes successfully, however the value of <code>recordId</code> is not changed by the <code>recordId = 2;</code> line. The log statement will write: <code>recordId 1</code> .	Script execution stops at the <code>recordId = 2;</code> line, and the following error is generated: <i>TypeError: Assignment to constant variable</i>

Behavior of for...each...in Statement

The `for...each...in` statement was deprecated as part of ECMA-357 (4x) and should no longer be used. In SuiteScript 2.0, the `for...each...in` statement is accepted. In SuiteScript 2.1, a `SyntaxError` is thrown if you try to use the `for...each...in` statement in your script. For more information about the deprecation of `for...each...in`, see [for..each..in](#).

Scenario	SuiteScript 2.0 Behavior	SuiteScript 2.1 Behavior
Use a <code>for...each...in</code> statement to process values in an object: <pre> 1 var obj = {fileType: "csv", encoding: "UTF-8", isOnline: "false"}; 2 for each(var value in obj) { 3 log.debug("value = " , value); 4 } </pre>	The <code>for...each...in</code> statement is accepted and the script continues to execute.	The following error is generated: <i>SyntaxError: SyntaxError: <eval>:3:4 Expected (but found each for each (var value in obj) {</i>

Formats for Converting Dates to Local Date Strings

JavaScript supports several date formats and ways to create a date. After you create a date, you can convert it to a local date string. In SuiteScript 2.0, the default format for the converted date string is the long format. In SuiteScript 2.1, the default format for the converted date string is the short format.

Scenario	SuiteScript 2.0 Behavior	SuiteScript 2.1 Behavior
Create a date and log two different formats for that date:	The default format is the long format, and additional properties passed are ignored.	The default format is the short format, and additional properties passed are ignored.

Scenario	SuiteScript 2.0 Behavior	SuiteScript 2.1 Behavior
<pre> 1 var event = new Date(Date.UTC(2012, 12, 21, 12)); 2 log.debug("date1 = ", event.toLocal DateString()); 3 log.debug("date2 = ", event.toLocal DateString('de-De', { year: 'numer ic', month: 'long', day: 'numeric' })); </pre>	The output is: ■ date1 = December 21, 2012 ■ date2 = December 21, 2012	The output is: ■ date1 = 2012-12-21 ■ date2 = 2012-12-21

Conditional Catch Blocks

In JavaScript, a try-catch statement can include an optional if statement within the catch block, however it is not ECMAScript compliant. In SuiteScript 2.0, a script that includes a conditional catch statement executes without error. In SuiteScript 2.1, a `SyntaxError` is thrown for any script that includes a conditional catch statement.

Scenario	SuiteScript 2.0 Behavior	SuiteScript 2.1 Behavior
Create a try-catch statement with an if statement in the catch block: <pre> 1 try { 2 // do something here 3 } catch (e if e instanceof TypeError) { // Don't use 4 log.debug("error = ", e); 5 } </pre>	The script completes execution without throwing an error.	The following error is generated: <pre> SyntaxError: SyntaxError: <eval>:2:11 Expected) but found if } catch (e if e instanceof TypeError) { // Don't use </pre>

The toSource Method

In JavaScript, the `toSource` method is used to return a string representing the source code of the object. However, this feature is obsolete and should not be used. In SuiteScript 2.0, a script that includes the `toSource` method for an object executes without error. In SuiteScript 2.1, an error is thrown for any script that includes the `toSource` method for an object. For more information about the `toSource` method, see [toSource](#).

Scenario	SuiteScript 2.0 Behavior	SuiteScript 2.1 Behavior
Log a Transaction source: <pre> 1 function Transaction(name) { 2 this.name = name; 3 } 4 var obj = new Transaction("myTrans"); 5 log.debug("Transaction = ", Transac tion.toSource()); 6 log.debug("Transaction Obj = ", obj.toSource()); </pre>	The script completes execution without throwing an error.	An error is thrown indicating <code>toSource</code> is not a valid function.

Set Decimal Number with Trailing Zeros

When you set a decimal number value in JavaScript, behavior depends on the format of the numerical value. In SuiteScript 2.0, if there are trailing zeros in a decimal value you set, the value is set as a double precision floating point number (double). In SuiteScript 2.1, if there are trailing zeros in a decimal value

you set, the value is set as an integer value. Note that in both SuiteScript 2.0 and SuiteScript 2.1, a value is always set as a double value if there are no trailing zeros in the decimal value you specify.

Scenario	SuiteScript 2.0 Behavior	SuiteScript 2.1 Behavior
Set a decimal value with trailing zeros: <pre>1 record.setValue('memo', 616.00)</pre>	record.setValue() saves a number as a double number: 616.0.	record.setValue() saves a number with trailing zeros as an Integer: 616.
Set a decimal value with no trailing zeros: <pre>1 record.setValue('memo', 616.01)</pre>	record.setValue() saves a number as a double number: 616.01.	record.setValue() saves a number as a double number: 616.01.

String Differences for RESTlet Post Method

In SuiteScript 2.0, a `JSON.stringify()` call is added internally to whatever is passed in the `post()` method of a `RESTlet`, which affects the value passed. In SuiteScript 2.1, a `JSON.stringify()` call is not added to the `post()` method for a `RESTlet`.

Scenario	SuiteScript 2.0 Behavior	SuiteScript 2.1 Behavior
Return <code>JSON.stringify</code> in a post function: <pre>1 // the following is included in a 2.x RESTlet 2 ... 3 function post() { 4 return JSON.stringify("flower"); 5 }</pre>	Returns <code>"\"flower\""</code> and a string length of 12. If you specify "flower" in the return statement, "flower" is returned with a string length of 8.	Returns "flower" and a string length of 8. If you specify "flower" in the return statement, flower is returned with a string length of 6.
<pre>1 let xmlRequest = new XMLHttpRequest(); 2 var url = "/app/site/hosting/restlet.nl?script=RESTLET_SCRIPT_ID&deploy=RESTLET_DEPLOYMENT_ID"; 3 4 xmlRequest.onreadystatechange = () => { 5 if (xmlRequest.readyState === 4) { 6 if (xmlRequest.status === 200) { 7 log.debug("xmlRequest.responseText=" + xmlRequest.responseText); 8 log.debug("xmlRequest.responseText.length=" + xmlRequest.responseText.length); 9 } else { 10 log.debug("xmlRequest.status=" + xmlRequest.status); 11 log.debug("xmlRequest.responseText=" + xmlRequest.responseText); 12 } 13 } 14 }; 15 xmlRequest.open("POST", url, true /* async */); 16 xmlRequest.setRequestHeader("Content-Type", "application/json"); 17 xmlRequest.send(JSON.stringify({})); // log statements appear after request is sent</pre>	The log statements are: xmlRequest.responseText="\"ok\"" xmlRequest.responseText.length=8	The log statements are: xmlRequest.responseText="ok" xmlRequest.responseText.length=4

RESTlet Return Type Difference

SuiteScript 2.1 changes the way that RESTlet responses are formatted for certain content types. When the Content-Type in the RESTlet header does not match the type that the RESTlet is returning, the results returned may not be what you are expecting.

Scenario	SuiteScript 2.0 Behavior	SuiteScript 2.1 Behavior
Set the Content-Type in a RESTlet header to application/json. For example, the following sample RESTlet script and sample code for calling the RESTlet. <pre> 1 /** 2 /** 3 * @NApiVersion 2.x 4 * @NScriptType restlet 5 */ 6 define([], function() { 7 function get() { 8 var x = {name:"jane", age:20}; 9 return JSON.stringify(x); 10 } 11 return { 12 get: get 13 } 14 }); </pre> <pre> 1 var host = window.location.host; 2 var url = 'https://' + host + '/app/site/hosting/ 3 restlet.nl?script=customscript296&deploy=customdeploy1'; 4 var xhttp = new XMLHttpRequest(); 5 xhttp.onreadystatechange = function() { 6 if (this.readyState == 4 && this.status == 200) { 7 alert(typeof xhttp.response); 8 alert(typeof JSON.parse(xhttp.response)); 9 } 10 }; 11 xhttp.open("GET", url, true); 12 xhttp.setRequestHeader('Content-Type', 'application/json'); 13 xhttp.send(); </pre>	The response is automatically converted to a string and that string is returned instead of an object. In the example scenario, the alert output is a string, SS2.0 gives string; but in SS2.1 it comes as object.	The return type for JSON will be an object. In the example scenario, the alert output is an object.

parseInt Difference

The parseInt function operates differently in SuiteScript 2.1 than in SuiteScript 2.0, in some situations. To avoid encountering incorrect results, follow best practices when using the parseInt function, which can be found at: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/parseInt.

Scenario	SuiteScript 2.0 Behavior	SuiteScript 2.1 Behavior
Parse a string value representing a number. <pre> 1 require([],function() { 2 var x = parseInt('08'); 3 log.debug('x', x); 4 }); </pre>	There is no value assigned to x.	The return value for x is 8.

Promises in SuiteScript 2.1 Server Scripts

SuiteScript 2.1 supports non-blocking asynchronous server-side promises expressed using the async, await, and promise keywords. However, these keywords are only valid for a subset of modules: N/http, N/

https, N/llm, N/query, N/search, and N/transaction. You will receive an error if you use the `async`, `await`, or `promise` keyword in a module other than the `N/http`, `N/https`, `N/llm`, `N/query`, `N/search`, or `N/transaction` modules. In other words, no other modules support the use of the `async`, `await`, and `promise` keywords in server scripts.

SuiteScript 2.x AI APIs

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

i Note: The content in this help topic pertains to SuiteScript 2.1.

SuiteScript AI APIs provide artificial intelligence (AI) capabilities directly within the NetSuite platform, letting you automate, analyze, and enhance your business processes using AI. These APIs leverage Oracle Cloud Infrastructure (OCI) services to enable features such as large language model (LLM) interaction, language translation, and document extraction. For more information about AI in NetSuite, see the help topic [NetSuite Features That Use AI](#).

The following table lists the available SuiteScript AI APIs, including the OCI service each one uses and links to full module documentation and code samples.

Module	Description	OCI Service Used	Links
N/documentCapture	The N/documentCapture module provides access to document extraction capabilities in scripts. This module lets you extract structured content and key information from a variety of document types (such as invoices, receipts, contracts, and so on) in PDF, TIFF, PNG, or JPG format. You can use this module to help automate document processing, reduce manual data entry, and enhance business workflows.	OCI Document Understanding	N/documentCapture Module N/documentCapture Module Script Samples Getting Started with the N/documentCapture Module
N/llm	The N/llm module provides access to large language model (LLM) capabilities in scripts. This module lets you generate text, summarize content, and extract information using AI-driven models directly from your SuiteScript code. You can use this module to help automate, enhance, or analyze business data using AI, while ensuring seamless integration with your NetSuite workflows and processes.	OCI Generative AI	N/llm Module N/llm Module Script Samples SuiteScript 2.x Generative AI APIs Generative AI Availability in NetSuite
N/machineTranslation	The N/machineTranslation module provides access to translation services in scripts. This module lets you translate text between supported languages, helping you work more effectively with multilingual content, localize business data, and improve user experiences for a global audience.	OCI Language	N/machineTranslation Module N/machineTranslation Module Script Samples

SuiteScript 2.x Generative AI APIs

Applies to: SuiteScript 2.x | SuiteCloud Developer | APIs

Note: The content in this help topic pertains to SuiteScript 2.1.

SuiteScript Generative AI APIs ([N/Ilm Module](#)) let you work with generative artificial intelligence (AI) in NetSuite using SuiteScript. You can use these APIs to send requests to large language models (LLMs) and to receive responses from them. NetSuite accesses LLMs through an integration with the [Oracle Cloud Infrastructure \(OCI\) Generative AI service](#).

To learn about other generative AI capabilities that NetSuite offers, see the help topic [NetSuite Features That Use AI](#).

Important: As you work with SuiteScript Generative AI APIs, keep the following considerations in mind:

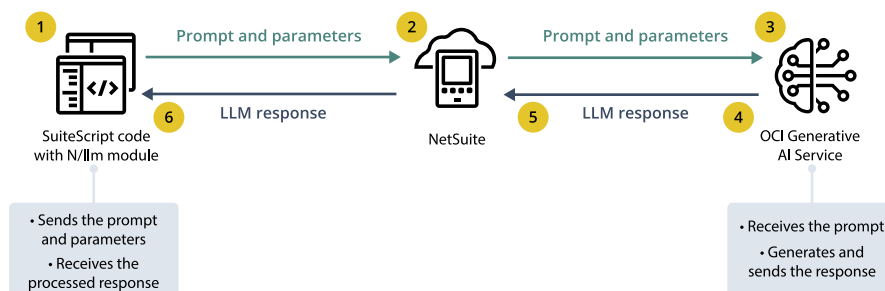
- Generative AI features, such as the N/Ilm module, use creativity in their responses. Make sure you validate the AI-generated responses for accuracy and quality. Oracle NetSuite isn't responsible or liable for the use or interpretation of AI-generated content.
- SuiteScript Generative AI APIs (N/Ilm module) are available only for accounts located in certain regions. For a list of these regions, see the help topic [Generative AI Availability in NetSuite](#).

How the SuiteScript Generative AI APIs Work

- A NetSuite developer uses the N/Ilm module in their SuiteScript code to send a request to the LLM.
- NetSuite accepts the prompt and any optional parameters included in the request and passes them to the OCI Generative AI service.
- The OCI Generative AI service processes the request using the LLM specified in your code. If you don't specify the LLM, the service uses the Cohere Command R LLM. The data never leaves Oracle, nor is it used by third parties for model training.

Note: When you use the N/Ilm module, your data may be processed globally according to the [Oracle Services Privacy Policy](#).

- The OCI Generative AI service sends the LLM response back to NetSuite.
- NetSuite processes the response and returns it to the SuiteScript code.
- The SuiteScript code uses the response per the rest of the script.



Supported LLMs

The `generateText` and `generateTextStreamed` methods have a `modelFamily` parameter that you can use to specify the LLM used, and a `modelParameters` object that lets you send supported parameters to the LLM with your prompt. For a list of supported models, see the help topic [llm.ModelFamily](#). For more information about model parameters, see the help topic [Model Parameter Values by LLM](#). If your code doesn't specify the model to use (through the `modelFamily` parameter), the Cohere Command R model is used.

For more information about chat model parameters in general, refer to [Pretrained Foundational Models in Generative AI](#) in the *Oracle Cloud Infrastructure Documentation*.

The `evaluatePrompt` and `evaluatePromptStreamed` methods take an existing prompt, along with values for any variables that the prompt uses, and send it to the LLM. You can use Prompt Studio to specify the model family that each prompt uses. For more information, see the help topic [Prompt Studio](#).

Usage Modes

There are three usage modes for using the N/llm module. The following table describes the usage modes in more detail.

Usage Mode	Description	When to Use This Mode
Free	Free, limited usage through NetSuite. When using the free limited-use mode, each successful response to NetSuite from the OCI Generative AI service counts as one use. When the capacity is used completely, the LLM module returns an error for subsequent calls until the next month.	- Testing the capability - Known, low quantity needed
On Demand	Unlimited, paid usage through your company's own Oracle Cloud account. When using the unlimited-use model through your company's own Oracle Cloud account, your company pays for on demand inferencing in the Oracle Generative AI service. To learn if this is the right choice for your company, refer to Paying for On Demand Inferencing in the <i>Oracle Cloud Infrastructure Documentation</i> . This mode requires that you complete the setup described in Using Your Own OCI Configuration for SuiteScript Generative AI APIs . You must provide your OCI credentials either in your SuiteScript code (using the <code>ociConfig</code> object) or on the Settings subtab of the AI Preferences page.	- Known medium quantity needed - Unknown quantity needed - Use cases that would fail if you ran out of capacity - You are developing a SuiteApp - You prefer pay-as-you-go service
Dedicated AI Cluster	Unlimited, paid usage through your company's own Oracle Cloud account. When using the unlimited-use model through your company's own Oracle Cloud account, your company pays for dedicated AI clusters in the Oracle Generative AI service. To learn if this is the right choice for your company, refer to Paying for Dedicated AI Clusters in the <i>Oracle Cloud Infrastructure Documentation</i> . This mode requires that you complete the setup described in Using Your Own OCI Configuration for SuiteScript Generative AI APIs . You must provide your OCI credentials either in your SuiteScript code (using the <code>ociConfig</code> object, including the <code>ociConfig.endpointId</code> value) or on the Settings subtab of the AI Preferences page.	- Known high quantity needed - Unknown quantity needed - Use cases that would fail if you ran out of capacity - You are developing a SuiteApp - Your use case requires a dedicated AI cluster

SuiteScript API Governance Limits for N/Ilm

The N/Ilm module is subject to SuiteScript API governance limits. The governance limits are included in individual method topics. For more information, see the help topics [SuiteScript Governance and Limits](#) and [SuiteScript 2.x API Governance](#).

Using Your Own OCI Configuration for SuiteScript Generative AI APIs

📘 Applies to: SuiteScript 2.x | SuiteCloud Developer | APIs

If your company needs more than the free monthly allocation of usage for SuiteScript Generative AI APIs, you can configure NetSuite to use your own Oracle Cloud account with the OCI Generative AI service. You can do this in two ways:

- The `ociConfig` parameters in N/Ilm module methods let you pass your Oracle Cloud account credentials to the OCI Generative AI service's large language model (LLM).
- The Settings subtab on the AI Preferences page lets you specify your Oracle Cloud account credentials when the Use OCI Configuration To Get Unlimited Usage box is checked. On the SuiteScript subtab, if the Use OCI Credentials for SuiteScript box is checked, these credentials are used when you call N/Ilm module methods in your scripts.

When you specify your own credentials, your scripts won't draw from the free usage pool in NetSuite for SuiteScript Generative AI APIs. Credentials that you pass using the `ociConfig` parameters in your scripts override those that you specify on the Settings subtab of the AI Preferences page.

Use these approaches when your company needs more than the free monthly allocation of usage for SuiteScript Generative AI APIs. When using the unlimited use model through your company's own Oracle Cloud account, your company pays for use of the Oracle Generative AI service. For more information, refer to the [Calculating Cost in Generative AI](#) section and the [Billing and Cost Management](#) section of the *Oracle Cloud Infrastructure Documentation*.

Configuring OCI Credentials

For instructions on configuring OCI credentials, see the help topic [Configure OCI Credentials for AI](#).

Testing OCI Configuration Parameters in Your Code

To test your OCI configuration parameters, add the `getRemainingFreeUsage` method to two places in a script that uses the `generateText` or `evaluatePrompt` methods: one time before `generateText` or `evaluatePrompt` and one time after.

If you didn't specify your OCI configuration parameters on the Settings subtab of the AI Preferences page, update your code to use the `ociConfig` parameters object.

If the configuration was correct, your script should run without errors, and you should see no change in the number returned by `getRemainingFreeUsage`.

If the configuration wasn't correct, a SuiteScript error will occur. Verify the following items:

- Do the OCI configuration parameters contain the correct values for your Oracle Cloud account?
- Were the NetSuite API Secret IDs entered correctly in your code or in the fields on the Settings subtab of the AI Preferences page? Do they match the secret IDs (and not the secret names) as shown on the API Secrets page?

- On the API Secrets page in NetSuite, were the OCI fingerprint and OCI private key secrets created correctly?
- If you aren't using the `ociConfig` parameters object, is the Use OCI Credentials for SuiteScript box checked on the SuiteScript subtab of the AI Preferences page?

Fix any items that are incorrect and run the script again. Repeat this step to resolve all configuration errors.

SuiteScript 2.x Analytic APIs

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

SuiteScript Analytic APIs help you work with analytical data in NetSuite using SuiteScript. These APIs support advanced queries and operations that let you to load, analyze, and update your data programmatically in server scripts.

The following table describes the APIs included in SuiteScript Analytic APIs, including links to relevant help topics for each API.

API	Description	Help Topics
Query API	The Query API lets you create constructed queries and run them using the N/query module. You can create conditions, join different record types, and run paged or non-paged queries.	N/query Module
SuiteQL API	The SuiteQL API lets you run queries using the SuiteQL query language. SuiteQL is based on SQL-92 and offers advanced query capabilities to access your NetSuite records and data.  Note: The SuiteQL API is for running SuiteQL queries with the N/query module. SuiteQL is available for other channels (such as SuiteAnalytics Connect), but these channels are not included in SuiteQL API.	SuiteQL SuiteQL in the N/query Module
Workbook API	The Workbook API lets you manage datasets and workbooks programmatically as part of SuiteAnalytics Workbook. You can create datasets, define conditions and joins, and set up visualizations like charts and pivots.	Workbook API N/dataset Module N/datasetLink Module N/workbook Module

Analytics Data Source and Supported Browsers

SuiteScript Analytic APIs are based on the analytics data source. This data source is a collection of NetSuite data that is grouped according to record types and fields. The structure of the analytics data source provides many capabilities and options to analyze your NetSuite data. For more information, see the help topic [Analytics Data Source Overview](#).

The analytics data source provides different information than previous data sources. For instance, Search and Report features (including SuiteScript modules such as the N/search module) use a different data source that was around before the analytics data source was introduced. The record types and fields supported in this data source may be different than those supported in the analytics data source.

You can use a browser to view a summary of the data included in a data source. A browser provides information about all record types and fields that are available for a data source. The information displayed in the browser depends on the underlying data source. For example, the SuiteScript Records Browser uses the Search and Report data source, so you can use this browser to find record type names and field names for Search and Report functionality. This browser doesn't use the analytics data source, so it's not useful for finding record type names and field names for SuiteScript Analytic APIs.

The following browsers are available for the analytics data source:

- **Records Catalog** — For more information, see the help topic [Records Catalog Overview](#).

When working with SuiteScript Analytic APIs, you can use the Records Catalog to find record type names and field names.

Workbook API

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The Workbook API lets you manage your datasets and workbooks programmatically as part of SuiteAnalytics Workbook. You can do the following:

- Create new datasets to define your data
- Create dataset conditions and joins with other record types
- Create new workbooks
- Define workbook visualizations, such as pivots and table views
- Run table views and obtain results
- Load existing datasets or workbooks and manage them

You must enable the SuiteAnalytics Workbook feature to use the Workbook API.

The Workbook API includes the following components:

- **SuiteScript modules** – The N/dataset, N/datasetLink, and N/workbook modules include all of the methods and objects you need to create and manage datasets and workbooks. The N/dataset module lets you define datasets to use for your workbooks. The N/dataset module is similar to the N/query module but is designed to work alongside the N/workbook module. Use the N/dataset module to create datasets for the Workbook API. The N/datasetLink module lets you link datasets so you can use data from both datasets in workbook visualizations. The N/workbook module lets you create workbooks, define supporting workbook components, and run pivots. For more information, see the help topics [N/dataset Module](#), [N/datasetLink Module](#), and [N/workbook Module](#).
- **Granular Workbook elements** – The N/dataset and N/workbook modules let you work with granular components that represent different areas in the SuiteAnalytics Workbook UI. For example, when creating a pivot with the N/workbook module, you can specify properties like name, axes, and sorting behavior. You can also define limiting filters (which filter results based on a limiting number) and conditional filters (which filter results based on simple or complex criteria).
- **Core plug-ins** – The new Dataset Builder Plug-in and Workbook Builder Plug-in let you create custom implementations for datasets and workbooks. You're able to provide custom implementations that meet your business needs. For more information, see the help topics [Dataset Builder Plug-in](#) and [Workbook Builder Plug-in](#).

Getting Started with the Workbook API

The following table lists tasks and links to help you get started with the Workbook API.

Task	Description	Links
Learn about the concepts in the Workbook API	Several topics are available to help you understand dataset and workbook concepts before you start using the Workbook API.	Workbook API Concepts
Follow a tutorial to create your first dataset	A tutorial is available to help you create a basic dataset using the Workbook API. This tutorial is a great starting point to explore the Workbook API's features.	Tutorial: Creating a Dataset Using the Workbook API

Task	Description	Links
Follow a tutorial to create a workbook and visualizations	A tutorial is available to help you create a workbook using the Workbook API, including a table view and pivot.	Tutorial: Creating a Workbook Using the Workbook API
Explore the objects and methods in the Workbook API	The N/dataset, N/datasetLink, and N/workbook modules have everything you need to create and manage datasets and workbooks.	N/dataset Module N/datasetLink Module N/workbook Module

Workbook API Concepts

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

This section describes the main concepts in the Workbook API and how to use them to create datasets and workbooks.

Use the following quick reference table for links to topics with more information.

Area	Concepts	Description
Datasets	Columns	Columns represent the record fields in a dataset that you want to include in the dataset query results.
	Conditions	Conditions let you filter the dataset query results based on criteria that you specify.
	Dataset Linking	You can link two datasets to use data from both datasets in a workbook.
	Joins	Datasets often join a root record type with other related record types to use data from the joined record types in a workbook.
Workbooks	Table Views Conditional Formatting Rules Table Columns	Table views let you explore your dataset query results in a simple tabular format.
	Pivots Data Dimensions Data Measures and Calculated Measures Expressions Pivot Axes Styles Selectors	Pivots help you analyze subsets of your dataset query results with advanced features.

Datasets

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Datasets are the basis for all workbook visualizations in your account. You create a query by combining fields from a root record type and any joined related types in a dataset. Your access to record types and fields depends on your account's features and your role's permissions in NetSuite.

For more information about datasets in SuiteAnalytics Workbook, see the help topic [Defining a Dataset](#).

You can create a dataset using [dataset.create\(options\)](#). This method creates a [dataset.Dataset](#) object. The type parameter is the only required one, and it sets the root record type for your dataset. For this parameter, you can load the N/query module and use values from the [query.Type](#) enum:

```
1 | var myDataset = dataset.create({
2 |     type: query.Type.CUSTOMER
3 | });
```

Alternatively, if you do not want to load the N/query module, you can use the string equivalents of the [query.Type](#) for this parameter:

```
1 | var myDataset = dataset.create({
2 |     type: 'customer'
3 | });
```

You can also provide these parameters for [dataset.create\(options\)](#):

- **columns** – The columns (fields) in the dataset. Use [dataset.createColumn\(options\)](#) to create columns. For more information, see [Columns](#).
- **condition** – A condition (criteria) to be applied to the dataset. Use [dataset.createCondition\(options\)](#) to create conditions. For more information, see [Conditions](#).
- **description** – A description for the dataset. Use this parameter only when you are creating a dataset using the Dataset Builder Plug-in. For more information, see the help topic [Dataset Builder Plug-in](#).
- **id** – A script ID for the dataset. If you do not specify one, an ID is created automatically. Use this parameter only when you are creating a dataset using the Dataset Builder Plug-in.
- **name** – A name for the dataset. Use this parameter only when you are creating a dataset using the Dataset Builder Plug-in.

```
1 | var myDataset = dataset.create({
2 |     columns: myColumns,
3 |     condition: myCondition,
4 |     type: 'customer'
5 | });
```

Columns

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

In a dataset, columns show the record fields you want to include in your query results. To use a field in a workbook visualization, it needs to be included as a column in the underlying dataset.

For more information about the types of fields you can create columns for in SuiteAnalytics Workbook, see the help topics [Formula Fields](#) and [Hierarchical Fields](#).

You can create a column using [dataset.createColumn\(options\)](#). This method creates a [dataset.Column](#) object. You can create a column using the following combinations of required parameters:

- **fieldId** – The field to use for the column.

```
1 | var myColumn = dataset.createColumn({
2 |     fieldId: 'email'
3 | });
```

- **formula and type** – A formula and the formula type to use for the column.

```

1 | var myColumn = dataset.createColumn({
2 |     formula: '{email}',
3 |     type: 'STRING'
4 | });

```

You can also provide these parameters for [dataset.createColumn\(options\)](#):

- **alias** – An alias for the column. You can use this alias to get an expression for the column to use in a workbook. Aliases can contain only letters, numbers, and underscores. For more information, see [Expressions](#).
- **join** – The joined record on which the field is present. Use [dataset.createJoin\(options\)](#) to create joins. For more information, see [Joins](#).
- **id** – A script ID for the column. You can use this ID to get an expression for the column to use in a workbook, similar to the alias parameter.
- **label** – A label for the column to display in the UI.

```

1 | var myColumn = dataset.createColumn({
2 |     alias: 'Email Column',
3 |     fieldId: 'email',
4 |     join: myJoin,
5 |     id: 'customscript_myColumn',
6 |     label: 'Email Column'
7 | });

```

Conditions

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Conditions help you filter dataset query results based on the criteria you set. Conditions can filter entire records and fields from a dataset, which affects all workbooks that are based on that dataset.

For more information about conditions in SuiteAnalytics Workbook, see the help topic [Dataset Criteria Filters](#).

You can create a condition using [dataset.createCondition\(options\)](#). This method creates a [dataset.Condition](#) object. You can create a condition using the following combinations of parameters:

- **column, operator, and values** – The column to apply the condition to, and an operator and values to use for the condition. Use [dataset.createColumn\(options\)](#) to create a column. Use values from the [query.Operator](#) enum (or their string equivalents). For more information, see [Columns](#).

```

1 | var myCondition = dataset.createCondition({
2 |     column: myColumn,
3 |     operator: query.Operator.EQUAL,
4 |     values: ['Smith']
5 | });

```

The values parameter is optional, and you should only include it when the operator requires values. For example, the `query.Operator.EMPTY` operator does not require values.

- **children and operator** – The child conditions to combine and an operator to use for the combination. You can use [dataset.createCondition\(options\)](#) to create individual conditions, then combine them using an AND or OR operation.

```

1 | var myCondition = dataset.createCondition({
2 |     children: [myFirstCondition, mySecondCondition],
3 |     operator: query.Operator.OR
4 | });

```


Dataset Linking

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

You can link two datasets that share at least one common field, like a date. Linking datasets is useful when you're working with record types that cannot be joined.

For more information about dataset linking in SuiteAnalytics Workbook, see the help topic [Linking Datasets](#).

To link datasets, use `datasetLink.create(options)`. This method creates a `datasetLink.DatasetLink` object. You'll need to provide two arrays: an array containing the datasets to link (as `dataset.Dataset` objects) and an array of expressions representing the columns to use to link the datasets. The columns you choose should have the same data type, like string or date. Create column expressions using `Dataset.getExpressionFromColumn(options)`. For more information, see [Expressions](#). You can also include an optional ID.

```
1 var myDatasetLink = datasetLink.create({
2   datasets: [myFirstDataset, mySecondDataset],
3   expressions: [[myFirstColumnExpression, mySecondColumnExpression]],
4   id: 'myDatasetLinkId'
5 });
```



Important: You can use linked datasets only in pivots, not in table views.

For more information, see the help topic [N/datasetLink Module](#).

Joins

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Joins help you add fields from other record types to your dataset. You can join multiple record types, even if they're not directly related to the main record type, using multilevel joining.

For more information about joining record types in SuiteAnalytics Workbook, see the help topic [Joining Record Types in a Dataset](#).

You can create a join using `dataset.createJoin(options)`. This method creates a `dataset.Join` object. You can create a join using the following combinations of parameters:

- `fieldId` – The field to use for the join. This type of join automatically infers the source and target record types for the join based on the record structure.

```
1 var myJoin = dataset.createJoin({
2   fieldId: 'email'
3 });
```

- `fieldId` and `join` – The field to use and an existing join. This type of join is a multilevel join. Use the `join` parameter only if a child join exists with the current join as its parent.

```
1 var myJoin = dataset.createJoin({
2   fieldId: 'phone',
3   join: myExistingJoin
4 });
```

- `fieldId` and either `source` or `target` – The field to use and the join direction. You might need to specify the source or target if the system can't automatically determine them based on the record structure.

For example, use the `target` parameter if you are joining to a record type that could define multiple inherited record types (such as entity records).

```
1 | var myTargetJoin = dataset.createJoin({
2 |   fieldId: 'phone',
3 |   target: 'entity'
4 | });
```

```
1 | var mySourceJoin = dataset.createJoin({
2 |   fieldId: 'phone',
3 |   source: 'contact'
4 | });
```

Workbooks

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Workbooks are for analyzing dataset query results using visualizations like table views and pivots. Every workbook visualization is based on a dataset, and you can use one dataset across multiple workbooks and visualizations. You can also use a different dataset for each visualization in a workbook. To create workbook visualizations, you need to use fields that are already part of the underlying dataset.

For more information about workbooks in SuiteAnalytics Workbook, see the help topic [Creating a Workbook](#).

To create a workbook, use [workbook.create\(options\)](#). This method creates a [workbook.Workbook](#) object. You can also provide these optional parameters to : [workbook.create\(options\)](#):

- `description` – A description that will show up in the SuiteAnalytics Workbook UI. Use this parameter only when you are creating a workbook using the Workbook Builder Plug-in. For more information, see the help topic [Workbook Builder Plug-in](#).
- `name` – A name for the workbook, which will appear in the SuiteAnalytics Workbook UI. Use this parameter only when you are creating a workbook using the Workbook Builder Plug-in.
- `pivots` – The pivots for the workbook. Use [workbook.createPivot\(options\)](#) to create pivots. For more information, see [Pivots](#).
- `tables` – The table views for the workbook. Use [workbook.createTable\(options\)](#) to create table views. For more information, see [Table Views](#).

```
1 | var myWorkbook = workbook.create({
2 |   description: 'A sample workbook',
3 |   name: 'My Workbook',
4 |   pivots: [myFirstPivot, mySecondPivot],
5 |   tables: [myFirstTable, mySecondTable]
6 | });
```

You don't need to specify the dataset when creating a workbook. You specify the dataset when you create workbook visualizations using methods such as [workbook.createTable\(options\)](#) and [workbook.createPivot\(options\)](#).

Table Views

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Table views let you explore your dataset query results in a simple tabular format. With table views, you can view your data without needing to customize or set up complex layouts or formulas. You can only create table views using fields that have been added to the underlying dataset.

For more information about table views in SuiteAnalytics Workbook, see the help topic [Workbook Table Views](#).

You can create a table view using [workbook.createTable\(options\)](#). This method creates a [workbook.Table](#) object. To create a table view, you'll need to provide the columns, underlying dataset, ID, and name. Use [workbook.createTableColumn\(options\)](#) to create table columns. For more information, see [Table Columns](#).

```
1 var myTable = workbook.createTable({
2   columns: [myFirstTableColumn, mySecondTableColumn, myThirdTableColumn],
3   dataset: myDataset,
4   id: 'customscript_myTable',
5   name: 'My Table View'
6 });
```

Conditional Formatting Rules

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Conditional formatting rules help you highlight important data in your workbook visualizations based on criteria you set. For instance, you can customize cell backgrounds, font sizes, styles, and more.

For more information about conditional formatting in SuiteAnalytics Workbook, see the help topic [Conditional Formatting](#).

You can create a conditional formatting rule using [workbook.createConditionalFormatRule\(options\)](#). This method creates a [workbook.ConditionalFormatRule](#) object. To create a rule, you'll need to provide these parameters:

- **filter** – A filter indicating when the conditional formatting rule should be applied. Use [workbook.createTableColumnFilter\(options\)](#) to create this filter as a [workbook.TableColumnFilter](#) object. To create a filter, combine an operator with a set of values. Use operators (or their string equivalents) from the [query.Operator](#) enum. Values are specified based on the column the filter is used for and the operator used.

```
1 var myFilter = workbook.createTableColumnFilter({
2   operator: query.Operator.ANY_OF,
3   values: [myArrayOfValues]
4 });
```

- **style** – The style to apply when the filter evaluates to true. Create a style using [workbook.createStyle\(options\)](#) which returns a [workbook.Style](#) object. When you create a style, you can use supporting methods such as [workbook.createColor\(options\)](#) and [workbook.createFontSize\(options\)](#) to define the different attributes of a style. For more information, see [Styles](#).

```
1 var myStyle = workbook.createStyle({
2   backgroundColor: workbook.createColor({
3     red: 255,
4     green: 255,
5     blue: 0
6   });
7 });
```

Use both parameters to create a conditional formatting rule:

```
1 var myRule = workbook.createConditionalFormatRule({
2   filter: myFilter,
3   style: myStyle
4 });
```

Once you have your rules, use [workbook.createConditionalFormat\(options\)](#) to combine them into a single [workbook.ConditionalFormat](#) object:

```
1 var myConditionalFormat = workbook.createConditionalFormat({
2   rules: [myFirstRule, mySecondRule]
3 });
```

To apply the rules to a column, pass a [workbook.ConditionalFormat](#) object to [workbook.createTableColumn\(options\)](#). For more information, see [Table Columns](#).

```
1 var myTableColumn = workbook.createTableColumn({
2   datasetColumnAlias: 'MyColumn',
3   conditionalFormats: [myFirstConditionalFormat, mySecondConditionalFormat]
4 });
```

Table Columns

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Table columns are based on dataset columns and appear in your table view. You can customize table columns with formatting rules, conditions, and sorting to get the look you want.

For more information about table columns in SuiteAnalytics Workbook, see the help topic [Workbook Table Views](#).

You can create a table column using [workbook.createTableColumn\(options\)](#). This method creates a [workbook.TableColumn](#) object. To create a table column, you'll need to provide the `datasetColumnAlias` and `conditionalFormats` parameters.

- The `datasetColumnAlias` parameter is the alias of a column in the underlying dataset. To ensure this value is available, specify an alias when you create each dataset column using [dataset.createColumn\(options\)](#).
- The `conditionalFormats` parameter applies formatting rules to the column. For more information, see [Conditional Formatting Rules](#).

```
1 var myTableColumn = workbook.createTableColumn({
2   datasetColumnAlias: 'MyColumn',
3   conditionalFormats: [myFirstConditionalFormat, mySecondConditionalFormat]
4 });
```

You can also provide these optional parameters for [workbook.createTableColumn\(options\)](#):

- `alias` – The alias for the table column.
- `condition` – Additional conditions for the table column.
- `label` – A label for the table column, which appears in the SuiteAnalytics Workbook UI.
- `sort` – The sorting behavior for the table column. Use [workbook.createSort\(options\)](#) to create sorts.
- `width` – The width of the table column in the SuiteAnalytics Workbook UI (in pixels).

```
1 var myTableColumn = workbook.createTableColumn({
2   alias: 'MyWorkbookTableColumn',
3   condition: myExtraCondition,
4   conditionalFormats: [myFirstConditionalFormat, mySecondConditionalFormat],
5   datasetColumnAlias: 'MyColumn',
6   label: 'My Workbook Table Column',
7   sort: mySort,
```

```

8 |     width: 100
9 | });

```

Pivots

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Pivots help you analyze specific parts of your dataset using features like dimensions, measures, filters, and report styles. To create a pivot, you need to use fields that are already part of the underlying dataset.

For more information about pivots in SuiteAnalytics Workbook, see the help topic [Workbook Pivot Tables](#).

You can create a pivot using [workbook.createPivot\(options\)](#). This method creates a [workbook.Pivot](#) object. To create a pivot, you'll need to provide these required parameters:

- **columnAxis** – The column axis for the pivot. Use [workbook.createPivotAxis\(options\)](#) to create axes. For more information, see [Pivot Axes](#).
- **dataset or datasetLink** – The underlying dataset or linked dataset for the pivot. You can provide either a dataset (using the dataset parameter) or a linked dataset (using the datasetLink parameter), but you can't use both simultaneously. For more information, see [Datasets](#) and [Dataset Linking](#).
- **id** – A script ID for the pivot. You can provide this ID to [Workbook.runPivot\(options\)](#) to obtain the results of the pivot.
- **name** – A name for the pivot.

```

1 | var myPivot = workbook.createPivot({
2 |     columnAxis: myColumnAxis,
3 |     dataset: myDataset,
4 |     id: 'MyPivot',
5 |     name: 'My Pivot',
6 |     rowAxis: myRowAxis
7 | });

```

You can also provide these optional parameters for [workbook.createPivot\(options\)](#):

- **aggregationFilters** – A set of conditional filters or limiting filters that are applied to the pivot. Use [workbook.createConditionalFilter\(options\)](#) to create conditional filters, and use [workbook.createLimitingFilter\(options\)](#) to create limiting filters. For more information about filters in SuiteAnalytics Workbook, see the help topic [Workbook Visualization Filters](#).
- **filterExpressions** – A set of simple, non-aggregated value-based filters for the pivot. You provide these filters as a set of [workbook.Expression](#) objects. Use [workbook.createExpression\(options\)](#) to create these expressions. For more information, see [Expressions](#).
- **portletName** – A name for the pivot when it is displayed as a portlet in NetSuite.
- **reportStyles** – A set of report styles for the pivot. Use [workbook.createReportStyle\(options\)](#) to create report styles. For more information, see [Styles](#).

```

1 | var myPivot = workbook.createPivot({
2 |     aggregationFilters: myAggregationFilters,
3 |     columnAxis: myColumnAxis,
4 |     dataset: myDataset,
5 |     filterExpressions: myFilterExpressions,
6 |     id: 'MyPivot',
7 |     name: 'My Pivot',
8 |     portletName: 'My Pivot as a Portlet',
9 |     reportStyles: [myFirstReportStyle, mySecondReportStyle],
10 |    rowAxis: myRowAxis
11 | });

```

Data Dimensions

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Data dimensions help you analyze and format pivot data from specific dataset columns. You can create data dimensions to see how one column's data relates to another column's data. For instance, you can create data dimensions for transaction type and posting period to see how many transactions occur per period.

For more information about data dimensions in SuiteAnalytics Workbook, see the help topic [Workbook Pivot Tables](#).

Start by creating data dimension items using [workbook.createDataDimensionItem\(options\)](#). This creates a [workbook.DataDimensionItem](#) object, where each item represents a dataset column you want to analyze. To create a data dimension item, you'll need to provide these parameters:

- **expression** – The expression for the data dimension item. Use [Dataset.getExpressionFromColumn\(options\)](#) to get an expression that represents the column in the underlying dataset. For more information, see [Expressions](#).
- **label** – A label for the data dimension, which appears in the SuiteAnalytics Workbook UI.

```
1 var myItem = workbook.createDataDimensionItem({
2   expression: myDataset.getExpressionFromColumn({
3     alias: 'MyColumn'
4   }),
5   label: 'My Item'
6 });
```

Once you have your data dimension items, use [workbook.createDataDimension\(options\)](#) to combine them into a single [workbook.DataDimension](#) object. You can use the optional **children** parameter to specify child data dimensions or measures, and you can use the optional **totalLine** parameter to define how the total line appears using values from the [workbook.TotalLine](#) enum.

```
1 var myDataDimension = workbook.createDataDimension({
2   children: [myFirstDDChild, mySecondDDChild],
3   items: [myFirstDDItem, mySecondDDItem],
4   totalLine: workbook.TotalLine.HIDDEN
5 });
```

You can provide an array of [workbook.DataDimension](#) objects when you create pivot axes using [workbook.createPivotAxis\(options\)](#). For more information, see [Pivot Axes](#).

```
1 var myColumnAxis = workbook.createPivotAxis({
2   root: workbook.createSection({
3     children: [myDataDimension]
4   }),
5   sortDefinitions: [mySortDefinition];
6 });
```

Data Measures and Calculated Measures

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Data measures help you aggregate data from a dataset column. For instance, you can create a data measure to count unique values in a column. Calculated measures allow you to do basic math using column values, other measures, or expressions. For example, you can create a calculated measure to find the difference between two other measures.

For more information about measures in SuiteAnalytics Workbook, see the help topic [Calculated Measures](#).

You can create a data measure using [workbook.createDataMeasure\(options\)](#). This method creates a [workbook.DataMeasure](#) object. To create a data measure, you'll need to provide these parameters:

- aggregation – The type of aggregation to use for the data measure. Use values from the [workbook.Aggregation](#) enum.
- expression or expressions – The expression (or expressions) for the data measure. Use expression for single-expression measures, and use expressions for multiple-expression measures. Use [Dataset.getExpressionFromColumn\(options\)](#) to get an expression that represents a column in the underlying dataset, or use [workbook.createExpression\(options\)](#) to create a custom expression. For more information, see [Expressions](#).

You can also add a `label` parameter to your data measure.

```
1 var myDataMeasure = workbook.createDataMeasure({
2   aggregation: workbook.Aggregation.SUM,
3   expressions: [dataset.getExpressionFromColumn({
4     alias: 'MyColumn'
5   })],
6   label: 'My Sum'
7 });
```

You can create a calculated measure using [workbook.createCalculatedMeasure\(options\)](#). This method creates a [workbook.CalculatedMeasure](#) object. To create a calculated measure, you only need to provide the `options.expression` parameter, which represents the expression for the calculated measure. Create the expression using [workbook.createExpression\(options\)](#) and use functions from the [workbook.ExpressionType](#) enum.



Important: For the top-level expression that you use in a calculated measure, only the PLUS, MINUS, DIVIDE, and MULTIPLY operands are valid. However, you can use other operands in expressions for each parameter in the top-level calculated measure.

You can also add a `label` parameter to your calculated measure.

```
1 var myCalculatedMeasure = workbook.createCalculatedMeasure({
2   expression: workbook.createExpression({
3     functionId: workbook.ExpressionType.MINUS,
4     parameters: {
5       operand1: myFirstOperandExpression,
6       operand2: mySecondOperandExpression
7     }
8   }),
9   label: 'My Calculated Measure'
10 });
```

Expressions

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Expressions are used to query or perform math operations on dataset data or calculate custom values. You can create expressions based on dataset columns or use custom functions for comparisons and other operations.

You can create a column-based expression using [Dataset.getExpressionFromColumn\(options\)](#). This method creates a [workbook.Expression](#) object. To create a column expression, provide either the `alias`

parameter (to specify a dataset column using its alias) or the `columnId` parameter (to specify a dataset column using its ID), but you can't provide both.

```
1 | var myColumnExpression = myDataset.getExpressionFromColumn({
2 |   alias: 'MyColumn'
3 | });
```

When you have a column expression, you can use it in various other methods.

- [workbook.createCalculatedMeasure\(options\)](#)
- [workbook.createDataDimensionItem\(options\)](#)
- [workbook.createDataMeasure\(options\)](#)
- [workbook.createReportStyleRule\(options\)](#)

You can create a custom expression using [workbook.createExpression\(options\)](#). This method creates a [workbook.Expression](#) object. To create a custom expression, you need to provide the `functionId` parameter, which specifies the operation. Use values from the [workbook.ExpressionType](#) enum for this parameter. The `parameters` parameter is optional and represents the parameters or operands for the function. Some functions don't need extra parameters or operands.

```
1 | var myCustomExpression = workbook.createExpression({
2 |   functionId: workbook.ExpressionType.CONSolidATE,
3 |   parameters: {
4 |     expression: myDataset.getExpressionFromColumn({
5 |       alias: 'MyColumn'
6 |     })
7 |   }
8 | });
```

For descriptions of each function you can use in an expression, see the help topic [workbook.ExpressionType](#).

Pivot Axes

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Pivot axes are the X and Y axes in a pivot, where you combine elements before creating the pivot. Create pivot axes with the data dimensions and measures you want in your pivot. You can also add sections (as [workbook.Section](#) objects) that wrap sets of data dimensions or measures. Pivot axes can have a sort definition to customize how elements are sorted.

For more information about pivots in SuiteAnalytics Workbook, see the help topic [Workbook Pivot Tables](#).

You can create a pivot axis using [workbook.createPivotAxis\(options\)](#). This method creates a [workbook.PivotAxis](#) object. To create a pivot axis, you need to provide the `root` parameter with the data dimensions and measures. You have two options for providing the `root` parameter value:

- For a single data dimension, use a [workbook.DataDimension](#) object directly:

```
1 | var myPivotAxis = workbook.createPivotAxis({
2 |   root: myDataDimension
3 | });
```

- For multiple data dimensions or measures, create a section using [workbook.createSection\(options\)](#) and add the data dimensions and measures to the section. You can use the `children` parameter of [workbook.createSection\(options\)](#) to create a hierarchy of data dimensions or measures to display on the axis.


```

1  var myPivotAxis = workbook.createPivotAxis({
2    root: workbook.createSection({
3      children: [
4        workbook.createDataDimension({
5          items: [
6            workbook.createDataDimensionItem({
7              label: 'My Child DD Item',
8              expression: workbook.createExpression({
9                functionId: 'DATE_TIME_PROPERTY',
10               parameters: {
11                 dateTime: myDataset.getExpressionFromColumn({
12                   alias: 'Date'
13                 }),
14                 property: 'YEAR',
15                 hierarchy: 'MONTH_BASED'
16               }
17             })
18           ]
19         })
20       ],
21       children: [myDataSubDimension]
22     }),
23     myCountMeasure,
24     myCalculatedMeasure
25   ]
26 });

```

You can also add a sort definition using the `sortDefinitions` parameter of [workbook.createPivotAxis\(options\)](#). The sort definition can only reference data dimension items added to the pivot axis by the `root` parameter.

Create a sort definition using [workbook.createSortDefinition\(options\)](#). You can use [workbook.createSortByDataDimensionItem\(options\)](#) to create a sort that applies to a specific data dimension, then add that sort to the sort definition using the `sortBy` parameter. You'll also need a selector for the data dimension, which you can create using [workbook.createPathSelector\(options\)](#).

```

1  var allSubNodesSelector = workbook.D descendantOrSelfNodesSelector;
2
3  var myPivotAxis = workbook.createPivotAxis({
4    root: workbook.createSection({
5      children: [myDataDimension],
6      totalLine: workbook.TotalLine.FIRST_LINE
7    }),
8    sortDefinitions: [
9      workbook.createSortDefinition({
10       sortBy: [
11         workbook.createSortByDataDimensionItem({
12           item: myDataDimensionItem,
13           sort: workbook.createSort({
14             ascending: true
15           })
16         })
17       ],
18       selector: workbook.createPathSelector({
19         elements: [
20           allSubNodesSelector,
21           workbook.createDimensionSelector({
22             dimension: myDataDimension
23           })
24         ]
25       })
26     })
27   ]
28 });
29

```

*// Note that this is the same data dimension
// used in the root parameter of createPivotAxis()*

You can also sort by measures using [workbook.createSortByMeasure\(options\)](#).

Selectors

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Selectors help you choose cells in a pivot to apply styles and sorting. Since pivots are hierarchical, selectors find the right cells based on that structure.

There are several types of selectors you can create:

- [workbook.ChildNodesSelector](#) – A special selector for child nodes. You don't need a method to create this selector.
- [workbook.DescendantOrSelfNodesSelector](#) – A generic selector for many different elements. You do not need to use a method to create this type of selector. Use this selector when creating more specific ones, like a path selector.

```
1 var myAllSubNodesSelector = workbook.DescendantOrSelfNodesSelector;
2
3 var myPathSelector = workbook.createPathSelector({
4   elements: [
5     myAllSubNodesSelector,
6     workbook.createDimensionSelector({
7       dimension: myDataDimension
8     })
9   ]
10 });
```

- [workbook.DimensionSelector](#) – A selector for dimensions, created using [workbook.createDimensionSelector\(options\)](#).

```
1 var myDimensionSelector = workbook.createDimensionSelector({
2   dimension: myDataDimension
3 });
```

- [workbook.MeasureSelector](#) – A selector for measures, created using [workbook.createMeasureSelector\(options\)](#).

```
1 var myMeasureSelector = workbook.createMeasureSelector({
2   measures: [myMeasure]
3 });
```

- [workbook.MeasureValueSelector](#) – A selector for measure values, created using [workbook.createMeasureValueSelector\(options\)](#). To create this selector, you need to provide column, row, and measure selectors.

```
1 var myRowSelector = workbook.DescendantOrSelfNodesSelector;
2 var myColumnSelector = workbook.DescendantOrSelfNodesSelector;
3 var myMeasureSelector = workbook.createMeasureSelector({
4   measures: [myMeasure]
5 });
6
7 var myMeasureValueSelector = workbook.createMeasureValueSelector({
8   rowSelector: myRowSelector,
9   columnSelector: myColumnSelector,
10  measureSelector: myMeasureSelector
11 });
```

- [workbook.PathSelector](#) – A selector for paths, created using [workbook.createPathSelector\(options\)](#).

```
1 var myAllSubNodesSelector = workbook.DescendantOrSelfNodesSelector;
2
3 var myPathSelector = workbook.createPathSelector({
4   elements: [
5     myAllSubNodesSelector,
```

```

6      workbook.createDimensionSelector({
7          dimension: myDimension
8      })
9  ]
10 });

```

Styles

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Styles help you customize cell formats in table views and pivots based on conditions. You can create standalone styles to customize cell attributes, and then add them to formatting rules.

For more information about styles in SuiteAnalytics Workbook, see the help topic [Conditional Formatting](#).

You can create a standalone style using [workbook.createStyle\(options\)](#). This method creates a [workbook.Style](#) object. You can also provide these optional parameters to [workbook.createStyle\(options\)](#):

- **backgroundColor** – The background color. Use values from the [workbook.Color](#) enum, or use [workbook.createColor\(options\)](#) to create a custom color.
- **backgroundImage** – The background image. Use values from the [workbook.Image](#) enum.
- **backgroundPosition** – The background position. Use values from the [workbook.Position](#) enum, or use [workbook.createPositionPercent\(options\)](#), [workbook.createPositionUnits\(options\)](#), or [workbook.createPositionValues\(options\)](#) to create a custom position.
- **color** – The font color. Use values from the [workbook.Color](#) enum, or use [workbook.createColor\(options\)](#) to create a custom color.
- **fontSize** – The font size. Use values from the [workbook.FontSize](#) enum.
- **fontStyle** – The font style. Use values from the [workbook.FontStyle](#) enum.
- **fontWeight** – The font weight. Use values from the [workbook.FontWeight](#) enum.
- **textAlign** – The text alignment. Use values from the [workbook.TextAlign](#) enum.
- **textDecorationColor** – The text decoration color. Use values from the [workbook.Color](#) enum, or use [workbook.createColor\(options\)](#) to create a custom color.
- **textDecorationLine** – The text decoration line. Use values from the [workbook.TextDecorationLine](#) enum.
- **textDecorationStyle** – The text decoration style. Use values from the [workbook.TextDecorationStyle](#) enum.

```

1  var myFirstStyle = workbook.createStyle({
2      backgroundColor: workbook.createColor({
3          red: 255,
4          green: 192,
5          blue: 203
6      })
7  });
8
9  var mySecondStyle = workbook.createStyle({
10     fontSize: workbook.FontSize.LARGE,
11     fontStyle: workbook.FontStyle.ITALIC,
12     fontWeight: workbook.FontWeight.BOLD
13 });

```

To create a report style, start by creating report style rules using [workbook.createReportStyleRule\(options\)](#), similar to creating conditional formatting rules for a conditional format. This creates a [workbook.ReportStyleRule](#) object. You'll need to provide these parameters:

- **expression** – An expression indicating when to apply the style. Create expressions using [workbook.createExpression\(options\)](#). For more information, see [Expressions](#).

- **style** – The style to apply when the expression evaluates to true. Create styles using [workbook.createStyle\(options\)](#).

```

1 var myReportStyleRule = workbook.createReportStyleRule({
2   expression: workbook.createExpression({
3     functionId: workbook.ExpressionType.COMPARE,
4     parameters: {
5       comparisonType: 'GREATER_OR_EQUAL',
6       operand1: workbook.createExpression({
7         functionId: workbook.ExpressionType.MEASURE_VALUE,
8         parameters: {
9           measure: myCalculatedMeasure
10        }
11      }),
12      operand2: workbook.createConstant(
13        workbook.createCurrency({
14          amount: 1,
15          id: 'USD'
16        })
17      )
18    }
19  }),
20  style: workbook.createStyle({
21    backgroundColor: workbook.createColor({
22      red: 255,
23      green: 192,
24      blue: 203
25    })
26  })
27 });

```

When you have your report style rules, use [workbook.createReportStyle\(options\)](#) combine them into a single [workbook.ReportStyle](#) object. You'll also need to provide a selector to choose the cell in the pivot where the style will be applied. For more information, see [Selectors](#).

```

1 var conditionalRowSelector = workbook.D descendantOrSelfNodesSelector;
2 var conditionalColumnSelector = workbook.D descendantOrSelfNodesSelector;
3 var conditionalMeasureSelector = workbook.createMeasureSelector({
4   measures: [myCalculatedMeasure]
5 });
6
7 var measureValueSelector = workbook.createMeasureValueSelector({
8   rowSelector: conditionalRowSelector,
9   columnSelector: conditionalColumnSelector,
10  measureSelector: conditionalMeasureSelector
11 });
12
13 var myReportStyle = workbook.createReportStyle({
14   selectors: [measureValueSelector],
15   rules: [myFirstReportStyleRule, mySecondReportStyleRule]
16 });

```

Tutorial: Creating a Dataset Using the Workbook API

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

In this tutorial, you'll learn how to:

- Load the N/dataset module
- Create an initial dataset
- Create dataset components (joins, columns, and conditions)
- Add dataset components to the initial dataset
- Save and run the dataset

You can write your script in a text editor of your choice. When the script is complete, you will use the SuiteScript Debugger to run the script and create the dataset. Finally, you will use the SuiteAnalytics Workbook UI to verify that the dataset was created successfully.

Navigating the Tutorial

This tutorial includes navigation links at the bottom of each page. You can use these links to go forward to the next step in the tutorial or go back to the previous step.

Use the following table to go to a specific step in the tutorial.

Step	Description
Prerequisites	This topic lists the prerequisites for this tutorial, including required NetSuite features and permissions.
Create an initial dataset	This step describes how to create an initial dataset with no components.
Create joins with other record types	This step describes how to create joins with other record types so you can include more fields in the dataset.
Create columns	This step describes how to create columns, which represent the fields to include in the dataset.
Create conditions	This step describes how to create conditions to filter the dataset.
Add components to the initial dataset	This step describes how to add the joins, conditions, and columns to the dataset.
Save and run the dataset	This step describes how to save the dataset and run it to obtain the results.

Related Help Topics

The following help topics provide more information about the concepts discussed in this tutorial:

- [Defining a Dataset](#)
- [Available Record Types](#)
- [Guidelines for Joining Record Types in SuiteAnalytics Workbook](#)
- [SuiteAnalytics Workbook Tutorial](#)
- [Navigating SuiteAnalytics Workbook](#)
- [SuiteScript Debugger](#)
- [N/dataset Module](#)

Next: [Prerequisites](#)

Prerequisites

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Before you start this tutorial, review the following prerequisites. If you are not sure whether your NetSuite account or role meet these prerequisites, contact your administrator.

Required Features

The following features must be enabled in your NetSuite account:

- **SuiteAnalytics Workbook** – This feature lets you access SuiteAnalytics Workbook. It is required both to access the SuiteAnalytics Workbook UI and to use the N/dataset, N/datasetLink, and N/workbook modules in a script.
- **Server SuiteScript** or **Client SuiteScript** – These features let you create and run server or client scripts in SuiteScript.

For more information about enabling features, see the help topic [Enabling Features](#).

Required Roles or Permissions

Your role and role permissions must let you do the following:

- Access the Analytics tab in the NetSuite UI
- View saved datasets and workbooks
- Create new datasets and workbooks
- Run server scripts using the SuiteScript Debugger

For more information about roles and permissions, see the help topic [NetSuite Users & Roles](#).

Previous: [Tutorial: Creating a Dataset Using the Workbook API](#)

Next: [Create an initial dataset](#)

Create an initial dataset

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Datasets are the basis for all workbooks in your account. In a dataset, you combine the fields of a root record type and any joined related record types to create a query. You can also create custom formula fields to calculate values that are not available through standard record type fields.

In this step, you will create an initial dataset object that uses the customer record type as the root record type.

1. Create a new file in a text editor of your choice. If your text editor is able to provide syntax highlighting based on file type, select JavaScript (.js) file highlighting.
2. Add the following code to load the N/dataset module:

```
1 | require(['N/dataset'], function(dataset) {
2 |
3 | });
```

In this tutorial, you load only the N/dataset module. However, when you work with datasets and workbooks in your production account, you should always load both the N/dataset and N/workbook modules in your scripts. This approach ensures that you can access all workbook-related objects and methods.

3. Create a dataset using [dataset.create\(options\)](#). Use the type parameter to specify the root record type for the dataset. Add the following code inside the require function definition:

```
1 | var tutorialDataset = dataset.create({
2 |   type: 'customer'
3 | });
```

This method returns a [dataset.Dataset](#) object. The string customer is the name of the customer record type. When you create a dataset, you must specify the name of the root record type to use. To obtain this name (as well as other record and field names), you can use the Records Catalog. For more information, see the help topic [Records Catalog Overview](#).

At this point, your script file should look similar to the following:

```

1 require(['N/dataset'], function(dataset) {
2   var tutorialDataset = dataset.create({
3     type: 'customer'
4   });
5 });

```

SuiteAnalytics Workbook UI

In the SuiteAnalytics Workbook UI, the root record type for a dataset appears in the info popup beside the dataset name.

The screenshot shows the Oracle NetSuite SuiteAnalytics Workbook interface. The top navigation bar includes 'ORACLE NETSUITE', a search bar, and various menu items like 'Activities', 'Transactions', 'Lists', 'Reports', 'Analytics', 'Documents', 'Setup', 'Customization', and 'Support'. The main content area displays 'Dataset | Cheps' Test Dataset' with an 'Employee' tag. A left sidebar lists various fields for the dataset, including 'Transaction Line', 'Billing Address', 'Created By Customer', 'Created By Employee', 'Created By Entity', 'Created By Partner', 'Created By Vendor', 'Entity', 'Entity Customer', 'Entity Employee', 'Entity Job', 'Entity Other Name', 'Entity Partner', 'Entity Vendor', 'Incoterm', 'Last Modified By Customer', 'Last Modified By Employee', 'Last Modified By Entity', 'Last Modified By Partner', 'Last Modified By Vendor', and 'Linked Tracking Numbers'. A right sidebar shows the 'Dataset Name' (Cheps' Test Dataset), 'Dataset Description' (This is Cheps' test dataset.), 'Root Record' (Transaction), 'Dataset ID' (custdataset_cheps), and 'App ID'. Below this, a table displays data for the dataset, with columns for DATE, SALES REP, ENTITY, and ENTITY (CUSTOMER): EMAIL. The table shows four rows of data, including transactions from 1/15/2002 to 2/13/2002. A footer note indicates 'Showing 4 rows out of 4 as a statistical sample overview of the entire dataset.'

Previous: [Prerequisites](#)

Next: [Create joins with other record types](#)

Create joins with other record types

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

SuiteAnalytics Workbook enables you to add fields from multiple record types to a single dataset. This includes record types that are more than one join away from the root record type of a dataset, enabling you to compile workbook source data from a diverse set of record types.

In this step, you will create objects to represent joins with other record types from the root record type.

1. In your script file, create a join using `dataset.createJoin(options)`. This join connects the root record type (customer) with the employee record type using the salesrep field. The salesrep field is located on the customer record. Add the following code after you define the dataset name and description in [Create an initial dataset](#):

```

1 var salesRepJoin = dataset.createJoin({
2   fieldId: 'salesrep'
3 });

```

The `dataset.createJoin(options)` method returns a `dataset.Join` object.

2. Create a second join to connect the root record type (customer) with the transaction record type using the entity field. The entity field on the transaction record type refers to the customer associated with that transaction. However, there is no join field on the customer record type that refers to the transactions for that customer.

To represent this relationship as a join object, you must specify the source parameter as well as the `fieldId` parameter. The source parameter refers to the record type to join from (transaction), and the `fieldId` parameter refers to the field on that record type to use for the join (entity). Add the following code after the first `dataset.createJoin(options)` call:

```
1 var transactionJoin = dataset.createJoin({
2   fieldId: 'entity',
3   source: 'transaction'
4 });
```

At this point, your script file should look similar to the following:

```
1 require(['N/dataset'], function(dataset) {
2   var tutorialDataset = dataset.create({
3     type: 'customer'
4   });
5
6   var salesRepJoin = dataset.createJoin({
7     fieldId: 'salesrep'
8   });
9   var transactionJoin = dataset.createJoin({
10    fieldId: 'entity',
11    source: 'transaction'
12  });
13 });
```

SuiteAnalytics Workbook UI

In the SuiteAnalytics Workbook UI, joined fields in a dataset are listed when you click a joined record type in the Records list. They are highlighted and appear at the top of the Fields list.

The screenshot shows the SuiteAnalytics Workbook UI for a dataset named 'Cheps' Test Dataset'. The interface includes a search bar, a list of fields, and a data preview section. The 'Contacts' field is highlighted in the Fields list, and its details are shown in the Criteria summary and Data Preview sections.

Fields List:

Field Name	Type
Entity ID	T
Alt. Email	T
Assist. Phone	T
Assistant	T
Category	T
Comments	T
Company	T
Contact is Inactive	T
Date Created	T
Default Billing Address	T
Default Shipping Address	T
Email	T
External ID	T
Fax	T
First Name	T
Home Phone	T
Internal ID	#
Job Title	T
Last Modified Date	T
Last Name	T
Lead Source	T

Criteria summary: Status AND Entity (Customer): Email

Data Preview:

TASTIC DATE	ENTITY	ENTITY (CUSTOMER): EMAIL	CONTACTS: ENTITY ID
002	Eric Schmidt	eschmidt@freeversion.com	
02	Amy Boughton	boughton751@freeversion.com	
002	Tony Matsuda	tonymat123@freeversion.com	
002	Phillip Van Hook	vanhook@freecheckbook.net	

Showing 4 rows out of 4 as a statistical sample overview of the entire dataset.

Previous: [Create an initial dataset](#)

Next: [Create columns](#)

Create columns

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Columns represent the record fields that are included in the dataset. Column definitions are used to create conditions that apply to the corresponding fields in the dataset.

In this step, you will create objects to represent columns for the dataset.

1. In your script file, create a column using `dataset.createColumn(options)`. This column represents the `entityid` field on the root customer record type. Add the following code after the two `dataset.createJoin(options)` calls in [Create joins with other record types](#):

```
1 | var entityIdColumn = dataset.createColumn({
2 |     fieldId: 'entityid'
3 | });
```

The `dataset.createColumn(options)` method returns a `dataset.Column` object.

2. Create a second column to represent the `email` field on the joined employee record type. Add the following code after the first `dataset.createColumn(options)` call:

```
1 | var emailColumn = dataset.createColumn({
2 |     fieldId: 'entityid',
3 |     join: salesRepJoin
4 | });
```

3. Create a third column to represent the `trandate` field (transaction date) on the joined transaction record type. Add the following code after the second `dataset.createColumn(options)` call:

```
1 | var trandateColumn = dataset.createColumn({
2 |     fieldId: 'trandate',
3 |     join: transactionJoin
4 | });
```

At this point, your script file should look similar to the following:

```
1 | require(['N/dataset'], function(dataset) {
2 |     var tutorialDataset = dataset.create({
3 |         type: 'customer'
4 |     });
5 |
6 |     var salesRepJoin = dataset.createJoin({
7 |         fieldId: 'salesrep'
8 |     });
9 |     var transactionJoin = dataset.createJoin({
10 |         fieldId: 'entity',
11 |         source: 'transaction'
12 |     });
13 |
14 |     var entityIdColumn = dataset.createColumn({
15 |         fieldId: 'entityid'
16 |     });
17 |     var emailColumn = dataset.createColumn({
18 |         fieldId: 'entityid',
19 |         join: salesRepJoin
20 |     });
21 |     var trandateColumn = dataset.createColumn({
22 |         fieldId: 'trandate',
23 |         join: transactionJoin
```

```

24 | });
25 | });

```

SuiteAnalytics Workbook UI

In the SuiteAnalytics Workbook UI, each included column (field) is highlighted at the top of the Fields list. You can also see each field as a column heading in the Data Grid.

The screenshot displays the SuiteAnalytics Workbook UI for a dataset named 'Cheps' Test Dataset'. The interface includes a 'Criteria summary' section with filters for 'Status' and 'Entity (Customer): Email'. Below this is a 'Data Preview' table with columns: DATE, SALES REP, ENTITY, and ENTITY (CUSTOMER): EMAIL. The 'DATE' column is highlighted with a red box. The table shows 4 rows of data.

DATE	SALES REP	ENTITY	ENTITY (CUSTOMER): EMAIL
1/15/2002		Eric Schmidt	eschmidt@freeversion.com
1/4/2002		Amy Boughton	boughton751@freeversion.com
1/28/2002		Tony Matsuda	tonymat123@freeversion.com
2/13/2002		Phillip Van Hook	vanhook@freecheckbook.net

Previous: [Create joins with other record types](#)

Next: [Create conditions](#)

Create conditions

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Conditions let you filter the query results in a dataset.

Note: A condition filters entire records and fields from a dataset, which affects all workbooks that are based on the dataset. For example, if you create a condition that filters invoice records from a dataset, workbooks based on that dataset will also not include invoice record data.

In this step, you will create objects to represent conditions for the dataset.

1. In your script file, create a condition using `dataset.createCondition(options)`. This condition filters any customer records that are inactive (that is, the Inactive box in the UI is checked). Because this condition applies to a field (`isinactive`) that does not have a corresponding `dataset.Column` object in your script, you must create a column for the field to use when creating the condition. Add the following code after the three `dataset.createColumn(options)` calls in [Create columns](#):

```

1 | var inactiveColumn = dataset.createColumn({
2 |     fieldId: 'isinactive'
3 | });
4 | var inactiveCondition = dataset.createCondition({

```

```

5 |     column: inactiveColumn,
6 |     operator: 'IS',
7 |     values: [false]
8 | });

```

The `dataset.createCondition(options)` method returns a `dataset.Condition` object. For a list of available operators, see the help topic [query.Operator](#).

2. Create a second condition that filters any transaction records with a transaction date on or before March 3, 2020. You already created a `dataset.Column` object for the `trandate` field in [Create columns](#), so you can use that object when creating the condition. Add the following code after the first `dataset.createCondition(options)` call:

```

1 | var trandateCondition = dataset.createCondition({
2 |     column: trandateColumn,
3 |     operator: 'AFTER',
4 |     values: ['3/3/2020']
5 | });

```

Note that the date value you provide must be formatted according to the preferences set in your NetSuite account.

At this point, your script file should look similar to the following:

```

1 | require(['N/dataset'], function(dataset) {
2 |     var tutorialDataset = dataset.create({
3 |         type: 'customer'
4 |     });
5 |
6 |     var salesRepJoin = dataset.createJoin({
7 |         fieldId: 'salesrep'
8 |     });
9 |     var transactionJoin = dataset.createJoin({
10 |         fieldId: 'entity',
11 |         source: 'transaction'
12 |     });
13 |
14 |     var entityIdColumn = dataset.createColumn({
15 |         fieldId: 'entityid'
16 |     });
17 |     var emailColumn = dataset.createColumn({
18 |         fieldId: 'entityid',
19 |         join: salesRepJoin
20 |     });
21 |     var trandateColumn = dataset.createColumn({
22 |         fieldId: 'trandate',
23 |         join: transactionJoin
24 |     });
25 |
26 |     var inactiveColumn = dataset.createColumn({
27 |         fieldId: 'isinactive'
28 |     });
29 |     var inactiveCondition = dataset.createCondition({
30 |         column: inactiveColumn,
31 |         operator: 'IS',
32 |         values: [false]
33 |     });
34 |     var trandateCondition = dataset.createCondition({
35 |         column: trandateColumn,
36 |         operator: 'AFTER',
37 |         values: ['3/3/2020']
38 |     });
39 | });

```

SuiteAnalytics Workbook UI

In the SuiteAnalytics Workbook UI, conditions appear in the Criteria summary area above the Data Grid.

Previous: [Create columns](#)

Next: [Add components to the initial dataset](#)

Add components to the initial dataset

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

After you create your dataset components, you can add them to the dataset you created in [Create an initial dataset](#).

In this step, you will add your dataset components to the initial dataset.

1. In your script file, assign the columns you created to the `Dataset.columns` property. This property accepts an array of `dataset.Column` objects. Add the following code after the two `dataset.createCondition(options)` calls in [Create conditions](#):

```
1 | tutorialDataset.columns = [entityIdColumn, emailColumn, trandateColumn];
```

2. Assign the conditions you created to the `Dataset.condition` property. This property accepts a single `dataset.Condition` object, which represents the set of all conditions for the dataset. We use `dataset.createCondition(options)` and the AND operator to combine the conditions you created earlier. Add the following code after you assign the columns above:

```
1 | tutorialDataset.condition = dataset.createCondition({
2 |   operator: 'AND',
3 |   children: [inactiveCondition, trandateCondition]
4 | });
```

At this point, your script file should look similar to the following:

```
1 | require(['N/dataset'], function(dataset) {
2 |   var tutorialDataset = dataset.create({
3 |     type: 'customer'
4 |   });
5 |
6 |   var salesRepJoin = dataset.createJoin({
7 |     fieldId: 'salesrep'
```

```

8   });
9   var transactionJoin = dataset.createJoin({
10     fieldId: 'entity',
11     source: 'transaction'
12   });
13
14   var entityIdColumn = dataset.createColumn({
15     fieldId: 'entityid'
16   });
17   var emailColumn = dataset.createColumn({
18     fieldId: 'entityid',
19     join: salesRepJoin
20   });
21   var trandateColumn = dataset.createColumn({
22     fieldId: 'trandate',
23     join: transactionJoin
24   });
25
26   var inactiveColumn = dataset.createColumn({
27     fieldId: 'isinactive'
28   });
29   var inactiveCondition = dataset.createCondition({
30     column: inactiveColumn,
31     operator: 'IS',
32     values: [false]
33   });
34   var trandateCondition = dataset.createCondition({
35     column: trandateColumn,
36     operator: 'AFTER',
37     values: ['3/3/2020']
38   });
39
40   tutorialDataset.columns = [entityIdColumn, emailColumn, trandateColumn];
41   tutorialDataset.condition = dataset.createCondition({
42     operator: 'AND',
43     children: [inactiveCondition, trandateCondition]
44   });
45   });

```

Previous: [Create conditions](#)

Next: [Save and run the dataset](#)

Save and run the dataset

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

When you are done creating your dataset, you can save it and run it. You must save the dataset before you can view it in the SuiteAnalytics Workbook UI. When you run the dataset, you receive a query result set that includes all of the rows in the dataset.

In this step, you will save the dataset and run it to obtain the query result set.

1. In your script file, save the dataset using [Dataset.save\(options\)](#). To save a dataset, you must provide a name, and you can optionally provide an ID (with the prefix `custdataset`) and a description. If you do not specify an ID, one is generated automatically. Add the following code after you add dataset columns and conditions in [Add components to the initial dataset](#):

```

1 | tutorialDataset.save({
2 |   name: 'My Tutorial Dataset',
3 |   description: 'This is a tutorial dataset.',
4 |   id: 'custdataset_tutorialDataset'
5 | });

```

2. Run the dataset using [Dataset.run\(\)](#). Add the following code after you save the dataset above:

```

1 | var results = tutorialDataset.run();

```

The `Dataset.run()` method returns a [query.ResultSet](#) object (which is located in the N/query module). You can also run the dataset as a paged query using `Dataset.runPaged(options)`, which returns a [query.PagedData](#) object (also located in the N/query module).

Your final script file should look similar to the following:

```

1  require(['N/dataset'], function(dataset) {
2      var tutorialDataset = dataset.create({
3          type: 'customer'
4      });
5
6      var salesRepJoin = dataset.createJoin({
7          fieldId: 'salesrep'
8      });
9      var transactionJoin = dataset.createJoin({
10         fieldId: 'entity',
11         source: 'transaction'
12     });
13
14     var entityIdColumn = dataset.createColumn({
15         fieldId: 'entityid'
16     });
17     var emailColumn = dataset.createColumn({
18         fieldId: 'entityid',
19         join: salesRepJoin
20     });
21     var trandateColumn = dataset.createColumn({
22         fieldId: 'trandate',
23         join: transactionJoin
24     });
25
26     var inactiveColumn = dataset.createColumn({
27         fieldId: 'isinactive'
28     });
29     var inactiveCondition = dataset.createCondition({
30         column: inactiveColumn,
31         operator: 'IS',
32         values: [false]
33     });
34     var trandateCondition = dataset.createCondition({
35         column: trandateColumn,
36         operator: 'AFTER',
37         values: ['3/3/2020']
38     });
39
40     tutorialDataset.columns = [entityIdColumn, emailColumn, trandateColumn];
41     tutorialDataset.condition = dataset.createCondition({
42         operator: 'AND',
43         children: [inactiveCondition, trandateCondition]
44     });
45
46     tutorialDataset.save({
47         name: 'My Tutorial Dataset',
48         description: 'This is a tutorial dataset.',
49         id: 'custdataset_tutorialDataset'
50     });
51
52     var results = tutorialDataset.run();
53 });

```

Testing Your Solution

Typically, you run scripts in NetSuite by associating the script with an entry point. When the entry point is triggered, the associated script runs. For this tutorial, you can test your solution by using the SuiteScript Debugger. You can access the debugger in your NetSuite account and copy the contents of your script file into the debugger. When you run the script, it creates the dataset that you defined in your script. You can verify that the dataset was created successfully by viewing it in the SuiteAnalytics Workbook UI. For more information about the SuiteScript Debugger, see the help topic [SuiteScript Debugger](#).

Previous: [Add components to the initial dataset](#)

Tutorial: Creating a Workbook Using the Workbook API

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

In this tutorial, you'll learn how to:

- Load a dataset to use for a workbook
- Create a table view with data from the dataset
- Define conditional formatting rules
- Create a pivot that includes defined axes and a report style
- Define data dimensions and dimension items
- Create data measures and calculated measures

You can write your script in a text editor of your choice. When the script is complete, you can use the Dataset Builder Plug-in and Workbook Builder Plug-in to create the dataset and workbook in your NetSuite account.



Tip: If you are new to the Workbook API and want to learn how to create a simple dataset, see [Tutorial: Creating a Dataset Using the Workbook API](#).

Navigating the Tutorial

This tutorial includes navigation links at the bottom of each page. You can use these links to go forward to the next step in the tutorial or go back to the previous step.

Use the following table to go to a specific step in the tutorial.

Step	Description
Prerequisites	This topic lists the prerequisites for this tutorial, including required NetSuite features and permissions.
Full scripts	This topic lists the complete scripts that you will create during the tutorial.
Create the dataset	This topic describes how to create the dataset to use for the workbook.
Set up the workbook	This topic describes how to set up the structure of the workbook and load the dataset.
Create a table view	This topic describes how to create a table view, including conditional formatting rules and table columns.
Create a pivot	This topic describes how to create a pivot, including data dimensions, data and calculated measures, report styles, and pivot axes.

Related Help Topics

The following help topics provide more information about the concepts discussed in this tutorial:

- [Creating a Workbook](#)
- [Available Record Types](#)
- [Workbook Table Views](#)

- [Workbook Pivot Tables](#)
- [Calculated Measures](#)
- [SuiteAnalytics Workbook Tutorial](#)
- [Navigating SuiteAnalytics Workbook](#)
- [N/workbook Module](#)

Next: [Prerequisites](#)

Prerequisites

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Before you start this tutorial, review the following prerequisites. If you are not sure whether your NetSuite account or role meet these prerequisites, contact your administrator.

Required Features

The following features must be enabled in your NetSuite account:

- **SuiteAnalytics Workbook** – This feature lets you access SuiteAnalytics Workbook. It is required both to access the SuiteAnalytics Workbook UI and to use the N/dataset, N/datasetLink, and N/workbook modules in a script.
- **Server SuiteScript** or **Client SuiteScript** – These features let you create and run server scripts or client scripts in SuiteScript.

For more information about enabling features, see the help topic [Enabling Features](#).

Required Roles or Permissions

Your role and role permissions must let you do the following:

- Access the Analytics tab in the NetSuite UI
- View saved datasets and workbooks
- Create new datasets and workbooks

For more information about roles and permissions, see the help topic [NetSuite Users & Roles](#).

Previous: [Tutorial: Creating a Workbook Using the Workbook API](#)

Next: [Full scripts](#)

Full scripts

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

This tutorial is designed to help you create a workbook using a step-by-step approach. You may want to see the full scripts that you will create, so they are listed here.

Dataset

This dataset includes fields related to transactions, transaction lines, and accounting impact. The dataset uses the transaction record as the base record type. It joins this record type with the transaction line record type using the transactionlines field. It also joins the transaction line record type with the transaction accounting line record type using the accountingimpact field. The dataset includes columns from all of these record types, including transaction line ID, transaction name, type, date, and so on.

This dataset is created using the Dataset Builder Plug-in. For more information, see the help topic [Dataset Builder Plug-in](#).

```

1  /**
2   * @NApiVersion 2.0
3   * @NScriptType datasetbuilderplugin
4   */
5   define(['N/dataset'], function (nDataset) {
6       return Object.freeze({
7           createDataset: function (context) {
8               // Create joins with other record types
9               var transactionlines = nDataset.createJoin({
10                   fieldId: 'transactionlines'
11               })
12
13               var accountingimpact = nDataset.createJoin({
14                   fieldId: 'accountingimpact',
15                   join: transactionlines
16               })
17
18               // Create the dataset and its columns
19               context.dataset = nDataset.create({
20                   type: 'transaction',
21                   columns: [
22                       nDataset.createColumn({
23                           fieldId: 'id',
24                           join: transactionlines,
25                           alias: 'TranlineID',
26                           label: 'Tranline ID'
27                       }),
28                       nDataset.createColumn({
29                           fieldId: 'trandisplayname',
30                           alias: 'Transaction',
31                           label: 'Transaction'
32                       }),
33                       nDataset.createColumn({
34                           fieldId: 'type',
35                           alias: 'Type',
36                           label: 'Type'
37                       }),
38                       nDataset.createColumn({
39                           fieldId: 'trandate',
40                           alias: 'Date',
41                           label: 'Date'
42                       }),
43                       nDataset.createColumn({
44                           fieldId: 'entity',
45                           alias: 'Entity',
46                           label: 'Entity'
47                       }),
48                       nDataset.createColumn({
49                           fieldId: 'postingperiod',
50                           alias: 'PostingPeriod',
51                           label: 'Posting Period'
52                       }),
53                       nDataset.createColumn({
54                           fieldId: 'posting',
55                           join: accountingimpact,
56                           alias: 'Posting',
57                           label: 'Posting'
58                       }),
59                       nDataset.createColumn({
60                           fieldId: 'account',
61                           join: accountingimpact,
62                           alias: 'Account',
63                           label: 'Account'
64                       }),
65                       nDataset.createColumn({
66                           fieldId: 'credit',
67                           join: accountingimpact,
68                           alias: 'Credit',
69                           label: 'Credit'

```

```

70         },
71         nDataset.createColumn({
72             fieldId: 'debit',
73             join: accountingimpact,
74             alias: 'Debit',
75             label: 'Debit'
76         })
77     ],
78     })
79 }
80 });
81 });

```

Workbook

This workbook loads the dataset defined in the previous section, and it includes a table view and pivot based on the data in the dataset. The workbook demonstrates features such as conditional formatting rules, data dimensions, data measures, and calculated measures.

This workbook is created using the Workbook Builder Plug-in. For more information, see the help topic [Workbook Builder Plug-in](#).

```

1  /**
2   * @NApiVersion 2.0
3   * @NScriptType workbookbuilderplugin
4   */
5  define(['N/workbook', 'N/dataset', 'N/runtime'], function(nWorkbook, nDataset, nRuntime){
6      return {
7          createWorkbook: function (context){
8              // Load the dataset for the workbook
9              var dataset = nDataset.load({
10                 id: 'customscript_transaction'
11             })
12
13             // Create a helper object for sort directions
14             var SORT = {
15                 ASCENDING: nWorkbook.createSort({
16                     ascending: true
17                 }),
18                 DESCENDING: nWorkbook.createSort({
19                     ascending: false
20                 })
21             }
22
23             // Create two conditional formatting rules
24             var yellowRule = nWorkbook.createConditionalFormatRule({
25                 filter: nWorkbook.createTableColumnFilter({
26                     operator: "LESS",
27                     values: [10]
28                 }),
29                 style: nWorkbook.createStyle({
30                     backgroundColor: nWorkbook.createColor({
31                         red: 255,
32                         green: 255,
33                         blue: 0,
34                     })
35                 })
36             });
37
38             var pinkRule = nWorkbook.createConditionalFormatRule({
39                 filter: nWorkbook.createTableColumnFilter({
40                     operator: "GREATER_OR_EQUAL",
41                     values: [0]
42                 }),
43                 style: nWorkbook.createStyle({
44                     backgroundColor: nWorkbook.createColor({
45                         red: 255,
46                         green: 192,
47                         blue: 203
48                     })
49                 })
50             });
51         }
52     };
53 });

```

```

49     })
50   });
51
52   // The color is set on first matching rule and is not overwritten by
53   // subsequent rules
54   var conditionalFormat = nWorkbook.createConditionalFormat({rules: [yellowRule, pinkRule]});
55
56   // Create a table view
57   var credit = nWorkbook.createTableColumn({datasetColumnAlias: 'Credit', conditionalFormats: [conditionalFormat]});
58   var debit = nWorkbook.createTableColumn({datasetColumnAlias: 'Debit', conditionalFormats: [conditionalFormat]});
59   var tableview = nWorkbook.createTable({id: 'view', name: 'Table', dataset: dataset, columns: [credit, debit]});
60
61   // Create data dimensions to be used in a pivot
62   var allSubNodesSelector = nWorkbook.DescendantOrSelfNodesSelector;
63
64   var typeItem = nWorkbook.createDataDimensionItem({
65     label: 'Type',
66     expression: dataset.getExpressionFromColumn({
67       alias: 'Type'
68     })
69   })
70
71   var postingPeriodItem = nWorkbook.createDataDimensionItem({
72     label: 'Posting Period',
73     expression: dataset.getExpressionFromColumn({
74       alias: 'PostingPeriod'
75     })
76   })
77
78   var typeDataDimension = nWorkbook.createDataDimension({
79     items: [typeItem]
80   })
81
82   var postingPeriodDataDimension = nWorkbook.createDataDimension({
83     items: [postingPeriodItem]
84   })
85
86   // Create data measure definitions to be used in a pivot
87   var countMeasure = nWorkbook.createDataMeasure({
88     label: 'Count',
89     expressions: [dataset.getExpressionFromColumn({
90       alias: 'TranlineID'
91     })],
92     aggregation: 'COUNT_DISTINCT'
93   });
94
95   var sumCredit = nWorkbook.createDataMeasure({
96     label: 'Sum Credit',
97     expression: nWorkbook.createExpression({
98       functionId: workbook.ExpressionType.SIMPLE_CONSOLIDATE,
99       parameters: {
100         expression: dataset.getExpressionFromColumn({
101           alias: 'Credit'
102         })
103       }
104     }),
105     aggregation: 'SUM'
106   });
107
108   var sumDebit = nWorkbook.createDataMeasure({
109     label: 'Sum Debit',
110     expression: nWorkbook.createExpression({
111       functionId: workbook.ExpressionType.SIMPLE_CONSOLIDATE,
112       parameters: {
113         expression: dataset.getExpressionFromColumn({
114           alias: 'Debit'
115         })
116       }
117     }),
118     aggregation: 'SUM'
119   });
120
121   // Create a calculated measure to be used in a pivot

```

```

122     var calculatedMeasure = nWorkbook.createCalculatedMeasure({
123         label: 'Balance',
124         expression: nWorkbook.createExpression({
125             functionId: workbook.ExpressionType.MINUS,
126             parameters: {
127                 operand1: nWorkbook.createExpression({
128                     functionId: workbook.ExpressionType.MEASURE_VALUE,
129                     parameters: {
130                         measure: sumCredit
131                     }
132                 }),
133                 operand2: nWorkbook.createExpression({
134                     functionId: workbook.ExpressionType.MEASURE_VALUE,
135                     parameters: {
136                         measure: sumDebit
137                     }
138                 })
139             }
140         })
141     });
142
143     var conditionalRowSelector = nWorkbook.DescendantOrSelfNodesSelector;
144     var conditionalColumnSelector = nWorkbook.DescendantOrSelfNodesSelector;
145     var conditionalMeasureSelector = nWorkbook.createMeasureSelector({measures: [calculatedMeasure]});
146
147     var measureValueSelector = nWorkbook.createMeasureValueSelector({
148         rowSelector: conditionalRowSelector,
149         columnSelector: conditionalColumnSelector,
150         measureSelector: conditionalMeasureSelector
151     });
152
153     // Create a report style
154     var rule = nWorkbook.createReportStyleRule({
155         expression: nWorkbook.createExpression({
156             functionId: workbook.ExpressionType.COMPARE,
157             parameters: {
158                 comparisonType: "GREATER_OR_EQUAL",
159                 operand1: nWorkbook.createExpression({
160                     functionId: workbook.ExpressionType.MEASURE_VALUE,
161                     parameters: {
162                         measure: calculatedMeasure
163                     }
164                 }),
165                 operand2: nWorkbook.createConstant(nWorkbook.createCurrency({
166                     amount: 1,
167                     id: "USD"
168                 }))
169             }
170         }),
171         style: nWorkbook.createStyle({
172             backgroundColor: nWorkbook.createColor({
173                 red: 255,
174                 green: 192,
175                 blue: 203
176             })
177         })
178     });
179
180     var reportStyle = nWorkbook.createReportStyle({
181         selectors: [measureValueSelector],
182         rules: [rule]
183     });
184
185     // Create pivot axes for the pivot
186     var rowAxis = nWorkbook.createPivotAxis({
187         root: nWorkbook.createSection({
188             children: [
189                 nWorkbook.createDataDimension({
190                     items: [
191                         nWorkbook.createDataDimensionItem({
192                             label: 'Date (Year)',
193                             expression: nWorkbook.createExpression({
194                                 functionId: workbook.ExpressionType.DATE_TIME_PROPERTY,

```

```

195         parameters: {
196             dateTime: dataset.getExpressionFromColumn({
197                 alias: 'Date'
198             }),
199             property: 'YEAR',
200             hierarchy: 'MONTH_BASED'
201         }
202     })
203 })
204 ],
205     children: [postingPeriodDataDimension]
206 },
207     countMeasure,
208     calculatedMeasure
209 ]
210 },
211     sortDefinitions: [
212         nWorkbook.createSortDefinition({
213             sortBy: [
214                 nWorkbook.createSortByDataDimensionItem({
215                     item: postingPeriodItem,
216                     sort: SORT.ASCENDING
217                 })
218             ],
219             selector: nWorkbook.createPathSelector({
220                 elements: [
221                     allSubNodesSelector,
222                     nWorkbook.createDimensionSelector({
223                         dimension: postingPeriodDataDimension
224                     })
225                 ]
226             })
227         })
228     ]
229 })
230
231 var columnAxis = nWorkbook.createPivotAxis({
232     root: nWorkbook.createSection({
233         children: [typeDataDimension],
234         totalLine: nWorkbook.TotalLine.FIRST_LINE
235     }),
236     sortDefinitions: [
237         nWorkbook.createSortDefinition({
238             sortBy: [
239                 nWorkbook.createSortByDataDimensionItem({
240                     item: typeItem,
241                     sort: SORT.ASCENDING
242                 })
243             ],
244             selector: nWorkbook.createPathSelector({
245                 elements: [
246                     allSubNodesSelector,
247                     nWorkbook.createDimensionSelector({
248                         dimension: typeDataDimension
249                     })
250                 ]
251             })
252         })
253     ]
254 })
255
256 // Create the pivot
257 var pivot = nWorkbook.createPivot({
258     id: 'pivot',
259     name: 'Pivot: Transaction types per posting period',
260     dataset: dataset,
261     rowAxis: rowAxis,
262     columnAxis: columnAxis,
263     reportStyles: [reportStyle]
264 })
265
266 // Create the workbook with the specified table view and pivot
267 context.workbook = nWorkbook.create({

```

```

268         tables: [tableview],
269         pivots: [pivot]
270     })
271 }
272 }
273 })

```

Previous: [Prerequisites](#)

Next: [Create the dataset](#)

Create the dataset

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

In this step, you will create the dataset for the workbook. All workbooks must use a dataset. In a dataset, you combine the fields of a root record type and any joined related record types to create a query.

 **Tip:** If you are new to the Workbook API and want to learn how to create a simple dataset, see [Tutorial: Creating a Dataset Using the Workbook API](#).

Create a new file in a text editor of your choice. In this file, add the following code to load the N/dataset module:

```

1  define(['N/dataset'], function (nDataset) {
2
3  });

```

This dataset is created using the Dataset Builder Plug-in. When you use this plug-in, your plug-in implementation script file must include the createDataset entry point. This entry point is triggered when the list of datasets is accessed in the SuiteAnalytics Workbook UI. The script file must return this entry point using `Object.freeze()`. For more information, see the help topic [Dataset Builder Plug-in](#).

Inside the define definition, add the following code for the createDataset entry point:

```

1  return Object.freeze({
2      createDataset: function (context) {
3
4      }
5  });

```

Inside this function definition, create two joins. The first one joins the base record type for the dataset (transaction) with the transaction line record type using the transactionlines field. The second one joins the transaction line record type with the transaction accounting line record type using the accountingimpact field.

```

1  var transactionlines = nDataset.createJoin({
2      fieldId: 'transactionlines'
3  })
4
5  var accountingimpact = nDataset.createJoin({
6      fieldId: 'accountingimpact',
7      join: transactionlines
8  })

```

The function you define for the createDataset entry point accepts a context parameter. To save a dataset using the Dataset Builder Plug-in, you create a `dataset.Dataset` object using `dataset.create(options)`, then assign this object to the `context.dataset` property.

Add the following code to create a dataset and assign it to the `context.dataset` property:

```
1 context.dataset = nDataset.create({
2   type: 'transaction',
3   columns: [ ]
4 })
```

Columns represent the fields on each record type that you want to include in the resulting dataset query. You can use [dataset.createColumn\(options\)](#) to create as many columns as you need. In this tutorial, we create columns for several fields, including transaction line ID, transaction name, date, posting period, and more.

Add the following columns to the dataset using the `columns` parameter of [dataset.create\(options\)](#):

```
1 columns: [
2   nDataset.createColumn({
3     fieldId: 'id',
4     join: transactionlines,
5     alias: 'TranlineID',
6     label: 'Tranline ID'
7   }),
8   nDataset.createColumn({
9     fieldId: 'trandisplayname',
10    alias: 'Transaction',
11    label: 'Transaction'
12  }),
13  nDataset.createColumn({
14    fieldId: 'type',
15    alias: 'Type',
16    label: 'Type'
17  }),
18  nDataset.createColumn({
19    fieldId: 'trandate',
20    alias: 'Date',
21    label: 'Date'
22  }),
23  nDataset.createColumn({
24    fieldId: 'entity',
25    alias: 'Entity',
26    label: 'Entity'
27  }),
28  nDataset.createColumn({
29    fieldId: 'postingperiod',
30    alias: 'PostingPeriod',
31    label: 'Posting Period'
32  }),
33  nDataset.createColumn({
34    fieldId: 'posting',
35    join: accountingimpact,
36    alias: 'Posting',
37    label: 'Posting'
38  }),
39  nDataset.createColumn({
40    fieldId: 'account',
41    join: accountingimpact,
42    alias: 'Account',
43    label: 'Account'
44  }),
45  nDataset.createColumn({
46    fieldId: 'credit',
47    join: accountingimpact,
48    alias: 'Credit',
49    label: 'Credit'
50  }),
51  nDataset.createColumn({
52    fieldId: 'debit',
53    join: accountingimpact,
54    alias: 'Debit',
55    label: 'Debit'
56  })
57 ]
```

57 |]

At this point, your dataset script file is complete and should look similar to the full script in the Dataset section of [Full scripts](#).

Previous: [Full scripts](#)

Next: [Set up the workbook](#)

Set up the workbook

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

In this step, you will create the initial structure for the workbook and load the dataset you created in the previous step. In later steps, you will add elements for a table view and pivot to this structure.

Create a new file in a text editor, and add the following code to load the N/dataset and N/workbook modules:

```
1 | define(['N/dataset', 'N/workbook'], function(nDataset,nWorkbook) {
2 |
3 | });
```

Similar to the dataset you created, this workbook is created using the Workbook Builder Plug-in. When you use this plug-in, your plug-in implementation script file must include the `createWorkbook` entry point. This entry point is triggered when the list of workbooks is accessed in the SuiteAnalytics Workbook UI. The script file must return this entry point. For more information, see the help topic [Workbook Builder Plug-in](#).

Inside the define definition, add the following code for the `createWorkbook` entry point:

```
1 | return {
2 |     createWorkbook: function (context) {
3 |
4 |     }
5 | }
```

Inside this function definition, use [dataset.loadDataset\(options\)](#) to load the dataset you created in the previous step. The ID of the dataset is the same ID that you specify when you upload the plug-in implementation script file to NetSuite. In this tutorial, the dataset ID is `customscript_transaction`.

```
1 | var dataset = nDataset.load({
2 |     id: 'customscript_transaction'
3 | })
```

We also create a helper object to hold values for sort directions, which will help us when we create pivot axes later in the tutorial. This helper object uses [workbook.createSort\(options\)](#) to create simple [workbook.Sort](#) objects representing ascending and descending sorts.

```
1 | var SORT = {
2 |     ASCENDING: nWorkbook.createSort({
3 |         ascending: true
4 |     }),
5 |     DESCENDING: nWorkbook.createSort({
6 |         ascending: false
7 |     })
8 | }
```

At this point, your script file should look similar to the following:

```
1 | define(['N/workbook', 'N/dataset'], function(nWorkbook, nDataset){
```



```

2      return {
3          createWorkbook: function (context){
4              var dataset = nDataset.load({
5                  id: 'customscript_transaction'
6              })
7
8              var SORT = {
9                  ASCENDING: nWorkbook.createSort({
10                      ascending: true
11                  }),
12                  DESCENDING: nWorkbook.createSort({
13                      ascending: false
14                  })
15              }
16          }
17      }
18  });

```

Previous: [Create the dataset](#)

Next: [Create a table view](#)

Create a table view

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

In this step, you will create a table view based on the data in our dataset. Table views let you analyze your data in a tabular format using specified columns and optional formatting rules. The following sections describe different aspects of creating a table view.

Conditional formatting rules

You can use conditional formatting rules to change the format of a table view cell based on conditions you specify. You can change a cell's background color, font size, font style, and other formatting properties.

In this tutorial, we use two conditional formatting rules:

- A rule that changes a cell's background color to yellow when the cell value is less than 10.
- A rule that changes a cell's background color to pink when the cell value is greater than or equal to 0.

These two rules may seem to conflict, but only the first matching rule is applied to a cell. Subsequent matching rules do not overwrite the formatting applied by the first matching rule. When we use these rules to create a conditional format, we add the rules in the order specified above. This approach means that if a cell matches both rules (for example, if the cell's value is 5), only the formatting for the first rule is applied and the cell background color is changed to yellow.

Use [workbook.createConditionalFormatRule\(options\)](#) to create the conditional formatting rules. This method accepts two parameters:

- **filter** – The filter used to determine whether a cell matches the rule. We use [workbook.createTableColumnFilter\(options\)](#) to create each filter.
- **style** – The style to apply when a cell matches the rule. We use [workbook.createStyle\(options\)](#) and [workbook.createColor\(options\)](#) to create the colors for each filter.

After the rules are created, use [workbook.createConditionalFormat\(options\)](#) to create the [workbook.ConditionalFormat](#) object that includes both rules.

```

1  var yellowRule = nWorkbook.createConditionalFormatRule({
2      filter: nWorkbook.createTableColumnFilter({
3          operator: "LESS",
4          values: [10]

```

```

5     }),
6     style: nWorkbook.createStyle({
7         backgroundColor: nWorkbook.createColor({
8             red: 255,
9             green: 255,
10            blue: 0,
11        })
12    })
13 });
14
15 var pinkRule = nWorkbook.createConditionalFormatRule({
16     filter: nWorkbook.createTableColumnFilter({
17         operator: "GREATER_OR_EQUAL",
18         values: [0]
19     }),
20     style: nWorkbook.createStyle({
21         backgroundColor: nWorkbook.createColor({
22             red: 255,
23             green: 192,
24             blue: 203
25         })
26     })
27 });
28
29 var conditionalFormat = nWorkbook.createConditionalFormat({
30     rules: [yellowRule, pinkRule]
31 });

```

Table columns

Now that we have a conditional format, we can use it to create the columns to include in the table view. Use [workbook.createTableColumn\(options\)](#) to create table view columns. To use this method, you need to specify the column in the dataset that you want to include. To do so, use the column alias that we set when we created the dataset.

```

1 var credit = nWorkbook.createTableColumn({
2     datasetColumnAlias: 'Credit',
3     conditionalFormats: [conditionalFormat]
4 });
5 var debit = nWorkbook.createTableColumn({
6     datasetColumnAlias: 'Debit',
7     conditionalFormats: [conditionalFormat]
8 });

```

Final table view

Finally, we use [workbook.createTable\(options\)](#) to create the final table view. Make sure you specify the dataset and columns to use.

```

1 var tableview = nWorkbook.createTable({
2     id: 'view',
3     name: 'Table',
4     dataset: dataset,
5     columns: [credit, debit]
6 });

```

At this point, your script file should look similar to the following:

```

1 define(['N/workbook', 'N/dataset'], function(nWorkbook, nDataset){
2     return {
3         createWorkbook: function (context) {
4             var dataset = nDataset.load({
5                 id: 'customscript_transaction'
6             })
7         }
8     }
9 });

```

```

8      var SORT = {
9          ASCENDING: nWorkbook.createSort({
10             ascending: true
11          }),
12          DESCENDING: nWorkbook.createSort({
13             ascending: false
14          })
15      }
16
17      var yellowRule = nWorkbook.createConditionalFormatRule({
18          filter: nWorkbook.createTableColumnFilter({
19             operator: "LESS",
20             values: [10]
21          }),
22          style: nWorkbook.createStyle({
23             backgroundColor: nWorkbook.createColor({
24                 red: 255,
25                 green: 255,
26                 blue: 0,
27             })
28          })
29      });
30
31      var pinkRule = nWorkbook.createConditionalFormatRule({
32          filter: nWorkbook.createTableColumnFilter({
33             operator: "GREATER_OR_EQUAL",
34             values: [0]
35          }),
36          style: nWorkbook.createStyle({
37             backgroundColor: nWorkbook.createColor({
38                 red: 255,
39                 green: 192,
40                 blue: 203
41             })
42          })
43      });
44
45      var conditionalFormat = nWorkbook.createConditionalFormat({
46          rules: [yellowRule, pinkRule]
47      });
48
49      var credit = nWorkbook.createTableColumn({
50          datasetColumnAlias: 'Credit',
51          conditionalFormats: [conditionalFormat]
52      });
53      var debit = nWorkbook.createTableColumn({
54          datasetColumnAlias: 'Debit',
55          conditionalFormats: [conditionalFormat]
56      });
57
58      var tableview = nWorkbook.createTable({
59          id: 'view',
60          name: 'Table',
61          dataset: dataset,
62          columns: [credit, debit]
63      });
64  }
65  }
66  });

```

Previous: [Set up the workbook](#)

Next: [Create a pivot](#)

Create a pivot

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

In this step, you'll create a pivot (also known as a pivot table) based on the data in the dataset. Pivots let you analyze different subsets of your data using more advanced features than are included for table

views. In a pivot, you can create data dimensions for the data you want to analyze, add data measures and calculated measures, and configure the axes of the pivot to display the information you want. The following sections describe different aspects of creating a pivot.

Data dimensions

You can use data dimensions to define the data you want to display and analyze in the pivot. You can create data dimensions based on columns in the dataset, then specify the data dimensions when you define the axes in the pivot.

In this tutorial, we are interested in the data from two columns: Type and Posting Period. We use [workbook.createDataDimensionItem\(options\)](#) to create a data dimension item for each column. This method accepts an expression that represents the column from the dataset. To obtain this expression, use [Dataset.getExpressionFromColumn\(options\)](#) and provide the alias of the column.

After creating the data dimension items, we create data dimensions for each item. A data dimension can include multiple data dimension items, but in our case, we have only one item per dimension. Use [workbook.createDataDimension\(options\)](#) to create each data dimension.

```

1 var typeItem = nWorkbook.createDataDimensionItem({
2   label: 'Type',
3   expression: dataset.getExpressionFromColumn({
4     alias: 'Type'
5   })
6 })
7
8 var postingPeriodItem = nWorkbook.createDataDimensionItem({
9   label: 'Posting Period',
10  expression: dataset.getExpressionFromColumn({
11    alias: 'PostingPeriod'
12  })
13 })
14
15 var typeDataDimension = nWorkbook.createDataDimension({
16   items: [typeItem]
17 })
18
19 var postingPeriodDataDimension = nWorkbook.createDataDimension({
20   items: [postingPeriodItem]
21 })

```

Data measures and calculated measures

Measures let you perform simple operations on the data in a column and display the results in the pivot. You can use two types of measures:

- **Data measure** – A data measure represents an operation on the data in a column or set of columns. You can use the operations defined in the [workbook.Aggregation](#) enum to aggregate the column data in different ways. For example, you can use the `workbook.Aggregation.COUNT_DISTINCT` value to count the number of distinct values that appear in a column.
- **Calculated measure** – A calculated measure represents a value that is calculated based on other column values or data measures. For example, you can calculate the difference between aggregated values in two columns.

In this tutorial, you will create three data measures (using [workbook.createDataMeasure\(options\)](#)) and one calculated measure (using [workbook.createCalculatedMeasure\(options\)](#)). The first data measure counts the number of distinct values in the Trancode column in the dataset. Similar to creating data dimension items, you must provide an expression that represents the dataset column to create the measure for.

```

1 var countMeasure = nWorkbook.createDataMeasure({
2   label: 'Count',
3   expressions: [dataset.getExpressionFromColumn({
4     alias: 'TranlineID'
5   })],
6   aggregation: 'COUNT_DISTINCT'
7 });

```

The second data measure sums the values in the Credit column in the dataset. Again, we need to provide an expression that represents the column, but we use `workbook.createExpression(options)` to create a custom expression (instead of obtaining one from a column using `Dataset.getExpressionFromColumn(options)`). When you use this method to create an expression, you can specify a function that represents the operation used in the expression using the values in `workbook.ExpressionType`. Each function accepts different parameters.

In our case, we use the `SIMPLE_CONSOLIDATE` function and provide an expression representing the column to consolidate. This function consolidates a currency expression based on the user's subsidiary and current accounting period.

```

1 var sumCredit = nWorkbook.createDataMeasure({
2   label: 'Sum Credit',
3   expression: nWorkbook.createExpression({
4     functionId: workbook.ExpressionType.SIMPLE_CONSOLIDATE,
5     parameters: {
6       expression: dataset.getExpressionFromColumn({
7         alias: 'Credit'
8       })
9     }
10  }),
11  aggregation: 'SUM'
12 });

```

The third data measure sums the values in the Debit column in the dataset. We use the same approach that we used for the Credit column.

```

1 var sumDebit = nWorkbook.createDataMeasure({
2   label: 'Sum Debit',
3   expression: nWorkbook.createExpression({
4     functionId: workbook.ExpressionType.SIMPLE_CONSOLIDATE,
5     parameters: {
6       expression: dataset.getExpressionFromColumn({
7         alias: 'Debit'
8       })
9     }
10  }),
11  aggregation: 'SUM'
12 });

```

Now we can create our calculated measure. This measure calculates the difference between the Sum Credit and Sum Debit data measures that we previously created. It uses the `workbook.ExpressionType.MEASURE_VALUE` function to obtain the values of each data measure, then it uses the `workbook.ExpressionType.MINUS` function to calculate the difference.

```

1 var calculatedMeasure = nWorkbook.createCalculatedMeasure({
2   label: 'Balance',
3   expression: nWorkbook.createExpression({
4     functionId: workbook.ExpressionType.MINUS,
5     parameters: {
6       operand1: nWorkbook.createExpression({
7         functionId: workbook.ExpressionType.MEASURE_VALUE,
8         parameters: {
9           measure: sumCredit
10        }
11      })
12    }
13  });

```

```

12         operand2: nWorkbook.createExpression({
13             functionId: workbook.ExpressionType.MEASURE_VALUE,
14             parameters: {
15                 measure: sumDebit
16             }
17         })
18     }
19 })
20 });

```

Selectors

Selectors are objects that select elements in a pivot for different purposes, such as applying style rules and configuring the pivot axes. Pivots are stored as hierarchical tree structures, and selectors take care of selecting the correct row, column, and measure when needed.

In the next section, we will create a report style for the pivot. This report style changes the formatting of cells in the pivot, similar to the conditional formatting rules we created for the table view. A report style must be supported by a measure value selector, which selects the cells to apply the style to based on a measure. We create this selector using [workbook.createMeasureValueSelector\(options\)](#). This method accepts a row selector, column selector, and measure selector. The row selector and column selectors are [workbook.D descendantOrSelfNodesSelector](#) objects, which are generic selectors that work in many situations. The measure selector must be created based on the measure to select (our calculated measure above) using [workbook.createMeasureValueSelector\(options\)](#).

We also create another [workbook.D descendantOrSelfNodesSelector](#), which we will use when we create the pivot axes in a later section.

```

1  var conditionalRowSelector = nWorkbook.D descendantOrSelfNodesSelector;
2  var conditionalColumnSelector = nWorkbook.D descendantOrSelfNodesSelector;
3  var conditionalMeasureSelector = nWorkbook.createMeasureSelector({
4      measures: [calculatedMeasure]
5  });
6
7  var measureValueSelector = nWorkbook.createMeasureValueSelector({
8      rowSelector: conditionalRowSelector,
9      columnSelector: conditionalColumnSelector,
10     measureSelector: conditionalMeasureSelector
11 });
12
13 var allSubNodesSelector = nWorkbook.D descendantOrSelfNodesSelector;

```

Report style

Now that we have the required selectors, we can create the report style. This report style uses expressions to determine whether the calculated measure value (Sum Credit minus Sum Debit) in a cell is greater than or equal to 1 US dollar (USD). If this condition is true, the cell color is changed to pink.

Use [workbook.createReportStyleRule\(options\)](#) to create a report style rule, then use [workbook.createReportStyle\(options\)](#) to create the report style based on the rules you provide. The [workbook.createReportStyle\(options\)](#) method also accepts a selector, and we provide the measure value selector we created in the previous section.

```

1  var rule = nWorkbook.createReportStyleRule({
2      expression: nWorkbook.createExpression({
3          functionId: workbook.ExpressionType.COMPARE,
4          parameters: {
5              comparisonType: "GREATER_OR_EQUAL",
6              operand1: nWorkbook.createExpression({
7                  functionId: workbook.ExpressionType.MEASURE_VALUE,
8                  parameters: {
9                      measure: calculatedMeasure

```

```

10     }
11     },
12     operand2: nWorkbook.createConstant(nWorkbook.createCurrency({
13         amount: 1,
14         id: "USD"})
15     )
16 },
17 ),
18 style: nWorkbook.createStyle({
19     backgroundColor: nWorkbook.createColor({
20         red: 255,
21         green: 192,
22         blue: 203
23     })
24 })
25 });
26
27 var reportStyle = nWorkbook.createReportStyle({
28     selectors: [measureValueSelector],
29     rules: [rule]
30 });

```

Pivot axes

Now it is time to assemble all of the elements and create the pivot. A pivot consists of a row axis and a column axis, and you create these axes using [workbook.createPivotAxis\(options\)](#). You must provide a root element for the axis, and this element contains a hierarchical set of data dimensions and measures for that axis. You must also provide a sort definition for the axis, which defines how items are sorted on the axis.

The row axis of our pivot contains a data dimension that applies to the Date column, meaning that the top level of this axis in the pivot will be organized by date. As a child of this dimension, we include the Posting Period data dimension that we created earlier. We also include the countMeasure and calculatedMeasure objects that we created, and everything is wrapped up in a [workbook.Section](#) object. For the sort definition, we specify an ascending sort based on the Posting Period data dimension item, and we use [workbook.createPathSelector\(options\)](#) and [workbook.createDimensionSelector\(options\)](#) to select the correct data dimension.

```

1  var rowAxis = nWorkbook.createPivotAxis({
2      root: nWorkbook.createSection({
3          children: [
4              nWorkbook.createDataDimension({
5                  items: [
6                      nWorkbook.createDataDimensionItem({
7                          label: 'Date (Year)',
8                          expression: nWorkbook.createExpression({
9                              functionId: workbook.ExpressionType.DATE_TIME_PROPERTY,
10                             parameters: {
11                                 dateTime: dataset.getExpressionFromColumn({
12                                     alias: 'Date'
13                                 }),
14                                 property: 'YEAR',
15                                 hierarchy: 'MONTH_BASED'
16                             }
17                         })
18                     ]
19                 },
20                 children: [postingPeriodDataDimension]
21             }),
22             countMeasure,
23             calculatedMeasure
24         ]
25     }),
26     sortDefinitions: [
27         nWorkbook.createSortDefinition({
28             sortBy: [
29                 nWorkbook.createSortByDataDimensionItem({

```

```

30         item: postingPeriodItem,
31         sort: SORT.ASCENDING
32     })
33 },
34 selector: nWorkbook.createPathSelector({
35     elements: [
36         allSubNodesSelector,
37         nWorkbook.createDimensionSelector({
38             dimension: postingPeriodDataDimension
39         })
40     ]
41 })
42 })
43 ]
44 })

```

We use a similar approach to create the column axis of the pivot. This axis contains the Type data dimension item, and we specify that we want to include a total line as the first line on the axis. For the sort definition, we specify an ascending sort based on the Type data dimension item.

```

1  var columnAxis = nWorkbook.createPivotAxis({
2      root: nWorkbook.createSection({
3          children: [typeDataDimension],
4          totalLine: nWorkbook.TotalLine.FIRST_LINE
5      }),
6      sortDefinitions: [
7          nWorkbook.createSortDefinition({
8              sortBy: [
9                  nWorkbook.createSortByDataDimensionItem({
10                     item: typeItem,
11                     sort: SORT.ASCENDING
12                 })
13             ],
14             selector: nWorkbook.createPathSelector({
15                 elements: [
16                     allSubNodesSelector,
17                     nWorkbook.createDimensionSelector({
18                         dimension: typeDataDimension
19                     })
20                 ]
21             })
22         })
23     ]
24 })

```

Final pivot

Finally, we use `workbook.createPivot(options)` to create the pivot, and we use `workbook.create(options)` to create the entire workbook, including the table view and pivot.

```

1  var pivot = nWorkbook.createPivot({
2      id: 'pivot',
3      name: 'Pivot: Transaction types per posting period',
4      dataset: dataset,
5      rowAxis: rowAxis,
6      columnAxis: columnAxis,
7      reportStyles: [reportStyle]
8  })
9
10 context.workbook = nWorkbook.create({
11     tables: [tableView],
12     pivots: [pivot]
13 })

```

At this point, your workbook script file is complete and should look similar to the full script in the Workbook section of [Full scripts](#).

You're done! Now you can use the Dataset Builder Plug-in and Workbook Builder Plug-in to upload your script files to NetSuite, and you can view the complete dataset and workbook in the Analytics area.

Previous: [Create a table view](#)

Workbook API Limitations

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

As you use the Workbook API, consider the following limitations:

- Charting features are not available in this release.
- When you use the Dataset Builder Plug-in or Workbook Builder Plug-in to create custom datasets or workbooks, you may receive an unexpected error, "Workbook does not exist" error, or "Workbook is no longer available" error when viewing the dataset or workbook in the SuiteAnalytics Workbook UI. These errors may occur if the code used to create the dataset or workbook in the plug-in is invalid or includes syntax errors. To work around this issue, see the following help topics:
 - For datasets, see the help topic [Creating a Dataset Builder Plug-in Script File](#).
 - For workbooks, see the help topic [Creating a Workbook Builder Plug-in Script File](#).

SuiteScript 2.x Script Types

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

SuiteScript 2.x offers several script types each with their own entry points as described in the following table.

Script Type	Description	Where They Execute
SuiteScript 2.x Bundle Installation Script Type	Bundle installation scripts are specialized server scripts that perform processes in target accounts as part of a bundle installation, update, or uninstallation. These processes include setup, configuration, and data management tasks that would otherwise have to be completed by account administrators. Entry points: ■ afterInstall(params) – defines the function that executes after a bundle is installed for the first time in a target account. ■ afterUpdate(params) – defines the function that executes after a bundle in a target account is updated. ■ beforeInstall(params) – defines the function that executes before a bundle is installed for the first time in a target account. ■ beforeUninstall(params) – defines the function that executes before a bundle is uninstalled from a target account. ■ beforeUpdate(params) – defines the function that executes before a bundle in a target account is updated.	On the server
SuiteScript 2.x Client Script Type	Client scripts are scripts that are executed by predefined event triggers in the client browser. They run on individual forms, can be deployed globally, and are applied to entity and transaction record types. They can validate user-entered data and auto-populate fields or sublists at various form events. Global client scripts enable centralized management of scripts that can be applied to an entire record type. Entry points: ■ fieldChanged(scriptContext) – defines the function that executes when a field is changed by a user or client call. ■ lineInit(scriptContext) – defines the function that executes when a line is initialized, such as when an existing line is selected or a new line is added (for example, to a transaction form). ■ localizationContextEnter(scriptContext) – defines the function that executes when the record enters the localization context that is specified on the script deployment record. ■ localizationContextExit(scriptContext) – defines the function that executes when the record exits the localization context. ■ pageInit(scriptContext) – defines the function that executes when the page completes loading or when the form is reset. ■ postSourcing(scriptContext) – defines the function that executes when a field that sources information from another field is modified. Executes on transaction forms only. ■ saveRecord(scriptContext) – defines the function that executes when a record is saved (that is, after the Save button is pressed but before the form is submitted).	On the client's browser

Script Type	Description	Where They Execute
	■ sublistChanged(scriptContext) – defines the function that executes after a sublist has been inserted, removed, or edited. ■ validateDelete(scriptContext) – defines the function that executes when an existing line in an edit sublist is deleted. ■ validateField(scriptContext) – defines the function that executes when a field is changed by a user or client side call. ■ validateInsert(scriptContext) – defines the function that executes when a sublist line is inserted into an edit sublist. ■ validateLine(scriptContext) – defines the function that executes before a line is added to an inline editor sublist or editor sublist.	
SuiteScript 2.1 Custom Tool Script Type	Custom tool scripts let external AI clients run SuiteScript code in response to natural language prompts, without you needing to log in to NetSuite. These tools have flexible entry points that are defined using a JSON schema.	On the server
SuiteScript 2.x Map/Reduce Script Type	The map/reduce script type is designed for scripts that need to handle large amounts of data. They provide a structured framework for processing a large number of records or a large amount of data and are best suited for situations where the data can be divided into small, independent parts. Entry points: ■ getInputData(inputContext) – defines the function that marks the beginning of the map/reduce script execution. This entry point invokes the input stage which is where input data is generated. ■ map(mapContext) – defines the function that invokes the map stage where input data is parsed into key-value pairs. ■ reduce(reduceContext) – defines the function that invokes the reduce stage where processing can occur for each key-value pair. ■ summarize(summaryContext) – defines the function that invokes the summarize stage where processed data is summarized.	On the server
SuiteScript 2.x Mass Update Script Type	Mass update scripts allows you to programmatically perform custom mass updates to update fields that are not available through general mass updates. These scripts can run complex calculations across many records. Entry points: ■ each(params) – defines the function that iterates through all applicable records allowing you to add logic to each record.	On the server
SuiteScript 2.x Portlet Script Type	Portlet scripts are used to create custom dashboard portlets. For example, you can use the portlet script type to create a portlet that is populated on-the-fly with company messages based on data within the system. Entry points: ■ render(params) – defines the function that executes when the Portlet script is triggered.	On the server, but rendered on the client's browser
SuiteScript 2.x RESTlet Script Type	RESTlets can be used to define custom RESTful integrations to NetSuite. You can make a RESTlet available for other applications to call, either from an external application or from within NetSuite. When an application or another script calls a RESTlet, the RESTlet script executes and, in some cases, returns a value to the calling application. Entry points:	On the server

Script Type	Description	Where They Execute
	■ delete – defines the function that executes when a DELETE request is sent to a RESTlet. An HTTP response body is returned. ■ get – defines the function that executes when a GET request is sent to a RESTlet. An HTTP response body is returned. ■ post – defines the function that executes when a POST request is sent to a RESTlet. An HTTP response body is returned. ■ put – defines the function that executes when a PUT request is sent to a RESTlet. An HTTP response body is returned.	
SuiteScript 2.x Scheduled Script Type	Scheduled scripts are server scripts that are executed (processed) with SuiteCloud Processors. You can deploy scheduled scripts so they are submitted for processing at a future time, or at future times on a recurring basis. You can also submit scheduled scripts on demand from the deployment record or from another script with the N/task Module. Entry points: ■ execute – defines the function that executes when the scheduled script is triggered.	On the server
SuiteScript 2.x SDF Installation Script Type	SDF installation scripts are used to perform tasks during deployment of a SuiteApp from SuiteCloud Development Framework (SDF) to your target account. They are automatically executed when a SuiteApp project is deployed. Entry points: ■ run(scriptContext) – defines what is executed when the script is specified to be run by the SDF deployment (in the deploy.xml file of an SDF project).	On SuiteApp project deployment
SuiteScript 2.x Suitelet Script Type	Suitelets allow you to build custom NetSuite pages and backend logic. They are server scripts that operate in a request-response model, and are invoked by HTTP GET or POST requests to system generated URLs. Suitelets enable the creation of dynamic web content and build NetSuite-looking pages, and they can be used to implement custom front and backends. Entry points: ■ onRequest(params) – defines the function that executes when the Suitelet is triggered.	On the server
SuiteScript 2.x User Event Script Type	User event scripts are executed when you perform certain actions on records, such as create, load, update, copy, delete, or submit. These scripts customize the workflow and association between your NetSuite entry forms. These scripts can also be used for additional processing before records are entered or for validating entries based on other data in the system. Entry points: ■ afterSubmit(context) – defines the function that executes immediately after a write operation on a record. ■ beforeLoad(context) – defines the function that executes whenever a read operation occurs on a record, and prior to returning the record or page. ■ beforeSubmit(context) – defines the function that executes prior to any write operation on the record.	On the server

Script Type	Description	Where They Execute
SuiteScript 2.x Workflow Action Script Type	Workflow action scripts allow you to create custom Workflow Actions that are defined on a record in a workflow. Entry points: ■ onAction(scriptContext) – defines a Workflow Action script trigger point.	On the server

SuiteScript Best Practices

- Always thoroughly test your code before using it on your live NetSuite data.
- Type all record, field, sublist, tab, and subtab IDs in lowercase in your SuiteScript code.
- Prefix all custom script IDs and deployment IDs with an underscore (_).
- Do not hard-code any passwords in scripts. The password and password2 fields are supported for scripting.
- If the same code is used across multiple forms, ensure that you test any changes in the code for **each** form that the code is associated with.
- Include proper error handling sequences in your script wherever data may be inconsistent, not available, or invalid for certain functions. For example, if your script requires a field value to validate another, ensure that the field value is available.
- Organize your code into reusable chunks. Many functions can be used in a variety of forms. Any reusable functions should be stored in a common library file and then called into specific event functions for the required forms as needed.
- Place all custom code and markup, including third party libraries, in your own namespace.



Important: Custom code must not be used to access the NetSuite DOM. Developers must use SuiteScript APIs to access NetSuite UI components.

- Use the built in Library functions whenever possible for reading/writing Date/Currency fields and for querying XML documents
- During script development, break your scripts into components, load them individually, and then test each one — inactivating all but the one you are testing when multiple components are tied to a single user event.
- When working with script type events, your function name should correspond with the event. For example, a pageInit event can be named PageInit or formAPageInit.
- Since name values can change, ensure that you use **static** ID values in your API calls where applicable.
- Although you can use any desired naming conventions for functions within your code, you should use custom namespaces or unique prefixes for all your function names.
- Thoroughly comment your code. This practice helps with debugging and development and assists NetSuite Customer Support in locating problems if necessary. However, note the inline comments are not allowed within an object.
- You must use the [runtime.getCurrentScript\(\)](#) function in the runtime module to reference script parameters. For example, use the following code to obtain the value of a script parameter named custscript_case_field:

```

1 | define(['N/runtime'], function(runtime) {
2 |     function pageInit(context) {
3 |         var strField = runtime.getCurrentScript().getParameter('SCRIPT', 'custscript_case_field');
4 |         ...

```

```
5 |    });
```

- Make sure that your script does not take a long time to execute. A script may execute for a long time if any or all of the following occur:
 - The script performs a large number of record operations without going over the usage limit.
 - The script processes a large number of transactions for the same records, such as items or lot numbers, without exceeding the usage unit limit.
 - The script causes a large number of user event scripts or workflows to execute.
 - The script performs database searches or updates that collectively take a long time to finish

Each server script type or application has a time limit for execution. This limit is not fixed and depends on the script type or application. If a single execution of a server script or application takes longer than the time limit for that script type or application, a `SSS_TIME_LIMIT_EXCEEDED` error is thrown. This error can also be thrown from a script that is executed by another script (for example, from a user event script that is executed by a scheduled script).

You can use SuiteScript Analysis to learn about when the script was installed and how it performed in the past. For more information, see the help topic [Analyzing Scripts](#).

SuiteScript 2.x Bundle Installation Script Type

📘 Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Bundle installation scripts are specialized server scripts that perform processes in target accounts when you install, update, or uninstall a bundle. These processes include setup, configuration, and data management tasks that would otherwise have to be done by account administrators.

Every bundle can include a bundle installation script that runs automatically when the bundle is installed, updated, or uninstalled. Each bundle installation script can contain triggers to be executed before install, after install, before update, after update, and after uninstallation.

Unlike other types of SuiteScript scripts, bundle installation scripts have no audience – they are always executed using the Administrator role and in the context of bundle installation, update, or uninstallation. Bundle installation scripts don't have event types.

A bundle installation script can be associated with multiple bundles. Before you can associate a script with a bundle, it must have a script record and at least one deployment. A bundle creator associates a bundle installation script with a bundle by selecting one of its deployments in the Bundle Builder. The script file and script record are automatically included in the bundle when it's added to target accounts. You can hide script file contents from target accounts by setting an option in the File Cabinet record.

If a bundle installation script fails, it stops the bundle installation, update, or uninstallation. Bundle installation scripts can include their own error handling in addition to errors thrown by SuiteBundler and SuiteScript. When a bundle installation script throws an error, it returns an **Installation Error** code followed by the message defined by the script author. For information about scripting with bundle installation scripts, see [SuiteScript 2.x Bundle Installation Script Entry Points and API](#).

For more information about bundle installation scripts, see the help topics [Using Bundle Installation Scripts](#) and [Bundle Support Across Account Types](#).

📘 Note: Custom modules aren't supported in bundle installation scripts.

You can use SuiteCloud Development Framework (SDF) to manage bundle installation scripts as part of file-based customization projects. For information about SDF, see . You can use the Copy to Account feature to copy a bundle installation script to another of your accounts. Each bundle installation script page has a Copy to Account option in the upper right corner. For information about Copy to Account, see the help topic [Copy to Account](#).

You can use SuiteScript Analytics to learn about when the script was installed and how it performed in the past. For more information, see the help topic [Analyzing Scripts](#).

Bundle Installation Script Sample

This script sample will make sure that the Time Off Management feature is enabled in the target NetSuite account before a bundle installation or a bundle update. If it isn't enabled, this script will warn the user.

You can write bundle installation scripts using either SuiteScript 2.0 or 2.1. This script sample is written in SuiteScript 2.0. You can update this script to use SuiteScript 2.1 features.

- For help with writing scripts in SuiteScript 2.x, see [SuiteScript 2.x Hello World](#) and [SuiteScript 2.x Entry Point Script Creation and Deployment](#).
- For more information about SuiteScript versions and SuiteScript 2.1, see [SuiteScript Versioning Guidelines](#) and [SuiteScript 2.1](#).

```

1  /**
2   * @ApiVersion 2.x
3   * @NScriptType BundleInstallationScript
4   */
5  define(['N/runtime'], function(runtime) {
6      function checkPrerequisites() {
7          if (!runtime.isFeatureInEffect({
8              feature: 'TIMEOFFMANAGEMENT'
9          })) {
10             throw 'The TIMEOFFMANAGEMENT feature must be enabled. ' +
11                 'Please enable the feature and try again.';
12         }
13     }
14
15     return {
16         beforeInstall: function(params) {
17             checkPrerequisites();
18         },
19         beforeUpdate: function(params) {
20             checkPrerequisites();
21         }
22     };
23 });

```

SuiteScript 2.x Bundle Installation Script Reference

📘 Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

A bundle installation script's functions run automatically during bundle install, update, or uninstall, based on these triggers or entry points:

- **beforeInstall** - executed before a bundle is installed for the first time in a target account.
- **afterInstall** - executed after a bundle is installed for the first time in a target account.
- **beforeUpdate** - executed before a bundle in a target account is updated.
- **afterUpdate** - executed after a bundle in a target account is updated.
- **beforeUninstall** - executed before a bundle is uninstalled from a target account.

A bundle installation script file should include at least one entry point. If you are using multiple entry points, you should include them in the same script file.

The following are example uses for each entry point:

- **beforeInstall**: Check the existing configuration and setup in the target account prior to bundle installation, and halt the installation with an error message if the target account doesn't meet minimum requirements to run the solution.
- **afterInstall**: Automate the setup and configuration of the bundled application after it's been installed in the target account, eliminating manual tasks.
- **afterInstall** or **afterUpdate**: Connect to an external system to fetch data and complete the setup of the bundled application.
- **beforeUpdate**: Manage required data changes in the target account prior to executing an upgrade.
- **beforeUninstall**: Reset configuration settings or remove data associated with the bundle being uninstalled.

Two specialized parameters are available to functions in bundle installation scripts to return the version of bundles as specified on the Bundle Basics page of the Bundle Builder.

- The **toversion** parameter returns the version of the bundle that will be installed in the target account. This parameter is available to **beforeInstall**, **afterInstall**, **beforeUpdate**, and **afterUpdate** entry point functions.
- The **fromversion** parameter returns the version of the bundle that's currently installed in the target account. This parameter is available to **beforeUpdate** and **afterUpdate** entry point functions.

A bundle installation script file can call functions in other script files if those files are added as library script files on the script record. Any .js library script files are automatically included in the bundle when it's added to target accounts.

Bundle installation scripts can call scheduled scripts, but only in the **afterInstall** and **afterUpdate** entry point functions. Calls to scheduled scripts aren't supported in the **beforeInstall**, **beforeUpdate**, or **beforeUninstall** entry points.

Bundle installation scripts have a 10,000 governance unit limit per execution.

You can create multiple deployments for each bundle installation script, each with different parameters, but only one deployment can be associated with each bundle. When you associate a bundle installation script with a bundle, you choose a specific script deployment.

Bundle installation scripts need to be executed with administrator privileges. The Execute as Role field should always be set to Administrator on the script deployment record.

Bundle installation scripts can only be run in target accounts if the Status is set to Released. You can set the Status to Testing if you want to debug the script but it must be set to Released when you are done testing.

Setting Up a Bundle Installation Script

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Complete the following steps to set up a bundle installation script.

Create the Bundle Installation Script File

Write your bundle installation script as a .js file and include at least one of the **beforeInstall**, **afterInstall**, **beforeUpdate**, **afterUpdate**, and **beforeUninstall** entry point functions, as needed. Your script can call functions in other files, but you will need to list those files as library script files on the NetSuite script record.

Add the Bundle Installation Script File to the File Cabinet

1. Go to Documents > Files > File Cabinet, and select the folder where you want to add the file.

You should add your file to the SuiteScripts folder, but it can be added to any other folder of your choice.

2. Click **Add File** and browse to the .js file.
3. In the File Cabinet folder where you added the bundle installation script file, click the **Edit** link next to file.
4. Check the **Available for SuiteBundles** box.
5. Optionally, you can check the **Hide in SuiteBundle** box.

Because this script file will be included in the bundle, by default its contents will be accessible to users of target accounts where the bundle is installed. If you don't want these users to see this file, you can set this option to hide it.

6. Click **Save**.

Create the Bundle Installation Script Record

1. Go to Setup > Customization > Scripts > New, and click **Bundle Installation**.
2. Complete fields in the script record and save.

Although you don't need to set every field on the Script record, you must at least provide a **Name** for the Script record, load your SuiteScript file to the record, and specify at least one of the executing functions on the Scripts subtab.

The screenshot shows the 'Scripts' subtab of a SuiteScript 2.x Bundle Installation Script record. The 'Script File' dropdown menu is open, showing '<Type then tab>'. Below the dropdown, a red rectangular box highlights five checkboxes: 'Before Install Function', 'After Install Function', 'Before Update Function', 'After Update Function', and 'Before Uninstall Function'. All checkboxes are currently unchecked.

3. These functions should all be in the main script file. If these functions call functions in other script files, you need to list those files as library script files.

Define Bundle Installation Script Deployment

After you create the bundle installation script record, you need to define the script deployment. To create a script deployment, do one of the following:

- When you save your Script record, you can immediately create a Script Deployment record by selecting **Save and Deploy** from the Script record **Save** button.
- If you clicked **Save**, immediately afterwards you can click **Deploy Script** on the script record.
- If you want to update a deployment that already exists, go to Customization > Scripting > Script Deployments. Click **Edit** next to the deployment record you want to edit.

Complete fields in the script deployment record and click **Save**.

- Be sure to check the **Execute as Admin** box.
- If you want to debug the script, set the **Status** to **Testing**. To enable the script to be run in a target account, you must set the **Status** to **Released**.

Associate the Script with a Bundle

Note: You need to have the SuiteBundler feature enabled in your account to access the Bundle Builder where you complete this task.

1. Start the Bundle Builder.
 - If you are creating a new bundle, go to Customization > SuiteBuilder > Create Bundle.
 - If you are editing an existing bundle, go to Customization > SuiteBundler > Search & Install Bundles > List, and select **Edit** from the Action menu for the desired bundle.
2. On the Bundle Basics page, select a bundle installation script deployment from the **Installation Script** dropdown.
3. Proceed through the remaining Bundle Builder steps, making definitions as necessary, and click **Save**. Note the following:
 - On the Select Objects page of the Bundle Builder, you don't have to explicitly add the bundle installation script. This script record and the related .js file are included automatically in the bundle, as are any other .js files that are listed as library script files on the script record.
 - After the bundle has been saved, this script record and related file(s) are listed as Bundle Components on the Bundle Details page.

Overview Components SuiteApp Info Messages Audit Trail	
Display Options ☐ Hide Components ☒ Show Components	
Export - CSV	
Name	ID
File Cabinet	
Files	
 NSEmail.js	5765
 NSCheckout.js	5766
 NSPostInstall.js	5511
Folders	
 Reference Checkout (34 KB)	729
 Custom Checkout (0 KB)	628
SuiteScripts	
Bundle Installation	
 NSRCPPostInstall (NSRCPPostInstall)	customscript11_2

SuiteScript 2.x Bundle Installation Script Entry Points and API

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Script Entry Point	Description
<code>afterInstall(params)</code>	Defines the function that executes after a bundle is installed for the first time in a target account.

Script Entry Point	Description
afterUpdate(params)	Defines the function that executes after a bundle in a target account is updated.
beforeInstall(params)	Defines the function that executes before a bundle is installed for the first time in a target account.
beforeUninstall(params)	Defines the function that executes before a bundle is uninstalled from a target account.
beforeUpdate(params)	Defines the function that executes before a bundle in a target account is updated.

afterInstall(params)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Description	Defines the function that's executed after a bundle is installed for the first time in a target account.
Returns	void
Since	2016.1

Parameters

Note: The `params` parameter is a JavaScript object. It's automatically passed to the script entry point by NetSuite.

Parameter	Type	Description	Since
<code>params.version</code>	string	The version of the bundle that's being installed in the target account.	2016.1

afterUpdate(params)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Description	Defines the function that's executed after a bundle in a target account is updated.
Returns	void
Since	2016.1

Parameters

Note: The `params` parameter is a JavaScript object. It's automatically passed to the script entry point by NetSuite.

Parameter	Type	Description	Since
<code>params.fromVersion</code>	string	The version of the bundle that's currently installed in the target account.	2016.1
<code>params.toVersion</code>	string	The new version of the bundle that's being installed in the target account.	2016.1

beforeInstall(params)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Description	Defines the function that's executed before a bundle is installed for the first time in a target account. Calls to scheduled scripts aren't supported.
Returns	void
Since	2016.1

Parameters

Note: The `params` parameter is a JavaScript object. It's automatically passed to the script entry point by NetSuite.

Parameter	Type	Description	Since
<code>params.version</code>	string	The version of the bundle that's being installed in the target account.	2016.1

beforeUninstall(params)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Description	Defines the function that's executed before a bundle is uninstalled from a target account. Calls to scheduled scripts aren't supported.
Returns	void
Since	2016.1

Parameters

Note: The `params` parameter is a JavaScript object. It's automatically passed to the script entry point by NetSuite.

Parameter	Type	Description	Since
<code>params.version</code>	string	The version of the bundle that's being uninstalled from the target account.	2016.1

beforeUpdate(params)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Description	Defines the function that's executed before a bundle in a target account is updated. Calls to scheduled scripts aren't supported.
Returns	void

Since	2016.1
-------	--------

Parameters

 **Note:** The `params` parameter is a JavaScript object. It's automatically passed to the script entry point by NetSuite.

Parameter	Type	Description	Since
<code>params.fromVersion</code>	string	The version of the bundle that's currently installed in the target account.	2016.1
<code>params.toVersion</code>	string	The new version of the bundle that's being installed in the target account.	2016.1

 **Warning:** Before the `beforeUpdate` entry point function runs, the bundle update process checks for custom objects and files that were in the previous bundle version but aren't in the new one. Then, the bundle update process deletes those custom objects and files from the previous version in the target account.

However, in case the `beforeUpdate` entry point function stops on any failed condition, the bundle update process stops the bundle update and the user is left with the previous bundle version, where the bundle update process has already deleted some custom objects and files, so the bundle may not be functioning correctly anymore.

You shouldn't remove unused custom objects and files from the new bundle version. You should remove such objects from the new bundle only after your whole install base is upgraded to the new bundle version.

SuiteScript 2.x Client Script Type

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Client scripts are scripts that are executed by predefined event triggers in the client browser. They validate user-entered data and auto-populate fields or sublists during form events. For details, see [SuiteScript 2.x Client Script Entry Points and API](#).

Scripts can be run on most standard records, custom record types, and custom NetSuite pages such as Suitelets. See the help topic [SuiteScript Supported Records](#) for a list of NetSuite records that support SuiteScript.

 **Important:** Client scripts only execute when you're in edit mode. If you've got a deployed client script with a `pageInit` entry point, it doesn't run when you view the form. It executes when you click **Edit**.

The following triggers can run a client script:

- Loading a form for editing
- Entering or changing a value in a field (before and after it is entered)
- Entering or changing a value in a field that sources another field
- Selecting a line item on a sublist

- Adding a line item (before and after it is entered)
- Saving a form

Record-level client scripts run after form-based clients and before user event scripts.

See the help topic [Script Type Usage Unit Limits](#) for details about client script governance.

 **Tip:** You can set the order in which client scripts execute on the Scripted Records page. See the help topic [The Scripted Records Page](#).

You can use SuiteCloud Development Framework (SDF) to manage client scripts in your file-based customization projects. For information about SDF, see . You can use the Copy to Account feature to copy an individual client script to another of your accounts. Each client script page has a clickable Copy to Account option in the upper right corner. For information about Copy to Account, see the help topic [Copy to Account](#).

You can use SuiteScript Analysis to learn about when the script was installed and how it performed in the past. For more information, see the help topic [Analyzing Scripts](#).

For additional information about SuiteScript 2.x client scripts, see the following:

- [SuiteScript 2.x Client Script Reference](#)
 - [Using the currentRecord Module in Client Scripts](#)
 - [Client Script Role Restrictions](#)
 - [Interfacing with Remote Objects in Client Scripts](#)
 - [Unrestricted Search Permissions in Client Scripts](#)
- [SuiteScript 2.x Client Script Entry Points and API](#)
 - [fieldChanged\(scriptContext\)](#)
 - [lineInit\(scriptContext\)](#)
 - [pageInit\(scriptContext\)](#)
 - [postSourcing\(scriptContext\)](#)
 - [saveRecord\(scriptContext\)](#)
 - [sublistChanged\(scriptContext\)](#)
 - [validateDelete\(scriptContext\)](#)
 - [validateField\(scriptContext\)](#)
 - [validateInsert\(scriptContext\)](#)
 - [validateLine\(scriptContext\)](#)
 - [localizationContextEnter\(scriptContext\)](#)
 - [localizationContextExit\(scriptContext\)](#)

SuiteScript Client Script Sample

For help with writing scripts in SuiteScript 2.x, see [SuiteScript 2.x Hello World](#) and [SuiteScript 2.x Entry Point Script Creation and Deployment](#).

The following sample shows a client script applied to a sales order form. Here's what the script does:

- When you edit the form, the pageInit event sets the customer field to a specific customer.
- When you save changes, the saveRecord event checks that the customer field is set and you've got at least one sales order item.
- When you edit a sales order item's quantity, the validateField event checks it's less than three.

- When you select a sales order item, the fieldChanged event updates a memo field to show it was selected.
- When you select a specific sales order item, the postSourcing event updates its price level.
- When you select an existing partner, the lineInit event switches the selected partner to a specific partner.
- When you delete a partner, the validateDelete event updates a memo field to show it was deleted.
- When you add or edit a partner, the validateInsert and validateLine events check their contribution is set to 100%.

```

1  /**
2   *@NApiVersion 2.x
3   *@NScriptType ClientScript
4   */
5  define(['N/error'], function(error) {
6      function pageInit(context) {
7          if (context.mode !== 'create')
8              return;
9          var currentRecord = context.currentRecord;
10         currentRecord.setValue({
11             fieldId: 'entity',
12             value: 107
13         });
14     }
15     function saveRecord(context) {
16         var currentRecord = context.currentRecord;
17         if (!currentRecord.getValue({fieldId: 'entity'}) || currentRecord.getLineCount({sublistId: 'item'}) < 1)
18             throw error.create({
19                 name: 'MISSING_REQ_ARG',
20                 message: 'Please enter all the necessary fields on the salesorder before saving'
21             });
22         return true;
23     }
24     function validateField(context) {
25         var currentRecord = context.currentRecord;
26         var sublistName = context.sublistId;
27         var sublistFieldName = context.fieldId;
28         var line = context.line;
29         if (sublistName === 'item') {
30             if (sublistFieldName === 'quantity') {
31                 if (currentRecord.getCurrentSublistValue({
32                     sublistId: sublistName,
33                     fieldId: sublistFieldName
34                 }) < 3)
35                     currentRecord.setValue({
36                         fieldId: 'otherrefnum',
37                         value: 'Quantity is less than 3'
38                     });
39             } else
40                 currentRecord.setValue({
41                     fieldId: 'otherrefnum',
42                     value: 'Quantity accepted'
43                 });
44         }
45     }
46     return true;
47 }
48 function fieldChanged(context) {
49     var currentRecord = context.currentRecord;
50     var sublistName = context.sublistId;
51     var sublistFieldName = context.fieldId;
52     var line = context.line;
53     if (sublistName === 'item' && sublistFieldName === 'item')
54         currentRecord.setValue({
55             fieldId: 'memo',
56             value: 'Item: ' + currentRecord.getCurrentSublistValue({
57                 sublistId: 'item',
58                 fieldId: 'item'
59             }) + ' is selected'
60         });

```

```

61     }
62     function postSourcing(context) {
63         var currentRecord = context.currentRecord;
64         var sublistName = context.sublistId;
65         var sublistFieldName = context.fieldId;
66         var line = context.line;
67         if (sublistName === 'item' && sublistFieldName === 'item')
68             if (currentRecord.getCurrentSublistValue({
69                 sublistId: sublistName,
70                 fieldId: sublistFieldName
71             }) === '39')
72                 if (currentRecord.getCurrentSublistValue({
73                     sublistId: sublistName,
74                     fieldId: 'pricelevels'
75                 }) !== '1-1')
76                     currentRecord.setCurrentSublistValue({
77                         sublistId: sublistName,
78                         fieldId: 'pricelevels',
79                         value: '1-1'
80                     });
81     }
82     function lineInit(context) {
83         var currentRecord = context.currentRecord;
84         var sublistName = context.sublistId;
85         if (sublistName === 'partners')
86             currentRecord.setCurrentSublistValue({
87                 sublistId: sublistName,
88                 fieldId: 'partner',
89                 value: '55'
90             });
91     }
92     function validateDelete(context) {
93         var currentRecord = context.currentRecord;
94         var sublistName = context.sublistId;
95         if (sublistName === 'partners')
96             if (currentRecord.getCurrentSublistValue({
97                 sublistId: sublistName,
98                 fieldId: 'partner'
99             }) === '55')
100                 currentRecord.setValue({
101                     fieldId: 'memo',
102                     value: 'Removing partner sublist'
103                 });
104         return true;
105     }
106     function validateInsert(context) {
107         var currentRecord = context.currentRecord;
108         var sublistName = context.sublistId;
109         if (sublistName === 'partners')
110             if (currentRecord.getCurrentSublistValue({
111                 sublistId: sublistName,
112                 fieldId: 'contribution'
113             }) !== '100.0%')
114                 currentRecord.setCurrentSublistValue({
115                     sublistId: sublistName,
116                     fieldId: 'contribution',
117                     value: '100.0%'
118                 });
119         return true;
120     }
121     function validateLine(context) {
122         var currentRecord = context.currentRecord;
123         var sublistName = context.sublistId;
124         if (sublistName === 'partners')
125             if (currentRecord.getCurrentSublistValue({
126                 sublistId: sublistName,
127                 fieldId: 'contribution'
128             }) !== '100.0%')
129                 currentRecord.setCurrentSublistValue({
130                     sublistId: sublistName,
131                     fieldId: 'contribution',
132                     value: '100.0%'
133                 });

```



```

134     return true;
135 }
136 function sublistChanged(context) {
137     var currentRecord = context.currentRecord;
138     var sublistName = context.sublistId;
139     var op = context.operation;
140     if (sublistName === 'item')
141         currentRecord.setValue({
142             fieldId: 'memo',
143             value: 'Total has changed to ' + currentRecord.getValue({fieldId: 'total'})
144         });
145 }
146
147 return {
148     pageInit: pageInit,
149     fieldChanged: fieldChanged,
150     postSourcing: postSourcing,
151     sublistChanged: sublistChanged,
152     lineInit: lineInit,
153     validateField: validateField,
154     validateLine: validateLine,
155     validateInsert: validateInsert,
156     validateDelete: validateDelete,
157     saveRecord: saveRecord
158 };
159 });

```

SuiteScript 2.x Client Script Reference

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Refer to the following guidance to help you write client scripts:

- You can use the `currentRecord` module to access the active record in your client script. For more information, see [Using the currentRecord Module in Client Scripts](#).
- You can specify that your client script only runs for certain roles. For more information, see [Client Script Role Restrictions](#).
- You can use the `N/log` module in a client script like you would in a server script, but if it's attached to a form (see [SuiteScript 2.x Form-Level Script Deployments](#)), log functions (`log.audit`, `log.debug`, `log.emergency`, `log.error`) won't work, and they'll be ignored. The script will run fine, but nothing gets logged. For logging in a client script attached to a form, use `console.log` instead.
- Client scripts interact with remote objects when they access the NetSuite database to create, load, copy, or change records. For more information, see [Interfacing with Remote Objects in Client Scripts](#).
- When you run a search in a client script, it uses the user's current role, which can cause issues if that role can't view the record. For information about how this restriction can be bypassed, see [Unrestricted Search Permissions in Client Scripts](#).

Also see the [Client Script Best Practices](#) section in the [SuiteScript Developer Guide](#) for a list of best practices to follow when using client scripts.

Using the currentRecord Module in Client Scripts

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

You can use the `currentRecord` module to access the record that is active in the current client context. For more information about the `currentRecord` module, see the help topic [N/currentRecord Module](#).

The following sample runs when the `saveRecord` entry point is triggered. This script uses the `currentRecord` module to check a Sales Order record, making sure the transaction date is within the last week and the total is correct.

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType ClientScript
4   */
5  define(['N/ui/message'],function(msg) {
6      function showErrorMessage(msgText) {
7          var myMsg = msg.create({
8              title: "Cannot Save Record",
9              message: msgText,
10             type: msg.Type.ERROR
11         });
12
13         myMsg.show({
14             duration: 5000
15         });
16     }
17
18     function saveRec(context) {
19         var rec = context.currentRecord;
20         var currentDate = new Date();
21         var oneWeekAgo = new Date(currentDate - 1000 * 60 * 60 * 24 * 7);
22
23         // Validate transaction date is not older than current time by one week
24         if (rec.getValue({fieldId: 'trandate'}) < oneWeekAgo) {
25             showErrorMessage("Cannot save sales order with trandate one week old.");
26             return false;
27         }
28
29         // Validate total is greater than 0
30         if (rec.getValue({fieldId: 'total'}) <= 0) {
31             showErrorMessage("Cannot save sales order with negative total amount.");
32             return false;
33         }
34         return true;
35     }
36
37     return {
38         saveRecord: saveRec
39     }
40 });

```

Client Script Role Restrictions

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Client scripts follow the permissions set up in your NetSuite account. You'll get an error if you try to run a client script to access a record without the right permissions.

The following client script attaches to a custom sales order form and executes when the fieldChanged entry point is triggered:

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType ClientScript
4   */
5  define(['N/search'],
6      function(search) {
7          function getSalesRepEmail(context) {
8              var salesRep = context.currentRecord.getValue({
9                  fieldId: 'salesrep'
10             });
11             var salesRepEmail = search.lookupFields({
12                 type: 'employee',
13                 id: salesRep,
14                 columns: ['email']
15             });
16             alert(JSON.stringify(salesRepEmail));
17         }

```

```

18
19     return {
20         fieldChanged: getSalesRepEmail
21     }
22 };

```

If you're logged in as an administrator, you'll get the alert when you load the sales order with this form. If you are logged in with a role that does not have permission to view/edit Employee records, you receive an error when you select the Sales Rep field.

To avoid this error, consider the following:

- Consider the types of users who will be using your custom form and running the script.
- Consider which record types users can't access. If everyone running the script needs access to those records, you might need to update their permissions (if you're an administrator).
- Consider rewriting the script to only use records that everyone can access.
- Consider writing the script as a user event script, and set the **Execute As Admin** preference on the Script Deployment page. Keep in mind that alerts only work in client scripts, not user event scripts. For more information about user event scripts, see [SuiteScript 2.x User Event Script Type](#).

Interfacing with Remote Objects in Client Scripts

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Client scripts interact with remote objects when they access the NetSuite database to create, load, copy, or change records.

The following client script loads a journal entry record and sets two line sublist lines when the saveRecord entry point is triggered.

```

1  /**
2   *@NApiVersion 2.0
3   *@NScriptType ClientScript
4   */
5  define(['N/record'], function(record){
6      function saveRecord(context){
7          var journalEntry = record.load({
8              id: 6,
9              type: record.Type.JOURNAL_ENTRY,
10             isDynamic: true
11         });
12         journalEntry.selectNewLine({
13             sublistId: 'line'
14         });
15         journalEntry.setCurrentSublistValue({
16             sublistId: 'line',
17             fieldId: 'account',
18             value: 1
19         });
20         journalEntry.setCurrentSublistValue({
21             sublistId: 'line',
22             fieldId: 'debit',
23             value: 1
24         });
25         journalEntry.setCurrentSublistValue({
26             sublistId: 'line',
27             fieldId: 'memo',
28             value: "Debit 1"
29         });
30         journalEntry.commitLine({
31             sublistId: 'line'
32         });
33         journalEntry.selectNewLine({
34             sublistId: 'line'

```

```

35     });
36     journalEntry.setCurrentSublistValue({
37         sublistId: 'line',
38         fieldId: 'account',
39         value: 1
40     });
41     journalEntry.setCurrentSublistValue({
42         sublistId: 'line',
43         fieldId: 'credit',
44         value: 1
45     });
46     journalEntry.setCurrentSublistValue({
47         sublistId: 'line',
48         fieldId: 'memo',
49         value: "Credit 1"
50     });
51     journalEntry.commitLine({
52         sublistId: 'line'
53     });
54
55     var recordId = journalEntry.save();
56     return true;
57 }
58 return {
59     saveRecord : saveRecord
60 };
61 });

```

Unrestricted Search Permissions in Client Scripts

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

When you run a search in a client script, it uses your current role's permissions. This can cause issues if your role doesn't have permission to view the record. Here's a way to get around this limitation:

Unrestrict search permissions on client scripts

1. Update your client script to send a GET HTTP request to a Suitelet that runs your search. You can pass values using URL parameters. For example:

```

1 | https.get.promise({
2 |     url: stSuiteletUrl + '&orderId=' + stOrder
3 | }).then(function(response){
4 |     var objResponse = JSON.parse(response.body);
5 | }).catch(function onRejected(reason) {
6 |     console.log(reason);
7 | });

```

2. Create the Suitelet that will run the search. Make sure it's set to run as an Administrator.
 - You can access URL parameters using the `context.request.parameters` argument. For example:

```

1 | let stOrderId = scriptContext.request.parameters.orderId;

```

- Return your search results using the `context.response.write()` method. For example:

```

1 | scriptContext.response.write({
2 |     output: JSON.stringify({
3 |         status: 'success',
4 |         results: arrResults
5 |     });
6 | });

```

The unrestricted results will be passed to the client script.

SuiteScript 2.x Client Script Entry Points and API

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Script Entry Point	Description
fieldChanged(scriptContext)	This is the function that runs when a user or client changes a field.
lineInit(scriptContext)	This function runs when a line is set up, like when you select an existing line or add a new one.
localizationContextEnter(scriptContext)	This function runs when the record enters a specific localization context, as set in the script deployment record.
localizationContextExit(scriptContext)	This function runs when the record exits that context.
pageInit(scriptContext)	This function runs when the page finishes loading or the form is reset.
postSourcing(scriptContext)	This function runs when you change a field that gets its info from another field. Executes on transaction forms only.
saveRecord(scriptContext)	This function runs when you save a record, after you click submit but before the form is sent.
sublistChanged(scriptContext)	This function runs after you add, remove, or edit a sublist.
validateDelete(scriptContext)	This function checks if it's okay to delete a line from an edit sublist.
validateField(scriptContext)	This function checks if a field change is valid when a user or client makes a change.
validateInsert(scriptContext)	This function checks if it's okay to add a new line to an edit sublist.
validateLine(scriptContext)	This function checks if it's okay to add a new line to an inline editor sublist before it happens.

fieldChanged(scriptContext)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Description	This function runs when a user or client changes a field. This event can also be triggered by beforeLoad user event scripts. You can use this event to do the following: - Give users more info based on what they enter. - Turn fields on or off based on user input. For an example, see SuiteScript Client Script Sample.  Note: Note that this event won't run if the field value is changed or entered in the page URL. Instead, use the pageInit function to handle updated field values in URLs. See pageInit(scriptContext).
Returns	void
Since	Version 2015 Release 2

Parameters

Note: The scriptContext parameter is a JavaScript object. NetSuite automatically sends it to your script's entry point.

Parameter	Type	Description	Since
scriptContext.currentRecord	currentRecord.CurrentRecord	The current form record. For more information about CurrentRecord object members, see the help topic CurrentRecord Object Members .	Version 2015 Release 2
scriptContext.sublistId	string	The sublist ID name.	Version 2015 Release 2
scriptContext.fieldId	string	The field ID name.	Version 2015 Release 2
scriptContext.line	string	The line number (zero-based index) if the field is in a sublist or a matrix. If it's not in a sublist or matrix, this value is undefined.	Version 2015 Release 2
scriptContext.column	string	The column number (zero-based index) if the field is in a matrix. If it's not in a matrix, this value is undefined.	Version 2015 Release 2

For an example of the fieldChanged entry point, see [SuiteScript Client Script Sample](#).

lineInit(scriptContext)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Description	This function runs when a line is set up.. This happens when you select an existing line or add a new one to a form. This event can behave like a pageInit event for line items in an inline editor sublist or editor sublist. For an example, see SuiteScript Client Script Sample .
Returns	void
Since	Version 2015 Release 2

Parameters

Note: The scriptContext parameter is a JavaScript object. NetSuite automatically sends it to your script's entry point.

Parameter	Type	Description	Since
scriptContext.currentRecord	currentRecord.CurrentRecord	The current form record.	Version 2015 Release 2

Parameter	Type	Description	Since
		For more information about CurrentRecord object members, see the help topic CurrentRecord Object Members .	
scriptContext.sublistId	string	The sublist ID name.	Version 2015 Release 2

For an example of the lineInit entry point, see [SuiteScript Client Script Sample](#).

localizationContextEnter(scriptContext)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Description	This function runs when a record enters a specific localization context set in the script deployment record.
Returns	void
Since	Version 2020 Release 1

Parameters

Parameter	Type	Description	Since
scriptContext.currentRecord	currentRecord.CurrentRecord	The current form record. For more information about CurrentRecord object members, see the help topic CurrentRecord Object Members .	Version 2015 Release 2
scriptContext.locale	string	The list of countries that represent the new localization context.	Version 2020 Release 1

If you've set up localization on the Context Filtering subtab, the pageInit entry point of the script is ignored, and no other entry points of the script are called before localizationContextEnter or after localizationContextExit. It is possible that the record may never enter a localization context. If that happens, none of the script's callbacks will run. For more information, see the help topic [Using the Context Filtering Subtab](#).

localizationContextExit(scriptContext)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Description	This function runs when a record leaves a localization context.
Returns	void
Since	Version 2020 Release 1

Parameters

Parameter	Type	Description	Since
scriptContext.currentRecord	currentRecord.CurrentRecord	The current form record. For more information about CurrentRecord object members, see the help topic CurrentRecord Object Members .	Version 2015 Release 2
scriptContext.locale	string	The list of countries that represent the new localization context.	Version 2020 Release 1

If you've set up localization on the Context Filtering subtab, the pageInit entry point of the script is ignored, and no other entry points of the script are called before localizationContextEnter or after localizationContextExit. The record might not ever enter a localization context. If that happens, none of the script's callbacks will run. For more information, see the help topic [Using the Context Filtering Subtab](#).

pageInit(scriptContext)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer



Important: Client scripts only run in edit mode. If you've got a deployed client script with a pageInit entry point, it doesn't run when you view the form. It runs when you click **Edit**.

Description	This function runs after the page loads or when the form is reset. You can use this function to: ■ Set default field values. ■ Turn fields on or off. ■ Change field availability or values based on the record's data. ■ Set initial field values using flags. ■ Show alerts if the loaded data is inconsistent or corrupted. ■ Get user login info and update field options or values accordingly. ■ Check that required fields for your custom code (but not necessarily required for the form) exist. For an example, see SuiteScript Client Script Sample.
Returns	void
Since	Version 2015 Release 2

Parameters



Note: The scriptContext parameter is a JavaScript object. NetSuite automatically sends it to your script's entry point.

Parameter	Type	Description	Since
scriptContext.currentRecord	currentRecord.CurrentRecord	The current form record.	Version 2015 Release 2

Parameter	Type	Description	Since
		For more information about CurrentRecord object members, see the help topic CurrentRecord Object Members .	
scriptContext.mode	string	The mode in which the record is being accessed. The mode can be one of these: - copy - create - edit	Version 2015 Release 2

For an example of the pageInit entry point, see [SuiteScript Client Script Sample](#).

postSourcing(scriptContext)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Description	This function runs when you click a field that gets its info from another field. Sometimes this happens as soon as you click the field. Other times, it happens after you've changed the field's value. This event works like fieldChanged event after all dependent field values have been set. It waits for all the related field changes to finish before running your function. This only works on transaction forms. For an example, see SuiteScript Client Script Sample.  Note: If a field doesn't have any related fields, this event won't be triggered by changes to that field.
Returns	void
Since	Version 2015 Release 2

Parameters

 Note: The scriptContext parameter is a JavaScript object. NetSuite automatically sends it to your script's entry point.
--

Parameter	Type	Description	Since
scriptContext.currentRecord	currentRecord.CurrentRecord	The current form record. For more information about CurrentRecord object members, see the help topic CurrentRecord Object Members .	Version 2015 Release 2
scriptContext.sublistId	string	The sublist ID name.	Version 2015 Release 2

Parameter	Type	Description	Since
scriptContext.fieldId	string	The field ID name.	Version 2015 Release 2

For an example of the postSourcing entry point, see [SuiteScript Client Script Sample](#).

saveRecord(scriptContext)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Description	This function runs when you save a record, after you click submit but before the form is sent. You can use this function to: ■ Show alerts before saving the data. ■ Turn on fields that were turned off by other functions. ■ Redirect the user to a specified URL. For an example, see SuiteScript Client Script Sample.
Returns	true if the record is valid and is saved. false otherwise.
Since	Version 2015 Release 2

Parameters

Note: The scriptContext parameter is a JavaScript object. NetSuite automatically sends it to your script's entry point.

Parameter	Type	Description	Since
scriptContext.currentRecord	currentRecord.CurrentRecord	The current form record. For more information about CurrentRecord object members, see the help topic CurrentRecord Object Members.	Version 2015 Release 2

For an example of the saveRecord entry point, see [SuiteScript Client Script Sample](#).

sublistChanged(scriptContext)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: The sublistChanged function replaces the SuiteScript 1.0 function recal.

Description	This function runs after you add, remove, or edit a sublist. For an example, see SuiteScript Client Script Sample.
--------------------	--

Returns	void
Since	Version 2015 Release 2

Parameters

Note: The scriptContext parameter is a JavaScript object. NetSuite automatically sends it to your script's entry point.

Parameter	Type	Description	Since
scriptContext.currentRecord	currentRecord.CurrentRecord	The current form record. For more information about CurrentRecord object members, see the help topic CurrentRecord Object Members .	Version 2015 Release 2
scriptContext.sublistId	string	The sublist ID name.	Version 2015 Release 2

For an example of the sublistChanged entry point, see [SuiteScript Client Script Sample](#).

validateDelete(scriptContext)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Description	This function checks if it's okay to delete a line from an edit sublist. For an example, see SuiteScript Client Script Sample .
Returns	true if the sublist line is valid and the delete is successful. false otherwise.
Since	Version 2015 Release 2

Parameters

Note: The scriptContext parameter is a JavaScript object. NetSuite automatically sends it to your script's entry point.

Parameter	Type	Description	Since
scriptContext.currentRecord	currentRecord.CurrentRecord	The current form record. For more information about CurrentRecord object members, see the help topic CurrentRecord Object Members .	Version 2015 Release 2
scriptContext.sublistId	string	The sublist ID name.	Version 2015 Release 2

Parameter	Type	Description	Since
scriptContext.lineCount	number	The number of lines to be deleted. This property is defined only when validateDelete is triggered by the Clear All Lines button in the UI.	Version 2020 Release 2

For an example of the validateDelete entry point, see [SuiteScript Client Script Sample](#).

validateField(scriptContext)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Description	This function checks if a field change is valid when a user or client makes a change. This event works on fields added by beforeLoad user event scripts. You can use this function to: - Check field lengths. - Limit field entries to a specific format. - Limit submitted values to a certain range. - Check the submission against another related field. For an example, see SuiteScript Client Script Sample.  Note: Note that this doesn't work for list or box fields.
Returns	true if the field is valid and the change is successful. false otherwise.
Since	Version 2015 Release 2

Parameters

 **Note:** The scriptContext parameter is a JavaScript object. NetSuite automatically sends it to your script's entry point.

Parameter	Type	Description	Since
scriptContext.currentRecord	currentRecord.CurrentRecord	The current form record. For more information about CurrentRecord object members, see the help topic CurrentRecord Object Members .	Version 2015 Release 2
scriptContext.sublistId	string	The sublist ID name.	Version 2015 Release 2
scriptContext.fieldId	string	The field ID name.	Version 2015 Release 2

Parameter	Type	Description	Since
scriptContext.line	string	The line number (zero-based index) if the field is in a sublist or a matrix. If it's not in a sublist or matrix, this value is undefined.	Version 2015 Release 2
scriptContext.column	string	The column number (zero-based index) if the field is in a matrix. If it's not in a matrix, this value is undefined.	Version 2015 Release 2

For an example of the validateField entry point, see [SuiteScript Client Script Sample](#).

validateInsert(scriptContext)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Description	This function checks if a new line in an edit sublist is valid. For an example, see SuiteScript Client Script Sample .
Returns	true if the sublist line is valid and the insertion is successful. false otherwise.
Since	Version 2015 Release 2

Parameters

Note: The scriptContext parameter is a JavaScript object. NetSuite automatically sends it to your script's entry point.

Parameter	Type	Description	Since
scriptContext.currentRecord	currentRecord.CurrentRecord	The current form record. For more information about CurrentRecord object members, see the help topic CurrentRecord Object Members .	Version 2015 Release 2
scriptContext.sublistId	string	The sublist ID name.	Version 2015 Release 2

For an example of the validateInsert entry point, see [SuiteScript Client Script Sample](#).

validateLine(scriptContext)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Description	This function checks if a new line is valid before it's added to an inline editor sublist or editor sublist.
--------------------	--

	This event works like a <code>saveRecord</code> event for line items in these sublists. For an example, see SuiteScript Client Script Sample .
Returns	true if the sublist line is valid and the addition is successful. false otherwise.
Since	Version 2015 Release 2

Parameters

 **Note:** The `scriptContext` parameter is a JavaScript object. NetSuite automatically sends it to your script's entry point.

Parameter	Type	Description	Since
<code>scriptContext.currentRecord</code>	currentRecord.CurrentRecord	The current form record. For more information about <code>CurrentRecord</code> object members, see the help topic CurrentRecord Object Members .	Version 2015 Release 2
<code>scriptContext.sublistId</code>	string	The sublist ID name.	Version 2015 Release 2

For an example of the `validateLine` entry point, see [SuiteScript Client Script Sample](#).

SuiteScript 2.1 Custom Tool Script Type

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Custom tool scripts let you build NetSuite tools that external AI clients can invoke through the NetSuite AI Connector Service. With these scripts, you can retrieve data, trigger actions, or perform most SuiteScript-supported operations—all from natural language prompts in your AI client.

By using custom tools, your AI client can provide more accurate responses to prompts and perform specific actions for you in NetSuite. You can deploy custom tools to your NetSuite account only through the SuiteCloud Development Framework (SDF), either as a SuiteApp project or an account customization project (ACP). Each tool must be defined in a JSON Schema.

 **Important:** The `N/http` and `N/https` modules are not supported in custom tool scripts.

Read the following topics to learn how custom tool scripts work, what their technical requirements are, and how to implement them effectively in your NetSuite account.

- [SuiteScript 2.1 Custom Tool Script Type Reference](#)
 - [Custom Tool Script Requirements](#)
 - [Custom Tool JSON Schema](#)
 - [Custom Tool Script Best Practices](#)
 - [Custom Tool Script Impact on Other Customizations](#)

- [Custom Tool Script Error Messages](#)
- [SuiteScript 2.1 Custom Tool Script Type Code Samples](#)
- [Creating Custom Tools for the NetSuite AI Connector Service](#)
- [Custom Tool Scripts as XML Definitions](#)

NetSuite provides samples of basic tools to help you get started with the NetSuite AI Connector Service and the custom tool script type. You can find sample tool implementations in the [MCP-Sample-Tools](#) directory of the [SuiteCloud Project Repository on Oracle Samples GitHub](#).

SuiteScript 2.1 Custom Tool Script Type Reference

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Building a custom tool in NetSuite takes several key steps, from implementing your tool's actions to defining how the external AI client should interact with it. As you develop your custom tool scripts, it's important to understand the deployment requirements and be aware of any limitations that can affect your implementation. The following topics cover the essential information you'll need when creating a custom tool script:

- [Custom Tool Script Requirements](#)
- [Custom Tool JSON Schema](#)
- [Custom Tool Script Best Practices](#)
- [Custom Tool Script Impact on Other Customizations](#)
- [Custom Tool Script Error Messages](#)

Custom Tool Script Requirements

In NetSuite, creating your own tool requires the following:

- **Custom Tool Script File** – Contains the SuiteScript code for your tool. Each method in the script can be defined as a separate tool in the tool schema. For guidance on writing code and structuring your custom tool scripts effectively, see [Custom Tool Script Best Practices](#).



Important: The N/http and N/https modules are not supported in custom tool scripts.

- **Custom Tool JSON Schema** – A JSON file that defines all properties for the actions in your custom tool. For information about the properties defined in the tool schema, see [Custom Tool JSON Schema](#).
- **Tool SDF Object** – The SDF object XML definition file associated with the custom tool script. In SuiteCloud Development Framework (SDF), custom tools are defined as [tool SDF](#) custom objects. For more information, see the help topic [Custom Tool Scripts as XML Definitions](#).

For complete samples of each file, see [SuiteScript 2.1 Custom Tool Script Type Code Samples](#).

Custom Tool JSON Schema

The following table describes the properties defined in the tool schema.

Note: If you include a property that is not listed in the following table, it will be ignored.

Property	Type	Required or Optional	Description
name	string	Required	Method name as written in the script file.
description	string	Required	Explains the tool's purpose. The AI client uses this information to decide when to use the tool.
inputSchema	object	Required	Details of the input parameters
inputSchema.type	object	Required	The type of the input parameter.
inputSchema.properties	object	Required	Key-value pairs for parameter names and their properties. Each parameter has a type and description. Example: <pre> 1 { 2 "firstname": { "type": "string", "description": "First Name" }, 3 "lastname": { "type": "string", "description": "Last Name" } 4 }</pre>
inputSchema.required	array<string>	Required	Array of required parameters, as defined in inputSchema.properties.
inputSchema.nullable	array<string>	Required for ChatGPT	Array of nullable parameters, as defined in inputSchema.properties. This parameter is ChatGPT-specific.
outputSchema	object	Required for ChatGPT, optional for other AI clients	Defines the expected output structure. If specified, the tool returns a structured output.
outputSchema.type	object	Required for ChatGPT, optional for other AI clients	The type of the output parameter.
outputSchema.properties	object	Required	Key-value pairs for parameter names and their properties. Each parameter has a type and description.
outputSchema.required	array<string>	Required	Array of required parameters, as defined in outputSchema.properties.
outputSchema.nullable	array<string>	Required for ChatGPT	Array of nullable parameters, as defined in outputSchema.properties. This parameter is ChatGPT-specific.
annotations	object	Optional	Additional metadata for the tool.
annotations.title	string	Optional	Tool title. If not specified, the tool name will be used.
annotations.readOnlyHint	boolean	Optional	Default value is false. If not specified, the default value will be used.

Property	Type	Required or Optional	Description
			If true, the tool doesn't change its environment.
annotations.idempotentHint	boolean	Optional	Default value is false. If not specified, the default value will be used. If true, calling the tool repeatedly with the same arguments have no further effect on its environment. This property applies only when readOnlyHint is set to false.
annotations.openWorldHint	boolean	Optional	Default value is false. If not specified, the default value will be used. If true, the tool can interact with external entities beyond its predefined domain. If false, the tool's interactions are limited to its defined scope.
annotations.destructiveHint	boolean	Optional	Default value is true. If not specified, the default value will be used. If true, the tool can remove or overwrite existing information in its environment. If false, the tool only adds new information and doesn't remove or change existing data. This property applies only when readOnlyHint is set to false.

Custom Tool Script Best Practices

Following a few best practices can help you write custom tool scripts that are reliable and simple to maintain. The following list contains best practices for developing custom tools.

- **Use unique method names** – Give each method a unique name to avoid conflicts and ensure clear mapping between your tool schema and script. This helps AI clients and developers find the right functionality.
- **Ensure tool descriptions are accurate** – Write clear, precise descriptions for each tool. Accurate descriptions help AI clients and developers understand, maintain, and extend your tools.
- **Stringify objects before returning** – Convert objects to JSON strings before returning them. This ensures compatibility with AI clients and avoids serialization issues. This also gives you a predictable data format for downstream processing.
- **Avoid semantic collision between tools** – Ensure that no two tools perform similar or confusing actions. Overlap can cause unpredictable AI behavior and make maintenance harder.
- **Plan each operation for the AI-as-customer-agent model** – The tool user is an AI acting for a customer. Make operations safe, auditable, and suitable for automation—not requiring human judgment.
- **Exclude data from untrusted sources in output** – Only include information from trusted or verified sources in your tool's output. This keeps your data reliable, protects against security risks, and ensures that AI-driven actions are based on trusted information.

Custom Tool Script Impact on Other Customizations

When external AI clients run custom tools, some scripts and customizations in NetSuite may also be triggered. The following table shows how these components are affected during execution.

Customization	Impact
User Event Scripts Workflow Action Scripts	Run as the role configured for the NetSuite AI Connector Service.
Workflow	If not set to run as administrator, workflows are triggered as the role configured for the NetSuite AI Connector Service.

Custom Tool Script Error Messages

When working with custom tool scripts, you may encounter various error messages that can help identify and resolve issues during development or deployment. The following table provides common error messages along with their descriptions, so you can quickly understand what each message means and how it might affect your workflow.

Error Message	Description
Access denied.	The user's role does not have access to the File Cabinet folder where the tools are located. Verify that access to the File Cabinet folder is not restricted for the role. For more information, see the help topic Restricting Access to File Cabinet Folders .
Couldn't load modules in the tool implementation. Make sure your custom tool script doesn't use unsupported modules like N/http or N/https. If the issue persists, check your script for syntax errors or other problems.	Possible causes of this error include: - Using unsupported SuiteScript modules in the tool implementation - Providing the wrong path for a custom module - Syntax error or other problems in the tool implementation
Invalid call, the following required properties are missing: [parameter1, parameter2]")	Some required parameters are missing.
The script execution context creation failed due to the following issue: <error details>	A critical error occurred during preparation, so the tool couldn't run.
This tool is not allowed	The tool needs a permission that the current session doesn't have.
There was an unexpected error while executing the tool: <error details>	The tool runs but fails on a line or throws an exception.
Tool execution failed	The tool doesn't return a result properly.

SuiteScript 2.1 Custom Tool Script Type Code Samples

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

This sample consists of the three requirements for building a custom tool: a custom tool script, a JSON file for the schema, and the SuiteCloud Development Framework (SDF) object XML file. These files show how to structure your custom tool before deployment to your NetSuite account.

In this example, the custom tool script defines two tool methods: `add()` and `reverseText()`. The `add()` tool method takes two parameters and returns their sum. The `reverseText()` method takes a string and returns the reversed string.

- [Custom Tool Script that Adds Numbers and Reverses a String](#)
- [Custom Tool Definition Schema Samples](#)
- [Custom Tool SDF XML Object Sample](#)

You can also find sample tool implementations in the [MCP-Sample-Tools](#) directory of the [SuiteCloud Project Repository on Oracle Samples GitHub](#). This project includes SuiteScript-based sample tools for performing common business tasks through conversational AI. These examples demonstrate the types of AI-powered solutions that you can build and deploy to your own NetSuite account.

NetSuite also provides the MCP Standard Tools SuiteApp, which is available in the SuiteApp Marketplace. For more information, see the help topic [MCP Standard Tools SuiteApp](#).

Custom Tool Script that Adds Numbers and Reverses a String

This sample shows a custom tool script that implements two tools: one for adding two numbers, and another for reversing a string.

Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see the help topic [SuiteScript 2.x Global Objects](#).

Important: This sample uses SuiteScript 2.1. For more information, see [SuiteScript 2.1](#).

```

1  /**
2   * exampletools.js
3   * @NApiVersion 2.1
4   * @NModuleScope Public
5   * @NScriptType CustomTool
6   */
7
8  define([], function() {
9      return {
10         add: async function (args) {
11             try {
12                 const a = args["a"];
13                 const b = args["b"];
14                 return {
15                     result: a+b
16                 };
17             } catch(e) {
18                 return {
19                     result: "",
20                     error: "There was an error executin the tool: " + e
21                 }
22             }
23         },
24         reverseText: async function (args) {
25             try {
26                 const text = args["text"];
27                 return {
28                     result: text.split('').reverse().join('')
29                 };
30             } catch(e) {
31                 return {
32                     result: "",
33                     error: "There was an error executin the tool: " + e
34                 }
35             }
36         }
37     }
38 })

```

```

37 }
38 });

```

Custom Tool Definition Schema Samples

This sample shows the JSON schema definition for the tools defined in the custom tool script. For more information about the properties of the JSON schema, see [Custom Tool JSON Schema](#).

```

1 {
2   {
3     "tools": [
4       {
5         "name": "add",
6         "description": "Add two numbers together",
7         "inputSchema": {
8           "type": "object",
9           "properties": {
10            "a": { "type": "number", "description": "First addend" },
11            "b": { "type": "number", "description": "Second addend" }
12          },
13          "required": ["a", "b"]
14        },
15        "outputSchema": {
16          "type": "object",
17          "properties": {
18            "result": {
19              "type": "number",
20              "description": "Value of adding the two input numbers"
21            },
22            "error": {
23              "type": "string",
24              "description": "Error message if execution fails"
25            }
26          },
27          "required": ["result"]
28        },
29        "annotations": {
30          "title": "Add Numbers",
31          "readOnlyHint": true,
32          "idempotentHint": true,
33          "openWorldHint": false
34        }
35      },
36      {
37        "name": "reverseText",
38        "description": "Reverses a given text string. Example: if the user says 'Reverse the text: I like apples', call this
39        tool with { \"text\": \"I like apples\" }. ",
40        "inputSchema": {
41          "type": "object",
42          "properties": {
43            "text": { "type": "string", "description": "Text to reverse" }
44          },
45          "required": ["text"]
46        },
47        "outputSchema": {
48          "type": "object",
49          "properties": {
50            "result": {
51              "type": "string",
52              "description": "Value of reversing the input text value"
53            },
54            "error": {
55              "type": "string",
56              "description": "Error message if execution fails"
57            }
58          },
59          "required": ["result"]
60        },
61        "annotations": {
62          "title": "Reverse Text",

```

```

62         "readOnlyHint": true,
63         "idempotentHint": true,
64         "openWorldHint": false
65     }
66 }
67 ]
68 }

```

This sample shows the JSON schema definition for the tools defined in the custom tool script, without using outputSchema.

```

1  {
2      "tools": [
3          {
4              "name": "add",
5              "description": "Add two numbers together",
6              "inputSchema": {
7                  "type": "object",
8                  "properties": {
9                      "a": { "type": "number", "description": "First addend" },
10                     "b": { "type": "number", "description": "Second addend" }
11                 },
12                 "required": ["a", "b"]
13             },
14             "annotations": {
15                 "title": "Add Numbers",
16                 "readOnlyHint": true,
17                 "idempotentHint": true,
18                 "openWorldHint": false
19             }
20         },
21         {
22             "name": "reverseText",
23             "description": "Reverses a given text string. Example: if the user says 'Reverse the text: I like apples', call this
24             tool with { \"text\": \"I like apples\" }. ",
25             "inputSchema": {
26                 "type": "object",
27                 "properties": {
28                     "text": { "type": "string", "description": "Text to reverse" }
29                 },
30                 "required": ["text"]
31             },
32             "annotations": {
33                 "title": "Reverse Text",
34                 "readOnlyHint": true,
35                 "idempotentHint": true,
36                 "openWorldHint": false
37             }
38         }
39     ]
40 }

```

If you are using ChatGPT, you need to define which schema properties are nullable. The following sample is the JSON schema definition for the custom tools when you use ChatGPT.

```

1  {
2      "tools": [
3          {
4              "name": "add",
5              "description": "Add two numbers together",
6              "inputSchema": {
7                  "type": "object",
8                  "properties": {
9                      "a": { "type": "number", "description": "First addend" },
10                     "b": { "type": "number", "description": "Second addend" }
11                 },
12                 "required": ["a", "b"],
13                 "nullable": []
14             },
15             "outputSchema": {
16                 "type": "object",

```

```

17         "properties": {
18             "result": {
19                 "type": "number",
20                 "description": "Value of adding the two input numbers"
21             },
22             "error": {
23                 "type": "string",
24                 "description": "Error message if execution fails"
25             }
26         },
27         "required": ["result"],
28         "nullable": ["error"]
29     },
30     "annotations": {
31         "title": "Add Numbers",
32         "readOnlyHint": true,
33         "idempotentHint": true,
34         "openWorldHint": false
35     }
36 },
37 {
38     "name": "reverseText",
39     "description": "Reverses a given text string. Example: if the user says 'Reverse the text: I like apples', call this
40     tool with { \"text\": \"I like apples\" }. ",
41     "inputSchema": {
42         "type": "object",
43         "properties": {
44             "text": { "type": "string", "description": "Text to reverse" }
45         },
46         "required": ["text"],
47         "nullable": []
48     },
49     "outputSchema": {
50         "type": "object",
51         "properties": {
52             "result": {
53                 "type": "string",
54                 "description": "Value of reversing the input text value"
55             },
56             "error": {
57                 "type": "string",
58                 "description": "Error message if execution fails"
59             }
60         },
61         "required": ["result"],
62         "nullable": ["error"]
63     },
64     "annotations": {
65         "title": "Reverse Text",
66         "readOnlyHint": true,
67         "idempotentHint": true,
68         "openWorldHint": false
69     }
70 }
71 ]

```

Custom Tool SDF XML Object Sample

This sample shows the SDF XML definition for the custom tool. In SDF, custom tools are defined as a [tool](#) SDF custom object. For more information, see the help topic [Custom Tool Scripts as XML Definitions](#).

```

1 <tool scriptid="customtool_test1">
2   <name>Test Tool 1</name>
3   <scriptfile>[/SuiteApps/com.netsuite.tools/tools/exampletools.js]</scriptfile>
4   <rpcschema>[/SuiteApps/com.netsuite.tools/tools/exampletools_schema.json]</rpcschema>
5   <exposeto3rdpartyagents>T</exposeto3rdpartyagents>
6   <permissions>
7     <permission>
8       <permkey>LIST_EMAILTEMPLATE</permkey>
9       <permlevel>FULL</permlevel>

```

```

10     </permission>
11     <permission>
12         <permkey>ADMI_CRMLIST</permkey>
13         <permlevel>VIEW</permlevel>
14     </permission>
15 </permissions>
16 </tool>

```

SuiteScript 2.x Map/Reduce Script Type

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The map/reduce script type is designed for scripts that need to handle large amounts of data. It's best for situations where you can divide the data into small, independent parts. When the script is executed, a structured framework automatically creates enough jobs to process all of these parts. You do not have to manage this process; NetSuite does it all for you. Another advantage of map/reduce is that these jobs can work in parallel and you can choose the level of parallelism when you deploy the script.

A map/reduce script can be run manually or on a schedule, like a scheduled script. Map/reduce scripts have several advantages over scheduled scripts. One advantage is that, if a map/reduce job violates certain aspects of NetSuite governance, the map/reduce framework automatically causes the job to yield and its work to be rescheduled, without disruption to the script. However, be aware that some aspects of map/reduce governance can't be handled through automatic yielding. For that reason, if you use this script type, you should familiarize yourself with the [Map/Reduce Governance](#) guidelines.

In general, use a map/reduce script when you need to process multiple records and can break your logic into smaller pieces. In contrast, a map/reduce script isn't as well suited to situations where you want to enact a long, complex function for each part of your data set. A complex series of steps might include loading and saving multiple records.

You can use SuiteCloud Development Framework (SDF) to manage map/reduce scripts as part of file-based customization projects. For information about SDF, see [. You can use the Copy to Account feature to copy an individual map/reduce script to another of your accounts. Each map/reduce script page has a clickable Copy to Account option in the upper right corner. For information about Copy to Account, see the help topic \[Copy to Account\]\(#\).](#)

You can use SuiteScript Analysis to learn about when the script was installed and how it performed in the past. For more information, see the help topic [Analyzing Scripts](#).

For more information about map/reduce scripts, see the following topics:

- [Map/Reduce Script Use Cases](#)
- [SuiteScript 2.x Map/Reduce Script Reference](#)
- [SuiteScript 2.x Map/Reduce Script Entry Points and API](#)
- [SuiteScript 2.x Map/Reduce Script Code Samples](#)

Also see the [Map/Reduce Script Best Practices](#) section in the [SuiteScript Developer Guide](#) for a list of best practices to follow when using client scripts.

Map/Reduce Script Use Cases

Map/reduce scripts are perfect for applying the same logic to multiple objects, one at a time. For instance, you can use map/reduce scripts for tasks like:

- Identify a list of purchase requisitions and transform each one into a purchase order.
- Search for invoices that meet certain criteria and apply a discount to each one.
- Search for customer records that appear to be duplicates, then process each apparent duplicate according to your business rules.

- Search for outstanding tasks assigned to sales reps, then send each person an email that summarizes their outstanding work.
- Identify files in the NetSuite File Cabinet, use the content of the files to create new documents, and upload the new documents to an external server.

SuiteScript 2.x Map/Reduce Script Reference

📌 Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Map/reduce scripts are unique compared to other SuiteScript 2.x script types, with distinct terminology and execution.

- To learn essential concepts behind map/reduce scripts, see [Map/Reduce Key Concepts](#).
- To learn about terms specific to map/reduce scripts, see [Map/Reduce Terminology](#).
- To learn about how map/reduce scripts are executed in stages, see [Map/Reduce Script Stages](#).
- One major benefit of map/reduce scripts is their ability to manage some of their own resource usage. This is made possible by a feature called yielding. To learn more about yielding, see [Map/Reduce Yielding](#).
- You can submit a map/reduce script in several ways. To learn how to submit a map/reduce script, see [Map/Reduce Script Submission](#).
- Like all script types, map/reduce scripts have usage limits in NetSuite. To learn more about governance on map/reduce scripts, see [Map/Reduce Governance](#).
- The Map/Reduce Script Status Page shows you script status and instance details. For more information about the Map/Reduce Script Status Page, see [Map/Reduce Script Status Page](#).
- Like any SuiteScript 2.x script, you'll need to test and potentially troubleshoot your map/reduce script. For more information about testing and troubleshooting map/reduce scripts, see [Map/Reduce Script Testing and Troubleshooting](#).
- A map/reduce script can be interrupted by server issues or uncaught errors. To learn how to handle these errors, see [Map/Reduce Script Error Handling](#).
- To learn about best practices for working with map/reduce scripts, see the help topic [Map/Reduce Script Best Practices](#).

Map/Reduce Key Concepts

📌 Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Inspired by the map/reduce paradigm, the general idea behind a map/reduce script is as follows:

1. Your script identifies some data that requires processing.
2. This data is split into key-value pairs.
3. Your script defines a function that the system invokes one time for each key-value pair.
4. Optionally, your script can also use a second round of processing.

The system can create multiple jobs for each round of processing and process the data in parallel, depending on how you deploy the script.

If you've worked with other SuiteScript 2.x scripts, you'll find that map/reduce scripts work differently. Before you start writing a map/reduce script, make sure you understand the differences. Keep the following in mind:

- [Map/reduce scripts are executed in stages](#)
- [The system supplements your logic](#)
- [The system provides robust context objects](#)

- Multiple jobs are used to execute one script
- Map/reduce scripts permit yielding and other interruptions

Map/reduce scripts are executed in stages

Most script types run as a single continuous process. In contrast, map/reduce scripts run in five stages, in a specific order.

You can control the script's behavior in four of the five stages. That is, each of these four stages corresponds to an entry point. Your corresponding function defines the script's behavior during that stage. For example:

- For the **getInputData** stage, you write a function that returns an object that can be transformed into a list of key-value pairs. For example, if your function returns a search of NetSuite records, the system runs the search. The key-value pairs would be the results of the search where each key would be the internal ID of a record and each value would be a JSON representation of the record's field IDs and values.
- For the **map** stage, you can optionally write a function that the system invokes one time for each key-value pair. If needed, your map function can generate new key-value pairs as output. If the script also uses a reduce function, this output data is sent as input to the shuffle and then the reduce stage. Otherwise, the new key-value pairs are sent directly to the summarize stage.
- You do not write a function for the **shuffle** stage. In this stage, the system organizes key-value pairs for the reduce stage, if you've defined one. These pairs may have been provided by the map function, if you're using one. If a map function was not used, the shuffle stage uses data provided by the **getInputData** stage. The shuffle stage groups this data by key to form a new set of key-value pairs, where each key is unique and each value is an array.

For example, suppose there are 100 key-value pairs. Let's say each key is an employee, and each value is a record they created. If there were only two unique employees, and one employee created 90 records, while the other created 10, then the shuffle stage would provide two key-value pairs. The keys are the employee IDs. One value is an array with 90 elements, and the other has 10.
- For the **reduce** stage, you write a function that runs one time for each key-value pair from the shuffle stage. This function can also generate key-value pairs to send to the summarize stage.
- In the **summarize** stage, your function can get and log stats about the script's work. It can also act on data from the reduce stage.

Note that you can omit either the map or reduce function. You can also skip the summarize function. For more details, review [SuiteScript 2.x Map/Reduce Script Entry Points and API](#).

The system supplements your logic

Most script types rely solely on the code in your script file. Map/reduce scripts work differently. In a map/reduce script, your logic is key, but the system also adds its own standardized logic. For instance, the system handles data transfer between stages. The system also calls your map and reduce functions multiple times. For this reason, think of the logic of the map and reduce functions as being similar to the logic you'd use in a loop. Each of these functions should perform a relatively small amount of work. For details about the system's behavior during and between the stages, see [Map/Reduce Script Stages](#).

The system provides robust context objects

For each entry point function you write in your map/reduce script, the system provides a context object. The system provides context objects to most SuiteScript 2.x entry points, but map/reduce entry point functions are especially robust. These objects contain data and properties that are critical to writing an effective map/reduce script. For instance, you can use these objects to get data from the previous stage and send output to the next one. These objects can also hold error data, usage stats, and other metrics. For details, see [SuiteScript 2.x Map/Reduce Script Entry Points and API](#).

Multiple jobs are used to execute one script

Map/reduce scripts run using [SuiteCloud Processors](#), which handle work through a series of **jobs**. Each job is run by a **processor**, a virtual unit of processing power. SuiteCloud Processors also handle scheduled scripts. However, these two script types are handled differently. For instance, the system uses only one job for a scheduled script. In contrast, the system creates multiple jobs for a single map/reduce script. The system creates at least one job per stage. The system can also create multiple jobs for the map and reduce stages. When there are multiple map and reduce jobs, they work independently and can run in parallel across processors. For this reason, the map and reduce stages are considered **parallel** stages.

In contrast, the `getInputData` and `summarize` stages each use one job. In each case, that job calls your function only one time. These stages are **serial** stages. The shuffle stage is also a serial stage.

Map/reduce scripts permit yielding and other interruptions

The map and reduce stages can be easily split into multiple jobs because they consist of independent function invocations. The structure is naturally flexible. It enables parallel processing and lets map and reduce jobs manage their own resource usage to some extent.

If a job takes up a processor for too long, the system can finish it after the current function is done. In this case, the system creates a new job to continue executing remaining key-value pairs. The new job starts either right after the original job finishes or later, depending on its priority and when it was submitted, to let higher-priority jobs run. For more details, see [Map/Reduce Yielding](#).

Note that the system has usage limits for map/reduce scripts that aren't managed through yielding. For details, see [Map/Reduce Governance](#).

Map/Reduce Terminology

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Term	Definition	More Information
buffer size	A script deployment option that sets the number of key-value pairs a map or reduce job can process before saving progress to the database. As a general best practice, leave this value set to 1.	■ Buffer Size
context object	An object that provides information about the current execution context of a map/reduce script. Includes properties and methods for accessing and manipulating data.	■ SuiteScript 2.x Map/Reduce Script Entry Points and API
deployment instance	A task that processes a script deployment. Created when a deployment is submitted for processing. Only one unfinished instance of a particular script deployment can exist at any time.	■ Map/Reduce Script Submission
exitOnError	Controls script behavior when an error interrupts a map or reduce function. If <code>exitOnError</code> is true and retries are used up, the script exits the current stage and moves to the summarize stage.	■ exitOnError
function invocation	An execution of a function. Some functions, like <code>getInputData</code> and <code>summarize</code> , are only invoked one time. Others, like map and reduce, are invoked multiple times. For example, if the <code>getInputData</code> stage provides 20 key-value pairs to the map stage, then the map function is invoked 20 times, one time for each pair.	■ SuiteScript 2.x Map/Reduce Script Type

Term	Definition	More Information
getInputData	The first stage in the processing of a map/reduce script. In this stage, your script returns an object that becomes a list of key-value pairs. Use the getInputData entry point to specify the function for this stage.	■ Map/Reduce Script Stages
governance	A system of rules governing your usage of NetSuite. Each script type, including map/reduce, has its own limits.	■ Map/Reduce Governance ■ SuiteScript Governance and Limits
hard limit	A map/reduce limit that interrupts the function if exceeded. Make sure you follow the Map/Reduce Script Best Practices section in the SuiteScript Developer Guide to avoid problems with hard limits.	■ Map/Reduce Governance ■ Map/Reduce Script Best Practices
job	A unit of execution handled by SuiteCloud Processors . Map/reduce scripts are processed in multiple jobs. Each stage has at least one job.	■ Map/Reduce Script Stages ■ SuiteCloud Processors
map	The second stage of a map/reduce script. A map function is optional, but your script must use either a map or reduce function (or both). The map function processes data from the getInputData stage. The map function runs one time for each key-value pair.	■ Map/Reduce Script Stages ■ map(mapContext)
map/reduce	A computing paradigm designed for the processing of large data sets.	■ https://en.wikipedia.org/wiki/MapReduce
map/reduce script type	A SuiteScript 2.x script type that uses the map/reduce paradigm.	■ Map/Reduce Key Concepts
parallel stage	A stage that can be handled by multiple jobs at the same time. The map and reduce stages are parallel stages. See also serial stage.	■ Map/Reduce Script Stages
priority	A job property. Job priority determines the order in which the scheduler sends jobs to the processor pool. You can set priorities on the deployment record or on the SuiteCloud Processors Priority Settings Page .	■ Map/Reduce Script Deployment Record ■ SuiteCloud Processors Priority Levels
processor	In the context of SuiteCloud Processors, a virtual unit of processing power that executes a job. It's a single processing thread, not a physical entity.	■ Concurrency Limit ■ SuiteCloud Processors
processor pool	The total processing power available to your NetSuite account. The number of processors available varies depending on your licensing. The Concurrency Limit field on the script deployment record determines how many processors are used.	■ Concurrency Limit ■ SuiteCloud Processors
reduce	The fourth stage of a map/reduce script. A reduce function is optional, but your script must use either a map or reduce function (or both). When used, a reduce function processes data provided by the getInputData stage or, if a map function is used, by the map stage. The reduce function runs once for each unique key. Each unique key passed to the reduce function has an array of values.	■ Map/Reduce Script Stages ■ reduce(reduceContext)
retryCount	A setting that applies when a map or reduce function is interrupted by an error or server restart. It determines	■ retryCount

Term	Definition	More Information
	whether the system retries the function for uncertain key-value pairs.	
script deployment	Script execution settings. These settings are stored in a deployment record. To run a map/reduce script, create a deployment record and submit it.	■ Map/Reduce Script Submission
serial stage	A stage that's handled by a single job. Examples of serial stages include getInputData, shuffle, and summarize.	■ Map/Reduce Script Stages
shuffle	The third stage of a map/reduce script. This stage matters if you're using a reduce function. The shuffle stage sorts data from getInputData or the map stage. The shuffle stage creates unique key-value pairs with array values. These pairs are sent to the reduce stage for more processing.	■ Map/Reduce Script Stages
soft limit	A governance rule that makes a job yield when surpassed. When the job yields, its work is rescheduled. Soft limit progress is checked after a function finishes. Therefore this limit won't interrupt a function.	■ Map/Reduce Governance
SuiteCloud Processors	Handles map/reduce and scheduled scripts.	■ SuiteCloud Processors
summarize	The final stage of a map/reduce script. You can use this stage to log data about the work of the script.	■ Map/Reduce Script Stages ■ summarize(summaryContext)
task	The set of work that represents a map/reduce deployment that has been submitted for processing. Each task has at least five jobs, one per stage.	■ Map/Reduce Script Submission
yielding	A behavior that helps map/reduce scripts manage resources. When a job surpasses a soft limit, it yields and its work is rescheduled. Yielding won't interrupt a function. Note also that this process is automatic. SuiteScript 2.x does not contain an API that lets you force a script to yield.	■ Yield After Minutes

Map/Reduce Script Stages

📌 Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The map/reduce script type goes through at least two of five possible stages.

The stages are processed in the following order. Each stage must finish before the next one starts.

- **getInputData** – Gets a collection of data. This stage is always processed first and is required. The input stage runs sequentially.
- **map** – Parses each row of data into a key-value pair. One pair (key-value) is passed per function invocation. If you skip this stage, you need a reduce stage. Data can be processed simultaneously in this stage.
- **shuffle** – Groups values by keys. This happens automatically after the map stage. You can't access this stage directly as it's handled by the framework. Data is processed sequentially in this stage.
- **reduce** – Evaluates the data in each group. One group (key-values) is passed per function invocation. If you skip this stage, you need a map stage. Data can be processed simultaneously in this stage.

- **summarize** – Summarizes the previous stages' output. Use this stage to summarize the data and write it to a file or send an email. This stage is optional and not part of the main map/reduce process. The summarize stage runs sequentially.

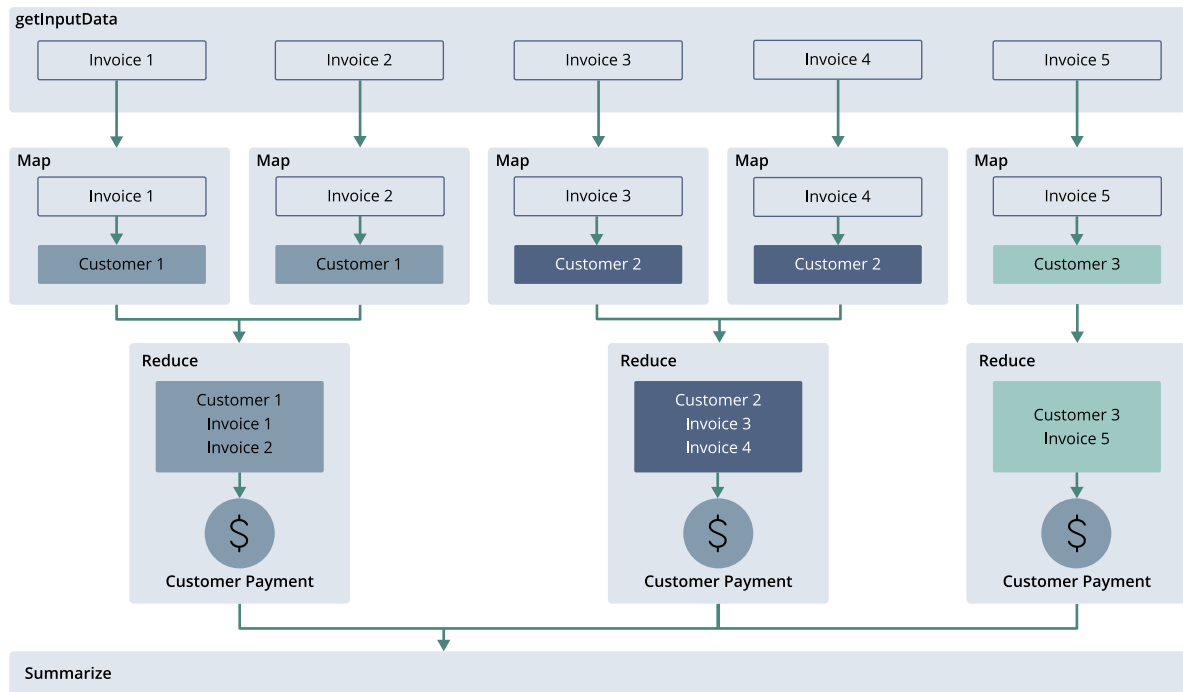
Note: You don't need to use both the map and reduce stages. You can skip one of them.

The following diagram illustrates these stages, in the context of processing a set of invoices.

Here's how the stages are used in this example:

- **getInputData** – Loads invoices that need payment.
- **map** – Pairs each invoice with the customer who should pay it. The output is key-value pairs with customerID as the key and the invoice as the value. The map function runs five times for five invoices.
- **reduce** – Groups invoices by customerID, with three unique groups. The reduce function runs three times for the three groups. To create a customer payment for every group, custom logic iterates over each group using customerID as the key.
- **summarize** – Custom logic gets metrics (like invoices paid) and sends them in an email

For a code sample similar to this example, see [Processing Invoices Example](#).



Passing Data to a Map or Reduce Stage

To avoid changing data accidentally, key-value pairs are converted to strings when passed between stages. For map/reduce scripts, SuiteScript 2.x checks if the data is a string and uses `JSON.stringify()` to convert it if needed.

JSON-serialized objects stay in JSON format. To avoid possible errors, SuiteScript does not automatically deserialize the data. For instance, trying to convert non-JSON data types like CSV or XML can cause errors. You can use `JSON.parse()` to convert the JSON string back to a JS object if you need to.

Map/reduce script keys (specifically, in `mapContext` or `reduceContext` objects) have a 3,000 character limit. In addition, error messages are returned when a key is longer than 3,000 characters or a value is

larger than 10 MB. Keys over 3,000 characters return a `KEY_LENGTH_IS_OVER_3000_BYTES` error. Values over 10 MB return a `VALUE_LENGTH_IS_OVER_10_MB` error.

If you have map/reduce scripts that use the `mapContext.write(options)` or `reduceContext.write(options)` methods, keep key strings under 3,000 characters and value strings under 10 MB. Consider the potential length of dynamically generated strings, which might exceed these limits. Also, try not to use keys instead of values to pass your data.

For more information about map/reduce limits see [Map/Reduce Governance](#).

Map/Reduce Yielding

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

One major benefit of map/reduce scripts is their ability to manage some of their own resource usage. This is made possible by a feature called yielding.

To understand yielding, note that map/reduce scripts are processed by [SuiteCloud Processors](#). A processor is a virtual unit of processing power that runs a job.

Yielding works by checking if a job has exceeded limits after each map or reduce function finishes. If it has, job gracefully ends its execution, making it possible for other jobs to gain access to the processor. A new job replaces the one that ended. The new job has the same priority as the old one, but a later timestamp.

Yielding happens when these limits are exceeded:

- The time limit set in the Yield After Minutes field on the script deployment record. You can set this value to any number between 3 and 60. The default is 60. For more details on this field, see [Yield After Minutes](#).
- The 10,000 usage unit limit per map or reduce job. For more details, see [Soft Limits on Long-Running Map and Reduce Jobs](#).

Since the system checks these limits between function calls, yielding won't interrupt a function. Additionally, be aware that a job does not yield until it has passed one of the relevant limits.

Yielding is separate from the limits for a single map or reduce function call. Those limits are described in [Hard Limits on Function Invocations](#). If you exceed the limit for a single function call, the system throws an `SSS_USAGE_LIMIT_EXCEEDED` error and stops the function, even if it's not finished.

For help understanding how the map and reduce stages can each have multiple function invocations, review [Map/Reduce Key Concepts](#).



Important: Yielding in map/reduce is automatic. You can't manually force a map/reduce job to yield using an API. This is different from how it worked in SuiteScript 1.0 for scheduled scripts.

Map/Reduce Script Submission

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

You can submit a map/reduce script in several ways:

- By [Scheduling a Map/Reduce Script Submission](#)
- By [Submitting an On-Demand Map/Reduce Script Deployment from the UI](#)
- By [Submitting an On-Demand Map/Reduce Script Deployment from a Script](#)

You can create multiple deployments for a single script record. This is helpful if you need to run the same script multiple times simultaneously or in a short period. For details, see [Submitting Multiple Deployments of the Same Script](#).

You'll need a map/reduce script deployment record for each process. For details about the fields available on this record, see [Map/Reduce Script Deployment Record](#).



Important: SuiteCloud Processors run all map/reduce script instances. Before submitting a map/reduce script for processing, review [SuiteCloud Processors](#).

Map/Reduce Script Deployment Record

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

You need to create a deployment record before running a map/reduce script.

Map/reduce script deployment records are similar to those for other script types. However, map/reduce deployments have some extra fields. Some of these fields are specific to [SuiteCloud Processors](#), which are used to run map/reduce scripts. Others are specific to map/reduce features. This topic covers all the available fields.

You can access a map/reduce deployment record in these ways:

- To edit an existing deployment record, go to Customization > Scripting > Script Deployments, find the record, and click **Edit**.
- To create a new deployment record, open the script record, and click **Deploy Script** button. For help creating a script record, see [Script Record Creation](#).

Body Fields

The following table summarizes the fields available on the map/reduce script deployment record. Note that some fields are available only when you edit or view an existing deployment record.

Field	Description
Script	A link to the script record associated with the deployment. This value cannot be changed, even on a new deployment record. If you begin the process of creating a deployment and realize that you selected the wrong script record, you must start the process over.
Title	The user-defined name for the deployment.
ID	A unique identifier for the deployment. You can customize the ID by entering a value in the ID field on a new record. Customize the ID if you plan to bundle the deployment to avoid naming conflicts. IDs must be lowercase and space-free. Use underscores to separate words. If you don't enter an ID, the system generates one. In both cases, the system automatically adds the prefix <code>customdeploy</code> to the ID created when the record is saved. You can change the ID on an existing deployment by clicking Change ID, but it's not preferred.
Deployed	A setting that shows whether the deployment is active. Check this box to run the script. Otherwise, the system uses the following behavior: ■ If the box is unchecked, and the Status is Not Scheduled, the Save and Execute option is no longer available, and you can't save and execute or submit it programmatically. ■ If the box is unchecked, and the deployment record's Status is Scheduled, the scheduled times are ignored and the deployment isn't submitted.
Status	A value that determines how and when a script deployment can be submitted for processing. The primary options are: ■ Scheduled —The script deployment is submitted for processing at the times indicated on the Schedule subtab.

Field	Description
	■ Not Scheduled — The script deployment is submitted for processing only when invoked manually, either through the UI or programmatically. Remember, the system only submits the deployment if the "Deployed" box is checked, regardless of the Status. For more details on this choice, see Status.
See Instances	A link to the Map/Reduce Script Status Page, filtered for all instances of this deployment record, for the current day. You can adjust the filters as needed. For details on working with the Map/Reduce Script Status page, see Map/Reduce Script Status Page.
Log Level	A value that determines what type of log messages are displayed on the Execution Log of both the deployment record and associated script record. The available levels are: ■ Debug — suitable for scripts still being tested; this level shows all debug, audit, error and emergency messages. ■ Use Audit, Error, or Emergency — suitable for scripts in production mode. Audit shows the most, Emergency shows the least. For more details on each level, see Log Level.
Execute As Role	The role used to run the script. For map/reduce, this is always set to Administrator and can't be changed.
Priority	A measure of how urgently this script should be processed relative to other map/reduce and scheduled scripts that have been submitted. This value applies to each job associated with the deployment. The priority affects when the SuiteCloud Processors sends jobs to the processor pool. For more details, see Priority.  Important: Understand SuiteCloud Processors before making changes. For details, see the help topic SuiteCloud Processors Priority Levels.
Concurrency Limit	Determines the number of SuiteCloud Processors that can be used to process the jobs associated with the script deployment. For more details on this field, see Concurrency Limit .
Submit All Stages At Once	Controls whether jobs are created for all map/reduce stages simultaneously. Only clear this field for low-priority script deployments. For more details on this field, see Submit All Stages At Once .
Yield After Minutes	A soft time limit on how long the script deployment's map and reduce jobs may run before yielding. Enter a value between 3 and 60. The system checks this time limit after each function invocation. If exceeded, the job yields to let other jobs run. A new job takes over the work from the yielded job. For more details on this field, see Yield After Minutes .
Buffer Size	A value that indicates how many key-value pairs a map or reduce job can process before information about the job's progress is saved to the database. A low Buffer Size reduces the risk of duplicate processing. Leave this value at 1 unless you have a specific reason to change it. For more details on this field, see Buffer Size .

Status

The Status field controls when a script deployment can be submitted. The default value is Not Scheduled.

Regardless of how the script deployment is submitted, it **does not necessarily execute** at the exact time scheduled, or at the exact time that it's manually invoked. There may be a short system delay, even if no scripts are before it. If there are scripts already waiting to be executed, the script may need to wait until others have completed. For details on this behavior, see the help topic [SuiteCloud Processors](#).

Scheduled

When a deployment's Status is set to **Scheduled**, the script runs on a one-time or recurring schedule. You set the schedule using the deployment record's [Schedule Subtab](#). Once you save, the deployment will be submitted automatically.

Note also:

- If you schedule a recurring submission with an end date, or a one-time submission, the status stays Scheduled even after the script finishes running.
- You can't run the script on-demand when it's Scheduled.

See also [Scheduling a Map/Reduce Script Submission](#).

Not Scheduled

When a deployment's Status is set to **Not Scheduled**, the deployment can be submitted on-demand. If you want the deployment to be submitted for processing, you must manually submit it, either through the NetSuite UI or programmatically. You can use:

- The **Save and Execute** option on the deployment record. See also [Submitting an On-Demand Map/Reduce Script Deployment from the UI](#).
- The `task.ScheduledScriptTask` API. See also [Submitting an On-Demand Map/Reduce Script Deployment from a Script](#).

You can only submit the script if there's no other instance already running. To run multiple instances at once, create multiple deployment records. For details, see [Submitting Multiple Deployments of the Same Script](#).

Testing

When a deployment's Status is set to **Testing**, only the script owner can test and debug without submitting for processing. You have several ways to test and debug your Map/Reduce script. For more information about Map/Reduce Script testing, see [Map/Reduce Script Testing and Troubleshooting](#).

Log Level

The Log Level field determines what type of log messages are displayed in the Execution Log.

For testing, use the **Debug** log level. This option includes more messages than the other log levels, including messages created by [log.debug\(options\)](#), [log.audit\(options\)](#), [log.error\(options\)](#), and [log.emergency\(options\)](#).

For production scripts, use one of these levels:

- **Audit** — This level includes a record of events that have occurred during the processing of the script (for example, "A request was made to an external site"). This level includes log messages created by [log.audit\(options\)](#), [log.error\(options\)](#), and [log.emergency\(options\)](#).
- **Error** — A log level set to Error shows only unexpected script errors, including log messages created by [log.error\(options\)](#) and [log.emergency\(options\)](#).
- **Emergency** — Includes only the most critical messages, including log messages created by [log.emergency\(options\)](#).

The default value is Debug.

Priority

When multiple scripts are submitted at the same time, some might need to wait. Use the **Priority** field to manage script processing. The Priority field determines how quickly scripts are processed relative to others submitted at the same time. The deployment's priority applies to all its jobs.

You have the following options:

- **High** — For critical deployments that need immediate processing. The scheduler sends high-priority jobs to the processor pool first.
- **Standard** — This is the default setting. This is a medium priority level. Medium-priority jobs are sent to the processor pool if there are no high-priority jobs waiting.
- **Low** — For deployments that can wait longer. Low-priority jobs are sent to the processor pool if there are no high- or medium-priority jobs waiting.



Important: You must understand [SuiteCloud Processors](#) before you change this setting. See the help topic [SuiteCloud Processors Priority Levels](#).

Concurrency Limit

The map/reduce script type allows parallel processing. With parallel processing, multiple SuiteCloud Processors can work together to run a single script instance. You can control the number of processors used for each script instance using the Concurrency Limit field on the script deployment record.



Note: This setting only applies to the map and reduce stages. Only these stages allow parallel processing.

For example, if you specify a concurrency limit of 5, the system creates five map jobs and five reduce jobs. If you don't set a limit, the system uses the maximum number of processors available to your account. The default value is 2. For more information, see the help topic [SuiteCloud Processors Processor Allotment Per Account](#).

When using a SuiteCloud project to modify the concurrency limit, SDF automatically adjusts it if the value exceeds the target account's limit. For example, if you set a concurrency limit of 10 and deploy to an account with a limit of 5, the Concurrency Limit field is set to 5. The XML representation of the map/reduce script remains at the original value of 10. For more information, see the help topic [Setting a Concurrency Limit on Your Map/Reduce Script Deployment in SDF](#).



Note: The Concurrency Limit field was introduced in 2017.2, as part of the SuiteCloud Processors feature. If you are editing a deployment record that was created prior to 2017.2, be aware that when your account was upgraded, the Concurrency Value field was initially set to a value that corresponds to the number of queues that had been saved for the Queues field.

Submit All Stages At Once

Each map/reduce script deployment instance is processed by multiple jobs. At least one job is created for each stage. Every map/reduce script must use either four or five stages: getInputData, shuffle, summarize, and either map or reduce (or both). However, the jobs for each stage aren't necessarily submitted at the same time. This behavior is controlled by the **Submit All Stages at Once** option.

Map/reduce stages must happen in a specific order. When Submit All Stages at Once is disabled, the system waits for each stage's prerequisite job to complete before submitting the next job.

In contrast, when Submit All Stages at Once is enabled, the system submits all stage jobs at the same time. This behavior increases the likelihood that all jobs associated with the script deployment instance finish, without gaps, before another script begins executing. However, be aware that this option **does not guarantee** that no gaps occur. For example, because a map/reduce job can yield, a long-running job may be forced to end, and a job associated with another script may begin executing in its place. Don't rely on this option if you need a strict script execution order. To enforce a strict sequence, have one script schedule another during the summarize stage. You can schedule a script programmatically using the [task.create\(options\)](#) method. For more details, see [Submitting an On-Demand Map/Reduce Script Deployment from a Script](#).

The Submit All Stages at Once option is enabled by default. In general, you should leave this option enabled.

Yield After Minutes

The Yield After Minutes field prevents long-running map or reduce jobs from taking over a processor.

Here's how it works: During map and reduce stages, the system checks the job's runtime after each function call. If the amount of elapsed time has surpassed the number of minutes identified in the Yield After Minutes field, the job gracefully ends its execution, and a new job is created to take its place. The new job has the same priority, but a later timestamp. This is called yielding.

Yield After Minutes defaults to 60, but you can set it between 3 and 60.

The system never interrupts a function invocation for this limit. The system only ends a job after the limit has been exceeded. For that reason, the degree to which the limit is surpassed varies depending on the duration of your function invocation. For example, if the Yield After Minutes limit is 3 minutes, but your function takes 15 minutes to complete, then in practice the job yields after 15 minutes, not 3 minutes.

Yielding is also affected by a governance limit. This limit is 10,000 usage units for each map and reduce job. This limit works similarly to the Yield After Minutes limit: The system waits until after each function invocation ends to determine whether the usage-unit limit has been surpassed. If it has, the job yields, even if the Yield After Minutes limit has not been exceeded.

See also [Map/Reduce Yielding](#) and [Soft Limits on Long-Running Map and Reduce Jobs](#).

Buffer Size

The **Buffer Size** controls how often a map or reduce job saves progress data. Generally, leave this field set to 1.

To understand this, let's review how map and reduce jobs work: The job first flags key-value pairs that need processing. Then it processes the flagged pairs. Then it saves data about the work done. This data includes details like processed pairs, usage points consumed, and more. This process repeats until the job yields or all pairs are processed.

The Buffer Size field determines how many pairs are flagged at one time. So if you leave this field set to its default of 1, the job flags one pair, processes it, and saves the data. Then it repeats the cycle.

You can set Buffer Size to any of the following values: 1, 2, 4, 8, 16, 32, or 64. However, a higher number increases the risk of processing key-value pairs twice if the job is interrupted by an application server restart. On the other hand, a higher number can be more efficient in certain situations.

Use the following guidance:

- Generally, leave it set to 1, especially when processing records.
- Choose a higher buffer size if the script performs fast algorithmic operations or if special circumstances require it.

Schedule Subtab

The following table summarizes the fields on the Schedule subtab. These settings are honored only if the deployment record's [Status](#) is set to Scheduled and the Deployed box is checked.

Field	Description
Single Event	The map/reduce script deployment is submitted only one time. Use this option if you want to schedule a future one-time submission.
Daily Event	The map/reduce script deployment is submitted every x number of days. If you schedule the submission to recur every x minutes or hours, the schedule starts over on the next scheduled day. For example, your deployment is set to submit daily, starting at 3:00 am and recurring every five hours. A scheduled script instance is submitted at 3:00 am, 8:00 am, 1:00 pm, 6:00 pm, and 11:00 pm. The schedule resets at midnight, with the next submission at 3:00 am.

Field	Description
Weekly Event	The map/reduce script deployment is submitted at least one time per scheduled week. If you schedule the submission to recur every x minutes or hours, the schedule starts over on the next scheduled day. For example, a deployment set to submit on Tuesday and Wednesday, starting at 3:00 am and recurring every 5 hours. The deployment is submitted on Tuesday at 3:00 am, 8:00 am, 1:00 pm, 6:00 pm, and 11:00 pm. On Wednesday, the schedule starts over and the next submission is at 3:00 am.
Monthly Event	The map/reduce script deployment is submitted at least one time per month.
Yearly Event	The scheduled script deployment is submitted at least one time per year.
Start Date	The first submission occurs on this date. This field is required if a one-time or recurring schedule is set.
Start Time	If you enter a Start Time, the first run starts then. When you save the deployment, NetSuite saves the Start Date and Start Time in your time zone. When another user opens the record, NetSuite shows them in their preferred time zone. Example: ■ User A (GMT +8, Manila) saves the deployment: □ Start Date = May 5 2025 □ Start Time = 8:00 a.m. ■ User B (GMT -5, Eastern) opens the record and sees: □ Start Date = May 4 2025 □ Start Time = 8:00 p.m. ■ User B clicks Save, and NetSuite stores the start values in GMT #5—regardless of whether the schedule changed. ■ When either user opens the record again, NetSuite shows the same moment in each user's time zone. User A still sees May 5 2025 8:00 a.m. (GMT +8), while User B sees May 4 2025 8:00 p.m. (GMT -5).  Note: NetSuite automatically applies time-zone rules, including Daylight Saving Time, based on the Start Date. For details on setting your time zone, see Time Zone in General Personal Preferences.
Repeat	If a value is selected, the first submission happens on that date and time. A new instance is created and submitted every x minutes or hours until the end of the start date. If applicable, the schedule resets on the next scheduled day. For example, your deployment is set to submit on Tuesday and Wednesday, starting at 3:00 pm and recurring every five hours. Submissions occur on Tuesday at 3:00 pm and 8:00 pm. On Wednesday, the schedule resets, and the next submission is at 3:00 pm.
End By	If a value is entered, the last submission ends by this date. If you schedule the submission to recur every x minutes or hours, a new script deployment instance is created and submitted every x minutes or hours until the end date.
No End Date	The schedule has no set end date.

Scheduling a Map/Reduce Script Submission

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

You can schedule map/reduce script deployments for processing. For example, you could configure the following schedules:

- A one-time submission, at a predefined time.
- Repeated submissions daily, weekly, monthly, or yearly.

To schedule a submission, set the Status field to **Scheduled**. Additionally, you must configure one or more upcoming times on the record's Schedule subtab.

You can schedule deployments every 15 minutes. For example, you could configure a script to run at 2:00 pm, 2:15 pm, 2:30 pm, and so on.

The Schedule subtab sets when the script deployment is submitted for processing. However, **the times you set on the Schedule subtab are not necessarily the times the script will execute**. Script deployment does not mean the script will execute precisely at 2:00 pm, 2:15 pm, 2:30 pm, and so on. There might be a short delay before the script runs, even if no other scripts are ahead of it. If other scripts are waiting, this script will wait until they're done. For more details about how map/reduce scripts are processed, see the help topic [SuiteCloud Processors](#).

The system only submits the deployment if the Deployed box is checked, regardless of the Status. To stop a map/reduce script, uncheck the Deployed box. For more details on this choice, see [Status](#).

 **Note:** The times displayed on the Schedule subtab use the session time zone, meaning that the times are displayed using the time zone of the user who is currently logged in. This time zone can be different from the time zone that is set for the company or the time zone of the map/reduce script owner.

To schedule a one-time or recurring map/reduce script submission:

1. Create your map/reduce script entry point script. This involves uploading a JavaScript file and creating a script record from it. To review a map/reduce script example, see [SuiteScript 2.x Map/Reduce Script Type](#). If this script is your first, see [SuiteScript 2.x Hello World](#) and [SuiteScript 2.x Entry Point Script Creation and Deployment](#).
2. Open the appropriate script record in view mode. Click the **Deploy Script** button.
3. When the script deployment record loads, check the **Deployed** box, if it's not already checked.
4. Set the **Status** field to **Scheduled**.
5. Set the remaining body fields. For help understanding the fields, see [Map/Reduce Script Deployment Record](#).
6. On the [Schedule Subtab](#), set the deployment options.

For example, if you wanted to **submit the script hourly**, you would configure the subtab as follows:

- Deployed = checked
- Daily Event = [radio button enabled]
- Repeat every 1 day
- Start Date = [today's date]
- **Start Time = 12:00 am**
- Repeat = every hour
- End By = [blank]
- No End Date = checked
- Status = Scheduled
- Log Level = Error

If the **Start Time** is set to any other time than 12:00 am (for example, it's set to 2:00 pm), the script will start at 2:00 pm, but then finish its hourly execution at 12:00 am. It will not resume until the next day at 2:00 pm.

7. Click **Save**.

Note: In some cases, you may want to submit a map/reduce script for processing multiple times simultaneously, or within a short time frame. However, the system won't let you submit a script deployment if a previous instance is already submitted and not finished. To solve this, create multiple deployments for the script. In other words, repeat Steps 2 through 7 of the procedure above as needed.

Submitting an On-Demand Map/Reduce Script Deployment from the UI

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

You can submit map/reduce scripts for processing on demand from the NetSuite UI. To submit a script in this way, use the **Save and Execute** command on the Script Deployment page. The **Status** field on the deployment must be set to **Not Scheduled**.

You can also submit an on-demand deployment programmatically. For details, see [Submitting an On-Demand Map/Reduce Script Deployment from a Script](#).

Note that after you submit an on-demand deployment of a script, the script **does not necessarily execute right away**. There might be a short delay before the script runs, even if no other scripts are ahead of it. If other scripts are waiting, this script will wait until they're done. For more details about how map/reduce scripts are processed, see the help topic [SuiteCloud Processors](#).

The system only submits the deployment if the Deployed box is checked, regardless of the Status. To stop a map/reduce script, uncheck the Deployed box. For more details on this choice, see [Status](#).

To submit an on-demand map/reduce script for processing from the UI:

1. Create your map/reduce script entry point script. This involves uploading a JavaScript file and creating a script record from it. To review a map/reduce script example, see [SuiteScript 2.x Map/Reduce Script Type](#). If this script is your first, see [SuiteScript 2.x Hello World](#) and [SuiteScript 2.x Entry Point Script Creation and Deployment](#).
2. Open the appropriate script record in view mode. Click the **Deploy Script** button.
3. When the script deployment record loads, check the **Deployed** box, if it's not already checked.
4. Set the Status field to **Not Scheduled**.
5. Set the remaining body fields. For help understanding the fields, see [Map/Reduce Script Deployment Record](#).
6. Select **Save and Execute** from the **Save** dropdown list.

Note: In some cases, you may want to submit a map/reduce script for processing multiple times simultaneously, or within a short time frame. However, the system won't let you submit a script deployment if a previous instance is already submitted and not finished. To solve this, create multiple deployments for the script. In other words, repeat Steps 2 through 6 of the procedure above as needed.

Submitting an On-Demand Map/Reduce Script Deployment from a Script

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

You can submit map/reduce scripts on demand from another server script type. You can submit a deployment this way by using the [task.MapReduceScriptTask](#). For the call to be successful, the **Status** field on the script deployment record must be set to **Not Scheduled**.

You can also submit an on-demand deployment from the UI. For details, see [Submitting an On-Demand Map/Reduce Script Deployment from the UI](#).

Note that after you submit an on-demand deployment of a script, the script **does not necessarily execute right away**. There might be a short delay before the script runs, even if no other scripts are ahead of it. If other scripts are waiting, this script will wait until they're done. For more details about how map/reduce scripts are processed, see the help topic [SuiteCloud Processors](#).

This method will ad-hoc start the Map/Reduce deployment with the identity/role being inherited from the script that called the method. To execute a map/reduce script from a server script, you need the Administrator role or a role with SuiteScript and SuiteScript Scheduling permissions. For more information, see the help topic [Permissions Documentation](#).

To submit an on-demand map/reduce script instance from a script:

1. Create your map/reduce script entry point script. This involves uploading a JavaScript file and creating a script record from it. To review a map/reduce script example, see [SuiteScript 2.x Map/Reduce Script Type](#). If this script is your first, see [SuiteScript 2.x Hello World](#) and [SuiteScript 2.x Entry Point Script Creation and Deployment](#).
2. Open the appropriate script record in view mode. Click the **Deploy Script** button.
3. When the script deployment record loads, check the **Deployed** box, if it's not already checked.
4. Set the Status field to **Not Scheduled**.
5. Set the remaining body fields. For help understanding the fields, see [Map/Reduce Script Deployment Record](#).
6. Click **Save**.
7. In the server script where you want to submit the map/reduce script, call `task.create(options)` to return a `task.MapReduceScriptTask` object:

```
1 | var scriptTask = task.create({taskType: task.TaskType.MAP_REDUCE});
```

8. Set the `MapReduceScriptTask.scriptId` and `MapReduceScriptTask.deploymentId` properties:

```
1 | scriptTask.scriptId = 'customscript1';
2 | scriptTask.deploymentId = 'customdeploy1';
```

The deployment ID property is optional when creating the map/reduce script task object. If you don't specify a deployment ID, the system will find and use an available deployment record for the script ID. For a deployment to be considered available, three conditions must be met: the deployment record must have a status of Not Scheduled, its Deployed option must be set to true, and no unfinished instances of the deployment can exist.

9. Call `MapReduceScriptTask.submit()` to submit the script for processing. For example:

```
1 | scriptTask.submit();
```

Note: In some cases, you may want to submit a map/reduce script for processing multiple times simultaneously, or within a short time frame. However, the system won't let you submit a script deployment if a previous instance is already submitted and not finished. To solve this, create multiple deployments for the script. In other words, repeat Steps 2 through 9 of the procedure above as needed. If you are using this approach, consider omitting the `deploymentId` property when you create the map/reduce script task object. When you omit the `deploymentId`, the system searches for and uses whichever deployment record is available.

Submitting Multiple Deployments of the Same Script

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The system does not permit you to submit a single script deployment for processing multiple times simultaneously. If a deployment is already submitted and not finished, you can't submit it again. You have to wait for the unfinished instance to finish.

To submit multiple instances of a script at the same time, create multiple deployments and submit each one as needed.

This approach has several benefits. For instance, you can process multiple script instances in parallel. Additionally, if you need a map/reduce script to be processed multiple times in the same general time frame, you can submit both instances at the same time without having to worry about the second submission failing with an error. The alternative is to monitor the first instance and submit the second one after it finishes.

Note that if a deployment has already been submitted but not yet finished, and you open the deployment record for editing in the UI, the deployment record's Save and Execute option isn't available. If you try to submit it programmatically, you'll get a `MAP_REDUCE_ALREADY_RUNNING` error.

For help creating or submitting a deployment, see the following topics:

- [Scheduling a Map/Reduce Script Submission](#)
- [Submitting an On-Demand Map/Reduce Script Deployment from the UI](#)
- [Submitting an On-Demand Map/Reduce Script Deployment from a Script](#)

Map/Reduce Governance

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

As with all script types, NetSuite imposes usage limits on map/reduce scripts. Map/reduce script governance rules have two main categories:

- Some limits interrupt the current function invocation if exceeded. These limits are known as **hard limits**.
- Other limits are managed automatically by the system. These limits don't interrupt function invocations. Instead, they can make a job yield and reschedule its work after the function invocation is complete. These limits are known as **soft limits**.

See the following sections for more details:

- [Hard Limits on Total Persisted Data](#)
- [Hard Limits on Function Invocations](#)
- [Soft Limits on Long-Running Map and Reduce Jobs](#)

Note that the system doesn't limit the total duration of a map/reduce script deployment instance. Users can't set a limit on this either. The system's limits regulate specific parts of the deployment, like a single function invocation's duration.

Note: One way that NetSuite measures a script's activity is through usage units. For more information about usage units, see the help topic [SuiteScript Governance and Limits](#).

Hard Limits on Total Persisted Data

A map/reduce script can't use more than 200MB of persisted data at any time. If you exceed this limit, the system throws a `PERSISTED_DATA_LIMIT_FOR_MAPREDUCE_SCRIPT_EXCEEDED` error. The script also ends its current function invocation, exits the current stage, and moves to the summarize stage. (This error does not occur in the summarize stage, because the total persisted data cannot be increased during that stage.)

Persisted data is calculated by adding up the following:

- Total size of all keys and values not yet mapped
- Total size of all keys and values not yet reduced
- Total size of all keys and values written as results in reduce

After processing, keys and values aren't counted toward the total storage size. If you have data that is preserved for internal usage, such as troubleshooting or analytics, the data will not be counted toward the total user-facing storage size enforced by the script.

The system takes into account any search results retrieved and returned by the input function. A large number of columns in a result set can increase the data usage significantly.

During the map and reduce stages, the total size is a measure of the keys and values yet to be processed. After processing, keys and values don't count toward the limit.

Hard Limits on Function Invocations

The table below shows the limits for map/reduce script function invocations. If you exceed these limits, the system throws an `SSS_USAGE_LIMIT_EXCEEDED` error. The system's response to this error depends on the stage and script configuration, as shown in the table.

Stage	Limits per function invocation	Response to <code>SSS_USAGE_LIMIT_EXCEEDED</code> error
getInputData	■ 10,000 units of API usage ■ 60 minutes of time ■ 1B of instructions	The script ends the function invocation and exits the stage. It proceeds directly to the summarize stage.
map	■ 1,000 units of API usage ■ 5 minutes of time ■ 100M of instructions	The response has two parts. You can configure the second part: 1. The function invocation ends, even if it's not finished with the current key-value pair. 2. Other jobs in Processing status are normally finished and their executions are not interrupted, but jobs in Pending status are canceled immediately. You can configure the script to retry the function for the same key-value pair. For details, see Configuration Options for Handling Map/Reduce Interruptions.
reduce	■ 5,000 units of API usage ■ 15 minutes of time ■ 100M of instructions	

Stage	Limits per function invocation	Response to SSS_USAGE_LIMIT_EXCEEDED error
summarize	10,000 units of API usage 60 minutes of time 1B of instructions	The script stops executing.

Script governance applies to each script invocation, not the entire execution.

As shown in the table above, a single execution of the **map** function should not:

- Use more than 1,000 usage points (same as Mass Update script)
- Run for more than 5 minutes
- Exceed the instruction count limit for User Event scripts

A single execution of the **reduce** function should not:

- Use more than 5,000 usage points
- Run for more than 15 minutes
- Exceed the instruction count limit for User Event scripts

A single execution of the **getInputData** or **summarize** function should not:

- Use more than 10,000 usage points
- Run for more than 60 minutes
- Exceed the instruction count limit for Scheduled scripts

If you're using map/reduce scripts as intended, you shouldn't come close to these limits, especially for map and reduce functions. In general, each invocation of a map or reduce function should do a relatively small portion of work. For more details, see the [Map/Reduce Script Best Practices](#) section in the [SuiteScript Developer Guide](#).

Soft Limits on Long-Running Map and Reduce Jobs

In addition to the limits described in [Hard Limits on Function Invocations](#), the system also has a soft limit of 10,000 usage units per map and reduce job.

To understand this limit, note that all map/reduce scripts are processed by [SuiteCloud Processors](#). A processor is a virtual unit of processing power that runs a job.

The 10,000-unit soft limit is a mechanism designed to prevent any processor from being monopolized by a long-running map or reduce job. During the map and reduce stages, after each function invocation, the system checks the total number of units that have been used by the job. If the total usage has surpassed 10,000 units, the job gracefully ends its execution and a new job is created to take its place. The new job has the same priority, but a later timestamp. This is called yielding.

Yielding is also affected by the script deployment record's Yield After Minutes field. This time limit works similarly to the 10,000-unit limit: The system checks the time limit after each function invocation ends. If the time limit is exceeded, the job yields, even if the 10,000-unit limit hasn't been reached. Yield After Minutes defaults to 60, but you can set it between 3 and 60. For more details, see [Yield After Minutes](#). See also [Map/Reduce Yielding](#).

If your map/reduce script triggers other operations, like scripts or workflows, they don't affect the initial time and usage limits. For example, if your map/reduce script triggers multiple user event scripts, the execution time of the user event scripts will not count toward the initial map/reduce time limit that is set. In these instances, the real execution time may exceed the soft limit of 60 minutes, but it won't count against script execution time.

Per enqueue limits are soft limits that are in queue at any particular stage of the script instance. The script checks per enqueue limits after each function invocation, but won't stop execution when the limit is reached. When the script exceeds the soft limit, it yields control of the queue after the current execution finishes.



Important: Yielding is separate from the limits for single map and reduce function invocations. Those limits are explained in [Hard Limits on Function Invocations](#). Exceeding these limits throws an SSS_USAGE_LIMIT_EXCEEDED error and ends the function invocation, even if it's not finished.

Character Limits on Map and Reduce Jobs

The character limit for keys in map/reduce scripts (specifically, in mapContext or reduceContext objects) is 3,000 characters. You'll get error messages if keys exceed 3,000 characters or values are over 10 MB. Keys over 3,000 characters return a KEY_LENGTH_IS_OVER_3000_BYTES error. Values over 10 MB return a VALUE_LENGTH_IS_OVER_10_MB error.

If you have map/reduce scripts that use the mapContext.write(options) or reduceContext.write(options) methods, make sure that key strings are shorter than 3,000 characters and value strings are smaller than 10 MB. Consider the potential length of dynamically generated strings, as they may exceed these limits. Avoid using keys to pass data; use values instead.

Map/Reduce Script Status Page

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The Map/Reduce Script Status Page lets you view script status, instance details, and retrieve details programmatically.

Viewing Map/Reduce Script Status

You can monitor map/reduce script execution from the UI's script status page. The script status page shows whether a deployment is pending, in progress, or can't be completed.

If all tasks are pending, you can cancel the deployment from this page.

From NetSuite, go to Customization > Scripting > Map/Reduce Script Status.

To help you understand and optimize the performance of script entry points used, you can drill down for more details about map stages, processing utilization, and timing. This information helps you understand how long stages or tasks take.

From the script status page, you can also view the deployment record and consider adjusting the concurrency limit.

Viewing Details of Script Instances

A script instance is a scheduled script or map/reduce script task submitted for processing. This script instance can also be called a task.

To view the details of a map/reduce script instance, from the Map/Reduce Script Status page, click **View Details**. Each row shows info about a map/reduce job in the script instance. A job always belongs to one of the stages of a map/reduce script, and there is at least one job per stage. There's exactly one job for each of the getInputData, shuffle, and summarize stages. The number of map and reduce jobs depends on concurrency limits and yields.

For more information, see [Map/Reduce Script Stages](#) and [Map/Reduce Yielding](#).



Important: Script execution details are deleted after 30 days. Note that the statuses on the Map/Reduce Script Status page are different from those on the Script Deployment page. The statuses on Script Deployment pages (Not Scheduled, Scheduled, Testing) indicate the type of deployment. Map/Reduce Script Status page statuses show the script's execution progress.

Programmatically Retrieving Script Instance Details

To get the script status using the [N/search Module](#), create and load a search using `scheduledscriptinstance` as the type argument.

To get the status using the [N/task Module](#), see the help topic [task.MapReduceScriptTask](#).

Map/Reduce Script Testing and Troubleshooting

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

You can test and debug your map/reduce script in several ways:

- Make a separate unit test suite as you develop and modify your map/reduce script. See [Map/Reduce Script Unit Testing](#).
- Check the execution logs. See the help topic [N/log Module](#).
- Monitor the script status. See [Map/Reduce Script Status Page](#) and [Programmatically Retrieving Script Instance Details](#).
- Detect any server restarts that interrupted map/reduce script execution. See [inputContext.isRestarted](#), [mapContext.isRestarted](#), [reduceContext.isRestarted](#), and [summaryContext.isRestarted](#).

The SuiteScript Debugger can't be used for deployed map/reduce script debugging. However, you may want to test any dependencies on other types of scripts. To test existing scripts, set the Script Deployment Status to Testing and ensure the logged-in user is the script owner. For information about the SuiteScript Debugger, see the help topic [SuiteScript Debugger](#).

Map/Reduce Script Unit Testing



Note: You cannot use the SuiteScript Debugger for deployed debugging of a map/reduce script type.

To test a map/reduce script on demand, split it into entry point sections. Use these sections to create unit tests. Each section should function as an entry point script that can be executed without external dependencies. If passing in modules, make sure that you use the `require` function and absolute paths.

To test map or reduce stages, you will need to create a mock context that seeds values and provides the dependent objects and parameters. Then, use assertion statements to check the states, behavior, inputs, and outputs. Note that the `getInputData` stage does not take parameters, so it will not require mock context and can be tested more conventionally.

To test a summarize stage, if it contains logic operating on a final set of data that has no return, use assertions and logs to gather information.

To assist development of your unit test, download the SuiteScript 2.x API files, specifically those representing the [mapContext](#), [reduceContext](#), and [summaryContext](#) objects. These files serve as a schema for testing or mocking properties and methods. To access the files, follow these steps:

1. From NetSuite, select Documents > Files > File Cabinet.
2. Select SuiteScript 2.0 API to download the zip folder.
3. Extract the `mapReduceContext` and `mapReduceSummary` .js files.

Map/Reduce Script Error Handling

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

A map/reduce script can be interrupted by:

- An application-server disruption, such as a NetSuite update, maintenance, or execution environment failure
- An uncaught error

Interruptions can leave parts of the script's work unfinished. The system has measures to handle this issue.

After an **application-server disruption**, the system automatically restarts the script, regardless of which stage was in progress when the failure occurred. For instance, if the failure occurred during the map or reduce stage, the map or reduce job restarts. In this case, the default behavior is that the restarted job invokes the map or reduce function again for all key-value pairs that were previously flagged for processing but not yet marked complete. You can modify this behavior using the [retryCount](#) option.

After an **uncaught error** is thrown, the behavior varies depending on the stage. If an error occurs during `getInputData`, the script moves to the `summarize` stage and doesn't restart `getInputData`. If an uncaught error is thrown during the map or reduce stage, the default behavior is that the current function invocation ends, even if incomplete, and the map or reduce job moves on to other key-value pairs that require processing. You can configure the script to re-run the map or reduce function for the pair that caused the error. You manage the system's behavior in this case by using the [retryCount](#) and [exitOnError](#) options.

To fully understand the system response to interruptions, see [System Response After a Map/Reduce Interruption](#).

In general, your script should include logic that checks to see whether a restart has occurred. If the script has been restarted, it should prevent duplicate processing. For more information, see [Adding Logic to Handle Map/Reduce Restarts](#).

For additional explanation of how each stage responds to a restart, see [Execution of Restarted Map/Reduce Stages](#).

System Response After a Map/Reduce Interruption

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

A map/reduce script can be interrupted at any time. For example, an application server disruption immediately stops the script. An uncaught error also stops the current function invocation, even if it's incomplete.

For more details, review the following sections:

- [System Response After an Application-Server Disruption](#)
- [System Response After an Uncaught Error](#)



Important: Regardless of the script configuration, include logic to check for restarts. If the script has been restarted, prevent duplicate processing. For details, see [Adding Logic to Handle Map/Reduce Restarts](#).

System Response After an Application-Server Disruption

An application disruption can happen due to a NetSuite update, maintenance, or execution environment failure. After the application server restarts, the script restarts, resuming the same stage it was in when interrupted.

When an application server restart interrupts the map or reduce stage, the system writes the `SSS_APP_SERVER_RESTART` error code to the relevant iterators. This error code is displayed with any uncaught error codes that were thrown.

For more details, see the following table.

Stage where interruption occurred	Script behavior	SSS_APP_SERVER_RESTART error code written to
getInputData	The script stops executing when disrupted. The system restarts the function after the application server restarts.	—
map	The script stops executing when disrupted. Any data that was saved during the previous invocation by using the <code>context.write()</code> method is discarded. Afterward, the response is as follows: 1. The system evaluates the <code>retryCount</code> config setting. If <code>retryCount</code> is set to a value greater than 0 or if the <code>retryCount</code> setting isn't used, the script tries to process the same set of key-value pairs it was processing when the application server became unavailable. This data includes all pairs that were flagged for processing but not marked complete. However, if <code>retryCount</code> is set to 0, the script moves on to Step 2 without attempting further processing for these key-value pairs. 2. The job moves on to other key-value pairs that require processing and were not previously flagged as in progress.	■ <code>mapContext.errors</code> — Contains the error codes recorded during previous attempts to process the current key-value pair. ■ <code>mapSummary.errors</code> — Contains all error codes recorded during the map stage.
reduce		■ <code>reduceContext.errors</code> — Contains the error codes recorded during previous attempts to process the current key-value pair. ■ <code>reduceSummary.errors</code> — Contains all error codes recorded during the reduce stage.
summarize	When the disruption occurs, the entire script stops executing. After the application server restarts, the system restarts the function.	—

System Response After an Uncaught Error

An uncaught error doesn't necessarily stop the script, but it can disrupt its work. Some of this behavior can be configured. For details, see the following table.

Stage where error occurred	Script behavior	Errors written to
getInputData	The script ends the function invocation and exits the stage. It goes directly to the summarize stage. This behavior cannot be configured.	<code>inputSummary.error</code>
map	The function invocation ends when an error occurs, even if incomplete. Any data that was saved during the invocation by using the <code>context.write()</code> method is discarded. Afterward, the system responds as follows: 1. The system evaluates the <code>retryCount</code> setting. If <code>retryCount</code> is set to a value	■ <code>mapContext.errors</code> — Contains the error codes recorded during previous attempts to process the current key-value pair. ■ <code>mapSummary.errors</code> — Contains all error codes recorded during the map stage.

Stage where error occurred	Script behavior	Errors written to
reduce	greater than 0, and the maximum number of retries has not yet been used, the script tries to process the same key-value pair again. The rest of the steps in this process are not used. - The system evaluates the <code>exitOnError</code> setting. If <code>exitOnError</code> is true, the script exits the stage and goes to summarize. The rest of the steps in this process are not used. - The job continues with other key-value pairs that need processing. It does not do any further work on the pair it was processing when the error occurred.	<code>reduceContext.errors</code> — Contains the error codes recorded during previous attempts to process the current key-value pair. <code>reduceSummary.errors</code> — Contains all error codes recorded during the reduce stage.
summarize	The script stops executing. This behavior cannot be configured.	—

Note: This table describes the behavior for the majority of errors, but a few errors result in different behavior. For example, if one of the jobs being processed in the map or reduce stage fails on `SSS_USAGE_LIMIT_EXCEEDED`, other jobs in Processing status are normally finished and their executions are not interrupted, but jobs in Pending status are canceled immediately. For details, see [Hard Limits on Total Persisted Data](#).

Configuration Options for Handling Map/Reduce Interruptions

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

A map/reduce script can be interrupted at any time. If the application server is disrupted, it'll immediately stop the script's execution. Additionally, an uncaught error, although it does not cause the script to stop executing, stops the current function invocation, even if it's not complete.

The system has two configuration options to fine-tune your script's response to interruptions. For details, see the following sections:

- `retryCount`
- `exitOnError`
- [Adding a Configuration Option to a Script](#)

retryCount

The `retryCount` option only applies to map and reduce stages. This option lets you configure your script to restart the map or reduce function for any key/pairs that were left in an uncertain state following an interruption, including application server restarts and uncaught errors.

The effect of this setting varies slightly depending on whether the script was disrupted by application server restart or an uncaught error. For example:

- When an **application server restart** occurs, the script cannot identify the exact key-value pair that was being processed when the interruption occurred. However, the script can identify the pairs that had been flagged for processing but were not yet marked as complete. In the event of a retry, the script retries processing for all of these pairs.

- When an **uncaught error** occurs, the script can identify the exact key-value pair that was being processed when the interruption occurred. When you're using the `retryCount` option, the script invokes the map or reduce function again for that specific key-value pair.

Valid values are 0 to 3. The number you choose for the `retryCount` setting is the number of retries permitted for **each** key-value pair that was left in an uncertain state after an interruption. For example, suppose you have `retryCount` set to 2, and an error interrupts a map job. In this case, the script would restart the function for the key-value pair that was being processed when the error was thrown. If the second function invocation for that pair was also interrupted by an error, the script would invoke the function for that key-value pair another time. However, if an error was thrown during the third attempt, the script would not retry processing again, because the `retryCount` setting permits only two retries. When the job moves on to a new key-value pair, two retries are available again.

The `retryCount` setting is optional. You can set a value for it in your map/reduce script's return statement, as described in [Adding a Configuration Option to a Script](#).

If you don't use `retryCount`, the behavior differs depending on whether the script was disrupted by an error or an application server restart. For example:

- When `retryCount` isn't used and an application server restart interrupts the script, the system **restarts** processing for all key-value pairs that were left in an uncertain state.
- When `retryCount` isn't used and an error interrupts the map or reduce function, the system **does not restart** processing for the key-value pair that was left in an uncertain state.

See also [System Response After an Uncaught Error](#) and [System Response After an Application-Server Disruption](#).



Note: When an uncaught error occurs, the system's response also depends on the `exitOnError` option's value. However, the script evaluates and reacts to the `retryCount` setting first.

exitOnError

`exitOnError` is used when an uncaught error occurs in the map or reduce stage. It's evaluated after the `retryCount` option.

When `exitOnError` is set to true, the script exits the stage after both of the following occur:

- An uncaught error occurs.
- All retries permitted by the `retryCount` option have been used.

This setting has no impact on the system's behavior after an application server restart. It's used only when an uncaught error is thrown.

The `exitOnError` setting is optional. You can set its value by adding it to the return statement in your map/reduce script, as described in [Adding a Configuration Option to a Script](#). Possible values are:

- **false** — The script continues processing in the current stage. (This is also the behavior used when the option isn't added to the script.)
- **true** — The script exits the stage and goes immediately to the summarize stage.

See also [System Response After an Uncaught Error](#).

Adding a Configuration Option to a Script

If you want to use the `retryCount` or `exitOnError` option, add them to the script's return statement in a config block. For example:

```
1 // Add additional code.
2 ...
3
4 return {
```



```

5
6     config:{
7         retryCount: 3,
8         exitOnError: true
9     },
10
11     getInputData: myGetInputData,
12     map: myMap,
13     reduce: myReduce,
14     summarize: mySummarize
15
16 };
17
18 ...
19 // Add additional code.

```

Logging Errors

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

With any map/reduce script, you should include logic to check for errors that might happen during the `getInputData`, `map`, and `reduce` stages. You can access data about errors using the context objects made available to each entry point. Use the properties shown in the following table.

Stage where error occurred	Property that contains data
<code>getInputData</code>	<code>inputSummary.error</code>
<code>map</code>	■ <code>reduceContext.errors</code> — Contains the error codes recorded during previous attempts to process the current key-value pair. ■ <code>mapSummary.errors</code> — Contains all error codes recorded during the map stage.
<code>reduce</code>	■ <code>reduceContext.errors</code> — Contains the error codes recorded during previous attempts to process the current key-value pair. ■ <code>reduceSummary.errors</code> — Contains all error codes recorded during the reduce stage.

Syntax

The following snippets shows how you can capture data about errors in various stages.

Map Stage

You can use this snippet in a map function. The snippet only logs data only if an error was thrown during a previous run of the map function for the same key-value pair currently being processed.

```

1 // Create a log entry showing each full serialized error thrown
2 // during previous attempts to process the current key-value pair.
3
4 mapContext.errors.iterator().each(function (key, error, executionNo){
5
6     log.error({
7         title: 'Map error for key: ' + key + ', execution no ' + executionNo,
8         details: error
9     });
10
11     return true;
12 });

```

Reduce Stage

You can use this snippet in a reduce function. The snippet only logs data if an error was thrown during a previous run of the reduce function for the same key-value pair currently being processed.

```

1 // Create a log entry showing each full serialized error thrown
2 // during previous attempts to process the current key-value pair.
3
4 reduceContext.errors.iterator().each(function (key, error, executionNo){
5     log.error({
6         title: 'Reduce error for key: ' + key + ', execution no ' + executionNo,
7         details: error
8     });
9     return true;
10 });

```

Summarize Stage

You can use these snippets in a summarize function. They log data about errors thrown during previous stages.

```

1 // If an error was thrown during the input stage, log the error.
2
3 if (summary.inputSummary.error)
4 {
5     log.error({
6         title: 'Input Error',
7         details: summary.inputSummary.error
8     });
9
10
11 // For each error thrown during the map stage, log the error, the corresponding key,
12 // and the execution number. The execution number indicates whether the error was
13 // thrown during the first attempt to process the key, or during a
14 // subsequent attempt.
15
16 summary.mapSummary.errors.iterator().each(function (key, error, executionNo){
17     log.error({
18         title: 'Map error for key: ' + key + ', execution no. ' + executionNo,
19         details: error
20     });
21     return true;
22 });
23
24
25 // For each error thrown during the reduce stage, log the error, the corresponding
26 // key, and the execution number. The execution number indicates whether the error was
27 // thrown during the first attempt to process the key, or during a
28 // subsequent attempt.
29
30 summary.reduceSummary.errors.iterator().each(function (key, error, executionNo){
31     log.error({
32         title: 'Reduce error for key: ' + key + ', execution no. ' + executionNo,
33         details: error
34     });
35     return true;
36 });

```

Execution of Restarted Map/Reduce Stages

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

A map/reduce script can have many jobs. The input, shuffle, and summarize stages each use one job, but multiple jobs can run in the map and reduce stages. Jobs can run in parallel within the map stage or reduce stages. Any of the jobs can be forcefully ended at any time. The impact depends on what the job was doing and which stage it was running in.

For details, see the following topics:

- [Termination of getInput Stage](#)
- [Termination of Shuffle Stage](#)

- Termination of Parallel Stages
- Termination of Summarize Stage

Termination of getInput Stage

The work of a serial stage (getInput, shuffle, and summarize stages) is done in a single job. If the getInput stage job is forcefully terminated, it's later restarted. The getInput part of the script can check if it's a restart by examining the `isRestarted` attribute of the context argument (`inputContext.isRestarted`). The script is being restarted if and only if (`context.isRestarted === true`).

The next stage gets its input from the return value of the getInput script, which is saved after the getInput stage finishes. Even a restarted getInput script should return the same data. The map/reduce framework makes sure no data is written twice.

However, if the getInput script changes other data (like creating NetSuite records), it should have code to handle duplicate processing. The script needs idempotent operations to ensure that these records are not created twice, if this is undesired.

Termination of Shuffle Stage

The shuffle stage doesn't have any custom code, so if its job is forcefully ended, it's restarted later and all the work gets redone. There is no impact other than that the stage takes longer to finish.

Termination of Parallel Stages

Map and reduce stages can run jobs in parallel, so they're considered parallel stages. An application restart affects parallel stages the same way. The following example covers what happens if a map stage restarts. A reduce stage will behave the same way.

The map stage's job is to run a map function on each key-value pair from the previous stage (getInput). Multiple jobs participate in the map stage. Map jobs claim key-value pairs (or a set number of them) that haven't had the map function run yet. The job flags these key-value pairs so no other job runs the map function on them. Then, the job sequentially executes the map function on the key-value pairs it flagged. The map stage finishes when all pairs have been processed.

There's no limit to how many jobs can be in the map stage, but concurrency is limited. Initially, the number of map jobs is equal to the selected concurrency in the corresponding map/reduce script deployment. However, to prevent a single map/reduce task from monopolizing all computational resources in the account, each map job can yield itself to allow other jobs to execute. The yield creates an additional map job and the number of yields is unlimited.



Note: This is a different type of yield compared to yield in a SuiteScript 1.0 scheduled script. In SuiteScript 1.0, the yield happens in the middle of a script execution. In a map job, the yield can happen only between two map function executions, and not in the middle of one.

If a map job is forcefully terminated, it's later restarted. First, the job executes the map function on all key-value pairs that it took and did not mark finished before termination. It's the only map job that can run the map function on those pairs—they can't be taken by another map job. After processing those pairs, the map job continues as usual, taking other unfinished pairs and running the map function on them.

In some cases, the map function can run again on multiple key-value pairs. The number of pairs the map function might run again depends on the buffer size you select on the deployment page. The buffer size determines the number of key-value pairs originally taken in a batch. The job marks the batch as finished only when the map function is executed on all of them. Therefore, if the map job is forcefully terminated in the middle of the batch, the entire batch will be processed from the beginning when the map job is restarted.

Note that the map/reduce framework deletes all key-value pairs written from a partially executed batch so they aren't written twice. Therefore, the map function does not need to check whether `mapContext.write(options)` for a particular key-value has already been executed. However, if the map

function changes other data, it must also use idempotent operations. For example, if a map function creates NetSuite records, the script should check that these records aren't created twice if that's not what you want.

To check if a map function execution is a part of a restarted batch, the script must examine the `isRestarted` attribute in the context argument ([mapContext.isRestarted](#)). The map function is in the restarted batch if and only if (`context.isRestarted === true`).

Be aware that a restarted value of true only means some part of the script might have already run. Even if `context.isRestarted === true`, a map function could run on a particular key-value for the first time. For example, the map job could be stopped after picking up a key-value pair but before running the map function on it. This is more likely if a high buffer value is set in the map/reduce deployment.

Termination of Summarize Stage

If the summarize stage job is forcefully ended, it's restarted later. The summarize part of the script can check if it's a restart by looking at the `isRestarted` attribute of the summary argument ([summaryContext.isRestarted](#)).

The script is being restarted if and only if (`summary.isRestarted === true`).

Adding Logic to Handle Map/Reduce Restarts

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Sometimes a script can fail because of an application server restart, like during a NetSuite update, maintenance, or an unexpected environment failure.

Restarts can forcefully stop an application at any time, so your scripts should handle restarts and recover from unexpected interruptions.

In a map/reduce script, each restarted piece of the script will automatically delete any internal map/reduce data that this piece created (for example, the key-value pairs that drive the execution of an entire mapping task). However, you must develop your own code to handle any parts of the script that modify additional data (for example, creation of NetSuite records like sales orders), which is *never* automatically deleted.

See the following topics to learn more about how restarts and map/reduce script execution:

- [Example 3: Design of a Robust Map/Reduce Script Example](#)
- [Example 4: A Problematic Map/Reduce Script Example](#)
- [Example 5: A Robust Map/Reduce Script Example](#)
- [Execution of Restarted Map/Reduce Stages](#)

i Note: A map or reduce function can also be restarted if interrupted by an uncaught error. For the script to restart in this situation, you must use the [retryCount](#) option. For additional details, see [System Response After an Uncaught Error](#).

Example 3: Design of a Robust Map/Reduce Script Example

The following script is designed to detect restarts at specific stages in processing and includes logic to run if a restart happens.

Consider this example as a basic template, where the comment `// I might do something differently` denotes implementation of a special function for each stage, to ensure that the script can repeat itself with the same result. Or, to run a recovery task, such as removing duplicate records.

The script includes a check on the `isRestarted` property for each entry point function. If the value of `isRestarted` is true, the example script shows a placeholder for invoking a function. This is meant as a placeholder where you can add logic to protect against restarts and data errors.

For more information about an interrupted map/reduce stage, see [Execution of Restarted Map/Reduce Stages](#).

```

1  /**
2   * @ApiVersion 2.x
3   * @NScriptType MapReduceScript
4   */
5  define([], function(){
6      return {
7          getInputData: function (context)
8          {
9              if (context.isRestarted)
10             {
11                 // I might do something differently
12             }
13             .
14             .
15             .
16             return inputForNextStage;
17         },
18         map: function (context)
19         {
20             if (context.isRestarted)
21             {
22                 // I might do something differently
23             }
24             .
25             .
26             .
27         },
28         reduce: function (context)
29         {
30             if (context.isRestarted)
31             {
32                 // I might do something differently
33             }
34             .
35             .
36             .
37         },
38         summarize: function (summary)
39         {
40             if (summary.isRestarted)
41             {
42                 // I might do something differently
43             }
44             .
45             .
46             .
47         }
48     }
49 });

```

Example 4: A Problematic Map/Reduce Script Example

This script is meant to perform a search and process the results, but it isn't fully prepared for an unexpected restart. The script still needs logic to help avoid unrecoverable states and the creation of bad or duplicate data if it's re-executed.

In Example 4, if the script is forcefully interrupted during the map stage, some sales orders might be updated twice when the map function is re-executed. See [Example 5: A Robust Map/Reduce Script Example](#) for an improved example.

Note that the other stages in this script don't need improvement for handling a restart. If the getInput stage runs again, the map/reduce framework makes sure each result is sent to the map stage only one time. In this script, the getInput stage doesn't change other data, so no special restart logic is needed. The reduce and summarize stages also don't change extra data—they only process internal map/reduce data.

```

1  /**
2   * @ApiVersion 2.x
3   * @NScriptType MapReduceScript
4   */
5  define(['N/search', 'N/record'],
6    function(search, record){
7      return {
8        getInputData: function (context)
9        {
10           var filter1 = search.createFilter({
11             name: 'mainline',
12             operator: search.Operator.IS,
13             values: true
14           });
15           var column1 = search.createColumn({name: 'recordtype'});
16           var srch = search.create({
17             type: search.Type.SALES_ORDER,
18             filters: [filter1],
19             columns: [column1]
20           });
21           return srch;
22         },
23         map: function (context)
24         {
25           var soEntry = JSON.parse(context.value);
26           var so = record.load({
27             type: soEntry.values.recordtype,
28             id: context.key
29           });
30           // UPDATE so FIELDS
31           so.save();
32
33           context.write({
34             key: soEntry.values.recordtype,
35             value: context.key
36           });
37         },
38         reduce: function (context)
39         {
40           context.write({
41             key: context.key,
42             value: context.values.length
43           });
44         },
45         summarize: function (summary)
46         {
47           var totalRecordsUpdated = 0;
48           summary.output.iterator().each(function (key, value)
49           {
50             log.audit({
51               title: key + ' records updated',
52               details: value
53             });
54             totalRecordsUpdated += parseInt(value);
55             return true;
56           });
57           log.audit({
58             title: 'Total records updated',
59             details: totalRecordsUpdated
60           });
61         }
62       }
63     }
64   });

```

Example 5: A Robust Map/Reduce Script Example

Comparing Example 5 to Example 4, a filter was added to the search in the getInput stage. The purpose is to filter out already processed sales orders. The filter makes it possible to re-execute the whole map/

reduce task repeatedly, because when the whole task is re-executed, the additional filter ensures that only unprocessed sales orders will be returned from the input stage and not all sales orders.

There are also substantial improvements to the map function. In Example 5, if the `((context.isRestarted === false))` condition is met, the script knows it's the first execution of the map function for the current key-value pair. It won't need to perform any additional checks and can go directly to the sales order record update.

When updating the sales order, the script sets the `custbody_processed_flag` flag and only checks this flag when needed. If `(context.isRestarted === true)`, it checks the processed flag and updates the order only if it hasn't already been updated.

The script has more checks and lookups than Example 4, but processing is still light. It uses a lookup method that doesn't load the full record, and if the processed flag is true, the record isn't loaded again.

In the map function, note that the `context.write(...)` statement isn't inside the if-statement because when the map function for a pair is restarted, any writes from the previous run are deleted. There's no need to check which writes have or haven't been done.

The reduce function hasn't changed from Example 4. Since the reduce stage only handles map/reduce internal data, the framework makes sure that even if the reduce function restarts, only the most recent writes for each pair go on to the next stage.

The summarize function didn't need any changes either, but it's a good practice to log any restarts. For example, you may see the "Total records updated" entry twice in the log for a single map/reduce run.

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType MapReduceScript
4   */
5  define(['N/search', 'N/record'],
6    function(search, record){
7      return {
8        getInputData: function (context)
9        {
10           var filter1 = search.createFilter({
11             name: 'mainline',
12             operator: search.Operator.IS,
13             values: true
14           });
15           var filter2 = search.createFilter({
16             name: 'custbody_processed_flag',
17             operator: search.Operator.IS,
18             values: false
19           });
20           var column1 = search.createColumn({name: 'recordtype'});
21           var srch = search.create({
22             type: search.Type.SALES_ORDER,
23             filters: [filter1, filter2],
24             columns: [column1]
25           });
26           return srch;
27         },
28         map: function (context)
29         {
30           var soEntry = JSON.parse(context.value);
31           var alreadyProcessed = false;
32           if (context.isRestarted)
33           {
34             var lookupResult = search.lookupFields({
35               type: soEntry.values.recordtype,
36               id: context.key,
37               columns: ['custbody_processed_flag']
38             });
39             alreadyProcessed = lookupResult.custbody_processed_flag;
40           }
41           if (!alreadyProcessed)
42           {

```

```

43     var so = record.load({
44         type: soEntry.values.recordtype,
45         id: context.key
46     });
47     // UPDATE so FIELDS
48     so.setValue({
49         fieldId: 'custbody_processed_flag',
50         value: true
51     });
52     so.save();
53 }
54
55 context.write({
56     key: soEntry.values.recordtype,
57     value: context.key
58 });
59
60 },
61 reduce: function (context)
62 {
63     context.write({
64         key: context.key,
65         value: context.values.length
66     });
67
68 },
69 summarize: function (summary)
70 {
71     if (summary.isRestarted)
72     {
73         log.audit({details: 'Summary stage is being restarted!'});
74     }
75     var totalRecordsUpdated = 0;
76     summary.output.iterator().each(function (key, value)
77     {
78         log.audit({
79             title: key + ' records updated',
80             details: value
81         });
82         totalRecordsUpdated += parseInt(value);
83         return true;
84     });
85     log.audit({
86         title: 'Total records updated',
87         details: totalRecordsUpdated
88     });
89 }
90 }
91 });

```

SuiteScript 2.x Map/Reduce Script Entry Points and API

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Map/Reduce Script Entry Points

The map/reduce script type has four entry points. These let you control the script's behavior and data flow within the map/reduce stages. For an overview of the stages, see [Map/Reduce Script Stages](#).

Entry Point	Context Object	Required/Optional	Description
getInputData(inputContext)	inputContext	Required	Marks the beginning of the map/reduce script execution and invokes the input stage. This function is invoked one time in the execution of the script.

Entry Point	Context Object	Required/Optional	Description
map(mapContext)	mapContext	Optional, but if this entry point is skipped, the reduce(reduceContext) entry point is required.	Invokes the map stage. If you use this entry point, the map function is invoked one time for each key-value pair provided by the getInputData(inputContext) function.
reduce(reduceContext)	reduceContext	Optional, but if this entry point is skipped, the map(mapContext) entry point is required.	Invokes the reduce stage. If you use this entry point, the reduce function is invoked one time for each key and list of values provided. Data comes from the map stage or, if that's not used, from the getInputData stage.
summarize(summaryContext)	summaryContext	Optional	Invokes the summarize stage. If you use this entry point, the summarize function is invoked one time in the execution of the script.

Map/Reduce Script API

The following tables describe properties that are available through the map/reduce entry points.

inputContext Object Members

The following members are called on [inputContext](#).

Member Type	Name	Return Type / Value Type	Description
Property	inputContext.isRestarted	boolean	Indicates whether the current invocation of the getInputData(inputContext) function represents a restart.
Object	inputContext.ObjectRef	object	And object that contains the input data.

The following members are called on [inputContext.ObjectRef](#).

Member Type	Name	Return Type / Value Type	Description
Property	ObjectRef.id	string number	The internal ID or script ID of the object. For example, this value could be a saved search ID.
	ObjectRef.type	string	The object's type.

mapContext Object Members

The following members are called on [mapContext](#).

Member Type	Name	Return Type / Value Type	Description
Property	mapContext.isRestarted	boolean	Indicates whether the current invocation of the map(mapContext) function is a restart. If isRestarted is true, that means the function was already called for this key-value pair but didn't finish successfully.

Member Type	Name	Return Type / Value Type	Description
	mapContext.executionNo	property	Indicates whether the current run of the map(mapContext) function is the first or a later attempt to process the key-value pair.
	mapContext.errors	iterator	Holds serialized errors that were thrown during previous attempts to run the map(mapContext) function on the current key-value pair.
	mapContext.key	string	The key to be processed during the current invocation of the map(mapContext) function.
	mapContext.value	string	The key that's being processed in this run of the map(mapContext) function.
Method	mapContext.write(options)	void	Writes the map(mapContext) output as key-value pairs. This data goes to the reduce stage (if it's used), or straight to the summarize stage.

reduceContext Object Members

The following members are called on [reduceContext](#).

Member Type	Name	Return Type / Value Type	Description
Property	reduceContext.isRestarted	boolean	Indicates whether the current invocation of the reduce(reduceContext) function is a restart. If isRestarted is true, the function was called before for this key-value pair but didn't finish successfully.
	reduceContext.executionNo	number	Indicates whether the current run of the reduce(reduceContext) function is the first or a later attempt to process the key-value pair.
	reduceContext.errors	iterator	Holds serialized errors that were thrown during previous attempts to run the reduce(reduceContext) function on the current key and its associated values.
	reduceContext.key	string	The input key to process during the reduce stage.
	reduceContext.values	string[]	The key being processed in this run of the reduce(reduceContext) function.
Method	reduceContext.write(options)	void	Writes the reduce(reduceContext) function as key-value pairs. This data goes to the summarize stage.

summaryContext Object Members

The following members are called on the [summaryContext](#).

Member Type	Name	Value Type	Description
Property	summaryContext.isRestarted	boolean (read-only)	Indicates whether the current invocation of the summarize(summaryContext) function is a restart. If isRestarted is true, the function was run before but didn't finish successfully.

Member Type	Name	Value Type	Description
	summaryContext.concurrency	number	The maximum concurrency number when running the map/reduce script.
	summaryContext.dateCreated	Date	The time and day when the script began running.
	summaryContext.seconds	number	The total time in seconds the script took to run.
	summaryContext.usage	number	The total number of usage units consumed during the processing of the script.
	summaryContext.yields	number	The total number of yields that occurred during the processing of the script.
	summaryContext.inputSummary	object	Object with data about the input stage.
	summaryContext.mapSummary	object	Object with data about the map stage.
	summaryContext.reduceSummary	object	Object with data about the reduce stage.
	summaryContext.output	iterator	Iterator with the keys and values saved from the reduce stage output.

inputSummary Object members

The following members are called on [summaryContext.inputSummary](#).

Member Type	Name	Value Type	Description
Property	inputSummary.dateCreated	Date	The time and day when the getInputData(inputContext) function began running.
	inputSummary.error	string	Holds serialized errors thrown from the getInputData(inputContext) function.
	inputSummary.seconds	number	The total time in seconds for the getInputData(inputContext) function run (not counting idle time).
	inputSummary.usage	number	The total number of usage units consumed by processing of the getInputData(inputContext) function.

mapSummary Members

The following members are called on [summaryContext.mapSummary](#).

Member Type	Name	Value Type	Description
Property	mapSummary.concurrency	number	Maximum concurrency number when running map(mapContext) .
	mapSummary.dateCreated	Date	The time and day when the first invocation of map(mapContext) function began.

Member Type	Name	Value Type	Description
	mapSummary.errors	iterator	Holds serialized errors thrown during the map stage.
	mapSummary.keys	iterator	Holds the keys passed to the map stage by the getInputData stage.
	mapSummary.seconds	number	The total time in seconds the map stage took.
	mapSummary.usage	number	The total number of usage units consumed during the map stage.
	mapSummary.yields	number	The total number of yields that occurred during the map stage.

reduceSummary Members

The following members are called on [summaryContext.reduceSummary](#).

Member Type	Name	Value Type	Description
Property	reduceSummary.concurrency	number	Maximum concurrency number when running reduce(reduceContext) .
	reduceSummary.dateCreated	Date	The time and day when the first invocation of the reduce(reduceContext) function began.
	reduceSummary.errors	iterator	Holds serialized errors thrown during the reduce stage.
	reduceSummary.keys	iterator	Holds the keys passed to the reduce stage.
	reduceSummary.seconds	number	The total time in seconds the reduce stage took.
	reduceSummary.usage	number	The total number of usage units consumed during the reduce stage.
	reduceSummary.yields	number	The total number of yields that occurred during the reduce stage.

getInputData(inputContext)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Description	Marks the start of the script's execution. The purpose of the input stage is to generate the input data. Runs when the getInputData entry point is triggered. This entry point is required. For information about the context object provided to this entry point, see inputContext. Note: If getInputData() returns a data structure with a non-string value, it's converted to a JSON string with <code>JSON.stringify()</code> before being stored.
Returns	Array Object File Query Dataset Search object reference (file, suiteql, query, search)

In simple scenarios, you can return a plain Array or a plain Object.

```
1 | return ["a", "b", "c"]
2 | return { a: 10, b: 20 }
```

Often, data comes from a query. In such case, return a SuiteQL object reference.

```
1 | return {
2 |   type: 'suiteql',
3 |   query: 'SELECT id, email FROM Employee WHERE currency = ?',
4 |   params: [currency]
5 | }
6 | // Then, in map phase, call JSON.parse(context.value).values
7 | // to get [123, "john@example.com"]
```

Alternatively, you can return Query / Search objects.

```
1 | return query.create(...)
2 | return search.create(...)
```

If the query / search exists, simply return a Query / Search object reference. There's no need to load anything.

```
1 | return { type: 'query', id: 123 }
2 | return { type: 'search', id: 123 }
```

Another common source of data are files. Map will be called for each line of the file.

```
1 | return file.load(...)
2 | return { type: 'file', id: 123 }
3 | return { type: 'file', path: '/SuiteScripts/data/names.txt' } // absolute path required
4 | // Then, in map phase, context.key is the line number (starting with 0)
5 | // and context.value is the line itself.
```

 **Note:** The file cannot contain empty lines.

Datasets are supported as well.

```
1 | return dataset.load(...)
```

Since 2015.2

Parameters

Parameter	Type	Required / Optional	Description
<code>inputContext</code>	Object	Required	Object that contains: ■ An indication of whether the current invocation of this method represents a restart ■ An Object that represents the input data For a description of each property in this object, see inputContext Object Members.

Errors

If this function throws an error, the job moves to the `summarize(summaryContext)` function. The serialized error is encapsulated in the `inputSummary.error` property.

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 // Add additional code
2 ...
3 function getInputData {
4 {
5
6     // Reference a saved search that returns a list of NetSuite records that
7     // require processing - for example, sales orders that are pending fulfillment.
8
9     return {
10         type: 'search',
11         id: 1234
12     };
13 }
14 ...
15 // Add additional code

```

inputContext

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	This object is passed to the getInputData(inputContext) entry point function. This object includes the following properties: ■ inputContext.isRestarted ■ inputContext.ObjectRef □ ObjectRef.id □ ObjectRef.type
Since	2016.1

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 return {
2     type: 'suiteql',
3     query: 'SELECT id, email FROM Employee WHERE currency = ?',
4     params: [currency]
5 }
6 // Then, in map phase, call JSON.parse(context.value).values
7 // to get [123, "john@example.com"]

```

inputContext.isRestarted

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates whether the current invocation of the getInputData(inputContext) function is the first. Usually, the getInput function runs just once. But if it's interrupted by an application server restart, the system restarts getInputData function.
-----------------------------	---

	If the value of this property is true, the getInputData(inputContext) has been restarted. You can use this property to make your script more robust.
Type	boolean (read-only)
Since	2016.1

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1  ...
2  function getInputData(context)
3  {
4      if (context.isRestarted)
5      {
6          log.debug('GET_INPUT isRestarted', 'YES');
7      }
8      else
9      {
10         log.debug('GET_INPUT isRestarted', 'NO');
11     }
12
13     var extractSearch = search.load({ id: 'customsearch35' });
14     return extractSearch;
15 }
16 ...

```

inputContext.ObjectRef

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	A plain object that defines the input data. For example, a SuiteQL query. You can return such object from getInputData(inputContext) .
Since	2015.2

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

These are examples of object references.

```

1  { type: 'query', id: 123 }
2  { type: 'search', id: 123 } { type: 'file', id: 123 }
3  { type: 'file', path: '/SuiteScripts/data/names.txt' } // absolute path required
4  {
5      type: 'suiteql',
6      query: 'SELECT id, email FROM Employee WHERE currency = ?',
7      params: [currency]
8  }

```

ObjectRef.id

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The internal ID or script ID of the object. For example, the saved search ID.
-----------------------------	---

Type	string number
Since	2015.2

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 | ...
2 | {
3 |   type: 'search',
4 |   id: 1234 //search internal id
5 | }
6 | ...

```

ObjectRef.type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The object's type.
Type	string
Values	'search'
Since	2015.2

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 | ...
2 | {
3 |   type: 'search',
4 |   id: 1234 //search internal id
5 | }
6 | ...

```

map(mapContext)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Description	Runs when the map entry point is triggered. Your map function's logic is applied to each key-value pair from the getInputData stage. Each function call processes one pair, then runs again for the next. The map stage outputs another set of key-value pairs. In the shuffle stage, these pairs are automatically grouped by key. For information about the context object provided to this entry point, see mapContext.
Returns	Void
Since	2015.2

Parameters

Parameter	Type	Required / Optional	Description
mapContext	Object	Required	Object that contains: ■ The key-value pairs to process in the map stage. ■ Logic to save data and pass it to the reduce stage. ■ Other properties you can use in the map function. For a description of each property in this object, see mapContext Object Members.

Errors

When an error is thrown, the job's behavior depends on the [retryCount](#) setting.

If the function restarts for a key-value pair it already tried to process, you can see any past errors in [mapContext.errors](#).

In the summary stage, you can review all map stage errors by using [mapSummary.errors](#).

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 // Add additional code
2 ...
3 function map(context)
4 {
5     for (var i = 0; context.value && i < context.value.length; i++)
6         if (context.value[i] !== ' ' && !PUNCTUATION_REGEXP.test(context.value[i]))
7             {
8                 context.write({
9                     key: context.value[i],
10                    value: 1
11                });
12             }
13 }
14 ...
15 ...
16 // Add additional code

```

mapContext

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Object that contains: ■ The key-value pairs to process in the map stage. ■ Logic to save data and pass it to the reduce stage. ■ Other properties you can use in the map function. This object includes the following properties and methods: ■ mapContext.isRestarted ■ mapContext.executionNo ■ mapContext.errors
---------------------------	---

	- mapContext.key - mapContext.value - mapContext.write(options)
Since	2015.2

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1  ...
2  function map(context)
3  {
4      for (var i = 0; context.value && i < context.value.length; i++)
5          if (context.value[i] !== ' ' && !PUNCTUATION_REGEXP.test(context.value[i]))
6              {
7                  context.write({
8                      key: context.value[i],
9                      value: 1
10                 });
11             }
12 }
13 ...

```

mapContext.isRestarted

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates whether the <code>map(mapContext)</code> function has run before for this key-value pair. For an overview of events that can cause a restart, see System Response After a Map/Reduce Interruption. When the map function restarts for a key-value pair, any data it wrote last time is deleted. Some of its logic might still have run before the interruption, though, so if <code>mapContext.isRestarted</code> is true, your script should take steps to avoid duplicate processing. For examples, see Adding Logic to Handle Map/Reduce Restarts. Related properties include <code>mapContext.executionNo</code> and <code>mapContext.errors</code>.
Type	boolean
Since	2016.1

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1  // Add additional code.
2  ...
3  function map(context) {
4      if (context.isRestarted)
5      {
6          // Add logic designed to assess how much processing was completed for this key-value pair and react accordingly.
7      }
8      else
9      {
10         // Let full processing continue for the key-value pair.
11     }
12     ...
13 } // Add additional code.

```

mapContext.executionNo

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates whether the current invocation of the map(mapContext) function is the first or a subsequent invocation for the current key-value pair. For an overview of events that can cause a restart, see System Response After a Map/Reduce Interruption. When the map function restarts for a key-value pair, any data it wrote last time is deleted. Some logic might have run before it was interrupted, so you might want to use mapContext.executionNo to add logic that avoids duplicate processing. For examples of how to write a robust map/reduce script, see Adding Logic to Handle Map/Reduce Restarts. Related properties include mapContext.isRestarted and mapContext.errors.
Type	number
Since	2018.1

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 // Add additional code
2 ...
3
4 if (context.executionNo === 1){
5     // Permit full processing of the key-value pair.
6 }
7
8 else if (context.executionNo === 2){
9     // Take steps to check whether any processing was previously completed.
10 }
11
12 else {
13     // Take other steps that might be necessary in the case of more than
14     // one previous attempt.
15 }
16 ...
17 // Add additional code

```

mapContext.errors

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Holds serialized errors that were thrown during previous attempts to run the map function on the current key-value pair. This iterator may also hold the SSS_APP_SERVER_RESTART error code, which is recorded if the function is interrupted by an application server restart. For an overview of events that can cause the map function to be invoked multiple times for a key-value pair, see System Response After a Map/Reduce Interruption.
Type	iterator
Since	2018.1

Members

Member	Type	Required/Optional	Description
iterator().each(parameters)	function	required	Executes one time for each error.

parameters

Member	Type	Required/Optional	Description
iteratorFunction(key, error, executionNo) See also functionParameters .	function	required	Provides logic to be executed during each iteration.

functionParameters

Parameter	Type	Required/Optional	Description
key	string	optional	Represents the key-value pair that the map function was attempting to process when the error occurred.
error	string	optional	A serialization of the error thrown.
executionNo	number	optional	Indicates whether the error occurred during the first or a subsequent attempt to process the key-value pair.

Syntax

The following snippets shows three ways you could use this iterator.

This code isn't a functional example. For a complete script sample, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 //Add additional code
2 ...
3
4 // Create a log entry showing each full serialized error, and the corresponding key.
5
6 context.errors.iterator().each(function (key, error, executionNo){
7     log.error({
8         title: 'Map error for key: ' + key + ', execution no ' + executionNo,
9         details: error
10    });
11    return true;
12 });
13
14
15 // Log only the name and description of each error thrown.
16
17 context.errors.iterator().each(function (key, error, executionNo){
18     var errorObject = JSON.parse(error);
19     log.error({
20         title: 'Reduce error for key ' + key + ', execution no. ' + executionNo,
21         details: errorObject.name + ': ' + errorObject.message
22    });
23    return true;
24 });
25
26
27 // Calculate and log the number of errors encountered.
28
29 var errorCount = 0;
30 context.errors.iterator().each(function() {

```

```

31     errorCount++;
32     return true;
33 });
34
35 log.audit({
36     title: 'Errors for map key: ' + context.key,
37     details: 'Total number of errors: ' + errorCount
38 });

```

mapContext.key

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The key to be processed during the map stage. ■ If the input is an array, the key is the element's index. ■ If the input is an object, the key is the object's key. ■ If the input is a result set, the key is the result's internal ID. If there's no internal ID, the key is the result's index.  Note: Each key cannot exceed 4000 bytes.
Type	string
Since	2015.2

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 // Add additional code
2 ...
3
4     context.write({
5         key: context.value[i],
6         value: 1
7     });
8 ...
9 // Add additional code

```

mapContext.value

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The value to be processed during the map stage. ■ If the input is an array, mapContext.value is the element's value. ■ If the input is an object, mapContext.value is the object's value. ■ If the input type is a result set, the mapContext.value is a search.Result object converted to a JSON string by using <code>JSON.stringify()</code>.  Note: Each value cannot exceed 1 megabyte.
Type	string

Since	2015.2
-------	--------

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 // Add additional code.
2 ...
3
4 // Assume that the search result is a list of phone call records. This snippet parses the results
5 // and uses the values of the title and the message fields from each record.
6
7 var searchResult = JSON.parse(context.value);
8
9 var title = searchResult.values.title;
10 var message = searchResult.values.message;
11
12 ...
13 // Add additional code.
```

mapContext.write(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Writes the key-value pairs to pass to the shuffle stage, and then to the reduce stage. If your script has both a map and a reduce function, you need to use this method so the reduce function runs.
Returns	Void
Since	2015.2

Parameters

i Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description
options.key	String or object. However, note that if you provide an object, the system calls JSON.stringify() on your input.	required	The key to write
options.value		required	The value to write

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 ...
2 function map(context)
3 {
4     for (var i = 0; context.value && i < context.value.length; i++)
5         if (context.value[i] !== ' ' && !PUNCTUATION_REGEXP.test(context.value[i]))
6             {
7                 context.write({
8                     key: context.value[i],
9                     value: 1
```

```

10 |         });
11 |     }
12 | }
13 | ...

```

reduce(reduceContext)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Description	Runs when the reduce entry point is triggered. The reduce function's logic is applied to each key and its list of values. Each call processes one key and its values, then runs again for the next set. Data is provided to the reduce stage by one of the following: ■ The <code>getInputData</code> stage — if your script has no map function. ■ The shuffle stage — if your script uses a map function. The shuffle stage follows the map stage. Its purpose is to sort data from the map stage by key. For information about the context object provided to this entry point, see reduceContext.
Returns	Void
Since	2015.2

Parameters

Parameter	Type	Required / Optional	Description
reduceContext	Object	Required	Object that has: ■ The data to process in the reduce stage. ■ Logic to save data and pass it to the summarize stage. ■ Other properties you can use in the reduce function. For a description of each property in this object, see reduceContext Object Members.

Errors

When an error is thrown, the job's behavior depends on the [retryCount](#) setting.

If the function restarts for a key-value pair it already tried to process, you can see any past errors in [reduceContext.errors](#).

In the summary stage, you can review all reduce stage errors by using [reduceSummary.errors](#).

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 | ...
2 | function reduce(context)
3 | {
4 |     context.write({

```

```

5         key: context.key ,
6         value: context.values.length
7     });
8 }
9 ...

```

reduceContext

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Contains the key-value pairs to process during the reduce stage. This object includes the following properties and methods: ■ reduceContext.isRestarted ■ reduceContext.executionNo ■ reduceContext.errors ■ reduceContext.key ■ reduceContext.values ■ reduceContext.write(options)
Since	2015.2

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 ...
2 function reduce(context)
3 {
4     context.write({
5         key: context.key ,
6         value: context.values.length
7     });
8 }
9 ...

```

reduceContext.isRestarted

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates whether the reduce(reduceContext) function has run before for the current key and values. For an overview of events that can cause a restart, see System Response After a Map/Reduce Interruption. When the reduce function restarts for a key-value pair, any data it wrote last time is deleted. Some logic might have run before it was interrupted, so if reduceContext.isRestarted is true, your script should take steps to avoid duplicate processing. For examples, see Adding Logic to Handle Map/Reduce Restarts. For examples, see Adding Logic to Handle Map/Reduce Restarts. Related properties include reduceContext.executionNo and reduceContext.errors.
Type	boolean
Since	2016.1

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 // Add additional code.
2 ...
3 function reduce (context) {
4   if (context.isRestarted)
5   {
6     // Add logic designed to assess how much processing was completed for this key
7     // and react accordingly.
8   }
9   else
10  {
11    // Let full processing continue for the current key and its values.
12  }
13  ...
14 // Add additional code.

```

reduceContext.executionNo

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates whether the current invocation of the reduce(reduceContext) function is the first, second, third, or fourth for the current key and its values. For an overview of events that can cause a restart, see System Response After a Map/Reduce Interruption. When the reduce function restarts for a key, any data it wrote last time is deleted. Some logic might have run before it was interrupted, so you might want to use <code>reduceContext.executionNo</code> to help avoid duplicate processing. For examples of how to write a robust map/reduce script, see Adding Logic to Handle Map/Reduce Restarts. Related properties include reduceContext.isRestarted and reduceContext.errors.
Type	number
Since	2018.1

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 // Add additional code
2 ...
3
4 if (context.executionNo === 1){
5   // Permit full processing of the key-value pair.
6 }
7
8 else if (context.executionNo === 2){
9   // Take steps to check whether any processing was previously completed.
10 }
11
12 else {
13   // Take other steps that might be necessary in the case of more than
14   // one previous attempt.
15 }
16 ...
17 // Add additional code

```

reduceContext.errors

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Holds errors thrown during earlier tries to run the reduce function for this key and its values. This iterator may also hold the SSS_APP_SERVER_RESTART error code, which is logged if the function is interrupted by an application server restart. For an overview of events that can cause the map function to be invoked multiple times for a key-value pair, see System Response After a Map/Reduce Interruption.
Type	iterator
Since	2018.1

Members

Member	Type	Required/Optional	Description
iterator().each(parameters)	function	required	Executes one time for each error.

parameters

Member	Type	Required/Optional	Description
iteratorFunction(key, error, executionNo) See also functionParameters .	function	required	Provides logic to be executed during each iteration.

functionParameters

Parameter	Type	Required/Optional	Description
key	string	optional	The key-value pair the reduce function was trying to process when the error occurred.
error	string	optional	A serialization of the error thrown.
executionNo	number	optional	Shows if the error occurred on the first attempt or a later try to process the key.

Syntax

The following snippets shows three ways you could use this iterator.

This code isn't a functional example. For a complete script sample, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 //Add additional code
2 ...
3
4 // Create a log entry showing each full serialized error, and the corresponding key.
5
6 context.errors.iterator().each(function (key, error, executionNo){
7     log.error({
8         title: 'Reduce error for key: ' + key + ', execution no ' + executionNo,
9         details: error

```

```

10     });
11     return true;
12 });
13
14
15 // Log only the name and description of each error thrown.
16
17 context.errors.iterator().each(function (key, error, executionNo){
18     var errorObject = JSON.parse(error);
19     log.error({
20         title: 'Reduce error for key ' + key + ', execution no. ' + executionNo,
21         details: errorObject.name + ': ' + errorObject.message
22     });
23     return true;
24 });
25
26
27 // Calculate and log the number of errors encountered.
28
29 var errorCount = 0;
30 context.errors.iterator().each(function() {
31     errorCount ++;
32     return true;
33 });
34
35 log.audit ({
36     title: 'Errors for reduce key: ' + context.key,
37     details: 'Total number of errors: ' + errorCount
38 });

```

reduceContext.key

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	If the map/reduce process includes a map stage, the reduce keys are the keys written by <code>mapContext.write(options)</code>. If the map stage is skipped, the reduce keys are provided by the <code>getInputData</code> stage: ■ If the input type is an array, the key is the index of the element. ■ If the input type is an object, the key is the key in the object. ■ If the input type is a result set, the key is the internal ID of the result.  Note: Each key cannot exceed 4000 bytes.
Type	string
Since	2015.2

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 ...
2 function reduce(context)
3 {
4     context.write({
5         key: context.key ,
6         value: context.values.length
7     });
8 }
9 ...

```

reduceContext.values

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	This array holds all values for a unique key passed to the reduce stage. These values are listed in lexicographical order. When the map/reduce process includes a map stage, the key-value pairs passed to the reduce stage are derived from the values written by <code>mapContext.write(options)</code>. When the map stage is skipped, the values are determined by the <code>getInputData</code> stage: ■ If the input type is an array, it's the value in the element. ■ If the input type is an object, it's the value in the object. ■ If the input type is a result set, the value is a <code>search.Result</code> object converted to a JSON string with <code>JSON.stringify()</code>. i Note: Each value cannot exceed 1 megabyte.
Type	string[]
Since	2015.2

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1  ...
2  function reduce(context)
3  {
4      context.write({
5          key: context.key ,
6          value: context.values.length
7      });
8  }
9  ...

```

reduceContext.write(options)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Writes key-value pairs.
Returns	Void
Since	2015.2

Parameters

i Note: The options parameter is a JavaScript object.
--

Parameter	Type	Required / Optional	Description
options.key	String or object. However, note that if you provide an object, the system calls <code>JSON.stringify()</code> on your input.	required	The key to write.
options.value		required	The value to write

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1  ...
2  function reduce(context)
3  {
4      context.write({
5          key: context.key,
6          value: context.values.length
7      });
8  }
9  ...

```

summarize(summaryContext)

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Description	Executes when the summarize entry point is triggered. When you add custom logic to this entry point function, that logic is applied to the result set. For information about the context object provided to this entry point, see summaryContext.
Returns	Void
Since	2015.2

Parameters

Parameter	Type	Required / Optional	Description
summaryContext	Object	Required	Holds statistics regarding the execution of a map/reduce script. For a description of each property in this object, see summaryContext Object Members.

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1  ...
2  function summarize(summary)
3  {
4      var type = summary.toString();
5      log.audit(type + ' Usage Consumed', summary.usage);
6      log.audit(type + ' Concurrency Number ', summary.concurrency);
7      log.audit(type + ' Number of Yields', summary.yields);
8      var contents = '';
9      summary.output.iterator().each(function (key, value)
10         {
11             contents += (key + ' ' + value + '\n');
12             return true;
13         });
14  ...

```

summaryContext

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Holds statistics regarding execution of a map/reduce script. This object includes the following properties: ■ summaryContext.isRestarted ■ summaryContext.concurrency ■ summaryContext.dateCreated ■ summaryContext.seconds ■ summaryContext.usage ■ summaryContext.yields ■ summaryContext.inputSummary □ inputSummary.dateCreated □ inputSummary.seconds □ inputSummary.usage □ inputSummary.error ■ summaryContext.mapSummary □ mapSummary.concurrency □ mapSummary.dateCreated □ mapSummary.keys □ mapSummary.seconds □ mapSummary.usage □ mapSummary.yields □ mapSummary.errors ■ summaryContext.reduceSummary □ reduceSummary.concurrency □ reduceSummary.dateCreated □ reduceSummary.keys □ reduceSummary.seconds □ reduceSummary.usage □ reduceSummary.yields □ reduceSummary.errors ■ summaryContext.output
Since	2015.2

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1  ...
2  function summarize(summary)
3  {
4      var type = summary.toString();
5      log.audit(type + ' Usage Consumed', summary.usage);
6      log.audit(type + ' Concurrency Number ', summary.concurrency);
7      log.audit(type + ' Number of Yields', summary.yields);

```

```

8      var contents = '';
9      summary.output.iterator().each(function (key, value)
10     {
11         contents += (key + ' ' + value + '\n');
12         return true;
13     });
14     ...

```

summaryContext.isRestarted

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Indicates whether the summarize(summaryContext) function was invoked again. To reduce problems if the Java virtual machine (JVM) restarts, NetSuite automatically restarts the current summary function. Any summary data written by the incomplete function is deleted. If the value is true, the current process invoked by summarize(summaryContext) was restarted. You can use this information to help you write a more robust map/reduce script that is designed to continue interrupted work as necessary.
Type	boolean (read-only)
Since	2016.1

Syntax

```

1  ...
2  function summarize(summary) {
3      if (summary.isRestarted)
4      {
5          log.debug('SUMMARY isRestarted', 'YES');
6      }
7      else
8      {
9          log.debug('SUMMARY isRestarted', 'NO');
10     }
11     log.debug('summarize', JSON.stringify(summary));
12     ...

```

summaryContext.concurrency

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The maximum concurrency number when executing parallel tasks for the map/reduce script.  Note: This number may be less than the number allocated on the script deployment. For example, tasks that remained in the pending state are not reflected in this number.
Type	number
Since	2015.2

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 | ...
2 | log.audit(type + ' Concurrency Number ', summary.concurrency);
3 | ...

```

summaryContext.dateCreated

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The time and day when the map/reduce script began running.
Type	Date
Since	2015.2

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 | ...
2 | log.audit(type + ' Creation Date', summary.dateCreated);
3 | ...

```

summaryContext.seconds

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Total seconds elapsed when running the map/reduce script.
Type	number
Since	2015.2

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 | ...
2 | log.audit(type + ' Total seconds elapsed', summary.seconds);
3 | ...

```

summaryContext.usage

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Total number of usage units consumed when running the map/reduce script.
Type	number
Since	2015.2

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).


```

1 | ...
2 | log.audit(type + ' Usage Consumed', summary.usage);
3 | ...

```

summaryContext.yields

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Total number of yields when running the map/reduce script.
Type	number
Since	2015.2

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 | ...
2 | log.audit(type + ' Number of Yields', summary.yields);
3 | ...

```

summaryContext.inputSummary

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Holds statistics regarding the input stage.
Since	2015.2

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 | ...
2 | logMetrics(summary.inputSummary);
3 | log.error('Input Error', summary.inputSummary.error);
4 |
5 | ...

```

inputSummary.dateCreated

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The time and day when getInputData(inputContext) began running.
Type	Date
Since	2015.2

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 | ...
2 | log.audit(' Creation Date', summary.inputSummary.dateCreated);
3 | ...

```

inputSummary.seconds

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Total seconds getInputData(inputContext) ran (not including idle time).
Type	number
Since	2015.2

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 | ...
2 | log.audit(' Time Elapsed', summary.inputSummary.seconds);
3 | ...

```

inputSummary.usage

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Total number of usage units consumed when running getInputData(inputContext) .
Type	number
Since	2015.2

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 | ...
2 | log.audit(' Usage', summary.inputSummary.usage);
3 | ...

```

inputSummary.error

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	If applicable, holds a serialized error that is thrown from getInputData(inputContext) .
Type	string
Since	2015.2

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 | ...
2 | logMetrics(summary.inputSummary);
3 | log.error('Input Error', summary.inputSummary.error);
4 |
5 | ...

```

summaryContext.mapSummary

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Holds statistics regarding the map stage
Since	2015.2

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 | ...
2 | logMetrics(summary.mapSummary);
3 | var mapKeys = [];
4 | summary.mapSummary.keys.iterator().each(function (key)
5 | {
6 |     mapKeys.push(key);
7 |     return true;
8 | });
9 | log.audit('MAP keys processed', mapKeys);
10 | summary.mapSummary.errors.iterator().each(function (key, error)
11 | {
12 |     log.error('Map Error for key: ' + key, error);
13 |     return true;
14 | });
15 | ...

```

mapSummary.concurrency

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The maximum concurrency number for executing parallel tasks during the map stage. Note: This number might be lower than the concurrency number set in the script deployment. For example, tasks that remained in the pending state are not reflected in this number.
Type	number
Since	2015.2

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 | ...
2 | log.audit('Concurrency', summary.mapSummary.concurrency);
3 | ...

```

mapSummary.dateCreated

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The time and day when <code>map(mapContext)</code> began running.
Type	Date
Since	2015.2

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1  ...
2  log.audit(' Creation Date', summary.mapSummary.dateCreated);
3  ...

```

mapSummary.keys

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Holds the keys that were passed to the map stage by the getInputData stage. Note that keys are sorted in the following way: ■ If all values are numbers, they're listed in numeric order. ■ If any values are strings, they're listed in lexicographical order.
Type	iterator
Since	2015.2

Members

Member	Type	Required/Optional	Description
<code>iterator().each(parameters)</code>	function	required	Executes one time for each key.

parameters

Member	Type	Required/Optional	Description
<code>iteratorFunction(key, executionCount, completionState)</code> See also functionParameters .	function	required	Provides logic to be executed during each iteration.

functionParameters

Parameter	Type	Required/Optional	Description
key	string	optional	Represents a key that was passed to the map stage.

Parameter	Type	Required/Optional	Description
executionCount	number	optional	The number of times the map function was called for the key.
completionState	string	optional	A setting that indicates whether the map function was executed successfully for the key. Possible values are 'COMPLETE', 'FAILED', and 'PENDING.' The system uses 'PENDING' if the script left the stage before it tried to process the key. This behavior can occur when exitOnError is set to true.

Syntax

The following snippets show three ways you could use this iterator.

This code isn't a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 //Add additional code.
2 ...
3
4 // Log all keys from the map stage.
5
6 var mapKeys = [];
7 summary.mapSummary.keys.iterator().each(function (key){
8     mapKeys.push(key);
9     return true;
10 });
11
12 log.debug({
13     title: 'Map stage keys',
14     details: mapKeys
15 });
16
17
18 // Create a log entry showing whether the map function was executed succesfully for each key.
19
20 summary.mapSummary.keys.iterator().each(function (key, executionCount, completionState){
21
22     log.debug({
23         title: 'Map key ' + key,
24         details: 'Outcome for map key ' + key + ': ' + completionState + ' // Number of attempts used: ' + executionCount
25     });
26
27     return true;
28 });
29
30
31
32 // Create a log entry showing the total number of keys for which the map function executed successfully.
33
34 var mapKeysProcessed = 0;
35 summary.mapSummary.keys.iterator().each(function (key, executionCount, completionState){
36
37     if (completionState === 'COMPLETE'){
38         mapKeysProcessed++;
39     }
40
41     return true;
42 });
43
44
45 log.debug({
46     title: 'Map key statistics',
47     details: 'Total number of map keys processed successfully: ' + mapKeysProcessed
48 });

```

mapSummary.seconds

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Total seconds elapsed when running <code>map(mapContext)</code> .
Type	number
Since	2015.2

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 | ...
2 | log.audit(' Time Elapsed', summary.mapSummary.seconds);
3 | ...

```

mapSummary.usage

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Total number of usage units consumed when running <code>map(mapContext)</code> .
Type	number
Since	2015.2

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 | ...
2 | log.audit(' Usage', summary.mapSummary.usage);
3 | ...

```

mapSummary.yields

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Total number of times yields when running <code>map(mapContext)</code> .
Type	number
Since	2015.2

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 | ...
2 | log.audit(' Total Yields', summary.mapSummary.yields);
3 | ...

```

mapSummary.errors

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Holds all serialized errors thrown from the map(mapContext) function. May also hold the SSS_APP_SERVER_RESTART error code, which is recorded in the event of an application server restart.
Type	iterator
Since	2015.2

Members

Member	Type	Required/Optional	Description
iterator().each(parameters)	function	required	Executes one time for each error.

parameters

Member	Type	Required/Optional	Description
iteratorFunction(key, error, executionNo) See also functionParameters .	function	required	Provides logic to be executed during each iteration.

functionParameters

Parameter	Type	Required/Optional	Description
key	string	optional	Represents a key that was passed to the map stage for processing.
error	string	optional	The error thrown during processing of the corresponding key.
executionNo	number	optional	Indicates whether the error occurred during the first, second, third, or fourth attempt to process the key.

Syntax

The following snippets show three ways you could use this iterator.

This code isn't a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 // Add additional code
2 ...
3 // Create a log entry showing each full serialized error
4 summary.mapSummary.errors.iterator().each(
5     function (key, error, executionNo) {
6         log.error({
7             title: 'Map error for key: ' + key + ', execution no. ' + executionNo,
8             details: error
9         });
10        return true;
11    }
12 );
13
14 // Log only the name and description of each error thrown

```

```

15 summary.mapSummary.errors.iterator().each(
16     function (key, error, executionNo) {
17         var errorObject = JSON.parse(error);
18         log.error({
19             title: Map error for key: ' + key + ', execution no. ' + executionNo,
20             details: errorObject.name + ': ' + errorObject.message
21         });
22         return true;
23     }
24 );
25
26 // Calculate and log the total number of errors encountered during the map stage
27 var errorCount = 0;
28 summary.mapSummary.errors.iterator().each(
29     function() {
30         errorCount++;
31         return true;
32     }
33 );
34
35 log.audit({
36     title: Map stage errors',
37     details: 'Total number of errors: ' + errorCount
38 });
39 ...
40 // Add additional code

```

summaryContext.reduceSummary

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Object Description	Holds statistics regarding the reduce stage.
Since	2015.2

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 ...
2 summary.reduceSummary.errors.iterator().each(function (key, error)
3 {
4     log.error('Reduce Error for key: ' + key, error);
5     return true;
6 });
7 ...

```

reduceSummary.concurrency

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The maximum concurrency number for executing parallel tasks during the reduce stage.  Note: This number might be lower than the concurrency set in the script deployment. For example, tasks that remained in the pending state are not reflected in this number.
Type	number
Since	2015.2

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```
1 | ...
2 | log.audit(' Concurrency', summary.reduceSummary.concurrency);
3 | ...
```

reduceSummary.dateCreated

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The time and day when reduce(reduceContext) began running.
Type	Date
Since	2015.2

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```
1 | ...
2 | log.audit(' Creation Date', summary.reduceSummary.dateCreated);
3 | ...
```

reduceSummary.keys

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Holds the keys that were passed to the reduce stage, either by the map stage, if it was used, or by the getInputData stage. These keys are listed in lexicographical order .
Type	iterator
Since	2015.2

Members

Member	Type	Required/Optional	Description
iterator().each(parameters)	function	required	Executes one time for each key.

parameters

Member	Type	Required/Optional	Description
iteratorFunction(key, executionCount, completionState) See also functionParameters .	function	required	Provides logic to be executed during each iteration.

functionParameters

Parameter	Type	Required/Optional	Description
key	string	optional	Represents a key that was passed to the reduce stage.
executionCount	number	optional	The number of times the reduce function was invoked for the key.
completionState	string	optional	A setting that indicates whether the reduce function was executed successfully for the key. Possible values are 'COMPLETE', 'FAILED', and 'PENDING.' The system uses 'PENDING' if the script left the stage before it tried to process the key. This behavior can occur when <code>exitOnError</code> is set to true.

Syntax

The following snippets show several ways you could use the `reduceSummary.keys` iterator. The code in the section isn't a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 //Add additional code.
2 ...
3
4 // Create a single log entry listing all of the keys that were passed to the reduce stage.
5
6 var reduceKeys = [];
7
8 summary.reduceSummary.keys.iterator().each(function (key){
9     reduceKeys.push(key);
10    return true;
11 });
12
13 log.debug({
14     title: 'Reduce stage keys',
15     details: reduceKeys
16 });
17
18
19 // Create a log entry for each key. The entry shows whether the reduce function was executed successfully for that key.
20
21 summary.reduceSummary.keys.iterator().each(function (key, executionCount, completionState){
22
23     log.debug({
24         title: 'Reduce key ' + key,
25         details: 'Outcome for reduce key ' + key + ': ' + completionState + ' // Number of attempts used: ' + executionCount
26     });
27
28     return true;
29
30 });
31
32
33 // creates a single log entry showing the total number of keys for which the reduce function was invoked successfully.
34
35 var reduceKeysProcessed = 0;
36 summary.reduceSummary.keys.iterator().each(function (key, executionCount, completionState){
37
38     if (completionState === 'COMPLETE'){
39         reduceKeysProcessed++;
40     }
41
42     return true;
43
44 });
45
46 log.debug({

```

```

47 |     title: 'Reduce key statistics',
48 |     details: 'Total number of reduce keys processed successfully: ' + reduceKeysProcessed
49 | });
50 |
51 | ...
52 | //Add additional code.

```

reduceSummary.seconds

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Total seconds elapsed when running reduce(reduceContext) .
Type	number
Since	2015.2

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 | ...
2 | log.audit(' Time Elapsed', summary.reduceSummary.seconds);
3 | ...

```

reduceSummary.usage

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Total number of usage units consumed when running reduce(reduceContext) .
Type	number
Since	2015.2

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1 | ...
2 | log.audit(' Usage', summary.mapSummary.usage);
3 | ...

```

reduceSummary.yields

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Total number of times yields when running reduce(reduceContext) .
Type	number
Since	2015.2

Syntax

The following code snippet shows the syntax for this member. It's not a functional example. For a complete script example, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1  ...
2  log.audit(' Total Yields', summary.reduceSummary.yields);
3  ...

```

reduceSummary.errors

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Holds all serialized errors thrown from the reduce(reduceContext) function. May also hold the SSS_APP_SERVER_RESTART error code, which is recorded in the event of an application server restart.
Type	iterator
Since	2015.2

Members

Member	Type	Required/Optional	Description
iterator().each(parameters)	function	required	Executes one time for each error.

parameters

Member	Type	Required/Optional	Description
iteratorFunction(key, error, executionNo) See also functionParameters .	function	required	Provides logic to be executed during each iteration.

functionParameters

Parameter	Type	Required/Optional	Description
key	string	optional	Represents a key that was passed to the reduce stage for processing.
error	string	optional	A serialization of the error thrown.
executionNo	number	optional	Indicates whether the error occurred during the first, second, third, or fourth attempt to process the key.

Syntax

The following snippets shows three ways you could use this iterator.

This code isn't a functional example. For a complete script sample, see [SuiteScript 2.x Map/Reduce Script Code Samples](#).

```

1  //Add additional code
2  ...
3
4  // Create a log entry showing each full serialized error.
5
6  summary.reduceSummary.errors.iterator().each(function (key, error, executionNo){
7      log.error({
8          title: 'Reduce error for key: ' + key + ', execution no. ' + executionNo,
9          details: error

```

```

10     });
11     return true;
12 });
13
14
15 // Log only the name and description of each error thrown.
16
17 summary.reduceSummary.errors.iterator().each(function (key, error){
18     var errorObject = JSON.parse(error);
19     log.error({
20         title: 'Reduce error for key: ' + key + ', execution no. ' + executionNo,
21         details: errorObject.name + ': ' + errorObject.message
22     });
23     return true;
24 });
25
26
27 // Calculate and log the total number of errors encountered during the reduce stage.
28
29 var errorCount = 0;
30 summary.reduceSummary.errors.iterator().each(function() {
31     errorCount ++;
32     return true;
33 });
34
35 log.audit ({
36     title: 'Reduce stage errors',
37     details: 'Total number of errors: ' + errorCount
38 });

```

summaryContext.output

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	Iterator that provides keys and values saved as output during the reduce stage. Note that keys are listed in lexicographical order .
Type	iterator
Since	2015.2

Members

Member	Type	Required/Optional	Description
iterator().each(parameters)	function	required	Executes one time for each key-value pair.

parameters

Member	Type	Required/Optional	Description
iteratorFunction(key, value). See functionParameters .	function	required	Provides logic to be executed during each iteration.

functionParameters

Parameter	Type	Required/Optional	Description
key	string	optional	The key saved at the end of the reduce stage by using the reduceContext.write(options) method.

Parameter	Type	Required/Optional	Description
value	string	optional	The value saved at the end of the reduce stage by using the reduceContext.write(options) method.

Syntax

```

1 // Add additional code
2 ...
3
4 // Create a variable to track how many key-value pairs were written.
5 var totalRecordsUpdated = 0;
6
7
8 // If the number of key-value pairs is expected to be manageable, log
9 // each one.
10
11 summary.output.iterator().each(function (key, value){
12     log.audit({
13         title: 'summary.output.iterator',
14         details: 'key: ' + key + ' / value: ' + value
15     });
16     totalRecordsUpdated++;
17     return true;
18 });
19
20 // Create a log entry showing the number of
21
22 log.debug({
23     title: 'Total records updated',
24     details: totalRecordsUpdated
25 });
26 ...
27 // Add additional code.
```

SuiteScript 2.x Map/Reduce Script Code Samples

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Two map/reduce script samples are provided:

- [Counting Characters Example](#) – counts how often each alphabet letter appears in a string. This basic script helps you understand how map/reduce scripts work.
- [Processing Invoices Example](#)– handles invoices and includes error-handling logic.

These script samples use SuiteScript 2.x. A newer version, SuiteScript 2.1, is also available and supports new language features that are included in the ES2019 specification. You can write map/reduce scripts using either SuiteScript 2.0 or SuiteScript 2.1.

- For help with writing scripts in SuiteScript 2.x, see [SuiteScript 2.x Hello World](#) and [SuiteScript 2.x Entry Point Script Creation and Deployment](#).
- For more information about SuiteScript versions and SuiteScript 2.1, see [SuiteScript Versioning Guidelines](#) and [SuiteScript 2.1](#).

Counting Characters Example

The following sample is a basic map/reduce script. This sample isn't meant for real-world business use, but rather to show how map/reduce scripts work.

This script defines a hard-coded string. The script counts each alphabet letter's occurrences in the string and creates a results file. Check the script comments for details on how the system processes it.

```

1  /**
2   * @ApiVersion 2.x
3   * @NScriptType MapReduceScript
4   */
5
6  define(['N/file'], function(file) {
7
8      // Define characters that should not be counted when the script performs its
9      // analysis of the text.
10     const PUNCTUATION_REGEX = /[^\u2000-\u206F\u2E00-\u2E7F\\'!"#$%&\'(\)\*\'+,-\.\:;<=>\?@\[\]\^_`\{\|\}~]/g;
11
12     // Use the getInputData function to return two strings (1) the quick brown fox and (2) jumps over the lazy dog.
13     function getInputData() {
14         return "the quick brown fox \n jumps over the lazy dog.".split('\n');
15     }
16
17     // After the getInputData function is executed, the system creates the following
18     // key-value pairs:
19     //
20     // key: 0, value: 'the quick brown fox'
21     // key: 1, value: 'jumps over the lazy dog.'
22
23     // The map function is invoked one time for each key-value pair. Each time the
24     // function is invoked, the relevant key-value pair is made available through
25     // the context.key and context.value properties.
26     function map(context) {
27
28         // Create a loop that examines each character in the string. Exclude spaces
29         // and punctuation marks.
30         for (var i = 0; context.value && i < context.value.length; i++) {
31             if (context.value[i] !== ' ' && !PUNCTUATION_REGEX.test(context.value[i])) {
32
33                 // For each character, invoke the context.write() method. This method saves
34                 // a new key-value pair. For the new key, save the character currently being
35                 // examined by the loop. For each value, save the number 1.
36
37                 context.write({
38                     key: context.value[i],
39                     value: 1
40                 });
41             }
42         }
43     }
44
45     // After the map function has been invoked for the last time, the shuffle stage
46     // begins. In this stage, the system sorts the 35 key-value pairs that were saved
47     // by the map function during its two invocations. From those pairs, the shuffle
48     // stage creates a new set of key-value pairs, where the each key is unique. In
49     // this way, the number of key-value pairs is reduced to 25. For example, the map
50     // stage saved three instances of { 'e','1'}. In place of those pairs, the shuffle
51     // stage creates one pair: { 'e', ['1','1','1']}. These pairs are made available to
52     // the reduce stage through the context.key and context.values properties.
53
54     // The reduce function is invoked one time for each of the 25 key-value pairs
55     // provided.
56     function reduce(context) {
57
58         // Use the context.write() method to save a new key-value pair, where the new key
59         // equals the key currently being processed by the function. This key is a letter
60         // in the alphabet. Make the value equal to the length of the context.values array.
61         // This number represents the number of times the letter occurred in the original
62         // string.
63
64         context.write({
65             key: context.key,
66             value: context.values.length
67         });
68     }
69

```

```

70 // The summarize stage is a serial stage, so this function is invoked only one
71 // time.
72 function summarize(context) {
73
74     // Log details about the script's execution.
75     log.audit({
76         title: 'Usage units consumed',
77         details: context.usage
78     });
79     log.audit({
80         title: 'Concurrency',
81         details: context.concurrency
82     });
83     log.audit({
84         title: 'Number of yields',
85         details: context.yields
86     });
87
88     // Use the context object's output iterator to gather the key-value pairs saved
89     // at the end of the reduce stage. Also, tabulate the number of key-value pairs
90     // that were saved. This number represents the total number of unique letters
91     // used in the original string.
92     var text = '';
93     var totalKeysSaved = 0;
94     context.output.iterator().each(function(key, value) {
95         text += (key + ' ' + value + '\n');
96         totalKeysSaved++;
97         return true;
98     });
99
100    // Log details about the total number of pairs saved.
101    log.audit({
102        title: 'Unique number of letters used in string',
103        details: totalKeysSaved
104    });
105
106    // Use the Nfile module to create a file that stores the reduce stage output,
107    // which you gathered by using the output iterator.
108    var fileObj = file.create({
109        name: 'letter_count_result.txt',
110        fileType: file.Type.PLAINTEXT,
111        contents: text
112    });
113
114    fileObj.folder = -15;
115    var fileId = fileObj.save();
116
117    log.audit({
118        title: 'Id of new file record',
119        details: fileId
120    });
121 }
122
123 // Link each entry point to the appropriate function.
124 return {
125     getInputData: getInputData,
126     map: map,
127     reduce: reduce,
128     summarize: summarize
129 };
130 });

```

The character limit for keys in map/reduce scripts (specifically, in mapContext or reduceContext objects) is reduced to 3,000 characters. You'll also get error messages if your key is too long or your value is over 10 MB. Keys over 3,000 characters will return a KEY_LENGTH_IS_OVER_3000_BYTES error. Values over 10 MB will return a VALUE_LENGTH_IS_OVER_10_MB error. For more information about map/reduce limits see [Map/Reduce Governance](#).

If you have map/reduce scripts that use the mapContext.write(options) or reduceContext.write(options) methods, keep your key strings under 3,000 characters and value strings under 10 MB. Consider the

potential length of any dynamically generated strings which may exceed these limits and avoid using keys, instead of values, to pass your data.

There's a 200 MB limit on data passed between stages in a single execution. Should persistent data exceed this limit, the script will require provisions for a data custom field or record to be used in either context.

Processing Invoices Example

The following example shows a sample script that processes invoices and contains logic to handle errors. This script does the following:

- Find the customers associated with all open invoices.
- Apply a location-based discount to each invoice.
- Write each invoice to the reduce stage so it's grouped by customer.
- Initialize a new CustomerPayment for each customer applied only to the invoices specified in the reduce values.
- Create a custom record capturing the details of the records that were processed.
- Send an email to administrators about any exceptions.

Before running this sample, create a custom record type with ID "customrecord_summary", and text fields with id "custrecord_time", "custrecord_usage", and "custrecord_yields".

Script Sample Prerequisites

1. In NetSuite, go to Customization > List, Records, & Fields > Record Types > New.
2. On the Custom Record Type page, enter a name.
3. In the ID field, enter "customrecord_summary".
4. Select Save.
5. From the Fields subtab, do the following:
 - Select New Field. Enter a label and set ID to "custrecord_time". Ensure that the Type field is set to Free-Form Text. Select Save & New.
 - Select New Field. Enter a label and set ID to "custrecord_usage". Ensure that the Type field is set to Free-Form Text. Select Save & New.
 - Select New Field. Enter a label and set ID to "custrecord_yields". Ensure that the Type field is set to Free-Form Text. Select Save.

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType MapReduceScript
4   */
5  define(['N/search', 'N/record', 'N/email', 'N/runtime', 'N/error'],
6    function(search, record, email, runtime, error)
7    {
8      function handleErrorAndSendNotification(e, stage)
9      {
10         log.error('Stage: ' + stage + ' failed', e);
11
12         var author = -5;
13         var recipients = 'notify@example.com';
14         var subject = 'Map/Reduce script ' + runtime.getCurrentScript().id + ' failed for stage: ' + stage;
15         var body = 'An error occurred with the following information:\n' +
16                   'Error code: ' + e.name + '\n' +
17                   'Error msg: ' + e.message;
18
19         email.send({
20           author: author,
21           recipients: recipients,

```

```

22         subject: subject,
23         body: body
24     });
25 }
26
27 function handleErrorIfAny(summary)
28 {
29     var inputSummary = summary.inputSummary;
30     var mapSummary = summary.mapSummary;
31     var reduceSummary = summary.reduceSummary;
32
33     if (inputSummary.error)
34     {
35         var e = error.create({
36             name: 'INPUT_STAGE_FAILED',
37             message: inputSummary.error
38         });
39         handleErrorAndSendNotification(e, 'getInputData');
40     }
41
42     handleErrorInStage('map', mapSummary);
43     handleErrorInStage('reduce', reduceSummary);
44 }
45
46 function handleErrorInStage(stage, summary)
47 {
48     var errorMsg = [];
49     summary.errors.iterator().each(function(key, value){
50         var msg = 'Failure to accept payment from customer id: ' + key + '. Error was: ' + JSON.parse(value).message
51 + '\n';
52         errorMsg.push(msg);
53         return true;
54     });
55     if (errorMsg.length > 0)
56     {
57         var e = error.create({
58             name: 'RECORD_TRANSFORM_FAILED',
59             message: JSON.stringify(errorMsg)
60         });
61         handleErrorAndSendNotification(e, stage);
62     }
63 }
64
65 function createSummaryRecord(summary)
66 {
67     try
68     {
69         var seconds = summary.seconds;
70         var usage = summary.usage;
71         var yields = summary.yields;
72
73         var rec = record.create({
74             type: 'customrecord_summary',
75         });
76
77         rec.setValue({
78             fieldId: 'name',
79             value: 'Summary for M/R script: ' + runtime.getCurrentScript().id
80         });
81
82         rec.setValue({
83             fieldId: 'custrecord_time',
84             value: seconds
85         });
86
87         rec.setValue({
88             fieldId: 'custrecord_usage',
89             value: usage
90         });
91
92         rec.setValue({
93             fieldId: 'custrecord_yields',
94             value: yields
95         });
96     }
97 }

```

```

94         rec.save();
95     }
96     catch(e)
97     {
98         handleErrorAndSendNotification(e, 'summarize');
99     }
100 }
101
102 function applyLocationDiscountToInvoice(recordId)
103 {
104     var invoice = record.load({
105         type: record.Type.INVOICE,
106         id: recordId,
107         isDynamic: true
108     });
109
110     var location = invoice.getText({
111         fieldId: 'location'
112     });
113
114     var discount;
115     if (location === 'East Coast')
116         discount = 'Eight Percent';
117     else if (location === 'West Coast')
118         discount = 'Five Percent';
119     else if (location === 'United Kingdom')
120         discount = 'Nine Percent';
121     else
122         discount = '';
123
124     invoice.setText({
125         fieldId: 'discountitem',
126         text: discount,
127         ignoreFieldChange : false
128     });
129     log.debug(recordId + ' has been updated with location-based discount.');
```

invoice.save();

```

130 }
131
132
133 function getInputData()
134 {
135     return search.create({
136         type: record.Type.INVOICE,
137         filters: [['status', search.Operator.IS, 'open']],
138         columns: ['entity'],
139         title: 'Open Invoice Search'
140     });
141 }
142
143 function map(context)
144 {
145     var searchResult = JSON.parse(context.value);
146     var invoiceId = searchResult.id;
147     var entityId = searchResult.values.entity.value;
148
149     applyLocationDiscountToInvoice(invoiceId);
150
151     context.write({
152         key: entityId,
153         value: invoiceId
154     });
155 }
156
157
158 function reduce(context)
159 {
160     var customerId = context.key;
161
162     var custPayment = record.transform({
163         fromType: record.Type.CUSTOMER,
164         fromId: customerId,
165         toType: record.Type.CUSTOMER_PAYMENT,
166         isDynamic: true

```

```

167     });
168
169     var lineCount = custPayment.getLineCount('apply');
170     for (var j = 0; j < lineCount; j++)
171     {
172         custPayment.selectLine({
173             sublistId: 'apply',
174             line: j
175         });
176         custPayment.setCurrentSublistValue({
177             sublistId: 'apply',
178             fieldId: 'apply',
179             value: true
180         });
181     }
182
183     var custPaymentId = custPayment.save();
184
185     context.write({
186         key: custPaymentId
187     });
188 }
189
190 function summarize(summary)
191 {
192     handleErrorIfAny(summary);
193     createSummaryRecord(summary);
194 }
195
196 return {
197     getInputData: getInputData,
198     map: map,
199     reduce: reduce,
200     summarize: summarize
201 };
202 });

```

SuiteScript 2.x Mass Update Script Type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Mass update scripts allow you to programmatically perform custom updates to fields that are not available through general mass updates. These scripts can perform complex calculations across multiple records, based on your script. You can run mass update scripts on demand, but you can't prioritize or schedule them for specific times. If you need to run a script at a specific time, look into using a scheduled or map/reduce script instead. For more information, see [SuiteScript 2.x Scheduled Script Type](#) and [SuiteScript 2.x Map/Reduce Script Type](#).

When you click Perform Update button on the Mass Update Preview Results page, the script runs on the server. You can't run a mass update script from within another type of script. You also cannot programmatically check the status of a mass update script or determine when the script started or ended.



Important: Keep the following things in mind when running a mass update script:

- When you run a custom mass update, it can trigger user event scripts if they're associated with the records you're updating.
- You can assign an audience to both mass update script deployments and mass updates. The script owner must make sure the audiences match. If the audiences don't match, the script won't run when users click the Perform Update button on the Mass Update page.
- To run a custom mass update, users need the right permission (Edit or Full) for the record type they're updating.
- Users must also have SuiteScript enabled in their accounts. (Administrators can go to Setup > Company > Enable Features. In the SuiteCloud subtab, check the Server SuiteScript box and the Client SuiteScript box.)

For information about scripting with mass update scripts, see [SuiteScript 2.x Mass Update Script Entry Points and API](#).

SuiteCloud Development Framework (SDF) lets you manage mass update scripts as part of your file-based customization projects. For information about SDF, see . The Copy to Account feature lets you copy a mass update script to another account. You'll find the Copy to Account option on the top right of each mass update script page. For information about Copy to Account, see the help topic [Copy to Account](#).

SuiteScript Analysis helps you understand when a script was installed and how it's performed over time. For more information, see the help topic [Analyzing Scripts](#).

Mass Update Script Sample

This code updates the probability field of all existing records to 61%.



Note: From the script deployment record, ensure that the **Applies To** field is set to **Opportunity**.

For help with writing scripts in SuiteScript 2.x, see [SuiteScript 2.x Hello World](#) and [SuiteScript 2.x Entry Point Script Creation and Deployment](#).

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType MassUpdateScript
4   */
5  define(['N/record'], (record) => {
6      function each(params) {
7          // Set the probability to 61%
8          let recOpportunity = record.load({
9              type: params.type,
10             id: params.id
11         });
12         recOpportunity.setValue('probability', 61);
13         recOpportunity.save();
14     }
15     return {
16         each: each
    
```

```

17 | };
18 | });

```

SuiteScript 2.x Mass Update Script Entry Points and API

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Script Entry Point	Description
<code>each(params)</code>	Defines the mass update script trigger point. Loops through all the records you want to update, so you can apply your logic to each one. See each(params) for parameters and syntax.

each(params)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Description	Iterates through each applicable record.
Returns	void
Since	Version 2016 Release 1

Parameters

Note: The `params` parameter is a JavaScript object. NetSuite automatically passes it to your script's entry point.

Parameter	Type	Description	Since
<code>params.id</code>	number	The ID of the record being processed by the mass update.	Version 2016 Release 1
<code>params.type</code>	string	The record type of the record being processed by the mass update.	Version 2016 Release 1

Syntax

Important: This code snippet shows the syntax for this entry point. This isn't a working example. For a full script sample, see [Mass Update Script Sample](#).

```

1 // Add additional code
2 ...
3 function each(params) {
4     // Set the probability to 61%
5     let recOpportunity = record.load({
6         type: params.type,
7         id: params.id
8     });
9     ...
10 // Add additional code

```

SuiteScript 2.x Portlet Script Type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Portlet scripts are run on the server and are rendered in the NetSuite dashboard. The following portlet types are supported:

- **Simple Form** – A data entry form that can include a submit button embedded into a portlet. This type supports the N/portlet module that can refresh and resize the portlet, as well as use record-level client script to implement validation. See the help topic [N/portlet Module](#).
- **Inline HTML** – An HTML-based portlet that is used to display free-form HTML such as images, Flash, and custom HTML.
- **Links and Indents** – A portlet that consists of rows of formatted content.
- **Simple List** – A standard list of user-defined column headers and rows.

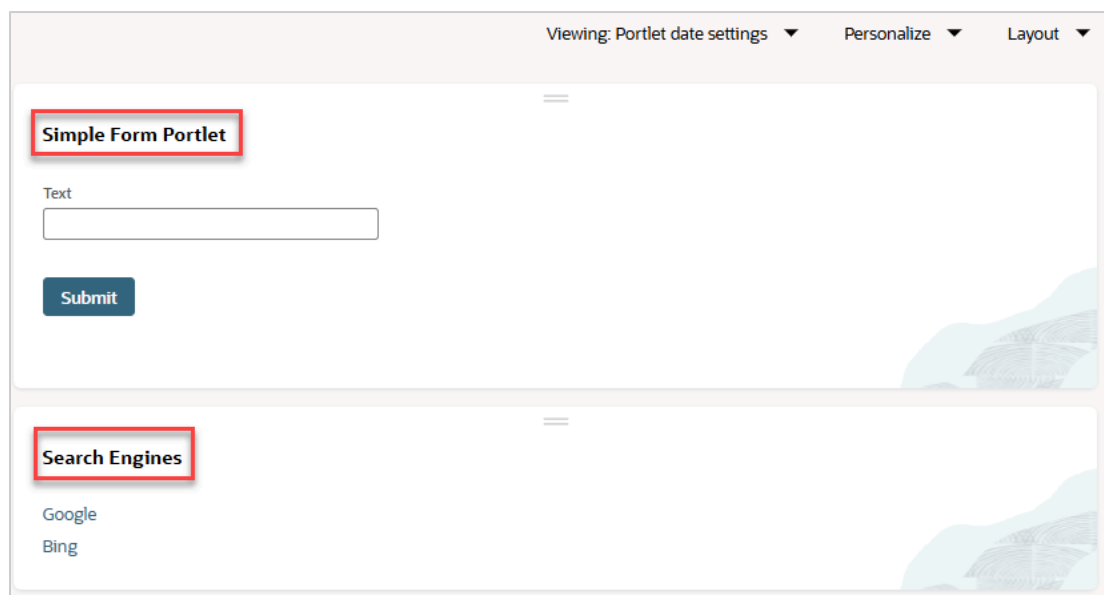
You can make the portlet script available as a SuiteApp portlet or as a custom portlet.

- **Dashboard SuiteApp Portlet** – You can designate that a portlet script should be used for a SuiteApp portlet. A SuiteApp portlet is a specialized type of custom portlet that provides direct access from users' dashboards to a SuiteApp installed in their account. This type of script is called a Dashboard SuiteApp portlet script. If a Dashboard SuiteApp portlet script is included in a SuiteApp, you can add a SuiteApp portlet to your dashboard from the SuiteApps tab of the Personalize Dashboard menu. For instructions, see the help topic [SuiteApp Portlets](#).

This type of portlet is supported for installed SuiteApps that include a dashboard component. A SuiteApp portlet script not only provides content for SuiteApp portlets. It also lets you include your choice of graphics as branding for the icons shown for SuiteApp portlets in the Personalize Dashboard window.

- **Custom Portlet** – To view content produced by a portlet script that is not intended for a SuiteApp portlet, you need to add a custom portlet to your dashboard and select the script in the portlet setup. For more information, see the help topics [Custom Portlets](#) and [Adding a Portlet to a Dashboard](#).

The following image shows a simple form portlet and a links portlet (labeled Search Engines) displayed on the NetSuite dashboard.



You can use SuiteCloud Development Framework (SDF) to manage portlet scripts as part of file-based customization projects. For information about SDF, see .

You can copy an individual portlet script to another account using the Copy to Account feature. Each portlet script page has a clickable Copy to Account option in the upper right corner. For information about Copy to Account, see the help topic [Copy to Account](#).

You can use SuiteScript Analysis to see when the script was installed and how it's performing. For more information, see the help topic [Analyzing Scripts](#).

See the following for more information about the Portlet Script type:

- [SuiteScript 2.x Portlet Script Reference](#)
 - [Creating and Deploying a Portlet Script](#)
 - [Guidelines for Creating a Dashboard SuiteApp Icon](#)
 - [Portlet Script Samples](#)
- [SuiteScript 2.x Portlet Script Entry Points and API](#)
- [render\(params\)](#)
- [Portlet Object](#)
 - [Portlet.addColumn\(options\)](#)
 - [Portlet.addEditColumn\(options\)](#)
 - [Portlet.addField\(options\)](#)
 - [Portlet.addLine\(options\)](#)
 - [Portlet.addRow\(options\)](#)
 - [Portlet.addRows\(options\)](#)
 - [Portlet.setSubmitButton\(options\)](#)
 - [Portlet.clientScriptFileId](#)
 - [Portlet.clientScriptModulePath](#)
 - [Portlet.html](#)
 - [Portlet.title](#)

SuiteScript 2.x Portlet Script Reference

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

To use a portlet script on the NetSuite dashboard, you need to create and deploy it first. For more information, see [Creating and Deploying a Portlet Script](#).

If you want to create a Dashboard SuiteApp Icon, see the [Guidelines for Creating a Dashboard SuiteApp Icon](#).

See [Portlet Script Samples](#) for a sample of each type of portlet script (simple form, HTML, links, and list).

Creating and Deploying a Portlet Script

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following is a basic process flow for creating and deploying a portlet script, including a Dashboard SuiteApp portlet script:

1. Create a script file and upload the file to your File Cabinet.
For more information about this process, see [SuiteScript 2.x Script Creation Process](#).
2. Create a script record for your portlet script. Ensure that the **Portlet Type** field is set to the correct portlet type used in your script.
For more information about this process, see [Creating a Script Record](#).
3. Create a script deployment record for your portlet script. For more information about the script deployment process, see [Methods of Deploying a Script](#).

If you're creating a script to be used for a SuiteApp portlet, check the **Dashboard SuiteApp** box. This option will let you access the portlet from the **SuiteApps** tab of **Personalize Dashboard**.



Note: After you have saved the script, you will not be able to clear the **Dashboard SuiteApp** box and make the portlet accessible from the **Standard Content** tab.

When you check the **Dashboard SuiteApp** box you can upload an image to the File Cabinet that represents the SuiteApp icon shown for the portlet in the **Personalize Dashboard** window. Note that only SVG images are supported for the icon. SVG is a vector format, so it assures perfect image scalability. For more information about icon guidelines, see [Guidelines for Creating a Dashboard SuiteApp Icon](#).

The following image indicates where the Dashboard fields are located:

The screenshot shows the 'Script Deployment' form. At the top are 'Save' and 'Cancel' buttons. Below is the 'Script' section with the name 'Employee Net Pay Variances'. The 'Title' field is labeled with an asterisk and contains 'Employee Net Pay Variances 2'. The 'ID' field is empty. The 'Deployed' checkbox is checked. Below this, the 'Dashboard SuiteApp' checkbox is checked and highlighted with a red box. To the right of this checkbox is an edit icon. Below the checkbox is the 'Icon' field, also with an edit icon, which contains the text 'oracle_logo.svg' and a dropdown arrow.



Note: It's best to avoid deleting script parameters for portlet scripts. Deleting these parameters can cause inconsistencies if the custom portlet is part of a published dashboard.

Guidelines for Creating a Dashboard SuiteApp Icon

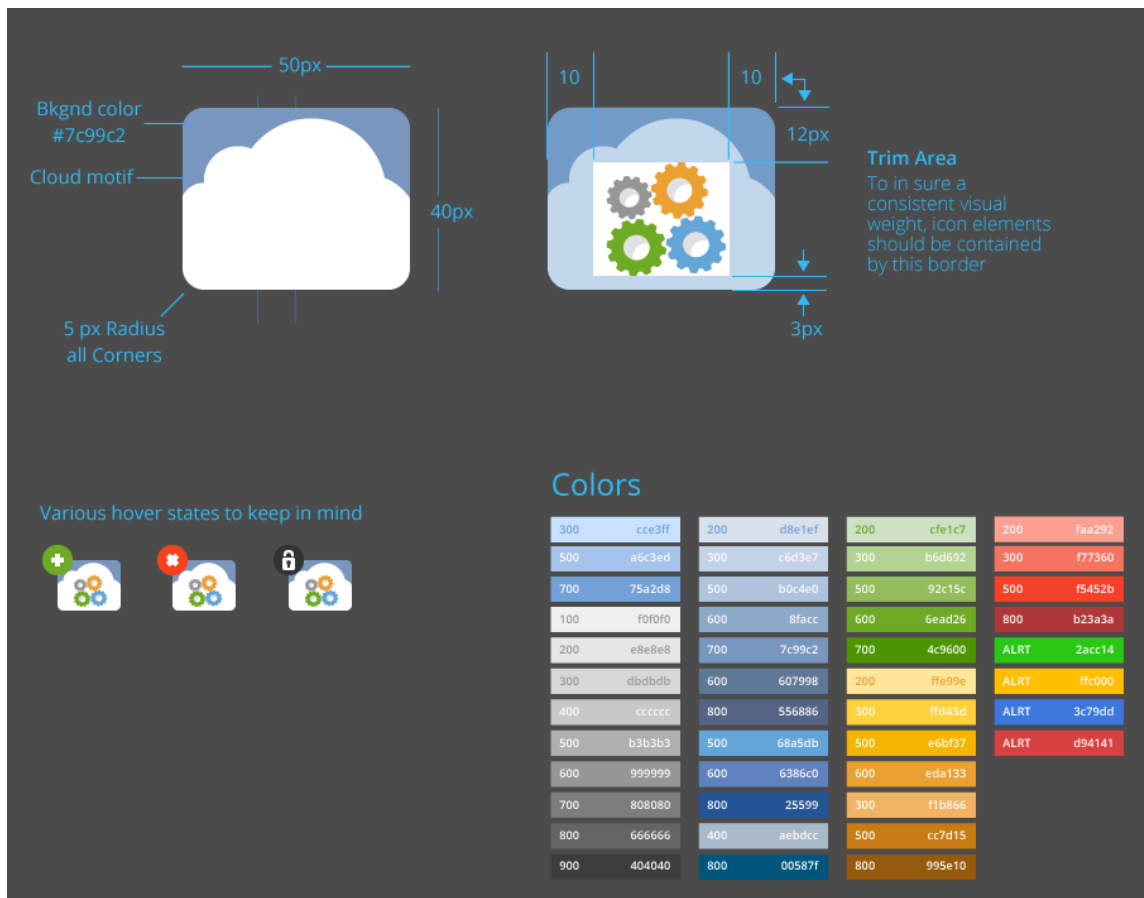
Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

To fit in with other dashboard icons, Dashboard SuiteApp icons need to follow some visual guidelines. The following guidelines preserve the visual characteristics of Dashboard SuiteApp icons:

- Dashboard SuiteApp icons use a unique background with a cloud motif. This background ensures a common size and shape for all Dashboard SuiteApp icons and provides a cloud motif that serves as an additional modifier. Click [here](#) to download the SVG file of the icon background.

- You should use a restricted color palette. This palette is a major contributor to the character of the icons used in NetSuite. Restrict your colors to this palette whenever possible. If additional colors are required, add them as special exceptions. See the image below for more information about this color palette.
- Graphical elements should be more geometrical than illustrative. Avoid using complex and irregular shapes. Try to reduce elements in your composition to their most primitive geometry.
- To add depth and detail, Dashboard SuiteApp icons use a simulated light source. This light source is evidenced by a cast shadow that proceeds down and to the right of elements in the composition at a 45 degree angle. Cast shadows are rendered with a black fill set to 10% opacity.
- Your icon image must be saved as an SVG file.

The following image shows the guidelines in use:



Portlet Script Samples

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

- [Simple Form Portlet Script Sample](#)
- [Inline HTML Portlet Script Sample](#)
- [Links and Indents Portlet Script Sample](#)
- [Simple List Portlet Script Sample](#)

For help with writing scripts in SuiteScript 2.x, see [SuiteScript 2.x Hello World](#) and [SuiteScript 2.x Entry Point Script Creation and Deployment](#).

Simple Form Portlet Script Sample

This sample creates a portlet that includes a simple form with a text field and a submit button.

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Portlet
4   */
5
6  define([], function() {
7      function render(params) {
8          var portlet = params.portlet;
9          portlet.title = 'Simple Form Portlet';
10
11          var fld = portlet.addField({
12              id: 'text',
13              type: 'text',
14              label: 'Text'
15          });
16
17          fld.updateLayoutType({
18              layoutType: 'normal'
19          });
20
21          fld.updateBreakType({
22              breakType: 'startcol'
23          });
24
25          portlet.setSubmitButton({
26              url: 'http://httpbin.org/post',
27              label: 'Submit',
28              target: '_top'
29          });
30      }
31
32      return {
33          render: render
34      };
35  });

```

Inline HTML Portlet Script Sample

This sample creates a portlet that displays simple HTML.

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Portlet
4   */
5
6  define([], function() {
7      function render(params) {
8          params.portlet.title = 'My Portlet';
9          var content = '<td><span><b>Hello!!!</b></span></td>';
10          params.portlet.html = content;
11      }
12
13      return {
14          render: render
15      };
16  });

```

Links and Indents Portlet Script Sample

This sample creates a portlet with two links.

```

1  /**

```

```

2  * @NApiVersion 2.x
3  * @NScriptType Portlet
4  */
5
6  define([], function() {
7      function render(params) {
8          var portlet = params.portlet;
9          portlet.title = 'Search Engines';
10         portlet.addLine({
11             text: 'NetSuite',
12             url: 'http://www.netsuite.com/'
13         });
14         portlet.addLine({
15             text: 'Oracle',
16             url: 'http://www.oracle.com/'
17         });
18     }
19
20     return {
21         render: render
22     };
23 });

```

Simple List Portlet Script Sample

This sample creates a portlet with a simple list.



Important: This sample references a custom entity field with the ID `custentity_multiselect`. Before attempting to use this sample, you must either create a field with that ID or edit the sample so that it references an existing custom entity field in your account. To create a custom entity field, see the help topic [Creating Custom Entity Fields](#).

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Portlet
4   */
5
6  define(['/N/search'], function(search) {
7      function render(params) {
8          // Check if the portlet is placed in the center column
9          var isDetail = (Number(params.column) === 2);
10         var portlet = params.portlet;
11
12         // Set the portlet title based on the column it is placed in
13         portlet.title = isDetail ? "My Detailed List" : "My List";
14
15         // Add columns to the portlet
16         portlet.addColumn({
17             id: 'internalid',
18             type: 'text',
19             label: 'ID',
20             align: 'LEFT'
21         });
22
23         portlet.addColumn({
24             id: 'companyname',
25             type: 'text',
26             label: 'Company Name',
27             align: 'LEFT'
28         });
29
30         // If the portlet is in the center column, add extra columns
31         if (isDetail) {
32             portlet.addColumn({
33                 id: 'email',
34                 type: 'text',
35                 label: 'E-mail',
36                 align: 'LEFT'

```

```

37     });
38     portlet.addColumn({
39         id: 'custentity_multiselect',
40         type: 'text',
41         label: 'Multiselect',
42         align: 'LEFT'
43     });
44 }
45
46 // Create a filter for the search to find customers with non-empty email field
47 var filter = search.createFilter({
48     name: 'email',
49     operator: search.Operator.ISNOTEMPTY
50 });
51
52 // Create a customer search with the specified columns
53 var customerSearch = search.create({
54     type: 'customer',
55     filters: filter,
56     columns: ['internalid', 'companyname', 'email', 'custentity_multiselect']
57 });
58
59 // Set the number of rows to show in the portlet based on the column it is placed in
60 var count = isDetail ? 15 : 5;
61
62 // Run the search and add the results to the portlet
63 customerSearch.run().each(function(result) {
64     portlet.addRow(result.getAllValues());
65     return --count > 0;
66 });
67 }
68
69 return {
70     render: render
71 };
72 });

```

SuiteScript 2.x Portlet Script Entry Points and API

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Script Entry Point	Description
render(params)	This function runs when the Portlet script is triggered and displays a portlet object on the NetSuite dashboard.

Portlet objects are used to encapsulate scriptable dashboard portlets. A portlet object is defined in a [Portlet Object](#), which defines columns, fields, lines, and rows, along with parameters and an optional submit button. For more information about the Portlet Object, see the following help topics:

- [render\(params\)](#)
- [Portlet Object](#)
 - [Portlet.addColumn\(options\)](#)
 - [Portlet.addEditColumn\(options\)](#)
 - [Portlet.addField\(options\)](#)
 - [Portlet.addLine\(options\)](#)
 - [Portlet.addRow\(options\)](#)
 - [Portlet.addRows\(options\)](#)
 - [Portlet.setSubmitButton\(options\)](#)

- [Portlet.clientScriptFileId](#)
- [Portlet.clientScriptModulePath](#)
- [Portlet.html](#)
- [Portlet.title](#)

render(params)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Description	Defines the portlet script trigger point.
Returns	void
Since	Version 2015 Release 2

Parameters

Note: The params parameter is a JavaScript object. NetSuite automatically passes it to your script entry point. The values for params are read-only.

Parameter	Type	Description	Since
params.portlet	Portlet Object	The Portlet object used for rendering. The availability of Portlet members depends on the type of Portlet that is passed in. For more information, see the following: ■ Simple Form Portlet Object Members ■ Inline HTML Portlet Object Members ■ Links and Indents Portlet Object Members ■ Simple List Portlet Object Members	Version 2015 Release 2
params.column	string (read-only)	The column index for the portlet on the dashboard. The index is a string representation of one of the following numeric values: 1. left column 2. center column 3. right column	Version 2015 Release 2
params.entity	string (read-only)	The customer ID for the selected customer.	Version 2015 Release 2

Portlet Object

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Portlet objects are used to encapsulate scriptable dashboard portlets. NetSuite automatically passes them to the render(params) entry point. For more information, see [render\(params\)](#).

Simple Form Portlet Object Members

These options are only for Form Portlet objects.

Member Type	Name	Return Type / Value Type	Description
Method	Portlet.addField(options)	serverWidget.Field	Adds a field to the form.
	Portlet.setSubmitButton(options)	serverWidget.Button	Adds a submit button to the form.
Property	Portlet.clientScriptFileId	number or string	The script file ID to be used in the portlet. Can be in either numerical or string format.
	Portlet.clientScriptModulePath	string	The script path to be used in the portlet.
	Portlet.title	string	The title of the portlet.

Inline HTML Portlet Object Members

These options are only for HTML Portlet objects.

Member Type	Name	Return Type / Value Type	Description
Property	Portlet.html	string	The complete HTML contents of the portlet.
	Portlet.title	string	The title of the portlet.

Links and Indents Portlet Object Members

These options are only for Links Portlet objects.

Member Type	Name	Return Type / Value Type	Description
Method	Portlet.addLine(options)	Object	Adds a line to the portlet.
Property	Portlet.title	string	The title of the portlet.

Simple List Portlet Object Members

These options are only for List Portlet objects.

Member Type	Name	Return Type / Value Type	Description
Method	Portlet.addColumn(options)	serverWidget.ListColumn	Adds a column to the portlet.
	Portlet.addEditColumn(options)	serverWidget.ListColumn	Adds an Edit or Edit/View column to the portlet.
	Portlet.addRow(options)	Object	Adds a row to the portlet.
	Portlet.addRows(options)	Object	Adds multiple rows to the portlet.

Member Type	Name	Return Type / Value Type	Description
Property	Portlet.title	string	The title of the portlet.

Portlet.addColumn(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a list column to the portlet.
Returns	serverWidget.ListColumn
Entry Point	render(params)
Since	2016.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.id	string	required	The internal ID of this column. Keep the internal ID in lowercase and space-free.	2016.2
options.label	string	required	The label of this column.	2016.2
options.type	string	required	The field type for this column. For more information about possible values, see the help topic serverWidget.FieldType .	2016.2
options.align	string	optional	The layout justification for this column. For more information about possible values, see the help topic serverWidget.LayoutJustification .	2016.2

Syntax

The following code snippet shows the syntax for this member. Note that this isn't a working example. For a complete script example, see [SuiteScript 2.x Portlet Script Type](#).

```

1  ...
2  var newColumn = params.portlet.addColumn({ id: 'column1', label: 'Text', type: serverWidget.FieldType.TEXT, align: serverWidget.LayoutJustification.RIGHT
3  });
4  ...

```

Portlet.addEditColumn(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds an Edit or Edit/View column to the portlet.
---------------------------	--

Returns	serverWidget.ListColumn
Entry Point	render(params)
Since	2016.2

Parameters

 **Note:** The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.column	string	required	The internal ID of the column to the left of which the Edit/View column is added.	2016.2
options.showHrefCol	string	optional	If set, it must contain a name of a column. This column's value decides if the View/Edit link is clickable for a given data row (T=clickable, F=non-clickable).	2016.2
options.showView	boolean	optional	If true, then an Edit/View column is added. Otherwise, only an Edit column is added. The default setting is false.	2016.2
options.link	string	optional	The Edit/View base link. (For example: /app/common/entity/employee.nl) The complete link is formed like this: <link>?<linkParamName>=<row data from linkParam>. (For example: /app/common/entity/employee.nl?id=123)	2019.2
options.linkParam	string	optional	The internal ID of the field in the row data where to take the parameter from. The default value is the value set in the options.column parameter.  Tip: In most cases, the value to use here is internalid	2019.2
options.linkParamName	string	optional	The name of the parameter. The default value is id.	2019.2

Syntax

The following code snippet shows the syntax for this member. Note that this isn't a working example. For a complete script example, see [SuiteScript 2.x Portlet Script Type](#).

```

1 ...
2 var portlet = params.portlet;
3 portlet.title = 'My Detailed List';
4 portlet.addColumn({ id: 'internalid', type: serverWidget.FieldType.TEXT, label: 'Number', align: serverWidget.LayoutJustification.LEFT
5 });
6 portlet.addColumn({ id: 'entityid', type: serverWidget.FieldType.TEXT, label: 'Name', align: serverWidget.LayoutJustification.LEFT
7 });
8 portlet.addEditColumn({ column: 'entityid', showView: true, showHrefCol: true, link: '/app/common/entity/custjob.n
  1', linkParam: 'internalid', linkParamName: 'id',

```

```

9  });
10 ...

```

Portlet.addField(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a field to the form.
Returns	serverWidget.Field
Entry Point	render(params)
Since	2016.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.id	string	required	The internal ID of the field. Keep the internal ID in lowercase, space-free, and prefix it with custpage if you're adding it to an existing page. For example, if you add a field that appears as Purchase Details , the field internal ID should be something similar to custpage_purchasedetails or custpage_purchase_details.	2016.2
options.label	string	required	The label for this field.	2016.2
options.type	string	required	The field type for the field. For more information about possible values, see the help topic serverWidget.FieldType .	2016.2
options.source	string	optional	The internal ID or script ID of the source list for this field if it is a select (List/Record) or multi-select field. Note: For radio fields, the source parameter needs to include the field's internal ID. Important: Once you've created a select or multi-select field from a record or list, you can't add more values using Field.addSelectOption(options). The select values are determined by the source record or list.	2016.2

Syntax

The following code snippet shows the syntax for this member. Note that this isn't a working example. For a complete script example, see [SuiteScript 2.x Portlet Script Type](#).

```

1 | ...
2 | var newField = params.portlet.addField({ id: 'textfield', type: serverWidget.FieldType.TEXT, label: 'text'
3 | });
4 | ...

```

Portlet.addLine(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a line to the portlet.
Returns	Object
Entry Point	render(params)
Since	2016.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.text	string	required	The text for the line.	2016.2
options.url	string	optional	The URL link.	2016.2
options.align	number	optional	The justification for the line. This value indicates the number of spaces to indent. Keep the align value between 0 and 5.	2016.2

Syntax

The following code snippet shows the syntax for this member. Note that this isn't a working example. For a complete script example, see [SuiteScript 2.x Portlet Script Type](#).

```

1 | ...
2 | params.portlet.addLine({ text: 'NetSuite', url: 'http://www.netsuite.com', align: 4
3 | });
4 | ...

```

Portlet.addRow(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a row to the portlet.
Returns	Object
Entry Point	render(params)
Since	2016.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.row	search.Result object	required	A row that consists of either a search. Result, or name/value pairs. Each pair should contain the value for the corresponding Column object in the list.	2016.2

Syntax

The following code snippet shows the syntax for this member. Note that this isn't a working example. For a complete script example, see [SuiteScript 2.x Portlet Script Type](#).

```

1  ...
2  params.portlet.addRow({ row: { columnid1: 'value1', columnid2: 'value2' }
3  });
4  ...

```

Portlet.addRow(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds multiple rows to the portlet.
Returns	Object
Entry Point	render(params)
Since	2016.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.rows	search.Result[] object[]	required	An array of rows that consist of either a search. Result array, or an array of name/value pairs. Each pair should contain the value for the corresponding Column object in the list.	2016.2

Syntax

The following code snippet shows the syntax for this member. Note that this isn't a working example. For a complete script example, see [SuiteScript 2.x Portlet Script Type](#).

```

1  ...
2  params.portlet.addRows({ rows: [{ columnid1: 'value1', columnid2: 'value2' }, { columnid1: 'value2', columnid2: 'value3' }]
3  });

```

4 | ...

Portlet.setSubmitButton(options)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Method Description	Adds a submit button to the form.
Returns	serverWidget.Button
Entry Point	render(params)
Since	2016.2

Parameters

Note: The options parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
options.url	string	required	The URL that the form posts data to.	2016.2
options.label	string	optional	The button label.	2016.2
options.target	string	optional	The target attribute of the form element, if it is different from the portlet's own embedded iframe. Supported values include standard HTML target attributes such as <code>_top</code> , <code>_parent</code> , and <code>_blank</code> . It can also be set to frame names and the NetSuite-specific identifier <code>_hidden</code> . Setting this value to <code>_hidden</code> allows submission to a backend that returns results to a hidden child iframe within the portlet's embedded iframe.	2016.2

Syntax

The following code snippet shows the syntax for this member. Note that this isn't a working example. For a complete script example, see [SuiteScript 2.x Portlet Script Type](#).

```

1 | ...
2 | params.portlet.setSubmitButton({ url: 'http://httpbin.org/post', label: 'Submit', target: '_top'
3 | });
4 | ...

```

Portlet.clientScriptFileId

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The script file ID to be used in the portlet.
Type	number or string
Entry Point	render(params)

Since	2016.2
--------------	--------

Errors

Error Code	Thrown If
PROPERTY_VALUE_CONFLICT	You tried to set this value, but the <code>Portlet.clientScriptModulePath</code> property value is already set. For more information, see Portlet.clientScriptFileId .

Syntax

The following code snippet shows the syntax for this member. Note that this isn't a working example. For a complete script example, see [SuiteScript 2.x Portlet Script Type](#).

```

1  ...
2  params.portlet.clientScriptFileId = 32;
3  ...

```

Portlet.clientScriptModulePath

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The script path to be used in the portlet.
Type	string
Entry Point	render(params)
Since	2016.2

Errors

Error Code	Thrown If
PROPERTY_VALUE_CONFLICT	You tried to set this value, but the <code>Portlet.clientScriptFileId</code> is already set. For more information, see Portlet.clientScriptFileId .

Syntax

The following code snippet shows the syntax for this member. Note that this isn't a working example. For a complete script example, see [SuiteScript 2.x Portlet Script Type](#).

```

1  ...
2  params.portlet.clientScriptModulePath = '/SuiteScripts/clientScript.js';
3  ...

```

Portlet.html

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The complete HTML contents of the portlet.
-----------------------------	--

Type	string
Entry Point	render(params)
Since	2016.2

Syntax

The following code snippet shows the syntax for this member. Note that this isn't a working example. For a complete script example, see [SuiteScript 2.x Portlet Script Type](#).

```
1 ...
2 params.portlet.html = htmlcontents;
3 ...
```

Portlet.title

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Property Description	The title of the portlet.
Type	string
Entry Point	render(params)
Since	2016.2

Syntax

The following code snippet shows the syntax for this member. Note that this isn't a working example. For a complete script example, see [SuiteScript 2.x Portlet Script Type](#).

```
1 ...
2 params.portlet.title = 'My Portlet';
3 ...
```

SuiteScript 2.x RESTlet Script Type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

A RESTlet is a SuiteScript you can call from outside NetSuite or another script. It only runs when called, and sometimes returns a value.

RESTlets help you pull data into NetSuite from other systems or send data out. You can also use RESTlets with other scripts to customize how NetSuite pages work.

For more details, see the following:

- [SuiteScript 2.x RESTlet Reference](#)

- [SuiteScript 2.x RESTlet Script and Request Examples](#)
- [SuiteScript 2.x RESTlet Script Entry Points and API](#)

You can manage RESTlet scripts with SuiteCloud Development Framework (SDF) in file-based projects. To learn more, see [. Use Copy to Account](#) to send a RESTlet script to another account. Just click Copy to Account in the upper right on the script page. For details, see the help topic [Copy to Account](#).

Use SuiteScript Analysis to check when a script was installed and how it's performed. For more, see the help topic [Analyzing Scripts](#).

SuiteScript 2.x RESTlet Reference

📘 Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

A RESTlet is a SuiteScript you can call from an external app or another SuiteScript. Depending on how you write it, it might also return data to the caller.

A RESTlet can do anything you can build with SuiteScript. In general, you can use a RESTlet to:

- Bring data into NetSuite or change NetSuite data from another system. In this way, RESTlets are an alternative to NetSuite's SOAP web services.
- Customize how NetSuite pages or features work. Here, RESTlets act as an alternative to other scripts, like Suitelets. Unlike a Suitelet, a RESTlet can send plain text or JSON data back to the caller.

Deploying and calling a RESTlet

When you build a RESTlet, you'll need to learn how to deploy and call it—these steps are a bit different than other script types. You'll also want to know about authentication, governance, and security for RESTlets.

Deploying a RESTlet. To use a RESTlet, follow the normal process for entry point scripts—create a script record and deployment record for your RESTlet script file. For details, see [Deploying a RESTlet](#).

Calling a RESTlet. When you save a deployment record, NetSuite gives you a URL you can use to call your RESTlet. Since RESTlets run only when called, this URL is key. See [Identifying a RESTlet in a Call](#) for more.

Once your RESTlet is live, you can call it with delete, get, post, or put. Depending on the method, you might put input in the URL or in the request body. Just make sure your script has an entry point that matches the method you use. For more, see [Selecting an HTTP/HTTPS Method for Calling a RESTlet](#).

For most RESTlet calls, add a Content-Type header so NetSuite knows how to read your input and format the response. See [Creating a Content-Type Header](#) for more.

RESTlet Authentication. A key benefit of RESTlets is that NetSuite always requires authentication. If the RESTlet call comes from outside NetSuite, you'll need to add an authentication header. Learn more [RESTlet Authentication](#).

Governance and Security. With RESTlets, keep governance and security rules in mind—they're a bit different for this script type. For more, see [RESTlet Governance and Security](#).

Debugging a RESTlet. You can debug RESTlet code just like any other SuiteScript—use the NetSuite Debugger or the debug client in SuiteCloud IDE. The client's available once you start a debug session. See the help topic [Debugging a RESTlet](#) for more.

Deploying a RESTlet

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Before you use a RESTlet, follow the usual script guidelines—make sure the script's set up right and create a script record.

To format your RESTlet correctly, make sure:

- The script must have the correct SuiteScript 2.x structure. For example, your script needs at least one entry point and function that matches the RESTlet script type. For more information about how to structure a SuiteScript 2.x script, see [Required Structure for Entry Point Scripts](#). The entry points you use decide how the RESTlet can be called. To learn more, see [SuiteScript 2.x RESTlet Script Entry Points and API](#).
- Your script needs the right JSDoc tags—at minimum, add `@NScriptType RESTlet` (any capitalization works). For details, see [Required JSDoc Tags for Entry Point Scripts](#).

Before you call a RESTlet, upload the script file to your File Cabinet and create both a script record and script deployment record. For help, see [SuiteScript 2.x Record-Level Script Deployments](#).

When creating these records for a RESTlet, be aware of the following:

- Enter clear info in your script record and deployment record ID fields. The system uses your text to create IDs, which can help you recognize the RESTlet in other scripts.
- Unlike other script types, RESTlets aren't tied to any specific record type—they're available on their own.
- The deployment record's Status field has "Released" and "Testing" options. The Status must be "Released" before you can call the RESTlet from outside NetSuite.
- Saving the deployment record creates a partial and full URL for calling your RESTlet—you'll see these on the page.

Script	Status
TP RESTlet Script Sample	Testing
Title	Log Level
TP RESTlet Script Sample	Debug
ID	URL
customdeploy1	/app/site/hosting/restlet.nl?script=619&deploy=1
☒ Deployed	External URL
	https://123456.restlets.api.netsuite.com/app/site/hosting/restlet.nl?script=619&deploy=1

If you use the full URL in an integration, be sure to add logic that finds the RESTlet domain dynamically. See [Identifying a RESTlet in a Call](#) for more info.

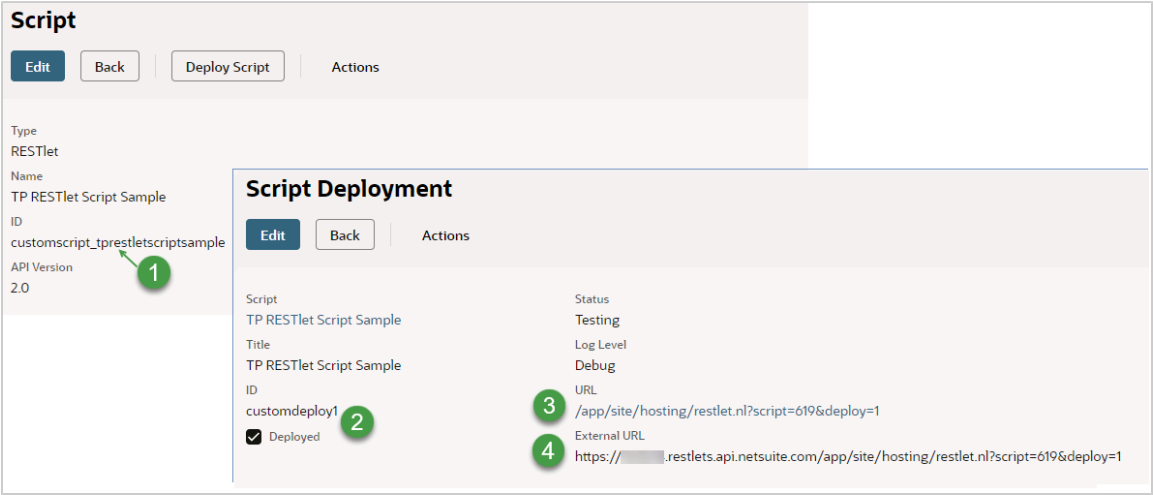
Identifying a RESTlet in a Call

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Before you call a RESTlet, you'll need to know how to identify it, which depends on whether you're calling it from inside or outside NetSuite. Usually, you'll use values from the script deployment record, and sometimes the script record ID too.

Internal Versus External Calls

The script and deployment records have everything you need to identify a RESTlet. The exact values to use depend on where the call comes from.



Source of the Call	Identify the RESTlet with:
An external client	A full URL, similar to the one shown on the script deployment record, in the External URL field (callout 4). For details, see Dynamically Generating a Full URL .
A client SuiteScript with an active session in the same NetSuite account where the RESTlet is deployed	Either of the following: ■ The partial URL shown on the script deployment record, in the URL field (callout 3). For an example, check out Example of a Client Script that Calls a RESTlet. ■ A URL generated by using the N/url module. To build the URL this way, use the script ID (from the script record) and the deployment ID (from the deployment record). For an example, check out Example of a Suitelet that Calls a RESTlet.
A server SuiteScript in the same NetSuite account where the RESTlet is deployed	A full URL, similar to the one shown on the script deployment record, in the External URL field callout (4). This URL must be dynamically generated, in one of the following ways: ■ By using the REST roles service. For details, see the help topic The REST Roles Service. ■ A URL generated by using the N/url module. To generate the URL this way, you need the script ID, which is viewable on the script record in the ID field (callout 1), in combination with the deployment ID, which is viewable on the script deployment record, in its ID field (callout 2). For an example, see Example of a Suitelet that Calls a RESTlet.
A server SuiteScript in a different NetSuite account from where the RESTlet is deployed	A full URL, similar to the one shown on the script deployment record, in the External URL field (callout 4). This URL must be dynamically generated by using the REST roles service. For details, see Dynamically Generating a Full URL .

 **Note:** If you are calling a RESTlet by using the delete or get method, you must extend the URL to include any input data that is required by the RESTlet's logic. For details, see [Selecting an HTTP/HTTPS Method for Calling a RESTlet](#).

Dynamically Generating a Full URL

When you save the deployment record, NetSuite gives you the full URL in the **External URL** field. But don't hard-code this URL—add logic to build the RESTlet domain dynamically instead.

You can do this in a couple of ways:

- To call a RESTlet from an external source, use NetSuite's roles service. For details on using this service, see the help topic [The REST Roles Service](#).
- To call a RESTlet from within NetSuite, you can use the N/url module. With this approach, you provide the ID values from the script record and script deployment record. For an example, see [Example of a Suitelet that Calls a RESTlet](#).

Note: Since 2017.2, you can use account-specific domains for RESTlets at .restlets.api.netsuite.com. The old data center-specific domains still work, too. For more info, see the help topic [URLs for Account-Specific Domains](#).

Selecting an HTTP/HTTPS Method for Calling a RESTlet

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

When you call a RESTlet, you can use one of four HTTP/HTTPS methods: delete, get, post, or put. Use the N/https module to connect SuiteScript scripts, RESTlets, and SuiteTalk REST endpoints without needing to reauthenticate. Use [https.requestRestlet\(options\)](#) or [https.requestSuiteTalkRest\(options\)](#) to send HTTPS requests to a RESTlet or REST endpoint.

When you call a RESTlet, keep these things in mind:

- A call's method must match an entry point defined in the RESTlet
- Each http/https method is dependent on the entry points within the RESTlet
- Input data is handled differently by each http method

A Call's Method Must Match an Entry Point

Each HTTP/HTTPS method (delete, get, post, put) matches a RESTlet entry point. Your call must use the method that's defined in the RESTlet's interface, or it won't work.

Here's an example: A Suitelet uses the https.get method to call a RESTlet. Since the RESTlet's interface includes a get entry point, the call works.

```
// Here, the Suitelet calls a RESTlet
var response = https.get({url: url, headers: headers});

// In this function, the RESTlet retrieves
// a standard NetSuite record.

function _get(context) {
    doValidation([context.recordtype, context.id], ['recordtype', 'id', 'GET']);
    return record.load({
        type: context.recordtype,
        id: context.id
    });
}

return {
    get : _get,
};
```

HTTP/HTTPS Methods are Dependent on RESTlet Script Entry Points

A RESTlet doesn't always have to follow the normal rules for HTTP/HTTPS methods. For example, you could write a get entry point that accepts data to post (though that's not best practice).

The following http methods are supported for RESTlets: delete, get, post, and put. These methods are defined at the following link: <https://www.w3.org/Protocols/rfc2616/rfc2616-sec9.html>.

Keep in mind, a RESTlet's behavior depends on how you write the script and might not match the intended behavior of the HTTP method. For example, you could have a get entry point that creates data or a post entry point that doesn't require input—but you should not do that

Input Data Is Handled Differently by Different HTTP Methods

RESTlet entry points handle input parameters differently, depending on which one you use.

How you pass input to a RESTlet depends on the HTTP method:

- For delete and get, put arguments in the URL when you call the RESTlet.
- For post and put, add arguments to the request body, using JSON or plain text.

HTTPS Methods for Secure Communication Between SuiteScripts and REST Endpoints

SuiteScript scripts, RESTlets, and SuiteTalk REST endpoints now share authentication from an existing session. Before, you had to log in again if you called one script from another. Now, the N/https methods let you skip reauthentication:

- [https.requestRestlet\(options\)](#) – This method sends an https request to a RESTlet and returns the response. Authentication headers are automatically added. The RESTlet endpoint identified by its script ID is passed as a parameter. The RESTlet will run with the same privileges as the calling script.
- [https.requestSuiteTalkRest\(options\)](#) – This method sends an https request to a SuiteTalk REST endpoint and returns the response. Authentication headers are automatically added. The REST endpoint identified by its URL is passed as a parameter.

This new method means you don't have to make a Suitelet available without login or set it to execute as Administrator just to pass information between SuiteScripts, RESTlets, or REST APIs.

 **Note:** These methods are just for calling RESTlets within NetSuite, not from outside applications.

Creating a Content-Type Header

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Depending on how you write your RESTlet, you might need to send a request body—and your RESTlet might return data you'll want to use.

If your RESTlet needs a request body, add a Content-Type header for the right format.

 **Note:** If you leave out a required Content-Type header, your request will fail with a 415: Unsupported Media Type error.

The following are suggested Content-Type headers:

- application/json – used for RESTlets that require request bodies that map values to fields.

- `application/xml` – used for RESTlets that are invoked through the GET method.
- `text/plain` – used for RESTlets that require limited or simple input.

Remember, Content-Type headers are case-sensitive—make sure you type them exactly as shown:

- Content-Type: `application/json`
- Content-Type: `application/xml`
- Content-Type: `text/plain`

For an example of a shell script that generates a Content-Type header, see [Example of a Shell Script that Calls a RESTlet](#).

If your RESTlet returns data, keep the following points in mind:

- If your RESTlet uses a request body and its response matches that format, the returned data will follow that same format.
- If you want the data format to be different from the request body, set the format using `restlet.createResponse(options)` when you return data.
- If there's no request body and your RESTlet returns data, the default is plain text.

SuiteScript 2.x RESTlet Error Handling

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Error handling for SuiteScript 2.x RESTlets falls into three categories: Supported http/https success codes, http/https error codes, and SuiteScript errors.

HTTP/HTTPS Success Codes

NetSuite mainly uses the **200 OK** status code for RESTlets. This means the request was successful, but it doesn't always mean everything worked as you planned. Sometimes, the RESTlet script catches a SuiteScript error but still returns **200 OK**—look for error details in the response body.

i Note: SuiteScript sometimes supports the 206 Partial Content status code. To learn more, see <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status/206>.

HTTP/HTTPS Error Codes

NetSuite supports these HTTP/HTTPS error codes for RESTlets. For more on these codes, see <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>.

Code	Description
302 Moved Temporarily	Your request went to the wrong NetSuite data center. Review your integration and make sure that it generates the NetSuite RESTlet domain dynamically. For details, see Dynamically Generating a Full URL .
400 Bad Request	The RESTlet request failed with a user error. Any unhandled runtime errors return a 400. But if your code catches the error, you'll see a 200 OK instead. For example, you'll get this error if you send a get request with a body (that's not allowed by HTTP standards). For details, see https://developer.mozilla.org/en-US/docs/Web/HTTP/Status/400 .
401 Unauthorized	There is no valid NetSuite login session for the RESTlet call.

Code	Description
403 Forbidden	The RESTlet request was sent to an invalid domain.
404 Not Found	The RESTlet called is not at the URL used in the call.
405 Method Not Allowed	The request method used is not valid.
415 Unsupported Media Type	An unsupported Content-Type was specified.
500 Internal Server Error	If a non-user error happens and you can't fix it by resending the request, contact Customer Support to file a case.
503 Service Unavailable	The NetSuite database is offline, or a database connection is not available.

SuiteScript Error Codes Used by RESTlets

Here are some SuiteScript errors you might see with RESTlets:

Code	Description	Notes
INVALID_LOGIN_ATTEMPT	Invalid login attempt.	This means there's a problem in the OAuth header—maybe the nonce, consumer key, token, or signature is invalid.
INVALID_LOGIN_CREDENTIALS	You have entered an invalid email address or account number. Please try again.	This means there's an issue with the NLAAuth header.
INVALID_REQUEST	The request could not be understood by the server due to malformed syntax.	You'll see this error if the OAuth header has bad syntax—like if the signature method, version, or timestamp parameter isn't accepted.
INVALID_RETURN_DATA_FORMAT	You should return {1}.	This error pops up if the response data format doesn't match what's set by the Content-Type header.
SSS_INVALID_SCRIPTLET_ID	That Suitelet is invalid, disabled, or missing. If you see this error, double-check that your URL points to the right RESTlet deployment ID.	If you receive this error, make sure that the URL points to the correct RESTlet script deployment ID.
UNEXPECTED_ERROR	An unexpected error occurred. Error ID: {1}	--
TWO_FA_REQD	Two-Factor Authentication required.	This means two-factor authentication (2FA) is required for the role, but it's missing.

SuiteScript Error Message Formatting for RESTlets

Here's how SuiteScript errors look for each supported Content-Type.

application/json

```
1 {
2   "error":
```

```

3      {
4        "code": "SSS_INVALID_SCRIPTLET_ID",
5        "message": "That Suitelet is invalid, disabled, or no longer exists."
6      }
7    }

```

application/xml

```

1    <error>
2      <code>SSS_INVALID_SCRIPTLET_ID</code>
3      <message>That Suitelet is invalid, disabled, or no longer exists.</message>
4    </error>

```

text/plain

```

1    error code: SSS_INVALID_SCRIPTLET_ID
2    error message: That Suitelet is invalid, disabled, or no longer exists.

```

RESTlet Governance and Security

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

RESTlet Governance

SuiteScript governance tracks usage units at both the API and script level. API-level usage is just like other SuiteScripts. At the script level, RESTlets let you use up to 5,000 units per script—that's five times more than Suitelets and most other scripts.

You're guaranteed up to 10MB per string for RESTlet input or output. Sometimes you might get a larger output, but 10MB is the official limit.

For more information, see the help topic [SuiteScript Governance and Limits](#).

SuiteScript doesn't support a logout operation like SOAP web services do.

RESTlet Security

RESTlet URLs are protected by TLS encryption—only encrypted requests are allowed. For more information, see the help topic [Supported TLS Protocol and Cipher Suites](#).



Important: Since version 2017.2, web services and RESTlet concurrency is managed per account and counts both web service and RESTlet requests together. For more, see the help topic [Web Services and RESTlet Concurrency Governance](#).

SuiteScript 2.x RESTlet Script and Request Examples

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Here are some RESTlet examples to show you different ways you can use a RESTlet:

- [Example Hello World RESTlet](#)
- [Example of a RESTlet that Retrieves, Deletes, Creates, and Upserts a NetSuite Record](#)
- [Example of a RESTlet that Adds Multiple Records](#)
- [Example of a RESTlet that Manipulates Scheduled Script](#)

- [Example of a Client Script that Calls a RESTlet](#)
- [Example of a Suitelet that Calls a RESTlet](#)
- [Example of a Shell Script that Calls a RESTlet](#)

For help with other SuiteScript 2.x scripts, check out [SuiteScript 2.x Hello World](#) and [SuiteScript 2.x Entry Point Script Creation and Deployment](#).

Example Hello World RESTlet

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

RESTlets can be a bit trickier than other script types. Other script types run when needed after deployment. With RESTlets, you deploy them but have to call them for them to run. To call a RESTlet, make sure you do these three things:

- Correctly identify your NetSuite account's RESTlet domain
- Use an HTTP method that matches an entry point in your script
- Use two required headers: Content-Type and Authorization.

For details on the required headers, see [Creating a Content-Type Header](#) and [RESTlet Authentication](#).

The following example RESTlet is a simple implementation of the standard Hello World example implemented using the RESTlet get entry point.

RESTlet

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Restlet
4   */
5  define([], function() {
6    return {
7      get : function() {
8        return "Hello World!"
9      }
10   }
11 });

```

Example Get Call and Response

You call this RESTlet with the http get method. Since it doesn't take any arguments, you don't need to add anything else to the URL. For testing, just use the value in the External URL field of the deployment record. But for integrations, make sure you dynamically find the RESTlet domain. See [Dynamically Generating a Full URL](#).

For example, you could call this RESTlet by using a URL like the following — one that does not include embedded parameters:

```
1 | https://<accountID>.restlets.api.netsuite.com/app/site/hosting/restlet.nl?script=482&deploy=1
```

Once you add the two required headers, your call will look like this:

```

1 | GET /app/site/hosting/restlet.nl?script=482&deploy=1 HTTP/1.1
2 | HOST: <accountID>.restlets.api.netsuite.com
3 | authorization: NLAAuth nlauth_account=12345, nlauth_email=john@smith.com, nlauth_signature=WelCome123
4 | content-type: application/json
5 | cookie: ...

```


The RESTlet returns this response:

```
1 | Hello World!
```

For help troubleshooting RESTlet errors, see [SuiteScript 2.x RESTlet Error Handling](#) and [Entry Point Script Validation Error Reference](#).

Example of a RESTlet that Retrieves, Deletes, Creates, and Upserts a NetSuite Record

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

This RESTlet uses each entry point to retrieve (get), delete, create (post), and upsert (put) NetSuite records.

RESTlet

```
1  /**
2  * @NApiVersion 2.x
3  * @NScriptType Restlet
4  */
5  define(['N/record', 'N/error'],
6  function(record, error) {
7      // Get a standard NetSuite record
8      function _get(context) {
9          return JSON.stringify(record.load({
10             type: context.recordtype,
11             id: context.id
12         }));
13     }
14     // Delete a standard NetSuite record
15     function _delete(context) {
16         record.delete({
17             type: context.recordtype,
18             id: context.id
19         });
20         return String(context.id);
21     }
22     // Create a NetSuite record from request params
23     function post(context) {
24         var rec = record.create({
25             type: context.recordtype
26         });
27         for (var fldName in context) {
28             if (context.hasOwnProperty(fldName))
29                 if (fldName !== 'recordtype')
30                     rec.setValue(fldName, context[fldName]);
31         }
32         var recordId = rec.save();
33         return String(recordId);
34     }
35     // Upsert a NetSuite record from request param
36     function put(context) {
37         var rec = record.load({
38             type: context.recordtype,
39             id: context.id
40         });
41         for (var fldName in context) {
42             if (context.hasOwnProperty(fldName))
43                 if (fldName !== 'recordtype' && fldName !== 'id')
44                     rec.setValue(fldName, context[fldName]);
45         }
46         rec.save();
47         return JSON.stringify(rec);
48     }
49 }
```

```

49     return {
50         get: _get,
51         delete: _delete,
52         post: post,
53         put: put
54     };
55 });

```

Example Get Call and Response

To get a record with this RESTlet, use the get method. In the URL, add values for record type and the internal ID of the record you want. The RESTlet's get function uses these parameters:

- context.recordtype
- context.id

Add each parameter to the URL using an ampersand, the parameter name, an equals sign, and its value:

```
1 | &[name of parameter]=[value]
```

For example, to get a phone call record with internal ID 9363, use a URL like this (make sure the script and deploy values are correct):

```
1 | https://<accountID>.restlets.api.netsuite.com/app/site/hosting/restlet.nl?script=474&deploy=1&recordtype=phonecall&id=9363
```

Using the get method with this URL, your request would look like this:

```

1 | GET /app/site/hosting/restlet.nl?script=474&deploy=1&recordtype=phonecall&id=9363 HTTP/1.1
2 | HOST: <accountID>.restlets.api.netsuite.com
3 | authorization: NLAUTH nlauth_account=12345, nlauth_email=john@smith.com, nlauth_signature=Welcme123
4 | content-type: application/json
5 | cookie: ...

```

In response, you'll get data like this:

```

1 | {
2 |   "id": "9363"
3 |   "type": "phonecall"
4 |   "isDynamic": false
5 |   "fields": {
6 |     "wfinstances": ""
7 |     "nlloc": "0"
8 |     "nlsub": "1"
9 |     "createddate": "5/18/2016 1:01 am"
10 |    "timezone": "America/Los_Angeles"
11 |    "accesslevel": "F"
12 |    "_eml_nkey_": "143659438"
13 |    "starttime": ""
14 |    "startdate": "5/18/2016"
15 |    "title": "Project kickoff"
16 |    "type": "call"
17 |    ...
18 |   }
19 |
20 |   "sublists": {
21 |     "contact": {
22 |       "currentline": {
23 |         "company": ""
24 |         "contact": ""
25 |         ...
26 |       }-
27 |     }-
28 |   }-
29 | }

```

Example Post Call and Response

To add a record with this RESTlet, use the post method. Arguments go in the request body, and the body needs to be in JSON, not plain text. For example:

```
1 | {"recordtype":"phonecall","type":"phonecall","title":"Project Kickoff"}
```

Send your post request to a URL that doesn't include extra parameters. For testing, use the value in the External URL field on the deployment record. For integrations, make sure you dynamically get the RESTlet domain. See [Dynamically Generating a Full URL](#).

For example, you can call this RESTlet with a URL like this (no parameters):

```
1 | https://<accountID>.restlets.api.netsuite.com/app/site/hosting/restlet.nl?script=474&deploy=1
```

If you make a post call with that request body and the right headers, your request would look like this:

```
1 | POST /app/site/hosting/restlet.nl?script=474&deploy=1 HTTP/1.1
2 | HOST: <accountID>.restlets.api.netsuite.com
3 | authorization: NLAUTH nlauth_account=12345, nlauth_email=john@smith.com, nlauth_signature=Welcome123
4 | content-type: application/json
5 | cookie: ...
```

The RESTlet then returns the internal ID of the new record. For example:

```
1 | 9564
```

Example of a RESTlet that Adds Multiple Records

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Here's an example of using a RESTlet to create multiple contact records in one call.

RESTlet

```
1 | /**
2 |  * @NApiVersion 2.x
3 |  * @NScriptType Restlet
4 |  */
5 | define([ 'N/record' ], function(record) {
6 |     return {
7 |         post: function(restletBody) {
8 |             var restletData = restletBody.data;
9 |             for (var contact in restletData) {
10 |                 var objRecord = record.create({
11 |                     type : record.Type.CONTACT,
12 |                     isDynamic : true
13 |                 });
14 |                 var contactData = restletData[contact];
15 |                 for (var key in contactData) {
16 |                     if (contactData.hasOwnProperty(key)) {
17 |                         objRecord.setValue({
18 |                             fieldId : key,
19 |                             value : contactData[key]
20 |                         });
21 |                     }
22 |                 }
23 |                 var recordId = objRecord.save({
24 |                     enableSourcing : false,
25 |                     ignoreMandatoryFields : false
```

```

26         });
27         log.debug(recordId);
28     }
29 }
30 }
31 });

```

Example Post Call

To create records with this RESTlet, use the post method and send your values for the new contacts in the request body. For example:

```

1 [{"data":[{"firstname":"John","middlename":"Robert","lastname":"Smith","subsidiary":"1"},{"firstname":"Anne","middle
  name":"Doe","lastname":"Smith","subsidiary":"1"}]}]

```

Example of a RESTlet that Manipulates Scheduled Script

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Here's a RESTlet example that passes a value to a scheduled script.

i Note: The scriptId and deploymentId in this sample are just placeholders—be sure to swap them out for real IDs from your NetSuite account before using the script.

```

1 /**
2  * @NApiVersion 2.x
3  * @NScriptType Restlet
4  */
5  define(['/N/task'], function(task) {
6      return {
7          get : function() {
8              var mrTask = task.create({
9                  taskType: task.TaskType.SCHEDULED_SCRIPT
10             });
11             mrTask.scriptId = 488;
12             mrTask.deploymentId = 'customdeploy_scheduledscript';
13             mrTask.params = {
14                 custscriptcustom_data: 'data'
15             };
16             mrTask.submit();
17             return "Scheduled script updated";
18         }
19     }
20 });

```

Example of a Client Script that Calls a RESTlet

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Here's an example of how a client script can call a RESTlet. Since the client script already has an active session, it uses a partial URL to call the RESTlet. That means you don't need to include the RESTlet domain in the URL. You'll find this partial URL in the **URL** field of the script deployment record for the RESTlet.

i Note: The partial URL in this example is just a placeholder—be sure to swap it for a valid value from your NetSuite account before using the script.

```

1 /**

```

```

2  * @ApiVersion 2.x
3  * @NScriptType ClientScript
4  */
5  define(['N/https'], function(https) {
6      return {
7          pageInit : function() {
8              var dataFromRestlet = https.get({
9                  url: '/app/site/hosting/restlet.nl?script=200&deploy=1'
10             });
11             console.log(dataFromRestlet.body);
12         }
13     }
14 });

```

Example of a Suitelet that Calls a RESTlet

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Here's an example of how a Suitelet can call a RESTlet. Since the Suitelet's in the same NetSuite account as the RESTlet, the script can use the N/url module to find the RESTlet's URL. For this approach, you'll need the script ID and script deployment ID for the RESTlet.

i Note: The scriptId, deploymentId, and authorization info here are just placeholders—make sure to swap them out with the real values from your NetSuite account before using the script.

```

1  /**
2  * @ApiVersion 2.x
3  * @NScriptType Suitelet
4  */
5  define(['N/https', 'N/url'], function(https, urlMod) {
6      return {
7          onRequest : function(options) {
8              var url = urlMod.resolveScript({
9                  scriptId: 'customscript196',
10                 deploymentId: 'customdeploy1',
11                 returnExternalUrl: true,
12                 params: {parametername: 'value'}
13             });
14
15             var headers = {'Authorization': 'NLAAuth nlauth_account=12345, nlauth_email=john@smith.com, nlauth_signature=Wel
16 come123, nlauth_role=3'};
17
18             var response = https.get({url: url, headers: headers});
19
20             options.response.write(response.body);
21         }
22     });
23

```

Example of a Shell Script that Calls a RESTlet

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Here's a shell script that shows you how to send a request to a RESTlet with NLAAuth authentication and a Content-Type of application/json. To learn how to build an NLAAuth header, see the help topic [Using User Credentials for RESTlet Authentication](#).

```

1  #!/bin/sh
2
3  #Enter your RESTlet url here.
4  my_url="https://<accountID>.restlets.api.netsuite.com/app/site/hosting/restlet.nl?script=126&deploy=1"

```

```

5
6 #Enter the body of your request in a file named data.json.
7 DATA_FILE="@data.json"
8
9 CONTENT_FLAG="Content-Type: application/json"
10
11 #Update the following line with valid values from your NetSuite account.
12 AUTH_STRING="Authorization: NLAUTH nlauth_account=123456, nlauth_email=jsmith%40ABC.com, nlauth_signature=xxxx"
13
14 #Send the request to your RESTlet and capture the response from your RESTlet.
15 /usr/bin/curl -H "${CONTENT_FLAG}" -H "${AUTH_STRING}" -d "${DATA_FILE}" $my_url > /tmp/restlet_response.txt

```

SuiteScript 2.x RESTlet Script Entry Points and API

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Script Entry Point	Description
get	Lets you retrieve something (like a NetSuite record). Runs when a get request calls the RESTlet—like http.get. Returns an HTTP response body.
delete	Lets you delete something (like a NetSuite record). Runs when a delete request calls the RESTlet—like http.delete. Returns an HTTP response body.
put	Lets you insert or upsert something (like a NetSuite record). Runs when a put request calls the RESTlet—like http.put. Returns an HTTP response body.
post	Lets you create something (like a NetSuite record). Runs when a post request calls the RESTlet—like http.post. Returns an HTTP response body.

delete

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Description	This function runs when a delete request is sent to a RESTlet—usually to delete something, like a NetSuite record.
Returns	The RESTlet returns an HTTP response body as either a string or an Object. - You'll get a string if the request Content-Type is 'text/plain'. - You'll get an Object if the Content-Type is 'application/json' or 'application/xml'.
Since	Version 2015 Release 2

Parameters

Note: NetSuite automatically passes requestParams as a JavaScript object to your RESTlet entry point.

Parameter	Type	Required / Optional	Description	Since
requestParams	Object	required, but automatically passed by NetSuite	No matter the content type, NetSuite passes URL parameters as a JavaScript object.	Version 2015 Release 2

get

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Description	This function runs when a get request calls the RESTlet—usually to retrieve something, like a NetSuite record.
Returns	The RESTlet returns an HTTP response body as either a string or an Object. - You'll get a string if the Content-Type is 'text/plain'. - You'll get an Object if the Content-Type is 'application/json' or 'application/xml'.
Since	Version 2015 Release 2

Parameters

Note: NetSuite automatically passes `requestParams` as a JavaScript object to your RESTlet entry point.

Parameter	Type	Required / Optional	Description	Since
<code>requestParams</code>	Object	required, but automatically passed by NetSuite	No matter the content type, NetSuite passes URL parameters as a JavaScript object.	Version 2015 Release 2

post

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Description	This function runs when a post request calls the RESTlet—usually to create something, like a NetSuite record.
Returns	The RESTlet returns an HTTP response body as either a string or an Object. - You'll get a string if the Content-Type is 'text/plain'. - You'll get an Object if the Content-Type is 'application/json' or 'application/xml'.
Since	Version 2015 Release 2

Parameters

Note: NetSuite automatically passes `requestParams` as a JavaScript object to your RESTlet entry point.

Parameter	Type	Required / Optional	Description	Since
<code>requestBody</code>	string Object	required, but automatically passed by NetSuite	The request body is a string if the Content-Type is 'text/plain', or a JavaScript object if the Content-Type is 'application/json' or 'application/xml'.	Version 2015 Release 2

put

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Description	This function runs when a PUT request calls the RESTlet—usually to insert or upsert something, like a NetSuite record.
Returns	The RESTlet returns an HTTP response body as either a string or an Object. - You'll get a string if the Content-Type is 'text/plain'. - You'll get an Object if the Content-Type is 'application/json' or 'application/xml'.
Since	Version 2015 Release 2

Parameters

Note: NetSuite automatically passes `requestParams` as a JavaScript object to your RESTlet entry point.

Parameter	Type	Required / Optional	Description	Since
<code>requestBody</code>	string Object	required, but automatically passed by NetSuite	The request body is a string if the Content-Type is 'text/plain', or a JavaScript object if the Content-Type is 'application/json' or 'application/xml'.	Version 2015 Release 2

SuiteScript 2.x Scheduled Script Type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Scheduled scripts are server scripts processed by [SuiteCloud Processors](#). You can set up scheduled scripts to run one time in the future or on a recurring schedule. You can also run scheduled scripts on-demand, either from the deployment record or another script using the [task.ScheduledScriptTask](#) API.

Note: To handle large datasets, use map/reduce scripts instead—these are newer and support parallel processing and better governance. See [SuiteScript 2.x Map/Reduce Script Type](#).

For additional information about SuiteScript 2.x scheduled scripts, see the following:

- [SuiteScript 2.x Scheduled Script Reference](#)
 - [Scheduled Script Submission](#)
 - [Scheduled Script Deployment Record](#)
 - [Scheduled Script Deployments that Continue to Use Queues](#)
 - [Scheduling a One Time or Recurring Scheduled Script Submission](#)
 - [Submitting an On-Demand Scheduled Script Instance from the UI](#)
 - [Submitting an On-Demand Scheduled Script Instance from Another Script](#)
 - [Scheduled Script Execution](#)
 - [Scheduled Script Debugging](#)

- [Scheduled Script Status Page](#)
 - [Scheduled Script Handling of Server Restarts](#)
- [SuiteScript 2.x Scheduled Script Entry Points and API](#)
 - [execute](#)
 - [context.InvocationType](#)
- [SuiteScript 2.x Scheduled Script Type Code Sample](#)

You can manage scheduled scripts with SuiteCloud Development Framework (SDF) as part of file-based projects. To learn more, see [. Use Copy to Account](#) to copy a scheduled script to another account—just click Copy to Account at the top right of the script page. For details, see the help topic [Copy to Account](#).

Use SuiteScript Analysis (part of the Application Performance Management SuiteApp) to see when a script was installed and how it's performed. For more, see the help topic [Analyzing Scripts](#).

For best practices, check the [Scheduled Script Best Practices](#) in the [SuiteScript Developer Guide](#).

Scheduled Script Use Cases

Use scheduled scripts for tasks that need to run on a set schedule or on-demand. Don't use them for big data jobs or long-running operations.

Here are a few good reasons to use a scheduled script:

- You need to log basic information about a recurring schedule
- You want to schedule a maintenance script to run later
- You need to create and then purge temporary records
- You need to asynchronously execute logic within another server script



Note: If you previously used SuiteScript 1.0 scheduled scripts, many of the use cases for those scripts now apply to the SuiteScript 2.x [SuiteScript 2.x Map/Reduce Script Type](#).

Scheduled Script Governance

Each scheduled script run can use up to 10,000 usage units. For more info, see the help topic [SuiteScript Governance and Limits](#).

SuiteScript 2.x scheduled scripts don't let you set recovery points or yield—there's no direct replacement for `nlapiYieldScript()` or `nlapiSetRecoverPoint()`. If you need to process lots of records, switch to the [SuiteScript 2.x Map/Reduce Script Type](#), which supports yielding and works much like scheduled scripts.

SuiteScript 2.x Scheduled Script Type Code Sample

This script sample finds and fulfills sales orders. You'll find more schedule script samples in the [SuiteScript 2.x Code Samples Catalog](#).

Before you submit this script:

1. Create a saved search for sales orders. Use [search.create\(options\)](#) and the [search.Type](#) enum to set it up with the right search id and search type.

2. Add a script parameter on the script record's **Parameters** subtab. The sample expects a search id from a script parameter that was set up with the script record.
 - Set the id to custscript_searchid.
 - Set Type to Free-Form Text.
 - Assign the saved search id to the parameter on the deployment record's **Parameters** subtab.

This sample uses `log.debug()`, but you don't have to load the `N/log` module—`log` is loaded by default for all script types. For more, see the help topic [log Object](#).

This script uses SuiteScript 2.0, but SuiteScript 2.1 is also available and includes new ES2019 features. You can write scheduled scripts with either version.

- For help writing SuiteScript 2.x scripts, check out [SuiteScript 2.x Hello World](#) and [SuiteScript 2.x Entry Point Script Creation and Deployment](#).
- For more about SuiteScript versions and SuiteScript 2.1, see [SuiteScript Versioning Guidelines](#) and [SuiteScript 2.1](#).

This sample is set to run on demand—not at a scheduled time. For details about script context, see [context.InvocationType](#).

```

1  /**
2   *@NApiVersion 2.x
3   *@NScriptType ScheduledScript
4   */
5  define(['N/search', 'N/record', 'N/email', 'N/runtime'],
6    function(search, record, email, runtime) {
7      function execute(context) {
8          if (context.type !== context.InvocationType.ON_DEMAND)
9              return;
10         var searchId = runtime.getCurrentScript().getParameter("custscript_searchid");
11         try {
12             search.load({
13                 id: searchId
14             }).run().each(function(result) {
15                 log.debug({
16                     details: 'transforming so : ' + result.id + ' to item fulfillment'
17                 });
18                 var fulfillmentRecord = record.transform({
19                     fromType: record.Type.SALES_ORDER,
20                     fromId: result.id,
21                     toType: record.Type.ITEM_FULFILLMENT,
22                     isDynamic: false
23                 });
24                 var lineCount = fulfillmentRecord.getLineCount('item');
25                 for (var i = 0; i < lineCount; i++) {
26                     fulfillmentRecord.setSublistValue('item', 'location', i, 1);
27                 }
28                 var fulfillmentId = fulfillmentRecord.save();
29                 var so = record.load({
30                     type: record.Type.SALES_ORDER,
31                     id: result.id
32                 });
33                 so.setValue('memo', fulfillmentId);
34                 so.save();
35                 return true;
36             });
37         } catch (e) {
38             var subject = 'Fatal Error: Unable to transform salesorder to item fulfillment!';
39             var authorId = -5;
40             var recipientEmail = 'notify@example.com';
41             email.send({
42                 author: authorId,
43                 recipients: recipientEmail,
44                 subject: subject,
45                 body: 'Fatal error occurred in script: ' + runtime.getCurrentScript().id + '\n\n' + JSON.stringify(e)
46             });

```

```
47     }
48   }
49   return {
50     execute: execute
51   };
52 });
```

SuiteScript 2.x Scheduled Script Reference

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

All scheduled scripts run with [SuiteCloud Processors](#). For more information, see the following:

- [Scheduled Script Submission](#)
- [Scheduled Script Execution](#)

You can debug scheduled scripts with the SuiteScript Debugger. For details, see [Scheduled Script Debugging](#).

Check the runtime status of your scheduled script instances on the Scheduled Script Status page. For more, see [Scheduled Script Status Page](#).

Sometimes a scheduled script might fail because the application server restarts—maybe from an update, maintenance, or some unexpected error. Restarts can end your script at any moment, so make sure your scripts can handle interruptions and recover. In SuiteScript 2.x, if there's a system failure, the script restarts from the beginning. For details, see [Scheduled Script Handling of Server Restarts](#).

Scheduled Script Submission

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

All scheduled script instances are executed (processed) by [SuiteCloud Processors](#). You can submit a scheduled script instance for processing in one of three ways:

- By [Scheduling a One Time or Recurring Scheduled Script Submission](#) from the script deployment record UI
- By [Submitting an On-Demand Scheduled Script Instance from the UI](#) with the **Save and Execute** option
- By [Submitting an On-Demand Scheduled Script Instance from Another Script](#) with the `task.ScheduledScriptTask` API

For any of these options, you'll need to create a [Scheduled Script Deployment Record](#).

How your scheduled script runs depends on its deployment type. See the following table for details, and read [Scheduled Script Deployment Record](#) for more about deployment.

		Submission			
		On demand			Scheduled
		Submit from Save & Execute on the UI	Submit from N/task	Submit in the debugger	Submit from scheduled deployment
Deployment Status	Testing	√	√	√	
	Scheduled				√
	Not Scheduled	√	√		



Important: After you submit a scheduled script, there might be a short system delay before it runs—even if no other scripts are ahead of it. If there are scripts already in the queue, your script will wait until those finish.

Scheduled Script Deployment Record

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Before you submit a scheduled script to [SuiteCloud Processors](#), make sure you've created at least one deployment record for the script.

Body Fields

Field	Description
Script	A link to the script record associated with the deployment Note: You can't change this value—even on a new deployment record. If you pick the wrong script record, you'll need to start over.
Title	The user-defined name for the deployment
ID	A unique identifier for the deployment: ■ If this field is left blank when the deployment is created, the system generates the ID. ■ Enter a custom ID if you plan to bundle the script and deployment for another account—this helps avoid naming conflicts. Use lowercase, no spaces, and “_” to separate words. Note: For any custom or system ID, NetSuite adds “customdeploy” to the start. You can edit this value on an existing deployment if needed; just click Edit on the deployment record, then click Change ID.
Deployed	This box marks whether the deployment is active—check it if you want your script to run. If it's not checked: ■ The script won't execute, even if there's a schedule set. ■ The Save and Execute option disappears from the Save dropdown when editing. ■ The script can't be submitted programmatically. For details, see Submitting an On-Demand Scheduled Script Instance from Another Script.
Status	This setting controls how and when a script can be submitted to SuiteCloud Processors. Options include: ■ Testing: You can test the script in SuiteScript Debugger as a deployed script. See the help topic Debugging Deployed SuiteScript 1.0 and SuiteScript 2.0 Server Scripts for more. Deployed debugging is only for the script owner. ■ Scheduled: You can set up single or recurring runs from the Schedule subtab, but you can't submit the script on-demand when it's set this way. Note: If you set up recurring runs, the deployment status stays as Scheduled on the Scheduled Script Status Page, even after the script finishes running. ■ Not Scheduled: If the script isn't scheduled, you can submit it on-demand using either: □ The Save and Execute button on the deployment record □ The task.ScheduledScriptTask API

Field	Description
	You can only run one on-demand instance of a script at a time—if one's still running, you can't start another. See Scheduled Script Submission for a table that compares scheduled script submissions to the Status value.
See Instances	A link to the Scheduled Script Status Page , filtered for all deployment instances of this deployment.
Log Level	This setting controls what's logged for this deployment—find entries on the Execution Log subtab. Options include: ■ Debug: Use Debug for testing mode—it shows all Audit, Error, and Emergency logs. ■ Audit: Use Audit for production—it keeps records of script events, like when a request is made to an external site. ■ Error: Use Error for production—this only logs unexpected script errors. ■ Emergency: Use Emergency for production—it only shows the most critical errors.
Execute as Role	For scheduled script deployments, this is always set to Administrator and can't be changed.
Priority	This setting affects when the processor scheduler sends jobs to the pool—by default, it's set to Standard. For more, see the help topic SuiteCloud Processors Priority Levels.  Note: Each SuiteScript 2.x scheduled script instance is handled by a single job within SuiteCloud Processors.  Important: Make sure you understand SuiteCloud Processors before changing this setting.. Possible options are: ■ High: Use High for critical jobs—the scheduler puts these at the front of the line. ■ Standard: Standard is the default medium priority—the scheduler sends these jobs next, after any high priority jobs. ■ Low: Use Low if your job can wait longer, and the scheduler processes these after high and standard jobs.
Queue (Deprecated)	This field's deprecated. If you click Remove Queue , it disappears from the deployment record..

Schedule Subtab

The Schedule subtab is always available, no matter what Status is chosen.

Field	Description
Single Event	This option submits the script only once—use it for a one-time future run.
Daily Event	This submits the script every set number of days. If you set it to go every x minutes or hours, the schedule restarts on the next scheduled day. For example, if your deployment submits daily at 3:00 am and repeats every 5 hours, instances are submitted at 3:00 am, 8:00 am, 1:00 pm, 6:00 pm, and 11:00 pm. At midnight, it restarts and the next one is at 3:00 am.
Weekly Event	This submits the script at least once a week. If it's set to recur every x minutes or hours, it starts over on the next scheduled day.

Field	Description
	For example, if it's set to run on Tuesday and Wednesday at 3:00 am, repeating every 5 hours, it submits at 3:00 am, 8:00 am, 1:00 pm, 6:00 pm, and 11:00 pm each day. The schedule starts over each day at 3:00 am.
Monthly Event	This submits the script at least once every month.
Yearly Event	This submits the script at least once every year.
Start Date	The first submission happens on this date—required for one-time or recurring schedules.
Start Time	If you enter a Start Time, the first run starts then. When you save the deployment, NetSuite saves the Start Date and Start Time in your time zone. When another user opens the record, NetSuite shows them in their preferred time zone. Example: ■ User A (GMT +8, Manila) saves the deployment: □ Start Date = May 5 2025 □ Start Time = 8:00 a.m. ■ User B (GMT -5, Eastern) opens the record and sees: □ Start Date = May 4 2025 □ Start Time = 8:00 p.m. ■ User B clicks Save, and NetSuite stores the start values in GMT #5—regardless of whether the schedule changed. ■ When either user opens the record again, NetSuite shows the same moment in each user's time zone. User A still sees May 5 2025 8:00 a.m. (GMT +8), while User B sees May 4 2025 8:00 p.m. (GMT -5).  Note: NetSuite automatically applies time-zone rules, including Daylight Saving Time, based on the Start Date. For details on setting your time zone, see Time Zone in General Personal Preferences.
Repeat	If you pick a date and time, the first submission happens then, and a new script runs every x minutes or hours until the day ends. The schedule restarts on the next scheduled day if needed. For example, if it's scheduled at 3:00 pm on Tuesday and Wednesday and set to recur every 5 hours, it runs at 3:00 pm and 8:00 pm on each day, and then the schedule restarts.
End By	If you set an end date, the last submission is then. If it's recurring, a new script runs every x minutes or hours until that end date.
No End Date	If you don't set an end date, the schedule goes on indefinitely.

Scheduled Script Deployments that Continue to Use Queues

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

For any scheduled script deployment created before [SuiteCloud Processors](#), the **Queue** field still shows up by default. This goes for accounts with or without SuiteCloud Plus. The deployment record has a **Remove Queue** option you can use if you want to stop using queues. Once you pick it, the deployment won't use a queue anymore—and you can't switch back.

The **Queue** field is there for deployments that need FIFO processing from a specific queue. But all jobs—whether they use queues or not—are handled by the same processor pool and compete with each other using the same processing rules.

Note: For deployments still using queues, make sure all jobs in the same queue have the same priority. Usually, the default (standard) priority works fine, but you can bump jobs in a queue to high priority if you always want a processor ready for them. Or, assign a bunch of lower-priority jobs to the same queue at low priority—they'll run one at a time that way.

Important: If your scripts rely on queue-based order (FIFO), update and test them before removing queues. Removing the queue could affect how your scripts run.

One workaround is to programmatically submit scripts in the order you want—use [task.ScheduledScriptTask](#) in the first script to kick off the second. This way, jobs reach the processor pool in the sequence you need.

Scheduling a One Time or Recurring Scheduled Script Submission

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Important: Before you submit a scheduled script, make sure you know how [SuiteCloud Processors](#) works.

You can set up scheduled scripts to run once in the future or on a regular daily, weekly, monthly, or yearly schedule. Make sure the Status field is set to **Scheduled** in the deployment record.

You can schedule deployment times every 15 minutes—like 2:00 pm, 2:15 pm, 2:30 pm, and so on.

On the Script Deployment page, the times you set in the Schedule subtab are when your script is submitted for processing—not the exact times it'll actually run. For example, setting it to 2:00 pm, 2:15 pm, and 2:30 pm doesn't guarantee execution right on the dot.

Set a scheduled script's deployment status to **Scheduled** if:

- It was in Testing, but is now ready for production.
- The script doesn't need to run immediately.
- The script should run at regular intervals.

Important: After you submit a script, there might be a short delay before it runs—even if nothing else is queued. If other scripts are already waiting, yours will have to wait for them to finish.

To set up a one-time or recurring scheduled script:

1. First, create your scheduled script entry point—this means your JavaScript file and its script record. For examples, check out [SuiteScript 2.x Scheduled Script Type Code Sample](#) or, if you're new, see [SuiteScript 2.x Hello World](#) and [SuiteScript 2.x Entry Point Script Creation and Deployment](#).
2. On the script record, click **Deploy Script**.
3. When the deployment record loads, make sure the **Deployed** box is checked—you can't submit the script unless it is.
4. Review the field descriptions in [Scheduled Script Deployment Record](#).
5. Set the Status field to **Scheduled**.
6. Fill in the other required fields.
7. Set the deployment options on the [Schedule Subtab](#).

If you want the script to run hourly all day, use these sample values:

- Deployed = checked
- Daily Event = [radio button enabled]
- Repeat every 1 day
- Start Date = [today's date]
- **Start Time = 12:00 am**
- Repeat = every hour
- End By = [blank]
- No End Date = checked
- Status = Scheduled
- Log Level = Error
- Execute as Role = Set to **Administrator**

If you set **Start Time** to something besides 12:00 am—like 2:00 pm—the script starts at 2:00 pm and keeps running hourly till midnight. Then it waits until 2:00 pm the next day to start up again.

8. Click **Save**.

Investigating Problems with Scheduled Script Submissions

After you submit the script, there might be a brief delay before it gets run, even if nothing else is queued.

If you think there's too much of a delay, check the following before reaching out to NetSuite Customer Support.

- Are you still using the old queuing system for scheduled scripts instead of the dynamic processor queue? If so, move your jobs to the dynamic queue. See [Scheduled Script Deployments that Continue to Use Queues](#) for more.
- Is one script waiting for another to finish? If jobs are queued because all processors are busy, it's not a real dependency—but consider these cases:
 - **Queue dependency:** The job is using queues and depends on the job ahead of it in the same queue.
 - **Async search dependency:** Scheduled and map/reduce scripts can be set up to run after an async search task finishes—they'll only start once that task is done.

You might also see delays if there are:

- Database access problems
- Intermittent network connectivity problems
- A required restart of system components

If you're still seeing long wait times, contact Customer Support and give them the details. Here's how you can tell if your scripts are using the old or new queue:

- If you have the Application Performance Monitoring SuiteApp:
 - Go to Customization > Performance > SuiteCloud Processors Monitor. In the Overview portlet, sort by average wait time to help Support. Jobs on the old queue have a number in the Queue column; "-None-" means the job uses the dynamic system.
 - You can also check the Script Queue Monitor and SuiteCloud Processors Job Details pages.
- If you don't have the APM SuiteApp, go to the [Scheduled Script Status Page](#). If you see a number in the Queue column, that's the old queue; if it's blank, it's dynamic.

- You can also check the Script Deployment record: if there's a **Remove Queue** button or a **Queue (Deprecated)** field, that script's using the old queue system.

Submitting an On-Demand Scheduled Script Instance from the UI

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Important: Before you submit a scheduled script, be sure you know how [SuiteCloud Processors](#) works.

You can submit a scheduled script on-demand by using the **Save and Execute** option on the Script Deployment page. Make sure the Status field is set to **Not Scheduled** or **Testing** before submitting a script on-demand.

You can submit an on-demand script only if there isn't another unfinished instance of it running.

Use the **Testing** status to debug and test your script. If you want to step through the script in the [SuiteScript Debugger](#), you'll need to use deployed debugging. See the help topics [Debugging Deployed SuiteScript 1.0 and SuiteScript 2.0 Server Scripts](#) and [Debugging Deployed SuiteScript 2.1 Server Scripts](#) for more details.



Important: Even if you submit a script on-demand, it might not start right away. There may be a short system delay, and if other scripts are in the queue, yours will wait until they finish.

To submit an on-demand scheduled script instance from the UI:

1. First, create your scheduled script entry point, which includes your JavaScript file and script record. Want an example? See [SuiteScript 2.x Scheduled Script Type Code Sample](#). New to scripts? Check out [SuiteScript 2.x Hello World](#) and [SuiteScript 2.x Entry Point Script Creation and Deployment](#).
2. On the associated script record, select the **Deploy Script** button.
3. When the script deployment record loads, click the **Deployed** box if it's not already checked. You can't submit the script unless this box is checked.
4. Review the scheduled script deployment field descriptions at [Scheduled Script Deployment Record](#).
5. Set the Status field to **Not Scheduled** or **Testing**.
6. Set the remaining body fields.
7. On the [Schedule Subtab](#), set all deployment options.
8. Click **Save and Execute** from the **Save** dropdown.

Submitting an On-Demand Scheduled Script Instance from Another Script

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Important: Before you submit a scheduled script, be sure you know how [SuiteCloud Processors](#) works.

You can use the `task.ScheduledScriptTask` API to programmatically submit a scheduled script to [SuiteCloud Processors](#).

You can submit an on-demand script only if there isn't another unfinished instance of it running.

To submit an on-demand scheduled script instance from another script:

1. First, create your scheduled script entry point, which includes your JavaScript file and script record. Need an example? Check out the [SuiteScript 2.x Scheduled Script Type Code Sample](#). New to scripts? See [SuiteScript 2.x Hello World](#) and [SuiteScript 2.x Entry Point Script Creation and Deployment](#).
2. On the associated script record, select the **Deploy Script** button.
3. When the script deployment record loads, click the **Deployed** box if it's not already checked. You can't submit the script unless this box is checked.
4. Review the scheduled script deployment field descriptions at [Scheduled Script Deployment Record](#).
5. Set the Status field to **Not Scheduled**.
6. Set the remaining body fields.
7. Click **Save**.
8. In the server script where you want to submit the scheduled script, call `task.create(options)`. This returns a `task.ScheduledScriptTask` object:

```
1 | var scriptTask = task.create({
2 |   taskType: task.TaskType.SCHEDULED_SCRIPT
3 | });
```

9. Set the `ScheduledScriptTask.scriptId` and `ScheduledScriptTask.deploymentId` properties:

```
1 | scriptTask.scriptId = 1234;
2 | scriptTask.deploymentId = 'customdeploy1';
```

10. Call `ScheduledScriptTask.submit()` to submit the scheduled script instance to [SuiteCloud Processors](#).

```
1 | var scheduledScriptTaskId = scriptTask.submit();
```

Scheduled Script Execution

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

[SuiteCloud Processors](#) run all scheduled script instances. To run a scheduled script, just submit it on-demand or set it up to run on a schedule. Want to learn more about submitting scheduled scripts? See [Scheduled Script Submission](#).

Each scheduled script gets its own job. The scheduler sends jobs to the processor pool based on their priority and submission order. Higher-priority jobs go first. When jobs have the same priority, they're sent in the order they were submitted. Want more info? See the help topic [SuiteCloud Processors Priority Levels](#).

Advanced settings in SuiteCloud Processors can also change the order jobs are sent to the processor pool. For details, see the help topic [SuiteCloud Processors Priority Elevation](#).

Scheduled Script Debugging

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the deployment record's **Testing** status to debug and test your script. If you want to step through your script in the [SuiteScript Debugger](#), you'll need to use deployed debugging.

Want more details? See the help topics [Debugging Deployed SuiteScript 1.0 and SuiteScript 2.0 Server Scripts](#) and [Debugging Deployed SuiteScript 2.1 Server Scripts](#).

Scheduled Script Status Page

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The Scheduled Script Status page shows the current and past runtime statuses for submitted scheduled script instances in your account.

Scheduled Script Status

Refresh

Filters

Date

today

From

11.9.2024

To

11.9.2024

Script

simplesched

Deployment ID

CUSTOMDEPLOY1

Queue

- All -

Status

- All -

API Version

- All -

Total: 3

API Version	Deployment ID	Date Created	Status	Start	End	Priority	Queue	% Complete	Cancel
2.0	customdeploy1	15.4.2024 4:09:55 pm	Complete	4:09:57 pm	4:09:59 pm	Standard		100.0%	
2.0	customdeploy1	15.4.2024 4:07:27 pm	Complete	4:07:28 pm	4:07:29 pm	Standard		100.0%	
2.0	customdeploy1	15.4.2024 4:06:43 pm	Complete	4:06:44 pm	4:06:45 pm	Standard		100.0%	

You've got two ways to access the Scheduled Script Status page. How you open the page determines which view you'll see first. Each view shows the same information but in a different order.

- Go to Customization > Scripting > Scheduled Script Status. The columns shown on the Scheduled Script Status page are:

Script	API Version	Deployment ID	Date Created	Status	Start	End	Priority	Queue	% Complete	Cancel
--------	-------------	---------------	--------------	--------	-------	-----	----------	-------	------------	--------

- Click the Status Page link on a Script Deployment record for a single scheduled script. The columns shown on the Scheduled Script Status page are:

API Version	Deployment ID	Date Created	Status	Start	End	Priority	Queue	% Complete	Cancel
-------------	---------------	--------------	--------	-------	-----	----------	-------	------------	--------

You can filter the scheduled scripts by Date, From, To, Script, Deployment ID, Queue, Status, or API Version.

Important: Script execution details get deleted after 30 days. The statuses on the Scheduled Script Status page aren't the same as those on the Script Deployment page. On the Script Deployment page, statuses like Not Scheduled, Scheduled, or Testing show the deployment type. On the Scheduled Script Status page, statuses show where your script is in its execution.

The following table describes each field on the page.

Field	Description
API Version	The API Version specified in the script using the @API Version JSDoc tag for the current version of the script. This means that if you submit a scheduled script using @API Version 2.0 and then submit it a second time using @API Version 2.1, the version that is shown in the API Version field will be 2.1 for all listings of the script.
Deployment ID	The deployment ID for the deployment record for the scheduled script.
Date Created	The date the script was created.
Status	The execution status for the scheduled script. This status will be one of the following: Canceled – This status indicates that due to a NetSuite server error, the script was canceled before or during script execution.

Field	Description
	- Complete – This status indicates the script completed normally. - Deferred – This status indicates that the script is eligible for processing, but has not been processed due to processing constraints. For example, deferred status occurs when one job must wait for another job to finish. - Failed – This status indicates that the script has been processed, but failed to complete normally. If your script failed, check it for possible errors. - Pending – This status indicates that the script has been submitted and is waiting to be processed. - Processing – This status indicates the script is currently running. - Retry – This status indicates that the script entered the processing state, but failed to complete normally. In this case, the script is eligible to be retried. The script retries automatically, so you don't need to create a new deployment. Check the script log to find out why it failed the first time (like a timeout, for example).
Start	The time the scheduled script execution started.
End	The time the scheduled script execution ended.
Priority	The priority for the scheduled script. For more information about priorities, see the help topic SuiteCloud Processors Priority Levels .
Queue	The queue for the scheduled script. This field only pertains to scheduled script deployments that occurred before the introduction of SuiteCloud Processors. For more information, see Scheduled Script Deployments that Continue to Use Queues .
% Complete	The script's percentage of completion. This field shows the following values: 0% - Shown when Status is Pending or Processing 100% - Shown when Status is Completed  Note: You can't script this field.

Scheduled Script Handling of Server Restarts

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Sometimes, a scheduled script might fail if the application server restarts—like for a NetSuite update, maintenance, or an unexpected issue. Restarts can force a script to stop anytime, so your scripts should handle restarts and recover from interruptions.

In SuiteScript 2.x, if a failure happens, the script restarts from the beginning. NetSuite knows when a scheduled script was stopped and restarts it as soon as the needed resources are ready. You don't manually script yields or recovery points in SuiteScript 2.x, so it's easier to manage restarts.

In your scheduled script, check the `context.type` attribute to see if the script was restarted (if `context.type === "aborted"`). For example:

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType ScheduledScript
4   */
5  define([], function(){
6      return {
7          execute: function (context)
8          {
9              if (context.type === 'aborted')
10             {

```

```

11      // I might do something differently
12      }
13      .
14      .
15      .
16    }
17  }
18  });

```

Want more details about the context argument? See [context.InvocationType](#).

The following example scripts demonstrate how restarts impact scheduled script execution:

- [Example: A Problematic Scheduled Script](#)
- [Example: A Robust Scheduled Script](#)

Example: A Problematic Scheduled Script

Scheduled scripts often search for data, then process the results.

In this example, the script updates each sales order in the system. The `filter1` filter ensures that the search returns exactly one entry per sales order. But if the script is stopped in the middle and restarted, some sales orders might get updated twice.

To avoid data issues from re-processing, make your script's operations idempotent—they should give the same result even if run more than one time (like stopping duplicate records). Or, your script should be able to fix mistakes, like removing duplicates if they happen.

The example script isn't idempotent, so many sales orders could get updated twice—possibly doubling a price from a repeated run.

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType ScheduledScript
4   */
5  define(['N/search', 'N/record'],
6    function(search, record){
7    return {
8      execute: function (context)
9      {
10        var filter1 = search.createFilter({
11          name: 'mainline',
12          operator: search.Operator.IS,
13          values: true
14        });
15        var srch = search.create({
16          type: search.Type.SALES_ORDER,
17          filters: [filter1],
18          columns: []
19        });
20
21        var pagedResults = srch.runPaged();
22
23        pagedResults.pageRanges.forEach(function(pageRange){
24          var currentPage = pagedResults.fetch({index: pageRange.index});
25          currentPage.data.forEach(function(result){
26            var so = record.load({
27              type: record.Type.SALES_ORDER,
28              id: result.id
29            });
30            //UPDATE so FIELDS>
31            so.save()
32          });
33        });
34      }
35    });
36  });

```

To fix scripts like this, check out [Example: A Robust Scheduled Script](#) and [Example 5: A Robust Map/Reduce Script Example](#).

Example: A Robust Scheduled Script

The following example shows how to prevent duplicate processing if your script restarts. This script uses `custbody_processed_flag` on the sales order. It is a custom boolean field that must be previously created in the UI. When a sales order is processed, the field is set to true, so your search skips flagged sales orders. If the script restarts, only unprocessed sales orders are handled.

```
1  /**
2   * @ApiVersion 2.x
3   * @NScriptType ScheduledScript
4   */
5  define(['N/search', 'N/record'],
6    function(search, record){
7      return {
8        execute: function (context)
9        {
10          var filter1 = search.createFilter({
11            name: 'mainline',
12            operator: search.Operator.IS,
13            values: true
14          });
15          var filter2 = search.createFilter({
16            name: 'custbody_processed_flag',
17            operator: search.Operator.IS,
18            values: false
19          });
20          var srch = search.create({
21            type: search.Type.SALES_ORDER,
22            filters: [filter1, filter2],
23            columns: []
24          });
25
26          var pagedResults = srch.runPaged();
27
28          pagedResults.pageRanges.forEach(function(pageRange){
29            var currentPage = pagedResults.fetch({index: pageRange.index});
30            currentPage.data.forEach(function(result){
31              var so = record.load({
32                type: record.Type.SALES_ORDER,
33                id: result.id
34              });
35              // UPDATE so FIELDS
36              so.setValue({
37                fieldID: 'custbody_processed_flag',
38                value: true
39              });
40              so.save()
41            });
42          });
43        }
44      }
45    });
```

SuiteScript 2.x Scheduled Script Entry Points and API

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Script Entry Point	Description
<code>execute</code>	This function runs when the scheduled script is triggered.

Script Entry Point	Description
context.InvocationType	An API enumeration that lists different script execution contexts.

execute

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Description	Sets the trigger point for the scheduled script.
Returns	void
Since	2015.2

Parameters

i Note: NetSuite automatically passes the scriptContext parameter—a JavaScript object—to the script entry point.

Parameter	Type	Required / Optional	Description	Since
scriptContext.type	string	required	This is the script execution context, with values shown in the context.InvocationType enum.	2015.2

context.InvocationType

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Enum Description	An enumeration that lists the string values for scheduled script execution contexts.
Module	SuiteScript 2.x Scheduled Script Type
Since	2015.2

Values

SCHEDULED	Normal execution based on the deployment options you set in the UI.
ON_DEMAND	The script runs when another script calls it with ScheduledScriptTask.submit() . i Note: The scheduled script must be set to Not Scheduled on the Script Deployment page.
USER_INTERFACE	The script runs from the UI when you click the Save & Execute button.
ABORTED	The script runs again automatically if the system went down during its execution.

SKIPPED	The script runs automatically after downtime if it missed its scheduled time.
---------	---

SuiteScript 2.x SDF Installation Script Type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The SDF installation script type is used to perform tasks during deployment of a SuiteApp from SuiteCloud Development Framework (SDF) to your target account. You can use this script type to perform setup, configuration, and data management tasks that would otherwise have to be completed by account administrators, such as:

- Setting up and verifying the target account
- Verifying that required permissions and features are enabled before installing
- Preventing installation if proper setup has not occurred
- Stopping installation if a requirement is missing in the account
- Migrating existing data (for instance, data needed for a SuiteApp)
- Programmatically creating fields on a custom record based on the account's enabled features

The SDF installation script type is similar to the bundle installation script type. However, this script type is designed for use with SuiteApps that are developed and deployed with the SDF, rather than with SuiteBundler.

For details about SDF installation scripts, see the following help topics:

- [SDF Installation Script Requirements](#)
- [Managing SDF Installation Scripts in SDF](#)
- [SDF Installation Script Samples](#)
- [SuiteScript 2.x SDF Installation Script Entry Points and API](#)

 [Executing Installation Scripts](#)

SDF Installation Script Requirements

To run an SDF installation script when deploying your SuiteApp, you need to link it to a script record and a script deployment record. For more information, see the help topic [Creating a Script Record](#).

You can create script records and script deployment records in a few ways: (1) within your SuiteCloud project as XML representations, (2) using SuiteScript, and (3) from within your NetSuite account. For more information, see the help topics [SDF Installation Scripts as XML Definitions](#) and [SDF Installation Scripts as SuiteScript](#).

SDF installation scripts have a limit of 10,000 units per run. For more information about governance, see the help topics [Script Type Usage Unit Limits](#) and [SuiteScript Governance and Limits](#).

Managing SDF Installation Scripts in SDF

You can use SuiteCloud Development Framework to manage SDF installation scripts as part of file-based customization projects. You can also add SDF installation scripts to your SuiteApp project deployment.

SuiteApp projects can include SDF installation scripts, which run automatically when the project is deployed. For more information, see the following help topics:

- [Customizing SuiteApp Project Deployment using SDF Installation Scripts](#)
- [SDF Installation Scripts as XML Definitions](#)
- [Defining a Script Record and Deployment for an SDF Installation Script](#)
- [Customizing SuiteApp Project Deployment using SDF Installation Scripts](#)

You can monitor SDF installation scripts and deployments using the deploy log in SuiteCloud IDE or the Execution Log on the Script or Script Deployment page. For more information, see the help topics [Deployment Logs for SuiteCloud Projects](#) and [Handling Exceptions and Monitoring SDF Installation Scripts](#).

Copy to Account Support for SDF Installation Scripts

Use the Copy to Account feature to copy an SDF installation script to another account. Find the Copy To Account option in the top right corner of each SDF installation script page. For information about Copy to Account, see the help topic [Copy to Account](#).

SDF Installation Script Samples

SDF installation script samples are available in the following SuiteCloud Development Framework help topics:

- [SDF Installation Script Example of Validating an Account Preference Before Installation](#)
- [SDF Installation Script Example of Customizing a SuiteApp Update](#)

SuiteScript 2.x SDF Installation Script Entry Points and API

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following entry point is defined for the SDF installation script type:

Script Entry Point	Description
run(scriptContext)	Defines what is executed when the script is specified to be run by the SDF deployment (in the deploy.xml file of a SuiteCloud project).

run(scriptContext)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Description	Defines what is executed when the script is specified to be run by the SDF deployment (in the deploy.xml file of a SuiteCloud project).
Returns	void

Since	2015.2
-------	--------

Parameters

Note: The scriptContext parameter is a JavaScript object.

Parameter	Type	Description	Since
fromVersion	string	The current SuiteApp version installed on the account. Specify Null if this is a new installation.	2015.2
toVersion	string	The version of the SuiteApp that will be installed on the account.	2015.2

For examples of the run entry point, see [SDF Installation Script Samples](#).

You can also define custom script parameters by adding a parameter to the script record (defined as a custom script field) and setting the parameter to a particular value on the deployment. Here's an example of how to get a script parameter from an SDF installation script:

```

1  /**
2   * @NApiVersion 2.0
3   * @NScriptType SDFInstallationScript
4   */
5
6  define(['N/runtime'], function(runtime) {
7      function run(params) {
8          var customParam = runtime.getCurrentScript().getParameter({name: 'custscript1'});
9      }
10     return {
11         run: run
12     };
13 });

```

See the help topic [Creating Script Parameters \(Custom Fields\)](#) for more information about script parameters. See the help topic [SDF Installation Script Example of Customizing a SuiteApp Update](#) for an example.

SuiteScript 2.x Suitelet Script Type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Suitelets are extensions of the SuiteScript API that let you build custom NetSuite pages and backend logic. Suitelets are server scripts that use a request-response model and respond to HTTP GET or POST requests at system-generated URLs.

There are two types of Suitelets:

1. **Suitelets** use UI objects to create custom pages that look like NetSuite pages. SuiteScript UI objects encapsulate the elements for building NetSuite-looking portlets, forms, fields, sublists, tabs, lists, and columns.
2. **Backend Suitelets** do not use any UI objects and execute backend logic, which can then be parsed by other parts of a custom application. Backend Suitelets work best when you use them for:

- Providing a service for backend logic to other SuiteScripts.
- Offloading server logic from client scripts to a backend Suitelet shipped without source code to protect sensitive intellectual property.



Note: Suitelets are not intended for use in systems integration use cases.

Suitelets are not intended to work inside web stores. Use online forms if you want to embed forms in a web store.



Important: RESTlets can be used as an alternative to backend Suitelets.

The concurrent request limit for Suitelets available without login is the same as for RESTlets. For information about the account limits, see the help topic [Web Services and RESTlet Concurrency Governance](#). For Suitelets that require login, there's currently no limit on concurrent requests.

You can use SuiteCloud Development Framework (SDF) to manage Suitelets as part of file-based customization projects. For information about SDF, see . You can use the Copy to Account feature to copy a Suitelet to another one of your accounts. Each Suitelet page has a Copy to Account option in the top right corner you can click. For information about Copy to Account, see the help topic [Copy to Account](#).

Custom Header use:

- Authorization header is reserved for OAuth-authenticated requests. To implement a custom authentication layer, use a different header name; for example X-Authorization.

You can use SuiteScript Analysis to see when the script was installed and how it's performed over time. For more information, see the help topic [Analyzing Scripts](#).

See the following for more information about the Suitelet script type:

- [SuiteScript 2.x Suitelet Script Reference](#)
 - [How Suitelet Scripts are Executed](#)
 - [Reserved Parameter Names in Suitelet URLs](#)
 - [Suitelet Script Deployment Page](#)
 - [Embedding HTML in Suitelets](#)
- [SuiteScript 2.x Suitelet Script Entry Points and API onRequest\(params\)](#)
- [SuiteScript 2.x Suitelet Script Type Code Samples](#)

Also see the [Suitelets and UI Object Best Practices](#) section in the [SuiteScript Developer Guide](#) for a list of best practices to follow when using client scripts.

SuiteScript 2.x Suitelet Script Reference

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

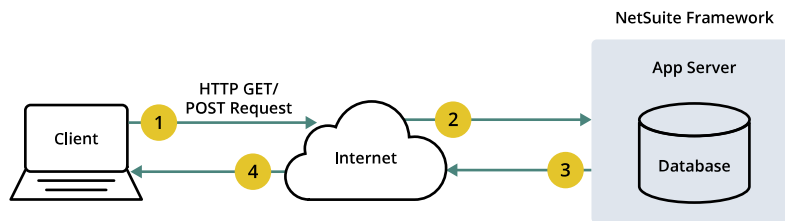
- [How Suitelet Scripts are Executed](#)

- [Reserved Parameter Names in Suitelet URLs](#)
- [Suitelet Script Deployment Page](#)
- [Embedding HTML in Suitelets](#)

How Suitelet Scripts are Executed

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following steps and diagram provide an overview of the Suitelet execution process:



1. Client initiates an HTTP GET or POST request (typically from a browser) for a system-generated URL. A web request object contains the data from the client's request. See the help topic [N/http Module](#).
2. The user's script is invoked, which gives the user access to the entire Server SuiteScript API as well as a web request and web response object.
3. NetSuite processes the user's script and returns a web response object to the client. The response can be in following forms:
 - Free-form text
 - HTML
 - RSS
 - XML
 - A browser redirect to another page on the Internet



Important: You can only redirect to external URLs from Suitelets that have the **Available Without Login** preference enabled on the script deployment.

- A browser redirect to an internal NetSuite page. The NetSuite page can be either a standard page or custom page that has been dynamically generated using UI objects.
4. The data shows up the user's browser.

Reserved Parameter Names in Suitelet URLs

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Some names are reserved and you shouldn't use them when naming custom parameters for Suitelet URLs.

The table below shows which parameter names are reserved:

Reserved Suitelet URL Parameter Names	
e	print
id	email
cp	q
l	si
popup	st
s	r
d	displayonly
_nodrop	nodisplay
sc	deploy
sticky	script
	ns-at

If any of your parameters use reserved names, your Suitelet might throw an error saying, "There are no records of this type." To avoid naming conflicts, you should prefix all custom URL parameters with **custom**. For example, use `custom_id` instead of `id`.

Suitelet Script Deployment Page

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Before you can run a Suitelet script, you need to create at least one deployment record for it.

A Suitelet deployment record is like other script types, but it has a few extra fields. This topic covers all available fields.

You can access a Suitelet script deployment record in the following ways:

- To edit a deployment record, go to Customization > Scripting > Script Deployments. Locate the appropriate record, and click the corresponding **Edit** link.
- To create a new deployment record, open the script record in view mode and click the **Deploy Script** button. For help creating a script record, see [Script Record Creation](#).

Suitelet Script Deployment Page Body Fields

The table below shows the body fields for Suitelet deployment records. Some fields only show up when you edit or view an existing record.

Field	Description
Script	A link to the script record for the deployment. You can't change it—even on a new deployment record. If you realize you picked the wrong record, you'll need to start over
Title	The user-defined name for the deployment.
ID	A unique identifier for the deployment.

	On a new record, you can set this ID in the ID field. If you plan to bundle the deployment for another account, it's best to use a custom ID to avoid naming conflicts. IDs must be lowercase and cannot use spaces. You can use an underscore as a delineator. If you don't enter an ID, the system creates one for you. In either case, it adds the customdeploy prefix when you save the record. Although it's not the preferred method, you can change the ID on an existing deployment by clicking Change ID.
Deployed	This setting shows if the deployment is active. You'll need to check this box for the script to run.
Status	The status can be set to Testing or Released. ■ Testing — The script runs for the script owner and chosen audience. ■ Released — The script runs for all specified audience members. See Errors Related to the Available Without Login URL to learn more about the relevance of the Released status when deploying internally and externally available Suitelets.
Event Type	Use the Event Type list to specify a script execution context at the time of script deployment. When you pick an event type, the script only runs for that event—even if the script file lists others.  Important: Event types you choose in the UI override those in the script file. For example, if you select delete in the Event Type list, the script only runs on delete—even if the script file has create. If you leave Event Type blank, your script only runs on the types listed in the script file.
Log Level	This setting controls what log messages show up in the Execution Log for deployment and script records. The available levels are: ■ Debug — suitable for scripts still being tested; this level shows all debug, audit, error and emergency messages. ■ Audit, Error, or Emergency — suitable for scripts in production mode. Audit is the most inclusive, and Emergency is the most limited.
Execute As Role	Shows which role runs the script.
Available Without Login	Shows if users without an active NetSuite session can access the Suitelet. See Setting Available Without Login .

Setting Available Without Login

When you check the **Available Without Login** box and save the script deployment record, an **External URL** appears on the Script Deployment page. Use this URL for Suitelets you want to make available to users without an active NetSuite session.

Only some SuiteScript APIs work in externally available Suitelets. If you want to use every SuiteScript API, your Suitelet needs a valid NetSuite session, which means users have logged in with their email and password.

Here are a few examples of when you might want to make a Suitelet externally available:

- Hosting one-off online forms, such as capturing partner conference registrations.
- Handling inbound partner communication, such as payment notifications from PayPal or Google Checkout, or creating an unsubscribe page for email campaigns—which needs account access but shouldn't require a login or hosted site.
- For third party integrations in which the Suitelet lives in the third-party Web site but needs to communicate to NetSuite using POST requests.

 **Note:** Suitelets are not intended for use in systems integration use cases.

 **Important:** Since Suitelets available without login don't require authentication, any data in them is less secure.

Errors Related to the Available Without Login URL

Depending on your Suitelet's use case, you'll use either the internal or external URL as the starting point. The following are some factors that determine whether a Suitelet will successfully run:

- Dependencies between the type of URL you are referencing (internal or external)
- The Suitelet deployment status (Testing or Released)
- Whether the external role **Online Form User** is selected in the External Roles field on the Audience subtab of the Script Deployment page.

 **Note:** To give internal users access, check the Select All box next to Internal Roles. To let external users who aren't logged in access the Suitelet, make sure **Online Form User** is selected in External Roles. Leave the other Audience subtab fields blank.

To be able to select the **Online Form User** role in the External Roles field, the Online Forms feature must be enabled. To enable the feature, go to Setup > Company > Enable Features. Click the CRM subtab, and check the **Online Forms** box in the Marketing section. If the role is still not available after enabling the feature, you may need to log out and log back in to see the updated roles list.

The following table summarizes these dependencies:

Suitelet URL Type	Deployment Status	Audience Configuration	Result
internal	Testing	The current user is included in the selected audience.	Suitelet executes successfully.
internal	Testing	The current user is not included the selected audience.	Suitelet executes successfully.
internal	Released	The current user is not included the selected audience.	Error message: You do not have privileges to view this page.
internal	Released	The current user is included in the selected audience.	Suitelet executes successfully.
external	Testing	The current user is not included the selected audience.	Error message: You are not allowed to go directly to this page.
external	Testing	The current user is included in the selected audience.	Error message: You are not allowed to go directly to this page.
external	Released	External role Online Form User is selected.	Suitelet executes successfully.

Suitelet URL Type	Deployment Status	Audience Configuration	Result
external	Released	External role Online Form User is not selected.	Error message: You do not have privileges to view this page.

Suitelet Script Deployment Page Audience Subtab

Use the Audience subtab on the Suitelet Script Deployment page to choose which roles can access your Suitelet.

The following table summarizes the Audience subtab fields.

Field	Description
Internal Roles	Choose the internal roles that can access your Suitelet. You need to select at least one role in either the Internal Roles or External Roles fields. To let all internal roles access it, check the Select All box.
External Roles	Choose the external roles that can access your Suitelet. You need to select at least one role in either the Internal Roles or External Roles fields. To give access to all external roles, select all external roles in the External Roles field.  Note: When giving access to multiple roles, especially external ones, make sure each role really needs access to the script.
Groups	Choose the groups that can access your Suitelet. If you don't select any, all groups can access it.
Partners	Choose the partners who can access your Suitelet. To let all partners access it, check the All Partners box.
Departments	Choose the departments that can access your Suitelet. If you don't pick any, all departments will have access.
Employees	Choose the employees that can access your Suitelet. To let all employees access it, check the All Employees box.

Suitelet Script Deployment Page Links Subtab

Use the Links subtab on the Suitelet Script Deployment page to add links to your Suitelet in NetSuite Centers. For example, create a link to a Suitelet from the Support Section in the Classic Center.

 **Note:** For scripts to run in the Customer, Vendor, Partner, or Employee Center, make sure you've selected the right role on the Audience subtab. Also, script files and any other files they use must have either Company-wide Usage or Available Without Login turned on in the File Cabinet. For more information, see the help topic [Permissions and File Cabinet Files](#).

The following table summarizes the Links subtab fields.

Field	Description
Center	The NetSuite Centers users can use to access your Suitelet.
Section	The NetSuite section of your Suitelet. Users access your Suitelet from this section.

Category	The Suitelet's category.
Label	The Suitelet's name. Users will see this label in the section menu after the category name.
Translation	If you've enabled Multi-Language for your NetSuite account, you can add translations for the Suitelet label in the Translation column.
Insert Before	Specify the section category you would like to insert your Suitelet before.

Embedding HTML in Suitelets

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The examples below show how to embed HTML in Suitelet code to add custom elements to a form. Embedding HTML in Suitelets is useful for adding components that are not available through the SuiteScript N/ui/serverWidget module.

- [Embedding Inline HTML in a Field](#)
- [Embedding HTML from a Linked HTML Page in a Suitelet](#)

Embedding Inline HTML in a Field

In a Suitelet, you can use [Form.addField\(options\)](#) to add inline HTML as a field on a form. Set the type to `INLINEHTML` and use the [Field.defaultValue](#) property to set the HTML value. For more information, see the help topic [N/ui/serverWidget Module](#).

```

1 var htmlImage = form.addField({
2   id: 'custpage_htmlfield',
3   type: serverWidget.FieldType.INLINEHTML,
4   label: 'HTML Image'
5 });
6 htmlImage.defaultValue = "<img src='https://<accountID>.app.netsuite.com/images/logos/netsuite-oracle.svg' alt='Oracle NetSuite logo'>";

```

The following screenshot shows a form with an `INLINEHTML` field containing an image field.

Embedding HTML from a Linked HTML Page in a Suitelet

In a Suitelet, you can use [https.get\(options\)](#) from the [N/https Module](#) module to embed HTML from a linked HTML page in the form created by the Suitelet. The Suitelet handles any data users submit to the HTML page. When you use a linked HTML page, you manage HTML code through strings, so it can be harder to manipulate data than it's with components created with [N/ui/serverWidget Module](#) methods. To pass data in the HTML string, your Suitelet code has to update values inside that string. As you add more values, the string gets more complicated.

The example below shows a simple volunteer sign-up sheet with two parts: the Suitelet code and the HTML. This Suitelet code uses the [https.get\(options\)](#) method from the `N/https` module to access the

content from the HTML page. It uses methods from the [N/record Module](#), [N/email Module](#), and [N/search Module](#) to collect, process, and respond to user-submitted data.

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Suitelet
4   */
5
6  define(['N/https', 'N/record', 'N/email', 'N/search'],
7    function callbackFunction(https, record, email, search) {
8      function getFunction(context) {
9        var contentRequest = https.get({
10          url: "https://LinkToFormPage.html" // see the next snippet
11        });
12        var contentDocument = contentRequest.body;
13        var sponsorid = context.request.parameters.sponsorid;
14
15        if (sponsorid && sponsorid != "" && sponsorid != null) {
16          contentDocument = contentDocument.replace("{{sponsorid}}", sponsorid);
17          log.debug("Setting Sponsor", sponsorid)
18        }
19
20        var projectid = context.request.parameters.projectid;
21
22        if (projectid && projectid != "" && projectid != null) {
23          contentDocument = contentDocument.replace("{{projectid}}", projectid);
24          log.debug("Setting Project", projectid);
25        }
26
27        context.response.write(contentDocument);
28      }
29
30      function postFunction(context) {
31        var params = context.request.parameters;
32        var emailString = "First Name: {{firstname}}\nLast Name: {{lastname}}\nEmail: {{email}}\nFacebook URL: {{custentity_fb_url}}\n";
33        var contactRecord = record.create({
34          type: "contact",
35          isDynamic: true
36        });
37
38        for (param in params) {
39          if (param === "company") {
40            if (params[param] != "{{sponsorid}}") {
41              contactRecord.setValue({
42                fieldId: param,
43                value: params[param]
44              });
45              var lkpfld = search.lookupFields({
46                type: "vendor",
47                id: params["company"],
48                columns: ["entityid"]
49              });
50              emailString += "\nSponsor: " + lkpfld.entityid;
51            }
52            else {
53              contactRecord.setValue({
54                fieldId: "custentity_sv_shn_isindi",
55                value: true
56              });
57            }
58          }
59          else {
60            if (param != "project") {
61              contactRecord.setValue({
62                fieldId: param,
63                value: params[param]
64              });
65              var replacer = "{{" + param + "}}";
66              emailString = emailString.replace(replacer, params[param]);
67            }
68          }
69        }
70      }
71    }
72  );

```

```

69     }
70
71     var contactId = contactRecord.save({
72         ignoreMandatoryFields: true,
73         enableSourcing: true
74     });
75
76     log.debug("Record ID", contactId);
77
78     if (params["project"] && params["project"] !== "" && params["project"] != null && params
79     ["project"] !== "{projectId}") {
80         var lkpfld = search.lookupFields({
81             type: "job",
82             id: params["project"],
83             columns: ["companyname"]
84         });
85
86         emailString += "\nProject Name: " + lkpfld.companyname;
87
88         var participationRec = record.create({
89             type: "customrecord_project_participants",
90             isDynamic: true
91         });
92
93         participationRec.setValue({
94             fieldId: "custrecord_participants_volunteer",
95             value: contactId
96         })
97
98         participationRec.setValue({
99             fieldId: "custrecord_participants_project",
100             value: params["project"]
101         })
102
103         var participationId = participationRec.save({
104             enableSourcing: true,
105             ignoreMandatoryFields: true
106         })
107     }
108
109     log.debug("Email String", emailString);
110
111     email.send({
112         author: -5,
113         recipients: 256,
114         subject: "New Volunteer Signed Up",
115         body: "A new volunteer has joined:\n\n" + emailString
116     });
117
118     email.send({
119         author: -5,
120         recipients: params["email"],
121         subject: "Thank you!",
122         body: "Thank you for volunteering:\n\n" + emailString
123     });
124
125     var contentRequest = https.get({
126         url: "https://LinkToFormCompletePage.html"
127     });
128
129     var contentDocument = contentRequest.body;
130
131     context.response.write(contentDocument);
132
133 }
134 function onRequestFxn(context) {
135     if (context.request.method === "GET") {
136         getFunction(context)
137     }
138     else {
139         postFunction(context)
140     }
141 }

```

```

142     }
143     return {
144         onRequest: onRequestFxn
145     };
146 });

```

The following HTML, contained in the LinkToFormPage.html file (see above snippet), creates the form that the Suitelet links to.

```

1 <form method="post" class="form-horizontal" action="https://LinkToSuitelet.js">
2   <table>
3     <tbody><tr>
4       <td>First Name</td>
5       <td class="col-md-8"><input class="form-control" id="firstname" placeholder="First Name" name="firstname" re
6     quired="" type="text"></td>
7     </tr>
8     <tr>
9       <td>Last Name</td>
10      <td class="col-md-8"><input class="form-control" id="lastname" placeholder="Last Name" name="lastname" re
11    quired="" type="text"></td>
12    </tr>
13    <tr>
14      <td>Email</td>
15      <td class="col-md-8"><input class="form-control" id="email" placeholder="email" name="email" required="" type="e
16    mail"></td>
17    </tr>
18    <tr>
19      <td>Facebook URL</td>
20      <td class="col-md-8"><input class="form-control" id="custentity_fb_url" placeholder="Facebook" name="custentity_f
21    b_url" required="" type="text"></td>
22    </tr>
23    <tr>
24      <td><input name="company" value="{{sponsoid}}" type="hidden">
25      <input name="project" value="{{projectid}}" type="hidden">
26    </td>
27    </tr>
28  </tbody></table>
29  <br>
30  <button type="submit" class="btn btn-inverse">Sign Up as a Volunteer</button>
31</form>

```

The following screenshot illustrates the form that is created by the linked HTML page.

The screenshot shows a web form with the following elements:

- First Name:** A text input field with the placeholder "First Name".
- Last Name:** A text input field with the placeholder "Last Name".
- Email:** A text input field with the placeholder "email".
- Facebook URL:** A text input field with the placeholder "Facebook".
- Sign Up as a Volunteer:** A dark blue button with white text.

SuiteScript 2.x Suitelet Script Entry Points and API

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Script Entry Point	Description
onRequest(params)	Defines the function that executes when the Suitelet is triggered.

onRequest(params)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Description	Defines the Suitelet script trigger point.
Returns	void
Since	Version 2015 Release 2

Parameters

Note: The `params` parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
<code>params.request</code>	http.ServerRequest	required	The incoming request.	Version 2015 Release 2
<code>params.response</code>	http.ServerResponse	required	The Suitelet response.	Version 2015 Release 2

SuiteScript 2.x Suitelet Script Type Code Samples

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

This topic contains examples of different types of Suitelets: Suitelets used to create custom pages (forms, list or assistants), and backend Suitelets.

For help with writing scripts in SuiteScript 2.x, see [SuiteScript 2.x Hello World](#) and [SuiteScript 2.x Entry Point Script Creation and Deployment](#).

- **Basic Samples**
 - [Write Your First Suitelet](#)
 - [Return a Simple XML Document](#)
 - [Add a Suitelet to a Tab](#)
- **Custom Forms**
 - [Write and Send Email](#)
 - [Create a Custom Form with an Inline Editor Sublist](#)
 - [Create a Custom Survey Form](#)
 - [Create a Form Containing Several Field Types, Reset and Submit Buttons, Tabs, and a Sublist](#)
 - [Create and Use Secret Keys](#)
 - [Create a Form with a Credential Field](#)
 - [Add a Secret Key Field to a Form](#)
 - [Add a Field that Displays Running Total to a Sublist](#)
 - [Parse Strings and Display Result](#)
- **Custom Lists**
 - [Create a Custom List](#)

- Custom Assistants
 - Sample Custom Assistant Script
- Suitelets with Embedded HTML
- Backend Suitelets
 - Find Plugin Implementations
 - Load an XML File and Obtain Child Element Values
 - Parse an XML File and Log Element Values
 - Redirect a New Sales Order
 - Render a PDF
 - Render Search Results into a PDF File
 - Retrieve the Name of the City Based on a ZIP Code
 - Return a File with Alternative Character Encoding
 - Write User and Session Information to the Response

Basic Samples

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

- Write Your First Suitelet
- Return a Simple XML Document
- Add a Suitelet to a Tab

Write Your First Suitelet

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

i Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see the help topic [SuiteScript 2.x Global Objects](#).

The following sample creates a Suitelet that shows an empty page with a Hello World header.

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Suitelet
4   */
5  define([], function() {
6      function onRequest(context) {
7          var html = '<html><body><h1>Hello World</h1></body></html>';
8          context.response.write(html);
9          context.response.setHeader({
10             name: 'Custom-Header-Demo',
11             value: 'Demo'
12         });
13     }
14
15     return {
16         onRequest: onRequest
17     };
18 });

```

The following screenshot shows the output of this script.

Hello World

Return a Simple XML Document

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see the help topic [SuiteScript 2.x Global Objects](#).

The following sample creates a Suitelet that returns a simple XML document. For information about executing Suitelet scripts, see [SuiteScript 2.x Suitelet Script Reference](#).

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Suitelet
4   */
5   define([], function() {
6       function onRequest(context) {
7           var xml = '<?xml version="1.0" encoding="utf-8" ?>' +
8               '<message>' + 'Hello World' + '</message>';
9           context.response.write(xml);
10          context.response.setHeader({
11              name: 'Custom-Header-Demo',
12              value: 'Demo'
13          });
14      }
15
16      return {
17          onRequest: onRequest
18      };
19  });

```

The following screenshot shows the output of this script.

```
<?xml version="1.0" encoding="utf-8" ?><message>Hello World</message>
```

Add a Suitelet to a Tab

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see the help topic [SuiteScript Debugger](#).

The following sample shows how to add a link to a Suitelet to a tab in a user event script.

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType UserEventScript

```

```

4  */
5  define(['N/record', 'N/runtime', 'N/ui/serverWidget', 'N/url'], function(record, runtime, serverWidget, url) {
6      function beforeLoad(scriptContext) {
7          var currentUserID = runtime.getCurrentUser().id;
8          /* If the record is edited or viewed, a tab called Sample Tab is added.
9           * Note that the script execution context is set to userinterface.
10          * This ensures that this script is ONLY invoked from a user event
11          * occurring through the UI.
12          */
13          if ((runtime.executionContext === runtime.ContextType.USER_INTERFACE) && (scriptContext.type === scriptContext.UserEventType.EDIT || scriptContext.type === scriptContext.UserEventType.VIEW)) {
14              //Creates the new tab Sample Tab on the form
15              var SampleTab = scriptContext.form.addTab({
16                  id: 'custpage_sample_tab',
17                  label: 'Sample Tab'
18              });
19
20              //On Sample Tab, create a field of type inlinehtml
21              var createNewReqLink = scriptContext.form.addField({
22                  id: 'custpage_new_req_link',
23                  type: 'inlinehtml',
24                  label: ' ',
25                  container: 'custpage_sample_tab'
26              });
27
28              //Define the parameters of the Suitelet that will be executed
29              //Replace the scriptId and the deploymentId with the values of your Suitelet
30              var linkURL = url.resolveScript({
31                  scriptId: 'customscriptsuiteletsample_yourfirstsamp',
32                  deploymentId: 'customdeploysuiteletsample_yourfirstsamp'
33              }) + '&recordid=' + scriptContext.newRecord.id;
34
35              //Create a link to launch the Suitelet.
36              createNewReqLink.defaultValue = '<B>Click <A HREF="' + linkURL + '">here</A> to create a new document signature request record.</B>';
37
38              //Add a sublist to Sample Tab
39              var signatureRequestSublist = scriptContext.form.addSublist({
40                  id: 'custpage_sig_req_sublist',
41                  type: 'list',
42                  label: 'Document Signature Requests',
43                  tab: 'custpage_sample_tab'
44              });
45              signatureRequestSublist.addField({
46                  id: 'custpage_req_name',
47                  type: 'text',
48                  label: 'Name'
49              });
50              signatureRequestSublist.addField({
51                  id: 'custpage_req_status',
52                  type: 'text',
53                  label: 'Status'
54              });
55              signatureRequestSublist.addField({
56                  id: 'custpage_req_created',
57                  type: 'date',
58                  label: 'Date Created'
59              });
60          }
61      }
62
63      return {
64          beforeLoad: beforeLoad,
65      };
66  });

```

Custom Forms

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

- Write and Send Email

- Create a Custom Form with an Inline Editor Sublist
- Create a Custom Survey Form
- Create a Form Containing Several Field Types, Reset and Submit Buttons, Tabs, and a Sublist
- Create and Use Secret Keys
- Create a Form with a Credential Field
- Add a Secret Key Field to a Form
- Add a Field that Displays Running Total to a Sublist
- Parse Strings and Display Result

Write and Send Email

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

i Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see the help topic [SuiteScript Debugger](#).

The following sample creates a Suitelet form that lets you write and send an email.

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Suitelet
4   */
5  /**
6   * A simple Suitelet for building an email form and sending out an email
7   * from the current user to the recipient email address specified on the form.
8   */
9  define(['N/email', 'N/runtime', 'N/ui/serverWidget'], function(email, runtime, serverWidget) {
10     function onRequest(context) {
11         if (context.request.method === 'GET') {
12             var form = serverWidget.createForm({
13                 title: 'Email Form'
14             });
15
16             var subject = form.addField({
17                 id: 'subject',
18                 type: serverWidget.FieldType.TEXT,
19                 label: 'Subject'
20             });
21             subject.layoutType = serverWidget.FieldLayoutType.NORMAL;
22             subject.updateBreakType = serverWidget.FieldBreakType.STARTCOL;
23             subject.isMandatory = true;
24
25             var recipient = form.addField({
26                 id: 'recipient',
27                 type: serverWidget.FieldType.EMAIL,
28                 label: 'Recipient email'
29             });
30             recipient.isMandatory = true;
31
32             var message = form.addField({
33                 id: 'message',
34                 type: serverWidget.FieldType.TEXTAREA,
35                 label: 'Message'
36             });
37             message.updateDisplaySize({
38                 height: 10,
39                 width: 60
40             });
41         }
42     }
43 }
```

```

40
41     form.addSubmitButton({
42         label: 'Send Email'
43     });
44
45     context.response.writePage(form);
46 }
47 else
48 {
49     var request = context.request;
50     var currentUser = runtime.getCurrentUser().id;
51     var subject = request.parameters.subject;
52     var recipient = request.parameters.recipient;
53     var message = request.parameters.message;
54     email.send({
55         author: currentUser,
56         recipients: recipient,
57         subject: subject,
58         body: message
59     });
60 }
61 }
62
63 return {
64     onRequest: onRequest
65 };
66 });

```

Create a Custom Form with an Inline Editor Sublist

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample creates a Suitelet that generates a sample form with a submit button, fields, and an inline editor sublist.

Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see the help topic [SuiteScript 2.x Global Objects](#).

Important: This sample uses SuiteScript 2.1. For more information, see [SuiteScript 2.1](#).

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType Suitelet
4   */
5  define(['N/ui/serverWidget'], (serverWidget) => {
6      const onRequest = (scriptContext) => {
7          if (scriptContext.request.method === 'GET') {
8              let form = serverWidget.createForm({
9                  title: 'Simple Form'
10             });
11
12             let field = form.addField({
13                 id: 'textfield',
14                 type: serverWidget.FieldType.TEXT,
15                 label: 'Text'
16             });
17             field.layoutType = serverWidget.FieldLayoutType.NORMAL;
18             field.updateBreakType({
19                 breakType: serverWidget.FieldBreakType.STARTCOL
20             });
21
22             form.addField({

```

```

23         id: 'datefield',
24         type: serverWidget.FieldType.DATE,
25         label: 'Date'
26     });
27     form.addField({
28         id: 'currencyfield',
29         type: serverWidget.FieldType.CURRENCY,
30         label: 'Currency'
31     });
32
33     let select = form.addField({
34         id: 'selectfield',
35         type: serverWidget.FieldType.SELECT,
36         label: 'Select'
37     });
38     select.addSelectOption({
39         value: 'a',
40         text: 'Albert'
41     });
42     select.addSelectOption({
43         value: 'b',
44         text: 'Baron'
45     });
46
47     let sublist = form.addSublist({
48         id: 'sublist',
49         type: serverWidget.SublistType.INLINEEDITOR,
50         label: 'Inline Editor Sublist'
51     });
52     sublist.addField({
53         id: 'sublist1',
54         type: serverWidget.FieldType.DATE,
55         label: 'Date'
56     });
57     sublist.addField({
58         id: 'sublist2',
59         type: serverWidget.FieldType.TEXT,
60         label: 'Text'
61     });
62
63     form.addSubmitButton({
64         label: 'Submit Button'
65     });
66
67     scriptContext.response.writePage(form);
68 } else {
69     const delimiter = /\u0001/;
70     const textField = scriptContext.request.parameters.textfield;
71     const dateField = scriptContext.request.parameters.datefield;
72     const currencyField = scriptContext.request.parameters.currencyfield;
73     const selectField = scriptContext.request.parameters.selectfield;
74     const sublistData = scriptContext.request.parameters.sublistdata.split(delimiter);
75     const sublistField1 = sublistData[0];
76     const sublistField2 = sublistData[1];
77
78     scriptContext.response.write(`You have entered: ${textField} ${dateField} ${currencyField} ${selectField} ${sublist
Field1} ${sublistField2}`);
79 }
80 }
81
82 return {onRequest}
83 });

```

Create a Custom Survey Form

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample creates a Suitelet that generates a customer survey form with inline HTML fields, radio fields, and a submit button.

Note: This script sample uses the define function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the require function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see the help topic [SuiteScript 2.x Global Objects](#).

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Suitelet
4   */
5   define(['N/ui/serverWidget'], function(serverWidget) {
6       function onRequest(context) {
7           var form = serverWidget.createForm({
8               title: 'Thank you for your interest in Wolfe Electronics',
9               hideNavBar: true
10          });
11
12          var htmlHeader = form.addField({
13              id: 'custpage_header',
14              type: serverWidget.FieldType.INLINEHTML,
15              label: ' '
16          }).updateLayoutType({
17              layoutType: serverWidget.FieldLayoutType.OUTSIDEABOVE
18          }).updateBreakType({
19              breakType: serverWidget.FieldBreakType.STARTROW
20          }).defaultValue = '<p style=\''font-size:20px\''>We pride ourselves on providing the best' +
21              ' services and customer satisfaction. Please take a moment to fill out our survey.</p><br><br>';
22
23          var htmlInstruct = form.addField({
24              id: 'custpage_p1',
25              type: serverWidget.FieldType.INLINEHTML,
26              label: ' '
27          }).updateLayoutType({
28              layoutType: serverWidget.FieldLayoutType.OUTSIDEABOVE
29          }).updateBreakType({
30              breakType: serverWidget.FieldBreakType.STARTROW
31          }).defaultValue = '<p style=\''font-size:14px\''>When answering questions on a scale of 1 to 10,' +
32              ' 1 = Greatly Unsatisfied and 10 = Greatly Satisfied.</p><br><br>';
33
34          var productRating = form.addField({
35              id: 'custpage_lblproductrating',
36              type: serverWidget.FieldType.INLINEHTML,
37              label: ' '
38          }).updateLayoutType({
39              layoutType: serverWidget.FieldLayoutType.NORMAL
40          }).updateBreakType({
41              breakType: serverWidget.FieldBreakType.STARTROW
42          }).defaultValue = '<p style=\''font-size:14px\''>How would you rate your satisfaction with our products?</p>';
43
44          form.addField({
45              id: 'custpage_rdoproducrating',
46              type: serverWidget.FieldType.RADIO,
47              label: '1',
48              source: 'p1'
49          }).updateLayoutType({
50              layoutType: serverWidget.FieldLayoutType.STARTROW
51          });
52          form.addField({
53              id: 'custpage_rdoproducrating',
54              type: serverWidget.FieldType.RADIO,
55              label: '2',
56              source: 'p2'
57          }).updateLayoutType({
58              layoutType: serverWidget.FieldLayoutType.MIDROW
59          });
60          form.addField({
61              id: 'custpage_rdoproducrating',
62              type: serverWidget.FieldType.RADIO,
63              label: '3',
64              source: 'p3'
65          }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.MIDROW});

```

```

66     form.addField({
67         id: 'custpage_rdoproducrating',
68         type: serverWidget.FieldType.RADIO,
69         label: '4',
70         source: 'p4'
71     }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.MIDROW});
72     form.addField({
73         id: 'custpage_rdoproducrating',
74         type: serverWidget.FieldType.RADIO,
75         label: '5',
76         source: 'p5'
77     }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.MIDROW});
78     form.addField({
79         id: 'custpage_rdoproducrating',
80         type: serverWidget.FieldType.RADIO,
81         label: '6',
82         source: 'p6'
83     }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.MIDROW});
84     form.addField({
85         id: 'custpage_rdoproducrating',
86         type: serverWidget.FieldType.RADIO,
87         label: '7',
88         source: 'p7'
89     }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.MIDROW});
90     form.addField({
91         id: 'custpage_rdoproducrating',
92         type: serverWidget.FieldType.RADIO,
93         label: '8',
94         source: 'p8'
95     }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.MIDROW});
96     form.addField({
97         id: 'custpage_rdoproducrating',
98         type: serverWidget.FieldType.RADIO,
99         label: '9',
100        source: 'p9'
101    }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.MIDROW});
102    form.addField({
103        id: 'custpage_rdoproducrating',
104        type: serverWidget.FieldType.RADIO,
105        label: '10',
106        source: 'p10'
107    }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.ENDROW});
108
109    var serviceRating = form.addField({
110        id: 'custpage_lblservicerating',
111        type: serverWidget.FieldType.INLINEHTML,
112        label: ' '
113    }).updateLayoutType({
114        layoutType: serverWidget.FieldLayoutType.NORMAL
115    }).updateBreakType({
116        breakType: serverWidget.FieldBreakType.STARTROW
117    }).defaultVaue = '<p style=\"font-size:14px\">How would you rate your satisfaction with our services?</p>';
118
119    form.addField({
120        id: 'custpage_rdoservicerating',
121        type: serverWidget.FieldType.RADIO,
122        label: '1',
123        source: 'p1'
124    }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.STARTROW});
125    form.addField({
126        id: 'custpage_rdoservicerating',
127        type: serverWidget.FieldType.RADIO,
128        label: '2',
129        source: 'p2'
130    }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.MIDROW});
131    form.addField({
132        id: 'custpage_rdoservicerating',
133        type: serverWidget.FieldType.RADIO,
134        label: '3',
135        source: 'p3'
136    }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.MIDROW});
137    form.addField({
138        id: 'custpage_rdoservicerating',

```

```

139         type: serverWidget.FieldType.RADIO,
140         label: '4',
141         source: 'p4'
142     }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.MIDROW});
143     form.addField({
144         id: 'custpage_rdoservicerating',
145         type: serverWidget.FieldType.RADIO,
146         label: '5',
147         source: 'p5'
148     }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.MIDROW});
149     form.addField({
150         id: 'custpage_rdoservicerating',
151         type: serverWidget.FieldType.RADIO,
152         label: '6',
153         source: 'p6'
154     }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.MIDROW});
155     form.addField({
156         id: 'custpage_rdoservicerating',
157         type: serverWidget.FieldType.RADIO,
158         label: '7',
159         source: 'p7'
160     }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.MIDROW});
161     form.addField({
162         id: 'custpage_rdoservicerating',
163         type: serverWidget.FieldType.RADIO,
164         label: '8',
165         source: 'p8'
166     }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.MIDROW});
167     form.addField({
168         id: 'custpage_rdoservicerating',
169         type: serverWidget.FieldType.RADIO,
170         label: '9',
171         source: 'p9'
172     }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.MIDROW});
173     form.addField({
174         id: 'custpage_rdoservicerating',
175         type: serverWidget.FieldType.RADIO,
176         label: '10',
177         source: 'p10'
178     }).updateLayoutType({layoutType: serverWidget.FieldLayoutType.ENDROW});
179
180     form.addSubmitButton({
181         label: 'Submit'
182     });
183
184     context.response.writePage(form);
185 }
186
187 return {
188     onRequest: onRequest
189 };
190 });

```

Create a Form Containing Several Field Types, Reset and Submit Buttons, Tabs, and a Sublist

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see the help topic [SuiteScript 2.x Global Objects](#).

The following code creates a Suitelet that generates a custom form containing several field types, tabs, a sublist, and a submit button. For steps to create this script, see [Sample Custom Form Script](#).

1 | /**

```

2  * @ApiVersion 2.x
3  * @NScriptType Suitelet
4  */
5  define(['N/ui/serverWidget'], function(serverWidget) {
6      function onRequest(context) {
7          if (context.request.method === 'GET') {
8
9              // Section One - Forms - See 'Steps for Creating a Custom Form' in topic 'Sample Custom Form Script'
10             var form = serverWidget.createForm({
11                 title: 'Customer Information'
12             });
13
14             var usergroup = form.addFieldGroup({
15                 id: 'usergroup',
16                 label: 'User Information'
17             });
18             usergroup.isSingleColumn = true;
19
20             var companygroup = form.addFieldGroup({
21                 id: 'companygroup',
22                 label: 'Company Information'
23             });
24
25             var select = form.addField({
26                 id: 'titlefield',
27                 type: serverWidget.FieldType.SELECT,
28                 label: 'Title',
29                 container: 'usergroup'
30             });
31             select.addSelectOption({
32                 value: 'Mr.',
33                 text: 'Mr.'
34             });
35             select.addSelectOption({
36                 value: 'Ms.',
37                 text: 'Ms.'
38             });
39             select.addSelectOption({
40                 value: 'Dr.',
41                 text: 'Dr.'
42             });
43
44             var fname = form.addField({
45                 id: 'fnamefield',
46                 type: serverWidget.FieldType.TEXT,
47                 label: 'First Name',
48                 container: 'usergroup'
49             });
50             fname.isMandatory = true;
51
52             var lname = form.addField({
53                 id: 'lnamefield',
54                 type: serverWidget.FieldType.TEXT,
55                 label: 'Last Name',
56                 container: 'usergroup'
57             });
58             lname.isMandatory = true;
59
60             form.addField({
61                 id: 'emailfield',
62                 type: serverWidget.FieldType.EMAIL,
63                 label: 'Email',
64                 container: 'usergroup'
65             });
66
67             var companyname = form.addField({
68                 id: 'companyfield',
69                 type: serverWidget.FieldType.TEXT,
70                 label: 'Company',
71                 container: 'companygroup'
72             });
73             companyname.defaultValue = 'Company Name';
74

```

```

75     form.addField({
76         id: 'phonefield',
77         type: serverWidget.FieldType.PHONE,
78         label: 'Phone Number',
79         container: 'companygroup'
80     });
81
82     form.addField({
83         id: 'urlfield',
84         type: serverWidget.FieldType.URL,
85         label: 'Website',
86         container: 'companygroup'
87     });
88
89     form.addSubmitButton({
90         label: 'Submit'
91     });
92
93     // Section Two - Tabs - See 'Steps for Adding a Tab to a Form' in topic 'Sample Custom Form Script'
94     var tab1 = form.addTab({
95         id: 'tab1id',
96         label: 'Payment'
97     });
98     tab1.helpText = 'Help Text Goes Here';
99
100    var tab2 = form.addTab({
101        id: 'tab2id',
102        label: 'Inventory'
103    });
104
105    form.addSubtab({
106        id: 'subtab1id',
107        label: 'Payment Information',
108        tab: 'tab1id'
109    });
110
111    form.addSubtab({
112        id: 'subtab2id',
113        label: 'Transaction Record',
114        tab: 'tab1id'
115    });
116
117    // Subtab One Fields
118    var ccselect = form.addField({
119        id: 'ccselectfield',
120        type: serverWidget.FieldType.SELECT,
121        label: 'Credit Card',
122        container: 'subtab1id'
123    });
124    ccselect.addSelectOption({
125        value: 'PayCard0',
126        text: 'Payment Card 0'
127    });
128    ccselect.addSelectOption({
129        value: 'PayCard1',
130        text: 'Payment Card 1'
131    });
132    ccselect.addSelectOption({
133        value: 'PayCard2',
134        text: 'Payment Card 2'
135    });
136
137    var expmonth = form.addField({
138        id: 'expmonth',
139        type: serverWidget.FieldType.SELECT,
140        label: 'Expiry Date:',
141        container: 'subtab1id'
142    });
143    expmonth.updateLayoutType({
144        layoutType: serverWidget.FieldLayoutType.STARTROW
145    });
146    expmonth.addSelectOption({
147        value: '01',

```



```

148         text: 'Jan'
149     });
150     expmonth.addSelectOption({
151         value: '02',
152         text: 'Feb'
153     });
154     expmonth.addSelectOption({
155         value: '03',
156         text: 'Mar'
157     });
158     expmonth.addSelectOption({
159         value: '04',
160         text: 'Apr'
161     });
162     expmonth.addSelectOption({
163         value: '05',
164         text: 'May'
165     });
166     expmonth.addSelectOption({
167         value: '06',
168         text: 'Jun'
169     });
170     expmonth.addSelectOption({
171         value: '07',
172         text: 'Jul'
173     });
174     expmonth.addSelectOption({
175         value: '08',
176         text: 'Aug'
177     });
178     expmonth.addSelectOption({
179         value: '09',
180         text: 'Sep'
181     });
182     expmonth.addSelectOption({
183         value: '10',
184         text: 'Oct'
185     });
186     expmonth.addSelectOption({
187         value: '11',
188         text: 'Nov'
189     });
190     expmonth.addSelectOption({
191         value: '12',
192         text: 'Dec'
193     });
194
195     var expyear = form.addField({
196         id: 'expyear',
197         type: serverWidget.FieldType.SELECT,
198         label: 'Expiry Year',
199         container: 'subtablid'
200     });
201     expyear.updateLayoutType({
202         layoutType: serverWidget.FieldLayoutType.ENDROW
203     });
204     expyear.addSelectOption({
205         value: '2020',
206         text: '2020'
207     });
208     expyear.addSelectOption({
209         value: '2019',
210         text: '2019'
211     });
212     expyear.addSelectOption({
213         value: '2018',
214         text: '2018'
215     });
216
217     var credfield = form.addCredentialField({
218         id: 'credfield',
219         label: 'Credit Card Number',
220         restrictToDomains: 'www.mysite.com',

```

```

221         restrictToScriptIds: 'customscript_my_script',
222         restrictToCurrentUser: false,
223         container: 'subtablid'
224     });
225     credfield.maxLength = 32;
226
227     // Subtab two Fields
228     form.addField({
229         id: 'transactionfield',
230         type: serverWidget.FieldType.LABEL,
231         label: 'Transaction History - Coming Soon',
232         container: 'subtab2id'
233     });
234
235     // Tab Two Fields
236     form.addField({
237         id: 'inventoryfield',
238         type: serverWidget.FieldType.LABEL,
239         label: 'Inventory - Coming Soon',
240         container: 'tab2id'
241     });
242
243     // Section Three - Sublist - See 'Steps for Adding a Sublist to a Form' in topic 'Sample Custom Form Script'
244     var sublist = form.addSublist({
245         id: 'sublistid',
246         type: serverWidget.SublistType.INLINEEDITOR,
247         label: 'Inline Sublist',
248         tab: 'tab2id'
249     });
250     sublist.addButton({
251         id: 'buttonId',
252         label: 'Print ',
253         functionName: '' // Add the function triggered on button click
254     });
255
256     // Sublist Fields
257     sublist.addField({
258         id: 'datefieldid',
259         type: serverWidget.FieldType.DATE,
260         label: 'Date'
261     });
262
263     sublist.addField({
264         id: 'productfieldid',
265         type: serverWidget.FieldType.TEXT,
266         label: 'Product'
267     });
268
269     sublist.addField({
270         id: 'qtyfieldid',
271         type: serverWidget.FieldType.INTEGER,
272         label: 'Quantity'
273     });
274
275     sublist.addField({
276         id: 'upfieldid',
277         type: serverWidget.FieldType.CURRENCY,
278         label: 'Unit Cost'
279     });
280
281     context.response.writePage(form);
282 } else {
283     // Section Four - Output - Used in all sections
284     var delimiter = /\u0001/;
285     var titleField = context.request.parameters.titlefield;
286     var fnameField = context.request.parameters.fnamefield;
287     var lnameField = context.request.parameters.lnamefield;
288     var emailField = context.request.parameters.emailfield;
289     var companyField = context.request.parameters.companyfield;
290     var phoneField = context.request.parameters.phonefield;
291     var urlField = context.request.parameters.urlfield;
292     var ccField = context.request.parameters.cctypefield;
293     var ccNumber = context.request.parameters.credfield;

```

```

294     var expMonth = context.request.parameters.expmonth;
295     var expYear = context.request.parameters.expyear;
296
297     context.response.write('You have entered:');
298     + '<br/> Name: ' + titleField + ' ' + fnameField + ' ' + lnameField
299     + '<br/> Email: ' + emailField
300     + '<br/> Company: ' + companyField
301     + '<br/> Phone: ' + phoneField + ' Website: ' + urlField
302     + '<br/> Credit Card: ' + ccField
303     + '<br/> Number: ' + ccNumber
304     + '<br/> Expiry Date: ' + expMonth + '/' + expYear);
305 }
306 }
307 return {
308     onRequest: onRequest
309 };
310 });

```

Create and Use Secret Keys

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample creates a simple Suitelet that requests user credentials, creates a secret key, and encodes a sample string.

Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see the help topic [SuiteScript 2.x Global Objects](#).

Note: The default maximum length for a secret key field is 32 characters. If needed, use the `Field.maxLength` property to change this value.

```

1  /**
2   * @ApiVersion 2.x
3   * @NScriptType Suitelet
4   */
5  define(['N/ui/serverWidget', 'N/runtime', 'N/crypto', 'N/encode'], (serverWidget, runtime, crypto, encode) => {
6      function onRequest(option) {
7          if (option.request.method === 'GET') {
8              let form = serverWidget.createForm({
9                  title: 'My Credential Form'
10             });
11             let skField = form.addSecretKeyField({
12                 id: 'mycredential',
13                 label: 'Credential',
14                 restrictToScriptIds: [runtime.getCurrentScript().id],
15                 restrictToCurrentUser: false
16             });
17             skField.maxLength = 200;
18
19             form.addSubmitButton();
20
21             option.response.writePage(form);
22         } else {
23             let form = serverWidget.createForm({
24                 title: 'My Credential Form'
25             });
26
27             const inputString = "YWJjZGVmZwo=";
28             let myGuid = option.request.parameters.mycredential;
29
30             // Create the key
31             let sKey = crypto.createSecretKey({
32                 guid: myGuid,
33                 encoding: encode.Encoding.UTF_8
34             });

```

```

35
36     try {
37         let hmacSha512 = crypto.createHmac({
38             algorithm: 'SHA512',
39             key: sKey
40         });
41         hmacSha512.update({
42             input: inputString,
43             inputEncoding: encode.Encoding.BASE_64
44         });
45         let digestSha512 = hmacSha512.digest({
46             outputEncoding: encode.Encoding.HEX
47         });
48     } catch (e) {
49         log.error({
50             title: 'Failed to hash input',
51             details: e
52         });
53     }
54
55     form.addField({
56         id: 'result',
57         label: 'Your digested hash value',
58         type: 'textarea'
59     }).defaultValue = digestSha512;
60
61     option.response.writePage(form);
62 }
63
64 return {
65     onRequest: onRequest
66 };
67 });

```

Create a Form with a Credential Field

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to use a Suitelet to create a form with a credential field. For more information about credential fields, see the help topic [Form.addField\(options\)](#).

Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see the help topic [SuiteScript 2.x Global Objects](#).

Note: The default maximum length for a credential field is 32 characters. If needed, use the `Field.maxLength` property to change this value.

The values for `restrictToDomains`, `restrictToScriptIds`, and `baseUrl` in this sample are placeholders. You must replace them with valid values from your NetSuite account.

Important: This sample uses SuiteScript 2.1. For more information, see [SuiteScript 2.1](#).

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType Suitelet
4   */
5
6  // This script creates a form with a credential field.
7  define(['N/ui/serverWidget', 'N/https', 'N/url'], (serverWidget, https, url) => {
8      function onRequest(context) {
9          if (context.request.method === 'GET') {
10             const form = serverWidget.createForm({
11                 title: 'Password Form'
12             });

```

```

13
14     const credField = form.addCredentialField({
15         id: 'password',
16         label: 'Password',
17         restrictToDomains: ['<accountID>.app.netsuite.com'],
18         restrictToCurrentUser: false,
19         restrictToScriptIds: 'customscript_my_script'
20     });
21
22     credField.maxLength = 32;
23
24     form.addSubmitButton();
25
26     context.response.writePage({
27         pageObject: form
28     });
29 }
30 else {
31     // Request to an existing Suitelet with credentials
32     let passwordGuid = context.request.parameters.password;
33
34     // Replace SCRIPTID and DEPLOYMENTID with the internal ID of the suitelet script and deployment in your account
35     let baseUrl = url.resolveScript({
36         scriptId: SCRIPTID,
37         deploymentId: DEPLOYMENTID,
38         returnExternalURL: true
39     });
40
41     let authUrl = baseUrl + '&pwd={' + passwordGuid + '}';
42
43     let secureStringUrl = https.createSecureString({
44         input: authUrl
45     });
46
47     let headers = ({
48         'pwd': passwordGuid
49     });
50
51     let response = https.post({
52         credentials: [passwordGuid],
53         url: secureStringUrl,
54         body: {authorization: ' ' + passwordGuid + ', data: anything can be here'},
55         headers: headers
56     });
57 }
58 }
59 return {
60     onRequest: onRequest
61 };
62 });

```

Add a Secret Key Field to a Form

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to add a secret key field.

i Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see the help topic [SuiteScript 2.x Global Objects](#).

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Suitelet
4   */
5   define(['N/ui/serverWidget', 'N/file', 'N/keyControl', 'N/runtime'], function(serverWidget, file, keyControl, runtime) {
6       function onRequest(context) {
7           var request = context.request;

```

```

8      var response = context.response;
9
10     if (request.method === 'GET') {
11         var form = serverWidget.createForm({
12             title: 'Enter Password'
13         });
14
15         var credField = form.addSecretKeyField({
16             id: 'custfield_password',
17             label: 'Password',
18             restrictToScriptIds: [runtime.getCurrentScript().id],
19             restrictToCurrentUser: true //Depends on use case
20         });
21         credField.maxLength = 64;
22
23         form.addSubmitButton();
24         response.writePage(form);
25     } else {
26         // Read the request parameter matching the field ID we specified in the form
27         var passwordToken = request.parameters.custfield_password;
28
29         var pem = file.load({
30             id: 422
31         });
32
33         var key = keyControl.createKey();
34         key.file = pem;
35         key.name = 'Test';
36         key.password = passwordToken;
37         key.save();
38     }
39 }
40 return {
41     onRequest: onRequest
42 };
43 });

```

Add a Field that Displays Running Total to a Sublist

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to add a field to the sublist that calculates and displays a running total for it.

i Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see the help topic [SuiteScript 2.x Global Objects](#).

```

1  /**
2   * @ApiVersion 2.x
3   * @NScriptType Suitelet
4   */
5  define(['N/ui/serverWidget', 'N/record'], function(serverWidget, record){
6      return {
7          onRequest: function(params) {
8              var form = serverWidget.createForm({
9                  title: 'Simple Form'
10             });
11             var sublistObj2 = form.addSublist({
12                 id: 'mylist',
13                 type: serverWidget.SublistType.INLINEEDITOR,
14                 label: 'List'
15             });
16             sublistObj2.addField({
17                 id: 'description',
18                 type: serverWidget.FieldType.TEXT,
19                 label: 'Description'

```

```

20     });
21     sublistObj2.addField({
22         id: 'amount',
23         type: serverWidget.FieldType.CURRENCY,
24         label: 'Amount'
25     });
26     sublistObj2.updateTotallingFieldId({
27         id: 'amount'
28     });
29     sublistObj2.setSublistValue({
30         id: 'description',
31         line: 0,
32         value: 'foo'
33     });
34     sublistObj2.setSublistValue({
35         id: 'amount',
36         line: 0,
37         value: '10'
38     });
39     sublistObj2.setSublistValue({
40         id: 'description',
41         line: 1,
42         value: 'bar'
43     });
44     sublistObj2.setSublistValue({
45         id: 'amount',
46         line: 1,
47         value: '15'
48     });
49     form.addSublist({
50         id: 'dummy',
51         type: serverWidget.SublistType.STATICLIST,
52         label: 'Dummy'
53     });
54     params.response.writePage(form);
55 }
56 };
57 });

```

Parse Strings and Display Result

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample parses a string (formatted according to the user's preferences) to a raw Date object, and then parses it back to the formatted string. This sample uses [format.parse\(options\)](#) and [format.format\(options\)](#).

This sample assumes the Date Format set in the preferences is MM/DD/YYYY. You may need to change the value for the date used in this script to match the preferences set in your account.

i Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Suitelet
4   */
5  define(['N/ui/serverWidget', 'N/format'], function(serverWidget, format) {
6      function parseAndFormatDateString() {
7          // Assuming Date format is MM/DD/YYYY
8          var initialFormattedDateString = "07/28/2015";
9          var parsedDateStringAsRawDateObject = format.parse({
10              value: initialFormattedDateString,
11              type: format.Type.DATE
12          });
13          var formattedDateString = format.format({

```

```

14         value: parsedDateStringAsRawDateObject,
15         type: format.Type.DATE
16     });
17     return [parsedDateStringAsRawDateObject, formattedDateString];
18 }
19 function onRequest(context) {
20     var data = parseAndFormatDateString();
21
22     var form = serverWidget.createForm({
23         title: "Date"
24     });
25
26     var fldDate = form.addField({
27         type: serverWidget.FieldType.DATE,
28         id: "date",
29         label: "Date"
30     });
31     fldDate.defaultValue = data[0];
32
33     var fldString = form.addField({
34         type: serverWidget.FieldType.TEXT,
35         id: "dateastext",
36         label: "Date as text"
37     });
38     fldString.defaultValue = data[1];
39
40     context.response.writePage(form);
41 }
42 return {
43     onRequest: onRequest
44 };
45 });

```

Custom Lists

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

- [Create a Custom List](#)
- [Add a Dynamic Link to a List in a Suitelet](#)

Create a Custom List

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

 **Note:** This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see the help topic [SuiteScript Debugger](#).

The following code creates a Suitelet that generates a custom list page. For steps to create this script, see [Steps for Creating a Custom List Page with SuiteScript 2.x](#).

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Suitelet
4   */
5  define(['N/ui/serverWidget'], function(serverWidget) {
6      function onRequest(context){
7          if(context.request.method === 'GET'){
8              // Section One - List - See 'Steps for Creating a Custom List Page', Step Five
9              var list = serverWidget.createList({
10                 title: 'Purchase History'
11             });
12
13             list.style = serverWidget.ListStyle.REPORT;

```



```

14
15     list.addButton({
16         id: 'buttonid',
17         label: 'Test',
18         functionName: '' // the function called when the button is pressed
19     });
20
21     // Section Two - Columns - See 'Steps for Creating a Custom List Page,' Step Seven
22     var datecol = list.addColumn({
23         id: 'column1',
24         type: serverWidget.FieldType.DATE,
25         label: 'Date',
26         align: serverWidget.LayoutJustification.RIGHT
27     });
28
29     list.addColumn({
30         id: 'column2',
31         type: serverWidget.FieldType.TEXT,
32         label: 'Product',
33         align: serverWidget.LayoutJustification.RIGHT
34     });
35
36     list.addColumn({
37         id: 'column3',
38         type: serverWidget.FieldType.INTEGER,
39         label: 'Quantity',
40         align: serverWidget.LayoutJustification.RIGHT
41     });
42
43     list.addColumn({
44         id: 'column4',
45         type: serverWidget.FieldType.CURRENCY,
46         label: 'Unit Cost',
47         align: serverWidget.LayoutJustification.RIGHT
48     });
49
50     list.addRows({
51         rows: [{column1: '05/30/2018', column2: 'Widget', column3: '4', column4: '4.50'},
52               {column1: '05/30/2018', column2: 'Sprocket', column3: '6', column4: '11.50'},
53               {column1: '05/30/2018', column2: 'Gizmo', column3: '9', column4: '1.25'}]
54     });
55     context.response.writePage(list);
56 }
57
58
59 return {
60     onRequest: onRequest
61 }
62 });

```

Add a Dynamic Link to a List in a Suitelet

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

i Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see the help topic [SuiteScript Debugger](#).

The code sample shown on this page adds a dynamic link, in this case, an entity record Dashboard, to a custom list column in a Suitelet.

i Note: For steps to create a custom list, see the help topic [SuiteScript Debugger](#)

We added the link using a View Dashboard icon. This icon shows when you hover over a record in any entity list (for example, Customers, Prospects, Leads, which you can view in List > Relationship). When you click the icon, it will direct you to a Dashboard page for that particular record.

The following steps show how to add the View Dashboard icon which will link an entity record with the corresponding Dashboard to a list in a Suitelet:

Add a View Dashboard Icon

1. Create a Transaction Body Field:
 - a. Go to Customization > Lists, Records, & Fields > Transaction Body Fields > New.
 - b. In **Basic Information** fill in the following fields:
 - In **Label**, enter: Dashboard.
 - In **Image ID**, enter: _dashboard_image.
 - In **Type**, select: Free-Form Text.
 - in **Store Value**: don't check the box.
 - c. In **Validation & Defaulting** subtab fill in the following fields:
 - In **Default Value**, enter (including the single quotation marks): '\'.
 - In **Formula**: check the box.
 - d. Click **Save**.
2. Deploy Suitelet script:
 - a. Create a JavaScript file named "SuiteletDashboard.js" with the following code:

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Suitelet
4   * @NModuleScope SameAccount
5   */
6  define(["N/ui/serverWidget", "N/search"],
7
8  function(serverWidget, search) {
9      function demoList(context) {
10         var list = serverWidget.createList({
11             title: 'Simple List'
12         });
13         list.style = context.request.parameters.style;
14         var column = list.addColumn({
15             id: 'custbody_dashboard_image',
16             label: 'Dashboard',
17             type: serverWidget.FieldType.TEXT,
18             align: serverWidget.LayoutJustification.LEFT
19         });
20         column.setURL({
21             url: "https://system.netsuite.com/app/center/card.nl?sc=-69" //URL of a Dashboard
22         });
23         column.addParamToURL({
24             param: 'entityid',
25             value: 'entity',
26             dynamic: true // the dynamic parameter that allows to link the URL of a Dashboard with the corresponding entity
27         });
28         list.addColumn({
29             id: 'trandate',
30             label: 'date',
31             type: serverWidget.FieldType.DATE,
32             align: serverWidget.LayoutJustification.LEFT
33         });
34         list.addColumn({
35             id: 'name_display',
36             label: 'Customer',
37             type: serverWidget.FieldType.TEXT,
38             align: serverWidget.LayoutJustification.LEFT
39         });
40         list.addColumn({

```

```

41         id: 'salesrep_display',
42         label: 'Sales Rep',
43         type: serverWidget.FieldType.TEXT,
44         align: serverWidget.LayoutJustification.LEFT
45     });
46     list.addColumn({
47         id: 'amount',
48         label: 'Amount',
49         type: serverWidget.FieldType.CURRENCY,
50         align: serverWidget.LayoutJustification.RIGHT
51     });
52
53     var searchEstimate = search.create({
54         type: search.Type.ESTIMATE,
55         filters: [{
56             name: 'mainline',
57             operator: 'is',
58             values: ['T']
59         }],
60         columns: [{
61             name: 'trandate'
62         }, {
63             name: 'entity'
64         }, {
65             name: 'name'
66         }, {
67             name: 'salesrep'
68         }, {
69             name: 'amount'
70         }, {
71             name: 'custbody_dashboard_image' // Dashboard link
72         }],
73     });
74     var searchEstimateResults = searchEstimate.run();
75     var results = searchEstimateResults.getRange({
76         start: 0,
77         end: 1000
78     });
79     list.addRows({
80         rows: results
81     });
82
83     context.response.writePage({
84         pageObject: list
85     });
86 }
87
88 return {
89     onRequest: demoList
90 };
91 });

```

- b. Go to Customization > Scripting > Scripts > New.
- c. Click **+**.
- d. Upload JavaScript file from Step 1.
- e. Click the **Create Script Record** button.
- f. In **Basic Information**, in the **Name** field, enter: View Dashboard Suitelet.
- g. On **Deployments** subtab, enter:
 - **Title:** View Dashboard Suitelet
 - **Status:** Released
- h. Click **Save**.

Troubleshooting

The following table shows errors that you might get while adding a creating a Transaction Body Field:

Error message	Solution
Your formula has an error in it. It could resolve to the wrong datatype, use an unknown function, or have a syntax error. Please go back, correct the formula, and re-submit.	Include single quotation marks when entering the value in the Default Value field (step 1c): <code>''</code>

Custom Assistants

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

■ [Sample Custom Assistant Script](#)

Sample Custom Assistant Script

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

For more information about custom assistant scripts, see [Creating Custom Assistants](#). Note that assistants cannot be used on externally available Suitelets.

The following screenshots show the output of the sample custom assistant script in this help topic.

The screenshot shows a web browser window with the URL `1013519.app.netsuite.com/app/site/hosting/scriptlet.nl?script=1743&deploy=1`. The page title is "New Supervisor". On the left, a "STEPS" sidebar shows "1 Select new supervisor" as the current step and "2 Review and Submit" as the next step. The main content area is titled "Select new supervisor" and contains two input fields: "NAME" and "EMPLOYEE". Each field has a dropdown arrow and a plus icon. At the bottom right, there are three buttons: "Cancel", "< Back", and "Next >". The "Next >" button is highlighted with a red rectangular box.

New Supervisor

STEPS

✓ 1 Select new supervisor

2 Review and Submit

Review and Submit
NAME

EMPLOYEE

Cancel < Back **Finish**

Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see the help topic [SuiteScript 2.x Global Objects](#).

The following sample creates an assistant with two steps. In the first one, you can select a new supervisor for the employee selected. The second step displays the selection and submits it.

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Suitelet
4   */
5  define(['N/ui/serverWidget'], function(serverWidget) {
6      return {
7          onRequest: function(context) {
8              var assistant = serverWidget.createAssistant({
9                  title: 'New Supervisor',
10                 hideNavBar: true
11             });
12             var assignment = assistant.addStep({
13                 id: 'assignment',
14                 label: 'Select New Supervisor'
15             });
16             var review = assistant.addStep({
17                 id: 'review',
18                 label: 'Review and Submit'
19             });
20
21             var writeAssignment = function() {
22                 assistant.addField({
23                     id: 'newsupervisor',
24                     type: serverWidget.FieldType.SELECT,
25                     label: 'Name',
26                     source: 'employee'
27                 });
28                 assistant.addField({
29                     id: 'assignedemployee',
30                     type: serverWidget.FieldType.SELECT,
31                     label: 'Employee',
32                     source: 'employee'
33                 });
34             };
35
36             var writeReview = function() {
37                 var supervisor = assistant.addField({
38                     id: 'newsupervisor',
39                     type: serverWidget.FieldType.TEXT,
40                     label: 'Name'
41                 });
42                 supervisor.defaultValue = context.request.parameters.inpt_newsupervisor;

```

```

43     supervisor.updateDisplayType({
44         displayType: serverWidget.FieldDisplayType.INLINE
45     });
46
47     var employee = assistant.addField({
48         id: 'assignedemployee',
49         type: serverWidget.FieldType.TEXT,
50         label: 'Employee'});
51     employee.defaultValue = context.request.parameters.inpt_assignedemployee;
52     employee.updateDisplayType({
53         displayType: serverWidget.FieldDisplayType.INLINE
54     });
55 }
56
57 var writeResult = function() {
58     var supervisor = context.request.parameters.newsupervisor;
59     var employee = context.request.parameters.assignedemployee;
60     context.response.write('Supervisor: ' + supervisor + '\nEmployee: ' + employee);
61 }
62
63 var writeCancel = function() {
64     context.response.write('Assistant was cancelled');
65 }
66
67 if (context.request.method === 'GET') //GET method means starting the assistant
68 {
69     writeAssignment();
70     assistant.currentStep = assignment;
71     context.response.writePage(assistant)
72 } else //POST method - process step of the assistant
73 {
74     if (context.request.parameters.next === 'Finish') //Finish was clicked
75         writeResult();
76     else if (context.request.parameters.cancel) //Cancel was clicked
77         writeCancel();
78     else if (assistant.currentStep.stepNumber === 1) { //transition from step 1 to step 2
79         writeReview();
80         assistant.currentStep = assistant.getNextStep();
81         context.response.writePage(assistant);
82     } else { //transition from step 2 back to step 1
83         writeAssignment();
84         assistant.currentStep = assistant.getNextStep();
85         context.response.writePage(assistant);
86     }
87 }
88 }
89 };
90 });

```

Suitelets with Embedded HTML

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

See [Embedding HTML from a Linked HTML Page in a Suitelet](#) for a complete example.

Backend Suitelets

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

- [Find Plugin Implementations](#)
- [Load an XML File and Obtain Child Element Values](#)
- [Parse an XML File and Log Element Values](#)
- [Redirect a New Sales Order](#)
- [Render a PDF](#)

- [Render Search Results into a PDF File](#)
- [Retrieve the Name of the City Based on a ZIP Code](#)
- [Return a File with Alternative Character Encoding](#)
- [Write User and Session Information to the Response](#)

Find Plugin Implementations

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows an implementation of a custom plug-in interface. To test this sample, you need a custom plug-in type with a script ID of `customscript_magic_plugin` and an interface with a single method, `int doTheMagic(int, int)`.

i Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see the help topic [SuiteScript 2.x Global Objects](#).

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType pluginTypeimpl
4   */
5  define(function() {
6      return {
7          doTheMagic: function(operand1, operand2) {
8              return operand1 + operand2;
9          }
10     };
11 });

```

The following Suitelet iterates through all implementations of the custom plug-in type `customscript_magic_plugin`. For the plug-in to be recognized, the Suitelet script record must specify the plug-in type on the Custom Plug-in Types subtab.

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Suitelet
4   */
5  define(['/N/plugin'], function(plugin) {
6      function onRequest(context) {
7          var impls = plugin.findImplementations({
8              type: 'customscript_magic_plugin'
9          });
10
11          for (var i = 0; i < impls.length; i++) {
12              var pl = plugin.loadImplementation({
13                  type: 'customscript_magic_plugin',
14                  implementation: impls[i]
15              });
16              log.debug('impl ' + impls[i] + ' result = ' + pl.doTheMagic(10, 20));
17          }
18
19          var pl = plugin.loadImplementation({
20              type: 'customscript_magic_plugin'
21          });
22          log.debug('default impl result = ' + pl.doTheMagic(10, 20));
23      }
24
25      return {
26          onRequest: onRequest
27      };
28 });

```

Load an XML File and Obtain Child Element Values

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample loads the BookSample.xml file from the File Cabinet, iterates through the individual book nodes, and accesses the child node values.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**
2   * @ApiVersion 2.x
3   * @NScriptType Suitelet
4   */
5   require(['N/xml', 'N/file'], function(xml, file) {
6       return {
7           onRequest: function(options) {
8               var sentence = '';
9               var xmlFileContent = file.load('SuiteScripts/BookSample.xml').getContents();
10              var xmlDocument = xml.Parser.fromString({
11                  text: xmlFileContent
12              });
13              var bookNode = xml.XPath.select({
14                  node: xmlDocument,
15                  xpath: '//b:book'
16              });
17
18              for (var i = 0; i < bookNode.length; i++) {
19                  var title = bookNode[i].firstChild.nextSibling.textContent;
20                  var author = bookNode[i].getElementsByTagName({
21                      tagName: 'b:author'
22                  })[0].textContent;
23                  sentence += 'Author: ' + author + ' wrote ' + title + '.\n';
24              }
25
26              options.response.write(sentence);
27          }
28      };
29  });

```

This script produces the following output when used with the BookSample.xml file:

```

1  Author: Giada De Laurentiis wrote Everyday Italian.
2  Author: J K. Rowling wrote Harry Potter.
3  Author: James McGovern wrote XQuery Kick Start.
4  Author: Erik T. Ray wrote Learning XML.

```

Parse an XML File and Log Element Values

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample parses the XML string stored in the `xmlString` variable. The sample selects all config elements in the `xmlDocument` node, loops through them, and logs their contents.

Note: This sample script uses the `require` function so that you can copy it into the SuiteScript Debugger and test it. You must use the `define` function in an entry point script (the script you attach to a script record and deploy). For more information, see [SuiteScript 2.x Script Basics](#) and [SuiteScript 2.x Script Types](#).

```

1  /**

```



```

2  /**
3   * @ApiVersion 2.x
4   * @NScriptType Suitelet
5   */
6  require(['N/xml'], function(xml) {
7      return {
8          onRequest: function(options) {
9              var xmlString = '<?xml version="1.0" encoding="UTF-8"?><config date="1465467658668" transient="false">Some content</config>';
10
11              var xmlDocument = xml.Parser.fromString({
12                  text: xmlString
13              });
14
15              var bookNode = xml.XPath.select({
16                  node: xmlDocument,
17                  xpath: '//config'
18              });
19
20              for (var i = 0; i < bookNode.length; i++) {
21                  log.debug('Config content', bookNode[i].textContent);
22              }
23          }
24      };
25  });

```

Redirect a New Sales Order

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to use a Suitelet to redirect to a new sales order record and set the entity field (which represents the customer).

i Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see the help topic [SuiteScript 2.x Global Objects](#).

! Important: The value used in this sample for the entity field is a placeholder. Before using this sample, replace the entity field value with a valid value from your NetSuite account. If you run a script with an invalid value, an error may occur.

```

1  /**
2   * @ApiVersion 2.1
3   * @NScriptType Suitelet
4   */
5
6  // This script redirects a new sales order record and sets the entity.
7  define(['N/record', 'N/http'], (record, http)=> {
8      function onRequest(context) {
9          context.response.sendRedirect({
10              type: http.RedirectType.RECORD,
11              identifier: record.Type.SALES_ORDER,
12              parameters: ({
13                  entity: 6
14              })
15          });
16      }
17      return {
18          onRequest: onRequest
19      };
20  });

```

Render a PDF

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following Suitelet generates and renders a PDF directly to the response.

The following sample shows how to render a PDF.

Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see the help topic [SuiteScript 2.x Global Objects](#).

```
1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Suitelet
4   */
5   define(['/N/xml'], function(xml) {
6       return {
7           onRequest: function(context) {
8               var xml = "<?xml version='1.0' encoding='UTF-8'>\n" +
9                   "<!DOCTYPE pdf PUBLIC \"-//big.faceless.org//report\" \"report-1.1.dtd\">\n" +
10                   "<pdf lang=\"ru-RU\" xml:lang=\"ru-RU\">\n" +
11                   "<head>\n" +
12                   "<link name=\"russianfont\" type=\"font\" subtype=\"opentype\" \" + \"src=\"NetSuiteFonts/verdana.ttf\" \" + \"src-
13                   bold=\"NetSuiteFonts/verdanab.ttf\" \" + \"src-italic=\"NetSuiteFonts/verdanai.ttf\" \" + \"src-bolditalic=\"NetSuiteFonts/verdan
14                   abi.ttf\" \" + \"bytes=\"2\"/>\n" +
15                   "</head>\n" +
16                   "<body font-family=\"russianfont\" font-size=\"18\">\nРусский текст</body>\n" +
17                   "</pdf>";
18               context.response.renderPdf(xml);
19           }
20       }
21   });
```

Render Search Results into a PDF File

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to render search results into a PDF.

Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see the help topic [SuiteScript 2.x Global Objects](#).

```
1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Suitelet
4   */
5   // This sample shows how to render search results into a PDF file.
6   define(['/N/render', '/N/search'], function(render, search) {
7       function onRequest(options) {
8           var request = options.request;
9           var response = options.response;
10
11           var xmlStr = '<?xml version="1.0" encoding="UTF-8">\n' +
12               '<!DOCTYPE pdf PUBLIC "-//big.faceless.org//report" "report-1.1.dtd">\n' +
13               '<pdf lang="ru-RU" xml:lang="ru-RU">\n' + "<head>\n" +
14               '<link name="russianfont" type="font" subtype="opentype" ' + 'src="NetSuiteFonts/verdana.ttf" ' + "src-
15               bold="NetSuiteFonts/verdanab.ttf" ' + 'src-italic="NetSuiteFonts/verdanai.ttf" ' + "src-bolditalic="NetSuiteFonts/verdan
16               abi.ttf" ' + 'bytes="2"/>\n' + "</head>\n" +
17               '<body font-family="russianfont" font-size="18">\n????? ????</body>\n' + "</pdf>";
```

```

16
17     var rs = search.create({
18         type: search.Type.TRANSACTION,
19         columns: ['trandate', 'amount', 'entity'],
20         filters: []
21     }).run();
22
23     var results = rs.getRange(0, 1000);
24     var renderer = render.create();
25     renderer.templateContent = xmlStr;
26     renderer.addSearchResults({
27         templateName: 'results',
28         searchResult: results
29     });
30
31     var newfile = renderer.renderAsPdf();
32     response.writeFile(newfile, false);
33 }
34
35 return {
36     onRequest: onRequest
37 };
38 });

```

Retrieve the Name of the City Based on a ZIP Code

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to use a Suitelet and a custom module to retrieve the name of a city based on a ZIP code. To speed up processing, the Suitelet uses a cache.

In this sample, the ZIP code is the key used to retrieve city names from the cache. A loader function is called if the city corresponding to the provided ZIP code (key) is not in the cache. This loader function is a custom module that loads a CSV file and uses it to find the requested value. This function is called `zipCodeDatabaseLoader` (included in the second script sample).

i Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see the help topic [SuiteScript 2.x Global Objects](#).

i Note: This sample depends on a CSV file that must exist before the script is run. The sample CSV file is available [here](#).

! Important: This sample uses SuiteScript 2.1. For more information, see [SuiteScript 2.1](#).

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType Suitelet
4   */
5
6  // This script retrieves the name of a city based on a ZIP code from a cache.
7  define(['/N/cache', '/SuiteScripts/zipToCityIndexCacheLoader'], function(cache, lib) {
8      const ZIP_CODES_CACHE_NAME = 'ZIP_CODES_CACHE';
9      const ZIP_TO_CITY_IDX_JSON = 'ZIP_TO_CITY_IDX_JSON';
10
11     function getZipCodeToCityLookupObj() {
12         const zipCache = cache.getCache({
13             name: ZIP_CODES_CACHE_NAME
14         });
15         const zipCacheJson = zipCache.get({
16             key: ZIP_TO_CITY_IDX_JSON,
17             loader: lib.zipCodeDatabaseLoader
18         });

```

```

19     return JSON.parse(zipCacheJson);
20 }
21
22 function findCityByZipCode(options) {
23     return getZipCodeToCityLookupObj()[String(options.zip)];
24 }
25
26 function onRequest(context) {
27     const start = new Date();
28     if (context.request.parameters.purgeZipCache === 'true') {
29         const zipCache = cache.getCache({
30             name: ZIP_CODES_CACHE_NAME
31         });
32         zipCache.remove({
33             key: ZIP_TO_CITY_IDX_JSON
34         });
35     }
36     const cityName = findCityByZipCode({
37         zip: context.request.parameters.zipcode
38     });
39
40     context.response.writeLine(cityName || 'Unknown :(');
41
42     if (context.request.parameters.auditPerf === 'true') {
43         context.response.writeLine('Time Elapsed: ' + (new Date().getTime() - start.getTime()) + ' ms');
44     }
45 }
46
47 return {
48     onRequest: onRequest
49 };

```

The following custom module provides the loader function used in the preceding Suitelet script sample. The loader function uses a CSV file to retrieve a value that was missing from a cache. This custom module does not need to include logic for placing the retrieved value into the cache. Whenever a value is returned through the options.loader parameter of the [Cache.get\(options\)](#) method, the value is automatically placed into the cache. This allows the loader function to serve as the sole method of populating a cache with values.

```

1  /**
2   * zipToCityIndexCacheLoader.js
3   * @NApiVersion 2.1
4   * @NModuleScope Public
5   */
6
7  //This custom module is a loader function that uses a CSV file to retrieve a value that was missing from a cache.
8  define(['N/file', 'N/cache'], function(file, cache) {
9      const ZIP_CODES_CSV_PATH = '/SuiteScripts/Resources/free-zipcode-CA-database-primary.csv';
10
11      function trimOuterQuotes(str) {
12          return (str || '').replace(/^"+/, '').replace(/"+$/, '');
13      }
14
15      function zipCodeDatabaseLoader(context) {
16          log.audit({
17              title: 'Loading Zip Codes',
18              details: 'Loading Zip Codes for ZIP_CODES_CACHE'
19          });
20          const zipCodesCsvText = file.load({
21              id: ZIP_CODES_CSV_PATH
22          }).getContents();
23          const zipToCityIndex = {};
24          const csvLines = zipCodesCsvText.split('\n');
25          util.each(csvLines.slice(1), function(el) {
26              var cells = el.split(',');
27              var key = trimOuterQuotes(cells[0]);
28              var value = trimOuterQuotes(cells[2]);
29              if (parseInt(key, 10))
30                  zipToCityIndex[String(key)] = value;
31          });
32          return zipToCityIndex;

```

```

33     }
34
35     return {
36         zipCodeDatabaseLoader: zipCodeDatabaseLoader
37     }
38 });

```

Return a File with Alternative Character Encoding

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

By default, Suitelet textual responses are encoded using UTF-8 encoding.

In SuiteScript 1.0, you can construct a response with an alternative encoding by using the `nlobjResponse.setEncoding()` function. However, there is no direct equivalent for this function in SuiteScript 2.x.

To return a file with an alternative encoding in SuiteScript 2.x, you can use the following modules:

- **N/file Module** – Use `file.create(options)` to create a file with a character encoding other than UTF-8.
- **N/https Module** – Use `ServerResponse.setHeader(options)` to specify the same character encoding in the charset parameter of the Content-Type header. Then, call `ServerResponse.writeFile(options)` to return the file in the response. This method preserves the file's encoding if the charset value specified in the header matches the file's encoding.

The following script sample shows how to create a file with an encoding other than UTF-8, and return the file through a Suitelet response with the file's encoding preserved.

i Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see the help topic [SuiteScript 2.x Global Objects](#).

```

1  /**
2   * @NApiVersion 2.0
3   * @NScriptType Suitelet
4   */
5  define(['N/https', 'N/file'], function(https, file)
6  {
7      function onRequest(context)
8      {
9          var outputFile = file.create({
10             name: "virtual.txt",
11             contents: "This response is encoded in iso-8859-1 -- éáíóú",
12             fileType: file.Type.PLAINTEXT,
13             encoding: file.Encoding.ISO_8859_1
14         });
15         // Charset in Content-Type header must match the file's encoding.
16         context.response.setHeader("Content-Type", "text/plain;charset=iso-8859-1");
17         context.response.writeFile(outputFile, true);
18     }
19     return {
20         onRequest: onRequest
21     };
22 });

```

Write User and Session Information to the Response

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following sample shows how to use a Suitelet to write user and session information for the currently executing script to the response.

Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see the help topic [SuiteScript 2.x Global Objects](#).

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Suitelet
4   */
5
6  // This script writes user and session information for the currently executing script to the response.
7  define(['N/runtime'], function(runtime) {
8      function onRequest(context) {
9          var remainingUsage = runtime.getCurrentScript().getRemainingUsage();
10         var userRole = runtime.getCurrentUser().role;
11         var currentSession = runtime.getCurrentSession();
12
13         // Set the current sessions's scope
14         currentSession.set({
15             name: 'scope',
16             value: 'global'
17         });
18
19         var sessionScope = runtime.getCurrentSession().get({
20             name: 'scope'
21         });
22
23         log.debug('Remaining Usage:', remainingUsage);
24         log.debug('Role:', userRole);
25         log.debug('Session Scope:', sessionScope);
26
27         context.response.write('Executing under role: ' + userRole
28             + '. Session scope: ' + sessionScope + '.');
29     }
30     return {
31         onRequest: onRequest
32     };
33 });

```

SuiteScript 2.x User Event Script Type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

User event scripts run on the NetSuite server whenever you create, load, update, copy, delete, or submit a record. Most standard NetSuite records and custom record types support user event scripts, except for records like personal ID (such as Driver's Licenses and Passports), some revenue recognition records, and some timecard-related records. For details on which records are supported, see the help topic [SuiteScript Supported Records](#).

You can use user event scripts to do things like:

- Implement custom validation on records
- Enforce user-defined data integrity and business rules
- Perform user-defined permission checking and record restrictions
- Implement real-time data synchronization
- Define custom workflows (redirection and follow-up actions)
- Customize forms.

For additional information about SuiteScript 2.x User Event Scripts, see the following:

- [SuiteScript 2.x User Event Script Reference](#)
 - [How User Event Scripts are Executed](#)
 - [SuiteScript 2.x User Event Script Tutorial](#)
- [SuiteScript 2.x User Event Script Entry Points and API](#)
 - [afterSubmit\(context\)](#)
 - [beforeLoad\(context\)](#)
 - [beforeSubmit\(context\)](#)
 - [context.UserEventType](#)

You can use SuiteCloud Development Framework (SDF) to manage user event scripts in file-based customization projects. For more about SDF, see [. You can also use Copy to Account to copy a user event script to another account. Click Copy to Account at the top right of the script page. For details, see the help topic \[Copy to Account\]\(#\).](#)

Use SuiteScript Analysis to see when a script was installed and how it performed in the past. To learn more, see the help topic [Analyzing Scripts](#).

For tips, check out the [User Event Script Best Practices](#) in the [SuiteScript Developer Guide](#).

SuiteScript 2.x User Event Script Sample

Here's a sample user event script for environments without the Team Selling feature. For more samples, see the [SuiteScript 2.x Code Samples Catalog: User Event Script Samples](#).

When you deploy this script on a customer record, it creates a follow-up phone call record each time a new customer is added.



Important: Before running this script, make sure you replace the salesrep internal ID with one that matches your account—use an ID for an employee marked as a sales rep. You'll find this option on the Human Resources subtab of the employee record. If you don't update the ID, the script might not work. This script is made for environments where customer records have a salesrep field. If Team Selling is enabled, customer records usually won't include a salesrep field.

For help with SuiteScript 2.x, see [SuiteScript 2.x Hello World](#) and [SuiteScript 2.x Entry Point Script Creation and Deployment](#).

```

1  /**
2   *@NApiVersion 2.x
3   *@NScriptType UserEventScript
4   */
5  define(['N/record'], function(record) {
6      function beforeLoad(context) {
7          if (context.type !== context.UserEventType.CREATE)
8              return;
9          var customerRecord = context.newRecord;
10         customerRecord.setValue('phone', '555-555-5555');
11         if (!customerRecord.getValue('salesrep'))
12             customerRecord.setValue('salesrep', 46); // replace '46' with one specific to your account
13     }
14     function beforeSubmit(context) {
15         if (context.type !== context.UserEventType.CREATE)
16             return;
17         var customerRecord = context.newRecord;
18         customerRecord.setValue('comments', 'Please follow up with this customer!');
19
20         if (!customerRecord.getValue('category')) {
21             throw error.create({ // you can change the type of error that is thrown
22                 name: 'MISSING_CATEGORY',

```

```

23         message: 'Please enter a category.'
24     })
25 }
26 }
27 function afterSubmit(context) {
28     if (context.type !== context.UserEventType.CREATE)
29         return;
30     var customerRecord = context.newRecord();
31     if (customerRecord.getValue('salesrep')) {
32         var call = record.create({
33             type: record.Type.PHONE_CALL,
34             isDynamic: true
35         });
36         call.setValue('title', 'Make follow-up call to new customer');
37         call.setValue('assigned', customerRecord.getValue('salesrep'));
38         call.setValue('phone', customerRecord.getValue('phone'));
39         try {
40             var callId = call.save();
41             log.debug('Call record created successfully', 'Id: ' + callId);
42         } catch (e) {
43             log.error(e.name);
44         }
45     }
46 }
47 return {
48     beforeLoad: beforeLoad,
49     beforeSubmit: beforeSubmit,
50     afterSubmit: afterSubmit
51 };
52 });

```

SuiteScript 2.x User Event Script Reference

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

User event scripts run during beforeLoad, beforeSubmit, and afterSubmit operations. To learn how each one works, see [How User Event Scripts are Executed](#).

To learn how to write a basic user event script, check out the [SuiteScript 2.x User Event Script Tutorial](#). In it, you'll create a script that adds a new employee record, updates fields, and creates a task.

For tips, see the [User Event Script Best Practices](#) section in the [SuiteScript Developer Guide](#).

How User Event Scripts are Executed

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer



Important: User event scripts can't be triggered by other user event scripts or workflows with a **Context of User Event Script**—so you can't chain them. But you can run a user event script from scheduled scripts, portlet scripts, or Suitelets.

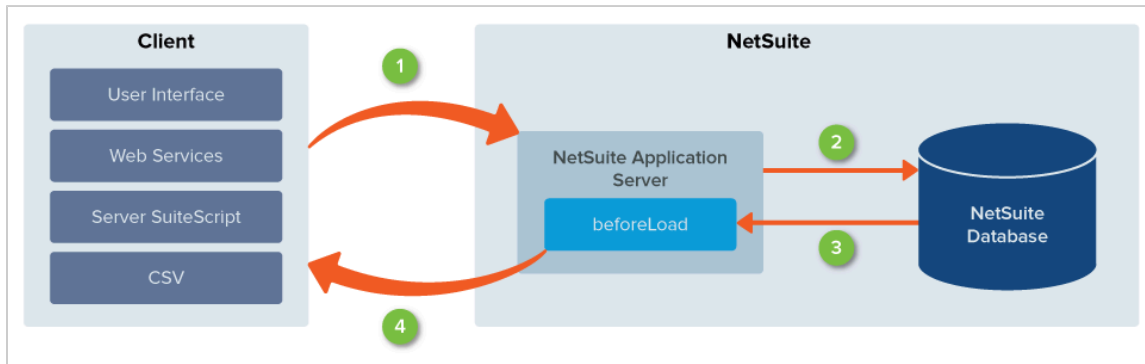
User event scripts run during beforeLoad, beforeSubmit, and afterSubmit events.



Tip: You can choose the order your user event scripts run on the Scripted Records page. See the help topic [The Scripted Records Page](#).

beforeLoad

The following diagram shows an overview of what occurs during a beforeLoad operation:

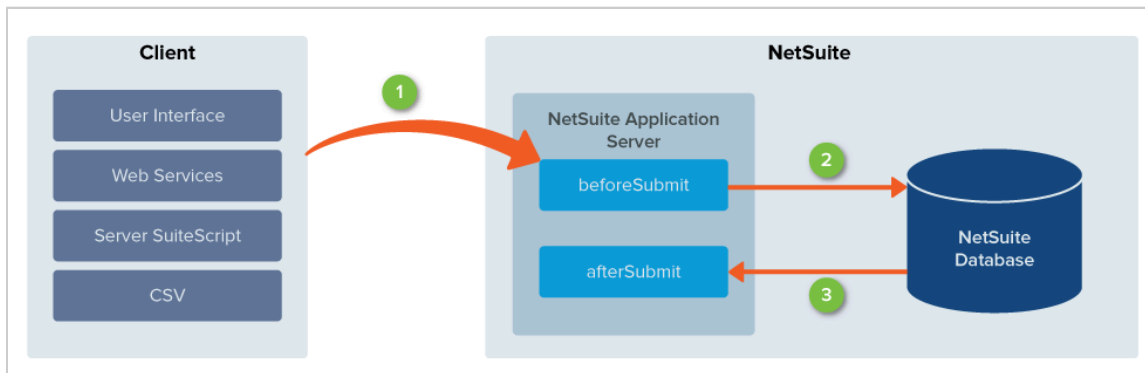


1. The client sends a request to read record data. This can come from the UI, SOAP and REST web services, CSV import, or server SuiteScript (not other user event scripts).
2. The application server checks permissions.
3. The database loads the info into the application server for processing. The `beforeLoad` operation happens here—before the data goes back to the client.
4. The client gets the validated and processed `beforeLoad` data.

Note: You can't source standard records during `beforeLoad`. Use the `pageInit` client script instead. See [pageInit\(scriptContext\)](#).

beforeSubmit and afterSubmit

The following diagram shows an overview of what occurs on `beforeSubmit` and `afterSubmit` operations:



1. The client sends a write request to the application server—this can be from the UI, SOAP/REST web services, server SuiteScript, CSV import, or XML. The server:
 - a. checks permissions
 - b. processes the data and runs validation checks during `beforeSubmit`

At this point, the data hasn't been saved to the database yet.

2. After validation, the data gets saved to the database.
3. When `afterSubmit` runs, it grabs the new data from the database and sends it to the application server for more processing. The `afterSubmit` actions on saved data can include:
 - a. sending email notifications about the new data
 - b. creating child records based on the new data
 - c. assigning tasks to employees using the saved data

Note: Asynchronous afterSubmit user events only work during webstore checkout.

SuiteScript 2.x User Event Script Tutorial

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

A user event script runs on the server and is triggered when you take action on a record. It can run logic in three situations:

- After you start to open the record, but before it loads.
- After you click Save to save the record.
- After the record's submitted to the database.

This tutorial walks you through how to set up a basic user event script. Here's what you'll do:

- [Sample Script Overview](#)
- Step One: [Check Your Prerequisites](#)
- Step Two: [Create the Script File](#)
- Step Three: [Review the Script \(Optional\)](#)
- Step Four: [Upload the Script File to NetSuite](#)
- Step Five: [Create a Script Record and Script Deployment Record](#)
- Step Six: [Test the Script](#)

Note: Before you start, check out [SuiteScript 2.x Script Basics](#) for an overview of key terms. For details about the structure of SuiteScript 2.x entry point scripts, see [SuiteScript 2.x Anatomy of a Script](#).

Sample Script Overview

The sample script is meant for the employee record. When you deploy it, here's what it does:

- When you start creating a new employee record, the script disables the Notes field before the page loads.
- After you fill out all the required info and click Save, the script adds a note saying no orientation date has been set yet.
- Once the record's submitted, the script creates a task to schedule the new employee's orientation and assigns it to their supervisor.

This script uses all three [SuiteScript 2.x User Event Script Type](#) entry points: [beforeLoad\(context\)](#), [beforeSubmit\(context\)](#), and [afterSubmit\(context\)](#).

Check Your Prerequisites

Before you start, make sure your system is set up correctly:

- [Enable the Feature](#)
- [Create an Employee Record](#)

Enable the Feature

You'll need to enable the Client and Server SuiteScript features in your account. For help, see the help topic [Enabling SuiteScript](#).

Create an Employee Record

You'll also need an employee record you can use as the supervisor for the test employee.

To see your current employee records, go to Lists > Employees. If you don't have one you can use, create a new employee. For help, see the help topic [Adding an Employee](#).

Create the Script File

Before you go any further, create the entry point script file. Just copy and paste the code below into your favorite text editor, then save it as createTask.js.

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType UserEventScript
4   */
5
6  // Load two standard modules.
7  define ( ['N/record', 'N/ui/serverWidget'] ,
8    // Add the callback function.
9    function(record, serverWidget) {
10      // In the beforeLoad function, disable the Notes field.
11      function myBeforeLoad (context) {
12        if (context.type !== context.UserEventType.CREATE)
13          return;
14        var form = context.form;
15        var notesField = form.getField({
16          id: 'comments'
17        });
18        notesField.updateDisplayType({
19          displayType: serverWidget.FieldDisplayType.DISABLED
20        });
21      }
22
23      // In the beforeSubmit function, add test to the Notes field.
24      function myBeforeSubmit(context) {
25        if (context.type !== context.UserEventType.CREATE)
26          return;
27        var newEmployeeRecord = context.newRecord;
28        newEmployeeRecord.setValue({
29          fieldId: 'comments',
30          value: 'Orientation date TBD.'
31        });
32      }
33
34      // In the afterSubmit function, begin creating a task record.
35      function myAfterSubmit(context) {
36        if (context.type !== context.UserEventType.CREATE)
37          return;
38        var newEmployeeRecord = context.newRecord;
39        var newEmployeeFirstName = newEmployeeRecord.getValue ({
40          fieldId: 'firstname'
41        });
42        var newEmployeeLastName = newEmployeeRecord.getValue ({
43          fieldId: 'lastname'
44        });
45        var newEmployeeSupervisor = newEmployeeRecord.getValue ({
46          fieldId: 'supervisor'
47        });
48        if (newEmployeeSupervisor) {
49          var newTask = record.create({
50            type: record.Type.TASK,
51            isDynamic: true
52          });
53          newTask.setValue({
54            fieldId: 'title',
55            value: 'Schedule orientation session for ' +
56              newEmployeeFirstName + ' ' + newEmployeeLastName
57          });

```

```
58         newTask.setValue({
59             fieldId: 'assigned',
60             value: newEmployeeSupervisor
61         });
62         try {
63             var newTaskId = newTask.save();
64             log.debug({
65                 title: 'Task record created successfully',
66                 details: 'New task record ID: ' + newTaskId
67             });
68         } catch (e) {
69             log.error({
70                 title: e.name,
71                 details: e.message
72             });
73         }
74     }
75 }
76
77 // Add the return statement that identifies the entry point functions.
78 return {
79     beforeLoad: myBeforeLoad,
80     beforeSubmit: myBeforeSubmit,
81     afterSubmit: myAfterSubmit
82 };
83 };
```

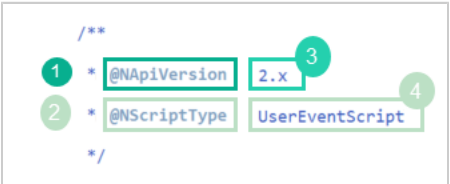
Review the Script (Optional)

If you want to see how the script is put together, check out the next sections.

- [JSDoc Tags](#)
- [beforeLoad Function](#)
- [beforeSubmit Function](#)
- [afterSubmit Function](#)
- [return Statement](#)

JSDoc Tags

The following image shows the JSDoc block for this script. To see what the numbered callouts mean, look at the table after the image.



Callout	Description
1 and 3	The @NApiVersion tag (set to 2.x) is required in all entry point scripts. Valid values are 2.0, 2.x, and 2.1
2 and 4	The @NScriptType tag (set to UserEventScript) shows what script type you're using. It's not case sensitive, but using Pascal case makes it easier to read.

beforeLoad Function

The next image shows the myBeforeLoad function. Because of how the script's [return Statement](#) is set up, this function runs when beforeLoad is called.

```

function myBeforeLoad ( 1context ) {
2
  if (context.type !== context.UserEventType.CREATE)
    return;
3
  var form = context.form;
  var notesField = form.getField({
    id: 'comments'
  });
  notesField.updateDisplayType({
    displayType: serverWidget.FieldDisplayType.DISABLED
  });
}

```

Callout	Description
1	The context object is available when beforeLoad runs. For a list of its properties, see beforeLoad(context) . This function uses two: type and form.
2	This statement makes the rest of the function run only if you're creating a new record. It checks the context object's type property to see what action you're taking. For possible actions, see context.UserEventType .
3	These statements disable the Notes field. They use the context object's form property to access the form, and the N/ui/serverWidget Module APIs to change field settings.

beforeSubmit Function

The next image shows the myBeforeSubmit function. Because of the [return Statement](#), this runs when beforeSubmit is called.

```

function myBeforeSubmit ( 1context ) {
2
  if (context.type !== context.UserEventType.CREATE)
    return;
3
  var newEmployeeRecord = context.newRecord;
  newEmployeeRecord.setValue({
    fieldId: 'comments',
    value: 'Orientation date TBD.'
  });
}

```

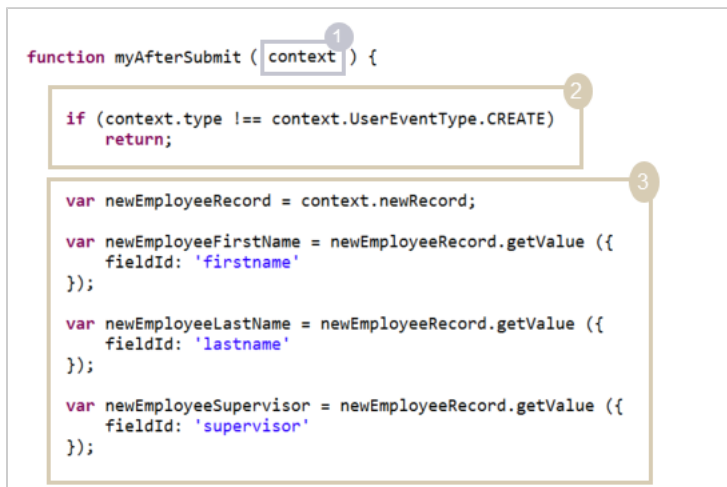
Callout	Description
1	The context object is available for beforeSubmit—see beforeSubmit(context) for its properties. This function uses type and newRecord.  Note: The context object's properties here are different from beforeLoad Function. The beforeLoad function has a form property; beforeSubmit doesn't, so you won't have access to the form in beforeSubmit.
2	This statement makes the rest of the function run only if you're creating a new record, using the context object's type property. See context.UserEventType for possible values.

Callout	Description
3	These statements add text to the Notes field by using the context object's newRecord property. The script uses Record.setValue(options) to add the text.

afterSubmit Function

The next image shows the myAfterSubmit function. Because of the [return Statement](#), this runs when afterSubmit is called.

The first part of the function gets data from the employee record:

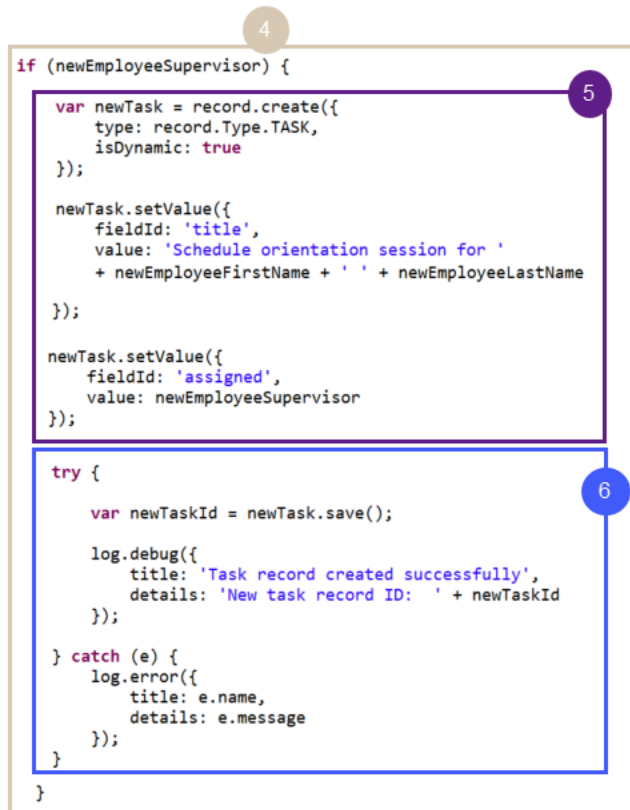


```
function myAfterSubmit ( context ) {
    if (context.type !== context.UserEventType.CREATE)
        return;

    var newEmployeeRecord = context.newRecord;
    var newEmployeeFirstName = newEmployeeRecord.getValue ({
        fieldId: 'firstname'
    });
    var newEmployeeLastName = newEmployeeRecord.getValue ({
        fieldId: 'lastname'
    });
    var newEmployeeSupervisor = newEmployeeRecord.getValue ({
        fieldId: 'supervisor'
    });
}
```

Callout	Description
1	The context object is available for afterSubmit—see afterSubmit(context) for a list of its properties. This function uses type and newRecord.
2	This statement makes the rest of the function run only if you're creating a new record, using the context object's type property. For actions, see context.UserEventType .
3	These statements use context.newRecord to access the submitted record and Record.getValue(options) to get the data.

The rest of the function creates the task record:



```

if (newEmployeeSupervisor) {
    var newTask = record.create({
        type: record.Type.TASK,
        isDynamic: true
    });

    newTask.setValue({
        fieldId: 'title',
        value: 'Schedule orientation session for '
            + newEmployeeFirstName + ' ' + newEmployeeLastName
    });

    newTask.setValue({
        fieldId: 'assigned',
        value: newEmployeeSupervisor
    });

    try {
        var newTaskId = newTask.save();

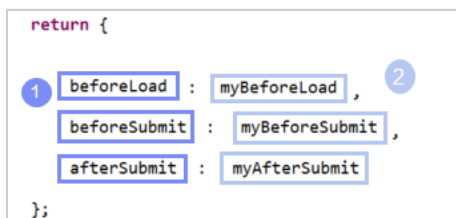
        log.debug({
            title: 'Task record created successfully',
            details: 'New task record ID: ' + newTaskId
        });
    } catch (e) {
        log.error({
            title: e.name,
            details: e.message
        });
    }
}

```

Callout	Description
4	There's a conditional statement—it checks if the Supervisor field has a value. If so, the logic inside the curly braces runs.
5	These statements use the N/record Module APIs to create the new task and set its values.
6	This code tries to save the new task record. If there are errors, they're logged in the script deployment record's execution log.

return Statement

The next image shows the callback function's return statement.



```

return {
    beforeLoad : myBeforeLoad,
    beforeSubmit : myBeforeSubmit,
    afterSubmit : myAfterSubmit
};

```

Callout	Description
1	Entry points. The callback function's return statement must use at least one entry point for the script type set by @NScriptType. See Callout 4 in JSDoc Tags .

Callout	Description
	This script uses all three user event script entry points.
2	Entry points functions. For each entry point, your script needs to define a function for it.

Upload the Script File to NetSuite

Once you've created your entry point script file, upload it to your NetSuite File Cabinet.

To upload the script file:

1. In the NetSuite UI, go to Documents > Files > SuiteScripts.
2. In the left pane, select the SuiteScripts folder and click **Add File**.
3. Follow the prompts to locate the createTask.js file (created in Step Two) in your local environment and upload it.

Even after you upload the file, you can edit it right from the File Cabinet if needed. For details, see the help topic [Editing Files in the File Cabinet](#).

Create a Script Record and Script Deployment Record

Before an entry point script can run in your account, you'll need to create a script record for the file and a script deployment record.

To create the script record and script deployment record:

1. Go to Customization > Scripting > Scripts > New.
2. In the **Script File** dropdown list, select createTask.js.
If you haven't uploaded the file yet, you can do it here—click the plus sign next to the dropdown. You might need to hover your cursor to the right to see the icon.
3. After you have selected the script file and it is displayed in the dropdown list, click the **Create Script Record** button.
If your file passes validation, the system adds a new script record and lists createTask.js on the **Scripts** subtab.
4. Fill out the required body fields as follows:
 - In the **Name** field, enter **Create Task Record User Event**.
 - In the **ID** field, enter **_ues_create_task_record**.
5. Click the **Deployments** subtab.
6. Add a line to the sublist, as follows:
 - Set the **Applies to** dropdown list to **Employee**.
 - In the **ID** field, enter **_ues_create_task_record**.

Leave the other fields as they are. The **Status** field is set to **Testing** by default, so the script won't run for other users. If you want everyone to use it later, change the status to **Released**.
7. Click **Save**.
The system will create the script and its deployment records.

Test the Script

Now that the script's deployed, let's make sure it works.

To test the script:

1. Start creating a new employee record: go to Lists > Employees > Employees > New.
2. On the new employee form, check that the Notes field is disabled. If beforeLoad worked, the field should be grayed out and not editable.
3. Fill in all required fields—these might vary based on your account settings, but you'll need at least:
 - **Name** — Enter a first name of John and last name of Smith.
 - **Subsidiary** — (OneWorld only) Choose an appropriate value from the dropdown list.
4. To make sure afterSubmit works, add a value for **Supervisor** if you haven't done so already.
5. Click **Save**. You'll see a success message, and the new record appears in View mode.
6. Check that beforeSubmit worked: The Notes field should say **Orientation date TBD**.

Confirmation
Employee successfully Saved

Employee John Smith Active

Edit Back Print Copy Actions

Primary Information

Employee ID	John Smith	Initials	JS
Mr./Ms...		Job Title	
Name	John Smith	Supervisor	Tom A Taylor

Notes
Orientation date TBD

7. Check that afterSubmit worked:
 - a. Select Activities > Scheduling > Tasks.
 - b. Make sure your view includes tasks assigned to all users.
 - c. Check that a task was created and assigned to the supervisor you picked earlier.

Tasks List Search Audit Trail Calendar

View: Task Basic Customize View New Task

Filters

Print Copy Edit Quick Sort: Due Date Total: 1

Edit View	Internal ID	Due Date	Task Title	Company	Priority	Status
<a>Edit <a>View	150022	10/21/2024	Schedule orientation schedule for John Smith		Medium	Not Started

Next Steps

If you want to customize this entry point script to call a custom module, see [SuiteScript 2.x Custom Module Tutorial](#).

SuiteScript 2.x User Event Script Entry Points and API

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Important: You can't run a user event script from another user event script, or from a workflow with a **Context** of **User Event Script**—so you can't chain them together. But you can run a user event script from inside a scheduled script, a portlet script, or a Suitelet.

Script Entry Point	Description
afterSubmit(context)	Runs right after a write operation on a record.
beforeLoad(context)	Runs whenever a record is read, before it's returned or the page is loaded.
beforeSubmit(context)	Runs before any write operation on the record.
context.UserEventType	Stores the string values for user event execution contexts.

afterSubmit(context)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Description	Runs the function right after a record is submitted. Use <code>afterSubmit</code> for anything you want to happen after a write operation—like sending an email, redirecting a browser, creating related records, or syncing with an external system. Notes: ■ The approve, cancel, and reject types are only for record types like sales orders, expense reports, timebills, purchase orders, and return authorizations. ■ Attaching or detaching a child custom record triggers an edit event. ■ Asynchronous <code>afterSubmit</code> user events only work during webstore checkout. This event can be used with the following <code>context.UserEventType</code>: ■ create ■ edit ■ xedit (inline editing—only returns the edited fields, not the full record) ■ delete ■ approve (only available for certain record types) ■ cancel (only available for certain record types) ■ reject (only available for certain record types) ■ pack (only available for certain record types, for example Item Fulfillment records) ■ ship (only available for certain record types, for example Item Fulfillment records) ■ dropship (for purchase orders with items specified as “drop ship”) ■ specialorder (for purchase orders with items specified as “special order”) ■ orderitems (for purchase orders with items specified as “order item”) ■ paybills (use this type to trigger <code>afterSubmit</code> for Vendor Payments from the Pay Bill page. No sublist line item info will be available, so you'll need a search or lookup for details.)
--------------------	--

Returns	void
Since	Version 2015 Release 2

Parameters

Note: The context parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
context.newRecord	record.Record	required	The new (or updated) record is read-only. To edit it, use record.load(options) to load the submitted record, make your changes, and submit it again.	Version 2015 Release 2
context.oldRecord	record.Record null	required if context.UserEventType is not equal to CREATE	The old record (the previous version) is also read-only. This parameter's empty when context.UserEventType is CREATE (a new record).	Version 2015 Release 2
context.type	string	required	This is the trigger type—set it using context.UserEventType .	Version 2015 Release 2

Want an example of afterSubmit? Check out the [SuiteScript 2.x User Event Script Sample](#).

beforeLoad(context)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Description	Runs a function before a record loads—whenever there's a read operation, and before the record or page is returned. This covers things like navigating to a record in the UI, reading it via SOAP web services, or just loading a record. You can't use beforeLoad to source standard records. For that, use the pageInit(scriptContext) client script instead. Notes: - beforeLoad user events don't trigger when you load or access an online form. - You can't update a record that's loaded in a beforeLoad script—if you try, that logic is ignored. - You can update records created in beforeLoad user events, though. - Attaching or detaching a child custom record triggers an edit event. This event can be used with the following context.UserEventType: - create - edit - view
--------------------	---

	- copy - print - email - quick view
Returns	void
Since	Version 2015 Release 2

Parameters

 **Note:** The context parameter is a JavaScript object.

Parameter	Type	Required/Optional	Description	Since
context.form	serverWidget.Form	required	The current form.	Version 2015 Release 2
context.newRecord	record.Record	required	The new record (the record being loaded).	Version 2015 Release 2
context.request	http.ServerRequest	optional	This is the HTTP request info sent by the browser. If a server triggers the event, you won't see this value.	Version 2015 Release 2
context.type	string	required	The type of operation invoked by the event (the trigger type). The type matches any value in the context.UserEventType enum. You can use this parameter to branch your script logic depending on the operation type. For example, only run delete logic when type equals delete. User event scripts should always check the type argument so you don't run code when you shouldn't.	Version 2015 Release 2

For an example of the beforeLoad entry point, see [SuiteScript 2.x User Event Script Sample](#).

beforeSubmit(context)

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Description	This function runs before a record is submitted—so before any write operation happens.
--------------------	--

	Any changes you make to the record in this script stick after the write operation. You can use <code>beforeSubmit</code> to validate the record, add restrictions or permission checks, or make last-minute changes. Notes: ■ The approve, cancel, and reject argument types only work for record types like sales orders, expense reports, timebills, purchase orders, and return authorizations. ■ Only <code>beforeLoad</code> and <code>afterSubmit</code> entry point functions run on the Message record type when a message is created by inbound email case capture—<code>beforeSubmit</code> scripts won't run. ■ User Event Scripts can't override custom field permissions. So, if a user's role permissions and a custom field's permissions don't match, <code>beforeSubmit</code> can't update the field—even if the script's set to run as Administrator. ■ If you attach or detach a child custom record from its parent, it triggers an edit event. ■ The context parameters for old and new records are always in standard mode. For details, see SuiteScript 2.x Standard and Dynamic Modes. This event can be used with the following <code>context.UserEventType</code>: ■ create ■ edit ■ xedit (inline editing; only returns the fields edited and not the full record) ■ delete ■ approve (only available for certain record types) ■ cancel (only available for certain record types) ■ reject (only available for certain record types) ■ pack (only available for certain record types, for example Item Fulfillment records) ■ ship (only available for certain record types, for example Item Fulfillment records) ■ markcomplete (specify this type for a <code>beforeSubmit</code> script to execute when users click Mark Complete on call and task records) ■ reassign (specify this type for a <code>beforeSubmit</code> script to execute when users click Grab on case records) ■ editforecast (specify this type for a <code>beforeSubmit</code> script to execute when users update opportunity and estimate records using the Forecast Editor)
Returns	void
Since	Version 2015 Release 2

Parameters

 **Note:** The context parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
<code>context.newRecord</code>	record.Record	required	The new (or updated) record.	Version 2015 Release 2
<code>context.oldRecord</code>	record.Record	required	The old record (how the record looked before).	Version 2015 Release 2
<code>context.type</code>	string	required	The trigger type—set with context.UserEventType .	Version 2015 Release 2

For an example of the beforeSubmit entry point, see [SuiteScript 2.x User Event Script Sample](#).

context.UserEventType

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

Enum Description	This holds the string values for user event execution contexts. The beforeLoad, beforeSubmit, and afterSubmit entry points use context.type, which takes one of the enum values.  Note: JavaScript doesn't have a built-in enumeration type. When SuiteScript 2.x docs say "enum," they mean a plain object where each key maps to a read-only string value.
Module	SuiteScript 2.x User Event Script Type
Since	Version 2015 Release 2

Values

■ APPROVE ■ CANCEL ■ CHANGEPASSWORD  Note: The CHANGEPASSWORD user event runs when a user updates their NetSuite password in the UI. That event only appears in the NetSuite account where the password was changed. ■ COPY ■ CREATE ■ DELETE ■ DROPSHIP ■ EDIT ■ EDITFORECAST ■ EMAIL ■ MARKCOMPLETE ■ ORDERITEMS ■ PACK ■ PAYBILLS ■ PRINT ■ QUICKVIEW ■ REASSIGN ■ REJECT ■ SHIP ■ SPECIALORDER ■ TRANSFORM ■ VIEW ■ XEDIT

Note: There's no execution context for VOID actions, so it isn't a supported user event context. When a transaction is voided, scripts set to run on EDIT might still trigger on VOID, but may not work as expected. To avoid issues, add conditions in your EDIT scripts to filter out transactions where the memo is "Void" or "Voided," and check for the voided field. See [Catch the Void Using a User Event Script](#) for details.

For an example of the context.UserEventType, see [SuiteScript 2.x User Event Script Sample](#).

Note: If you attach or detach a child custom record from its parent, it triggers an edit event.

SuiteScript 2.x Workflow Action Script Type

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Workflow action scripts let you create custom Workflow Actions on a record in a workflow. They're handy for working with sublists since sublist fields aren't available through the Workflow Manager. You can also use workflow action scripts to run advanced logic you can't do with built-in actions.

For more on SuiteFlow workflows, check out these topics:

- [SuiteFlow Overview](#)
- [Working with Workflows](#)
- [SuiteFlow Reference and Examples](#)

For details about scripting with workflow action scripts, see [Creating and Using Workflow Action Scripts](#) and [SuiteScript 2.x Workflow Action Script Entry Points and API](#).

You can use SuiteCloud Development Framework (SDF) to manage these scripts in file-based projects. For more, see [Use the Copy to Account feature to copy a workflow action script to another account](#). Click Copy to Account at the top right of the page. For more info, see the help topic [Copy to Account](#).

You can use SuiteScript Analysis to see when the script was installed and how it's performed. To learn more, see the help topic [Analyzing Scripts](#).

Workflow Action Script Sample

This sample shows how to save a return value from a custom action script to a workflow field.

- You want to get a value from the Item sublist and use the value as a condition in the workflow. You use `record.getSublistValue` in the script and return this in the workflow.
- You want to check if a certain item exists in the Item sublist. The script returns "0" if it doesn't and "1" if it does.
- You want to make sure that all items in the Item sublist have a quantity equal to or greater than 1 (similar case as above).

Before you use this script, make sure you've done the following:

- Make sure the script returns a value – you can set this on the Parameters subtab of the Script record page.

- In SuiteFlow, create a workflow field that matches the data type of the script return value.
- In a workflow state, add the custom action (your Workflow Action script).
- Add the Workflow Action script's return value to the Store Result In field under custom action Parameters.

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType WorkflowActionScript
4   */
5  define([], function() {
6      function onAction(scriptContext){
7          log.debug({
8              title: 'Start Script'
9          });
10         var newRecord = scriptContext.newRecord;
11         var itemCount = newRecord.getLineCount({
12             sublistId: 'item'
13         });
14         log.debug({
15             title: 'Item Count',
16             details: itemCount
17         });
18         for (var i = 0; i < itemCount; i++){
19             var quantity = newRecord.getSublistValue({
20                 sublistId: 'item',
21                 fieldId: 'quantity',
22                 line: i
23             });
24             log.debug({
25                 title: 'Quantity of Item ' + i,
26                 details: quantity
27             });
28             if (quantity === 0){
29                 return 0;
30             }
31         }
32         log.debug({
33             title: 'End Script'
34         });
35         return 1;
36     }
37     return {
38         onAction: onAction
39     }
40 });

```

See the SuiteFlow help topics for additional samples, such as [Storing a Return Value from a Custom Action Script in a Workflow Field](#). You can find more examples if you search “workflow action script” on Suite Answers—some may be in SuiteScript 1.0.

Creating and Using Workflow Action Scripts

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

You set up a workflow action script like any other SuiteScript type: go to Customization > Scripting > Scripts > New.

With a workflow action script, you can create generic Custom Actions available for all record types. Choose **All Records** in the Applies To dropdown on the Script Deployment page.

Applies To *	ID	Deploy
- All Records -		Yes

These custom actions will then show up in all workflows, no matter the record type. For example, you can build a generic, parameterized action to set sales rep values. Then set the parameters inside the workflow and run the “Set Sales Rep (Custom)” action to use values specific to that workflow.

If you deploy a workflow action script to All Records, you can't also pick another record type—otherwise, you'll get an error. When set to All Records, the script appears as a **custom** action in all workflows.

You can create parameters for a workflow definition in the customization details in Workflow Manager.

You can also set the return value type for a workflow action script on the Parameters subtab of the script record. If your workflow (or workflow state) has fields of the same type, you can set this value to be saved to a specific field.

Additional Example

The following is a custom action workflow action script that sets the sales rep on the record in the workflow.

```

1  /**
2   * @NApiVersion 2.1
3   */
4
5  function onAction (scriptContext) {
6    const newRecord = scriptContext.newRecord;
7    const myScript = runtime.getCurrentScript();
8    const scriptParamSalesRep = myScript.getParameter({
9      name: 'custscript_salesrep'
10    });
11    newRecord.setValue({
12      fieldId: 'salesrep',
13      value: scriptParamSalesRep
14    });
15  }

```

Notice that you don't need to use the record type or record ID parameters (they're still sent, though); scriptContext gives you everything you need for the current record in the workflow.

Also, when running workflow action scripts, the current record context is **workflow**. See the help topic [runtime.executionContext](#) for more about what triggered the script.

SuiteScript 2.x Workflow Action Script Entry Points and API

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Script Entry Point	Description
onAction(scriptContext)	This defines a Workflow Action script trigger point.

onAction(scriptContext)

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Description	This defines a Workflow Action script trigger point. For a script example, check out Workflow Action Script Sample .
Returns	void
Since	Version 2016 Release 1

Parameters

Note: The scriptContext parameter is a JavaScript object.

Parameter	Type	Required / Optional	Description	Since
scriptContext.newRecord	record.Record	required	The new record. You can't use record.Record.save() here.	2016.1
scriptContext.oldRecord	record.Record	required	The old record. You can't use record.Record.save() here.	2016.1
scriptContext.form	serverWidget.Form	optional	The current form the script uses to work with the record (only available in beforeLoad).	2016.2
scriptContext.type	string	optional	The event type—like create, edit, view, or delete.	2016.2
scriptContext.workflowId	integer	optional	The internal ID of the workflow that's calling the script.	2016.2

SuiteScript 2.x Record Actions and Macros

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

SuiteScript 2.x supports APIs that provide the programmatic equivalent of clicking a button in the NetSuite user interface. The record action and macro APIs let you use SuiteScript to run the same business logic as a UI button click. These APIs can boost productivity by automating routine tasks that used to require manual work in the UI.

For more information, see the following topics:

- [Overview of Record Action and Macro APIs](#)
- [Supported Record Macros](#)
- [Supported Record Actions](#)

Overview of Record Action and Macro APIs

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

NetSuite records offer two alternatives for executing native NetSuite logic: a user either clicks a UI button or runs a script that calls the API corresponding to the button. These script and UI alternatives both produce the same results. Macro and action APIs make scripting easier and more flexible. These APIs are supported for all SuiteScript 2.x client and server script types. Macro and action APIs also can lower governance usage because they can execute complex business logic in one API call instead of multiple API calls. Governance is charged per action, and the cost depends on the action type. For macros, no governance is charged, because changes executed by macros are saved as part of record submits.

Record actions provide a convenient way to update the state of one or more records that are in view mode. Changes that the execution of an action API makes to records are persisted in the database immediately. You don't need to consider required roles, permissions, or other conditions for a record action to execute. The conditions required to execute an action are embedded in the record action API. If the conditions aren't met, the action doesn't execute. Approve and reject are two example use cases for record actions. These actions can be applied to a single record or to multiple records of the same type, as a bulk process. When an approve or reject action is executed on a record, the approval status of the record is saved immediately.

Record macros provide an automated way to execute business logic on a record as it is edited. Changes that the execution of a macro API makes to a record are not persisted until the record is saved. An example use case for a record macro is a preview of the calculated tax amount for a sales order's items. This macro API executes after items are entered on a sales order. It results in the display of the calculated tax amount on the sales order. However, the tax amount is not saved until the record is saved. A macro API is applied only to one single record at a time. When you save changes to a record, it may trigger changes to related records.

You'll need to use two types of APIs to work with record macros and actions in your scripts:

- Generic APIs that let you get and run actions or macros:
 - Record action APIs are part of the [N/action Module](#). They include generic members for getting and executing actions on a record type. For details, see the help topic [N/action Module](#).
 - Record macro APIs are part of the [N/record Module](#). They include generic members for getting and executing macros on a record type. For details, see the help topics [Record Object Members](#) and [Macro Object Members](#).

- Individual APIs that implement specific logic on a specific record type:
 - A limited number of Individual actions for specific record types are supported. For details, see [Supported Record Actions](#).
 - A limited number of individual macros for specific record types. For details, see [Supported Record Macros](#).

Note: For information about working with record actions in REST web services, see the help topic [Executing Record Actions](#).

Executing Actions and Macros Using SuiteScript 2.0

Supported Record Actions

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Record actions make it convenient to update records that are currently being viewed. For more information, see [Overview of Record Action and Macro APIs](#).

Record macro APIs are part of the [N/action Module](#). For more information, see the help topic [N/action Module](#).

The following table lists currently supported record actions. The table is organized by record type and includes the UI button labels. For details about a specific action, click its Action ID.

Note: For information about working with record actions in REST web services, see the help topic [Executing Record Actions](#).

Record Type	Action ID	UI Button Label
Bonus	cancel	Exclude from Payroll
Charge	clearBudgetAmounts	Clear Values
Charge	distributeBudgetTotalAmount	Distribute Total
Charge	selectAllBudgetLines	Mark All
Charge	setBudgetAmountsToCalculated	Set to Calculated
Charge	unselectAllBudgetLines	—
Inventory Count	approve	Approve
Inventory Count	completecount	Complete Count
Inventory Count	reject	Reject
Inventory Count	startcount	Start Count
Invoice	markforgrouping	Mark for Grouping
Invoice	removeinvoicefromgroup	Remove From Grouping
Invoice	unmarkforgrouping	Unmark for Grouping
Invoice Group	groupinvoices	Group Invoices

Invoice Group	removeinvoicesfromgroup	Unlink Invoices
Revenue Recognition Event	removeinvoicesfromgroup	Recalculate % Complete Override
Revenue Arrangement	allocate	Allocate
Supply Change Order	approve	Approve all pending
Supply Change Order	reject	Reject all pending
Supply Plan Definition	launch	Launch
Time (TimeBill)	approve	Approve
Time (TimeBill)	reject	Reject
Time (TimeBill)	retract	Retract
Time (TimeBill)	submit	Submit
Vendor Payment	confirm	Confirm
Vendor Payment	decline	Decline
Weekly Timesheet	approve	Approve All Pending
Weekly Timesheet	reject	Reject All Pending
Weekly Timesheet	retract	Retract
Weekly Timesheet	submit	Submit

Supported Record Macros

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Record macros automate business logic when you edit a record. For more information, see [Overview of Record Action and Macro APIs](#).

Record macro APIs are part of the [N/record Module](#). For more information, see the help topics [Record Object Members](#) and [Macro Object Members](#).

The following table lists currently supported record macros. The table is organized by record type and includes the UI button labels. For details about a specific macro, click its Macro ID.

Record Type	Macro ID	UI Button Label
Billing Rate Card	modifyPriceByPercent	Recalculate
Project Charge Rule	copyResources	Copy Resources from Tasks
Project Work Breakdown Structure Macros	getAmountFieldValue	Get Amount Field Value
Project Work Breakdown Structure Macros	getAmountsFields	Get Amounts Fields
Project Work Breakdown Structure Macros	getUnmatchedActuals	Get Unmatched Actuals
Project Work Breakdown Structure Macros	setAmountFieldValue	Set Amount Field Value
Sales Order	autoAssignLocations	Auto Assign Locations

Transaction Record Macros	calculateTax	Preview Tax
Transaction Record Macros	getSummaryTaxTotals	Get Summary Tax Totals
Weekly Timesheet	checkTimeLimits	Check Time Limits
Weekly Timesheet	copyFromWeek	Copy From Week

SuiteScript 2.x JSDoc Validation

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

i Note: If you're using SuiteScript 2.0 for your scripts, consider updating your scripts to use SuiteScript 2.1. This version includes new language features that are provided by the ECMAScript 2019 specification. For more information, see [SuiteScript 2.1](#) and [SuiteScript Versioning Guidelines](#).

JSDoc 3 is a documentation generator for JavaScript source code. Developers insert specific comment blocks into their source code, and these comment blocks can be extracted to generate documentation. These comment blocks start with `/**` and end with `*/`.

JSDoc also includes its own markup language, which is made up of JSDoc tags. These tags are prefaced with the `@` symbol. The JSDoc tags are added to the comment blocks and are used to specify the various entities of a JavaScript file (for example, `@param`).

```
1  /**
2   * Creates a file
3   * @param {string} name - file name
4   * @param {string} fileType - file type
5   * @param {string} contents - file content
6   * @returns {Object} file.File
7  */
```

JSDoc parses the source code for each comment block. JSDoc 3 also allows you to create custom JSDoc tags. These tags can be defined to trigger events (for example, displaying a certain page).

SuiteScript 2.x includes several custom tags, including `@NApiVersion` and `@NScriptType`. Some of these tags are required in each entry point script uploaded to NetSuite. To learn about these tags, see [SuiteScript 2.x JSDoc Tags](#).

When a SuiteScript 2.x script record is requested, NetSuite uses JSDoc 3 to evaluate the entry point script and parse the code for the required JSDoc tag. This tag is used to validate the SuiteScript version.

i Note: SuiteScript 2.x users can use JSDoc 3 to create their own documentation for scripts, custom modules, and SuiteApps. To take advantage of this tool, developers must download JSDoc 3 from the official website. For more information about JSDoc 3, see <https://jsdoc.app/>.

JSDoc Comment Blocks

To be recognized as valid JSDoc content, JSDoc tag comment blocks must start with `/**` and end with `*/`. JSDoc tags consist of a key-value pair. The key is a string starting with the `@` symbol and ending with the first white space after this key string. The value starts with the next non-whitespace character and ends with the next carriage return. Each comment line in the block starts with `*`.

The following table lists examples of valid and invalid JSDoc formatting.

Valid Examples	Invalid Examples
<pre>1 /** 2 * @NApiVersion 2.x 3 */</pre>	<pre>1 /** 2 * @NApiVersion 2.x 3 */</pre> The JSDoc tag comment block does not start with <code>/**</code>. <pre>1 // @NApiVersion 2.x</pre>

Valid Examples	Invalid Examples
	The JSDoc tag comment is a single-line comment and not a block. <pre> 1 /** 2 * @NApiVersion 2.x*/ </pre> The JSDoc tag comment block does not include a carriage return after the value.

SuiteScript 2.x JSDoc Tags

The following table describes the available SuiteScript 2.x JSDoc tags. SuiteScript 2.x entry point scripts must include the following two tags:

- @NApiVersion
- @NScriptType

For more information about entry point validation, including possible errors, see [SuiteScript 2.x Entry Point Script Validation](#).

JSDoc Tag	Possible Values	Required/Optional	Description
@NApiVersion	2.0 2.1 2.x 2.X	Required for entry point scripts Optional for custom modules	This tag is used in two ways: ■ For SuiteScript entry point scripts, this tag is a required declaration. It indicates to NetSuite which SuiteScript version to use. ■ For SuiteScript custom modules that are not entry point scripts, this tag is an optional declaration to specify the script version. This tag can be used to prevent compatibility issues if a script that references your custom module uses another SuiteScript version with language features or syntax that your custom module does not support. The 2.x or 2.X value usually represents the latest version of SuiteScript that is generally available and does not represent any versions that are released as beta features. However, this does not apply to SuiteScript 2.1. In this release, the 2.x value indicates that a script uses SuiteScript 2.0, not SuiteScript 2.1. You can still use SuiteScript 2.1 and all of its features in your scripts, but your 2.x scripts will not run as SuiteScript 2.1 scripts until a future release. For more information about SuiteScript versioning, see SuiteScript Versioning Guidelines.
@NScriptType	BundleInstallationScript ConsolidatedRateAdjustorPlugin CustomGLPlugin ClientScript CustomTool EmailCapturePlugin MapReduceScript MassUpdateScript PaymentGatewayPlugin PluginTypeImpl Portlet	Required for entry point scripts	This tag identifies the type of script defined in the file.

JSDoc Tag	Possible Values	Required/Optional	Description
	PromotionsPlugin Restlet ScheduledScript ShippingPartnersPlugin Suitelet TaxCalculationPlugin UserEventScript WorkflowActionScript		
@NModuleScope	SameAccount TargetAccount Public	Optional	This tag is used to control access to scripts and custom modules. ■ If the value is set to SameAccount, access to the module is limited to other modules from the same bundle, as well as modules native to the same source account and any associated sandboxes and Release Preview accounts. Source code is not hidden at runtime. ■ If the value is set to TargetAccount, access to the module is limited to other modules from the same bundle, as well as modules native to the same source account, target account, and any associated sandboxes and Release Preview accounts. Source code is hidden at runtime. ■ If the value is set to Public, any script in the account can load and use the module. Source code is hidden at runtime. The default value is SameAccount. Note: If your script contains SuiteScript 2.1 syntax that includes classes and will be included in a bundle, the @NModuleScope JSDoc tag must be set to SameAccount. For more information on each of these values, see Controlling Access to Scripts and Custom Modules.

Controlling Access to Scripts and Custom Modules

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

You can define the scope of environments (associated NetSuite accounts, sandboxes, and bundles) that may access a script. Access refers to whether the module can be loaded and invoked by another module. Modules are either loaded by NetSuite (using entry point modules) or by other modules (as libraries).



Important: Before deploying a SuiteApp, make sure that you review the module scope. This will help you to avoid:

- Deploying a SuiteApp that doesn't work as expected because it can't access a necessary module in another bundle.
- Providing wider access than desired to third parties and other accounts.

To set the scope for a script, you must include the appropriate value using the JSDoc tag.

If you don't set the scope, it defaults to SameAccount. For information about adding a JSDoc tag, see [SuiteScript 2.x JSDoc Validation](#).

The following table lists and describes the access control modifiers (supported module scopes) that are available in SuiteScript 2.x:

ModuleScope Value	Description	Access Disallowed When:	Visibility of Source Code	Examples
SameAccount	Limits script access to modules native to the same environment where the script was created and uploaded. This environment includes the account from which a bundle is installed and can also include the related family of sandbox accounts. For more information about sandbox accounts, see the help topic Understanding NetSuite Account Types. A script file that is 'native' to the environment is added to an account using an authenticated user session.	A bundled module from a different source account attempts to import the module.	Visible during runtime	- Installing a customization from a single sandbox to a production environment that is linked to the same account. - Distributing a bundle with private business logic as an ISV (Independent Software Vendor).
TargetAccount	Limits script access to modules native to the same source or target environment as the script. A source environment includes the NetSuite account (and any associated sandboxes and Release Preview accounts) from which a bundle is installed into another account. A target environment includes the NetSuite account (and any associated sandboxes and Release Preview accounts) into which a bundle is installed (whether using Bundle Copy or Bundle Install).	A bundled module that is not native to the target or source account attempts to import the module.	Hidden during runtime	- Account administrators distributing private business logic between accounts they control, and the modules in the bundle are needed by other modules created in the target account - Distributing a bundle with public APIs intended for import only by modules native to the target account.
Public	Any bundle (native and third party) that is installed in the account can run the script.	The script is not installed in the account.	Hidden during runtime	- Customers installing customizations from multiple sandbox accounts to production, where modules from different sandboxes need to load each other. - Installing a customization from a

ModuleScope Value	Description	Access Disallowed When:	Visibility of Source Code	Examples
				development account to a sandbox or production account. ■ Developing open source code.

For more information about packaging and distributing bundles (SuiteApps), see the help topic [SuiteBundler Overview](#).



Note: SuiteBundler is still supported but will not be updated with any new features.

To take advantage of new features for packaging and distributing customizations, you can use the Copy to Account and SuiteCloud Development (SDF) features instead of SuiteBundler.

Copy to Account is an administrator tool that you can use to copy custom objects between your accounts. The tool can copy one custom object at a time, including dependencies and data. For more information, see the help topic [Copy to Account](#).

SuiteCloud Development Framework is a development framework that you can use to create SuiteApps from an integrated development environment (IDE) on your local computer. For more information, see .

NetSuite Development Accounts and Module Scope



Warning: If you use a NetSuite Development account to work on a module that you intend to distribute or test, you should choose a module scope value rather than accept the default.

Keep in mind that development accounts are limited to 5 users and isolated from production and sandbox accounts. Set the value to Public if other bundles or accounts depend on using a module that is sourced from a development account.

If the default module scope is used on a script in a development account that is packaged into a bundle and installed to production or sandbox accounts, that script will not be accessible to modules installed from other NetSuite accounts. For example, when developers use multiple developer accounts to collaborate on a project, the SameAccount module scope might not be an appropriate access control level. This scope prevents modules installed from different accounts from loading one another.

For more information about development accounts, see The Development Account Environment, [NetSuite Development Accounts](#), and the [SuiteApp Development Process with SuiteBundler](#).

SuiteScript 2.x Entry Point Script Creation and Deployment

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Creating a Script Record

After you write an entry point script, you must take the following steps to run the script in your NetSuite account:

1. Upload the script file to your File Cabinet. The file must have all required script elements as described in [SuiteScript 2.x Entry Point Script Validation](#).
2. Deploy your script:
 - Deploy your script on one or more record types, as described in [SuiteScript 2.x Record-Level Script Deployments](#). All entry point script types can be deployed at the record level.
 - Deploy your script on a form, as described in [SuiteScript 2.x Form-Level Script Deployments](#). Only client scripts can be deployed at the form level.

To understand the difference between deploying a client script at the record level versus the form level, see [Record-Level and Form-Level Script Deployments](#).

Record-Level and Form-Level Script Deployments

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

A client script can be deployed either at the record level or at the form level. All other entry point script types can be deployed only at the record level.

Record-level Deployment

When you deploy a script at the record level, it's deployed globally on one or more record types and will run on all forms associated with that record type. For example, you would deploy a script to run on the employee record type to have the script run whenever the employee record is used, regardless of which form it is being used on.

With record-level deployment, you can limit script execution by configuring it to be available only to certain audiences. With this approach, the script runs only when the record is used by people in certain roles, groups, or other classifications, as configured on the Audience subtab of the script deployment record.

However, with record-level deployment, you can't limit the script to run on only one form as you can with form-level deployment. The script runs the same way on all forms associated with the record type it is deployed on.

Note: Record-level client scripts can also be used on forms and lists that have been generated through the use of Suitelets. Form-based client scripts can't be used by Suitelets.

Form-level Deployment

When you deploy a client script at the form level, you attach it to a custom form for a record type, and it only runs when that form is used. For example, you could attach a client script to a custom entry form for the employee record type. When a user opens the employee record using the standard entry form, the script won't run. It will only run when the user opens the custom form for the employee record.

You can attach a client script to any custom entry form, custom transaction form, or custom address form. However, you can't limit the audience for a form-level script as you can with a record-level script deployment.

Hidden form fields can't be accessed or manipulated by client scripts.

For information about deploying a script at the record level, see [SuiteScript 2.x Record-Level Script Deployments](#). For information about deploying a client script at the form level, see [SuiteScript 2.x Form-Level Script Deployments](#).

SuiteScript 2.x Entry Point Script Validation

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

For a SuiteScript 2.x entry point script to work, it must contain certain required elements. The system checks each script file for these elements when you upload it to the File Cabinet and saves data about your file which is used when you create a script record for the script.

If NetSuite detects that a required element within the script file is missing or incorrectly formatted, it returns an error. Errors can be returned when you upload the file to the File Cabinet or when you create a script record for the script file. Errors can also be returned when you attach a client script to a custom form.

For more information, see the following help topics:

- [Entry Point Script Validation Guidelines](#)
- [Entry Point Script Validation Examples](#)
- [Entry Point Script Validation Error Reference](#)

Entry Point Script Validation Guidelines

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

For an entry point script to be valid, its structure must meet certain requirements. If it doesn't meet all these requirements, you won't be able to create a script record for the script, and you won't be able to attach the script to a form. In some cases, you won't be able to upload the script file to the File Cabinet if it doesn't meet all requirements.

Note: If a correctly formatted script is already attached to a custom form or has a script record, you can't edit and save changes to the script file that would introduce errors during validation.

For more information, see the following:

- [Entry Point Script Validation Terms](#)

- Required Structure for Entry Point Scripts
- Required JSDoc Tags for Entry Point Scripts

Entry Point Script Validation Terms

As you learn about script validation requirements and errors, it's important to know certain terms. These terms are described in the following illustration and table.

```

/**
 * @NApiVersion 2.x
 * @NScriptType UserEventScript
 */
define(['N/record'],
  function(record) {

    function beforeLoad(context) {
      if (context.type !== context.UserEventType.CREATE)
        return;
      var customerRecord = context.newRecord;
      customerRecord.setValue('phone', '555-555-5555');
      if (!customerRecord.getValue('salesrep'))
        customerRecord.setValue('salesrep', 46);
    }
    function beforeSubmit(context) {
      if (context.type !== context.UserEventType.CREATE)
        return;
      var customerRecord = context.newRecord;
      customerRecord.setValue('comments', 'Please follow up with this customer!');
    }
    function afterSubmit(context) {
      if (context.type !== context.UserEventType.CREATE)
        return;
      var customerRecord = context.newRecord;
      if (customerRecord.getValue('salesrep')) {
        var call = record.create({
          type: record.Type.PHONE_CALL,
          isDynamic: true
        });
        call.setValue('title', 'Make follow-up call to new customer');
        call.setValue('assigned', customerRecord.getValue('salesrep'));
        call.setValue('phone', customerRecord.getValue('phone'));
        try {
          var callId = call.save();
          log.debug('Call record created successfully', 'Id: ' + callId);
        } catch (e) {
          log.error(e.name);
        }
      }
    }

    return {
      beforeLoad: beforeLoad,
      beforeSubmit: beforeSubmit,
      afterSubmit: afterSubmit
    };
  });

```

Callout	Description
1	JSDoc tags
2	JSDoc tag values
3	Entry point function definitions
4	Script type interface (or simply interface)
5	Entry points
6	Entry point functions (also called script type functions)

At a high level, the following rules apply:

- Each script must be properly structured as described in [Required Structure for Entry Point Scripts](#).
- Each script file must include at least two JSDoc tags: `@NApiVersion` and `@NScriptType`, and they must have valid values. Also, the value for the `@NScriptType` tag must be compatible with the script. For more information, see [Required JSDoc Tags for Entry Point Scripts](#).

Required Structure for Entry Point Scripts

An entry point script can't be associated with a script record or custom form unless it meets certain requirements. For the script to be valid, these requirements must be met:

- The script must include one, and only one, properly structured define statement. The script can't include multiple define statements.
- The define statement can't include direct references to SuiteScript 2.x objects or enums. All such references must be wrapped in a function.
- The script's return statement must include an interface whose entry points are associated with one, and only one, script type. In other words, the return statement must include only one script type interface. See [Entry Point Script Validation Terms](#).
- The interface for the return statement must include at least one entry point.
- Each entry point must correspond with an entry point function.
- All entry point functions must be defined in the same file.
- The script type interface implemented must match the `@NScriptType` value.
- The script must not contain any JavaScript syntax errors.

Required JSDoc Tags for Entry Point Scripts

Every entry point script must contain the `@NApiVersion` and `@NScriptType` JSDoc tags as described in the following table. This table includes information about possible validation errors. For more information about validation errors, see [Entry Point Script Validation Error Reference](#).

JSDoc Tag	Valid Values	Purpose	Possible Validation Errors
<code>@NApiVersion</code>	2.0 2.1 2.x 2.X	Identifies the SuiteScript version to use. 2.x and 2.X are equivalent.	■ FAILED_TO_VALIDATE_SCRIPT_FILE — If you omit this tag within a script file that is otherwise properly formatted, you'll receive this error when you upload the script file to the File Cabinet. ■ INVALID_API_VERSION — If you use an invalid value for this tag, you'll receive this error when you upload the script to the File Cabinet.
<code>@NScriptType</code>	BundleInstallationScript ConsolidatedRateAdjustorPlugin CustomGLPlugin CustomTool ClientScript EmailCapturePlugin	Identifies the type of script defined in the file.	■ WRONG_SCRIPT_TYPE — If you specify a value that is incompatible with the entry points included in the script, you'll receive this error when you upload the script file to the File Cabinet. ■ FAILED_TO_VALIDATE_SCRIPT_FILE — If you omit this tag within a script file that is otherwise properly formatted, you can upload the script to the File Cabinet, but you'll receive this error when you create a script record for the script or attach the script to a form.

JSDoc Tag	Valid Values	Purpose	Possible Validation Errors
	MapReduceScript MassUpdateScript PaymentGatewayPlugin PluginTypeImpl Portlet PromotionsPlugin Restlet ScheduledScript ShippingPartnersPlugin Suitelet TaxCalculationPlugin UserEventScript WorkflowActionScript		

For more information about working with JSDoc tags in SuiteScript 2.x, see [SuiteScript 2.x JSDoc Validation](#).



Tip: If you are currently using SuiteScript 2.0 for your scripts, consider updating your scripts to use SuiteScript 2.1. SuiteScript 2.1 includes new language features that are provided by a newer version of ECMAScript. For more information, see [SuiteScript 2.1](#) and [SuiteScript Versioning Guidelines](#).

Entry Point Script Validation Examples

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Here are some examples of correct and incorrect SuiteScript scripts:

- Example of a Valid Script that Includes All Required Elements
- Example of an Invalid Script — Missing Return Statement
- Example of an Invalid Script — Missing Define Statement
- Example of an Invalid Script — Missing @NScriptType JSDoc Tag
- Example of an Invalid Script — Missing @NApiVersion JSDoc Tag

Example of a Valid Script that Includes All Required Elements

The following script is valid. It includes a define statement, a return statement with an entry point, and an entry point function that corresponds with the entry point. Plus, the @NScriptType JSDoc tag matches the script type interface, and the @NApiVersion JSDoc tag is correct.

```

1  /**
2   * @NApiVersion 2.1
3   * @NScriptType UserEventScript
4   */

```



```

5
6 define(['N/record'], (record) => {
7   function myBeforeSubmitFunction(context) {
8     if (context.type !== context.UserEventType.CREATE)
9       return;
10    let customerRecord = context.newRecord;
11    customerRecord.setValue({
12      fieldId: 'comments',
13      value: 'Please follow up with this customer!'
14    });
15  }
16  return {
17    beforeSubmit: myBeforeSubmitFunction
18  };
19 });

```

Example of an Invalid Script — Missing Return Statement

The following script is invalid because it doesn't have a return statement. If you try to upload a script that doesn't include a return statement, the system returns the error "SuiteScript 2.0 entry point scripts must implement one script type function." For more information, see [SCRIPT_OF_API_VERSION_20_MUST ...](#)

```

1 /**
2  * @NApiVersion 2.1
3  * @NScriptType UserEventScript
4  */
5
6 define(['N/record'], (record) => {
7   function myBeforeSubmitFunction(context) {
8     if (context.type !== context.UserEventType.CREATE)
9       return;
10    let customerRecord = context.newRecord;
11    customerRecord.setValue({
12      fieldId: 'comments',
13      value: 'Please follow up with this customer!'
14    });
15  }
16 });

```

Example of an Invalid Script — Missing Define Statement

The following script is invalid because it uses a require statement instead of a define statement. If you try to upload a script that includes a require statement, the system returns the error "SuiteScript 2.0 entry point scripts must implement one script type function." For more information, see [SCRIPT_OF_API_VERSION_20_MUST ...](#). For more information about using the require statement, see the help topic [require Function](#).

```

1 /**
2  * @NApiVersion 2.x
3  * @NScriptType UserEventScript
4  */
5
6 require(['N/record'], (record) => {
7   function myBeforeSubmitFunction(context) {
8     if (context.type !== context.UserEventType.CREATE)
9       return;
10    let customerRecord = context.newRecord;
11    customerRecord.setValue({
12      fieldId: 'comments',
13      value: 'Please follow up with this customer!'
14    });
15  }
16 });

```

Example of an Invalid Script — Missing @NScriptType JSDoc Tag

The following script is invalid because it doesn't have an @NScriptType JSDoc tag. You can upload the script file, however, if you try to create a script record for the script, the system returns the error “@NScriptType is required for 2.0 entry point script.” For more information, see [MISSING_SCRIPT_TYPE](#).

```

1  /**
2   * @NApiVersion 2.1
3   */
4
5  define(['N/record'], (record) => {
6      function myBeforeSubmitFunction(context) {
7          if (context.type !== context.UserEventType.CREATE)
8              return;
9          let customerRecord = context.newRecord;
10         customerRecord.setValue({
11             fieldId: 'comments',
12             value: 'Please follow up with this customer!'
13         });
14     }
15     return {
16         beforeSubmit : myBeforeSubmitFunction
17     };
18 });

```

Example of an Invalid Script — Missing @NApiVersion JSDoc Tag

The following script is invalid because it doesn't have an @NApiVersion JSDoc tag. If you try to upload a script file that doesn't include the @NApiVersion JSDoc tag, the system returns the error “Failed to validate script file.” For more information, see [FAILED_TO_VALIDATE_SCRIPT_FILE](#).

```

1  /**
2   * @NScriptType UserEventScript
3   */
4
5  define(['N/record'], function(record) {
6      function myBeforeSubmitFunction(context) {
7          if (context.type !== context.UserEventType.CREATE)
8              return;
9          let customerRecord = context.newRecord;
10         customerRecord.setValue({
11             fieldId: 'comments',
12             value: 'Please follow up with this customer!'
13         });
14     }
15     return {
16         beforeSubmit : myBeforeSubmitFunction
17     };
18 });

```

Entry Point Script Validation Error Reference

❗ Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following table describes validation errors that can occur when you upload an entry point script, create a script record, or attach a client script to a custom form. Some errors can also be returned when

you edit a script file that has already been uploaded to NetSuite, attached to a script record, or attached to a custom form.

Error Code	Error Text (Description)
CANNOT_CHANGE_API_VERSION	This file is used by a SuiteScript {1} script; you cannot change the API version of the file. {1} — SuiteScript version (1.0, 2.0, etc).
FAILED_TO_VALIDATE_SCRIPT_FILE	Failed to validate script file: {1} {1} — the script file name.
INVALID_API_VERSION	Invalid JSDoc tag value; valid values are: @NApiVersion [2.X, 2.x, 2.0 2.1]
INVALID_JSDOC_TAG_VALUE	Invalid JSDoc tag value; valid values are: {1} {1} — the valid values based on the specific invalid JSDoc tag.
MISSING_SCRIPT_TYPE	The @NScriptType JSDoc tag is required for 2.0 entry point scripts.
MULTIPLE_DEFINE_CALLS	Invalid define call, define should only be called one time per module. Define calls found at the following line numbers: {1} {1} — the lines in the script file.
SCRIPT_OF_API_VERSION_20_CANNOT_IMPLEMENT_MORE_THAN_ONE_SCRIPT_TYPE_INTERFACES (See SCRIPT_OF_API_VERSION_20_CANNOT ...)	SuiteScript 2.0 entry point scripts cannot implement functions for more than one script type.
SCRIPT_OF_API_VERSION_20_MUST_IMPLEMENT_A_SCRIPT_TYPE_INTERFACE (See SCRIPT_OF_API_VERSION_20_MUST ...)	SuiteScript 2.0 entry point scripts must implement one script type function.
SS_V2_FILE_USED_FOR_FORM_SCRIPTS_MUST_IMPLEMENT_CLIENT_SCRIPT_TYPE (See SS_V2_FILE_USED_FOR_FORM_SCRIPTS)	SuiteScript version 2 file used for form scripts must implement client script type.
SYNTAX_ERROR	Syntax error: {1} {1} — additional information specific to the error.
THE_FILE_IS_USED_AS_SCRIPT_TYPE_1_CANNOT_BE_CHANGED_TO_2 (See THE_FILE_IS_USED_AS_SCRIPT_TYPE_1 ...)	The file is used as script type {1}, cannot be changed to {2} {1} and {2} — the script types.
WRONG_SCRIPT_TYPE	Script file includes @NScriptType {1}; this script type cannot be used to {2}. {1} — the script type specified; {2} — the script's entry points.

CANNOT_CHANGE_API_VERSION

Full Error Text:	This file is used by a SuiteScript {1} script; you cannot change the API version of the file. {1} — the SuiteScript version (that is, 1.0, 2.0, etc).
Occurs:	This error occurs when you edit a script file that is already attached to a script record or a custom form. Specifically, if you try to edit and save changes to the @NApiVersion JSDoc tag value. For example, you'll see this error if you try to remove the @NApiVersion 2.0 JSDoc tag from a version 2.0 script.

	Or, if you try to add the <code>@NApiVersion 2.0</code> JSDoc tag to a version 1.0 script, you'll see this error.
Correction:	–
Other Notes:	In some cases, this error is preceded by the INVALID_API_VERSION error, which is included if the new <code>@NApiVersion</code> JSDoc tag value you are trying to use is invalid. Note that SuiteScript 1.0 files don't use the <code>@NApiVersion</code> JSDoc tag. If you are still using SuiteScript 1.0 scripts, you should consider converting them to SuiteScript 2.0 or SuiteScript 2.1. For help creating and uploading SuiteScript 1.0 files, see the Running Scripts in NetSuite Overview topic in the SuiteScript 1.0 Documentation topic.

FAILED_TO_VALIDATE_SCRIPT_FILE

Full Error Text:	Failed to validate script file: {1} {1} — the script file name.
Occurs:	This error occurs if your script file isn't structured correctly. For example, you'll see this error if your script is formatted correctly for SuiteScript 2.0 in all ways except that it is missing the <code>@NApiVersion</code> JSDoc tag. This error may also be displayed if you don't correctly initialize global variables, as described in Resolve Error:"SuiteScriptModuleLoaderError FAIL_TO_EVALUATE_SCRIPT_1 All SuiteScript API Modules are unavailable while executing your define callback" when Saving SuiteScript 2.0 File
Correction:	Check your script file to make sure that it contains all required elements and is structured correctly.
Other Notes:	For more information about creating valid scripts, see SuiteScript 2.x Entry Point Script Validation .

INVALID_API_VERSION

Full Error Text:	Invalid JSDoc tag value; valid values are: <code>@NApiVersion [2.X, 2.x, 2.0, 2.1]</code>
Occurs:	This error occurs when you try to upload a script file with an invalid <code>@NApiVersion</code> JSDoc tag value. It can also be displayed if you try to modify the value of the <code>@NApiVersion</code> JSDoc tag in a script file that has already been uploaded.
Correction:	Check your script file to make sure the <code>@NApiVersion</code> JSDoc tag has a valid value. Valid values are: 2.0, 2.x, and 2.X. 2.x and 2.X are equivalent.
Other Notes:	For more information about the <code>@NApiVersion</code> JSDoc tag values, see SuiteScript 2.x JSDoc Tags .

INVALID_JSDOC_TAG_VALUE

Full Error Text:	Invalid JSDoc tag value; valid values are: {1} {1} — the valid values based on the specific invalid JSDoc tag.
Occurs:	This error occurs if your script file uses an invalid value for any JSDoc tag.
Correction:	Check your script file to make sure the value you used for the JSDoc tag is valid. Verify that it doesn't include typos or other errors.
Other Notes:	For more information about JSDoc tag values, see SuiteScript 2.x JSDoc Tags .

MULTIPLE_DEFINE_CALLS

Full Error Text:	Invalid define call, define should only be called one time per module. Define calls found at the following line numbers: {1} {1} — the lines in the script file where the define calls are.
Occurs:	This error occurs if your script includes more than one define statement. Each script can have only one define statement.
Correction:	Check your script file to make sure there is only one define statement, and rework it if multiple defines statements are in your script.
Other Notes:	For more information about using define calls, see the help topic define Object .

MISSING_SCRIPT_TYPE

Full Error Text:	@NScriptType is required for 2.0 entry point script
Occurs:	This error occurs if you try to create a script record for a SuiteScript 2.x script that doesn't include the @NScriptType JSDoc tag. This error also occurs if you remove the @NScriptType JSDoc tag from a script file that is attached to a script record.
Correction:	Edit the script file and add the @NScriptType JSDoc tag with the appropriate value. Also, don't remove the @NScriptType JSDoc tag from any script file that is already attached to a script record.
Other Notes:	This error occurs when you create a script record for your script file. You'll be able to successfully upload your script file to the File Cabinet, but you won't be able to create a script record for it. In this case, successfully uploading a script file doesn't mean that the file is valid.

SCRIPT_OF_API_VERSION_20_CANNOT ...

Full Error Text:	SuiteScript 2.0 entry point scripts cannot implement functions for more than one script type. The full code for this error is: SCRIPT_OF_API_VERSION_20_CANNOT_IMPLEMENT_MORE_THAN_ONE_SCRIPT_TYPE_INTERFACES
Occurs:	This error occurs if the interface (a return statement) of your script file includes entry points from more than one script type.
Correction:	Check your script to make sure the interface contains only entry points for one script type (the script type indicated in the @NScriptType JSDoc tag). If necessary, create additional script files to implement additional script type entry points.
Other Notes:	For a description of all entry points for each script type, see SuiteScript 2.x Script Types .

SCRIPT_OF_API_VERSION_20_MUST ...

Full Error Text:	SuiteScript 2.0 entry point scripts must implement one script type function. The full error code for this error is: SCRIPT_OF_API_VERSION_20_MUST_IMPLEMENT_A_SCRIPT_TYPE_INTERFACE.
------------------	--

Occurs:	This error occurs when your script is missing an interface (a return statement) or that an error exists within the interface. This error can occur when you try to upload a file, or when you try to edit a file that was previously uploaded.
Correction:	Check your script file to make sure that at least one entry point for the script type (as indicated in the @NScriptType JSDoc tag) is included in the interface.
Other Notes:	For a description of all entry points for each script type, see SuiteScript 2.x Script Types .

SS_V2_FILE_USED_FOR_FORM_SCRIPTS

Full Error Text:	SuiteScript 2.0 file used for form scripts must implement client script type. The full error code for this error is: SS_V2_FILE_USED_FOR_FORM_SCRIPTS_MUST_IMPLEMENT_CLIENT_SCRIPT_TYPE.
Occurs:	This error occurs if you try to attach a script type other than a client script to a custom form. Only a client script can be attached to a form (that is, deployed at the form level). All other types of entry point scripts can only be deployed at the record level.
Correction:	Make sure that the script you are attempting to attach to a custom form is a client script. Otherwise, deploy the script at the record level.
Other Notes:	For more information about working with client scripts on custom form, see SuiteScript 2.x Form-Level Script Deployments . For more information about deploying other types of entry point scripts at the record level, see SuiteScript 2.x Record-Level Script Deployments .

SYNTAX_ERROR

Full Error Text:	Syntax error: {1} {1} — additional information specific to the error.
Occurs:	This error occurs if your script contains JavaScript syntax errors.
Correction:	Check to make sure your script follows all JavaScript syntax rules.
Other Notes:	There are several resources available for JavaScript syntax rules including two JavaScript tutorial sites at: https://www.w3schools.com/js/default.asp and https://developer.mozilla.org/en-US/docs/Web/JavaScript .

THE_FILE_IS_USED_AS_SCRIPT_TYPE_1 ...

Full Error Text:	The file is used as script type {1}, cannot be changed to {2}. {1} and {2} — the script types. The full error code for this error is: THE_FILE_IS_USED_AS_SCRIPT_TYPE_1_CANNOT_BE_CHANGED_TO_2.
Occurs:	This error may occur if you edit a script file that is attached to an existing script record to use a different script type interface from what it previously used, and a different @NScriptType value.
Correction:	To avoid this error, make your changes in a script file that isn't already associated with a script record. Then create a new script record based on the updated script file.

Other Notes:	—
--------------	---

WRONG_SCRIPT_TYPE

Full Error Text:	Script file includes @NScriptType {1}; this script type cannot be used to {2} {1} — the script type specified; {2} — the script's entry points.
Occurs:	This error occurs if the @NScriptType JSDoc tag value isn't compatible with the entry points in the script's interface.
Correction:	Check your script to make sure the @NScriptType JSDoc tag value corresponds to the entry points in the script. If the tag's value doesn't correspond to the script type interface, you cannot upload the file to the File Cabinet.
Other Notes:	For more information about script types, their definitions, and entry points, see SuiteScript 2.x Script Types .

SuiteScript 2.x Record-Level Script Deployments

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

After you create an entry point script and it passes validation, as described in [SuiteScript 2.x Entry Point Script Validation](#), you'll need to deploy the script on records in your NetSuite Account by following these steps:

1. Upload the script file to the File Cabinet.
2. Create a script record for your script file, as described in [Script Record Creation](#). The script record identifies the script's internal ID, whether the script is active, and other details.
3. Deploy the script, as described in [Script Deployment](#). You can use script deployments to configure details for how and when the script runs. The choices you make vary depending on the script's type.

Script Record Creation

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

After you create an entry point script and uploaded it to the File Cabinet, you can create a script record for it. You must create a script record before you can deploy your script.

When you create a script record, you set some fields manually and the system automatically fills in other fields based on data in the script file. These automatically filled fields are described in the following table.

Field on the Script Record	Value Taken From
API Version	The @NApiVersion JSDoc tag in your script file.
Script File	The name of your script file.

Field on the Script Record	Value Taken From
Type	The @NScriptType JSDoc tag in your script file.
Functions (on the Script subtab)	The entry points defined within your script.

Note: Your script must be properly structured and validated before it can be used to create a script record. For more information, see [SuiteScript 2.x Entry Point Script Validation](#).

For more information about creating a script record, see the following help topics:

- [Creating a Script Record](#)
- [Working With Script Parameters](#)

Creating a Script Record

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

How you create a script record depends on the script's type. The following procedure shows general steps that apply to all script types. For certain script types, additional steps may be required.

To create a script record:

1. Go to Customization > Scripting > Scripts > New.
The system displays the Upload Script File page.
2. In the **Script File** list, select the appropriate SuiteScript (.js) file. The list shows all versions of SuiteScript script files that you have uploaded to your File Cabinet.
If the file you want to use hasn't been uploaded yet, you can upload it from this page. Point to the area at the right of the list to display a Plus icon.

Click the Plus icon to display a popup window where you can select and upload your .js script file from your local environment. Fill in the required fields to select your file and then click **Save**.

Note: Client scripts or scripts referenced inside a client script can't have the **Hide in SuiteBundle** preference enabled. Enabling the **Hide in SuiteBundle** preference on script files that need to be accessed on the client side will cause a validation error on save of the record.

File - Google Chrome

acct[redacted].app.netsuite.com/app/common/media/mediaitem.nl?restricttype=JAVASCRIPT&l=T&...

File

Save Cancel

Attach From *

Computer

File Name

Folder *

SuiteScripts

URL

Select File

Choose File No file chosen

Character Encoding

Unicode (UTF-8)

☐ Inactive

☐ Available for SuiteBundles

☐ Hide in SuiteBundle

☐ Available Without Login

☐ Company-wide Usage

☐ Generate URL Time Stamp

- After you select the file, click **Create Script Record**.

Upload Script File

Reset Cancel Create Script Record

SCRIPT FILE *

newUserEventScript.js

The Script page is displayed with your .js script file listed on the Scripts subtab. Fields that are automatically filled in based on your script (API Version, Script File, Type, and Functions) are read-only and can't be changed. Remember, these fields are automatically filled in based on the content of your script file.

Note: If you are redirected to a page that prompts you to select a script type, that means that in Step 2 you identified a SuiteScript 1.0 file. You can click the Back button in your browser to return to the previous page and select a different file. If the script you want to use is a 1.0 version script, consider converting it to SuiteScript 2.x.

4. Enter a name in the **Name** field.
5. In the **ID** field, optionally enter a custom ID for the script record. If the **ID** field is left blank, a system-generated internal ID is created when you save the record.
6. If the **Portlet Type** field is displayed, select the appropriate value. This field is available only for the portlet script type, and it is a required field.
7. In the **Description** field, optionally enter a description for the script.
8. If appropriate, change the value of the **Owner** field. By default, this field is set to the currently logged-in user.
9. If appropriate, check the **Inactive** box. Marking a script record inactive prevents the script from executing. When a script is set to Inactive, any deployments associated with the script are also inactive.
10. The **Scripts** subtab shows your script file name. You can select a different script file using icons next to the **Script File** field.
11. On the **Parameters** subtab, define any parameters (custom fields) that are used by functions in the script. For information about script parameters, see [Working With Script Parameters](#).
12. On the **Unhandled Errors** subtab, optionally define the people to be notified if any script errors occur.

Three types of error notifications are sent:

- An initial email is sent on the first occurrence of an error within an hour.
- A second email with aggregated error information for every hour is sent. (The error counter is reset when this email is sent.)
- A third email is sent for the first 100 errors that have occurred after the error counter is set to 0.

For example, if an error is thrown 130 times within an hour, an initial email is sent. Then, after the 100th occurrence, another email is sent. Since there are an additional 30 occurrences within the

same hour, a final summary email is sent at the end of the hour. During the second hour, if there are only 50 occurrences of the error, only one summary email is sent at the end of that hour.

By default, the **Notify Script Owner** box is checked. You can also identify other people to receive notifications, as follows:

- Check the **Notify All Admins** box, if all administrators should be notified.
- Select one or more groups from the **Groups** list. To define new groups, go to Lists > Relationships > Groups > New.
- Enter the email addresses of any other users who should be notified. You can enter a comma-separated list of email addresses.

 **Note:** SuiteScript execution and errors can be monitored using the Script Execution Log subtab. Script execution details can be viewed on the Script page, the Script Deployment page, and the SuiteScript Debugger. For more information, see the help topic [SuiteScript Monitoring, Auditing, and Logging](#).

13. Optionally, define a deployment for the script record by clicking the **Deployments** subtab and adding a line to the sublist. For more information about adding a line to this list, see [Deploying a Script by using the Deployments Sublist](#). You can also add a script deployment without adding a deployment to this list and clicking **Save and Deploy** when you are ready to save your script. This will display the Script Deployment page where you'll define your deployment.
14. If this is a client script, optionally add lines to the **Buttons** sublist, which appears on the **Scripts** subtab. You can translate the labels for these buttons. This feature is available in accounts with multi-language feature only.
15. If the **Scripts** subtab includes the **Custom Plug-in Types** sublist, optionally add lines to this list.
16. To save the new record, click one of the following:
 - **Save** – to save and close the record.
 - **Save and Deploy** – to save the script record and open the Script Deployment page that lets you create a new script deployment record.

When you save the script record using **Save**, the system shows the new script record on the Script page in view mode. The Scripts subtab lists all possible entry point functions that could exist for the script type used in your script. The entry points used in your script are checked. For example:

Script

← →

List Search Copy to

Edit

Back

Deploy Script

Actions

Type

User Event

Name

New User Event Script

ID

customscriptusereventscript

API Version

2.0

Description

Owner

Smith, John

☐ Inactive

Scripts

Parameters

Unhandled Errors

Execution Log

Deployments

System Notes

Script File

preview uc_ue_calculateCommissionSO.js download Edit

☐ Before Load Function

☐ Before Submit Function

☒ After Submit Function

Edit

Back

Deploy Script

Actions

The entry point functions listed on the Scripts subtab can't be edited. To add or remove an entry point function, you'll need to edit the script and add or remove the appropriate entry point functions. When you save your changes, the script record's list of entry point functions is updated to match the contents of the script file.

Working With Script Parameters

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

You can use script parameters to configure your script either through script deployment or by the end user. Script parameters should be created whenever you need to parameterize a script that is deployed multiple times. Using script parameters allows you to customize the behavior of the script for each deployment. For more information about using script parameters, see the help topic [Creating Script Parameters \(Custom Fields\)](#).

Script Deployment

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Before an entry point script will run in your NetSuite account, it must be deployed. You can deploy a script when you create a script record, or you can deploy it later. The deployment settings available vary depending on the script type and on how you deploy the script.

When you deploy a script, NetSuite creates a script deployment record. Script deployment records are listed at Customization > Scripting > Script Deployments. Deployments are also listed on the Deployments subtab of the script record.

Multiple deployments can be created for the same script record. When multiple deployments exist, they are executed in the order in which they are listed on the Deployments subtab. This sequence typically corresponds with the order in which the deployments were created.

To create and edit a script deployment record, you must use a role with Edit level SuiteScript permissions. Make sure you set the correct permissions and features. For more information, see the help topic [Setting Roles and Permissions for SuiteScript](#).

On a script deployment record, you can set up context filtering to determine how and when the script runs. You set up context filtering on the Context Filtering subtab when you create or edit a script deployment record. There are two types of context filtering:

- **Execution context filtering** — Execution contexts provide information about how a script is triggered to process. For example, a script can be triggered in response to an action in the NetSuite application or an action occurring in another context, such as a web services integration. For more information, see the help topic [Execution Contexts](#).
- **Localization context filtering** — Localization contexts provide information about the countries or regions in which a script can run. For example, you can specify that a script should run only on records that are associated with Canada. NetSuite automatically determines the localization context for records and transactions based on their values for country fields such as subsidiary and tax nexus. For more information, see the help topic [Record Localization Context](#).

For more information about setting up context filtering, see the help topic [Using the Context Filtering Subtab](#).

For more information about script deployment, see the following help topics:

- [Methods of Deploying a Script](#)
- [Deploying a Script by using the Deployments Sublist](#)
- [Deploying a Script by using the Script Deployment Record](#)
- [Updating a Script Deployment](#)
- [Managing Web Store Performance Impact](#)

Methods of Deploying a Script

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

To run an entry point script in your NetSuite account, you'll need a script deployment that decides how and when it runs. Here are the ways you can create a script deployment:

- Using the Deployments subtab on the script record (see [Work with the Deployments Subtab/Sublist](#)).
- Using the script deployment record (see [Use the Script Deployment Record](#)).
- Using N/record module methods (see [Use N/record Module Methods](#)).

In most cases, you can also use the above methods to edit a script deployment.

Work with the Deployments Subtab/Sublist

You can deploy a script when you create a script record by using the Deployments subtab on the script record page. This lets you define values for many deployment fields, although not all. For some script types, you can also use the Deployments subtab when you are editing an existing script record. However, for other script types, the Deployments subtab is read-only when you are editing the script record. For

more information about using the Deployment sublist, see [Deploying a Script by using the Deployments Sublist](#).

Use the Script Deployment Record

You can deploy a script by using the script deployment record. Here's why you might want to use this approach:

- The script deployment record lets you access a greater number of fields than the Deployments sublist.
- If you've already created a script record, in some cases it isn't possible to edit the Deployments sublist.

For more information about using the script deployment record, see [Deploying a Script by using the Script Deployment Record](#).

Use N/record Module Methods

You can create a deployment programmatically by using the [record.create\(options\)](#) method in a script. When creating a script deployment record, set the `options.type` parameter to `record.Type.SCRIPT_DEPLOYMENT`. Similarly, if you want to modify a script deployment record programmatically, you can load it by using [record.load\(options\)](#). You can also modify an existing deployment record by using other [N/record Module](#) methods.

For help with the field IDs available on the script deployment record, refer to the [SuiteScript Records Browser](#).

Deploying a Script by using the Deployments Sublist

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

You can use the Deployments sublist when you create a script record that you want to deploy. When you deploy a script using the Deployments sublist, you can define values for some deployment fields but not all. For example, if you want to set up context filtering (execution context filtering or localization context filtering), you must do it in the script deployment record. You can't set context filtering on the Deployments subtab. For more information, see the help topic [Using the Context Filtering Subtab](#).

Depending on the script type, you can also use the Deployments sublist when you are editing an existing script record. However, for some script types, the Deployments sublist is read-only when you are editing the script record.

For information about other ways of deploying a script, see [Methods of Deploying a Script](#).

i Note: Make sure that you have the correct permissions and features for deploying a script. For more information, see [Methods of Deploying a Script](#).

Be aware that the script deployment process varies depending on the script type. The following procedure describes basic steps for using the Deployments sublist. For certain script types, additional steps may be required.

To deploy a script by using the Deployments sublist:

1. Open the script record by going to Customization > Scripting > Scripts.
2. Locate the script you want to create a script deployment for. Click the corresponding **Edit** link.

3. Click the **Deployments** subtab.
4. Add a value to the sublist, as follows:
 - If the **Title** column is displayed, enter a title.
 - If the **Applies to** column is displayed, select the record type where you want to deploy the script. To deploy the script on all record types supported in SuiteScript, click the **Add Multiple** button to select additional record types.
 - Enter a meaningful id in the **ID** column. The script deployment record ID lets you work with the deployment programmatically. Note that the system automatically adds a prefix of `customdeploy` to the value you enter. If you don't specify an ID, a system-generated ID is created for you.
 - Check or clear the **Deployed** box, as appropriate. For most script types, the default value is **Yes**. However, for map/reduce and scheduled scripts, the default is **Not Scheduled**.
 - In the **Status** column, select the appropriate deployment status. Note that the available values vary depending on the script type. See the help topic [Setting Script Deployment Status](#).
 - If the **Event Type** list is displayed, optionally select a value from the list. This value identifies the event type that triggers the script execution. Only one event type can be selected. If you need to have multiple event types trigger your script, you'll need to create additional script deployments. See the help topic [Setting Script Execution Event Type from the UI](#).
 - Optionally, in the **Log Level** field, specify which type of log messages will appear on the **Execution Log** subtab when the script is executed. See the help topic [Setting Script Execution Log Levels](#).
 - If the **Execute as Role** column appears, optionally select whether you want the script to run using Administrator privileges, regardless of the permissions of the currently logged-in user. See the help topic [Executing Scripts Using a Specific Role](#).
 - Click **Add** to add the new line to the sublist.
5. Click **Save**.

The system saves the deployment. If you view the Deployments subtab again, the deployment you created is represented as a link. You can click the link to view the script deployment record that has been created. In some cases, this page lets you configure additional fields. For more information, see [Updating a Script Deployment](#).

Deploying a Script by using the Script Deployment Record

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Deploying a script using a script deployment record lets you access more fields than when you use the Deployments sublist on the script record. If you have already created a script record, in some cases, it isn't possible to edit the Deployments sublist on the script record.

To deploy a script using the script deployment record:

1. When you save your script record, you can immediately create a script deployment record by selecting **Save and Deploy** from the script record **Save** button.
If you want to update a deployment that already exists, go to Customization > Scripting > Script Deployments and select **Edit** next to the deployment you want to update.
2. On the Script Deployment page:
 - For Suitelet, scheduled, and portlet scripts, provide a name for the deployment in the **Title** field.

- For user event and client scripts, select the record the script will run against in the **Applies To** field. In the **Applies To** field, you can also select **All Records** to deploy the script to all records that officially support SuiteScript. (For a list of these records, see the help topic [SuiteScript Supported Records](#).)
- 3. In the **ID** field, optionally enter a custom scriptId for the deployment. If you don't create a scriptId, a system-generated internalId is created for you. NetSuite will add a customdeploy at the beginning of the ID you enter.
- 4. Optionally, clear the **Deployed** box if you don't want to deploy the script. The box is checked by default. A script will only run in NetSuite when the **Deployed** box is selected.
- 5. In the **Status** field, set the script deployment status. See the help topic [Setting Script Deployment Status](#).
- 6. Optionally, in the **Event Type** list, specify an event type for the script execution. See the help topic [Setting Script Execution Event Type from the UI](#).
- 7. Optionally, the **Log Level** field, specify which log messages will appear on the **Execution Log** subtab after the script is executed. See the help topic [Setting Script Execution Log Levels](#).
- 8. In the **Execute as Role** field, select whether you want the script to run using Administrator privileges, regardless of the permissions of the currently logged in user. See the help topic [Executing Scripts Using a Specific Role](#).
- 9. Check the **Available Without Login** box (for Suitelets only) to generate an External URL on save of this deployment. See the help topic [Setting Available Without Login](#).

Note: External URLs use account-specific domains. As a best practice, use `url.resolveScript(options)` to discover the Suitelet URL instead of hard-coding when possible.

- 10. For script types with the **Audience** subtab, specify the roles, groups, or individuals who should be affected by the script. See the help topic [Defining Script Audience](#).

For script types with the **Schedule** subtab, define when and how often the script should run. See [Scheduled Script Execution](#) and [Scheduling a Map/Reduce Script Submission](#).

Note: The time displayed on the Schedule subtab of scheduled and map/reduce script types represents the Set Preference time zone of the last user who edited the script deployment.

- 11. Optionally (for Suitelets only), on the **Links** subtab, create a menu link for the Suitelet if you want to launch your Suitelet from the UI. See [Suitelet Script Deployment Page Links Subtab](#).
- 12. Optionally, for script types with the **Context Filtering** subtab, specify the execution and localization contexts for the script. See the help topic [Using the Context Filtering Subtab](#).
- 13. Optionally, on the **Execution Log** subtab, create custom views for all script logging details. See the help topic [Using the Script Execution Log Subtab](#).
- 14. Click **Save**.

Note that for portlet scripts, you must enable the portlet to display on your dashboard (see the help topic [Custom Portlets](#)).

Updating a Script Deployment

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

You can edit an existing script deployment using either the Deployments sublist on the script record or by editing the script deployment record.

For some script types, you can make changes by editing the script record's Deployments sublist. However, for other script types, the Deployments sublist is read-only. For these script types, you must edit the deployment using the script deployment record. You can do this programmatically using the [N/record Module](#), or you can do this using the script deployment record entry form. This topic describes updating the deployment using the script deployment record entry form.

When you open the script deployment record for editing, you have access to more fields than are available on the Deployments sublist. For instance, the script record Deployments sublist lets you select the record the script deployment applies to, set a custom ID for the script deployment, select whether the script is to be deployed, set the status (Testing or Release), set the event type the deployment applies to, and set the log level for the deployment. On a script deployment record, you can set and select all of the same fields and additionally set the role the script is to be executed as for the deployment. You can also set and view additional script deployment details using the subtabs on the script deployment record:

- **Audience** – When a script is deployed, it runs only in the accounts of the specified audience. This subtab lets you specify the roles that make up the script audience. In SuiteScript 2.x, this subtab is available for the following script types: client, portlet, RESTlet, Suitelet, and user event. See the help topic [Defining Script Audience](#).
- **Scripts** – For a client or user event script, you can view a summary of the scripts that are associated with the script deployment record.
- **Links** – For a Suitelet, you can specify the menu paths that permit users to access the Suitelet. See [Suitelet Script Deployment Page Links Subtab](#).
- **Schedule** – For a map/reduce or scheduled script, you can configure the schedule that determines when the script runs. See [Scheduled Script Deployment Record](#).
- **Context Filtering** – You can set up context filtering to determine how and when a script runs. See the help topic [Using the Context Filtering Subtab](#).
- **Execution Log** – You can use to log data when your script executes.
- **System Notes** – You can view history details of the deployment on the System Notes subtab. The System Notes subtab also lists the contexts that applied each time the script executed. For more information, see the help topic [Execution Contexts](#).



Important: You can't edit a deployment if the script associated with the deployment is currently running in NetSuite. You must wait until the script stops running.

To update a script deployment:

1. Go to Customization > Scripting > Script Deployments.
2. Locate the deployment you want to edit and click the corresponding **Edit** link.
3. Make changes on fields at the top of the Script Deployment page as well as in each subtab, as needed.
4. Click **Save**.

Managing Web Store Performance Impact

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

When deploying a user event script, you can permit or prevent the deployment from being triggered by web store activity by specifying execution contexts on the **Context Filtering** subtab of the script deployment record. Execution contexts let you control whether the deployment is triggered by particular

events, such as UI actions or web store activity. For more information, see the help topic [Execution Contexts](#).

The following execution contexts apply to web store activity:

- Web Application
- Web Store

Scripts can significantly slow web store performance. By preventing script deployments from being triggered by web store activity, you can improve web store response times.

You can set up execution context filtering when you deploy a user event script. You can also set up execution context filtering by editing a script deployment record for a user event script, as described in the following procedure.

To enable or disable web store triggers for a user event script deployment:

1. Open the script deployment record, for a user event script, for editing. For example, go to Customization > Scripting > Script Deployments. Locate the appropriate script deployment record and click the corresponding **Edit** link.
2. On the script deployment record, on the **Context Filtering** subtab, specify the execution contexts for the script. If you select Web Application or Web Store, those contexts are enabled for your script. If you don't select them, they are disabled for your script.
3. Click **Save**.

Note: Another option for optimizing web store performance is the Asynchronous afterSubmit Sales Order Processing feature. When you enable this feature, all afterSubmit user events and workflows triggered by web store checkout run asynchronously. For more information, see the help topic [Commerce Features](#).

Viewing System Notes

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Note: This topic applies to System Notes only. For information about viewing System Notes v2, see the help topic [Viewing System Notes v2](#).

You can view history details of a script record or script deployment record in System Notes.

Note: In previous releases, details about these records was listed on its **History** subtab. However, that subtab is no longer updated. New activity is captured on the record's System Notes subtab.

To view System Notes of a script or script deployment record:

1. Go to Customization > Scripting > Scripts to find a script record or go to Customization > Scripting > Script Deployments to find a script deployment record.
2. Locate the record you want to view and click **View**.

3. Click the **System Notes** subtab. The history details are included on this subtab.

System Notes show the following information:

- Date when the change was made
- Who made the change (the SET BY field)
- Context for the change (for example, UI)
- Type of change, for example, Edit
- Field changed
- Old value
- New value
- Role of the user who made the change



Note: If you run a scheduled script through Save & Execute or using the scheduling function, the SET BY field will show "System".

If you run a scheduled script from a Suitelet (not available externally), RESTlet, or User Event script, the SET BY field will show the user who triggered the script. This user will be also shown as the user who created or modified any record created or modified by the scheduled script.

If you run a scheduled script from a Suitelet available externally (Available Without Login), an error will be thrown: You don't have privileges to perform this operation.

For more information about system notes, see the help topic [System Notes Overview](#).

SuiteScript 2.x Form-Level Script Deployments

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Sometimes, you may want to deploy a client script on only one form. To do this, you attach the script to the form. You can attach a client script to a custom entry form, a custom transaction form, or a custom address form.

Only client scripts can be attached to forms (all other types of entry point scripts are deployed at the record level). When you attach a client script to a form, it runs on that form only.

For more information about the difference between form-level and record-level script deployments, see [Record-Level and Form-Level Script Deployments](#). For help with record-level deployment, see [SuiteScript 2.x Record-Level Script Deployments](#).

With both custom entry forms and custom transaction forms, you can also include logic in the script to create a custom action such as a button or a menu item. For custom address forms, you can deploy the script on the form, but you can't configure custom actions.

For more information about deploying a client script at the form level and creating custom actions, see the following help topics:

- [Attaching a Client Script to a Form](#)
- [Configuring a Custom Action](#)

Attaching a Client Script to a Form

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

You can attach a client script to a form after you have made sure that it is a valid script, as described in [SuiteScript 2.x Entry Point Script Validation](#).

After you attach a script to a form, it is deployed on that form and will run whenever the form is used. You can attach a client script to a custom entry form, a custom transaction form, or a custom address form.



Important: You must have at least the Edit level SuiteScript permission to attach a script to a custom form by editing the Custom Code subtab of the form record. Users with the View level SuiteScript permission can see the Custom Code subtab, but they can't edit it.

To attach a client script to a form:

1. Go to the form that you want to attach the script to. For example, go to Customization > Forms > Entry Forms.
2. Locate the appropriate form and click the corresponding **Edit** link.
3. Go to the **Custom Code** subtab.
4. In the **Script File** list, select the SuiteScript 2.x script that you want to attach. If the script file you want to use hasn't yet been uploaded to the File Cabinet, you can upload it from this page. To upload a file, point to the area at the right of the list to display a Plus icon. Click this icon to display a popup window. You can use this popup window to upload a file from your local environment.

After you select the file, it is displayed in the **Script File** field. The **SuiteScript API Version** field gets updated based on your selected file. The page also updates to include a list of all possible client script entry point functions. Check marks are displayed next to the functions that are included in your script. For example, this script uses includes only the Page Init entry point function:

The screenshot shows the 'Custom Code' subtab of a form record. At the top, there are tabs for 'Subtabs', 'Field Groups', 'Fields', 'Actions', 'Sublists', 'QuickView', and 'Custom Code'. Below the tabs, the 'Script File' dropdown is set to 'validate_order_build.js'. To the right of the dropdown are a plus icon and a document icon. Below the dropdown, the 'SuiteScript API Version' is '2.0'. A list of entry point functions follows, each with a checkbox:

- ☐ Page Init Function
- ☒ Save Record Function
- ☐ Validate Field Function
- ☐ Field Changed Function
- ☐ Post Sourcing Function
- ☐ Line Init Function
- ☒ Validate Line Function
- ☐ Validate Insert Function
- ☐ Validate Delete Function
- ☐ Recalc Function

The boxes on the **Custom Code** subtab can't be edited directly. Instead, you edit the script to add or remove functions as appropriate and the boxes automatically reflect the newly added or

removed functions when you save the changes to your script file (and upload it to the File Cabinet as needed).

5. Click **Save**.

Configuring a Custom Action

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

In some cases, you may want to configure a custom action for a custom form. A custom action can be either a custom button or a custom menu item. A custom button appears at the top of the page. A custom menu item appears as an option when the user points to the Actions label:

The screenshot shows the 'Sales Order' form in NetSuite. At the top, there are buttons for 'Save', 'Cancel', and 'Auto Fill'. To the right of these buttons is an 'Actions' button. A dropdown menu is open from the 'Actions' button, showing three options: 'Send for Auditing' (highlighted with a red box), 'Memorize', and 'Go To Register'. Below the buttons, there is a section for 'Approval Status' with a dropdown menu and a checkbox for 'Accounting Approval'. Below that is a section for 'Primary Information' with a 'Custom Form' dropdown menu (set to 'Custom Sales Order'), an 'Order #' text field (containing 'SORD100599'), a 'PO #' text field, and a 'Sales Channel' dropdown menu.

You configure custom action elements by using logic contained in the client script attached to your form.

Note: Custom actions can be configured for custom entry and custom transaction forms only. A custom action can't be configured for a custom address form.

To configure a custom action on a custom form:

1. Open the custom form, if not already open. To open a custom form, go to Customization > Forms > Transaction Forms. Locate the appropriate form and click the corresponding **Edit** link.
2. Go to the **Actions** subtab.
3. Go to the **Custom Actions** subtab.
4. In the **Label** column, enter a label for your button or menu item.
5. In the **Function** column, enter the name of the appropriate entry point function from your client script.
6. In the **Display as** column, select **Button** or **Menu** as appropriate.
7. Click **Save**.

Note: For more information about custom actions and standard actions, see the help topic [Configuring Buttons and Actions](#).

SuiteScript 2.x Custom Modules

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

With SuiteScript 2.x, you can create custom modules—including third-party, AMD, and non-AMD modules. This keeps your code modular and easy to read or update, and lets you build or use extra API functionality.

You can build a custom module to:

- Group reusable functions in a common library file. Use custom modules to organize helper functions or custom APIs into reusable sections—then load them in your entry point script.
- Add custom modules to SuiteApps and share them with third parties.
- Import a third-party API.
- Separate utilities from business logic.

Note: Custom modules aren't supported in bundle installation scripts.

To see how a custom module is built, check out [SuiteScript 2.x Anatomy of a Custom Module Script](#). You can also follow the [SuiteScript 2.x Custom Module Tutorial](#).

While building your custom module, these topics might help:

- [Module Dependency Paths](#)
- [Naming a Custom Module](#)
- [Custom Module Examples](#)
- [Troubleshooting Errors](#)
- [Custom Modules Frequently Asked Questions](#)

Custom Module Prerequisites

Before you create or use a custom module, get familiar with these topics:

- [SuiteScript 2.x Script Basics](#)
- [SuiteScript 2.x Entry Point Script Validation](#)
- [SuiteScript 2.x Global Objects](#) and [Module Dependency Paths](#)
- [Controlling Access to Scripts and Custom Modules](#)

Creating a Custom Module

To create your custom module, you'll need to use the [define Object](#). Always use define when making custom modules, entry point scripts, or referencing script files with a relative path.

Every custom module script has to return an object when its entry point runs. This returned object could be something simple, like a string—but usually it's a function. As with regular entry point scripts, this function gets a context object as its argument.

You can add regular and custom JSDoc tags to your module scripts. Use `@NApiVersion` and `@NModuleScope` to show the API version and module scope. For non-entry modules, `@NApiVersion` shows which SuiteScript version you're using—and prevents problems if a referencing script is on a different version. `@NModuleScope` controls whether other scripts and accounts can access your module. To learn more, see [Controlling Access to Scripts and Custom Modules](#).

For info on JSDoc tags, see [SuiteScript 2.x JSDoc Validation](#).

You can use JSDoc (and your own tags) to make docs for scripts, custom modules, and SuiteApps. Read more on JSDoc from the official site: <https://jsdoc.app/>.

For examples, see [Custom Module Examples](#).

Using Your Custom Module

To use your custom module, you'll need another script to reference it. Make a separate entry point script to load and trigger your custom module (using an absolute path with the [require Function](#)). For more on paths, see [Module Dependency Paths](#).

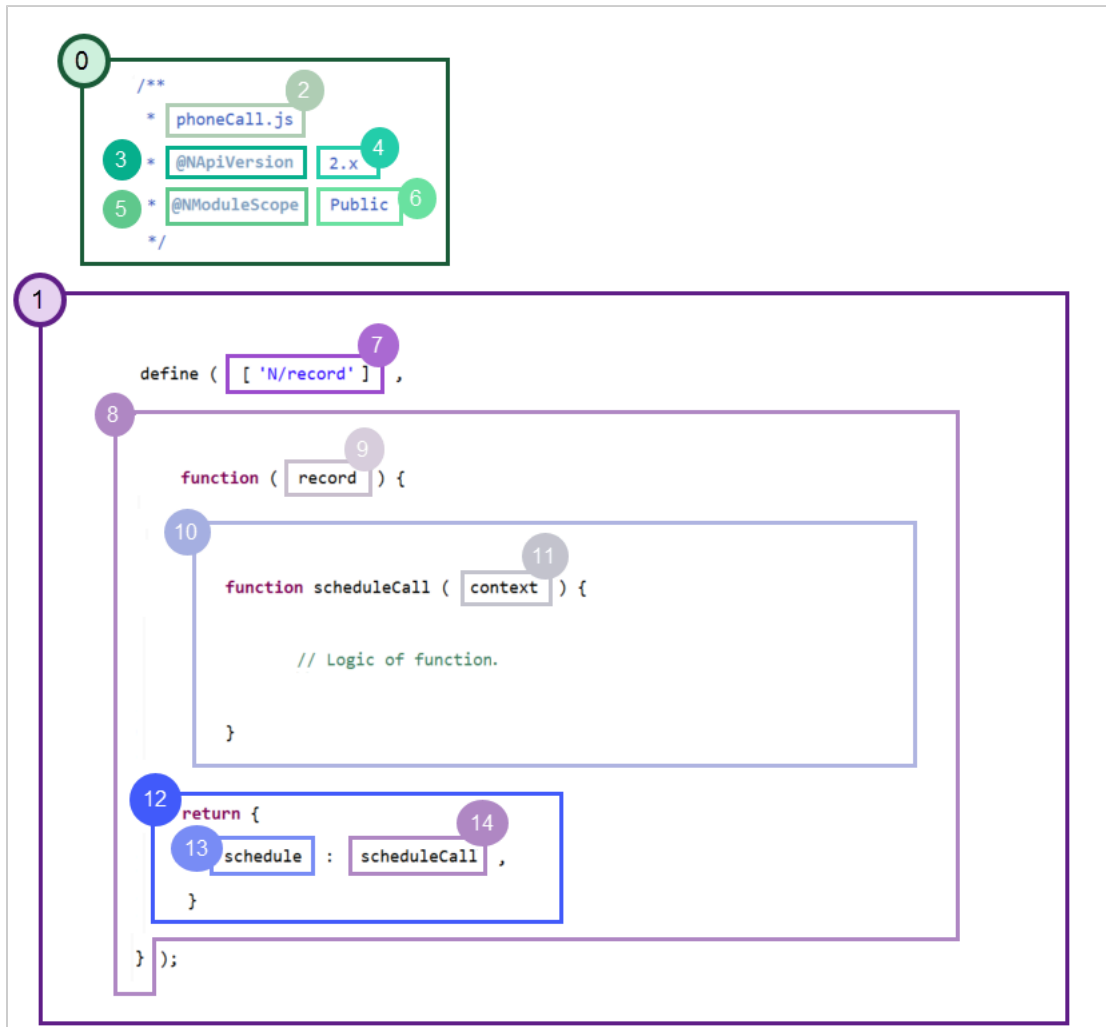
After you've made your custom module and reference script, upload them to your NetSuite File Cabinet (Documents > Files > File Cabinet > SuiteScripts). For any entry point script referencing your module, create a script record and deployment record. See [SuiteScript 2.x Entry Point Script Creation and Deployment](#) for more.

To learn how module naming works, see [Naming a Custom Module](#).

SuiteScript 2.x Anatomy of a Custom Module Script

📘 Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

All SuiteScript 2.x custom module scripts have to follow the same basic structure. The diagram below shows that structure. For an explanation of the numbered components of the script, refer to the table that follows the diagram.



General Area	Callout	Description
0 — JSDoc tags and other annotations	2	The title of the file that holds this script. It isn't required, but it can be useful. Any script that loads this module has to use this name.
	3 and 4	The <code>@NApiVersion</code> tag and its value. This tag isn't required in a custom module script, but adding it can help avoid compatibility issues with future versions of SuiteScript.
	5 and 6	The <code>@NModuleScope</code> tag and its value. This tag is optional. You can use it to limit other scripts' access to this module. For more information about this tag, see Controlling Access to Scripts and Custom Modules .
1 — define function	7	The <code>define</code> function's first argument, which is a list of modules required by the script. This script uses only one: the N/record Module , which lets the script interact with records.
	8	The <code>define</code> function's second argument, which is a callback function.
	9	The arguments of the callback function. Because this script loads only one module, the callback function takes only one argument. This represents the N/record Module and you can use it anywhere in the function to access its APIs. You can name it whatever you want, but it's best to pick something similar to the module name.

General Area	Callout	Description
	10	The script's entry point function. As you can see in the return statement (Callout 12), this function connects to the entry point named schedule.
	11	The context object that's available when the schedule entry point is called. Its property values are set by the script that calls this custom module. For an example, see SuiteScript 2.x Custom Module Tutorial .
	12	The callback function's return statement.
	13	The custom module's entry point: every custom module script needs at least one. Entry point scripts use a standard entry point from @NScriptType JSDoc tag, but for custom modules, you can use any name you want in the return statement.
	14	The function that's returned when the schedule entry point is called. For each entry point, your custom module needs to return an object, usually a function.

Note: For details on the standard structure for an entry point script, see [SuiteScript 2.x Anatomy of a Script](#).

SuiteScript 2.x Custom Module Tutorial

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

A custom module script holds logic you can use in other scripts. If you reuse logic in several scripts, store it in a custom module instead of copying it—this is a lot more efficient.

This tutorial walks you through creating a custom module script and updating an entry point script to use that module.

This tutorial includes and overview and several tasks:

- [Sample Script Overview](#)
- [Deploy the Prerequisite Script](#)
- [Create the Custom Module Script File](#)
- [Review the Script \(Optional\)](#)
- [Upload the Custom Module Script File to NetSuite](#)
- [Modify the Entry Point Script](#)
- [Upload the Revised User Event Script](#)
- [Test the Script](#)

Sample Script Overview

The custom module script in this tutorial automatically creates a phone call record. After you create this module and upload it to your NetSuite File Cabinet, you can call this module from an entry point script.

When you've finished updating the entry point script (as described in this tutorial), the custom module runs each time a new employee record is created. If the new employee record has values for Phone and Supervisor, the module schedules a call between the supervisor and the new employee.

Deploy the Prerequisite Script

If you have not deployed the user event script described in [SuiteScript 2.x User Event Script Tutorial](#), deploy it now.

Create the Custom Module Script File

Copy and paste the following code into the text editor of your choice. Save the file and name it `phoneCall.js`.

```

1  /**
2   * phoneCall.js
3   * @NApiVersion 2.x
4   * @NModuleScope Public
5   */
6
7
8  // This script create a record, so it loads the N/record module.
9
10 define(['N/record'], ,
11
12     // The next line marks the beginning of the callback
13     // function. The 'record' argument is an object that
14     // represents the record module.
15
16     function (record) {
17
18         // The next line marks the beginning of the entry point
19         // function.
20
21         function scheduleCall (context) {
22             var newPhoneCall = record.create({
23                 type: record.Type.PHONE_CALL,
24                 isDynamic: true
25             });
26
27             newPhoneCall.setValue({
28                 fieldId: 'title',
29                 value: context.phoneCallTitle
30             });
31
32             newPhoneCall.setValue({
33                 fieldId: 'assigned',
34                 value: context.phoneCallOwner
35             });
36
37             newPhoneCall.setText({
38                 fieldId: 'phone',
39                 text: context.phoneNumber
40             });
41
42             try {
43                 var newPhoneCallId = newPhoneCall.save();
44
45                 log.debug({
46                     title: 'Phone call record created successfully',
47                     details: 'New phone call record ID: ' + newPhoneCallId
48                 });
49
50             } catch (e) {
51                 log.error({
52                     title: e.name,
53                     details: e.message
54                 });
55             }
56         }

```

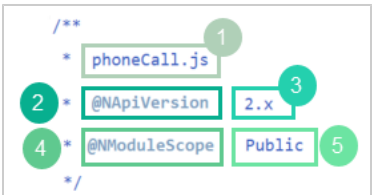
```
57
58 // Add the return statement that identifies the entry point function.
59 return {
60     schedule: scheduleCall,
61 }
62 };
```

Review the Script (Optional)

If you want to learn more about how this script is structured, check out the next sections. Note that these images don't show the entire script. For more details, refer to the comments in [Create the Custom Module Script File](#).

JSDoc Tags

A custom module script isn't required to have JSDoc tags, but you should use them. The following illustration shows this sample's JSDoc block.



Callout	Description
1	The title of the file that holds this script. You don't have to list this annotation, but it can be useful. Any script that loads this module has to use the file's name.
2 and 3	The @NApiVersion tag and its value. You don't have to include this tag in a custom module script, but you might want to so you avoid compatibility issues with future versions of SuiteScript. Valid values for this tag are 2.0, 2.x, and 2.X.
4 and 5	The @NModuleScope tag and its value. This tag is optional—you might want to use it to limit what other scripts can access in this module. For more information about this tag, see Controlling Access to Scripts and Custom Modules .

Entry Point Function

Every custom module script has to define an object that's returned when the script's entry point is called. This object could be a static value, like a string. However, it's much more common for custom module scripts to return a function, like this script does. Because of the way the [Return Statement](#) is set up, the entry point function shown in the following diagram is invoked when the script's schedule entry point is invoked.

Like an entry point function in a standard script, this function takes a context object as its argument.

```
function scheduleCall ( context ) {  
  
    var newPhoneCall = record.create({  
        type: record.Type.PHONE_CALL,  
        isDynamic: true  
    });  
  
    newPhoneCall.setValue({  
        fieldId: 'title',  
        value: context.phoneCallTitle  
    });  
  
    newPhoneCall.setValue({  
        fieldId: 'assigned',  
        value: context.phoneCallOwner  
    });  
  
    newPhoneCall.setText({  
        fieldId: 'phone',  
        text: context.phoneNumber  
    });  
  
    try {  
        var newPhoneCallId = newPhoneCall.save();  
  
        log.debug({  
            title: 'Phone call record created successfully',  
            details: 'New Phone call record ID: ' + newPhoneCallId  
        });  
    } catch (e) {  
        log.error({  
            title: e.name,  
            details: e.message  
        });  
    }  
}
```

Callout	Description
1	The context object that's available when the schedule entry point is called. The property values for this object are set in the script that calls this custom module.
2	This statement uses the record.create(options) method to start making a phone call record.
3	These statements set fields on the phone call record. They use the context object's properties, along with the Record.setValue(options) and Record.setText(options) methods. The values of the context object — phoneCallTitle, phoneCallOwner, and phoneCallNumber — must be defined in the script that calls the custom module.
4	This try/catch block tries to save the new phone call record with Record.save(options) method. If saving fails, the block catches and logs the error that caused the issue.

Return Statement

Like an entry point script, the callback function in a custom module script needs a return statement. The return statement must include at least one entry point.

```
return {  
    1 schedule : 2 scheduleCall,  
}
```

Callout	Description
1	The custom module's entry point. As with an entry point script, every custom module script must use at least one entry point. The difference is an entry point script has to use a standard entry point that matches the script type. With a custom module, you create the entry point, so you can use the return statement to name it however you like.
2	A reference to an object. This example references a function. This structure is probably the most common. But the entry point could also reference another object, static value. Whatever you reference, it has to be defined in the same script file.

Upload the Custom Module Script File to NetSuite

After you create your custom module script file, upload it to your NetSuite File Cabinet.

To upload the script file:

1. In the NetSuite UI, go to Documents > File > SuiteScripts.
2. In the left pane, pick the SuiteScripts folder and click **Add File**.
3. Follow the prompts to find the phoneCall.js file in your local environment and upload it to the SuiteScripts folder.

Even after you upload the file, you can edit it in the File Cabinet if you need to. For details, see the help topic [Editing Files in the File Cabinet](#).

Modify the Entry Point Script

After you upload a custom module script file, you don't need to take any other actions for it to be available. Unlike entry point scripts, you do not have to create a script record or script deployment record for it. However, to use the custom module script, it needs to be referenced by a script that's already been deployed.

In the next steps, you'll update an entry point script to use the phoneCall.js module, as well as the user event script from the [SuiteScript 2.x User Event Script Tutorial](#). To update the file, you can use either of the following approaches:

- If you want to copy and paste the updated script directly from this help topic, skip ahead to [Copy the Full Script](#).
- If you want to read about how to make the needed edits yourself, refer to [Update the Script](#).

Update the Script

This procedure tells you how to update createTask.js so that it uses the phoneCall.js custom module file. For more information about how dependencies should be defined in the following procedure, see [Module Dependency Paths](#).

To update the script:

1. Open the createTask.js file for editing.
2. Add /phoneCall.js to the first argument of the define function to update your dependencies.

```
define ( ['N/record', 'N/ui/serverWidget', './phoneCall' ] ,
```

3. Add another dependency to the callback function's first argument. This object represents the phone call module. You can use it to access the phone call module's API.

```
function(record, serverWidget, phone) {
```

4. In the script's afterSubmit function, add logic to get the employee's phone number.

```
function myAfterSubmit(context) {

    if (context.type !== context.UserEventType.CREATE)
        return;

    var newEmployeeRecord = context.newRecord;

    var newEmployeeFirstName = newEmployeeRecord.getValue ({
        fieldId: 'firstname'
    });

    var newEmployeeLastName = newEmployeeRecord.getValue ({
        fieldId: 'lastname'
    });

    var newEmployeeSupervisor = newEmployeeRecord.getValue ({
        fieldId: 'supervisor'
    });

    var newEmployeePhoneNumber = newEmployeeRecord.getValue({
        fieldId: 'phone'
    });
}
```

5. Create a block of code that does the following:
 - Add a conditional test to check if the employee record has values for both the phone number and supervisor fields (Callout 1).
 - Create an object called phoneData that sets all the fields needed for the phone call module's schedule method (Callout 2).
 - Call the schedule method, passing the phoneCall object as its argument (Callout 3).

```
1 if (newEmployeeSupervisor && newEmployeePhoneNumber) {

    2 var phoneData =
        {
            3 phoneNumber : newEmployeePhoneNumber, 4
            phoneCallOwner : newEmployeeSupervisor,
            phoneCallTitle : 'Welcome ' + newEmployeeFirstName +
                           ' ' + newEmployeeLastName
        };

    3 phone.schedule(phoneData);
}
```

6. Upload the revised script file to your NetSuite File Cabinet, overwriting the prior version.

Copy the Full Script

Here's the fully updated user event script. If you haven't already created the script file using the steps described in [Create the Client SuiteScript Script File Step by Step](#), copy and paste the following code into the text editor of your choice. Save the file and name it createTask.js.

```

1  /**
2   *
3   * @NApiVersion 2.x
4   * @NScriptType UserEventScript
5   */
6
7  define(['N/record', 'N/ui/serverWidget', './phoneCall'],
8    function(record, serverWidget, phone) {
9
10     // In the beforeLoad function, disable the
11     // Notes field on the record.
12
13     function myBeforeLoad (context) {
14       if (context.type !== context.UserEventType.CREATE)
15         return;
16       var form = context.form;
17
18       var notesField = form.getField({
19         id: 'comments'
20       });
21
22       notesField.updateDisplayType({
23         displayType: serverWidget.FieldDisplayType.DISABLED
24       });
25     }
26
27     // In the beforeSubmit function, add a message to the Notes field.
28
29     function myBeforeSubmit(context) {
30       if (context.type !== context.UserEventType.CREATE)
31         return;
32       var newEmployeeRecord = context.newRecord;
33       newEmployeeRecord.setValue('comments', 'Orientation date TBD.');
```

```

62     var newEmployeePhoneNumber = newEmployeeRecord.getValue({
63         fieldId: 'phone'
64     });
65
66     // If the user entered a value for the supervisor field,
67     // create a task record for the supervisor.
68
69     if (newEmployeeSupervisor) {
70         var newTask = record.create({
71             type: record.Type.TASK,
72             isDynamic: true
73         });
74
75         newTask.setValue({
76             fieldId: 'title',
77             value: 'Schedule orientation session for ' +
78                 newEmployeeFirstName + ' ' + newEmployeeLastName
79         });
80
81         newTask.setValue({
82             fieldId: 'assigned',
83             value: newEmployeeSupervisor
84         });
85
86         try {
87             var newTaskId = newTask.save();
88             log.debug({
89                 title: 'Task record created successfully',
90                 details: 'New task record ID: ' + newTaskId
91             });
92
93         } catch (e) {
94             log.error({
95                 title: e.name,
96                 details: e.message
97             });
98         }
99     }
100
101     // If the user entered values for both the supervisor and
102     // phone number fields, use the phoneCall module to schedule a phone call.
103
104     if (newEmployeeSupervisor && newEmployeePhoneNumber) {
105         var phoneData =
106             {
107                 phoneNumber: newEmployeePhoneNumber,
108                 phoneCallOwner: newEmployeeSupervisor,
109                 phoneCallTitle: 'Welcome ' + newEmployeeFirstName +
110                     ' ' + newEmployeeLastName
111             };
112         phone.schedule(phoneData);
113     }
114 }
115
116 return {
117     beforeLoad: myBeforeLoad,
118     beforeSubmit: myBeforeSubmit,
119     afterSubmit: myAfterSubmit
120 };
121 };
122 });

```

Upload the Revised User Event Script

After you update the createTask.js file, upload it to your NetSuite File Cabinet.

To upload the script file:

1. In the NetSuite UI, go to Documents > File > SuiteScripts.

2. In the left pane, pick the SuiteScripts folder and click **Add File**.
3. Follow the prompts to locate the createTask.js file in your local environment and upload it to the SuiteScripts folder.
4. If the system says a file with that name already exists, click OK to overwrite it.

Test the Script

Now that both files are uploaded, make sure the custom module works as expected.

To test the script:

1. Start creating a new employee record by going to Lists > Employees > Employees > New.
2. In the new employee form, check if the Notes field is disabled. If the beforeSubmit entry point works, the field will be gray and you can't edit it.
3. Enter values for the required fields. These fields might vary depending on your account's features and customizations. At minimum, they include:
 - **Name** — Enter Susan as the first name and Johnson as the last name.
 - **Subsidiary** — (OneWorld only) Choose a value from the dropdown list.
4. To make sure that the custom module logic is used, enter values in the **Supervisor** and **Phone** fields.
5. Click **Save**. You'll see a success message, and the system will show the new record in View mode.
6. Check that the phone call record was created successfully:
 1. Select Activities > Scheduling > Phone Calls.
 2. Check the filtering options to make sure phone calls for all employees are shown
 3. Make sure a phone call was scheduled to welcome the new employee.

Phone Calls

View

Phone Call Default

Customize View

New Phone Call

Filters

Edit

Edit | View

Internal ID

Subject

Phone Call Date

Phone Number

Priority

Status

1839

Welcome Susan Johnson

11.9.2017

(650) 555-1212

Medium

Scheduled

Module Dependency Paths

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

To specify module paths or names, use the [define Object](#) or [require Function](#).

SuiteScript supports the following path types:

Path Type	Description
Absolute Paths	Use an absolute path to start from the root folder in the File Cabinet.

Path Type	Description
	Absolute paths start with a forward slash (/), like <code>"/SuiteScripts/MyApp/Util"</code> .
Relative Paths	Relative paths start from the dependent module's location, so you have more flexibility when moving nested folders around in the File Cabinet. Relative paths start with a period (".") or two periods (".."). For example: if your module is in <code>"/SuiteScripts/MyApp"</code>, the relative path to a sibling file is <code>"/Util"</code>, which is the same as <code>"/SuiteScripts/MyApp/Util"</code>.  Note: Don't use relative paths in global contexts.
Bundle Virtual Paths	A bundle virtual path points to a File Cabinet location using a valid bundle ID. Bundle virtual paths start with a forward slash followed by a period (/.). For example: <code>/.bundle/<bundle id>/SS2_CustomModuleTest.js/</code>

You can also reference modules using custom module names—a global identifier that lets you refer to a module by name instead of by path. For more, see [Naming a Custom Module](#).

Absolute Paths

 **Note:** Don't use file extensions in the path.

Start absolute paths with a "/" to point to the top-level File Cabinet directory. Remember, files follow their parent folders' restrictions, so make sure referenced library files are available to the right users.

In the following example, the Module Loader expects the custom module files `lib1.js` and `lib2.js` to be located in the `SuiteScripts` top-level directory in the File Cabinet.

```

1 // myModule.js
2 define(['./SuiteScripts/lib1', './SuiteScripts/lib2'], // myModule has a dependency on modules lib1 and lib2
3     ...
4 );
```

Relative Paths

 **Note:** Don't use file extensions in the path. Make sure library files are available to users who need them.

You can use relative paths to point to modules in subfolders from your current module's location.

In the following example, the Module Loader expects the custom module files `lib1.js` and `lib2.js` to be located in the `lib` subdirectory, the same File Cabinet directory that contains `myModule.js`.

```

1 // myModule.js
2 define(['./lib/lib1', './lib/lib2'], // myModule has a dependency on modules lib1 and lib2
3     ...
4 );
```

Bundle Virtual Paths

Note: Don't use file extensions in the path.

Use a bundle virtual path, starting with a forward slash followed by a period ("/."), to point to a bundle File Cabinet location. You'll need a valid bundle ID.

In the following example, the Module Loader is looking for common libraries in shared bundles.

```
1 // myModule.js
2 require(['./bundle/101'], // myModule has a dependency on modules with the bundle id 100 and 101
3 ...
4 );
```

Sometimes, a virtual path helps if a bundle is moved or its ID changes. When a bundle's deprecated or copied, SuiteScript 2.x checks the chain and looks for the file in several places. If a bundle's replaced with a new version (and new ID), scripts using the old path in the target account switch to the new bundle's ID—the virtual path resolves it automatically.

Note: You can't use the source account bundle ID for a virtual bundle path, because the path depends on a real file path for an existing bundle—which doesn't exist at creation in the source account.

Naming a Custom Module

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

You can set up a custom module name to make re-use easier. Once you configure the name, you can require the module without worrying about its path.

Loading custom modules by name also helps you use third-party libraries and may help avoid naming conflicts.

You'll need to call `define(id, [dependencies,] moduleObject)` and set up a [require Function](#).

To load a custom module by name:

1. Create your custom module file and upload it to the File Cabinet. For example, make a `math.js` file with your code and save it in SuiteScripts/Example.

```
1 ...
2 let myMath = {
3   add: function(num1, num2) {
4     return num1 + num2
5   },
6   subtract: function(num1, num2) {
7     return num1 - num2
8   },
9   multiply: function(num1, num2) {
10    return num1 * num2
11  },
12  divide: function(num1, num2) {
13    return num1 / num2
14  },
15 }
```

```
16 | ...
```

2. To load your module by name, add the module's file name (alias) and its path under the `paths` parameter in a JSON file (like `myconfig.json`).

```
1 | ...
2 | {
3 |   "paths": {
4 |     "math": "/SuiteScripts/Example/math.js"
5 |   }
6 |   "shim": {
7 |     "math": {
8 |       "exports": "myMath"
9 |     }
10 |   }
11 | }
12 | ...
```

3. To load the custom module in your SuiteScript 2.x script, include the `@NAmdConfig` JSDoc tag with your `.json` file name and pass the custom module name to `define Object`.

```
1 | /**
2 |  * @NApiVersion 2.x
3 |  * @NScriptType UserEventScript
4 |  * @NAmdConfig ./myconfig.json
5 |  */
6 |
7 | define(['math'], function (math) {
8 |   return {
9 |     beforeLoad: function beforeLoad() {
10 |       log.debug({
11 |         title: 'test',
12 |         details: myMath.add(1,2)
13 |       });
14 |     }
15 |   }
16 | });
```

For more about the `@NAmdConfig` JSDoc tag, see the help topic [require Configuration](#).

Custom Module Examples

❗ Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The examples below show how to use the `define Object` and `require Configuration`

- [Define a custom utility module](#)
- [Import the custom utility module](#)
- [Define a custom module for a SuiteApp](#)
- [Import a third-party JavaScript library](#)

Define a custom utility module

The following file defines a custom module with its own API. For example, you can let multiple scripts use an incremental counter by requiring and importing this module.

To avoid version issues, this script uses the `@NApiVersion` tag.

```

1  /**
2   * counter.js
3   * @NApiVersion 2.1
4   */
5  define(function(){
6     let counter = 0;
7
8     function incrementValue() {
9         counter++;
10    }
11    function getValue() {
12        return counter;
13    }
14
15    return {
16        increment: incrementValue,
17        value: getValue
18    }
19 });

```

Import the custom utility module

This example uses the [define Object](#) to include both a built-in SuiteScript module and a custom module.

It uses the module's file path to include the utility as a dependency—best practice is to skip the .js extension.

```

1  /**
2   * customRecord.js
3   * @NApiVersion 2.x
4   */
5  define(['N/record', './counter'],
6     function(record, counter){
7         function createCustomRec() {
8             record.create();
9             counter.increment();
10        }
11
12        return {
13            createCustomRecord: createCustomRec
14        }
15    });

```

Define a custom module for a SuiteApp

To make a bundled custom module available to third parties, add the `@NModuleScope` JSDoc tag and set it to `public`.

```

1  /**
2   * @NApiVersion 2.x
3   * @NModuleScope public
4   */

```

Then use [define Object](#) so your module is treated as an AMD module. To use a module bundled inside an external SuiteApp, pass the bundle's file path with its bundle ID to `define()`, like this:

```

1  define(['./bundle/<bundle ID>/<module path>'],
2     function(<module name>){
3         <logic goes here>
4     }
5 );

```

Import a third-party JavaScript library

There are two types of imports for third-party libraries:

- [Import a third-party library](#)
- [Add a non-AMD library](#)

Import a third-party library

Some third-party libraries are AMD compatible, so you can set a require config with the module's path.

For example, create a JsLibraryConfig.json file with this code and save it in the File Cabinet:

```

1  ...
2  {
3    "paths": {
4      "coolthing": "/SuiteScripts/myFavoriteJsLibrary"
5    }
6  }
7  ...

```

Next, use the @NAmdConfig JSDoc tag to set the relative path from your script to the config file:

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType UserEventScript
4   * @NAmdConfig ./JsLibraryConfig.json
5   */
6
7  define(['coolthing'],
8    function (coolthing) {
9      return {
10         beforeLoad: function beforeLoad(ctx) {
11           coolthing.times(2, function () {
12             log.debug({
13               title: 'log',
14               details: 'log'
15             });
16           });
17         }
18       };
19     });

```

You can do something similar to use a specific version of jQuery:

First, add jQueryConfig.json to the File Cabinet and set the path:

```

1  ...
2  {
3    "paths": {
4      "jquery": "/SuiteScripts/myjQueryLib"
5    }
6  }
7  ...

```

Then in your script, reference the path to the JSON config in the @NAmdConfig JSDoc tag. For example: * @NAmdConfig /SuiteScripts/myjQueryLib.json.

```

1  ...
2  * @NAmdConfig ./jQueryConfig.json
3  */

```

```

4
5 define(['jquery'], function(jquery) {
6 });
7 ...

```



Note: If you want to use jQuery with a Suitelet, import it using an on-demand client script attached with `Form.clientScriptFileId` or `Form.clientScriptModulePath`.



Important: SuiteScript doesn't support direct DOM access to the NetSuite UI—only use SuiteScript APIs.

Add a non-AMD library

In this example, upload `math.js` to `/SuiteScripts/ShimExample`—it's not AMD compatible.

```

1 var myMath = {
2   add: function(num1, num2) {
3     return num1 + num2;
4   },
5   subtract: function(num1, num2) {
6     return num1 - num2;
7   },
8   multiply: function(num1, num2) {
9     return num1 * num2;
10  },
11  divide: function(num1, num2) {
12    return num1 / num2;
13  },
14 }

```

If the library isn't AMD compatible, use a require config with JSON parameters. Here, you add `shim` and `paths` parameters in `MathConfig.json`, also saved in `ShimExample`.

```

1 ...
2 {
3   "paths": {
4     "math": "SuiteScripts/ShimExample/math.js"
5   },
6   "shim": {
7     "math": {
8       "exports": "myMath"
9     }
10  }
11 }
12 ...

```

You can then load the module as a dependency in an entry point script (like a user event script) as long as you include a valid `@NAmdConfig` JSDoc tag with the path to the JSON config.

```

1 /**
2  * @NApiVersion 2.x
3  * @NScriptType UserEventScript
4  * @NAmdConfig /SuiteScript/ShimExample/MathConfig.json
5  */
6
7 define(['math'], function (math) {
8   return {
9     beforeLoad: function beforeLoad() {
10       log.debug({
11         title: 'test',
12         details: myMath.add(1,2)
13       });
14     }
15   };
16 }

```

```

14 |     }
15 | }
16 | });

```

Troubleshooting Errors

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Module not found

You'll see this error if you're using the `require` Function. Because `require` doesn't have a global context, relative paths won't work unless you import `require()` as a dependency of `define()`.

If you get a "Module does not exist" error, try switching relative paths to absolute paths.

For more information, see the help topic [require Function](#).

You do not have permission to load this module

Check your module scope settings. For details, see [Controlling Access to Scripts and Custom Modules](#).

Script is behaving inconsistently or erratically

If your script uses a custom module or library, make sure the file is in a File Cabinet location without restrictions. Remember, files and folders inherit parent folder restrictions, which can keep some roles from running the script.

Custom Modules Frequently Asked Questions

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

When should I use `require` versus `define`?

Always use the [define Object](#) for entry point scripts or when creating new modules. Use the [require Function](#) to load and use existing modules.

There's a performance advantage to using the `require` Function. Whenever possible, try to use `require`. The `define` Object imports all dependencies up front, while `require` only loads them as needed.

Here is a summary of when to use or not use the `define` Object and `require` Function:

Object/Function	When to use	Caveat
define Object	- To define entry point scripts - To create new modules - To export modules - To import modules using a relative path	If you use <code>define</code> , you won't get the performance boost of progressive loading.

require Function	■ To import modules using an absolute path (relative path requires context) ■ To step through code in the SuiteScript debugger ■ To support building a test framework ■ For progressive loading of dependencies	If you use require, you might need to import define first to get the right context. For example, to import a custom module by relative path: <pre> 1 define(['require'], function(require){ 2 require(['../ParentFolder/customFile'], function(custom File){ 3 //your code 4 }); 5 }); </pre>
------------------	--	---

Are third-party libraries supported?

Yes—you can define the library as a custom module, set a global identifier to reference it, and set it up as a dependency. For more examples of importing third-party and non-AMD libraries, see [Import a third-party library](#) and [Import a third-party JavaScript library](#).

SuiteScript 2.x Scripting Records and Subrecords

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

For information about using SuiteScript 2.x to work with records and subrecords, see the following topics:

- [SuiteScript 2.x Scripting Records](#)
- [SuiteScript 2.x Scripting Subrecords](#)

SuiteScript 2.x Scripting Records

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

For details on using SuiteScript 2.x to work with records, see the following sections:

- [SuiteScript 2.x Standard and Dynamic Modes](#)
- [SuiteScript 2.x Record Modules](#)

SuiteScript 2.x Standard and Dynamic Modes

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

When you create, copy, load, or transform records in SuiteScript, you can work with records in standard or dynamic mode.

Standard mode:

When a SuiteScript 2.x script creates, copies, loads, or transforms a record in standard mode, the record's body fields and sublist line items are not sourced, calculated, and validated until the record is saved with [Record.save\(options\)](#). You might also see this referred to as "deferred dynamic mode" in the SuiteScript help.

When you work with a record in standard mode, in most cases, you don't need to set values in any particular order. When you submit the record, NetSuite will process the fields and sublist items in the right order, no matter how your script is organized. See [Getting Text in SuiteScript 2.x Record Modes](#).

Dynamic mode:

When a SuiteScript 2.x script creates, copies, loads, or transforms a record in dynamic mode, the record's body fields and sublist line items are sourced, calculated, and validated in real time. Dynamic mode makes the record behave like it would in the NetSuite UI.

Note: In dynamic mode, the Legacy Tax field **Tax Code** isn't automatically transferred when a requisition becomes a purchase order. For more information, see [Tax Code Limitations When Using SuiteScript 2.x](#).

In dynamic mode, the order of setting field values is important. This might feel limiting to some developers. You'll probably need to check the UI often when scripting in dynamic mode. For instance, when creating an invoice in the UI, you wouldn't set the Terms field before the Customer field. That's because setting the Customer field will override the Terms field value. On an invoice, the value of Terms is sourced from the terms specified on the Customer record. This behavior also applies to dynamic scripting. If you don't set field values in the right order in your script, some values might get overridden.

How you can tell if a record is in dynamic mode:

You can determine if a record is in dynamic mode using these two SuiteScript 2.x properties:

- [Record.isDynamic](#) (for server scripts)
- [CurrentRecord.isDynamic](#) (for client scripts)

Record Modes and User Event Scripts

User event scripts that use `newRecord` or `oldRecord` objects always run in standard mode. This is why you might see the `SSS_INVALID_API_USAGE` error in these situations:

- When the user event script executes on a record that is being created, and the script attempts to use [Record.getText\(options\)](#) without first using [Record.setText\(options\)](#) for the same field.
- When the user event script executes on an existing record or on a record being created through copying, and the script uses [Record.setValue\(options\)](#) on a field before using [Record.getText\(options\)](#) for the same field.

Getting Text in SuiteScript 2.x Record Modes

In dynamic mode, you can use [Record.getText\(options\)](#) without limitation, but in standard mode, there are some limits. In standard mode, you can use this method **only** in these situations:

- You can use [Record.getText\(options\)](#) on any field where the script has already used [Record.setText\(options\)](#).
- If you are loading or copying a record, you can use [Record.getText\(options\)](#) on any field except those where the script has already changed the value with [Record.setValue\(options\)](#).

Record Module Method Considerations

Be aware that the [record.create\(options\)](#), [record.copy\(options\)](#), [record.load\(options\)](#), and [record.transform\(options\)](#) methods work in standard mode by default. To use these methods in dynamic mode, set the `.isDynamic` property for each one.

SuiteScript 2.x Record Modules

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

With SuiteScript 2.x, you use the [N/record Module](#) and [N/currentRecord Module](#) to script with records.

In **server scripts**, use the [N/record Module](#). See the following topics for examples of working with records in server scripts:

- [SuiteScript 2.x User Event Script Sample](#)
- [SuiteScript 2.x User Event Script Tutorial](#)
- [SuiteScript 2.x Custom Module Tutorial](#)

In **client scripts**:

- Use [N/currentRecord Module](#) methods to interact with the current record.
- Use the [N/record Module](#) to load and interact with remote records.

See the following topics for examples of working with records in client scripts:

- [SuiteScript Client Script Sample](#)

- [Using the currentRecord Module in Client Scripts](#)
- [Interfacing with Remote Objects in Client Scripts](#)
- [Disable Field on Client PageInit using SuiteScript 2.0](#)

SuiteScript 2.x Scripting Subrecords

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

For details on using SuiteScript 2.x to work with subrecords, see the following topics:

- [About Subrecords](#)
- [Subrecord Scripting in SuiteScript 2.x Compared With 1.0](#)
- [Scripting Subrecords that Occur on Sublist Lines](#)
- [Scripting Subrecords that Occur in Body Fields](#)

About Subrecords

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

When working with records in SuiteScript, you'll often deal with various field types. You may see fields with a data type of summary. The only way to fill these fields is by saving a subrecord to them. To work with these fields, you need to know how to script with subrecords.

Saving subrecords to summary fields can be more complicated than setting values for other field types. However, subrecords are similar to records. So if you know how to work with records, you already know a great deal about working with subrecords. This topic summarizes the similarities and differences.

i Note: Note that subrecords don't work with the Advanced Employee Permissions feature. For more information, see the help topic [Before Enabling the Advanced Employee Permissions Feature](#).

Subrecords

Subrecords are a way to store data in NetSuite.

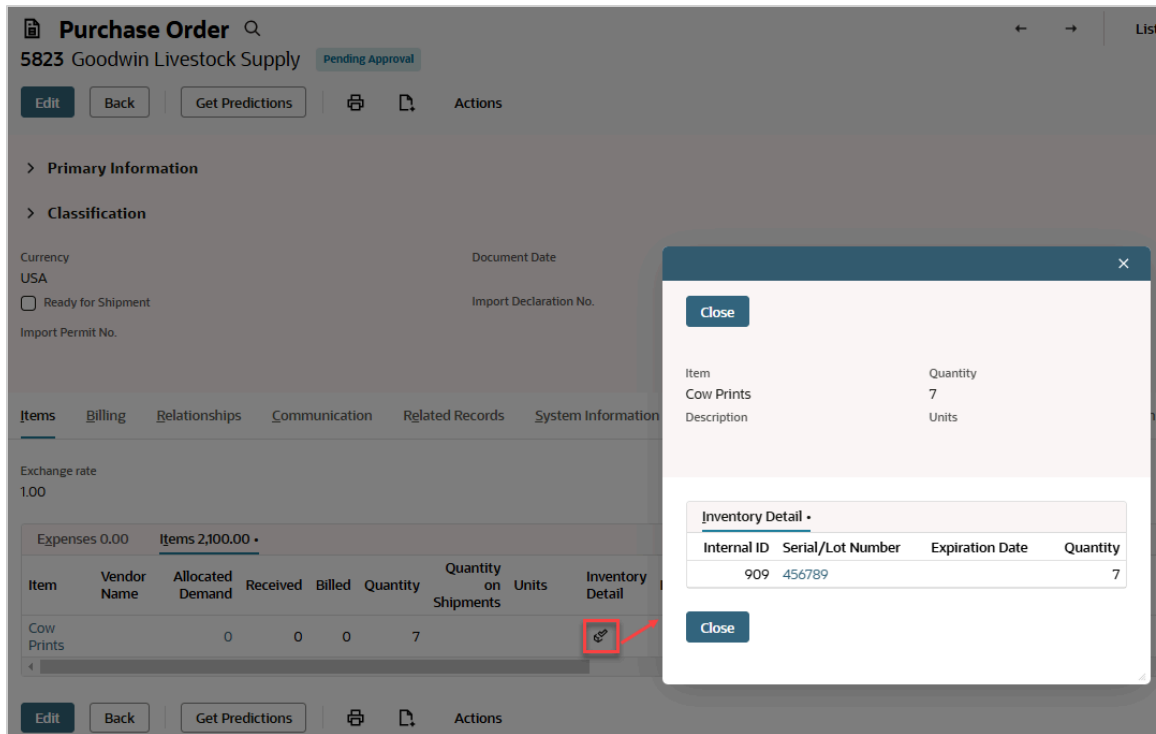
Like records, subrecords are classified by type. Common subrecord types include address, inventory detail, and order schedule.

Each subrecord type has its own purpose and set of fields. For example:

- An address subrecord holds an address with fields like city, state, and zip.
- An order schedule subrecord is for purchase schedules, with fields like startdate and enddate.
- An inventory detail subrecord stores data like serial numbers for inventory items, with a sublist to hold this data.

At the same time, subrecords differ from records in some ways. For example, while records can exist independently, a subrecord exists solely to hold information about a specific record. You can only work with subrecords in the context of their parent record.

In the UI, you usually open a subrecord by clicking an icon on the parent record. This opens the subrecord form in a new window. For example, the following illustration shows, at left, a purchase order with one line in its Items sublist. If you click the icon, it opens a window showing the inventory detail subrecord.



To access this same subrecord from a script, start by loading the purchase order. Then you would use the `Record.getSublistSubrecord(options)` method to open the subrecord. For example:

```

1  ...
2  // Load the purchase order.
3  var rec = record.load({
4      type: record.Type.PURCHASE_ORDER,
5      id: 7,
6      isDynamic: false
7  });
8
9  // Retrieve the subrecord. For sublistId, use the ID of the relevant sublist on
10 // the purchase order record type. For fieldId, use the ID of the summary field
11 // on the sublist that holds the subrecord.
12 var subrec = rec.getSublistSubrecord({
13     sublistId: 'item',
14     line: 0,
15     fieldId: 'inventorydetail'
16 });
17
18 ...

```

For a full script example, see [Creating an Inventory Detail Subrecord Example](#).

Subrecord Scripting Overview

Use the following guidelines when scripting with subrecords:

- [When Subrecords Are Read-Only](#)
- [Look Up Details About the Summary Field](#)
- [Look Up the Subrecord's Field and Sublist IDs](#)
- [Use Record Methods to Get and Set Values](#)
- [Do Not Explicitly Save a Subrecord](#)

When Subrecords Are Read-Only

You can use subrecords in both client and server scripts. But subrecords are read-only if you retrieve their parent records in these ways:

- Through the context object provided to a client script, or through [currentRecord.get\(\)](#).
- Through the context object provided to a beforeLoad user event script.

For more details, see [Supported Deployments for Subrecord Scripting](#).

Look Up Details About the Summary Field

To work with a subrecord, you need to know about the summary field that contains it. You'll also need to know if the summary field is on the parent record's body or on a sublist. You need both pieces of information to instantiate the subrecord. If it's a sublist field, you'll also need the sublist ID.

If you are not sure where the field is situated, you can review the record in the UI. You can also check the reference page for the record in the [SuiteScript Records Browser](#). On the reference page for each record type, the **Fields** table lists all of the record type's body fields. The tables listed under the heading **Sublists** show sublist fields. For more details, see [Finding Details About Parent Record Types](#).

Look Up the Subrecord's Field and Sublist IDs

Like records, all subrecords have required and optional fields. You need the field IDs to set values for these fields. For sublist fields, you'll also need the sublist ID. You can find this info in the [SuiteScript Records Browser](#). The browser lists subrecord types alongside record types. For each subrecord type, a reference page includes the IDs for all of the elements on the subrecord, including a sublist, if one exists, and all of the subrecord's fields. For more details, see [Finding Details About Subrecord Types](#).

Additionally, if you have the Show Internal IDs preference enabled, you can use the UI to find the IDs for subrecord body fields. To view a field's ID, click its label. This will open a popup with the ID. For help enabling this preference, see the help topic [Setting the Internal ID Preference](#).

Use Record Methods to Get and Set Values

Scripting with subrecords is similar to scripting with records in many ways.

For example, when your script instantiates a subrecord, the system returns one of the same objects that it uses to represent records. These objects include [record.Record](#) and [currentRecord.CurrentRecord](#).

You can use many of the same methods for subrecords as you do for records. For example:

- In server scripts, use [Record.setValue\(options\)](#) to set values on record or subrecord body fields.
- In a client script, use [CurrentRecord.getValue\(options\)](#) to retrieve the value stored in either a record or subrecord body field.

Some record methods and properties don't work with subrecords. These exceptions are noted in the property descriptions in the [N/record Module](#) and [N/currentRecord Module](#) topics.

Do Not Explicitly Save a Subrecord

After you have created or updated a subrecord, you do not explicitly save it. Rather, after you have set all required fields on the subrecord, you simply save the parent record. Saving the parent record also saves the subrecord.

Creating an Inventory Detail Subrecord Example

Here's an example of how to create a subrecord.

This example creates a purchase order with one inventory item in its sublist. The example also creates an inventory detail subrecord to store a lot number for the item.

Before using this example in your NetSuite account, do the following:

- Make sure the [Advanced Bin / Numbered Inventory Management](#) feature is enabled, at Setup > Company > Enable Features, on the Items & Inventory subtab.
- Verify that you have at least one vendor, one location, and one lot-numbered inventory item defined in your system. Make a note of each record's internal ID.
- Where noted in the script example comments, replace the hardcoded IDs with valid values from your NetSuite account.

This example uses standard mode, but you can also use dynamic mode. For more details about both approaches, see [Using SuiteScript 2.x to Create a Subrecord in a Sublist Field](#).

Note: For help deploying a user event script, see [SuiteScript 2.x Entry Point Script Creation and Deployment](#).

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType UserEventScript
4   */
5
6  define(['N/record'], function(record){
7      function myAfterSubmit(context){
8          // Create the purchase order.
9          var rec = record.create({
10             type: record.Type.PURCHASE_ORDER,
11             isDynamic: false
12         });
13
14         // Set body fields on the purchase order. Replace both of these
15         // hardcoded values with valid values from your NetSuite account.
16         rec.setValue({
17             fieldId: 'entity',
18             value: '2'
19         });
20
21         rec.setValue({
22             fieldId: 'location',
23             value: '2'
24         });
25
26         // Insert a line in the item sublist.
27         rec.insertLine({
28             sublistId: 'item',
29             line: 0
30         });
31
32         // Set the required fields on the line. Replace the hardcoded value
33         // for the item field with a valid value from your NetSuite account.
34         rec.setSublistValue({
35             sublistId: 'item',
36             fieldId: 'item',
37             line: 0,
38             value: '7'
39         });
40
41         rec.setSublistValue({
42             sublistId: 'item',
43             fieldId: 'quantity',
44             line: 0,
45             value: 1
46         });
47
48         // Instantiate the subrecord. To use this method, you must
49         // provide the ID of the sublist, the number of the line you want
50         // to interact with, and the ID of the summary field.

```

```

51
52     var subrec = rec.getSublistSubrecord({
53         sublistId: 'item',
54         line: 0,
55         fieldId: 'inventorydetail'
56     });
57
58     // Insert a line in the subrecord's inventory assignment sublist.
59     subrec.insertLine({
60         sublistId: 'inventoryassignment',
61         line: 0
62     });
63
64     subrec.setSublistValue({
65         sublistId: 'inventoryassignment',
66         fieldId: 'quantity',
67         line: 0,
68         value: 1
69     });
70
71     // Set the lot number for the item. Although this value is
72     // hardcoded, you do not have to change it, because it doesn't
73     // reference a record in your account. For this example,
74     // the value can be any string.
75     subrec.setSublistValue({
76         sublistId: 'inventoryassignment',
77         fieldId: 'receiptinventorynumber',
78         line: 0,
79         value: '01234'
80     });
81
82     // Save the record. Note that the subrecord object does
83     // not have to be explicitly saved.
84
85     try {
86         var recId = rec.save();
87         log.debug({
88             title: 'Record created successfully',
89             details: 'Id: ' + recId
90         });
91
92     } catch (e) {
93         log.error({
94             title: e.name,
95             details: e.message
96         });
97     }
98 }
99
100 return {
101     afterSubmit: myAfterSubmit
102 };
103 });

```

For additional script samples, see [Scripting Subrecords that Occur on Sublist Lines](#) and [Scripting Subrecords that Occur in Body Fields](#).

Supported Deployments for Subrecord Scripting

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

A script can interact with subrecord instances only if it uses a supported deployment.

To understand supported deployments, it's important to understand that every subrecord has a parent record. For more details on this relationship, see [About Subrecords](#).

The following types of deployments are supported:

- [Server Scripts Deployed on Parent Records](#)

- [Client Scripts Deployed on Parent Records \(with Limitations\)](#)
- [Client Scripts Deployed on Custom Address Forms](#)

Server Scripts Deployed on Parent Records

To interact with a subrecord, deploy your server script on the parent record type. Alternatively, you can deploy the script on a different record type — but then use the script to load the parent record. After you've loaded the parent record, you can interact with the subrecord.

Subrecord methods don't work in beforeLoad user event scripts, unless you create or load a parent record and interact with the subrecord through that parent.

You can't deploy a server script directly to a subrecord type.

Client Scripts Deployed on Parent Records (with Limitations)

A client script may not create subrecords on the current record and is limited to read-only access of existing subrecords on the current record. The client script may remove the subrecord from the current record.

You can't deploy a client script directly to a subrecord type. But you can customize an address form, as explained in the next section.

Client scripts can't be deployed directly to subrecord types.

Client Scripts Deployed on Custom Address Forms

You can create custom forms for address subrecords if needed. This process is described in [Customizing Address Forms](#). With a custom address form, you can add a client script using the Custom Code subtab. In these scripts, you can work with the subrecord like you would with a regular record. This process is not covered in this chapter.

Body Field Subrecords and Sublist Subrecords

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

When a subrecord occurs on a record, it is always represented by a single field on the record. In the [SuiteScript Records Browser](#), this field is always listed as a field of type summary.

A field with a subrecord can be either a body field or a sublist field. For example, the subsidiary record has a body field called mainaddress, which stores an address subrecord. By contrast, the employee record permits the creation of multiple addresses, and each address is described in a sublist line. The sublist includes a field with the address subrecord.

Where the summary field is placed affects how you create the subrecord.

For an example of each type of placement, see the following sections:

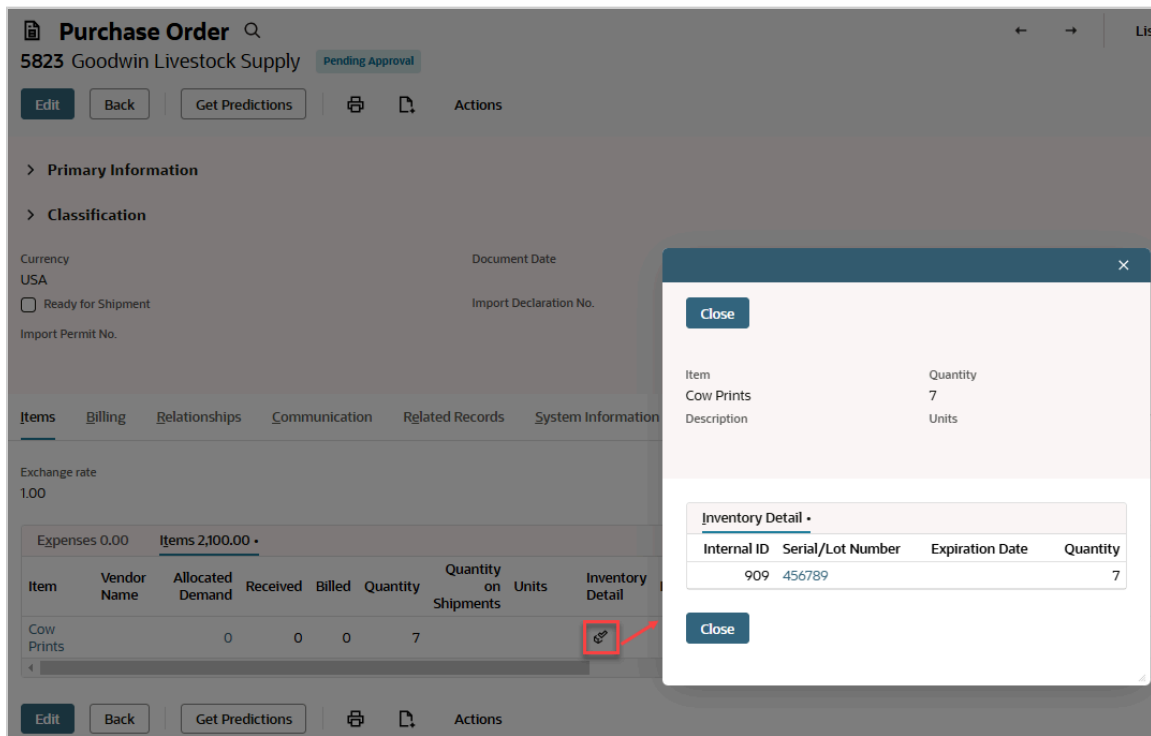
- [Subrecord that Occurs in a Sublist Field Example](#)
- [Subrecord that Occurs in a Body Field Example](#)

Subrecord that Occurs in a Sublist Field Example

Many subrecord types can occur in a sublist field.

For instance, a purchase order's item sublist might have an Inventory Detail column, depending on your account settings. If the item on the line is a serialized or lot-numbered item, you can create a subrecord instance in this column.

In the UI, click the icon in the Inventory Detail column to view and edit the subrecord. This opens a new window showing the subrecord.



Note that other fields on the sublist line aren't part of the subrecord. For example, in the preceding screenshot, the values in the Item, Vendor Name, Received, Billed, and Quantity columns are not part of the subrecord.

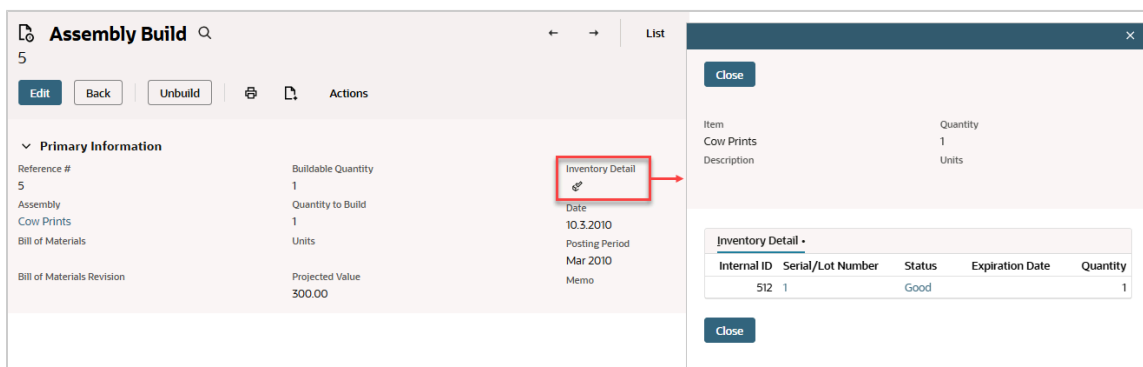
For details on using SuiteScript 2.x to work with subrecords that exist on sublist lines, see [Scripting Subrecords that Occur on Sublist Lines](#).

Subrecord that Occurs in a Body Field Example

A subrecord can also occur in a body field. The same subrecord type can be a sublist field on one record and a body field on another.

For example, depending on the configuration of your account, the assembly build record can include an Inventory Detail body field. If the assembled item is serialized or lot-numbered, you can create an Inventory Detail subrecord in that field.

In the UI, click the icon under the Inventory Detail label to view and edit the subrecord in a new window.



For details on using SuiteScript 2.x to work with subrecords that exist in body fields, see [Scripting Subrecords that Occur in Body Fields](#).

Structure of a Subrecord

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The fields available in a subrecord vary depending on the subrecord's type. Subrecords can have body fields, sublists, or both.

When working with subrecord fields, keep the following in mind:

- After creating a subrecord, you can set values on its body and sublist fields using the same methods as for records.
- Sublists within subrecords usually aren't labeled in the UI. To find the name of a subrecord's sublist, refer to the [SuiteScript Records Browser](#). For details on using the Records Browser to find details on subrecords, see [Finding Subrecord Details in the Records Browser](#).

For examples of subrecords that are structured in different ways, see the following sections:

- [Writable Body Fields Example](#)
- [Writable Sublist Example](#)
- [Writable Body Fields and Sublist Example](#)

Writable Body Fields Example

The address subrecord has several writable body fields, such as city, state, and zip. It has no sublist.

OK Cancel

Country *
United States

Attention
[Text Field]

Addressee
John Smith

Phone
[Text Field]

Address 1
2955 Campus Drive

Address 2
Suite 100

City
San Mateo

State
California

Zip
94403-2511

Address
John Smith
2955 Campus Drive
Suite 100
San Mateo CA 94403-2511
United States

☐ Override

Map

After creating an address subrecord, you can set body field values using the [Record.setValue\(options\)](#) method, the same as if you were setting values on an instance of a record. For more details, see [Creating an Address Sublist Subrecord Example](#) and [Creating an Address on a Subsidiary Record Example](#).

Writable Sublist Example

The landed cost subrecord has a sublist that lets you list individual expenses associated with merchandise you have received. The body fields on this subrecord are read-only, but the sublist is writable. To add details about an expense, you add a line to the sublist.

COST CATEGORY	AMOUNT
Customs	115.00
Shipping and Handling	100.00
Port Fees	130.00

After creating a landed cost subrecord, you can set sublist field values using `setSublistValue()` and `setCurrentSublistValue()`, like you would with a record's sublist.

For more details, see [Creating a Landed Cost Sublist Subrecord Example](#).

Writable Body Fields and Sublist Example

The order schedule subrecord has a sublist that lets you configure how and when upcoming purchase orders are to be created. The subrecord has body fields for scheduling options, like whether purchase orders need to be created manually. It also has a sublist for entering dates for future purchase orders.

OK Close

Create Purchase Orders *
Manually

Create Schedule *
Manually

Total Quantity
3

Clear All Lines Autogenerate

Release	Order	Date *	Quantity *	Allocated Demand	Memo
	Purchase Order #5762	21.8.2014	1	0	
	Purchase Order #5763	22.8.2014	1	0	
		23.8.2014	1	0	

☐

✓ Add X Cancel + Insert Remove

OK Close

After creating an order schedule subrecord, you can set field values using the same methods as for records. Use `setValue()` for body fields. For sublist fields, use `setCurrentSublistValue()` or `setSublistValue()`.

For more details, see [Creating an Order Schedule Sublist Subrecord Example](#).

Finding Subrecord Details in the Records Browser

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

As with records, you can find important information about subrecords in the [SuiteScript Records Browser](#). The Records Browser has details on each subrecord type. It also includes information about record types that can be parents to subrecords. For more information, see the following sections:

- [Finding Details About Parent Record Types](#)
- [Finding Details About Subrecord Types](#)

Finding Details About Parent Record Types

Every subrecord instance must have a parent record. An instance of a subrecord exists only to provide information about an instance of a record.

The parent record has a field that holds or references the subrecord. To create a subrecord, you need to reference this field. These fields are always identified in the Records Browser as fields of type summary. For example:

item - Items				
Internal ID	Type	Label	Required	Help
billvariancestatus	text		false	
catchupperiod	select		false	
commitinventory	select	Commit	false	
createpo	select	Create PO	false	
createwo	checkbox	Create WO	false	
daysbeforeexpiration	text		false	
deferrevrec	checkbox		false	
description	textarea	Description	false	
id	text		false	
inventorydetail	summary	Inventory Detail	false	
isclosed	text		false	
isvsoebundle	text		false	
item	select	Item	true	
itemid	text		false	
itemsource	select	Item Source	false	
itemsubtype	text		false	
itemtype	text		false	
line	text		false	
linenumber	integer		false	

Summary fields can be on the parent record's body or in a sublist. Knowing where the field is helps you create the subrecord correctly. In the reference page for each record type in the Records Browser, the **Fields** table lists all of the record type's body fields. The tables listed under the heading **Sublists** shows sublist fields. See also [Body Field Subrecords and Sublist Subrecords](#).

In some cases, the values set for other fields on a record can affect the availability or behavior of the summary field. For example:

- On an assembly build record, the availability of the inventorydetail summary field varies depending on the value of the record's item field. The summary field is available only if the item field references a serialized or lot-numbered assembly item. For an example of working with this record-subrecord combination, see [Creating an Inventory Detail Subrecord on a Body Field Example](#).
- On a vendor bill record, the landedcost summary field is available only if the landedcostperline body field is set to true. For an example of working with this record-subrecord combination, see [Creating a Landed Cost Sublist Subrecord Example](#).
- On a sales transaction, the value of the shippingaddress summary field is affected by the shipaddresslist field. For general details about shipping and billing addresses, see [Scripting Transaction Shipping and Billing Addresses](#). For a script example, see [Using SuiteScript 2.x to Create a New Shipping Address Example](#).

Finding Details About Subrecord Types

The Records Browser has a reference page for each subrecord type, like record types. Subrecords are listed alphabetically alongside records.

Each subrecord has a label below its internal ID.

Inventory Detail

Internal ID: inventorydetail

Subrecord

Fields

Internal ID	Type	napiSubmitField	Label	Required	Help
customform	select	false	Custom Form	false	Select the form you want to use to enter this record. You can select the standard form, a custom form you have already created, or select New to create a custom form.
externalid	text	false	ExternalId	false	
item	select	false	Item	false	Enter or select one or more items to determine which items the attributes are copied to. Click the Select Multiple icon to open the Choose Item window and select more than one item. Note: If the source item is a different item type than the target item, data in extraneous fields is copied over but does not affect calculations. For example, if the source item processed is a time-phased planning item but the target item is not a time-phased planning item, fields relevant to time-phased planning will be populated on the target item but the data is not used for calculations.
itemdescription	text	false	Description	false	A more complete Description of the item.
location	select	false	Location	false	Select a location to associate with this item. To use a location, the Multi-Location Inventory feature must be enabled. * If the Multi-Location Inventory feature is not enabled: Selecting an item record location limits the items that certain roles can access. For example, Role A is set up to access only items associated with Location One. Therefore, employees who have Role A can access only items associated with Location One. Note: If the Multi-Location Inventory feature is not enabled, you cannot use this field to track inventory by locations. For example, you cannot track how many widgets you have in stock in Location One. For more information click here. * If the Multi-Location Inventory feature is enabled: Selecting an item record location classifies the item by that location to limit the items that certain roles can access. To track inventory per location, use the Locations subtab at the bottom of this form. For more information click here. Select New to enter a new location record. Go to Setup > Company > Locations for details about existing location records.
quantity	float	false	Quantity	false	In the Quantity to Build field, enter the number of assembly items you want to build. You cannot enter a quantity that exceeds the amount displayed in the Buildable Quantity field.
tolocation	text	false	Location	false	
unit	select	false	Units	false	If the assembly uses Units of Measure, the base units are displayed in the Units field.

Sublists

inventoryassignment - Inventory Detail

Internal ID	Type	Label	Required	Help
binnumber	select	Bin	true	
expirationdate	date	Expiration Date	false	

The reference page displays a subrecord's scriptable fields and sublists, like record types.

In the UI, subrecord sublists usually don't have labels. But the Records Browser shows the name of each sublist.

About the Address Subrecord

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The address subrecord has some unique characteristics. This means working with the address subrecord might be a bit different than with other subrecords. For details, see the following sections:

- [Billing and Shipping Addresses Can Be Sourced from Other Records](#)
- [The Address Subrecord Can Have Custom Forms](#)
- [Address Data Is Summarized on the Parent Record](#)

Note: See also [Scripting Transaction Shipping and Billing Addresses](#).

Billing and Shipping Addresses Can Be Sourced from Other Records

Most subrecords are unique to the record they're created on. However, in some cases, a single address subrecord instance can be referenced by multiple records.

For example: You can create multiple addresses for an entity. When you create a transaction for that entity, you can use one of its existing addresses for shipping or billing. This means setting shipping or billing addresses can work a bit differently than other address fields. For details, see [Addresses Can Be Sourced from Entity Records](#).

The Address Subrecord Can Have Custom Forms

Compared with other subrecords, the address subrecord is unique in that you can create custom entry forms for it. For example, you may want to create custom forms for different countries.

If your account has multiple address forms, and if you are not seeing the expected results, the reason could be related to the form. In particular, if your script is using dynamic mode, the first value you set on the subrecord form should be country. The reason is that if you set a value for country that differs from the default, the form resets when the country value changes. Set the country value first as a best practice.

Note: The **Override**, **Phone**, and **Zip** fields of the address subrecord aren't available as search filters from the customer record.

Address Data Is Summarized on the Parent Record

Most subrecords don't show a summary of their details on the parent record in the UI. You have to open the subrecord in its own window to view its data.

The address subrecord is an exception to this rule. When you enter address subrecord details and go back to the main record view, the system usually shows a summary of what you entered. For example, in the following screenshot, subrecord data is summarized in the Address column.

Communication <u>Address</u> Human Resources ACH/Direct Deposit Time Tracking Payroll Commission Related Records Marketing Access System Information						
ID	Default Shipping	Home	Label	Address *	Edit	
	Yes	Yes	Company Headquarters	John Smith 2955 Campus Drive Suite 100 San Mateo CA 94403-2511 United States		

This summary shows the value of the `addrtext` field on the address subrecord. This value is created through the use of a template and generated from other values entered on the subrecord, such as the values for the city and state fields. Each address form can have its own template for determining the `addrtext` value. You can view an address form's template by checking its record at Customization > Forms > Address Forms.

An error in the `addrtext` field doesn't mean there's an error in the other subrecord values. If an error exists in the summary, review the template to make sure that it is capturing the values you intend.

To view the values for all of the fields on the subrecord, do one of the following:

- Use one of the following methods to retrieve the subrecord: `getCurrentSublistSubrecord()`, `getSublistSubrecord()`, or `getSubrecord()`. For an example, see [Retrieving an Address Subrecord Example](#).
- In the UI, open the subrecord for editing in its own window.

Client Scripts Attached to the Address Subrecord

Client scripts on address subrecords in transactions might run on both the server and client. This is normal behavior, so be aware of it. To run the logic only one time, wrap it in an `if` statement that exits the script on the server. For example:

```
1 | if (typeof document !== 'undefined') {
```



```

2 // client script logic
3 }

```

If a client script attached to an address subrecord loads the [N/currentRecord Module](#), the script fails. Although you can't load the [N/currentRecord Module](#) in a client script that is attached to the address subrecord, you can get `currentRecord` from `context.currentRecord`:

```

1 define([],function(){
2     function saveRecord(context){
3         var currentRecord = context.currentRecord;
4         return true;
5     }
6     return {
7         saveRecord: saveRecord
8     }
9 });

```

See the help topic [currentRecord.CurrentRecord](#).

You can attach client scripts to custom entry forms, custom transaction forms, or custom address forms. See [Attaching a Client Script to a Form](#).

For information about client scripts, see [SuiteScript 2.x Client Script Type](#).

Subrecord Scripting in SuiteScript 2.x Compared With 1.0

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Compared with SuiteScript 1.0, SuiteScript 2.x introduces the following changes in how you script subrecords:

- [A Single Method Lets You Create and Load Subrecords](#)
- [Subrecords Do Not Have to Be Explicitly Saved](#)
- [To Create Addresses, You Must Use Subrecord Methods](#)

A Single Method Lets You Create and Load Subrecords

In SuiteScript 1.0, there are separate APIs for creating and editing subrecords. For example, you'd use `nlapiCreateSubrecord` to create a subrecord and `nlapiEditSubrecord` to edit a subrecord.

In SuiteScript 2.x, the same method can create or edit a subrecord. These methods all have the word **get** in their names. For example, you use the `getSubrecord()` method to create or load a subrecord that exists on the body of a record. You use the `getSublistSubrecord()` or `getCurrentSublistSubrecord()` method to create or load a subrecord that exists on a sublist line.

When you use any of these methods, the system responds with the following logic:

- If a subrecord already exists in that field, it gets loaded.
- If not, a new subrecord is created. Then you can set values on the field. The subrecord is saved when you save the record.

Subrecords Do Not Have to Be Explicitly Saved

In SuiteScript 1.0, you had to explicitly save a subrecord before saving the record. However, in SuiteScript 2.x, after you create a subrecord or make changes to one, you are not required to explicitly save the subrecord (and no methods exist for that purpose). New subrecords are saved when you save the record. This also applies if you update an existing subrecord. Updates are saved when you save the record.

To Create Addresses, You Must Use Subrecord Methods

Before address subrecords existed, addresses were represented as body fields or sublist lines on records.

After address subrecords were introduced, SuiteScript 1.0 was updated to support two ways of working with addresses:

- You can use subrecord APIs (the preferred method)
- You can interact with addresses using the legacy approach of setting values for the address body and sublist fields that used to exist. This support was made possible by logic added to the system that read the values set in this manner and created an address subrecord on behalf of the 1.0 script. Because this support exists in 1.0, these deprecated fields are displayed in the SuiteScript Records Browser as available fields.

In SuiteScript 2.x, you **must** use subrecord methods to create an address. The system doesn't support the old address body and sublist fields. For that reason, to create, edit, or load an address in SuiteScript 2.x, you must instantiate the address subrecord by referencing the appropriate summary field.

Scripting Subrecords that Occur on Sublist Lines

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Often, a subrecord is accessed through a field on a sublist line. For example:

- Every purchase order must include a list of items. Depending on the item's configuration, the sublist line might need an inventory detail subrecord.
- A vendor bill can have a list of items. If the bill is configured to track landed cost per line, each line in the Items sublist may include a landed cost subrecord.
- An employee may have multiple addresses. Each address is stored in a subrecord, and each subrecord is associated with a line in the Address sublist.

In each case, the sublist line also has fields that are not part of the subrecord. The subrecord is linked to only one field on the sublist line. In the SuiteScript Records Browser, this field is always identified as a field of type summary.

For more details, see the following sections:

- [Using SuiteScript 2.x to Create a Subrecord in a Sublist Field](#)
- [Using SuiteScript 2.x to Edit a Subrecord that Occurs in a Sublist Field](#)
- [Using SuiteScript 2.x to Retrieve a Sublist Subrecord](#)

Using SuiteScript 2.x to Create a Subrecord in a Sublist Field

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

A sublist line can include a field that references a subrecord, depending on the record type and other factors. Often, you need to add the subrecord when creating the sublist line. In other cases, you can add the subrecord later.

To create a sublist subrecord, your script must use the [N/record Module](#). You can use either dynamic or standard mode. For details, see the following sections:

- [Creating a Sublist Subrecord in Dynamic Mode](#)
- [Creating a Sublist Subrecord in Standard Mode](#)

Subrecords can also be in a record's body field. For details on working with subrecords when they occur in body fields, see [Scripting Subrecords that Occur in Body Fields](#). For an overview of the difference between these two types of placement, see [Body Field Subrecords and Sublist Subrecords](#).

Note: For more details about the methods referenced in this topic, see the help topic [Record Object Members](#).

Creating a Sublist Subrecord in Dynamic Mode

If your script uses dynamic mode, you can use the following procedure to create a subrecord in a sublist field.

To learn about SuiteScript scripting modes, see [SuiteScript 2.x Standard and Dynamic Modes](#).

To create a sublist subrecord in dynamic mode:

1. Create a new record and set its required fields, or load an existing record to update it.
2. Do one of the following:
 - If you want to create a new sublist line, create the line using the [Record.selectNewLine\(options\)](#) method. Set any required values on the sublist line with the [Record.setCurrentSublistValue\(options\)](#) method.
 - If you want to add a new subrecord to an existing sublist line, identify that line using the [Record.selectLine\(options\)](#) method.
3. Create the new subrecord using the [Record.getCurrentSublistSubrecord\(options\)](#) method. The method requires two arguments:
 - A `sublistId`, which identifies the sublist.
 - A `fieldId`, which identifies the field on the sublist that contains the subrecord. In the Records Browser, the field that holds the subrecord is always identified as a field of type summary.

For example, you could use an expression like the following to create an order schedule subrecord on an item sublist:

```

1 ...
2 var orderScheduleSubrecord = blanketPurchaseOrder.getCurrentSublistSubrecord({
3   sublistId: 'item',
4   fieldId: 'orderschedule'
5 });
6 ...

```

4. Use [Record.setValue\(options\)](#) method to set body fields on the subrecord as needed. For example, on an order schedule subrecord, you could use the following expression to set a value for the Create Purchase Orders select field.

```

1 ...
2 orderScheduleSubrecord.setValue({
3   fieldId: 'createpurchaseorder',
4   value: 'LEAD'
5 });
6 ...

```

Note that not all subrecords have body fields that can be edited.

5. If the subrecord has a sublist, generally you are required to add at least one line to the sublist. For each line, use the following guidelines:
 - Create the line with the [Record.selectNewLine\(options\)](#) method.
 - Set required values on the line with the [Record.setCurrentSublistValue\(options\)](#) method.

- Save the subrecord's sublist line with the [Record.commitLine\(options\)](#) method.

For example, if you create an order schedule subrecord, you could use the following expressions to create a line on the subrecord's schedule sublist:

```

1 ...
2 orderScheduleSubrecord.selectNewLine
3   sublistId: 'schedule',
4 };
5
6 orderScheduleSubrecord.setCurrentSublistValue({
7   sublistId: 'schedule',
8   fieldId: 'quantity',
9   value: 1
10 });
11
12 orderScheduleSubrecord.setCurrentSublistValue({
13   sublistId: 'schedule',
14   fieldId: 'trandate',
15   value: dateVariable
16 });
17
18 orderScheduleSubrecord.commitLine({
19   sublistId: 'schedule'
20 });
21 ...

```

6. Save the sublist line that holds the subrecord with the [Record.commitLine\(options\)](#) method.
7. Save the record with the [Record.save\(options\)](#) method.

Note: For a full script examples, see [Creating an Inventory Detail Sublist Subrecord Example](#) and [Creating an Address Sublist Subrecord Example](#).

Creating a Sublist Subrecord in Standard Mode

If your script uses standard mode, you can use the following procedure to create a subrecord in a sublist field.

To learn about SuiteScript scripting modes, see [SuiteScript 2.x Standard and Dynamic Modes](#)

To create a sublist subrecord in standard mode:

1. To add a new record, create it and set its required body fields. To update an existing record, load it.
2. If you want to create a new sublist line, create the line using the [Record.insertLine\(options\)](#) method. Set any required fields on the sublist line.
3. Create the new subrecord with the [Record.getSublistSubrecord\(options\)](#) method. This method takes three arguments:
 - A `sublistId`, which identifies the sublist.
 - A `fieldId`, which identifies the field on the sublist that contains the subrecord. In the Records Browser, the field that holds the subrecord is always identified as a field of type summary.
 - A line number.

For example, you could use an expression like the following to create an inventory detail subrecord on the first line in an item sublist:

```

1 ...
2 inventoryDetailSubrecord = rec.getSublistSubrecord({
3   sublistId: 'item',
4   fieldId: 'inventorydetail',
5   line: 0

```

```

6 | });
7 | ...

```

4. Use the [Record.setValue\(options\)](#) method to set body fields on the subrecord. Be aware that not all subrecords have writable body fields.
5. If the subrecord has a sublist, generally you are required to add at least one line to the sublist. For each line, use the following guidelines:
 - Use the [Record.insertLine\(options\)](#) method to create the line.
 - Set values on the line with the [Record.setSublistValue\(options\)](#) method.

For example, if you were creating an order schedule subrecord, you could use the following expressions to create a line on the subrecord's schedule sublist:

```

1 | ...
2 | subrecordInvDetail.setSublistValue({
3 |     sublistId: 'inventoryassignment',
4 |     fieldId: 'receiptinventorynumber',
5 |     line: 0,
6 |     value: '012345'
7 | });
8 | ...

```

6. Save the record using the [Record.save\(options\)](#) method.



Note: For a full script example of creating a sublist subrecord in standard mode, see [Creating a Landed Cost Sublist Subrecord Example](#).

Creating an Inventory Detail Sublist Subrecord Example

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following example shows how to create a purchase order record that includes an inventory detail subrecord. The script adds a line to the item sublist and creates an inventory detail subrecord on that line.

To use this example, make sure you meet these requirements:

- The Advanced Bin / Numbered Inventory Management feature must be enabled at Setup > Company > Enable Features, on the Items & Inventory subtab.
- The item you add to the sublist should be a lot-numbered inventory item.
- The receiptinventorynumber value must be unique in your system.

Note how this field differs from the issueinventorynumber field, both available within the inventory detail subrecord, in terms of usage. Make sure that your script does not set both fields at the same time. Use the appropriate field based on whether or not the transaction adds items to your inventory, as follows:

- **Issue Inventory Number** (ID: **issueinventorynumber**) – In this field, you can select lot or serial numbers that exist in your inventory. Be sure to reference this field to select numbers for items that you include in a sales order, item fulfillment, inventory status change, and negative adjustments, among others.
- **Receipt Inventory Number** (ID: **receiptinventorynumber**) – In this field, you can enter lot or serial numbers for items that you want to add to your inventory. Be sure to use this field to assign numbers to items within item receipts and positive adjustments, among others. Refer to the example in this topic to see how this field is used for the purchase order transaction.

This example uses dynamic mode, but you can also use standard mode to add the subrecord. For general details about using either approach to add a sublist subrecord, see [Using SuiteScript 2.x to Create a Subrecord in a Sublist Field](#).

To learn about SuiteScript scripting modes, see [SuiteScript 2.x Standard and Dynamic Modes](#)

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType UserEventScript
4   */
5
6  define([ 'N/record' ],function(record){
7      function afterSubmit(context)
8          // Create the purchase order.
9          var rec = record.create({
10             type: record.Type.PURCHASE_ORDER,
11             isDynamic: true
12         });
13
14         // Set body fields on the purchase order.
15         rec.setValue({
16             fieldId: 'entity',
17             value: '1663'
18         });
19
20         rec.setValue({
21             fieldId: 'location',
22             value: '6'
23         });
24
25         // Create one line in the item sublist.
26         rec.selectNewLine({
27             sublistId: 'item'
28         });
29
30         rec.setCurrentSublistValue({
31             sublistId: 'item',
32             fieldId: 'item',
33             value: '299'
34         });
35
36         rec.setCurrentSublistValue({
37             sublistId: 'item',
38             fieldId: 'quantity',
39             value: 1
40         });
41
42         // Create the subrecord for that line.
43         var subrec = rec.getCurrentSublistSubrecord({
44             sublistId: 'item',
45             fieldId: 'inventorydetail'
46         });
47
48         // Add a line to the subrecord's inventory assignment sublist.
49         subrec.selectNewLine({
50             sublistId: 'inventoryassignment'
51         });
52
53         subrec.setCurrentSublistValue({
54             sublistId: 'inventoryassignment',
55             fieldId: 'quantity',
56             value: 2
57         });
58
59         subrec.setCurrentSublistValue({
60             sublistId: 'inventoryassignment',
61             fieldId: 'receiptinventorynumber',
62             value: '01234'
63         });
64
65         // Save the line in the subrecord's sublist.
66         subrec.commitLine({
67             sublistId: 'inventoryassignment'
68         });
69
70         // Save the line in the record's sublist
71         rec.commitLine({

```

```

72     sublistId: 'item'
73   });
74
75   // Save the record.
76   try {
77     var recId = rec.save();
78     log.debug({
79       title: 'Record created successfully',
80       details: 'Id: ' + recId
81     });
82   } catch (e) {
83     log.error({
84       title: e.name,
85       details: e.message
86     });
87   }
88 }
89 return {
90   afterSubmit: afterSubmit
91 };
92 });

```

Creating an Address Sublist Subrecord Example

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following example shows how to create an employee record and populate the Address sublist with one line. The script also creates an address subrecord on the sublist line.

This example uses dynamic mode, but you can also use standard mode to add the subrecord. For general details about using either approach to add a sublist subrecord, see [Using SuiteScript 2.x to Create a Subrecord in a Sublist Field](#).

To learn about SuiteScript scripting modes, see [SuiteScript 2.x Standard and Dynamic Modes](#)

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType UserEventScript
4   */
5
6  define([ 'N/record' ],function(record){
7    function afterSubmit(context){
8
9      // Create the record.
10     var rec = record.create({
11       type: record.Type.EMPLOYEE,
12       isDynamic: true
13     });
14
15     // Set the required body fields.
16     rec.setValue({
17       fieldId: 'firstname',
18       value: 'John'
19     });
20
21     rec.setValue({
22       fieldId: 'lastname',
23       value: 'Smith'
24     });
25
26     rec.setValue({
27       fieldId: 'subsidiary',
28       value: '1'
29     });
30
31     // Create a line in the Address sublist.
32     rec.selectNewLine({
33       sublistId: 'addressbook'
34     });
35

```

```

36 // Set an optional field on the sublist line.
37 rec.setCurrentSublistValue({
38     sublistId: 'addressbook',
39     fieldId: 'label',
40     value: 'Primary Address'
41 });
42
43 // Create an address subrecord for the line.
44 var subrec = rec.getCurrentSublistSubrecord({
45     sublistId: 'addressbook',
46     fieldId: 'addressbookaddress'
47 });
48
49 // Set body fields on the subrecord. Because the script uses
50 // dynamic mode, you should set the country value first. The country
51 // value determines which address form is to be used, so by setting
52 // this value first, you ensure that the values for the rest
53 // of the form's fields will be set properly.
54 subrec.setValue({
55     fieldId: 'country',
56     value: 'US'
57 });
58
59 subrec.setValue({
60     fieldId: 'city',
61     value: 'San Mateo'
62 });
63
64 subrec.setValue({
65     fieldId: 'state',
66     value: 'CA'
67 });
68
69 subrec.setValue({
70     fieldId: 'zip',
71     value: '94403'
72 });
73
74 subrec.setValue({
75     fieldId: 'addr1',
76     value: '2955 Campus Drive'
77 });
78
79 subrec.setValue({
80     fieldId: 'addr2',
81     value: 'Suite 100'
82 });
83
84 // Save the sublist line.
85 rec.commitLine({
86     sublistId: 'addressbook'
87 });
88
89 // Save the record.
90 try{
91     var recId = rec.save();
92
93     log.debug({
94         title: 'Record created successfully',
95         details: 'Id: ' + recId
96     });
97 } catch (e){
98     log.error({
99         title: e.name,
100         details: e.message
101     });
102 }
103
104 return {
105     afterSubmit: afterSubmit
106 };
107 });

```


Creating an Order Schedule Sublist Subrecord Example

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following example creates a blanket purchase order record. It adds one line to the item sublist and creates an order schedule subrecord on that line.

To use this example, enable the Blanket Purchase Order feature at Setup > Company > Enable Features, on the Transactions subtab.

This example uses dynamic mode, but you can also use standard mode to add the subrecord. For general details about using either approach to add a sublist subrecord, see [Using SuiteScript 2.x to Create a Subrecord in a Sublist Field](#).

To learn about SuiteScript scripting modes, see [SuiteScript 2.x Standard and Dynamic Modes](#)

```

1  /**
2   * @ApiVersion 2.x
3   * @NScriptType UserEventScript
4   */
5
6  define([ 'N/record' ],function(record) {
7      function afterSubmit(context) {
8          // Create the record.
9          var rec = record.create({
10             type: record.Type.BLANKET_PURCHASE_ORDER,
11             isDynamic: true
12         });
13
14         // Set body fields on the record.
15         rec.setValue({
16             fieldId: 'entity',
17             value: '1663'
18         });
19
20         rec.setValue({
21             fieldId: 'location',
22             value: '6'
23         });
24
25         rec.setValue({
26             fieldId: 'memo',
27             value: '456789'
28         });
29
30         // Create one line in the item sublist.
31         rec.selectNewLine({
32             sublistId: 'item',
33             line: 0
34         });
35
36         rec.setCurrentSublistValue({
37             sublistId: 'item',
38             fieldId: 'item',
39             value: '500'
40         });
41
42         rec.setCurrentSublistValue({
43             sublistId: 'item',
44             fieldId: 'quantity',
45             value: '1'
46         });
47
48         // Create the subrecord for that line.
49         var subrec = rec.getCurrentSublistSubrecord({
50             sublistId: 'item',
51             fieldId: 'orderschedule'
52         });
53     }
54 }
```

```

54 // Set a field on the body of the subrecord.
55 subrec.setValue({
56     fieldId: 'createpurchaseorder',
57     value: 'LEAD'
58 });
59
60 // Create a line in the subrecord's sublist.
61 subrec.selectNewLine({
62     sublistId: 'schedule',
63 });
64
65 subrec.setCurrentSublistValue({
66     sublistId: 'schedule',
67     fieldId: 'quantity',
68     value: 1
69 });
70
71 var nextQuarter = new Date();
72 nextQuarter.setDate(nextQuarter.getDate() + 90);
73
74 subrec.setCurrentSublistValue({
75     sublistId: 'schedule',
76     fieldId: 'trandate',
77     value: nextQuarter
78 });
79
80 // Save the line in the subrecord's sublist.
81 subrec.commitLine({
82     sublistId: 'schedule'
83 });
84
85 // Save the line in the record's sublist
86 rec.commitLine({
87     sublistId: 'item'
88 });
89
90 // Save the record.
91 try {
92     var recId = rec.save();
93     log.debug({
94         title: 'Record created successfully',
95         details: 'Id: ' + recId
96     });
97 } catch (e) {
98     log.error({
99         title: e.name,
100         details: e.message
101     });
102 }
103
104 return {
105     afterSubmit: afterSubmit
106 };
107 });

```

Creating a Landed Cost Sublist Subrecord Example

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following example creates a vendor bill record. This example creates one line in the item sublist. It also sets the Landed Cost per Line option to true. With this configuration, it is possible to create a landed cost subrecord for each line.

To use this example, enable the Landed Cost feature at Setup > Company > Enable Features on the Items & Inventory subtab.

This example uses standard mode, but you can also use dynamic mode to add the subrecord. For general details about using either approach to add a sublist subrecord, see [Using SuiteScript 2.x to Create a Subrecord in a Sublist Field](#).

To learn about SuiteScript scripting modes, see [SuiteScript 2.x Standard and Dynamic Modes](#)

```

1  /**
2   * @ApiVersion 2.x
3   * @NScriptType UserEventScript
4   */
5
6  define([ 'N/record' ],function(record) {
7      function afterSubmit(context) {
8          // Create the record.
9          var rec = record.create({
10             type : record.Type.VENDOR_BILL,
11             isDynamic: false
12         });
13
14         // Set body fields on the record.
15         rec.setValue({
16             fieldId: 'entity',
17             value: '1663'
18         });
19
20         rec.setValue({
21             fieldId: 'location',
22             value: '6'
23         });
24
25         rec.setValue({
26             fieldId: 'tranid',
27             value: '101A'
28         });
29
30         // Set the Landed Cost per Line field to true.
31         rec.setValue({
32             fieldId: 'landedcostperline',
33             value: true
34         });
35
36         // Add an item to the Item sublist.
37         rec.insertLine({
38             sublistId: 'item',
39             line: 0
40         });
41
42         // Set values on the sublist line.
43         rec.setSublistValue({
44             sublistId: 'item',
45             fieldId: 'item',
46             line: 0,
47             value: '599'
48         });
49
50         rec.setSublistValue({
51             sublistId: 'item',
52             fieldId: 'quantity',
53             line: 0,
54             value: 1
55         });
56
57         rec.setSublistValue({
58             sublistId: 'item',
59             fieldId: 'location',
60             line: 0,
61             value: '6'
62         });
63
64         // Create the subrecord.
65         var subrec = rec.getSublistSubrecord({
66             sublistId: 'item',
67             fieldId: 'landedcost',
68             line: 0
69         });
70
71         // Add a line to the subrecord's Landed Cost Data sublist.

```

```

72     subrec.insertLine({
73         sublistId: 'landedcostdata',
74         line: 0
75     });
76
77     // Set values on the subrecord's sublist line.
78     subrec.setSublistValue({
79         sublistId: 'landedcostdata',
80         fieldId: 'costcategory',
81         line: 0,
82         value: 2
83     });
84
85     subrec.setSublistValue({
86         sublistId: 'landedcostdata',
87         fieldId: 'amount',
88         line: 0,
89         value: 17.85
90     });
91
92     // Save the record.
93     try {
94         var recId = rec.save();
95         log.debug({
96             title: 'Record created successfully',
97             details: 'Id: ' + recId
98         });
99     } catch (e) {
100         log.error({
101             title: e.name,
102             details: e.message
103         });
104     }
105 }
106 return {
107     afterSubmit: afterSubmit
108 };
109 });

```

Using SuiteScript 2.x to Edit a Subrecord that Occurs in a Sublist Field

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Your script can edit a subrecord in a sublist field in some cases.

To edit a sublist subrecord, your script must use the [N/record Module](#). You can use either dynamic or standard mode. For details, see the following sections:

- [Editing a Subrecord in Dynamic Mode](#)
- [Editing a Subrecord in Standard Mode](#)

To learn about SuiteScript scripting modes, see [SuiteScript 2.x Standard and Dynamic Modes](#)

Editing a Subrecord in Dynamic Mode

If your script uses dynamic mode, you can use the following procedure to edit a subrecord that occurs in a sublist field.

To edit a subrecord in dynamic mode:

1. Load the record.
2. Use the [Record.selectLine\(options\)](#) method to identify the sublist and line that contain the subrecord that you want to update.
3. Get the subrecord using the [Record.getCurrentSublistSubrecord\(options\)](#) method. The method requires two arguments:
 - A `sublistId`.
 - A `fieldId`, which identifies the field on the sublist that contains the subrecord. In the Records Browser, the field that holds the subrecord is always identified as a field of type summary.

For example, suppose you are working with an entity record, such as an employee or customer. You could use an expression like the following to load an address subrecord from the entity's Address sublist:

```

1 | ...
2 | var addressSubrecord = rec.getCurrentSublistSubrecord({
3 |   sublistId: 'addressbook',
4 |   fieldId: 'addressbookaddress'
5 | });
6 | ...

```

4. Update the subrecord's body fields using the [Record.setValue\(options\)](#) method as needed. For example, you could use an expression like the following to update a value on the address subrecord:

```

1 | ...
2 | addressSubrecord.setValue({
3 |   fieldId: 'city',
4 |   value: 'St. Petersburg'
5 | });
6 | ...

```

However, note that some subrecords do not have writable body fields.

5. If the subrecord has a sublist you want to modify, follow these steps for each line you want to change:
 1. Identify the line you want to change using the [Record.selectLine\(options\)](#) method.
 2. For each value you want to change, use the [Record.setCurrentSublistValue\(options\)](#) method to identify the field and the new value.
 3. Save your changes to the subrecord's sublist line using the [Record.commitLine\(options\)](#) method.
6. Save the line that holds the subrecord with the [Record.commitLine\(options\)](#) method.
7. Save the record with the [Record.save\(options\)](#) method.

Note: For a full script example that shows editing a sublist subrecord in dynamic mode, see [Updating an Order Schedule Sublist Subrecord Example](#).

Editing a Subrecord in Standard Mode

If your script uses standard mode, you can use the following procedure to edit a subrecord that occurs in a sublist field.

To learn about SuiteScript scripting modes, see [SuiteScript 2.x Standard and Dynamic Modes](#)

To edit a subrecord in standard mode:

1. Load the record.
2. Retrieve the subrecord using the [Record.getSublistSubrecord\(options\)](#) method. This method takes three arguments:
 - A sublistId.
 - A fieldId, which identifies the field on the sublist that contains the subrecord. In the Records Browser, the field that holds the subrecord is always identified as a field of type summary.
 - A line number, which identifies the sublist line that contains the subrecord you want to change.

For example, you could use an expression like the following to load an inventory detail subrecord from an item sublist:

```

1 | ...
2 | inventoryDetailSubrecord = rec.getSublistSubrecord({
3 |     sublistId: 'item',
4 |     fieldId: 'inventorydetail',
5 |     line: 0
6 | });
7 | ...

```

3. Use the [Record.setValue\(options\)](#) method to update the subrecord's body fields. Be aware that not all subrecords have writable body fields.
4. If the subrecord has a sublist whose values you want to modify, use the [Record.setSublistValue\(options\)](#) method to update the appropriate value. The method requires four arguments:
 - A sublistId.
 - A fieldId, which identifies the field you want to change.
 - A line number, which identifies the sublist line you want to change.
 - The new value.

For example, if you were updating an inventory detail subrecord, you could use the following expression to update the serial number on the first line of the inventory assignment sublist:

```

1 | ...
2 | inventoryDetailSubrecord.setSublistValue({
3 |     sublistId: 'inventoryassignment',
4 |     fieldId: 'receiptinventorynumber',
5 |     line: 0,
6 |     value: '56789'
7 | });
8 | ...

```

5. Save the record with the `save()` method.

 **Note:** For a full script example showing how to modify a subrecord in standard mode, see [Updating an Address Subrecord Example](#).

Updating an Order Schedule Sublist Subrecord Example

 **Applies to:** SuiteScript 2.x | APIs | SuiteCloud Developer

The following example loads an existing blanket purchase order record. It selects a line on the record's item sublist, retrieves the subrecord associated with that line, and makes changes to the subrecord.

To use this example, you must meet the following prerequisites:

- The Blanket Purchase Order feature must be enabled at Setup > Company > Enable Features, on the Transactions subtab.
- The blanket purchase order record that you reference should have at least one line in the item sublist. The line should reference an existing order schedule subrecord.

This example uses dynamic mode, but you can also use standard mode to update the subrecord. For general details about using either approach to update a sublist subrecord, see [Using SuiteScript 2.x to Edit a Subrecord that Occurs in a Sublist Field](#). To learn about SuiteScript scripting modes, see [SuiteScript 2.x Standard and Dynamic Modes](#)

```

1  define([ 'N/record' ],function(record) {
2      function afterSubmit(context) {
3          // Load the blanket purchase order record.
4          var rec = record.load({
5              type: record.Type.BLANKET_PURCHASE_ORDER,
6              id: 3319,
7              isDynamic: true
8          });
9
10         // Select the sublist and line.
11         rec.selectLine({
12             sublistId: 'item',
13             line: 0
14         });
15
16         // Retrieve the subrecord.
17         var subrec = rec.getCurrentSublistSubrecord({
18             sublistId: 'item',
19             fieldId: 'orderschedule'
20         });
21
22         // Select the appropriate line in the subrecord's sublist.
23         subrec.selectLine({
24             sublistId: 'schedule',
25             line: 0
26         });
27
28         // Identify the field to be modified, and set new value.
29         var nextQuarter = new Date();
30         nextQuarter.setDate(nextQuarter.getDate() + 90);
31
32         subrec.setCurrentSublistValue({
33             sublistId: 'schedule',
34             fieldId: 'trandate',
35             value: nextQuarter
36         });
37
38         // Save the subrecord's sublist line.
39         subrec.commitLine({
40             sublistId: 'schedule'
41         });
42
43         // Save the item sublist line that contains the subrecord.
44         rec.commitLine({
45             sublistId: 'item'
46         });
47
48         // Save the record.
49         try {
50             var recId = rec.save();
51             log.debug({
52                 title: 'Record updated successfully',
53                 details: 'Id: ' + recId
54             });
55         } catch (e) {
56             log.error({
57                 title: e.name,
58                 details: e.message
59             });
60         }

```

```

61     }
62     return {
63         afterSubmit: afterSubmit
64     };
65 });

```

Updating an Address Subrecord Example

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following example shows how to update an employee address. Each address on the employee record is stored in a subrecord linked to a line in the Address sublist.

This example uses standard mode, but you can also use dynamic mode to edit the subrecord. For general details about using either approach to update a sublist subrecord, see [Using SuiteScript 2.x to Edit a Subrecord that Occurs in a Sublist Field](#).

To learn about SuiteScript scripting modes, see [SuiteScript 2.x Standard and Dynamic Modes](#)

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType UserEventScript
4   */
5
6  define([ 'N/record' ], function(record) {
7      function afterSubmit(context) {
8          // Load the record.
9          var rec = record.load({
10             type: record.Type.EMPLOYEE,
11             id: 1863,
12             isDynamic: false
13         });
14
15         // Retrieve the subrecord to be modified.
16         var subrec = rec.getSublistSubrecord({
17             sublistId: 'addressbook',
18             fieldId: 'addressbookaddress',
19             line: 0
20         });
21
22         // Change a field on the subrecord.
23         subrec.setValue({
24             fieldId: 'addr1',
25             value: '15 Main Street'
26         });
27
28         // Save the record.
29         try {
30             var recId = rec.save();
31             log.debug({
32                 title: 'Record updated successfully',
33                 details: 'Id: ' + recId
34             });
35         } catch (e) {
36
37             log.error({
38                 title: e.name,
39                 details: e.message
40             });
41         }
42     }
43     return {
44         afterSubmit: afterSubmit
45     };
46 });

```


Using SuiteScript 2.x to Retrieve a Sublist Subrecord

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

In some cases, you may want to retrieve data from a subrecord that occurs in a sublist field.

To retrieve a sublist subrecord, use the [N/record Module](#). You can use either dynamic or standard mode. For details, see the following sections:

- [Retrieving a Sublist Subrecord in Dynamic Mode](#)
- [Retrieving a Subrecord in Standard Mode](#)

Retrieving a Sublist Subrecord in Dynamic Mode

If your script uses dynamic mode, you can use the following procedure to retrieve a subrecord that occurs in a sublist field.

To learn about SuiteScript scripting modes, see [SuiteScript 2.x Standard and Dynamic Modes](#)

To retrieve a sublist subrecord in dynamic mode:

1. Load the parent record.
2. Select the line that contains the subrecord with the [Record.selectLine\(options\)](#).
3. Get the subrecord using the [Record.getCurrentSublistSubrecord\(options\)](#) method. This method takes two arguments:

- A `sublistId`.
- A `fieldId`, which identifies the field on the sublist that contains the subrecord. In the Records Browser, the field that holds the subrecord is always identified as a field of type summary.

For example, let's say you're working with an employee or customer record. You could use an expression like the following to load an address subrecord from the entity's Address sublist:

```
1 ...
2 var addressSubrecord = rec.getCurrentSublistSubrecord({
3     sublistId: 'addressbook',
4     fieldId: 'addressbookaddress'
5 });
6 ...
```

4. To retrieve a value from the subrecord's body, use the [Record.getValue\(options\)](#). For example, you could use an expression like the following to retrieve a detail from the body of an address subrecord:

```
1 ...
2 var cityValue = addressSubrecord.getValue({
3     fieldId: 'city'
4 });
5 ...
```

5. If you want to retrieve a value from the subrecord's sublist, use the [Record.getSublistValue\(options\)](#) method.

Note: For a full script example showing how to retrieve a sublist subrecord in dynamic mode, see [Retrieving an Order Schedule Subrecord Example](#).

Retrieving a Subrecord in Standard Mode

If your script uses standard mode, use the following procedure to retrieve a sublist subrecord.

To learn about SuiteScript scripting modes, see [SuiteScript 2.x Standard and Dynamic Modes](#)

To retrieve a subrecord in standard mode

1. Load the parent record.
2. Retrieve the subrecord with the [Record.getSublistSubrecord\(options\)](#) method. The method requires three arguments:
 - A sublistId.
 - A fieldId, which identifies the field on the sublist that contains the subrecord. In the Records Browser, the field that holds the subrecord is always identified as a field of type summary.
 - A line number

For example, you could use an expression like the following to load an order schedule subrecord from a blanket purchase order record:

```

1  ...
2  var orderScheduleSubrecord = rec.getSublistSubrecord({
3      sublistId: 'item',
4      fieldId: 'orderschedule',
5      line: 0
6  });
7  ...

```

3. To retrieve a value from the subrecord's body, use the [Record.getValue\(options\)](#) method.
4. To retrieve a value from the subrecord's sublist, use the [Record.getSublistValue\(options\)](#) method.

For example, the order schedule subrecord has a schedule sublist. If you wanted to retrieve the date value from the second line of the schedule sublist, you would use an expression like the following:

```

1  ...
2  var dateValue = orderScheduleSubrecord.getSublistValue({
3      sublistId: 'schedule',
4      fieldId: 'trandate',
5      line: 1
6  });
7  ...

```

Note: For a full script example showing how to retrieve a sublist subrecord in standard mode, see [Retrieving an Address Subrecord Example](#).

Retrieving an Order Schedule Subrecord Example

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following example loads a blanket purchase order record. It selects a line on the item sublist and retrieves the associated order schedule. It reads two values from the subrecord and logs them.

To use this script, make sure you meet these requirements:

- The Blanket Purchase Order feature must be enabled at Setup > Company > Enable Features , on the Transactions subtab.
- The blanket purchase order record you reference must have at least one line in the Item sublist, and that line should include an order schedule subrecord.

This example uses dynamic mode, but you can also use standard mode to load the subrecord. For general details about using either approach to retrieve a sublist subrecord, see [Using SuiteScript 2.x to Retrieve a Sublist Subrecord](#).

To learn about SuiteScript scripting modes, see [SuiteScript 2.x Standard and Dynamic Modes](#)

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType usereventscript
4   */
5
6  define([ 'N/record' ],function(record) {
7      function afterSubmit(context) {
8          // Load the record.
9          var rec = record.load({
10             type: record.Type.BLANKET_PURCHASE_ORDER,
11             id: 3120,
12             isDynamic: true
13         });
14
15         // Retrieve the subrecord.
16
17         rec.selectLine({
18             sublistId: 'item',
19             line: 0
20         });
21
22         var subrec = rec.getCurrentSublistSubrecord({
23             sublistId: 'item',
24             fieldId: 'orderschedule',
25         });
26
27         // Create a variable and initialize it to the
28         // value of the trandate field.
29         var dateValue = subrec.getSublistValue({
30             sublistId: 'schedule',
31             fieldId: 'trandate',
32             line: 0
33         });
34
35         // Create a variable and initialize it to the
36         // value of the memo field.
37         var memoValue = subrec.getSublistValue({
38             sublistId: 'schedule',
39             fieldId: 'memo',
40             line: 0
41         });
42
43         // Print the retrieved values to the execution log.
44         try {
45             log.debug({
46                 title: 'date value',
47                 details: 'date value: ' + dateValue
48             });
49             log.debug({
50                 title: 'memo value',
51                 details: 'memo value: ' + memoValue
52             });
53         } catch (e) {
54             log.error({
55                 title: e.name,
56                 details: e.message
57             });
58         }
59     }
60     return {
61         afterSubmit: afterSubmit
62     };
63 });

```

Retrieving an Address Subrecord Example

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

This example loads an employee record and selects a line on the address sublist. It retrieves a value from the sublist line that's not part of the subrecord. It also retrieves the address subrecord and reads one of its fields. The script logs both values.

This example uses standard mode, but you can also use dynamic mode to load the subrecord. For general details about using either approach to retrieve a sublist subrecord, see [Using SuiteScript 2.x to Retrieve a Sublist Subrecord](#).

To learn about SuiteScript scripting modes, see [SuiteScript 2.x Standard and Dynamic Modes](#)

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType usereventscript
4   */
5
6  define([ 'N/record' ],function(record) {
7      function afterSubmit(context) {
8          // Load the record.
9          var rec = record.load({
10             type: record.Type.EMPLOYEE,
11             id: 1863,
12             isDynamic: false
13         });
14
15         // Retrieve a value from the sublist line
16         var labelValue = rec.getSublistValue({
17             sublistId: 'addressbook',
18             fieldId: 'label',
19             line: 0
20         });
21
22         // Retrieve the subrecord associated with that same line.
23         var subrec = rec.getSublistSubrecord({
24             sublistId: 'addressbook',
25             fieldId: 'addressbookaddress',
26             line: 0
27         });
28
29         // Create a variable to initialize it to the
30         // value of the subrecord's city field.
31         var cityValue = subrec.getValue({
32             fieldId: 'city'
33         });
34
35         // Print the retrieved values to the execution log.
36         try {
37             log.debug({
38                 title: 'label value',
39                 details: 'label value: ' + labelValue
40             });
41
42             log.debug({
43                 title: 'city value',
44                 details: 'city value: ' + cityValue
45             });
46         } catch (e) {
47             log.error({
48                 title: e.name,
49                 details: e.message
50             });
51         }
52     }
53     return {
54         afterSubmit: afterSubmit
55     }
56 }

```

```

55 | };
56 | });

```

Scripting Subrecords that Occur in Body Fields

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Sometimes, a subrecord is accessed through a body field on a record.

Most subrecords referenced from body fields behave in the same general way. But there are some differences when working with subrecords used as shipping or billing addresses on transactions.

For details, see the following sections:

- [Body Field Subrecords](#)
- [Transaction Shipping and Billing Addresses](#)

Body Field Subrecords

Here are some examples of typical subrecords referenced from body fields:

- An assembly build record may need to include the serial numbers of each assembly being tracked. This info would be stored in an inventory detail subrecord linked to a body field on the assembly build record.
- A subsidiary record may include a main address, a shipping address, and a return address. Each of these addresses is linked to a field on the record's body.
- The company information preferences page includes the preferred main address, shipping address, and return address for your organization. Each of these addresses is linked to a field on the record's body.

For details about working with these types of subrecords, see the following sections:

- [Using SuiteScript 2.x to Create a Body Field Subrecord](#)
- [Using SuiteScript 2.x to Update a Body Field Subrecord](#)
- [Using SuiteScript 2.x to Retrieve a Body Field Subrecord](#)

Transaction Shipping and Billing Addresses

Some transactions have body fields for shipping and billing addresses. Each of these fields holds an address subrecord. Working with these subrecords can be different from other body field subrecords. For details, see [Scripting Transaction Shipping and Billing Addresses](#).

Using SuiteScript 2.x to Create a Body Field Subrecord

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Some records have body fields that reference subrecords. Scripting body field subrecords is a bit different from sublist subrecords.

To create a subrecord, your script must use the [N/record Module](#). You can use either dynamic or standard mode. For details on each approach, see the following sections:

- [Creating a Body Field Subrecord in Dynamic Mode](#)
- [Creating a Body Field Subrecord in Standard Mode](#)



Important: If you are scripting the shipping address or billing address on a transaction, see [Scripting Transaction Shipping and Billing Addresses](#).

Creating a Body Field Subrecord in Dynamic Mode

If your script uses dynamic mode, you can use the following procedure to create a subrecord in a body field.

To learn about SuiteScript scripting modes, see [SuiteScript 2.x Standard and Dynamic Modes](#)

To create a body field subrecord in dynamic mode:

1. To add a new record, create it and set its required body fields. To update an existing record, load it.
2. Create the subrecord using the [Record.getSubrecord\(options\)](#). The method requires one argument: A `fieldId`, which identifies the field on the record holding the subrecord. In the Records Browser, the field that holds the subrecord is always identified as a field of type summary.

For example, you could use an expression like the following to create an inventory detail subrecord on an assembly build record:

```
1 | ...
2 | var inventoryDetailSubrecord = rec.getSubrecord({
3 |   fieldId: 'inventorydetail'
4 | });
5 | ...
```

3. Use the [Record.setValue\(options\)](#) to set body fields on the subrecord. Be aware that not all subrecords have writable body fields.
4. If the subrecord has a sublist, you usually need to add at least one line to the sublist. For each line, use the following guidelines:
 - Create the line with the [Record.selectNewLine\(options\)](#).
 - Set required values on the line with the [Record.setCurrentSublistValue\(options\)](#).
 - Save the subrecord's sublist line with the [Record.commitLine\(options\)](#).

For example, if you were creating an inventory detail subrecord, you could use the following expression to create a line on the subrecord's inventory assignment sublist:

```
1 | ...
2 | inventoryDetailSubrecord.selectNewLine({
3 |   sublistId: 'inventoryassignment',
4 | });
5 |
6 | inventoryDetailSubrecord.setCurrentSublistValue({
7 |   sublistId: 'inventoryassignment',
8 |   fieldId: 'receiptinventorynumber',
9 |   value: '012345'
10 | });
11 |
12 | inventoryDetailSubrecord.commitLine({
13 |   sublistId: 'inventoryassignment'
14 | })
```

```
15 | ...
```

5. Save the record.



Note: For full script example showing how to create a body field subrecord using dynamic mode, see [Creating an Address on a Subsidiary Record Example](#) and [Creating an Inventory Detail Subrecord on a Body Field Example](#).

Creating a Body Field Subrecord in Standard Mode

If your script uses standard mode, use the following procedure to create a subrecord on the body field of a record.

To learn about SuiteScript scripting modes, see [SuiteScript 2.x Standard and Dynamic Modes](#)

To create a body field subrecord in standard mode:

1. If adding a new record, create it and set its required body fields. If updating an existing record, load it.
2. Create the subrecord using the [Record.getSubrecord\(options\)](#) method. This method requires one argument: A fieldId, which identifies the body field on the record that contains the subrecord. In the Records Browser, the field that holds the subrecord is always identified as a field of type summary.

For example, you could use an expression like the following to create an address subrecord on a location record:

```
1 | ...
2 | var addressSubrecord = rec.getSubrecord({
3 |   fieldId: 'mainaddress'
4 | });
5 | ...
```

3. Use the [Record.setValue\(options\)](#) method to set body fields on the subrecord. Be aware that not all subrecords have writable body fields.
4. If the subrecord has a sublist, generally you are required to add at least one line to the sublist. For each line, use the following guidelines:
 - Create the line with the [Record.insertLine\(options\)](#) method.
 - Set required values on the line with the [Record.setSublistValue\(options\)](#) method.

For example, if you were creating an inventory detail subrecord, you could use the following expressions to create a line on the subrecord's inventory assignment sublist:

```
1 | ...
2 | inventoryDetailSubrecord.insertLine({
3 |   sublistId: 'inventoryassignment',
4 |   line: 0
5 | });
6 |
7 | inventoryDetailSubrecord.setSublistValue({
8 |   sublistId: 'inventoryassignment',
9 |   fieldId: 'receiptinventorynumber',
10 |   line: 0,
11 |   value: '12345'
12 | });
13 | ...
```

5. Save the record.

Creating an Address on a Subsidiary Record Example

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following example shows how to create a subsidiary record that includes an address. The address data is in an address subrecord linked to the mainaddress field.

To use this example, you must meet the following prerequisites:

- You must have a OneWorld account.
- The value you use for the subsidiary record's name field must be unique in your system.

This example uses dynamic mode, but you can also use standard mode to add the subrecord. For general details about using either approach, see [Using SuiteScript 2.x to Create a Body Field Subrecord](#).

To learn about SuiteScript scripting modes, see [SuiteScript 2.x Standard and Dynamic Modes](#)

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType UserEventScript
4   */
5
6  define([ 'N/record' ],function(record) {
7      function afterSubmit(context) {
8          var rec = record.create({
9              type: record.Type.SUBSIDIARY,
10             isDynamic: true
11         });
12
13         // Set body fields on the record.
14         rec.setValue({
15             fieldId: 'name',
16             value: 'US Subsidiary'
17         });
18
19         rec.setValue({
20             fieldId: 'state',
21             value: 'CA'
22         });
23
24         // Create the address subrecord.
25         var subrec = rec.getSubrecord({
26             fieldId: 'mainaddress',
27         });
28
29         subrec.setValue({
30             fieldId: 'city',
31             value: 'San Mateo'
32         });
33
34         subrec.setValue({
35             fieldId: 'state',
36             value: 'CA'
37         });
38
39         subrec.setValue({
40             fieldId: 'zip',
41             value: '94403-2511'
42         });
43
44         subrec.setValue({
45             fieldId: 'addr1',
46             value: '2955 Campus Drive'
47         });
48     }

```



```

49     subrec.setValue({
50         fieldId: 'addr2',
51         value: 'Suite 100'
52     });
53
54     // Save the record.
55     try {
56         var recId = rec.save();
57         log.debug({
58             title: 'Record created successfully',
59             details: 'Id: ' + recId
60         });
61     } catch (e) {
62         log.error({
63             title: e.name,
64             details: e.message
65         });
66     }
67 }
68 return {
69     afterSubmit : afterSubmit
70 };
71 });

```

Creating an Inventory Detail Subrecord on a Body Field Example

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following example shows how to create an assembly build record that includes an inventory detail subrecord. In this case, the subrecord is contained in a body field called inventory detail.

To use this example, you must meet the following prerequisites:

- The Advanced Bin / Numbered Inventory Management feature must be enabled at Setup > Company > Enable Features, on the Items & Inventory subtab.
- The item selected for the item body field should be a serialized assembly item.
- The receiptinventorynumber value must be unique in your system.

This example uses dynamic mode, but you can also use standard mode to add the subrecord. For general details about using either approach to add a sublist subrecord, see [Using SuiteScript 2.x to Create a Body Field Subrecord](#).

To learn about SuiteScript scripting modes, see [SuiteScript 2.x Standard and Dynamic Modes](#)

```

1  require(['N/record'], function(record) {
2
3      // Create a lot numbered assembly item.
4      var item = record.create({
5          type: record.Type.LOT_NUMBERED_ASSEMBLY_ITEM,
6          isDynamic: true
7      });
8      item.setValue({
9          fieldId: 'itemid',
10         value: 'lot numbered item'
11     });
12     item.setValue({
13         fieldId: 'taxschedule',
14         value: 2
15     });
16     item.selectNewLine({
17         sublistId: 'member'
18     });
19     item.setCurrentSublistValue({

```

```

20         sublistId: 'member',
21         fieldId: 'item',
22         value: 233
23     });
24     item.commitLine({
25         sublistId: 'member'
26     });
27     var itemId = item.save();
28
29     // Create the assembly build record.
30     var rec = record.create({
31         type: record.Type.ASSEMBLY_BUILD,
32         isDynamic: true
33     });
34
35     // Set body fields.
36     rec.setValue({
37         fieldId: 'subsidiary',
38         value: '1'
39     });
40     rec.setValue({
41         fieldId: 'location',
42         value: '1'
43     });
44     rec.setValue({
45         fieldId: 'item',
46         value: itemId
47     });
48     rec.setValue({
49         fieldId: 'quantity',
50         value: 1
51     });
52
53     // Create the inventory detail subrecord.
54     var subrec = rec.getSubrecord({
55         fieldId: 'inventorydetail'
56     });
57
58     // Create a line on the subrecord's inventory assignment sublist.
59     subrec.selectNewLine({
60         sublistId: 'inventoryassignment',
61     });
62     subrec.setCurrentSublistValue({
63         sublistId: 'inventoryassignment',
64         fieldId: 'receiptinventorynumber',
65         value: '012345'
66     });
67     subrec.setCurrentSublistValue({
68         sublistId: 'inventoryassignment',
69         fieldId: 'quantity',
70         value: 1
71     });
72     subrec.commitLine({
73         sublistId: 'inventoryassignment'
74     });
75
76     // Save the record.
77     var recId = rec.save();
78     log.debug({
79         title: 'Record created successfully',
80         details: 'Id: ' + recId
81     });
82 });

```

Using SuiteScript 2.x to Update a Body Field Subrecord

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

You can edit an existing body field subrecord if the parent record's business logic allows it.

To edit a subrecord, your script must use the [N/record Module](#). You can use either dynamic or standard mode.

To learn about SuiteScript scripting modes, see [SuiteScript 2.x Standard and Dynamic Modes](#)

To update a body field subrecord:

1. Load the record.
2. Retrieve the subrecord with the [Record.getSubrecord\(options\)](#) method. The method requires one argument: A fieldId, which identifies the field on the sublist that contains the subrecord. In the Records Browser, the field that holds the subrecord is always identified as a field of type summary.

For example, you could use an expression like the following to load the inventory detail subrecord associated with an assembly build record:

```
1 | ...
2 | var subrec = call.getSubrecord({
3 |     fieldId: 'inventorydetail'
4 | });
5 | ...
```

3. Use the [Record.setValue\(options\)](#) method update the subrecord's body fields. Be aware that not all subrecords have writable body fields.
4. If the subrecord has a sublist you want to change, you can:
 - If your script uses dynamic mode, use the following steps for each value you want to change:
 1. Identify the line you want to change using the [Record.selectLine\(options\)](#) method.
 2. For each value you want to change, use the [Record.setCurrentSublistValue\(options\)](#) method to identify the field and the new value.
 3. Save your changes to the subrecord's sublist line with the [Record.commitLine\(options\)](#) method.
 - If your script uses standard mode, use the [Record.setSublistValue\(options\)](#) method to update each field that you want to change. This field takes four arguments:
 - A sublistId.
 - A fieldId, which identifies the field on the sublist that contains the subrecord.
 - A line number, which identifies the sublist line that contains the subrecord you want to change.
 - The new value.
5. Save the record with the [Record.save\(options\)](#) method.

Note: For full script example showing how to edit a body field subrecord using dynamic mode, see [Editing a Body Field Address Subrecord Example](#) and [Editing a Body Field Inventory Detail Subrecord Example](#).

Editing a Body Field Address Subrecord Example

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following example loads a subsidiary record. It also loads and edits the address subrecord in the mainaddress field.

You need a OneWorld account to use this script.

For general details on updating body field subrecords, see [Using SuiteScript 2.x to Update a Body Field Subrecord](#).

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType UserEventScript
4   */
5  define([ 'N/record' ],function(record) {
6      function afterSubmit(context) {
7          // Load the record.
8          var rec = record.load({
9              type: record.Type.SUBSIDIARY,
10             id: 1,
11             isDynamic: true
12         });
13
14         // Load the subrecord.
15
16         var subrec = rec.getSubrecord({
17             fieldId: 'mainaddress'
18         });
19
20         // Make changes to one field.
21         subrec.setValue({
22             fieldId: 'addr1',
23             value: '12331-A Riata Trace Parkway'
24         });
25
26         // Save the record.
27         try {
28             var recId = rec.save();
29             log.debug({
30                 title: 'Record updated successfully',
31                 details: 'Id: ' + recId
32             });
33
34         } catch (e) {
35             log.error({
36                 title: e.name,
37                 details: e.message
38             });
39         }
40     }
41     return {
42         afterSubmit : afterSubmit
43     };
44 });

```

Editing a Body Field Inventory Detail Subrecord Example

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following example loads an assembly build record. It also loads the subrecord in the Inventory Detail field and saves a change to it.

To use this example, make sure you meet these requirements:

- The Advanced Bin / Numbered Inventory Management feature must be enabled at Setup > Company > Enable Features, on the Items & Inventory subtab.
- The assembly build record that you load must already have an inventory detail subrecord.
- The new value you choose for receiptinventorynumber must be unique in your system.

For general details on updating body field subrecords, see [Using SuiteScript 2.x to Update a Body Field Subrecord](#).

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType UserEventScript
4   */
5
6  define([ 'N/record' ],function(record) {
7      function afterSubmit(context) {
8          // Load the record.
9          var rec = record.load({
10             type: record.Type.ASSEMBLY_BUILD,
11             id: 4918,
12             isDynamic: true
13         });
14
15         // Load the subrecord.
16
17         var subrec = rec.getSubrecord({
18             fieldId: 'inventorydetail'
19         });
20
21         // Identify a line on the subrecord's sublist.
22
23         subrec.selectLine({
24             sublistId: 'inventoryassignment',
25             line: 0
26         });
27
28         // Make changes to one sublist field.
29
30         subrec.setCurrentSublistValue({
31             sublistId: 'inventoryassignment',
32             fieldId: 'receiptinventorynumber',
33             value: '890123'
34         });
35
36         // Save the record.
37         try {
38             var recId = rec.save();
39
40             log.debug({
41                 title: 'Record updated successfully',
42                 details: 'Id: ' + recId
43             });
44
45         } catch (e) {
46             log.error({
47                 title: e.name,
48                 details: e.message
49             });
50         }
51     }
52     return {
53         afterSubmit: afterSubmit
54     };
55 });

```

Using SuiteScript 2.x to Retrieve a Body Field Subrecord

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

You might need to retrieve data from a subrecord in a body field.

To retrieve a subrecord, your script can use either the [N/record Module](#) or the [N/currentRecord Module](#). You can use either dynamic or standard mode.

To learn about SuiteScript scripting modes, see [SuiteScript 2.x Standard and Dynamic Modes](#)

To use SuiteScript 2.x to retrieve a body field subrecord:

1. Load the record.
2. Get the subrecord using the `getSubrecord()` method. This method requires one argument: A `fieldId`, which identifies the field on the sublist that contains the subrecord. In the Records Browser, the field that holds the subrecord is always identified as a field of type summary.

For example, you could use an expression like the following to load an address subrecord stored in the body field of a location record:

```
1 | ...
2 | var subrec = call.getSubrecord({
3 |   fieldId: 'mainaddress'
4 | });
5 | ...
```

3. To retrieve a value from the subrecord's body, use the `getValue()` method. For example, you could use an expression like the following to retrieve a detail from the body of an address subrecord:

```
1 | ...
2 | var cityValue = subrec.getValue({
3 |   fieldId: 'city'
4 | });
5 | ...
```

4. To retrieve a value from the subrecord's sublist, use the `getSublistValue()` method. For example, you could use an expression like the following to retrieve a detail from the sublist of an inventory detail subrecord:

```
1 | ...
2 | var cityValue = subrec.getValue({
3 |   sublistId: 'inventoryassignment',
4 |   fieldId: 'receiptinventorynumber',
5 |   line: 4,
6 | });
7 | ...
```

Retrieving a Body Field Address Subrecord Example

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Many record types have body fields that reference address subrecords. A similar design exists with the Company Information page, which is accessed at Setup > Company > Company Information. Although it's not a record, you can interact with it like one. Like a record, it has body fields that reference subrecords. It includes `mainaddress`, `shippingaddress`, and `returnaddress` subrecords. The following example shows how to retrieve values from each of these addresses.

Note that this page also has **state** and **country** fields. These fields are set separately from the subrecords. In SuiteScript, you interact with them like any standard body field.

To get the most out of this example, first populate the Company Information page with values in these fields:

- State
- County
- Address
- Shipping address

■ Return address

For general details on retrieving body field subrecords, see [Using SuiteScript 2.x to Update a Body Field Subrecord](#).

```

1  /**
2   *@NApiVersion 2.x
3   *@NScriptType UserEventScript
4   */
5  define(['N/config', 'N/record'], function(config, record) {
6      function afterSubmit(context) {
7          // The Company Information page is not a standard record. You access it by
8          // using the config.load method.
9
10         var companyInfo = config.load({
11             type: config.Type.COMPANY_INFORMATION
12         });
13
14         // Retrieve some of the preferences stored on the main part of the
15         // Company Information page. You interact with these preferences as if
16         // they were body fields.
17
18         var companyName = companyInfo.getValue({
19             fieldId: 'companyname'
20         });
21
22         var employerId = companyInfo.getValue({
23             fieldId: 'employerid'
24         });
25
26         // The Company Information page also includes some address fields on
27         // the body of the record. Retrieve these fields.
28         var state = companyInfo.getValue({
29             fieldId: 'state'
30         });
31
32         var country = companyInfo.getValue({
33             fieldId: 'country'
34         });
35
36         // Retrieve details from the subrecord that represents the main address.
37         // As a first step, instantiate the subrecord.
38         var mainAddress = companyInfo.getSubrecord({
39             fieldId: 'mainaddress'
40         });
41
42         // Retrieve values from the main address subrecord.
43         var mainAddressCity = mainAddress.getValue({
44             fieldId: 'city'
45         });
46
47         var mainAddressState = mainAddress.getValue({
48             fieldId: 'state'
49         });
50
51         // Retrieve details from the subrecord that represents the shipping address.
52         // To start, instantiate the subrecord.
53         var shippingAddress = companyInfo.getSubrecord('shippingaddress');
54
55         // Retrieve values from the subrecord.
56         var shippingAddressCity = shippingAddress.getValue({
57             fieldId: 'city'
58         });
59
60         var shippingAddressState = shippingAddress.getValue({
61             fieldId: 'state'
62         });
63
64         // Retrieve details from the subrecord that represents the return address.
65         // To start, instantiate the shipping address subrecord.
66         var returnAddress = companyInfo.getSubrecord('returnaddress');
67     }

```

```

68      // Retrieve values from the subrecord.
69      var returnAddressCity = returnAddress.getValue({
70          fieldId: 'city'
71      });
72
73      var returnAddressState = returnAddress.getValue({
74          fieldId: 'state'
75      });
76
77      // Write selected details to the log.
78      log.debug ({
79          title: 'mainAddressState',
80          details: mainAddressState
81      });
82
83      log.debug ({
84          title: 'shippingAddressState',
85          details: shippingAddressState
86      });
87
88      log.debug ({
89          title: 'returnAddressState',
90          details: returnAddressState
91      });
92      }
93      return {
94          afterSubmit: afterSubmit
95      };
96  });

```

Scripting Transaction Shipping and Billing Addresses

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

A transaction's shipping address is stored in an address subrecord linked to a body field. The billing address is stored similarly. Working with these subrecords is similar to working with other body field subrecords. However, there are some differences. For details, see the following sections:

- [Addresses Can Be Sourced from Entity Records](#)
- [A Default Address May Be Used](#)
- [New Shipping and Billing Addresses Are Always Custom](#)

Note: See also [About the Address Subrecord](#).

Addresses Can Be Sourced from Entity Records

Many transactions are associated with an entity. The entity record may already have addresses defined on its Address subtab. When working with a transaction, you can use one of these existing addresses as the shipping or billing address.

If you choose to reference an existing address, your script does not have to create an address subrecord or use any subrecord methods. Instead, you can pick an address with a select field that is associated with the summary field. For example, to designate an existing address as the shipping address on a sales order, you use the shipaddresslist select field. This field identifies the subrecord used by the shippingaddress summary field. For an example of how to select an existing address using the shipaddresslist select field, see [Using SuiteScript 2.x to Select an Existing Shipping Address Example](#).

A Default Address May Be Used

Entities can have default shipping or billing addresses. If you create a transaction without setting a shipping or billing address, the entity's default is used.

To override the default, use `removeSubrecord()` to clear the summary field before setting your new subrecord values. Alternatively, set the `shipaddresslist` field to null before setting your new address values. For an example, see [Using SuiteScript 2.x to Create a New Shipping Address Example](#).

New Shipping and Billing Addresses Are Always Custom

When creating a sales order in the UI, you must choose New or Custom in the Ship to Select field to enter a new shipping address. If you select New, the address you enter is also saved to the customer record. If you select Custom, the new address is not saved to the customer record. The address is only saved on the transaction.

In SuiteScript, you don't have the option to choose between New or Custom. Any new shipping or billing address created by your script is treated as a Custom address. The address is saved on the transaction, but it cannot be saved to the entity record.

Using SuiteScript 2.x to Create a New Shipping Address Example

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following example creates a sales order record with one line in the item sublist. It also creates a shipping address for the transaction. It overrides any default shipping address on the customer record.

i Note: For details about working with a transaction's shipping or billing address, see [Scripting Transaction Shipping and Billing Addresses](#).

To use this example, make sure you meet these requirements:

- As already shown, when you create the new shipping address for the sales order, country field is set first (because the example uses dynamic mode – To learn about SuiteScript scripting modes, see [SuiteScript 2.x Standard and Dynamic Modes](#)). For more information, see [About the Address Subrecord](#).

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType UserEventScript
4   */
5
6  define([ 'N/record' ],function(record) {
7      function afterSubmit(context){
8          // Create the record.
9          var rec = record.create({
10             type: record.Type.SALES_ORDER,
11             isDynamic: true
12         });
13
14         // Set body fields on the record.
15         rec.setValue({
16             fieldId: 'entity',
17             value: '2163'
18         });
19
20         rec.setValue({
21             fieldId: 'memo',
22             value: '102A'
23         });
24
25         // Create the subrecord.
26         var subrec = rec.getSubrecord({
27             fieldId: 'shippingaddress'
28         });
29
30         // Set values on the subrecord.
31         // Set country field first when script uses dynamic mode
32         subrec.setValue({

```

```

33     fieldId: 'country',
34     value: 'US'
35   });
36
37   subrec.setValue({
38     fieldId: 'city',
39     value: 'New York'
40   });
41
42   subrec.setValue({
43     fieldId: 'state',
44     value: 'New York'
45   });
46
47   subrec.setValue({
48     fieldId: 'zip',
49     value: '10018'
50   });
51
52   subrec.setValue({
53     fieldId: 'addr1',
54     value: '8 W 40th St.'
55   });
56
57   // Create a line in the item sublist.
58   rec.selectNewLine({
59     sublistId: 'item'
60   });
61
62   rec.setCurrentSublistValue({
63     sublistId: 'item',
64     fieldId: 'item',
65     value: '100'
66   });
67
68   rec.setCurrentSublistValue({
69     sublistId: 'item',
70     fieldId: 'quantity',
71     value: '11'
72   });
73   } catch (e) {
74   }
75
76   rec.commitLine({
77     sublistId: 'item'
78   });
79
80   // Save the record.
81   try {
82     var recId = rec.save();
83     log.debug({
84       title: 'Record created successfully',
85       details: 'Id: ' + recId
86     });
87     log.error({
88       title: e.name,
89       details: e.message
90     });
91   }
92   return {
93     afterSubmit: afterSubmit
94   };
95 });

```

Using SuiteScript 2.x to Select an Existing Shipping Address Example

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script creates a sales order record with one in line in the item sublist. It also sets the value of the shipping address field to the value of an address from the customer record. Because the script

selects an existing address subrecord, it does not use subrecord methods. It identifies the address subrecord by internal ID.

Note: For details about working with a transaction's shipping or billing address, see [Scripting Transaction Shipping and Billing Addresses](#).

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType UserEventScript
4   */
5
6  define([ 'N/record' ],function(record) {
7      function afterSubmit(context) {
8          var rec = record.create({
9              type: record.Type.SALES_ORDER,
10             isDynamic: true
11         });
12
13         // Set body fields on the record.
14         rec.setValue({
15             fieldId: 'entity',
16             value: '110'
17         });
18
19         // Set the shipping address to the value of
20         // an address on the customer record.
21         rec.setValue({
22             fieldId: 'shipaddresslist',
23             value: '395713'
24         });
25
26         // Create one line in the item sublist.
27         rec.selectNewLine({
28             sublistId: 'item'
29         });
30
31         rec.setCurrentSublistValue({
32             sublistId: 'item',
33             fieldId: 'item',
34             value: '100'
35         });
36
37         rec.setCurrentSublistValue({
38             sublistId: 'item',
39             fieldId: 'quantity',
40             value: '3'
41         });
42
43         rec.commitLine({
44             sublistId: 'item'
45         });
46
47         // Save the record.
48         try {
49             var recId = rec.save();
50             log.debug({
51                 title: 'Record created successfully',
52                 details: 'Id: ' + recId
53             });
54         } catch (e) {
55             log.error({
56                 title: e.name,
57                 details: e.message
58             });
59         }
60     }
61     return {
62         afterSubmit: afterSubmit
63     };
64 });

```

Using SuiteScript 2.x to Retrieve a Shipping Address Example

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

This user event script retrieves all fields from a transaction's shipping address.

Deploy this script on a sales order or another transaction with a shippingaddress summary field. After creating a new transaction with a shipping address, the address will be logged in the script deployment record's execution log.

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType UserEventScript
4   */
5
6  define(['N/record'], function(record) {
7      function beforeSubmit(context) {
8          if (context.type == context.UserEventType.CREATE) {
9              var salesOrder = context.newRecord;
10             var shipToAddress = salesOrder.getSubrecord({
11                 fieldId: 'shippingaddress'
12             });
13
14             log.debug({
15                 title: 'shipping address',
16                 details: JSON.stringify(shipToAddress)
17             });
18         }
19     }
20     return {
21         beforeSubmit: beforeSubmit
22     };
23 });

```

Using SuiteScript 2.x to Retrieve one Value from a Shipping Address Example

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following script loads a sales order record. It retrieves a value from the shipping address and logs it.

Note: For details about working with a transaction's shipping or billing address, see [Scripting Transaction Shipping and Billing Addresses](#).

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType ClientScript
4   */
5
6  define(['N/record'], function(record) {
7      function pageInit() {
8          // Load the record.
9          var rec = record.load({
10             type: record.Type.SALES_ORDER,
11             id: 5025,
12             isDynamic: true
13         });
14
15         // Retrieve the subrecord.
16         var subrec = rec.getSubrecord({
17             fieldId: 'shippingaddress'
18         });
19     }
20 });

```

```
20 // Create a variable and initialize it to the
21 // value of the subrecord's city field.
22 var cityValue = subrec.getValue({
23     fieldId: 'city'
24 });
25
26 // Print the value to the execution log.
27 try {
28     log.debug({
29         title: 'city value',
30         details: 'city value: ' + cityValue
31     });
32 } catch (e) {
33     log.error({
34         title: e.name,
35         details: e.message
36     });
37 }
38 }
39 return {
40     pageInit: pageInit
41 }
42 };
```

SuiteScript 2.x Custom Pages

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

You can use SuiteScript 2.x to create custom forms and list pages, and work with UI objects.

You can create custom forms using a Suitelet, Portlet, or user event script that triggers on the `beforeLoad` entry point. You can add buttons, fields, field groups, tabs, sublists, and page links to a custom form. For more information, see [SuiteScript 2.x Custom Forms](#).

You can create custom list pages using a Suitelet or Portlet. You can add buttons, columns, page links, and rows to a custom list page. For more information, see [SuiteScript 2.x Custom List Pages](#).

Creating custom forms and pages lets you build a custom UI that optimizes your NetSuite environment and maintains its look and feel. You have several options to create and manage your custom UI. You can use Customization, SuiteScript 2.x UI Components, or HTML. For more information, see [SuiteScript 2.x Working with UI Objects](#).



Important: SuiteScript only allows access to the NetSuite UI through its APIs, not the Document Object Model (DOM).

SuiteScript 2.x Custom Forms

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

You can create custom forms using a Suitelet, Portlet, or user event script triggered on the `beforeLoad` entry point. Buttons, fields (configured or specialized), field groups, tabs and subtabs, sublists, and page links can all be included on a custom form.

When you create a custom form, you can build a custom UI to optimize your NetSuite environment while maintaining the NetSuite look and feel. You have a few options for creating and managing your custom UI. You can use Customization, SuiteScript 2.x UI Components, or HTML. For more information, see [SuiteScript 2.x Working with UI Objects](#).

Supported Script Types for Custom Form Creation

You can use the following script types to create custom forms:

- **Suitelet:** Suitelets offer the most flexibility for creating forms. With Suitelets, you're in control of the data and can process requests and responses directly. For information about Suitelets, see [SuiteScript 2.x Suitelet Script Type](#).
- **Portlet:** Portlets are rendered within dashboard portlets. For information about portlet scripts, see [SuiteScript 2.x Portlet Script Type](#).
- **Before Load User Event Script:** User event scripts are executed on the server when a user performs certain actions on records. Use a before load user event script to customize an existing form when a user edits a record. For information about before load user event scripts, see [SuiteScript 2.x User Event Script Type](#) and [beforeLoad\(context\)](#).

Supported UI Components for Custom Forms

You can use the following elements in your custom forms: buttons, fields (configured or specialized), field groups, tabs and subtabs, sublists, and page links.

Buttons

Buttons can trigger specific actions. The Submit button saves your changes to a record. You can also attach a client script to a form and trigger it with a custom button.

- For information about adding a custom button to a form, see the help topic [Form.addButton\(options\)](#).
- For information about adding a submit button to a form, see the help topic [Form.addSubmitButton\(options\)](#).

Fields

Add a customized field to a form to collect specific information from a user. Use the [serverWidget.FieldType](#) enumeration to specify the field type. Most field types have built-in error checking to ensure correct formatting.

- For information about adding a custom field to a form, see the help topic [Form.addField\(options\)](#).

Configuring Fields

After you've added a field, you can customize its position on the form by placing them within a field group, or on a tab or subtab.

- For information about field positioning and layout, see [Positioning Fields on Forms](#).
- For information about placing a field on a tab or subtab, see [Steps for Adding a Tab to a Form](#).

You can set various field properties using enum values. Here are some common field properties you can set:

- Use [serverWidget.FieldDisplayType](#) to control how a field is displayed. For example, you can disable, hide, or make fields read-only.
- Use [Field.isMandatory](#) to indicate whether a field is required.
- For other field configuration properties, see the help topic [serverWidget.Field](#).

Specialized Fields

The Secret Key field is a special field that can be used with the [N/crypto Module](#) to perform encryption or hashing.

- For information about using a Secret Key field, see the help topic [Form.addSecretKeyField\(options\)](#).

Credential fields are special text fields for storing credentials like passwords, used to invoke third-party services. You can limit access to Credential fields by domain, script, or user. Credential field data is encrypted, not stored in plain text.

- For information about using a Credential field, see the help topic [Form.addCredentialField\(options\)](#).

Field Groups

Use field groups to organize and manage fields on a form. Some of the properties listed in [serverWidget.FieldGroup](#) can provide additional field group customization.

- For details, see the help topic [Form.addFieldGroup\(options\)](#).

Tabs and Subtabs

You can add tabs or subtabs to a form to organize large groups of fields so that a user can use the form more easily.

- For information about adding a tab to a form, see [Steps for Adding a Tab to a Form](#) and [Form.addTab\(options\)](#).

Sublists

A sublist is a list of child records that you can add to a parent record form.

- For information about adding a sublist to a form, see [Steps for Adding a Sublist to a Form](#), and [Form.addSublist\(options\)](#).

Page Links

Page links on form can be either breadcrumb links or crosslinks. Breadcrumb links display a series of links leading to the location of the current page. Crosslinks help with navigation and can include links like Forward, Back, List, Search, and Customize.

- For more information about page link types, see the help topic [serverWidget.FormPageLinkType](#).
- For more information about adding page links to a form, see the help topic [Form.addPageLink\(options\)](#).

Positioning Fields on Forms

You can add field groups to position fields together with other closely related fields on custom forms. Use the properties in [serverWidget.FieldGroup](#) to add field groups to forms. You also can use [Field.updateLayoutType\(options\)](#) to define the positioning and placement of fields.

Note: Note that some field group properties, like collapsing and hiding, aren't supported on forms.

The following screenshot shows a single column field group above a double column field group.

The screenshot displays a form titled "Customer Information" with a "Submit" button. It contains two distinct field groups, each outlined with a red border. The first group, "User Information", is a single-column layout containing a "Title" dropdown menu (set to "Mr."), and text input fields for "First Name", "Last Name", and "Email". The second group, "Company Information", is a double-column layout. It features a "Company" section with a "Company Name" field and a "Phone Number" field, and a "Website" section with a "Website" field.

Field Layout Type

The `serverWidget.FieldLayoutType` enumerations contain values used by `Field.updateLayoutType(options)` to position fields outside of a field group and on the same row.

Use `OUTSIDE` to position a field outside a field group. You can also use `OUTSIDEABOVE` and `OUTSIDEBELOW` to position a field above or below a field group. The following screenshot displays three text fields positioned outside of a field group by setting these Field Layout Type values.

The screenshot shows a form titled "usergroup Field Group". Inside the group, there are four fields: "Title" (a dropdown menu with "Mr." selected), "First Name" (a text input), "Last Name" (a text input), and "Email" (a text input). Outside the group, there are three text input fields. Two are positioned above the group, labeled "Outside Above Field" and "Outside Field". One is positioned below the group, labeled "Outside Below Field". All three external fields are highlighted with red boxes.

The following code sample illustrates how to set Field Layout values to produce the field placement in the screenshot.

```

1 //Add additional code
2 ...
3 var outsideAbove = form.addField({
4   id: 'outsideabovefield',
5   type: serverWidget.FieldType.TEXT,
6   label: 'Outside Above Field'
7 });
8 outsideAbove.updateLayoutType({
9   layoutType: serverWidget.FieldLayoutType.OUTSIDEABOVE
10 });
11 ...
12 //Add additional code

```

You can use the `STARTROW`, `MIDROW`, and `ENDROW` enumerations to position fields together in the same row. For instance, you can group similar fields like first name and last name together. The following screenshot displays three text fields positioned together using these Field Layout Type values.

The screenshot shows a form titled "usergroup Field Group". Inside the group, there are four fields: "Title" (a dropdown menu with "Mr." selected), "First Name" (a text input), "Last Name" (a text input), and "Email" (a text input). At the bottom of the form, outside the group, there are three text input fields labeled "STARTROW", "MIDROW", and "ENDROW". These three fields are positioned side-by-side and are highlighted with a red box.

The following code sample illustrates how to set Field Layout values to produce the field placement in the screenshot.

```

1 //Add additional code
2 ...
3 var startRow = form.addField({
4   id: 'startrow',
5   type: serverWidget.FieldType.TEXT,
6   label: 'STARTROW',
7   container: 'usergroup'
8 });
9
10 startRow.updateLayoutType({
11   layoutType: serverWidget.FieldLayoutType.STARTROW
12 });
13
14 var midRow = form.addField({
15   id: 'midrow',
16   type: serverWidget.FieldType.TEXT,
17   label: 'MIDROW',
18   container: 'usergroup'
19 });
20
21 midRow.updateLayoutType({
22   layoutType: serverWidget.FieldLayoutType.MIDROW
23 });
24
25 var endRow = form.addField({
26   id: 'endrow',
27   type: serverWidget.FieldType.TEXT,
28   label: 'ENDROW',
29   container: 'usergroup'
30 });
31
32 endRow.updateLayoutType({
33   layoutType: serverWidget.FieldLayoutType.ENDROW
34 });
35
36 ...
37 //Add additional code

```

Field Break Type

The `serverWidget.FieldBreakType` enumeration contains the values set by `Field.updateBreakType(options)` to define how fields are divided across columns and rows on forms.

- Use the `STARTCOL` value to move a field to a new column. The following screenshot shows the difference between the EMAIL field when the break type is set to `NONE` (top image) and when it is set to `STARTCOL` (bottom image).

Note: If the `isSingleColumn` property (for a field group) is set to `true`, then breaking columns doesn't apply. In single column mode, column breaks aren't allowed.

The image displays two screenshots of a SuiteScript 2.x custom form. Both screenshots show a 'usergroup Field Group' containing several fields: 'Title' (a dropdown menu with 'Mr.' selected), 'First Name', 'Last Name', and 'Email'. In the top screenshot, the 'Email' field is highlighted with a red rectangular box. In the bottom screenshot, the 'Email' field is also highlighted with a red rectangular box, but the box contains the text 'FieldBreakType = STARTCOL', indicating the field's configuration.

The following code sample illustrates how to set the `FieldBreakType` values to produce the field placement in the screenshot.

```

1 //Add additional code
2 ...
3 var email = form.addField({
4   id: 'emailfield',
5   type: serverWidget.FieldType.EMAIL,
6   label: 'Email',
7   container: 'usergroup'
8 });
9
10 email.updateBreakType({
11   breakType : serverWidget.FieldBreakType.STARTCOL
12 });
13 ...
14 //Add additional code

```

Use the `STARTROW` value to move a field to a new row. Note that `STARTROW` only works on fields outside of a field group. By default, `NONE` is used, which doesn't move a field to a new

row. For information about how to position a field outside of a field group, see the help topics [serverWidget.FieldLayoutType](#) and [Field.updateLayoutType\(options\)](#).

The following screenshot shows the difference between a field with a break type set to STARTROW and STARTCOL.

The image contains two screenshots of a NetSuite custom form, each showing a 'usergroup Field Group' with fields for Title, First Name, Last Name, and Email. In the top screenshot, a field labeled 'Outside Above Field' with 'FieldBreakType=STARTROW' is shown above the group. In the bottom screenshot, a field labeled 'Outside Above Field' with 'FieldBreakType=STARTCOL' is shown to the left of the group.

The following code sample illustrates how to set the FieldBreakType to STARTROW to produce the field placement in the screenshot.

```

1 //Add additional code
2 ... outsideAbove = form.addField({
3   id: 'outsideabovefield',
4   type: serverWidget.FieldType.TEXT,
5   label: 'OUTSIDEABOVE'
6 });
7
8 outsideAbove.updateLayoutType({
9   layoutType: serverWidget.FieldLayoutType.OUTSIDEABOVE
10 });
11
12 outsideAbove.updateBreakType({
13   layoutType: serverWidget.FieldLayoutType.STARTROW
14 });
15 ...
16 //Add additional code

```

Sample Custom Form Script

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following screenshot displays a form created by a Suitelet. You can use the sample code provided below to create this form.

Steps for Creating a Custom Form with SuiteScript 2.x

First, create a script file. To create this file, do one of the following:

- If you want to copy and paste the completed script directly from this help topic, go to the [Complete Custom Form Script Sample](#).
- If you want to walk through the steps for creating the script, complete the following procedure.

To create a custom form, complete the following steps:

1. First, create a Suitelet and add the necessary JSDoc tags.

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Suitelet
4   */

```

2. Add the define function to load the [N/ui/serverWidget Module](#) module.

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Suitelet
4   */
5  define(['N/ui/serverWidget'],
6         function(serverWidget) {
7  });

```

- To enable a Submit button, create an `onRequest` function. In the `onRequest` function, add an if statement to check if the request method is 'GET'. After the if statement, create a return statement to support the GET request method.

Note: This sample is divided into four sections, as noted in the code comments. Each part shows where to add the corresponding sample code. Section two of the sample code is referenced in [Steps for Adding a Tab to a Form](#). Section three of the sample code is referenced in [Steps for Adding a Sublist to a Form](#) sections.

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Suitelet
4   */
5  define(['N/ui/serverWidget'],
6    function(serverWidget) {
7      function onRequest(context){
8          if(context.request.method === 'GET'){
9              context.response.writePage(form);
10             // Section One - Forms - See "Steps for Creating a Form"
11             // Section Two - Tabs - See "Steps for Adding a Tab to a Form"
12             // Section Three - Sublist - See "Steps for Adding a Sublist to a Form"
13          }else{
14              // Section Four - Output - Used in all sections
15          }
16      }
17      return {
18      }
19  }
20  });

```

- In section one, call the `createForm` method using `serverWidget.createForm(options)`.

```

1  var form = serverWidget.createForm({
2      title: 'Enter Customer Information'
3  });

```

- In section one, create Field Groups to organize the fields using `Form.addFieldGroup(options)`. Use the properties defined in `serverWidget.FieldGroup` to control the appearance and behavior of the field groups.

```

1  var usergroup = form.addFieldGroup({
2      id : 'usergroup',
3      label : 'User Information'
4  });
5  usergroup.isSingleColumn = true;
6
7  var companygroup = form.addFieldGroup({
8      id : 'companygroup',
9      label : 'Company Information'
10 });

```

- In section one, add a Submit button to your form using `Form.addSubmitButton(options)`. Customize the button text by changing the `label` property.

```

1  form.addSubmitButton({
2      label: 'Submit'
3  });

```

- In section one, create the fields you need using `Form.addField(options)`. Don't forget to set the `id`, `type`, and `label` properties correctly. Use the `container` property to specify the field group. Use the properties defined in `serverWidget.Field` to customize your fields. This sample uses `Field.isMandatory` to define several required fields, and `Field.defaultValue` to set the default value of the Company name field.

Some fields, such as the email and phone fields, provide basic error checking to make sure the field contents meet the requirements for the field. For more information, see the help topic [serverWidget.FieldType](#).

Note: When modifying or adding custom fields, use the `custpage_` prefix to the identifier to avoid conflicts with standard fields.

```

1  var select = form.addField({
2      id: 'titlefield',
3      type: serverWidget.FieldType.SELECT,
4      label: 'Title',
5      container: 'usergroup'
6  });
7
8  select.addSelectOption({
9      value: 'Mr.',
10     text: 'Mr.'
11 });
12
13 select.addSelectOption({
14     value: 'MS.',
15     text: 'Ms.'
16 });
17
18 select.addSelectOption({
19     value: 'Dr.',
20     text: 'Dr.'
21 });
22
23 var fname = form.addField({
24     id: 'fnamefield',
25     type: serverWidget.FieldType.TEXT,
26     label: 'First Name',
27     container: 'usergroup'
28 });
29 fname.isMandatory = true;
30
31 var lname = form.addField({
32     id: 'lnamefield',
33     type: serverWidget.FieldType.TEXT,
34     label: 'Last Name',
35     container: 'usergroup'
36 });
37 lname.isMandatory = true;
38
39 form.addField({
40     id: 'emailfield',
41     type: serverWidget.FieldType.EMAIL,
42     label: 'Email',
43     container: 'usergroup'
44 });
45
46 var companyname = form.addField({
47     id: 'companyfield',
48     type: serverWidget.FieldType.TEXT,
49     label: 'Company',
50     container: 'companygroup'
51 });
52 companyname.defaultValue = 'Company Name';
53
54 form.addField({
55     id: 'phonefield',
56     type: serverWidget.FieldType.PHONE,
57     label: 'Phone Number',
58     container: 'companygroup'
59 });
60
61 form.addField({
62     id: 'urlfield',

```

```

63 |     type: serverWidget.FieldType.URL,
64 |     label: 'Website',
65 |     container: 'companygroup'
66 | });

```

8. In section four, define a variable for each field in this sample.

```

1 | var delimiter = /\u0001/;
2 | var titleField = context.request.parameters.titlefield;
3 | var fnameField = context.request.parameters.fnamefield;
4 | var lnameField = context.request.parameters.lnamefield;
5 | var emailField = context.request.parameters.emailfield;
6 | var companyField = context.request.parameters.companyfield;
7 | var phoneField = context.request.parameters.phonefield;
8 | var urlField = context.request.parameters.urlfield;

```

9. In section four, use `context.response.write` to create an output statement. This statement shows how the Submit button works by displaying the field data as output.

```

1 | context.response.write('You have entered:');
2 |   + '<br/> Name: ' + titleField + ' ' + fnameField + ' ' + lnameField
3 |   + '<br/> Email: ' + emailField
4 |   + '<br/> Company: ' + companyField
5 |   + '<br/> Phone: ' + phoneField + ' Website: ' + urlField);

```

Steps for Adding a Tab to a Form

This sample uses the code created in [Sample Custom Form Script](#) to show how to add tab and subtab UI components to a form.

Keep in mind the following limitations when adding tabs to a form:

- You cannot add a single tab. If you create a single tab, the contents of the tab appear on the form without the tab bar.
 - A tab won't show up until you've added an object to the form. After you have added a field, you can set the container property to indicate where the field appears on the tab.
1. In section two, use `Form.addTab(options)` to add two tabs to the form. You can use `Tab.helpText` to add inline help text to guide users.



Note: A value for `tab.id` must be all lowercase and have no spaces. When adding a tab to an existing page, use the `custpage` prefix to avoid conflicts with standard tabs.

```

1 | // Section Two - Tabs
2 |
3 | var tab1 = form.addTab({
4 |   id : 'tab1id',
5 |   label : 'Payment'
6 | });
7 | tab1.helpText = 'Help Text Goes Here';
8 |
9 | var tab2 = form.addTab({
10 |   id : 'tab2id',
11 |   label : 'Inventory'
12 | });

```

2. In section two, use `Form.addSubtab(options)` to add two subtabs. Set the `tab` property to `tab1id` to position the subtab on the first tab you created.

```

1 | form.addSubtab({
2 |   id : 'subtab1id',

```



```

3      label : 'Payment Information',
4      tab: 'tablid'
5  });
6
7  form.addSubtab({
8      id : 'subtab2id',
9      label : 'Transaction Record',
10     tab: 'tablid'
11 });

```

3. In section two, add more fields to your form. To assign a field to a tab, use the container property to indicate the tab where you want to place the field.

The following sample adds a Credential field to the form. Credential fields are text fields used to store credentials in NetSuite. When you submit the form, the credit card number is encrypted and not shown in plain text. For information about using Credential fields, see the help topic [Form.addCredentialField\(options\)](#).

```

1  //Subtab One Fields
2  var ccselect = form.addField({
3      id: 'ccselectfield',
4      type: serverWidget.FieldType.SELECT,
5      label: 'Credit Card',
6      container: 'subtablid'
7  });
8
9  ccselect.addSelectOption({
10     value: 'PayCard0',
11     text: 'Payment Card 0'
12 });
13
14 ccselect.addSelectOption({
15     value: 'PayCard1',
16     text: 'Payment Card 1'
17 });
18
19 ccselect.addSelectOption({
20     value: 'PayCard2',
21     text: 'Payment Card 2'
22 });
23
24
25 var expmonth = form.addField({
26     id: 'expmonth',
27     type: serverWidget.FieldType.SELECT,
28     label: 'Expiry Month',
29     container: 'subtablid'
30 });
31
32 expmonth.updateLayoutType({
33     layoutType: serverWidget.FieldLayoutType.STARTROW});
34
35 expmonth.addSelectOption({
36     value: '01',
37     text: 'Jan'
38 });
39
40 expmonth.addSelectOption({
41     value: '02',
42     text: 'Feb'
43 });
44
45 expmonth.addSelectOption({
46     value: '03',
47     text: 'Mar'
48 });
49
50 expmonth.addSelectOption({
51     value: '04',
52     text: 'Apr'

```

```

53 });
54
55 expmonth.addSelectOption({
56     value: '05',
57     text: 'May'
58 });
59
60 expmonth.addSelectOption({
61     value: '06',
62     text: 'Jun'
63 });
64
65 expmonth.addSelectOption({
66     value: '07',
67     text: 'Jul'
68 });
69
70 expmonth.addSelectOption({
71     value: '08',
72     text: 'Aug'
73 });
74
75 expmonth.addSelectOption({
76     value: '09',
77     text: 'Sep'
78 });
79
80 expmonth.addSelectOption({
81     value: '10',
82     text: 'Oct'
83 });
84
85 expmonth.addSelectOption({
86     value: '11',
87     text: 'Nov'
88 });
89
90 expmonth.addSelectOption({
91     value: '12',
92     text: 'Dec'
93 });
94
95 var expyear = form.addField({
96     id: 'expyear',
97     type: serverWidget.FieldType.SELECT,
98     label: 'Expiry Year',
99     container: 'subtab1id'
100 });
101
102 expyear.updateLayoutType({
103     layoutType: serverWidget.FieldLayoutType.ENDROW});
104
105 expyear.addSelectOption({
106     value: '2020',
107     text: '2020'
108 });
109
110 expyear.addSelectOption({
111     value: '2019',
112     text: '2019'
113 });
114
115 expyear.addSelectOption({
116     value: '2018',
117     text: '2018'
118 });
119
120 var credfield = form.addCredentialField({
121     id: 'credfield',
122     label: 'Credit Card Number',
123     restrictToDomains: 'www.mysite.com',
124     restrictToScriptIds: 'customscript_my_script',
125     restrictToCurrentUser: false,

```

```

126     container : 'subtab1id'
127   });
128   credfield.maxLength = 32;
129
130   // Subtab Two fields
131   form.addField({
132     id: 'transactionfield',
133     type: serverWidget.FieldType.LABEL,
134     label: 'Transaction History - Coming Soon',
135     container: 'subtab2id'
136   });
137
138   //Tab Two fields
139   form.addField({
140     id: 'inventoryfield',
141     type: serverWidget.FieldType.LABEL,
142     label: 'Inventory - Coming Soon',
143     container: 'tab2id'
144   });

```

4. In section four, define a variable for each field you have added to the tabs. In this sample, these variables show how the Submit button works.

```

1 var ccField = context.request.parameters.cctypefield;
2 var ccNumber = context.request.parameters.credfield;
3 var expMonth = context.request.parameters.expmmonth;
4 var expYear = context.request.parameters.expyear;

```

5. In section four, update the output to show the variables when the Submit button is clicked.

```

1 context.response.write('You have entered:'
2   + '<br/> Name: ' + titleField + ' ' + fnameField + ' ' + lnameField
3   + '<br/> Email: ' + emailField
4   + '<br/> Company: ' + companyField
5   + '<br/> Phone: ' + phoneField + ' Website: ' + urlField
6   + '<br/> Credit Card: ' + ccField
7   + '<br/> Number: ' + ccNumber
8   + '<br/> Expiry Date: ' + expMonth + '/' + expYear);

```

Steps for Adding a Sublist to a Form

This sample uses the code created in [Steps for Adding a Tab to a Form](#) to show how to add a sublist to a form.

1. In section three, add a sublist to the form using [Form.addSublist\(options\)](#). Set the tab property to tab2id to position the sublist on the second tab. This is an inline editor sublist. For information about sublist types, see the help topic [serverWidget.SublistType](#).

```

1 var sublist = form.addSublist({
2   id : 'sublistid',
3   type : serverWidget.SublistType.INLINEEDITOR,
4   label : 'Inventory',
5   tab: 'tab2id'
6 });

```

2. In section three, add a button to the sublist using [Sublist.addButton\(options\)](#). In this example, the button doesn't do anything, but you can use the `functionName` property to specify the function triggered when the button is clicked.

```

1 sublist.addButton({
2   id : 'buttonid',
3   label : 'Print ',
4   functionName: '' // Add the function triggered on button click
5 });

```

3. In section three, add fields to the sublist using [Sublist.addField\(options\)](#). For more information about supported field types, see the help topic [serverWidget.FieldType](#).

Note: [Sublist.addField\(options\)](#) does not support the inline HTML field type.

```

1  sublist.addField({
2      id : 'datefieldid',
3      type : serverWidget.FieldType.DATE,
4      label : 'Date'
5  });
6
7  sublist.addField({
8      id : 'productfieldid',
9      type : serverWidget.FieldType.TEXT,
10     label : 'Product'
11 });
12
13 sublist.addField({
14     id : 'qtyfieldid',
15     type : serverWidget.FieldType.INTEGER,
16     label : 'Quantity'
17 });
18
19 sublist.addField({
20     id : 'upfieldid',
21     type : serverWidget.FieldType.CURRENCY,
22     label : 'Unit Cost'
23 });

```

Complete Custom Form Script Sample

Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see the help topic [SuiteScript 2.x Global Objects](#).

The following code creates a Suitelet that generates a custom form containing several field types, tabs, a sublist, and a submit button. For steps to create this script, see [Sample Custom Form Script](#).

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Suitelet
4   */
5  define(['N/ui/serverWidget'], function(serverWidget) {
6      function onRequest(context) {
7          if (context.request.method === 'GET') {
8
9              // Section One - Forms - See 'Steps for Creating a Custom Form' in topic 'Sample Custom Form Script'
10             var form = serverWidget.createForm({
11                 title: 'Customer Information'
12             });
13
14             var usergroup = form.addFieldGroup({
15                 id: 'usergroup',
16                 label: 'User Information'
17             });
18             usergroup.isSingleColumn = true;
19
20             var companygroup = form.addFieldGroup({
21                 id: 'companygroup',
22                 label: 'Company Information'
23             });
24
25             var select = form.addField({
26                 id: 'titlefield',
27                 type: serverWidget.FieldType.SELECT,

```

```

28         label: 'Title',
29         container: 'usergroup'
30     });
31     select.addSelectOption({
32         value: 'Mr.',
33         text: 'Mr.'
34     });
35     select.addSelectOption({
36         value: 'MS.',
37         text: 'Ms.'
38     });
39     select.addSelectOption({
40         value: 'Dr.',
41         text: 'Dr.'
42     });
43
44     var fname = form.addField({
45         id: 'fnamefield',
46         type: serverWidget.FieldType.TEXT,
47         label: 'First Name',
48         container: 'usergroup'
49     });
50     fname.isMandatory = true;
51
52     var lname = form.addField({
53         id: 'lnamefield',
54         type: serverWidget.FieldType.TEXT,
55         label: 'Last Name',
56         container: 'usergroup'
57     });
58     lname.isMandatory = true;
59
60     form.addField({
61         id: 'emailfield',
62         type: serverWidget.FieldType.EMAIL,
63         label: 'Email',
64         container: 'usergroup'
65     });
66
67     var companyname = form.addField({
68         id: 'companyfield',
69         type: serverWidget.FieldType.TEXT,
70         label: 'Company',
71         container: 'companygroup'
72     });
73     companyname.defaultValue = 'Company Name';
74
75     form.addField({
76         id: 'phonefield',
77         type: serverWidget.FieldType.PHONE,
78         label: 'Phone Number',
79         container: 'companygroup'
80     });
81
82     form.addField({
83         id: 'urlfield',
84         type: serverWidget.FieldType.URL,
85         label: 'Website',
86         container: 'companygroup'
87     });
88
89     form.addSubmitButton({
90         label: 'Submit'
91     });
92
93     // Section Two - Tabs - See 'Steps for Adding a Tab to a Form' in topic 'Sample Custom Form Script'
94     var tab1 = form.addTab({
95         id: 'tab1id',
96         label: 'Payment'
97     });
98     tab1.helpText = 'Help Text Goes Here';
99
100    var tab2 = form.addTab({

```

```

101         id: 'tab2id',
102         label: 'Inventory'
103     });
104
105     form.addSubtab({
106         id: 'subtablid',
107         label: 'Payment Information',
108         tab: 'tablid'
109     });
110
111     form.addSubtab({
112         id: 'subtab2id',
113         label: 'Transaction Record',
114         tab: 'tablid'
115     });
116
117     // Subtab One Fields
118     var ccselect = form.addField({
119         id: 'cctypefield',
120         type: serverWidget.FieldType.SELECT,
121         label: 'Credit Card',
122         container: 'subtablid'
123     });
124     ccselect.addSelectOption({
125         value: 'PayCard0',
126         text: 'Payment Card 0'
127     });
128     ccselect.addSelectOption({
129         value: 'PayCard1',
130         text: 'Payment Card 1'
131     });
132     ccselect.addSelectOption({
133         value: 'PayCard2',
134         text: 'Payment Card 2'
135     });
136
137     var expmonth = form.addField({
138         id: 'expmonth',
139         type: serverWidget.FieldType.SELECT,
140         label: 'Expiry Date:',
141         container: 'subtablid'
142     });
143     expmonth.updateLayoutType({
144         layoutType: serverWidget.FieldLayoutType.STARTROW
145     });
146     expmonth.addSelectOption({
147         value: '01',
148         text: 'Jan'
149     });
150     expmonth.addSelectOption({
151         value: '02',
152         text: 'Feb'
153     });
154     expmonth.addSelectOption({
155         value: '03',
156         text: 'Mar'
157     });
158     expmonth.addSelectOption({
159         value: '04',
160         text: 'Apr'
161     });
162     expmonth.addSelectOption({
163         value: '05',
164         text: 'May'
165     });
166     expmonth.addSelectOption({
167         value: '06',
168         text: 'Jun'
169     });
170     expmonth.addSelectOption({
171         value: '07',
172         text: 'Jul'
173     });

```

```

174     expmonth.addSelectOption({
175         value: '08',
176         text: 'Aug'
177     });
178     expmonth.addSelectOption({
179         value: '09',
180         text: 'Sep'
181     });
182     expmonth.addSelectOption({
183         value: '10',
184         text: 'Oct'
185     });
186     expmonth.addSelectOption({
187         value: '11',
188         text: 'Nov'
189     });
190     expmonth.addSelectOption({
191         value: '12',
192         text: 'Dec'
193     });
194
195     var expyear = form.addField({
196         id: 'expyear',
197         type: serverWidget.FieldType.SELECT,
198         label: 'Expiry Year',
199         container: 'subtablid'
200     });
201     expyear.updateLayoutType({
202         layoutType: serverWidget.FieldLayoutType.ENDROW
203     });
204     expyear.addSelectOption({
205         value: '2020',
206         text: '2020'
207     });
208     expyear.addSelectOption({
209         value: '2019',
210         text: '2019'
211     });
212     expyear.addSelectOption({
213         value: '2018',
214         text: '2018'
215     });
216
217     var credfield = form.addCredentialField({
218         id: 'credfield',
219         label: 'Credit Card Number',
220         restrictToDomains: 'www.mysite.com',
221         restrictToScriptIds: 'customscript_my_script',
222         restrictToCurrentUser: false,
223         container: 'subtablid'
224     });
225     credfield.maxLength = 32;
226
227     // Subtab two Fields
228     form.addField({
229         id: 'transactionfield',
230         type: serverWidget.FieldType.LABEL,
231         label: 'Transaction History - Coming Soon',
232         container: 'subtab2id'
233     });
234
235     // Tab Two Fields
236     form.addField({
237         id: 'inventoryfield',
238         type: serverWidget.FieldType.LABEL,
239         label: 'Inventory - Coming Soon',
240         container: 'tab2id'
241     });
242
243     // Section Three - Sublist - See 'Steps for Adding a Sublist to a Form' in topic 'Sample Custom Form Script'
244     var sublist = form.addSublist({
245         id: 'sublistid',
246         type: serverWidget.SublistType.INLINEEDITOR,

```

```

247         label: 'Inline Sublist',
248         tab: 'tab2id'
249     });
250     sublist.addButton({
251         id: 'buttonId',
252         label: 'Print ',
253         functionName: '' // Add the function triggered on button click
254     });
255
256     // Sublist Fields
257     sublist.addField({
258         id: 'datefieldid',
259         type: serverWidget.FieldType.DATE,
260         label: 'Date'
261     });
262
263     sublist.addField({
264         id: 'productfieldid',
265         type: serverWidget.FieldType.TEXT,
266         label: 'Product'
267     });
268
269     sublist.addField({
270         id: 'qtyfieldid',
271         type: serverWidget.FieldType.INTEGER,
272         label: 'Quantity'
273     });
274
275     sublist.addField({
276         id: 'upfieldid',
277         type: serverWidget.FieldType.CURRENCY,
278         label: 'Unit Cost'
279     });
280
281     context.response.writePage(form);
282 } else {
283     // Section Four - Output - Used in all sections
284     var delimiter = /\u0001/;
285     var titleField = context.request.parameters.titlefield;
286     var fnameField = context.request.parameters.fnamefield;
287     var lnameField = context.request.parameters.lnamefield;
288     var emailField = context.request.parameters.emailfield;
289     var companyField = context.request.parameters.companyfield;
290     var phoneField = context.request.parameters.phonefield;
291     var urlField = context.request.parameters.urlfield;
292     var ccField = context.request.parameters.cctypefield;
293     var ccNumber = context.request.parameters.credfield;
294     var expMonth = context.request.parameters.expmnth;
295     var expYear = context.request.parameters.expyear;
296
297     context.response.write('You have entered:'
298         + '<br/> Name: ' + titleField + ' ' + fnameField + ' ' + lnameField
299         + '<br/> Email: ' + emailField
300         + '<br/> Company: ' + companyField
301         + '<br/> Phone: ' + phoneField + ' Website: ' + urlField
302         + '<br/> Credit Card: ' + ccField
303         + '<br/> Number: ' + ccNumber
304         + '<br/> Expiry Date: ' + expMonth + '/' + expYear);
305 }
306 }
307 return {
308     onRequest: onRequest
309 };
310 });

```


SuiteScript 2.x Custom List Pages

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

You can create custom list pages using either a Suitelet or a Portlet. Custom list pages can include buttons, columns, page links, and rows.

Supported Script Types for Custom List Page Creation

You can use the following script types to create custom list pages:

- **Suitelet:** Suitelets offer the most flexibility for creating lists. Suitelets can process requests and responses directly, giving you control over the data included in a list. For information about Suitelets, see [SuiteScript 2.x Suitelet Script Type](#).
- **Portlet:** Portlets are rendered within dashboard portlets. For information about portlet scripts, see [SuiteScript 2.x Portlet Script Type](#).

Supported UI Components for Custom List Pages

Your custom list page can include buttons, columns, page links, and rows.

Buttons

Buttons let you trigger custom functions on your list page.

- For information about adding a button, see the help topic [List.addButton\(options\)](#).

Columns

Columns display record values in editable or read-only fields. You can define column properties using the field types listed in [serverWidget.FieldType](#). An Edit column is a special column that adds links to edit and view each record or row in the list.

- For information about adding columns to a list page, see the help topic [List.addColumn\(options\)](#).
- For information about adding an edit column, see the help topic [List.addEditColumn\(options\)](#).

Note: URL columns let you add a website to each row. Use [ListColumn.setURL\(options\)](#) to specify the base URL for the website, and [ListColumn.addParamToURL\(options\)](#) to assign additional properties to a URL column.

Page Link

List pages can have two types of page links: breadcrumb links and crosslinks. Breadcrumb links display a series of links leading to the location of the current page. Crosslinks help with navigation and can include links like Forward, Back, List, Search, and Customize.

- For information about page link types, see the help topic [serverWidget.FormPageLinkType](#).
- For information about adding page links, see the help topic [List.addPageLink\(options\)](#).

Rows

Each row in a list represents a single record. You can add one or more rows to a custom list page by specifying the column ID and value for each row.

- For more information about adding a single row, see the help topic [List.addRow\(options\)](#).
- For more information about adding multiple rows, see the help topic [List.addRows\(options\)](#).

Sample

```

1  /**
2   * @NScriptType Suitelet
3   * @NApiVersion 2.x
4   */
5   define([ 'N/ui/serverWidget'], function(serverWidget) {
6       return {
7           onRequest : function(context) {
8               var list = serverWidget.createList({
9                   title: "List"
10              });
11              list.addColumn({
12                  id: 'column1',
13                  type: serverWidget.FieldType.TEXT,
14                  label: 'Text',
15                  align: serverWidget.LayoutJustification.LEFT
16              });
17              context.response.writePage(list);
18          }
19      });

```

Sample Custom List Page Script

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

The following screenshot displays a list page created by a Suitelet. You can use the sample code provided below to create this list yourself.

Purchase History				
Test				
Purchase History				
Date	Product	Quantity	Unit Cost	
05/30/2018	Widget	4	4.50	
05/30/2018	Sprocket	6	11.50	
05/30/2018	Gizmo	9	1.25	

Steps for Creating a Custom List Page with SuiteScript 2.x

Before proceeding, you must create a script file. To create this file, you can do one of the following:

- If you want to copy and paste the completed script directly from this help topic, go to [Complete Custom List Page Sample Script](#).
- If you want to walk through the steps for creating the script, complete the following procedure.

Here's how to create a list page:

1. Create a Suitelet and add the necessary JSDoc tags.

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Suitelet
4   */

```

2. Add the define function to load the [N/ui/serverWidget Module](#) module.

```

1  /**
2   *@NApiVersion 2.x
3   *@NScriptType Suitelet
4   */
5   define(['N/ui/serverWidget'],
6     function(serverWidget) {
7   });

```

3. To enable a Submit button, create an onRequest function. In the onRequest function create an if statement for the context request method, and set it to 'GET'. After the if statement, create a return statement to support the GET request method.



Note: This sample is divided into two sections, as noted in the code comments. Each part shows where to add the corresponding sample code.

```

1  /**
2   *@NApiVersion 2.x
3   *@NScriptType Suitelet
4   */
5   define(['N/ui/serverWidget'],
6     function(serverWidget) {
7       function onRequest(context){
8         if(context.request.method === 'GET'){
9           // Section One - List - See 'Steps for Creating a List', Step Five
10          // Section Two - Columns - See 'Steps for Creating a List', Step Seven
11          context.response.writePage(list);
12        }else{
13        }
14      }
15      return {
16        onRequest: onRequest
17      }
18    }
19  });

```

4. In section one, create a list page using [serverWidget.createList\(options\)](#). You can use [serverWidget.ListStyle](#) to define the style of the list page.

```

1  var list = serverWidget.createList({
2    title: 'Purchase History'
3  });
4  list.style = serverWidget.ListStyle.REPORT;

```

5. In section one, below the declaration of the list, use [List.addButton\(options\)](#) to create a button on the list. Use the `functionName` property to call the function triggered when the button is clicked.

```

1  list.addButton({
2    id : 'buttonid',
3    label : 'Test',
4    functionName: '' // the function called when the button is pressed
5  });

```

6. In section two, use [List.addColumn\(options\)](#) to define the columns in the list. The column type is specified according to the [serverWidget.FieldType](#) enumeration.

Use the `align` property to justify the position of the column.



Important: Note that not all field types are supported by [List.addColumn\(options\)](#), see the help topic [serverWidget.FieldType](#).

```

1  var datecol = list.addColumn({

```

```

2      id : 'column1',
3      type : serverWidget.FieldType.DATE,
4      label : 'Date',
5      align : serverWidget.LayoutJustification.RIGHT
6    });
7
8    list.addColumn({
9      id : 'column2',
10     type : serverWidget.FieldType.TEXT,
11     label : 'Product',
12     align : serverWidget.LayoutJustification.RIGHT
13   });
14
15   list.addColumn({
16     id : 'column3',
17     type : serverWidget.FieldType.INTEGER,
18     label : 'Quantity',
19     align : serverWidget.LayoutJustification.RIGHT
20   });
21
22   list.addColumn({
23     id : 'column4',
24     type : serverWidget.FieldType.CURRENCY,
25     label : 'Unit Cost',
26     align : serverWidget.LayoutJustification.RIGHT
27   });

```

7. In section two, use [List.addRow\(options\)](#) to define a single row on the list. Or use [List.addRows\(options\)](#) to define multiple rows at one time.

```

1 list.addRow({
2   row : { column1 : '02/12/2018', column2 : 'Widget', column3: '4', column4:'4.50' }
3 });

```

```

1 list.addRows({
2   rows :[{column1 : '02/12/2018', column2 : 'Widget', column3: '4', column4:'4.50'},
3     {column1 : '02/14/2018', column2 : 'Sprocket', column3: '6', column4:'11.50'},
4     {column1 : '02/16/2018', column2 : 'Gizmo', column3: '9', column4:'1.25'}]
5 });

```

Complete Custom List Page Sample Script

Note: This script sample uses the define function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the require function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see the help topic [SuiteScript Debugger](#).

The following code creates a Suitelet that generates a custom list page. For steps to create this script, see [Steps for Creating a Custom List Page with SuiteScript 2.x](#).

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Suitelet
4   */
5  define(['N/ui/serverWidget'], function(serverWidget) {
6    function onRequest(context){
7      if(context.request.method === 'GET'){
8        // Section One - List - See 'Steps for Creating a Custom List Page', Step Five
9        var list = serverWidget.createList({
10          title: 'Purchase History'
11        });
12
13        list.style = serverWidget.ListStyle.REPORT;
14      }
15    }
16  });

```

```

15     list.addButton({
16         id: 'buttonid',
17         label: 'Test',
18         functionName: '' // the function called when the button is pressed
19     });
20
21     // Section Two - Columns - See 'Steps for Creating a Custom List Page', Step Seven
22     var datecol = list.addColumn({
23         id: 'column1',
24         type: serverWidget.FieldType.DATE,
25         label: 'Date',
26         align: serverWidget.LayoutJustification.RIGHT
27     });
28
29     list.addColumn({
30         id: 'column2',
31         type: serverWidget.FieldType.TEXT,
32         label: 'Product',
33         align: serverWidget.LayoutJustification.RIGHT
34     });
35
36     list.addColumn({
37         id: 'column3',
38         type: serverWidget.FieldType.INTEGER,
39         label: 'Quantity',
40         align: serverWidget.LayoutJustification.RIGHT
41     });
42
43     list.addColumn({
44         id: 'column4',
45         type: serverWidget.FieldType.CURRENCY,
46         label: 'Unit Cost',
47         align: serverWidget.LayoutJustification.RIGHT
48     });
49
50     list.addRows({
51         rows: [{column1: '05/30/2018', column2: 'Widget', column3: '4', column4: '4.50'},
52               {column1: '05/30/2018', column2: 'Sprocket', column3: '6', column4: '11.50'},
53               {column1: '05/30/2018', column2: 'Gizmo', column3: '9', column4: '1.25'}]
54     });
55     context.response.writePage(list);
56 }
57 }
58
59 return {
60     onRequest: onRequest
61 }
62 });

```

SuiteScript 2.x Working with UI Objects

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

SuiteScript UI Objects let you build custom UIs and assistants. For more information, see:

- [Custom UI Development](#) – lets you build a custom user interface to optimize your NetSuite environment.
- [UI Component Overview](#) – a collection of objects you can use to customize NetSuite with using SuiteScript 2.x.
- [SuiteScript 2.x UI Modules](#) – API modules for building a custom UI with SuiteScript 2.x
- [Using HTML](#) – lets you use HTML to add elements you can't find in the NetSuite UI components.
- [Creating Custom Assistants](#) – lets you create custom assistants (also called wizards) with a series of steps that users follow to complete a bigger task.

Custom UI Development

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

You can build a custom user interface to optimize your NetSuite environment and keep the NetSuite look and feel. This lets you create the forms and pages your organization needs. There are several ways to create and manage your custom UI, so you can choose what works best for you. You can use Customization, SuiteScript 2.x UI components, and HTML in Suitelets.

Customization

Customization is a point-and-click tool that lets you change the look and feel of your user interface. It gives you plenty of options without needing to know SuiteScript. You can use Customization to change forms by reorganizing fields in field groups or subtabs, or to hide and show fields.

Customization is easier than developing with SuiteScript, but it's less flexible and only lets you modify or copy existing forms.

For more information, see the help topic [Customization Overview](#).

SuiteScript 2.x UI Components

SuiteScript 2.x UI components are objects and methods in the [N/ui/serverWidget Module](#) API module. SuiteScript 2.x gives you lots of control and flexibility to create custom pages and forms. You can also place and position UI components, like fields, with more precision. This is useful when you're designing forms for custom preprinted paper.

Additionally, SuiteScript lets you automate things like validations, calculations, or creating and updating records.

SuiteScript gives you more control than Customization, but it's harder to learn and doesn't let you use existing form templates.

For more information, see [UI Component Overview](#).

HTML

NetSuite lets you use inline and embedded HTML to customize your user interface. You can also build your entire UI with HTML components and embed it into a Suitelet for processing. HTML is useful for creating pages and forms that meet specific brand needs, but it's more difficult to process and manage form data from HTML than from a NetSuite UI Component.

For more information, see [Using HTML](#).

UI Component Overview

i Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

UI Components are objects you can use to customize your NetSuite experience with SuiteScript 2.x. With server-side scripts, you can create custom UIs and still keep the NetSuite look and feel. SuiteScript UI components are generated on the server and accessible through client scripts.

i Note: SuiteScript does not support direct access to the NetSuite UI through the Document Object Model (DOM). NetSuite UI should only be accessed using SuiteScript APIs.

You can customize the NetSuite user interface using four script types:

- **Workflow Action** scripts lets you manipulate custom workflow actions that are defined on a record in a workflow. For more information, see [SuiteScript 2.x Workflow Action Script Type](#).
- **Suitelets** are the most commonly used script type because they provide the most flexibility and functionality. You can use Suitelets to create forms, lists, assistants, and support in-line HTML. For more information, see [SuiteScript 2.x Suitelet Script Type](#).
- **Portlets**, like Suitelets, let you create forms, lists, and display links or inline HTML. Unlike Suitelets, Portlets only show up on Dashboard pages and can't be displayed on their own. For more information, see [SuiteScript 2.x Portlet Script Type](#).
- **User Event** scripts can also be used to change custom forms. For more information, see [SuiteScript 2.x User Event Script Type](#).

This guide focuses on developing custom user interfaces using Suitelets.

Page Types

When you create a custom UI with a Suitelet, you can use three types of pages (Form, List, and Assistant) combined with the standard NetSuite UI components and HTML to build what you need.

- **Form** – A form displays information in NetSuite. Forms have fields that show or collect user data, which can then be submitted and processed by a Suitelet. You can also use forms to show or output information stored in a record.
For information about forms, supported UI components, and how to create a form, see [SuiteScript 2.x Custom Forms](#).
For information about the API module, see the help topic [serverWidget.Form](#).
- **List** – A list page shows information in a table with rows and columns. Lists can show search results and let users update or edit data.
For information about how to create a list, see [SuiteScript 2.x Custom List Pages](#).
For information about the API module, see the help topic [serverWidget.List](#).
- **Assistant** – An assistant is a wizard with multiple steps to help users finish a bigger goal. Each step covers part of the task.
For information about how to create an assistant, see [Creating Custom Assistants](#).
For information about the API module, see the help topic [serverWidget.Assistant](#).

Standard UI Components

There are four main NetSuite UI components you can use to build a page, list, or assistant. These components are standard in NetSuite, though their properties may change depending on the page type.

- **Button** – A custom button is something you can add to your UI. You control its visibility, label, and what it does.
For information about the API module, see the help topic [serverWidget.Button](#).
- **Field** – Custom fields let you record and display information in your UI.
For more information about types of fields, see the help topic [serverWidget.FieldType](#).
For information about the API module, see the help topic [serverWidget.Field](#).
- **Tab** – A tab is a special section in your UI. Using multiple tabs lets you show more information on a single page.
For information about the API module, see the help topic [serverWidget.Tab](#).
- **Sublist** – A sublist is a list you can embed on your page. Custom sublists can show information from a saved search.

For information about the API module, see the help topic [serverWidget.Sublist](#).

Properties and Enumerations

Each UI component has properties and methods you can use to customize its functionality and appearance. These configuration values are stored as enumerations and give you plenty of flexibility when designing your user interface.

Examples of enumerations and properties include:

- The [serverWidget.FieldType](#) enumerations specify the field type and define how they behave on your form.
- The [serverWidget.FieldLayoutType](#) enumerations help you position a field outside a field group or on the same row.
- The [serverWidget.FieldBreakType](#) enumerations control field break types.
- The [Assistant.isNotOrdered](#) property specifies if your assistant is sequential or non-sequential.

SuiteScript 2.x UI Modules

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

There are three API modules you can use to build a custom user interface with SuiteScript 2.x:

- [N/ui/serverWidget Module](#) – The serverWidget module has UI components for working with user interfaces in NetSuite. You can use it to create custom pages, forms, lists, and widgets with the NetSuite look and feel.
- [N/ui/message Module](#) – The message module lets you display and manage messages at the top of your user interface.
- [N/ui/dialog Module](#) – The dialog module lets you create modal dialog boxes to present options or alerts. It uses JavaScript promises to manage dialogs asynchronously.

Using HTML

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Using HTML in a Suitelet lets you add elements that aren't available in the NetSuite UI components. With HTML, you can customize and style your pages to match your organization's brand.

Inline HTML

Use HTML to define a custom field on pages generated by Suitelets. For more information, see [SuiteScript 2.x Suitelet Script Type](#).

To add HTML to your Suitelet form, use [Form.addField\(options\)](#) to add a field, set the field type to `INLINEHTML`, and use [Field.defaultValue](#) to set the HTML value.

HTML input is supported only in inline HTML fields.

```
1 var htmlImage = form.addField({
2     id: 'custpage_htmlfield',
3     type: serverWidget.FieldType.INLINEHTML,
4     label: 'HTML Image'
5 });
6 htmlImage.defaultValue = "<img src='https://<accountID>.app.netsuite.com/images/logos/netsuite-oracle.svg'
  alt='NetSuite-Oracle logo'>"
```




Important: NetSuite Forms and SuiteScript don't support direct access to the NetSuite UI through the Document Object Model (DOM). You should access the NetSuite UI only through SuiteScript APIs. Inline HTML fields on Form pages, except Suitelet Forms, will be deprecated in a future release. If you need to load content from external sources in Inline HTML fields, you must implement Subresource Integrity. For more information, see [Subresource Integrity](#).

HTML Suitelets

HTML Suitelets use HTML elements to build a UI rather than NetSuite UI components. The HTML is embedded in the Suitelet with the [https.get\(options\)](#) method from the [N/https Module](#), and the Suitelet manages the data returned from the form. Unlike a custom form, which returns prebuilt UI objects, an HTML Suitelet returns a string with the HTML code, so manipulating and passing data is difficult. Because HTML is stored as a string, to pass data to or from an HTML Suitelet, you need to modify and update values in the string. The more values you use, the more complex the string manipulation will be.

The following example shows a simple volunteer sign-up sheet using an HTML Suitelet. You can look at this example in two parts.

UI Built in HTML

The UI is managed entirely by HTML and uses standard HTML form elements instead of NetSuite UI components.

The following code sample and screenshot illustrate the form UI built in HTML.

```

1 <form method="post" class="form-horizontal" action="https://LinkToSuitelet.js">
2   <table>
3     <tbody><tr>
4       <td>First Name</td>
5       <td class="col-md-8"><input class="form-control" id="firstname" placeholder="First Name" name="firstname" re
6     quired="" type="text"></td>
7     </tr>
8     <tr>
9       <td>Last Name</td>
10      <td class="col-md-8"><input class="form-control" id="lastname" placeholder="Last Name" name="lastname" re
11    quired="" type="text"></td>
12    </tr>
13    <tr>
14      <td>Email</td>
15      <td class="col-md-8"><input class="form-control" id="email" placeholder="email" name="email" required="" type="e
16    mail"></td>
17    </tr>
18    <tr>
19      <td>Facebook URL</td>
20      <td class="col-md-8"><input class="form-control" id="cuententity_fb_url" placeholder="Facebook" name="cuententity_f
21    b_url" required="" type="text"></td>
22    </tr>
23    <tr>
24      <td colspan="2">
25        <input name="company" value="{{sponsoid}}" type="hidden">
26        <input name="project" value="{{projectid}}" type="hidden">
27      </td>
28    </tr>
29  </tbody></table>
30  <br>
31  <button type="submit" class="btn btn-inverse">Sign Up as a Volunteer</button>

```

```
24 | </form>
```

Form Processing Managed by the Suitelet

The Suitelet manages the information collected from the HTML form. This Suitelet uses the [https.get\(options\)](#) method from the [N/https Module](#) to get the content from the HTML page, and uses methods from the [N/record Module](#), [N/email Module](#), and [N/search Module](#) to collect, process, and respond to user data.

```
1  /**
2  *
3  * @NApiVersion 2.x
4  * @NScriptType Suitelet
5  *
6  */
7
8  define(['N/https', 'N/record', 'N/email', 'N/search'],
9
10     function callbackFunction(https, record, email, search) {
11
12         function getFunction(context) {
13
14             var contentRequest = https.get({
15                 url: "https://LinkToFormPage.html"
16             });
17             var contentDocument = contentRequest.body;
18             var sponsorid = context.request.parameters.sponsorid;
19
20             if (sponsorid) {
21                 contentDocument = contentDocument.replace("{{sponsorid}}", sponsorid);
22                 log.debug("Setting Sponsor", sponsorid)
23             }
24
25             var projectid = context.request.parameters.projectid;
26
27             if (projectid) {
28                 contentDocument = contentDocument.replace("{{projectid}}", projectid);
29                 log.debug("Setting Project", projectid);
30             }
31
32             context.response.write(contentDocument);
33         }
34
35         function postFunction(context) {
36
37             var params = context.request.parameters;
38
39             var emailString = "First Name: {{firstname}}\nLast Name: {{lastname}}\nEmail: {{email}}\nFacebook URL: {{custentity_fb_url}}\n";
40
41             var contactRecord = record.create({
42                 type: "contact",
43                 isDynamic: true
44             });
45
46             for (param in params) {
47                 if (param === "company") {
48                     if (params[param] !== "{{sponsorid}}") {
49                         contactRecord.setValue({
50                             fieldId: param,
51                             value: params[param]
52                         });
53                     }
54                     var lkpfld = search.lookupFields({
55                         type: "vendor",
56                         id: params["company"],
57                         columns: ["entityid"]
58                     });
59                     emailString += "\nSponsor: " + lkpfld.entityid;
60                 }
61                 else {
```

```

61         contactRecord.setValue({
62             fieldId: "custentity_sv_shn_isindi",
63             value: true
64         })
65     }
66 }
67 else {
68     if (param !== "project") {
69         contactRecord.setValue({
70             fieldId: param,
71             value: params[param]
72         });
73         var replacer = "{{" + param + "}}";
74         emailString = emailString.replace(replacer, params[param]);
75     }
76 }
77 }
78
79 var contactId = contactRecord.save({
80     ignoreMandatoryFields: true,
81     enableSourcing: true
82 });
83
84 log.debug("Record ID", contactId);
85
86 if (params["project"]
87 ["project"] !== "{{projectid}}") {
88
89     var lkpfld = search.lookupFields({
90         type: "job",
91         id: params["project"],
92         columns: ["companyname"]
93     });
94
95     emailString += "\nProject Name: " + lkpfld.companyname;
96
97     var participationRec = record.create({
98         type: "customrecord_project_participants",
99         isDynamic: true
100     });
101
102     participationRec.setValue({
103         fieldId: "custrecord_participants_volunteer",
104         value: contactId
105     })
106
107     participationRec.setValue({
108         fieldId: "custrecord_participants_project",
109         value: params["project"]
110     })
111
112     var participationId = participationRec.save({
113         enableSourcing: true,
114         ignoreMandatoryFields: true
115     })
116 }
117
118 log.debug("Email String", emailString);
119
120 email.send({
121     author: -5,
122     recipients: 256,
123     subject: "New Volunteer Signed Up",
124     body: "A new volunteer has joined:\n\n" + emailString
125 });
126
127 email.send({
128     author: -5,
129     recipients: params["email"],
130     subject: "Thank you!",
131     body: "Thank you for volunteering:\n\n" + emailString
132 });
133

```

```

134     var contentRequest = https.get({
135         url: "https://LinkToFormCompletePage.html"
136     });
137
138     var contentDocument = contentRequest.body;
139
140     context.response.write(contentDocument);
141
142 }
143 function onRequestFxn(context) {
144
145     if (context.request.method === "GET") {
146         getFunction(context)
147     }
148     else {
149         postFunction(context)
150     }
151
152 }
153 return {
154     onRequest: onRequestFxn
155 };
156 });

```

Note: You can simplify creating HTML Suitelets by using an HTML templating engine. NetSuite servers run FreeMarker, a Java library that generates text outputs from templates, to create dynamic pages and forms. For more information about FreeMarker, see <http://freemarker.org/docs/index.html>

Creating Custom Assistants

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

You can use the assistant page type to create custom assistants. An assistant (also called a wizard) has a series of steps that users complete to finish a larger task.

For examples of the NetSuite built-in assistants, see the help topic [Understanding NetSuite Assistants](#).

You build assistants with Suitelets and the [N/ui/serverWidget Module](#).

Note: You can't use the assistant on externally available Suitelets.

Important: SuiteScript doesn't support direct access to the NetSuite UI through the Document Object Model (DOM). You should only access the NetSuite UI using SuiteScript APIs.

Assistant Creation Process

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

To develop an assistant, complete the following steps:

1. **Create a new assistant and define steps.**
Create a new assistant in the Suitelet and define each step.
2. **Build assistant pages.**
Use fields, field groups, and sublists to build the assistant pages for each step.
3. **Process assistant pages.**
For each assistant page, construct a response to the user's navigation. At a minimum, render each step or page with a GET request. You can also use a POST request to process the user's data before redirecting them to another step.

Supported UI Components for Assistants

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

Use the following UI components to build a custom assistant.

Fields

Use fields to display and collect data. Fields are added to the current step as needed. For more information, see the help topic [Assistant.addField\(options\)](#).

Field Groups

Use field groups to organize fields in a step. For more information, see the help topic [Assistant.addFieldGroup\(options\)](#).

Field groups can be collapsed to save space on the page. For more information, see the help topics [FieldGroup.isCollapsed](#) and [FieldGroup.isCollapsible](#).

Sublists

Use a sublist to show the results of a saved search.

Note: INLINEEDITOR is the only sublist type supported by the assistant page type.

For more information, see the help topic [Assistant.addSublist\(options\)](#).

Buttons

Unlike form and list page types, you don't need to add buttons to an assistant. Buttons are generated automatically.

Steps

Steps are the main UI components in an assistant because they define each page. You can set step sequence, placement, and positioning with these properties.

- To add a step to your assistant, see the help topic [Assistant.addStep\(options\)](#).
- To indicate whether the steps must be completed in a particular order, see the help topic [Assistant.isNotOrdered](#).

Note: Ordered steps appear vertically along the left side. Unordered steps show up horizontally at the top of the assistant.

- To add help text to your steps, see the help topic [AssistantStep.helpText](#).

Sample Custom Assistant Script

Applies to: SuiteScript 2.x | APIs | SuiteCloud Developer

For more information about custom assistant scripts, see [Creating Custom Assistants](#). Note that assistants cannot be used on externally available Suitelets.

The following screenshots show the output of the sample custom assistant script in this help topic.

Note: This script sample uses the `define` function, which is required for an entry point script (a script you attach to a script record and deploy). You must use the `require` function if you want to copy the script into the SuiteScript Debugger and test it. For more information, see the help topic [SuiteScript 2.x Global Objects](#).

The following sample creates an assistant with two steps. In the first one, you can select a new supervisor for the employee selected. The second step displays the selection and submits it.

```

1  /**
2   * @NApiVersion 2.x
3   * @NScriptType Suitelet
4   */
5   define(['N/ui/serverWidget'], function(serverWidget) {
6       return {
7           onRequest: function(context) {
8               var assistant = serverWidget.createAssistant({
9                   title: 'New Supervisor',
10                  hideNavBar: true
11              });
12              var assignment = assistant.addStep({
13                  id: 'assignment',
14                  label: 'Select New Supervisor'

```

```

15     });
16     var review = assistant.addStep({
17         id: 'review',
18         label: 'Review and Submit'
19     });
20
21     var writeAssignment = function() {
22         assistant.addField({
23             id: 'newsupervisor',
24             type: serverWidget.FieldType.SELECT,
25             label: 'Name',
26             source: 'employee'
27         });
28         assistant.addField({
29             id: 'assignedemployee',
30             type: serverWidget.FieldType.SELECT,
31             label: 'Employee',
32             source: 'employee'
33         });
34     }
35
36     var writeReview = function() {
37         var supervisor = assistant.addField({
38             id: 'newsupervisor',
39             type: serverWidget.FieldType.TEXT,
40             label: 'Name'
41         });
42         supervisor.defaultValue = context.request.parameters.inpt_newsupervisor;
43         supervisor.updateDisplayType({
44             displayType: serverWidget.FieldDisplayType.INLINE
45         });
46
47         var employee = assistant.addField({
48             id: 'assignedemployee',
49             type: serverWidget.FieldType.TEXT,
50             label: 'Employee' });
51         employee.defaultValue = context.request.parameters.inpt_assignedemployee;
52         employee.updateDisplayType({
53             displayType: serverWidget.FieldDisplayType.INLINE
54         });
55     }
56
57     var writeResult = function() {
58         var supervisor = context.request.parameters.newsupervisor;
59         var employee = context.request.parameters.assignedemployee;
60         context.response.write('Supervisor: ' + supervisor + '\nEmployee: ' + employee);
61     }
62
63     var writeCancel = function() {
64         context.response.write('Assistant was cancelled');
65     }
66
67     if (context.request.method === 'GET') //GET method means starting the assistant
68     {
69         writeAssignment();
70         assistant.currentStep = assignment;
71         context.response.writePage(assistant)
72     } else //POST method - process step of the assistant
73     {
74         if (context.request.parameters.next === 'Finish') //Finish was clicked
75             writeResult();
76         else if (context.request.parameters.cancel) //Cancel was clicked
77             writeCancel();
78         else if (assistant.currentStep.stepNumber === 1) { //transition from step 1 to step 2
79             writeReview();
80             assistant.currentStep = assistant.getNextStep();
81             context.response.writePage(assistant);
82         } else { //transition from step 2 back to step 1
83             writeAssignment();
84             assistant.currentStep = assistant.getNextStep();
85             context.response.writePage(assistant);
86         }
87     }

```

```
88 |     }  
89 |     };  
90 | });
```


Transitioning from SuiteScript 1.0 to SuiteScript 2.x

Applies to: SuiteScript 1.0 | SuiteScript 2.x | APIs | SuiteCloud Developer

Oracle NetSuite encourages you to move from SuiteScript 1.0 to SuiteScript 2.x. If you are using SuiteScript 1.0 for your scripts, consider converting these scripts to SuiteScript 2.0 or SuiteScript 2.1. SuiteScript 1.0 scripts continue to be supported, however, SuiteScript 1.0 functionality is no longer being updated, and no new feature development or enhancement work is being done for SuiteScript 1.0.

You should use SuiteScript 2.x for any new or substantially revised scripts to take advantage of new features, APIs, and functionality enhancements.

In general, SuiteScript 2.x supports everything SuiteScript 1.0 does. However, there is not always a direct mapping between functions and objects in SuiteScript 1.0 and modules and methods available in SuiteScript 2.x. And some functionality provided by SuiteScript 1.0 can now be done using JavaScript.

See the following help topics for guidance on transitioning your scripts from SuiteScript 1.0 to SuiteScript 2.x:

- [Overview of the Differences Between SuiteScript 1.0 and SuiteScript 2.x](#)
- [Differences Between SuiteScript 1.0 and SuiteScript 2.x Script Types](#)
- [SuiteScript 1.0 APIs Not Directly Mapped to a SuiteScript 2.x Module](#)
- [Differences in Similar SuiteScript 1.0 and SuiteScript 2.x Capabilities](#)
- [Converting a SuiteScript 1.0 Script to a SuiteScript 2.x Script](#)
- [Using SuiteScript 2.x Scripts with SuiteScript 1.0 Scripts](#)

For more information about specific features of SuiteScript 2.x, see the following help topics:

- [SuiteScript 2.1](#)
- [SuiteScript 2.x Script Creation Process](#)
- [SuiteScript 2.x Terminology](#)
- [SuiteScript 2.x Script Types](#)
- [SuiteScript 2.x Global Objects](#)
- [SuiteScript 2.x Global Objects](#)
- [SuiteScript 2.x Scripting Records and Subrecords](#)
- [SuiteScript Reserved Words](#)
- [SuiteScript 2.x JSDoc Validation](#)
- [SuiteScript 2.x Entry Point Script Creation and Deployment](#)
- [SuiteScript 2.x Custom Modules](#)
- [SuiteScript 2.x API Reference](#)
- [SuiteScript 2.x Code Samples Catalog](#)
- [SuiteScript Developer Guide](#)
- [SuiteScript FAQ](#)

Overview of the Differences Between SuiteScript 1.0 and SuiteScript 2.x

i Applies to: SuiteScript 1.0 | SuiteScript 2.x | APIs | SuiteCloud Developer

In general, SuiteScript 2.x provides support for all functionality in SuiteScript 1.0. However, there is not always a direct mapping between functions and objects in SuiteScript 1.0 and modules and methods available in SuiteScript 2.x. And some functionality provided by SuiteScript 1.0 can now be done using JavaScript.

Because SuiteScript 2.x is a modular language, there are several general differences between how scripts are written in SuiteScript 2.x and how they are written in SuiteScript 1.0. These differences include:

- Scripts now have a specified structure with entry points and entry point functions (see [Script Structure, Entry Points, Entry Point Functions](#))
- Scripts now have access to a context object (see [Context Objects](#))
- Scripts now use and support JSDoc (see [JSDoc](#))
- There are now reserved words that cannot be used in SuiteScript scripts (see [Reserved Words](#))
- Enumerated values are now available (see [Enumerated Values](#))
- Scripts now use modules instead of functions and objects (see [Modules](#))
- Arguments or parameters are now specified as objects of key: value pairs (see [Arguments and Parameters](#))
- Scripting subrecords in SuiteScript 2.x is fundamentally different from scripting subrecords in SuiteScript 1.0 (see [Scripting Subrecords](#))

There are also more specific differences concerning arguments and parameters, such as:

- Some argument or parameter names have changed.
- Some arguments or parameters to functions are now properties.
- Some SuiteScript 2.x module methods have extra parameters compared to their SuiteScript 1.0 versions.

For more information about these differences and other specific differences, see [Differences in Similar SuiteScript 1.0 and SuiteScript 2.x Capabilities](#).

Also note that some SuiteScript 1.0 functions and objects are not directly mapped to a SuiteScript 2.x module. For a list of these functions and objects, see [SuiteScript 1.0 APIs Not Directly Mapped to a SuiteScript 2.x Module](#).

Script Structure, Entry Points, Entry Point Functions

Each SuiteScript 2.x script type includes one or more entry points that are exclusive to that script type. An entry point represents the juncture at which the system grants control of the NetSuite application to the script. When you include an entry point in your script, you tell the system that it should execute a function defined within the script. This function is called an entry point function. With many script types, the available entry points are analogous to the types of events that trigger your script. For example, the entry points in a client script represent events that can occur during a browser session, such as `pageInit` and `fieldChanged`. In comparison, the scheduled script type has only one entry point, `execute`, which represents the point at which a schedule executes the script or a user manually executes the script.

For more information about the required structure for SuiteScript 2.x scripts, see [SuiteScript 2.x Anatomy of a Script](#).

For more information about entry points and entry point functions, see [SuiteScript 2.x Entry Point Script Creation and Deployment](#).

For more information about entry points for each specific script type, see [SuiteScript 2.x Script Types](#).

Context Objects

Every SuiteScript 2.x script, whether it's an entry point script or a custom module, must use an entry point. Each entry point must also correspond to an object (usually a function) defined within the script. When the object is a function, it is referred to as an entry point function. In most cases, an entry point function has access to a system-provided object that lets your script access data and take actions specific to the context in which the script is executing. For that reason, this object is often referred to as a context object.

For more information about context objects, see [Context Objects for Standard and Custom Entry Points](#).

JSDoc

SuiteScript 2.x lets you use JSDoc tags and comment blocks. In fact, you'll need to include two SuiteScript JSDoc tags in your SuiteScript 2.x scripts: `@ApiVersion` and `@ScriptType`. You can document your scripts by adding JSDoc comment blocks and tags right in your code.

For more information, see the help topic [SuiteScript 2.x JSDoc Validation](#).

Reserved Words

Like most programming languages, SuiteScript includes a list of reserved words. You can't use reserved words as variable names, labels, or function names in languages based on ECMAScript, like JavaScript. Since SuiteScript is based on ECMAScript, you also can't use ECMAScript reserved words as variable or function names in your SuiteScript scripts. If you use a reserved word as a variable or function name, you'll probably get a syntax error when your script runs.

There are also additional reserved parameter names when using SuiteScript Suitelets. For more information about these reserved parameter names, see [Reserved Parameter Names in Suitelet URLs](#).

 **Note:** SuiteScript 2.x provides global objects including [log Object](#) and [util Object](#). If you are using the words 'log' or 'util' as variable names in your SuiteScript 1.0 scripts and you have both SuiteScript 1.0 and SuiteScript 2.x scripts running on the same record you will receive an error. Both the 'log' and 'util' words are reserved in SuiteScript 2.x. You'll need to update any SuiteScript 1.0 scripts that use 'log' or 'util' as variable names.

For more information about SuiteScript reserved words, see [SuiteScript Reserved Words](#).

Enumerated Values

Although JavaScript does not include an enumeration type, in the SuiteScript 2.x documentation we use the term 'enumeration' (or enum) to describe a plain JavaScript object with a flat, map-like structure. Within this object, each key points to a read-only string value.

Enumerations typically take the form of a constant with an associated value. For example, the [N/runtime Module](#) includes the `runtime.Permission` enum that includes permission values of `FULL`, `EDIT`, `CREATE`, `VIEW`, and `NONE`. Or, the [N/error Module](#) includes the `error.Type` enum that includes values for possible errors that may be generated from the execution of a script.

Some enumeration values are static and some are dynamic. Static values are available to all SuiteScript 2.x scripts regardless of the type of account or which features are enabled. Dynamic values are determined based on the specific account type and enabled features. In most cases, the help center documentation indicates that some values are dynamic and may not be available to all scripts in all accounts.

Modules

All SuiteScript 2.x APIs are organized into a set of modules. Each module's name reflects its functionality. For example, the [N/record Module](#) lets you interact with NetSuite records and the [N/https Module](#) lets you make HTTPS requests to external web services. Most SuiteScript 2.x modules must be explicitly loaded by a script before the script can access that module's methods, properties, and enums. You load each module you want to use by specifying it as an argument to the `define` function. Some SuiteScript modules are also globally available and do not need to be explicitly loaded.

For more information about the modular architecture of SuiteScript 2.x, see [Modular Architecture](#).

For more information about the `define` function, see the help topic [define Object](#).

For more information about globally available objects, see the help topic [SuiteScript 2.x Global Objects](#).

Arguments and Parameters

There are several differences between how arguments or parameters are used in SuiteScript 1.0 and how they are used in SuiteScript 2.x. Some of these differences are:

- In SuiteScript 2.x, method parameters are specified as objects of key: value pairs. In SuiteScript 1.0, arguments are passed to API methods as individual variables or values. In SuiteScript 2.x, they are typically passed as objects. Standard methods expect all parameters to be passed in the form of a single object, that has one or many properties (key: values pairs).
- Some argument or parameter names are slightly different in SuiteScript 1.0 than in SuiteScript 2.x for the same comparable API method.
- Some arguments or parameters to functions in SuiteScript 1.0 are now properties in SuiteScript 2.x modules.
- Some SuiteScript 2.x module methods have additional parameters than their SuiteScript 1.0 function or object counterparts.

The [SuiteScript 1.0 to SuiteScript 2.x API Map](#) help topic provides links to each SuiteScript 1.0 API and each corresponding SuiteScript 2.x module, method, or property. You can use this map to compare arguments and parameters between similar functions. The map also includes notes that point out SuiteScript 1.0 functions that are now module properties in SuiteScript 2.x.

Scripting Subrecords

Scripting subrecords in SuiteScript 2.x is fundamentally different from scripting subrecords in SuiteScript 1.0. For example, in SuiteScript 2.x:

- You can use a single method to create or load a subrecord.
- You don't have to save subrecords explicitly—they're saved when you save the parent record.
- You must use subrecord methods to work with the address subrecord; addresses are now implemented using subrecords rather than individual fields on a record.

For more information about scripting subrecords in SuiteScript 2.x, see the help topics [SuiteScript 2.x Scripting Subrecords](#) and [Subrecord Scripting in SuiteScript 2.x Compared With 1.0](#).

Differences Between SuiteScript 1.0 and SuiteScript 2.x Script Types

i Applies to: SuiteScript 1.0 | SuiteScript 2.x | APIs | SuiteCloud Developer

SuiteScript 2.x includes all the script types from SuiteScript 1.0. Plus, SuiteScript 2.x adds two more script types: map/reduce and SDF installation scripts.

In SuiteScript 1.0, the term 'event types' is used to describe how a script was triggered. In SuiteScript 2.x, it's now called an 'entry point'. All SuiteScript 2.x scripts have defined entry points, which generally map directly to the event types supported by the same script type in SuiteScript 1.0. For more information about SuiteScript 2.x entry points, see [SuiteScript 2.x Entry Point Script Creation and Deployment](#).

See the following help topics for specific information about the differences in each script type:

- [Bundle Installation Script Type Differences](#)
- [Client Script Type Differences](#)
- [Map/Reduce Script Type Differences](#)
- [Mass Update Script Type Differences](#)
- [Portlet Script Type Differences](#)
- [RESTlet Script Type Differences](#)
- [Scheduled Script Type Differences](#)
- [SDF Installation Script Type Differences](#)
- [Suitelet Script Type Differences](#)
- [User Event Script Type Differences](#)
- [Workflow Action Script Type Differences](#)

For more information about specific SuiteScript 2.x script types, see the following help topics:

- [SuiteScript 2.x Bundle Installation Script Type](#)
- [SuiteScript 2.x Client Script Type](#)
- [SuiteScript 2.x Map/Reduce Script Type](#)
- [SuiteScript 2.x Mass Update Script Type](#)
- [SuiteScript 2.x Portlet Script Type](#)
- [SuiteScript 2.x RESTlet Script Type](#)
- [SuiteScript 2.x Scheduled Script Type](#)
- [SuiteScript 2.x SDF Installation Script Type](#)
- [SuiteScript 2.x Suitelet Script Type](#)
- [SuiteScript 2.x User Event Script Type](#)
- [SuiteScript 2.x Workflow Action Script Type](#)

Bundle Installation Script Type Differences

The only difference between bundle installation scripts in SuiteScript 1.0 and bundle installation scripts in SuiteScript 2.x is that the toversion parameter in SuiteScript 1.0 event types is now called version in the beforeInstall, afterInstall, and beforeUninstall entry points in SuiteScript 2.x.

For more information about the bundle installation script type in SuiteScript 2.x, see [SuiteScript 2.x Bundle Installation Script Type](#).



Note: SuiteBundler is still supported, but it will not be updated with any new features.

To take advantage of new features for packaging and distributing customizations, you can use the Copy to Account and SuiteCloud Development (SDF) features instead of SuiteBundler.

Copy to Account is an administrator tool that you can use to copy custom objects between your accounts. The tool can copy one custom object at a time, including dependencies and data. For more information, see the help topic [Copy to Account](#).

SuiteCloud Development Framework is a development framework that you can use to create SuiteApps from an integrated development environment (IDE) on your local computer. For more information, see .

Client Script Type Differences

Here's how client scripts differ between SuiteScript 1.0 and 2.x:

- In SuiteScript 2.x, a client script includes a new parameter: `currentRecord`.
- In SuiteScript 2.x, the `fieldChanged` entry point includes a new parameter: `column`.
- The type parameter in the SuiteScript 1.0 `pageInit` event type is called `mode` in the SuiteScript 2.x `pageInit` entry point.
- The `recalc` event type in SuiteScript 1.0 is now the `sublistChanged` entry point in SuiteScript 2.x.
- In SuiteScript 2.x, the `validateDelete` entry point includes a new parameter: `lineCount`.
- In SuiteScript 2.x, the `validateField` entry point includes a new parameter: `column`.
- In SuiteScript 2.x, there are two new entry points: `localizationContextEnter` and `localizationContextExit`.

For more information about the client script type in SuiteScript 2.x, see [SuiteScript 2.x Client Script Type](#).

Map/Reduce Script Type Differences

The map/reduce script is a new script type for SuiteScript 2.x that is designed for scripts that need to handle large amounts of data. It is best suited for situations where the data can be divided into small, independent parts. There's no equivalent script type in SuiteScript 1.0.

For more information about the map/reduce script type in SuiteScript 2.x, see [SuiteScript 2.x Map/Reduce Script Type](#).

Mass Update Script Type Differences

Here's how mass update scripts differ between SuiteScript 1.0 and 2.x:

- In SuiteScript 2.x, a mass update script includes the `each` entry point. There is no equivalent event type in SuiteScript 1.0.

- The `rec_type` and `rec_id` parameters used in SuiteScript 1.0 mass update scripts are replaced with `params.id` and `params.type` parameters for the each entry point in SuiteScript 2.x.

For more information about the mass update script type in SuiteScript 2.x, see [SuiteScript 2.x Mass Update Script Type](#).

Portlet Script Type Differences

Here's how portlet scripts differ between SuiteScript 1.0 and 2.x:

- In SuiteScript 2.x, you can use a portlet for a SuiteApp.
- In SuiteScript 2.x, a portlet script includes the `render` entry point. There is no equivalent event type in SuiteScript 1.0.
- The following methods are included in a SuiteScript 1.0 `nlobjPortlet` object, but are not included in the SuiteScript 2.x `Portlet` object: `setRefreshInterval(n)`, `setScript(scriptId)`.
- The following functions included in a SuiteScript 1.0 `nlobjPortlet` object are now properties in a SuiteScript 2.x `Portlet` object:
 - The `setHtml(html)` function in SuiteScript 1.0 is the `Portlet.html` property in SuiteScript 2.x.
 - The `setTitle(title)` function in SuiteScript 1.0 is the `Portlet.title` property in SuiteScript 2.x.
- The parameters for the `nPortletObject.addColumn` method have different names in the `Portlet.addColumn` method in SuiteScript 2.x:
 - The `name` parameter in SuiteScript 1.0 is the `id` parameter in SuiteScript 2.x.
 - The `just` parameter in SuiteScript 1.0 is the `align` parameter in SuiteScript 2.x.
- The `Portlet.addEditColumn` method in SuiteScript 2.x has new parameters: `link`, `linkParam`, and `linkParamName`.
- The `name` parameter for the SuiteScript 1.0 `nPortletObject.addField` method is now called `id` in the SuiteScript 2.x `Portlet.addField` method.
- The `indent` parameter for the SuiteScript 1.0 `nPortletObject.addLine` method is now called `align` in the SuiteScript 2.x `Portlet.addField` method.
- SuiteScript 2.x portlet scripts include two new properties: `Portlet.clientScriptFileId` and `Portlet.clientScriptModulePath`.

For more information about the portlet script type in SuiteScript 2.x, see [SuiteScript 2.x Portlet Script Type](#).

RESTlet Script Type Differences

The only difference between RESTlet scripts in SuiteScript 1.0 and RESTlet scripts in SuiteScript 2.x is that the RESTlet in SuiteScript 2.x includes the `delete`, `get`, `post`, and `put` entry points. These entry points are equivalent to the `delete`, `get`, `put`, and `post` functions (http methods) in SuiteScript 1.0.

- In SuiteScript 2.x, a RESTlet includes the `delete`, `get`, `post`, and `put` entry points. These entry points are equivalent to the `delete`, `get`, `put`, and `post` functions (http methods) in SuiteScript 1.0.

Note: The SuiteScript 2.x documentation for RESTlet script error handling includes more detail than previously provided for SuiteScript 1.0. For more information about error handling with RESTlet scripts, see [SuiteScript 2.x RESTlet Error Handling](#).

For more information about the client script type in SuiteScript 2.x, see the help topic [SuiteScript 2.x RESTlet Script Type](#).

Scheduled Script Type Differences

Here's how scheduled scripts differ between SuiteScript 1.0 and 2.x:

- In SuiteScript 2.x, a scheduled script includes the execute entry point. There is no equivalent event type in SuiteScript 1.0.
- The SuiteScript 1.0 `nlapIScheduleScript` function is replaced with the SuiteScript 2.x `task.create(options)` and `task.ScheduledScriptTask`.
- The type parameter in the SuiteScript 1.0 `nlapIScheduledScript` method is replaced with the `context.InvocationType` enum in SuiteScript 2.x. The values are similar: `SCHEDULED`, `ON_DEMAND`, `USER_INTERFACE`, `ABORTED`, and `SKIPPED`.
- The `nlapISetRecoverPoint` function in SuiteScript 1.0 has no SuiteScript 2.x equivalent. The functionality provided by this function is not needed because of the way scheduled scripts execute in SuiteScript 2.x.
- The `nlapiYieldScript` function in SuiteScript 1.0 has no SuiteScript 2.x equivalent. The functionality provided by this function is not needed because of the way scheduled scripts execute in SuiteScript 2.x.

For more information about the scheduled script type in SuiteScript 2.x, see [SuiteScript 2.x Scheduled Script Type](#).

SDF Installation Script Type Differences

The SDF installation script is a new script type for SuiteScript 2.x that is used to perform tasks during deployment of a SuiteApp from the SuiteCloud Development Framework (SDF) to a target account. There is no equivalent SuiteScript 1.0 script type.

For more information about the SDF installation script type in SuiteScript 2.x, see [SuiteScript 2.x SDF Installation Script Type](#).

Suitelet Script Type Differences

The only difference between Suitelet scripts in SuiteScript 1.0 and Suitelet scripts in SuiteScript 2.x is that in SuiteScript 2.x, a Suitelet includes the `onRequest` entry point. There is no equivalent event type in SuiteScript 1.0.

For more information about the Suitelet script type in SuiteScript 2.x, see [SuiteScript 2.x Suitelet Script Type](#).

User Event Script Type Differences

Here's how user event scripts differ between SuiteScript 1.0 and 2.x:

- The `beforeLoad` entry point in SuiteScript 2.x includes a new parameter: `newRecord`.

- The `beforeSubmit` entry point in SuiteScript 2.x includes two new parameters: `oldRecord` and `newRecord`.
- The `afterSubmit` entry point in SuiteScript 2.x includes two new parameters: `oldRecord` and `newRecord`.
- The type parameter in each entry point in SuiteScript 2.x is now set using the `context.UserEventType` enum.

For more information about the user event script type in SuiteScript 2.x, see [SuiteScript 2.x User Event Script Type](#).

Workflow Action Script Type Differences

Here's how workflow action scripts differ between SuiteScript 1.0 and 2.x:

- In SuiteScript 2.x, a workflow action script includes the `onAction` entry point. There is no equivalent event type for in SuiteScript 1.0.
- The `id` parameter in SuiteScript 1.0 is called `workflowId` in SuiteScript 2.x.
- There are two new parameters in SuiteScript 2.x: `newRecord` and `oldRecord`.

For more information about the workflow action script type in SuiteScript 2.x, see [SuiteScript 2.x Workflow Action Script Type](#).

Differences in Similar SuiteScript 1.0 and SuiteScript 2.x Capabilities

i Applies to: SuiteScript 1.0 | SuiteScript 2.x | APIs | SuiteCloud Developer

Some SuiteScript 1.0 capabilities work differently in SuiteScript 2.x, such as logging, printing, scripting subrecords, and resolving URLs.

Logging

Instead of using the SuiteScript 1.0 `nlapilogExecution` function to log all types of log messages, SuiteScript 2.x provides a different method for each type of log message: audit, debug, emergency, and error. Each method is available in the [N/log Module](#).

- The [log.audit\(options\)](#) method logs an audit message that appears on the Execution Log subtab only if the Log Level on the script deployment is set to Audit or Debug.
- The [log.debug\(options\)](#) method logs a debug message that appears on the Execution Log subtab only if the Log Level on the script deployment is set to Debug.
- The [log.emergency\(options\)](#) method logs an emergency message that appears on the Execution Log subtab if the Log Level on the script deployment is set to Audit, Debug, Error, or Emergency. In other words, emergency messages always appear.
- The [log.error\(options\)](#) method logs an error message that appears on the Execution Log subtab only if the Log Level on the script deployment is set to Audit, Debug, or Error.

For more information, see the help topics:

- [log Object](#)
- [N/log Module](#)

Printing and Template Creation

Instead of using the SuiteScript 1.0 functions to print a record or create a template, in SuiteScript 2.x you use the methods in the [N/render Module](#) module, shown in the following table:

SuiteScript 1.0 Function	SuiteScript 2.x Method
<code>nlapicreateEmailMerger(templated)</code>	<code>render.mergeEmail(options)</code>
<code>nlapicreateTemplateRenderer()</code>	<code>render.create()</code>
<code>nlapiprintRecord(type, id, mode, properties)</code>	<code>render.bom(options)</code> , <code>render.packingSlip(options)</code> , <code>render.pickingTicket(options)</code> , <code>render.statement(options)</code> , <code>render.transaction(options)</code>
<code>nlapixmlToPDF(xmlstring)</code>	<code>render.xmlToPdf(options)</code> , <code>TemplateRenderer.renderAsPdf()</code>

For more information, see the help topic [N/render Module](#).

Scripting Subrecords

Scripting subrecords in SuiteScript 2.x is fundamentally different from scripting subrecords in SuiteScript 1.0. Compared with SuiteScript 1.0, SuiteScript 2.x introduces the following changes in how you script subrecords:

- You can use a single method to create or load a subrecord
- SuiteScript 2.x subrecords do not have to be explicitly saved; they are saved when you save the parent record
- You must use subrecord methods to work with the address subrecord; addresses are not implemented using subrecords rather than individual fields on a record

For more information about scripting subrecords in SuiteScript 2.x, see [SuiteScript 2.x Scripting Subrecords](#) and [Subrecord Scripting in SuiteScript 2.x Compared With 1.0](#).

Resolving URLs

Instead of using the SuiteScript 1.0 `nlapiresolveURL` function to resolve all URLs, SuiteScript 2.x provides a different method to resolve a record, a script, or a task. Each method is available in the [N/url Module](#).

- The `url.resolveDomain(options)` method used to resolve a domain name.
- The `url.resolveRecord(options)` method used to resolve a record URL.
- The `url.resolveScript(options)` method used to resolve a script URL.
- The `url.resolveTaskLink(options)` method is used to resolve a task link.

For more information about the methods available to resolve URLs, see the help topic [N/url Module](#)

SuiteScript 1.0 APIs Not Directly Mapped to a SuiteScript 2.x Module

i Applies to: SuiteScript 1.0 | SuiteScript 2.x | APIs | SuiteCloud Developer

Some SuiteScript 1.0 functions and objects do not directly map to any SuiteScript 2.x module method, property, or enum. This includes SuiteScript 1.0 APIs that either do not have direct SuiteScript 2.x API

equivalents or the mapping is not intuitive (such as a mapping from a method to a property, or to a new module altogether).

The following table provides the functions and objects that do not have direct or intuitive mapping to SuiteScript 2.x. This information is a summary of what is included in the [SuiteScript 1.0 to SuiteScript 2.x API Map](#).

Refer to the [SuiteScript 1.0 Documentation](#) for information about SuiteScript 1.0 functions and objects.

SuiteScript 1.0 Function or Object	What to use when writing SuiteScript 2.x scripts
Functions	
<code>nlapiAddDays(d, days)</code>	Standard JavaScript supplies date functions to add or subtract days from a Date object. SuiteScript 2.x is also compatible with third-party JavaScript APIs that provide date functions.
<code>nlapiAddMonths(d, months)</code>	Standard JavaScript supplies date functions to add or subtract months from a Date object. SuiteScript 2.x is also compatible with third-party JavaScript APIs that provide date functions.
<code>nlapiEncrypt(s, algorithm, key)</code>	Hashing and HMAC functionality are provided by the N/crypto Module . Encoding functionality is provided by the N/encode Module .
<code>nlapiGetCurrentLineItemDateTimeValue(type, fieldId, timeZone)</code>	This functionality is provided by the N/format Module . Specifically, the format.Timezone enum used in the format.format(options) and format.parse(options) methods.
<code>nlapiGetDateTimeValue(fieldId, timeZone)</code>	
<code>nlapiGetLineItemDateTimeValue(type, fieldId, lineNum, timeZone)</code>	
<code>nlapiSetCurrentLineItemDateTimeValue(type, fieldId, dateTime, timeZone)</code>	
<code>nlapiSetDateTimeValue(fieldId, dateTime, timeZone)</code>	
<code>nlapiSetLineItemDateTimeValue(type, fieldId, lineNum, dateTime, timeZone)</code>	
<code>nlapiSetRecoveryPoint()</code>	
<code>nlapiYieldScript()</code>	Scripting recovery points for scheduled scripts are no longer necessary. See SuiteScript 2.x Scheduled Script Type and Map/Reduce Script Type Differences .
Objects	
<code>nlobjCredentialBuilder.replace(string1, string2)</code>	This functionality is not provided by a SuiteScript 2.x module or method. You need to manually perform the replacement of credential strings within your script.
<code>nlobjPortlet.setRefreshInterval(n)</code>	Although there is a Portlet object in SuiteScript 2.x, that object does not include a refresh function. It is no longer needed in SuiteScript 2.x scripts.
<code>nlobjRecord.getCurrentLineItemDateTimeValue(type, fieldId, timeZone)</code>	This functionality is provided by the N/format Module . Specifically, the format.format(options) and format.parse(options) methods.
<code>nlobjRecord.getDateTimeValue(fieldId, timeZone)</code>	
<code>nlobjRecord.getLineItemDateTimeValue(type, fieldId, lineNum, timeZone)</code>	

SuiteScript 1.0 Function or Object	What to use when writing SuiteScript 2.x scripts
nlobjRecord.setCurrentLineItemDateTimeValue(type, fieldId, dateTime, timeZone)	
nlobjRecord.setDateTimeValue(fieldId, dateTime, timeZone)	
nlobjRecord.setLineItemDateTimeValue(type, fieldId, lineNum, dateTime, timeZone)	
nlobjResponse.setContentType(type, name, disposition).	The functionality to specify content type and encoding type is specified using HTTP headers in SuiteScript 2.x. For more information, see the help topics HTTP Header Information and HTTPS Header Information .
nlobjResponse.setEncoding(encodingType).	
nlobjSublist.setLineItemValues(values)	Use the Sublist.setSublistValue(options) method of the N/ui/serverWidget Module to set each value in the sublist individually.
nlobjSubrecord.cancel() and nlobjSubrecord.commit()	Scripting subrecords in SuiteScript 2.x is fundamentally different from scripting subrecords in SuiteScript 1.0. For more information about scripting subrecords in SuiteScript 2.x, see the help topics SuiteScript 2.x Scripting Subrecords and Subrecord Scripting in SuiteScript 2.x Compared With 1.0 .

Along with functions and objects that are not directly mapped from SuiteScript 1.0 to SuiteScript 2.x, there are also some functions and objects in SuiteScript 1.0 that map to SuiteScript 2.x, but those mapping may not be intuitive. For example, the functions and objects provided to perform CSV imports in SuiteScript 1.0 are now provided by the [N/task Samples](#) in SuiteScript 2.x. All mappings are provided in the [SuiteScript 1.0 to SuiteScript 2.x API Map](#). The map includes notes to aid in understanding mappings between SuiteScript 1.0 and SuiteScript 2.x that are not intuitive. Also refer to all help topics for the mapped SuiteScript 2.x modules to fully understand the difference (if any) in functionality provided by SuiteScript 1.0 and by SuiteScript 2.x.

Further, some SuiteScript 1.0 functions and objects map to SuiteScript 2.x module properties instead of module methods. The [SuiteScript 1.0 to SuiteScript 2.x API Map](#) clearly indicates these type of mappings. Look in the Notes column – it will indicate the mapping is to a SuiteScript 2.x module property.

If converting your SuiteScript 1.0 scripts to SuiteScript 2.x is not feasible in your situation, you can use SuiteScript 2.x RESTlet scripts to extend the capabilities of your existing SuiteScript 1.0 scripts. For more information, see [Using SuiteScript 2.x Scripts with SuiteScript 1.0 Scripts](#).

Converting a SuiteScript 1.0 Script to a SuiteScript 2.x Script

i Applies to: SuiteScript 1.0 | SuiteScript 2.x | APIs | SuiteCloud Developer

The general steps you take to convert a SuiteScript 1.0 script to a SuiteScript 2.x script depend on whether you are converting an entry point script or a custom module script.

- If your script is for a specific script type, such as user event or client script, you will first need to edit the script so that the correct script type structure is in place. This includes adding JSDoc comments, the define function, and a return statement at a minimum. For more information about proper script structure, see the help topic [SuiteScript 2.x Anatomy of a Script](#).

- After you place the script into the proper structure, you can start converting individual SuiteScript 1.0 calls into their corresponding SuiteScript 2.x calls according to the [SuiteScript 1.0 to SuiteScript 2.x API Map](#). Note that this order is not a strict order of steps to take; however, you may find it easier to place the script into the proper script structure first and then convert each call statement.
- If your script is a custom module and not a specific script type, you can convert each SuiteScript 1.0 call into its corresponding SuiteScript 2.x call according to the [SuiteScript 1.0 to SuiteScript 2.x API Map](#).

Here are a few additional notes to consider when converting your scripts:

- Some functions and objects in SuiteScript 1.0 are now supported in SuiteScript 2.x by using JavaScript, such as Date functions.
- Some functions and objects in SuiteScript 1.0 operate a little differently in SuiteScript 2.x, such as logging and printing. See the help topic [Differences in Similar SuiteScript 1.0 and SuiteScript 2.x Capabilities](#).
- Some function and objects in SuiteScript 1.0 do not completely map to any SuiteScript 2.x module, method, property, or enum. See the help topic [SuiteScript 1.0 APIs Not Directly Mapped to a SuiteScript 2.x Module](#).

The [Sample SuiteScript 1.0 to SuiteScript 2.0 and SuiteScript 2.1 Conversions](#) help topic shows you a side-by-side comparison of SuiteScript 1.0 and SuiteScript 2.x scripts for each script type and custom modules.

If converting your SuiteScript 1.0 scripts to SuiteScript 2.x is not feasible in your situation, you can use SuiteScript 2.x RESTlet scripts to extend the capabilities of your existing SuiteScript 1.0 scripts. For more information, see [Using SuiteScript 2.x Scripts with SuiteScript 1.0 Scripts](#).

Sample SuiteScript 1.0 to SuiteScript 2.0 and SuiteScript 2.1 Conversions

Applies to: SuiteScript 1.0 | SuiteScript 2.x | APIs | SuiteCloud Developer

This help topic gives you example conversions from SuiteScript 1.0 to SuiteScript 2.0 or 2.1 scripts. Each example shows the SuiteScript 1.0 script on the left and the SuiteScript 2.0/2.1 script on the right.

Conversions for the following scripts are included:

- Bundle installation script one: [Confirm bundle installation, enable features, create account](#)
- Client script one: [Set default fields values on a record](#)
- Client script two: [Disable fields on a record](#)
- Client script three: [Display user profile information](#)
- Mass update script one: [Update fields on sales order and estimate records](#)
- Portlet script one: [Build a simple portlet that posts data to a servlet](#)
- RESTlet script one: [Using RESTlet get, post, put, and delete entry points and methods](#)
- Scheduled script one: [Fulfill and bill sales orders each day](#)
- Suitelet script one: [Create a simple form](#)
- User event script one: [Implement end of month sales order promotions](#)
- Workflow action script one: [Set the sales rep on the record in a workflow](#)

- Custom module script one: [Create an item receipt from a purchase order](#)
- Custom module script two: [Create and run a joined search](#)
- Custom module script three: [Create CSV imports](#)

Confirm bundle installation, enable features, create account

i Applies to: SuiteScript 1.0 | SuiteScript 2.x | APIs | SuiteCloud Developer

This sample is a bundle installation script that makes sure the bundle being installed is version 2.0, and that the Work Orders and Multiple Currencies features are enabled, and then creates an account record.

The conversion of this script from SuiteScript 1.0 to SuiteScript 2.x includes the following:

- JSDoc tags are added at the top of the script to indicate the script version and script type
- The define statement is added which imports the N/runtime and N/error modules.
- The return statement is added with the beforeInstall, beforeUpdate, and afterUpdate entry points.
- The beforeInstall, beforeUpdate, and afterUpdate entry point functions are created.
- The beforeInstall, beforeUpdate, and afterUpdate entry point functions are converted to SuiteScript 2.x.
 - The toversion argument is changed to the params.toVersion or params.version parameter.
 - The fromversion argument is changed to the params.fromVersion parameter.
 - The nlapiCreateRecord function is changed to the record.create method.
 - The setFieldValue function is changed to the Record.setValue method.
 - The nlapiSubmitRecord function is changed to the Record.save method.
- The checkFeatureEnabled function is converted to SuiteScript 2.x.
 - The nlapiGetContext function isn't needed anymore.
 - The getFeature function is changed to the runtime.isFeatureInEffect method.
 - The nlapiLogExecution function is changed to the log.debug method.
 - The nlobjError object is changed to the error.create method.

SuiteScript 1.0 Script	SuiteScript 2.x Script
<pre> 1 function beforeInstall(toversion){ 2 checkFeatureEnabled('WORKORDERS'); 3 if (toversion.toString() == "2.0"){ 4 checkFeatureEnabled('MULTICURRENCY'); 5 } 6 } 7 8 function afterInstall(toversion){ 9 var randomnumber=Math.floor(Math.random()*10000); 10 var objRecord = nlapiCreateRecord('account'); 11 objRecord.setFieldValue('accttype','Bank'); 12 objRecord.setFieldValue('acctnumber',randomnumber); 13 objRecord.setFieldValue('acctname','Acct '+toversion); 14 nlapiSubmitRecord(objRecord, true); 15 } 16 17 function beforeUpdate(fromversion, toversion){ 18 checkFeatureEnabled('WORKORDERS'); </pre>	<pre> 1 /** 2 * @NApiVersion 2.x 3 * @NScriptType BundleInstallationScript 4 */ 5 6 define(['N/runtime', 'N/error'], function (runtime, error) { 7 function checkFeatureEnabled(featureId){ 8 isInEffect = runtime.isFeatureInEffect({ 9 feature: featureId 10 }); 11 } 12 if (isInEffect){ 13 log.debug({ 14 title: Feature, 15 details: featureId + 'enabled' 16 }); 17 } else { 18 var featureError = error.create({ </pre>

SuiteScript 1.0 Script	SuiteScript 2.x Script
<pre> 19 if (toversion.toString() == "2.0") { 20 checkFeatureEnabled('MULTICURRENCY'); 21 } 22 } 23 24 function afterUpdate(fromversion, toversion){ 25 if (fromversion.toString() != toversion.toString()){ 26 var randomnumber=Math.floor(Math.random()*10000); 27 var objRecord = nlapiCreateRecord('account'); 28 objRecord.setFieldValue('accttype','Bank'); 29 objRecord.setFieldValue('acctnumber',randomnumber); 30 objRecord.setFieldValue('acctname','Acct '+toversion); 31 nlapiSubmitRecord(objRecord, true); 32 } 33 } 34 35 function checkFeatureEnabled(featureId){ 36 var objContext = nlapiGetContext(); 37 var feature = objContext.getFeature(featureId); 38 39 if (feature){ 40 nlapiLogExecution('DEBUG','Feature',featureId+' enabled'); 41 } else { 42 throw new nlobjError('INSTALLATION_ERROR','Feature '+ 43 featureId+' must be enabled. Please enable the feature and try 44 again.');</pre>	<pre> 19 name: 'INSTALLATION_ERROR', 20 message: 'Feature' + featureId + 'must be enabled. Please enable the feature and try again.' 21 }); 22 throw featureError; 23 } 24 } 25 26 return { 27 beforeInstall: function beforeInstall(params) { 28 checkFeatureEnabled('WORKORDERS'); 29 30 if (params.version.toString() === "2.0") { 31 checkFeatureEnabled('MULTICURRENCY'); 32 } 33 }, 34 35 beforeUpdate: function beforeUpdate(params) { 36 checkFeatureEnabled('WORKORDERS'); 37 38 if (params.toVersion.toString() === "2.0") { 39 checkFeatureEnabled('MULTICURRENCY'); 40 } 41 }, 42 43 afterUpdate: function afterUpdate(params) { 44 if (params.fromVersion.toString() != tover 45 sion.toString()) { 46 var randomnumber = Math.floor(Math.random()*10000); 47 var objRecord = record.create({ 48 type: 'account' 49 }); 50 objRecord.setValue({ 51 fieldId: 'accttype', 52 value: 'Bank' 53 }); 54 objRecord.setValue({ 55 fieldId: 'acctnumber', 56 value: randomnumber 57 }); 58 objRecord.setValue({ 59 fieldId: 'acctname', 60 value: 'Acct '+toversion 61 }); 62 objRecord.save({ 63 enableSourcing: true 64 }); 65 } 66 }; 67 });</pre>

Set default fields values on a record

Applies to: SuiteScript 1.0 | SuiteScript 2.x | APIs | SuiteCloud Developer

This sample is a client script that sets default field values on a record.

The conversion of this script from SuiteScript 1.0 to SuiteScript 2.1 includes the following:

- JSDoc tags are added at the top of the script to indicate the script version and script type.
- The define statement is added which imports the N/record module.
- The return statement is added to the pageInit entry point.
- A record is loaded using the record.load method
- The nlapiGetFieldValue function is changed to the Record.getValue method.
- The nlapiGetFieldText function is changed to the Record.getText method.

- The `nlapiSetFieldText` function is changed to the `Record.setText` method.

SuiteScript 1.0 Script	SuiteScript 2.1 Script
<pre> 1 function pageInit() { 2 if ((nlapiGetFieldValue('fieldA').length === 0) (nlapiGetField 3 Text('fieldA') === "valueA")) 4 { 5 nlapiSetFieldText('fieldA', nlapiGetFieldText('valueB')); 6 } 7 } </pre>	<pre> 1 /** 2 * @NApiVersion 2.1 3 * @NScriptType ClientScript 4 */ 5 6 define([], () => { 7 function pageInit(context) { 8 const myRecord = context.currentRecord; 9 const myFieldValue = myRecord.getValue({ 10 fieldId: 'fieldA' 11 }); 12 const myFieldText = myRecord.getText({ 13 fieldId: 'fieldA' 14 }); 15 16 if ((myFieldValue.length === 0) (myFieldText 17 === 'valueA')) 18 { 19 myRecord.setValue({ 20 fieldId: 'fieldA', 21 value: 'valueB' 22 }); 23 } 24 } 25 return { 26 pageInit: pageInit 27 }; 28 }); </pre>

Disable fields on a record

Applies to: SuiteScript 1.0 | SuiteScript 2.x | APIs | SuiteCloud Developer

This sample is a client script that disables fields on a record.

The conversion of this script from SuiteScript 1.0 to SuiteScript 2.1 includes the following:

- JSDoc tags are added at the top of the script to indicate the script version and script type.
- The `define` statement is added which imports the `N/record` module.
- The `return` statement is added with the `pageInit` entry point.
- A record is loaded using the `record.load` method.
- The `nlapiDisableField` function is changed to the `Record.isDisabled` property.

SuiteScript 1.0 Script	SuiteScript 2.1 Script
<pre> 1 function pageInit() { 2 nlapiDisableField('custrecord_other_fieldA', true); 3 nlapiDisableField('custrecord_other_fieldB', true); 4 } </pre>	<pre> 1 /** 2 * @NApiVersion 2.1 3 * @NScriptType ClientScript 4 */ 5 6 define([], () => { 7 function pageInit(context) { 8 const myRecord = context.currentRecord; 9 const myRecordFieldA = myRecord.getField({ 10 fieldId: 'custrecord_other_fieldA' 11 }); 12 const myRecordFieldB = myRecord.getField({ 13 fieldId: 'custrecord_other_fieldB' 14 }); 15 } 16 }); </pre>

SuiteScript 1.0 Script	SuiteScript 2.1 Script
	<pre> 15 16 myRecordFieldA.isDisabled = true; 17 myRecordFieldB.isDisabled = true; 18 } 19 20 return { 21 pageInit: pageInit 22 }; 23 };</pre>

Display user profile information

i Applies to: SuiteScript 1.0 | SuiteScript 2.x | APIs | SuiteCloud Developer

This sample is a client script that displays user profile information.

The conversion of this script from SuiteScript 1.0 to SuiteScript 2.1 includes the following:

- JSDoc tags are added at the top of the script to indicate the script version and script type.
- The define statement is added which imports the N/runtime and N/ui/message modules.
- The return statement is added with the pageInit entry point.
- The nlapiGetUser function is changed to the runtime.getCurrentUser method and the runtime.User.name property.
- The nlapiGetRole function is changed to the runtime.User.role property.
- The nlapiGetDepartment function is changed to the runtime.User.department property.
- The nlapiGetLocation function is changed to the runtime.User.location property.
- The alert function is changed to the message.create and message.show functions.

SuiteScript 1.0 Script	SuiteScript 2.1 Script
<pre> 1 function pageInit(){ 2 var userName = nlapiGetUser(); 3 var userRole = nlapiGetRole(); 4 var userDept = nlapiGetDepartment(); 5 var userLoc = nlapiGetLocation(); 6 7 alert("Current User Information" + "\n\n" + 8 "Name: " + userName + "\n" + 9 "Role: " + userRole + "\n" + 10 "Dept: " + userDept + "\n" + 11 "Loc: " + userLoc 12); 13 }</pre>	<pre> 1 /** 2 * @ApiVersion 2.1 3 * @NScriptType ClientScript 4 */ 5 6 define(['N/runtime', 'N/ui/message'], (runtime, message) => { 7 function pageInit(context) { 8 const myUser = runtime.getCurrentUser(); 9 const userName = myUser.name; 10 const userRole = myUser.role; 11 const userDept = myUser.department; 12 const userLoc = myUser.location; 13 14 let myMessageText = "Current User Information" + "\n\n" + 15 "Name: " + userName + "\n" + 16 "Role: " + userRole + "\n" + 17 "Dept: " + userDept + "\n" + 18 "Loc: " + userLoc; 19 20 const myMessage = message.create({ 21 type: INFORMATION, 22 title: 'User Information', 23 message: myMessageText, 24 duration: 10000 25 }); 26 myMessage.show(); 27 } 28 29 return { 30 pageInit: pageInit 31 }; 32 }</pre>

SuiteScript 1.0 Script	SuiteScript 2.1 Script
	32 });

Update fields on sales order and estimate records

Applies to: SuiteScript 1.0 | SuiteScript 2.x | APIs | SuiteCloud Developer

This sample is a mass update script that updates fields on sales order and estimate records.

The conversion of this script from SuiteScript 1.0 to SuiteScript 2.1 includes the following:

- JSDoc tags are added at the top of the script to indicate the script version and script type.
- The define statement is added which imports the N/record and N/runtime modules.
- The return statement is added with the each entry point.
- The nlapiLoadRecord function is changed to the record.load method.
- The nlapiGetContext and getSetting functions are changed to the runtime.getCurrentScript and the runtime.Script.getParameter methods.
- The setFieldValue method is changed to the Record.setValue method.
- The nlapiSubmitRecord method is changed to the Record.save method.

SuiteScript 1.0 Script	SuiteScript 2.1 Script
<pre> 1 function updateDepartment(rec_type, rec_id) { 2 var transaction = nlapiLoadRecord(rec_type, rec_id); 3 transaction.setFieldValue('department', nlapiGetContext().getSetting('SCRIPT', 'custscript_dept_update')); 4 nlapiSubmitRecord(transaction, false, true); 5 } </pre>	<pre> 1 /** 2 * @NApiVersion 2.1 3 * @NScriptType MassUpdateScript 4 */ 5 6 define(['N/record', 'N/runtime'], (record, runtime) => { 7 function each(params) { 8 const transaction = record.load({ 9 type: params.type, 10 id: params.id 11 }); 12 const scriptObj = runtime.getCurrentScript(); 13 14 const myDepartment = scriptObj.getParameter({ 15 name: 'custscript_dept_update' 16 }); 17 18 transaction.setValue({ 19 fieldId: 'department', 20 value: myDepartment 21 }); 22 transaction.save({ 23 enableSourcing: false, 24 ignoreMandatoryFields: true 25 }); 26 } 27 return { 28 each: each 29 }; 30 }); </pre>

Build a simple portlet that posts data to a servlet

Applies to: SuiteScript 1.0 | SuiteScript 2.x | APIs | SuiteCloud Developer

This sample is a portlet script builds a simple a simple for that posts data to a servlet.

The conversion of this script from SuiteScript 1.0 to SuiteScript 2.0 includes the following:

- JSDoc tags are added at the top of the script to indicate the script version and script type.
- The define statement is added which imports the N/portlet module.
- The return statement is added with the render entry point.
- The addField, addSelectOption, and setSubmitButton functions are changed to methods that accept an object as the parameter.
- The setLayoutType function is changed to the updateLayout and updateBreakType methods.

SuiteScript 1.0 Script	SuiteScript 2.0 Script
<pre> 1 function demoSimpleFormPortlet(portlet, column) { 2 portlet.setTitle('Simple Form Portlet'); 3 4 var fld = portlet.addField('text', 'text', 'Text'); 5 6 fld.setLayoutType('normal', 'startcol'); 7 8 portlet.addField('integer', 'integer', 'Integer'); 9 10 portlet.addField('date', 'date', 'Date'); 11 12 var select = portlet.addField('fruit', 'select', 'Select'); 13 14 select.addSelectOption('a', 'Oranges'); 15 16 select.addSelectOption('b', 'Apples'); 17 18 select.addSelectOption('c', 'Bananas'); 19 20 portlet.addField('textarea', 'textarea', 'Textarea'); 21 22 portlet.setSubmitButton('http://httpbin.org/post', 'Submit'); 23 }</pre>	<pre> 1 /** 2 * @NApiVersion 2.0 3 * @NScriptType Portlet 4 */ 5 6 define([], function () { 7 function render(params) { 8 var portlet = params.portlet; 9 10 portlet.title = 'Simple Form Portlet'; 11 12 var fld = portlet.addField({ 13 id: 'text', 14 type: 'text', 15 label: 'Text' 16 }); 17 18 fld.updateLayoutType({ 19 layoutType: 'normal' 20 }); 21 22 fld.updateBreakType({ 23 breakType: 'startcol' 24 }); 25 26 var select = portlet.addField({ 27 id: 'fruit', 28 type: 'select', 29 label: 'Select' 30 }); 31 32 select.addSelectOption({ 33 value: 'a', 34 text: 'Oranges' 35 }); 36 37 select.addSelectOption({ 38 value: 'b', 39 text: 'Apples' 40 }); 41 42 select.addSelectOption({ 43 value: 'c', 44 text: 'Bananas' 45 }); 46 47 portlet.addField({ 48 id: 'textarea', 49 type: 'textarea', 50 label: 'Textarea' 51 }); 52 53 portlet.setSubmitButton({ 54 url: 'http://httpbin.org/post', 55 label: 'Submit', 56 target: '_top' 57 }); 58 } 59 return { 60 render: render 61 }; 62 });</pre>

Using RESTlet get, post, put, and delete entry points and methods

Applies to: SuiteScript 1.0 | SuiteScript 2.x | APIs | SuiteCloud Developer

This sample is a RESTlet that shows how to use the get, post, put, and delete entry points and methods in a RESTlet script.

The conversion of this script from SuiteScript 1.0 to SuiteScript 2.1 includes the following:

- JSDoc tags are added at the top of the script to indicate the script version and script type.
- The define statement is added which imports the N/record and N/error modules.
- The return statement is added with the onRequest entry point.
- The nlapiLoadRecord function is changed to the record.load method.
- The nlapiCreateRecord function is changed to the record.create method.
- The record.setFieldValue function is changed to the Record.setValue method.
- The nlapiSubmitRecord function is changed to the Record.save method.
- The nlapiDeleteRecord function is changed to the record.delete method.

SuiteScript 1.0 Script	SuiteScript 2.1 Script
<pre> 1 // (GET method sample - get a standard NetSuite record) 2 function getRecord(datain) { 3 return nlapiLoadRecord(datain.recordtype, datain.id); 4 } </pre>	<pre> 1 /** 2 *@NApiVersion 2.1 3 *@NScriptType Restlet 4 */ 5 define(['N/record', 'N/error'], (record, error) => { 6 7 // Support method called by other methods for validating 8 function doValidation(args, argNames, methodName) { 9 for (let i = 0; i < args.length; i++) { 10 if (!args[i] && args[i] !== 0) { 11 throw error.create({ 12 name: 'MISSING_REQ_ARG', 13 message: 'Missing a required argument: [' + 14 argNames[i] + ']' for method: ' + methodName 15 }); 16 } 17 } 18 } 19 20 // (GET method - get a standard NetSuite record) 21 function _get(context) { 22 doValidation([context.recordtype, context.id], ['record 23 type', 'id'], 'GET'); 24 return JSON.stringify(record.load({ 25 type: context.recordtype, 26 id: context.id 27 })); 28 } 29 30 // (POST method - create a NetSuite record) 31 function post(context) { 32 doValidation([context.recordtype], ['recordtype'], 'POST'); 33 let rec = record.create({ 34 type: context.recordtype 35 }); 36 for (var fldName in context) 37 if (context.hasOwnProperty(fldName)) 38 if (fldName !== 'recordtype') 39 rec.setValue(fldName, context[fldName]); 40 let recordId = rec.save(); 41 return String(recordId); 42 } 43 44 // (DELETE method - delete a standard NetSuite record) 45 function _delete(context) { </pre>
<pre> 1 // (POST method sample - create a standard NetSuite record) 2 function createRecord(datain) { 3 var err = new Object(); 4 5 if (!datain.recordtype) { 6 err.status = "failed"; 7 err.message= "missing recordtype"; 8 return err; 9 } 10 11 var record = nlapiCreateRecord(datain.recordtype); 12 13 for (var fieldName in datain) { 14 if (datain.hasOwnProperty(fieldName)) { 15 if (fieldName !== 'recordtype' && fieldName !== 'id') { 16 var value = datain[fieldName]; 17 if (value && typeof value !== 'object') { 18 record.setFieldValue(fieldName, value); 19 } 20 } 21 } 22 } 23 var recordId = nlapiSubmitRecord(record); 24 return recordId; 25 } </pre>	<pre> 29 doValidation([context.recordtype], ['recordtype'], 'POST'); 30 let rec = record.create({ 31 type: context.recordtype 32 }); 33 for (var fldName in context) 34 if (context.hasOwnProperty(fldName)) 35 if (fldName !== 'recordtype') 36 rec.setValue(fldName, context[fldName]); 37 let recordId = rec.save(); 38 return String(recordId); 39 } 40 41 // (DELETE method - delete a standard NetSuite record) 42 function _delete(context) { </pre>
<pre> 1 // (DELETE method sample - delete a standard NetSuite record) 2 function deleteRecord(datain) { 3 nlapiDeleteRecord(datain.recordtype, datain.id); 4 } </pre>	<pre> 42 // (DELETE method - delete a standard NetSuite record) 43 function _delete(context) { </pre>

SuiteScript 1.0 Script	SuiteScript 2.1 Script
	<pre> 44 doValidation([context.recordtype, context.id], ['record type', 'id'], 'DELETE'); 45 record.delete({ 46 type: context.recordtype, 47 id: context.id 48 }); 49 return String(context.id); 50 } 51 52 // (PUT method - upsert a NetSuite record) 53 function put(context) { 54 doValidation([context.recordtype, context.id], ['record type', 'id'], 'PUT'); 55 let rec = record.load({ 56 type: context.recordtype, 57 id: context.id 58 }); 59 for (let fldName in context) 60 if (context.hasOwnProperty(fldName)) 61 if (fldName !== 'recordtype' && fldName !== 'id') 62 rec.setValue(fldName, context[fldName]); 63 rec.save(); 64 return JSON.stringify(rec); 65 } 66 67 return { 68 get: _get, 69 delete: _delete, 70 post: post, 71 put: put 72 }; 73 </pre>

Fulfill and bill sales orders each day

Applies to: SuiteScript 1.0 | SuiteScript 2.x | APIs | SuiteCloud Developer

This sample is a scheduled script that fulfills and bills sales orders created each day.

The conversion of this script from SuiteScript 1.0 to SuiteScript 2.0 includes the following:

- JSDoc tags are added at the top of the script to indicate the script version and script type.
- The define statement is added which imports the N/search and N/record modules.
- The return statement is added with the onRequest entry point.
- The nlobjSearchFilter function is changed to the search.createFilter method.
- The nlobjSearchColumn function is changed to the search.createColumn method.
- The nlapiSearchRecord function is changed to the search.run.each method.
- The nlapiTransformRecord function is changed to the record.transform method.
- The nlapiSubmitRecord function is changed to the Record.save method.

SuiteScript 1.0 Script	SuiteScript 2.0 Script
<pre> 1 function processOrdersCreatedToday(type) 2 { 3 //only execute when run from the scheduler 4 if (type != 'scheduled' && type != 'skipped') return; 5 6 var filters = new Array(); 7 filters[0] = new nlobjSearchFilter('mainline', null, 'is', 'T'); 8 filters[1] = new nlobjSearchFilter('trandate', null, 'equal To', 'today'); 9 </pre>	<pre> 1 /** 2 * @NApiVersion 2.0 3 * @NScriptType ScheduledScript 4 */ 5 6 // This script creates multiple sales records and logs the record creation progress. 7 define(['N/search', 'N/record'], function(search, record) { 8 function processOrdersCreatedToday(){ 9 if ((context.type != context.InvocationType.SCHEDULED) (context.type != context.InvocationType.SKIPPED)) { </pre>

SuiteScript 1.0 Script	SuiteScript 2.0 Script
<pre> 10 var searchresults = nlapiSearchRecord('salesorder', null, filter 11 s, null, new nlobjSearchColumn('terms')); 12 for (var i = 0; searchresults != null && i < searchresult 13 s.length; i++) 14 { 15 var id = searchresults[i].getId(); 16 var fulfillRecord = nlapiTransformRecord('salesorder', 17 id, 'itemfulfillment'); 18 nlapiSubmitRecord(fulfillRecord); 19 20 var billType = searchresults[i].getValue('paymentmethod') 21 == null ? 'invoice' : 'cashsale'; 22 var billRecord = nlapiTransformRecord('salesorder', id, 23 billType); 24 nlapiSubmitRecord(billRecord); 25 } </pre>	<pre> 10 var filters = new Array(); 11 12 filters[0] = search.createFilter({ 13 name: 'mainline', 14 operator: 'is', 15 values: 'T' 16 }); 17 filters[1] = search.createFilter({ 18 name: 'trandate', 19 operator: 'equalTo', 20 values: 'today' 21 }); 22 23 var searchcolumn = search.createColumn({ 24 name: 'terms' 25 }) 26 27 var search = search.create({ 28 type: search.type.SALESORDER, 29 filters: filters, 30 columns: searchcolumn 31 }); 32 33 search.run().each(function(result) { 34 35 var id = result.getValue({ 36 name: 'id' 37 }); 38 39 var fulfillRecord = record.transform({ 40 fromType: 'salesorder', 41 fromId: id, 42 toType: 'itemfulfillment' 43 }); 44 45 fulfillRecord.save(); 46 47 var paymentmethod = result.getValue({ 48 name: 'paymentmethod' 49 }); 50 51 var billType = paymentmethod ? 'invoice' 52 : 'cashsale'; 53 54 var billRecord = record.transform({ 55 fromType: 'salesorder', 56 fromId: id, 57 toType: billType 58 }); 59 60 billRecord.save(); 61 }); 62 63 return { 64 execute: processOrdersCreatedToday 65 } 66 } 67 }); </pre>

Create a simple form

Applies to: SuiteScript 1.0 | SuiteScript 2.x | APIs | SuiteCloud Developer

This sample is a Suitelet that creates a simple form.

The conversion of this script from SuiteScript 1.0 to SuiteScript 2.x includes the following:

- JSDoc tags are added at the top of the script to indicate the script version and script type.
- The define statement is added which imports the N/ui/serverWidget module.
- The return statement is added with the onRequest entry point.

- The `nlapiCreateForm` function is changed to the `serverWidget.createForm` method.
- The `field.setLayoutType` function is changed to the `serverWidget.FieldLayoutType.NORMAL` and `serverWidget.FieldBreakType.STARTCOL` enums.
- The `dumpResponse` function is rewritten to use the context parameter.

SuiteScript 1.0 Script	SuiteScript 2.x Script
<pre> 1 function demoSimpleForm(request, response) { 2 if (request.getMethod() == 'GET') { 3 var form = nlapiCreateForm('Simple Form'); 4 var field = form.addField('textfield','text', 'Text'); 5 field.setLayoutType('normal', 'startcol'); 6 form.addField('datefield','date', 'Date'); 7 form.addField('currencyfield','currency', 'Currency'); 8 form.addField('textareafield','textarea', 'Textarea'); 9 10 var select = form.addField('selectfield','select', 'Select'); 11 select.addSelectOption('', ''); 12 select.addSelectOption('a', 'Albert'); 13 select.addSelectOption('b', 'Baron'); 14 select.addSelectOption('c', 'Chris'); 15 select.addSelectOption('d', 'Drake'); 16 select.addSelectOption('e', 'Edgar'); 17 18 var sublist = form.addSubList('sublist','inlineeditor', 'In line Editor Sublist'); 19 sublist.addField('sublist1', 'date', 'Date'); 20 sublist.addField('sublist2', 'text', 'Text'); 21 sublist.addField('sublist3', 'currency', 'Currency'); 22 sublist.addField('sublist4', 'textarea', 'Large Text'); 23 sublist.addField('sublist5', 'float', 'Float'); 24 25 form.addSubmitButton('Submit'); 26 27 response.writePage(form); 28 } 29 else { 30 dumpResponse(request,response); 31 } 32 }</pre>	<pre> 1 /** 2 * @NApiVersion 2.x 3 * @NScriptType Suitelet 4 */ 5 6 define(['N/ui/serverWidget'], function (serverWidget) { 7 function onRequest(context) { 8 if (context.request.method === 'GET') { 9 var form = serverWidget.createForm({ 10 title: 'Simple Form' 11 }); 12 13 var field = form.addField({ 14 id: 'custpage_text', 15 type: serverWidget.FieldType.TEXT, 16 label: 'Text' 17 }); 18 19 field.layoutType = serverWidget.FieldLayoutType.NORMAL; 20 21 field.updateBreakType({ 22 breakType: serverWidget.FieldBreakType.STARTCOL 23 }); 24 25 form.addField({ 26 id: 'custpage_date', 27 type: serverWidget.FieldType.DATE, 28 label: 'Date' 29 }); 30 31 form.addField({ 32 id: 'custpage_currencyfield', 33 type: serverWidget.FieldType.CURRENCY, 34 label: 'Currency' 35 }); 36 37 var select = form.addField({ 38 id: 'custpage_selectfield', 39 type: serverWidget.FieldType.SELECT, 40 label: 'Select' 41 }); 42 43 select.addSelectOption({ 44 value: 'a', 45 text: 'Albert' 46 }); 47 48 select.addSelectOption({ 49 value: 'b', 50 text: 'Baron' 51 }); 52 53 var sublist = form.addSubList({ 54 id: 'sublist', 55 type: serverWidget.SublistType.INLINEEDITOR, 56 label: 'Inline Editor Sublist' 57 }); 58 59 sublist.addField({ 60 id: 'sublist1', 61 type: serverWidget.FieldType.DATE, 62 label: 'Date' 63 }); 64 65 sublist.addField({ 66 id: 'sublist2', 67 type: serverWidget.FieldType.TEXT, 68 label: 'Text' 69 }); 70 } 71 } 72 }</pre>

SuiteScript 1.0 Script	SuiteScript 2.x Script
	<pre> 71 form.addSubmitButton({ 72 label: 'Submit Button' 73 }); 74 75 context.response.writePage(form); 76 } else { 77 var delimiter = /\u0001/; 78 var textField = context.request.parameters.textfield; 79 var dateField = context.request.parameters.datefield; 80 var currencyField = context.request.parameters.currency 81 field; 82 var selectField = context.request.parameters.selectfield; 83 var sublistData = context.request.parameters.sublistda 84 ta.split(delimiter); 85 var sublistField1 = sublistData[0]; 86 var sublistField2 = sublistData[1]; 87 88 context.response.write('You have entered: ' + textField 89 + ' ' + dateField + ' ' 90 + currencyField + ' ' + selectField + ' ' + sublist 91 Field1 + ' ' + sublistField2); 92 } 93 } 94 95 return { 96 onRequest: onRequest 97 }; 98 99 }); </pre>

Implement end of month sales order promotions

Applies to: SuiteScript 1.0 | SuiteScript 2.x | APIs | SuiteCloud Developer

This sample is a user event script that implements end of month sales order promotions.

The conversion of this script from SuiteScript 1.0 to SuiteScript 2.0 includes the following:

- JSDoc tags are added at the top of the script to indicate the script version and script type
- The define statement is added which imports the N/currentRecord, N/runtime, and N/ui/serverWidget modules
- The return statement is added with the beforeLoad entry point
- The nlapiGetContext function is changed to the context.UserEventType property.
- The getExecutionContext function is changed to the runtime.executionContext property.
- The nlapiLogExecution function is changed to the log.debug method.
- The form function parameter is changed to the form context component.

SuiteScript 1.0 Script	SuiteScript 2.0 Script
<pre> 1 function customizeUI_SalesOrderBeforeLoad(type, form) { 2 var currentContext = nlapiGetContext(); 3 4 if (type == 'create' && currentContext.getExecutionContext() 5 == 'userinterface') 6 { 7 var fieldId = 'custpage_eom_promotion'; 8 var fieldLabel = 'Eligible EOM promotion'; 9 var today = new Date(); 10 var month = today.getMonth(); 11 var date = today.getDate(); 12 13 nlapiLogExecution('DEBUG', 'month date', month + ' ' + date); 14 15 //February </pre>	<pre> 1 /* 2 * @NApiVersion 2.0 3 * @NScriptType UserEventScript 4 */ 5 6 define(['N/currentRecord', 'N/runtime', 'N/ui/serverWidget'], func 7 tion (currentRecord, runtime, serverWidget) { 8 function customizeUI_SalesOrderBeforeLoad(context) { 9 if (context.UserEventType === context.UserEventType.CREATE) { 10 if (runtime.executionContext === runtime.Context 11 Type.USEREVENT) { 12 var fieldId = 'custpage_eom_promotion'; 13 var fieldLabel = 'Eligible EOM promotion'; 14 var today = new Date(); 15 var month = today.getMonth(); </pre>

SuiteScript 1.0 Script	SuiteScript 2.0 Script
<pre> 15 if (month==1) 16 { 17 if (date==24 date==25 date==26 date==27 date==28 date==29) 18 form.addField(fieldId, 'checkbox', fieldLabel); 19 } 20 //31-day months 21 else if (month==0 month==2 month==4 month==6 month==7 month==9 month==11) 22 { 23 if (date==27 date==28 date==29 date==30 date==31) 24 form.addField(fieldId, 'checkbox', fieldLabel); 25 } 26 //30-day months 27 else 28 { 29 if (date==26 date==27 date==28 date==29 date==30) 30 form.addField(fieldId, 'checkbox', fieldLabel); 31 } 32 } 33 } </pre>	<pre> 14 var date = today.getDate(); 15 16 log.debug({ 17 title: 'month date', 18 details: month + ' ' + date 19 }); 20 21 // February 22 if (month === 1) { 23 if (date === 24 date === 25 date === 26 date === 27 date === 28 date === 29) { 24 context.form.addField({ 25 id: fieldId, 26 label: fieldLabel, 27 type: serverWidget.FieldType.CHECKBOX 28 }); 29 } 30 // 31-day months 31 } else if (month === 0 month === 2 month === 4 month === 6 month === 7 32 month === 9 month === 11) { 33 if (date === 27 date === 28 date === 29 date === 30 date === 31) { 34 context.form.addField({ 35 id: fieldId, 36 label: fieldLabel, 37 type: serverWidget.FieldType.CHECKBOX 38 }); 39 } 40 } else { 41 if (date === 26 date === 27 date === 28 date === 29 date === 30) { 42 context.form.addField({ 43 id: fieldId, 44 label: fieldLabel, 45 type: serverWidget.FieldType.CHECKBOX 46 }); 47 } 48 } 49 } 50 } 51 } 52 } 53 } 54 return { 55 beforeLoad: customUI_SalesOrderBeforeLoad 56 }; 57 } </pre>

Set the sales rep on the record in a workflow

Applies to: SuiteScript 1.0 | SuiteScript 2.x | APIs | SuiteCloud Developer

This sample is a user event script that sets the sales rep on the record in a workflow. To get a new record in SuiteScript 2.x, place this code in a user event script's beforeLoad, beforeSubmit, or afterSubmit entry point. For this example, the beforeLoad entry point is used.

The conversion of this script from SuiteScript 1.0 to SuiteScript 2.1 includes the following:

- JSDoc tags are added at the top of the script to indicate the script version and script type.
- The define statement is added which imports the N/currentRecord and N/runtime modules.
- The return statement is added with the beforeLoad entry point.
- The nlapiGetNewRecord function is changed to context.newRecord.
- The nlapiGetContext and getSetting functions are changed to the runtime.getCurrentScript and the runtime.Script.getParameter methods.
- The setFieldValue function is changed to the Record.setValue method.

SuiteScript 1.0 Script	SuiteScript 2.1 Script
<pre> 1 function changeSalesRep() { 2 nlapiGetNewRecord().setFieldValue('salesrep', nlapiGetContext() 3 t().getSetting('SCRIPT', 'custscript_salesrep')); 4 } </pre>	<pre> 1 /* 2 * @NApiVersion 2.1 3 * @NScriptType UserEventScript 4 */ 5 6 define(['N/currentRecord', 'N/runtime'], (currentRecord, runtime) => 7 { 8 function changeSalesRep(context) { 9 const myRecord = context.newRecord; 10 const scriptObj = runtime.getCurrentScript(); 11 const myScriptParam = scriptObj.getParameter({ 12 name: 'custscript_salesrep' 13 }); 14 15 myRecord.setValue({ 16 fieldId: 'salesrep', 17 value: myScriptParam 18 }); 19 } 20 21 return { 22 beforeLoad: changeSalesRep 23 }; 24 }); </pre>

Create an item receipt from a purchase order

Applies to: SuiteScript 1.0 | SuiteScript 2.x | APIs | SuiteCloud Developer

This sample is a custom module script that creates an item receipt from a purchase order. This is either a custom module or an individual function within a user event or other server script, so no JSDoc or define statement or return statement or entry point function is needed. But you'll need to load the N/record and N/email modules for the function to work.

The conversion of this script from SuiteScript 1.0 to SuiteScript 2.1 includes the following:

- JSDoc tags are added at the top of the script to indicate the script version and script type.
- The `nlapiTransformRecord` function is changed to the `record.transform` method.
- The `getLineItemValue` function is changed to the `Record.getSublistValue` method.
- The `setLineItemValue` function is changed to the `Record.setSublistValue` method.
- The `nlapiSubmitRecord` function is changed to the `Record.save` method.
- The `nlapiSendEmail` function is changed to the `email.send` method.

SuiteScript 1.0 Script	SuiteScript 2.1 Script
<pre> 1 function transformPurchaseOrder() { 2 var fromRecord; 3 var fromId; 4 var toRecord; 5 var trecord; 6 var qty; 7 8 fromRecord = 'purchaseorder'; 9 fromId = 26; 10 toRecord = 'itemreceipt'; 11 12 trecord = nlapiTransformRecord(fromRecord, fromId, toRecord); 13 qty = trecord.getLineItemValue('item', 'quantity', 1); 14 trecord.setLineItemValue('item', 'quantity', 1, '2'); 15 var idl = nlapiSubmitRecord(trecord, true); 16 } </pre>	<pre> 1 function transformPurchaseOrder() { 2 const fromRecord = 'purchaseorder'; 3 const fromId = 26; 4 const toRecord = 'itemreceipt'; 5 6 const trecord = record.transform({ 7 fromType: fromRecord, 8 fromId: fromId, 9 toType: toRecord, 10 }); 11 12 const qty = trecord.getSublistValue({ 13 sublistId: 'item', 14 fieldId: 'quantity', 15 line: 1 16 }); 17 } </pre>

SuiteScript 1.0 Script	SuiteScript 2.1 Script
<pre> 17 nlapiSendEmail(-5, -5, 'Transform Email' + 'Original Qty = ' + qty 18 + ' ' + 'Record Created = ' + idl, null); } </pre>	<pre> 18 trecord.setSublistValue({ 19 sublistId: 'item', 20 fieldId: 'quantity', 21 line: 1, 22 value: '2' 23 }); 24 25 const idl = trecord.save({ 26 enableSourcing: true 27 }); 28 29 email.send({ 30 author: -5, 31 recipients: -5, 32 subject: 'Transform Email' + 'Original Qty = ' + qty + ' ' 33 + 'Record Created = ' + idl, 34 body: null, 35 }); </pre>

Create and run a joined search

i Applies to: SuiteScript 1.0 | SuiteScript 2.x | APIs | SuiteCloud Developer

This sample is a custom module script that creates and runs a joined search. This sample can be used as a custom module or as an individual function within a user event or other server script, so no JSDoc or define statement or return statement or entry point function is needed. But you'll need to load the N/ search module for the function to work.

The conversion of this script from SuiteScript 1.0 to SuiteScript 2.x includes the following:

- The `nlobjSearchFilter` function is changed to the `search.createFilter` method.
- The `nlobjSearchColumn` function is changed to the `search.createColumn` method.
- The `nlapiSearchRecord` function is changed to the `Search.run` method.

SuiteScript 1.0 Script	SuiteScript 2.x Script
<pre> 1 function myJoinedSearch() { 2 // Create a filters array and define search filters for an Item search 3 var filters = new Array(); 4 5 // filter by a specific customer (121) on the Pricing record 6 filters[0] = new nlobjSearchFilter('customer', 'pricing', 'is', '121'); 7 8 // filter by a currency type (USA) on the Pricing record 9 filters[1] = new nlobjSearchFilter('currency', 'pricing', 'is', '1'); 10 11 // set search return columns for Pricing search 12 var columns = new Array(); 13 14 // return data from pricelevel and unitprice fields on the Pricing record 15 columns[0] = new nlobjSearchColumn('pricelevel', 'pricing'); 16 columns[1] = new nlobjSearchColumn('unitprice', 'pricing'); 17 18 // specify name as a search return column. There is no join set in this field. 19 // This is the Name field as it appears on Item records. 20 columns[2] = new nlobjSearchColumn('name'); 21 22 // execute the Item search, which uses data on the Pricing record as search 23 // filters 24 var searchresults = nlapiSearchRecord('item', null, filters, columns); 25 } </pre>	<pre> 1 function myJoinedSearch () { 2 var filters = new Array(); 3 4 filters[0] = search.createFilter({ 5 name: 'customer', 6 join: 'pricing', 7 operator: 'is', 8 values: '121' 9 }); 10 11 filters[1] = search.createFilter({ 12 name: 'currency', 13 join: 'pricing', 14 operator: 'is', 15 values: '1' 16 }); 17 18 var columns = new Array(); 19 20 columns[0] = search.createColumn({ 21 name: 'pricelevel', 22 join: 'pricing' 23 }); 24 columns[1] = search.createColumn({ 25 name: 'unitprice', 26 join: 'pricing' 27 }); 28 columns[2] = search.createColumn({ 29 name: 'name' 30 }); </pre>

SuiteScript 1.0 Script	SuiteScript 2.x Script
	<pre> 31 32 var searchresults = search.run(); 33 } </pre>

Create CSV imports

Applies to: SuiteScript 1.0 | SuiteScript 2.x | APIs | SuiteCloud Developer

This sample includes two scripts used to create CSV imports. These samples can be used as a custom module or as individual function within a user event or other server script, so no JSDoc or define statement or return statement or entry point function is needed. But you'll need to load the N/task and N/file modules for the function to work.

The conversion of this script from SuiteScript 1.0 to SuiteScript 2.x includes the following:

- The `nlapiLoadFile` function is changed to the `file.load` method.
- The `nlapiCreateCSVImport` function is changed to the `task.create` method using the `task.TaskType.CSV_IMPORT` type value.
- The `job.setMapping` function is changed to the `job.mappingId` property.
- The `job.setPrimaryFile` function is changed to the `job.importFile` property.
- The `job.setLinkedFile` function is changed to the `job.linkedFiles` property (which is an Object).
- The `job.setOption` function is changed to the `job.name` property.
- The `nlapiSubmitCSVImport` function is changed to the `job.submit` method.

SuiteScript 1.0 Script	SuiteScript 2.x Script
<pre> 1 var mappingFieldId = 2; 2 var primaryFile = nlapiLoadFile(73); 3 4 var job = nlapiCreateCSVImport(); 5 job.setMapping(mappingFieldId); 6 job.setPrimaryFile(primaryFile); 7 job.setOption("jobName", "jobImport"); 8 9 var jobId = nlapiSubmitCSVImport(job); </pre>	<pre> 1 function CSVImport() { 2 var mappingFieldId = 2; 3 var primaryFile = file.load({ 4 id: 73 5 }); 6 7 var job = task.create({ 8 taskType: task.TaskType.CSV_IMPORT 9 }); 10 11 job.mappingId = mappingFieldId; 12 job.importFile = primaryFile; 13 job.name = 'jobImport'; 14 15 var jobId = job.submit(); 16 } </pre>
<pre> 1 var mappingFieldId = "CUSTIMPOTentityMultiFile"; 2 3 var job = nlapiCreateCSVImport(); 4 job.setMapping(mappingFieldId); 5 6 job.setPrimaryFile(nlapiLoadFile(73)); 7 8 job.setLinkedFile("addressbook", nlapiLoadFile(74)); 9 job.setOption("jobName", "jobImport"); 10 11 var jobId = nlapiSubmitCSVImport(job); </pre>	<pre> 1 function CSVMultiFileImport() { 2 var mappingFieldId = 'CUSTIMPOTentityMultiFile'; 3 4 var job = task.create({ 5 taskType: task.TaskType.CSV_IMPORT 6 }); 7 8 job.mappingId = mappingFieldId; 9 10 var primaryFile = file.load({ 11 id: 73 12 }); 13 job.importFile = primaryFile; 14 15 var linkedFile = file.load({ 16 id: 74 17 }); 18 } </pre>

SuiteScript 1.0 Script	SuiteScript 2.x Script
	<pre> 19 job.linkedFiles = {'addressbook': linkedFile}; 20 21 job.name = 'jobImport'; 22 23 var jobId = job.submit(); 24 }</pre>

Using SuiteScript 2.x Scripts with SuiteScript 1.0 Scripts

Applies to: SuiteScript 1.0 | SuiteScript 2.x | APIs | SuiteCloud Developer

If converting your SuiteScript 1.0 scripts to SuiteScript 2.x is not feasible in your situation, you can use SuiteScript 2.x to develop any new functionality within a RESTlet and call the RESTlet from your SuiteScript 1.0 script using the `nlapirequestRestlet()` function.

This approach lets you take advantage of the features, APIs, and functionality enhancements available in SuiteScript 2.x and use them to build new features and extend the capabilities of your existing SuiteScript 1.0 scripts.

To use SuiteScript 2.x scripts with SuiteScript 1.0 scripts, you need to do the following:

- Create a separate RESTlet script to contain your SuiteScript 2.x code. For information about RESTlets, see [SuiteScript 2.x RESTlet Script Type](#).
- Update your SuiteScript 1.0 script to add the `nlapirequestRestlet(scriptId, deploymentId, urlParams, body, headers, httpMethod)` function with the details of the RESTlet script.

`nlapirequestRestlet(scriptId, deploymentId, urlParams, body, headers, httpMethod)`

Applies to: SuiteScript 1.0 | APIs | SuiteCloud Developer

Note: The content in this help topic pertains to SuiteScript 1.0. You can use this function in SuiteScript 1.0 scripts to call RESTlets written in SuiteScript 2.x code. For more information, see [Using SuiteScript 2.x Scripts with SuiteScript 1.0 Scripts](#).

For information about SuiteScript 1.0, see the help topic [SuiteScript 1.0 Documentation](#).

Description	Sends an HTTPS request to a RESTlet and returns the response. Authentication headers are automatically added. The RESTlet runs with the same privileges as the calling script. If you use custom headers, make sure the names of these headers do not contain underscores.
Returns	nlobjResponse object
Supported Script Types	Server scripts
Governance	10 units
Since	2023.1

Parameters

Parameter	Type	Required / Optional	Description	Since
scriptId	string number	required	The internal ID or script ID of the script record. Specify internal ID as a number. Specify script ID as a string.	2023.1
deploymentId	string	required	The script ID of the script deployment record.	2023.1
urlParams	Object<string, string>	optional	The parameters to be appended to the target URL as a query string.	2023.1
body	string Object<string, string>	required, if httpMethod is POST or PUT.	The PUT/POST data. This parameter is ignored if httpMethod is not POST or PUT.	2023.1
headers	Object<string, string>	optional	The HTTPS headers.	2023.1
httpMethod	string	optional	The HTTPS method (DELETE, GET, HEAD, POST, PUT). The default value is GET if body is not specified, or POST if body is specified.	2023.1

Errors

Error Code	Message	Thrown If
INVALID_SCRIPT_DEPLOYMENT_ID_1	—	The deploymentId parameter does not reference a valid deployment for the script.
SSS_AUTHORIZATION_HEADER_NOT_ALLOWED	—	The authorization header is set.
SSS_INVALID_HEADER	—	The headers parameter is in an invalid format or contains an invalid header.
SSS_INVALID_SCRIPT_ID_1	—	The scriptId parameter does not reference a RESTlet script.
SSS_INVALID_URL_PARAMS	—	The urlParams parameter is in an invalid format.
SSS_MISSING_REQD_ARGUMENT	Missing a required argument: {param name}	The body, deploymentID, or scriptID parameter is not specified.
SSS_REQUEST_LOOP_DETECTED	—	The script executes a recursive function that has exceeded the limit for the number of times a script can call itself using an HTTPS request.

Syntax

```
1 | // Add additional code
```

```
2  ...
3  var myRestletHeaders = {
4      myHeaderType: 'Test',
5      myHeaderStuff: 'This is my header',
6      myHeaderId: 7
7  };
8  var myUrlParameters = {
9      myFirstParameter: 'firstparam',
10     mySecondParameter: 'secondparam'
11 }
12 var myRestletResponse = nlapiRequestRestlet(
13     99,
14     'deploy1',
15     myUrlParameters,
16     'My Restlet body',
17     myRestletHeaders,
18     'GET'
19 );
```